

Alex's Anthology of Algorithms
Common Code for Contests in Concise C++

Draft v0.98, Jun 15, 2026

Alex Li

© 2026. All rights reserved.

This page is intentionally left blank.

Preface

Browse the latest version online at: algorithms.alexli.ca

Contributions are always welcome on GitHub: github.com/alxli/algorithm-anthology

Introduction

Welcome to a practical collection of concise C++ implementations of algorithms, data structures, and contest utilities. This is not a first textbook; it is a codebook about implementation details, interfaces, edge cases, and the choices that make familiar algorithms usable in real code. Use it:

- as a reference when you know an algorithm conceptually but want a clean implementation;
- as a printed codebook for contests that allow prepared material, including ICPC-style events;
- as a checklist for edge cases, complexity, overflow, indexing, and API choices;
- as a map of related techniques: slower versions, faster versions, variants, and common reductions sit near one another.

For learning, do not start by copying. Study the idea, prove the invariant or recurrence, implement it yourself, then compare; the differences reveal the practical choices. Each section gives only enough theory to orient the implementation: problem, interface, assumptions, and complexity. Source files are organized as documentation, reusable implementation, then example usage. The reusable part ends at the **Example Usage** marker; everything after exists only so the file compiles and demonstrates itself. When adapting a section, drop the example block and keep the implementation above it.

Guiding principles:

- *Clarity*: A reader familiar with the algorithm should follow the code without reverse-engineering it. Tricks, non-obvious invariants, overflow risks, and language assumptions are documented.
- *Concision*: Contest code is found, read, adapted, and typed under pressure. Short code helps when it removes noise, not when it hides the idea. So the code here is often more verbose than compact ICPC snippets; the extra lines buy clearer names, configurable types, safer defaults, reset functions, explicit handling of variants, and more.
- *Efficiency*: Expected asymptotic behavior and reasonable constants. Both simpler and more robust versions are often included, but educational-only sections are labeled so.
- *Adaptability*: Interfaces make common changes easy, such as swapping a numeric type, changing a combine function, exposing an edge ID, or choosing whether boundaries are included.
- *Consistency*: Similar structures expose similar operations, with repeated naming and example styles, so moving between chapters is easy.

- *Portability*: Targets modern C++17 and avoids unnecessary dependencies; the few contest-useful exceptions are noted at each use.

This began as a personal contest codebook, so it favors single-file snippets, predictable APIs, compact examples, and low constants. Use it actively: annotate it, delete what you dislike, rename functions to match your habits, and stress-test anything you intend to trust. Cheers!

Portability Notes

Examples target a modern GCC or Clang environment with C++17. A typical compile command:

```
g++ -std=c++17 -O2 -Wall -pedantic
```

Most code assumes common contest-platform type sizes: 8-bit `char`, 32-bit `int` and `float`, 64-bit `double` and `long long`. `long double` width and precision are implementation-dependent; sections relying on it say so.

The code targets ISO C++17 unless noted. The most common exceptions:

- GCC/Clang integer extensions like `__int128` or `__uint128_t`, usually guarded with a portable fallback when overflow safety matters.
- Compiler builtins like `__builtin_popcount()` and `__builtin_clz()` — convenient and widely available, though C++20's `<bit>` header standardizes many of them. See <https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html> for GCC's reference.
- GNU policy-based data structures like `__gnu_pbds::tree`, used in a few sections to show useful non-standard components.
- A few documented representation assumptions, such as `signbit_()` in the math utilities assuming an IEEE-like floating-point sign bit layout.

Contents

Preface	i
1 Elementary Algorithms	1
1.1 Array Transformations	1
1.1.1 Sorting Algorithms	1
1.1.2 Array Rotation	7
1.1.3 Counting Inversions	9
1.1.4 Coordinate Compression	11
1.1.5 Selection (Quickselect)	13
1.1.6 3-Way Partitioning (Dutch National Flag)	14
1.2 Arrays and Dynamic Programming	15
1.2.1 Prefix Sums	15
1.2.2 Difference Array	17
1.2.3 Two Pointers	18
1.2.4 Iterated Function Cycle Detection	19
1.2.5 Sliding Window Maximum	22
1.2.6 Aggregate Queue	23
1.2.7 Monotonic Stack	25
1.2.8 Minimum Excluded Value	27
1.2.9 Longest Increasing Subsequence	28
1.2.10 0-1 Knapsack	29
1.2.11 Unbounded Knapsack and Coin Change	30
1.2.12 Subset Sum	31
1.2.13 Largest Zero Submatrix	33
1.3 Greedy and Scheduling	34
1.3.1 Interval Scheduling	34
1.3.2 Weighted Interval Scheduling	35
1.3.3 Interval Partitioning	36
1.3.4 Deadline Scheduling	37
1.3.5 Minimizing Late Jobs (Moore-Hodgson)	38

1.4	Streaming and Randomized Algorithms	39
1.4.1	Fisher-Yates Shuffle	39
1.4.2	Reservoir Sampling	40
1.4.3	Maximum Subarray Sum (Kadane)	42
1.4.4	Majority Element (Boyer-Moore)	44
1.4.5	Online Statistics (Welford)	45
1.4.6	Frequent Elements (Misra-Gries)	46
1.5	Bit Manipulation	47
1.5.1	Bit Operations and Masks	47
1.5.2	Subset and Submask Iteration	49
1.5.3	Gray Code	51
1.5.4	XOR Basis	52
1.5.5	Binary Trie	53
1.5.6	Bitwise Convolution (FWHT)	56
2	Data Structures	60
2.1	Linked Lists	60
2.1.1	Singly Linked List	60
2.1.2	Doubly Linked List	62
2.1.3	LRU Cache	65
2.2	Heaps and Priority Queues	66
2.2.1	Binary Heap	66
2.2.2	Leftist Heap	68
2.2.3	Skew Heap	71
2.2.4	Pairing Heap	73
2.3	Dictionaries and Ordered Sets	76
2.3.1	Treap	76
2.3.2	AVL Tree	81
2.3.3	Red-Black Tree	84
2.3.4	Splay Tree	90
2.3.5	Size Balanced Tree	93
2.3.6	Interval Treap	97
2.3.7	Hash Table (Chaining)	101
2.3.8	Hash Table (Open Addressing)	104
2.3.9	Skip List	109
2.4	Range Queries in One Dimension	112
2.4.1	Sparse Table	112
2.4.2	Disjoint Sparse Table	113
2.4.3	Square Root Decomposition	115

2.4.4	Mo's Algorithm	117
2.4.5	Segment Tree (Point Update)	118
2.4.6	Segment Tree (Range Update)	122
2.4.7	Iterative Segment Tree (Point Update)	127
2.4.8	Iterative Segment Tree (Range Update)	130
2.4.9	Segment Tree Beats	135
2.4.10	Sparse Segment Tree	139
2.4.11	Persistent Segment Tree	144
2.4.12	Merge Sort Tree	145
2.4.13	Wavelet Tree	147
2.4.14	Cartesian Tree	149
2.4.15	Implicit Treap	150
2.5	Range Queries in Two Dimensions	154
2.5.1	Quadtree (Point Update)	154
2.5.2	Quadtree (Range Update)	158
2.5.3	2D Segment Tree (Point Update)	161
2.5.4	2D Range Tree	166
2.5.5	K-d Tree for 2D Range Query	168
2.5.6	K-d Tree for Nearest Neighbor	170
2.6	Fenwick Trees	172
2.6.1	Fenwick Tree (Point Update, Range Query)	172
2.6.2	Fenwick Tree (Range Update, Point Query)	174
2.6.3	Fenwick Tree (Range Update, Range Query)	175
2.6.4	Sparse Fenwick Tree (Range Update, Range Query)	177
2.6.5	2D Fenwick Tree	179
2.6.6	2D Sparse Fenwick Tree	181
2.6.7	Offline 2D Fenwick Tree	183
2.7	Disjoint Sets and Tree Structures	186
2.7.1	Disjoint Set Union	186
2.7.2	Sparse Disjoint Set Union	187
2.7.3	Rollback Disjoint Set Union	189
2.7.4	Lowest Common Ancestor (Sparse Table)	191
2.7.5	Lowest Common Ancestor (Segment Tree)	193
2.7.6	Virtual Tree	196
2.7.7	Weighted Tree Path Queries (Binary Lifting)	199
2.7.8	Heavy Light Decomposition	201
2.7.9	Euler Tour Tree	206
2.7.10	Link-Cut Tree	210

3	Strings	217
3.1	String Utilities	217
3.2	Hashing	224
3.2.1	Hash Functions	224
3.2.2	Rolling Hash	229
3.2.3	Rolling Hash Deque	232
3.2.4	Dynamic Rolling Hash	235
3.2.5	Zobrist Hashing	237
3.3	Pattern Matching	239
3.3.1	String Searching (KMP)	239
3.3.2	String Searching (Z Algorithm)	240
3.3.3	String Searching (Aho-Corasick)	241
3.3.4	Palindrome Searching (Manacher)	244
3.3.5	Lyndon Factorization (Duval)	246
3.3.6	De Bruijn Sequence	247
3.4	Suffix Arrays and LCP	248
3.4.1	Suffix Array and LCP (Manber-Myers)	248
3.4.2	Suffix Array and LCP (Counting Sort)	250
3.4.3	Suffix Array and LCP (DC3)	252
3.5	String Data Structures	256
3.5.1	Trie	256
3.5.2	Radix Tree	259
3.5.3	Suffix Tree (Ukkonen)	264
3.5.4	Suffix Automaton	268
3.5.5	Eertree	270
3.6	Sequence Dynamic Programming	272
3.6.1	Longest Common Substring	272
3.6.2	Longest Common Subsequence	273
3.6.3	Sequence Alignment (Needleman-Wunsch)	275
3.7	Expression Parsing	277
3.7.1	Recursive Descent Parsing (Simple)	277
3.7.2	Recursive Descent Parsing	279
3.7.3	Shunting Yard Parsing	282
3.8	Encoding and Compression	286
3.8.1	Entropy and Frequency Tables	286
3.8.2	Run-Length Encoding	288
3.8.3	Base64 Encoding	289
3.8.4	Huffman Coding	290

3.8.5	Burrows-Wheeler Transform	292
-------	-------------------------------------	-----

4 Graphs 295

4.1	DFS and Tree Algorithms	295
4.1.1	Graph Class and DFS	295
4.1.2	Topological Sort	297
4.1.3	Tree Centers and Diameter	299
4.1.4	Tree Rerooting DP	301
4.1.5	Centroid Decomposition	304
4.1.6	Prufer Code	305
4.1.7	Cycle Finding	307
4.1.8	Eulerian Trails	310
4.2	Connectivity	313
4.2.1	Connected Components	313
4.2.2	Strongly Connected Components (Kosaraju)	315
4.2.3	Strongly Connected Components (Tarjan)	316
4.2.4	Dominator Tree (Lengauer-Tarjan)	318
4.2.5	Articulation Points, Biconnected Components, Block-Cut Forest	320
4.2.6	Bridges, 2-Edge-Connected Components, Bridge Forest	323
4.2.7	Online Bridge Finding	326
4.2.8	2-SAT	329
4.2.9	Functional Graph	330
4.2.10	Offline Dynamic Connectivity	333
4.3	Shortest Paths	336
4.3.1	Unweighted Shortest Path (BFS)	336
4.3.2	0-1 Shortest Path (Deque BFS)	337
4.3.3	Shortest Path (Dijkstra)	339
4.3.4	Shortest Path (Bellman-Ford)	340
4.3.5	Shortest Path (SPFA)	342
4.3.6	DAG Shortest Path (Topological DFS)	344
4.3.7	All-Pairs Shortest Path (Floyd-Warshall)	345
4.4	Spanning Trees	347
4.4.1	Minimum Spanning Tree (Prim)	347
4.4.2	Minimum Spanning Tree (Kruskal)	349
4.4.3	Minimum Spanning Arborescence (Edmonds)	350
4.5	Flows and Cuts	352
4.5.1	Maximum Flow (Ford-Fulkerson)	352
4.5.2	Maximum Flow (Edmonds-Karp)	353
4.5.3	Maximum Flow (Dinic)	356

4.5.4	Maximum Flow (Dinic with Decomposition)	358
4.5.5	Maximum Flow (Push-Relabel)	362
4.5.6	Minimum-Cost Maximum-Flow (SPFA)	365
4.5.7	Minimum-Cost Maximum-Flow (Potentials)	367
4.5.8	Gomory-Hu Tree	370
4.5.9	Global Minimum Cut (Stoer-Wagner)	373
4.6	Matching and Assignment	375
4.6.1	Maximum Bipartite Matching (Kuhn)	375
4.6.2	Maximum Bipartite Matching (Hopcroft-Karp)	377
4.6.3	Minimum Vertex Cover (Konig)	378
4.6.4	Maximum Graph Matching (Blossom)	381
4.6.5	Maximum Weight Graph Matching (Weighted Blossom)	383
4.6.6	Assignment (Hungarian Algorithm)	389
4.6.7	Stable Marriage (Gale-Shapley)	391
4.7	Exponential Graph Problems	392
4.7.1	Maximum Clique (Bron-Kerbosch)	392
4.7.2	Graph Coloring (Backtracking)	394
4.7.3	Edge Coloring (Misra-Gries)	396
4.7.4	Shortest Hamiltonian Path and Cycle (Held-Karp)	398
5	Optimization	402
5.1	Monotone Predicate Search	402
5.1.1	Binary Search	402
5.1.2	Exponential Search	404
5.2	Unimodal and Continuous Search	405
5.2.1	Ternary and Golden Section Search	405
5.2.2	Hill Climbing	407
5.2.3	Simulated Annealing	408
5.3	Dynamic Programming Optimization	410
5.3.1	Monotone Queue DP Optimization	410
5.3.2	Divide-and-Conquer DP Optimization	411
5.3.3	SMAWK Row Minima	412
5.3.4	Knuth DP Optimization	414
5.3.5	Convex Hull Trick (Semi-Dynamic)	416
5.3.6	Convex Hull Trick (Fully-Dynamic)	417
5.3.7	Li Chao Tree	419
5.4	Numerical Methods	421
5.4.1	Root Finding (Bracketing)	421
5.4.2	Root Finding (Iteration)	422

5.4.3	Polynomial Root Finding (Differentiation)	423
5.4.4	Polynomial Root Finding (Laguerre)	425
5.4.5	Polynomial Root Finding (Ehrlich-Aberth)	428
5.4.6	Integration (Simpson)	431
5.4.7	Linear Programming (Simplex)	432
6	Mathematics	436
6.1	Math Utilities	436
6.2	Combinatorics	444
6.2.1	Combinatorial Calculations	444
6.2.2	Enumerating Arrangements	447
6.2.3	Enumerating Permutations	450
6.2.4	Enumerating Combinations	453
6.2.5	Enumerating Partitions	457
6.2.6	Enumerating Generic Combinatorial Sequences	460
6.3	Number Theory	463
6.3.1	GCD, LCM, Mod Inverse, Chinese Remainder	463
6.3.2	Modular Arithmetic	468
6.3.3	Primitive Roots	472
6.3.4	Discrete Logarithm (Baby-Step Giant-Step)	474
6.3.5	Modular Square Root (Tonelli-Shanks)	476
6.3.6	Jacobi Symbol	477
6.3.7	Prime Generation	479
6.3.8	Prime Tests and Factorization	481
6.3.9	Euler's Totient Function	489
6.3.10	Mobius Function and Inclusion-Exclusion	491
6.4	Arbitrary Precision Arithmetic	493
6.4.1	Big Integers (Simple)	493
6.4.2	Big Integers	496
6.4.3	Rational Numbers	506
6.5	Linear Algebra	509
6.5.1	Matrix Utilities	509
6.5.2	Row Reduction	516
6.5.3	Determinant and Inverse	518
6.5.4	LU Decomposition	522
6.5.5	Characteristic Polynomial	525
6.5.6	Matrix-Tree Theorem	527
6.5.7	Sparse Matrix	529
6.6	Polynomials	534

6.6.1	Number Theoretic Transform	534
6.6.2	Fast Fourier Transform	536
6.6.3	Polynomial Operations	538
6.6.4	Berlekamp-Massey	542
6.6.5	Linear Recurrence (Kitamasa)	544
6.6.6	Polynomial Interpolation	545
6.7	Game Theory	547
6.7.1	Nim	547
6.7.2	Nim Product	548
6.7.3	Sprague-Grundy	550
6.7.4	Grundy Numbers on DAG	551
6.7.5	Minimax and Alpha-Beta Pruning	552
7	Geometry	555
7.1	Geometry Primitives	555
7.1.1	Point	555
7.1.2	Line	559
7.1.3	Circle	562
7.1.4	Triangle	565
7.1.5	Rectangle	567
7.2	Angles, Distances, and Intersections	569
7.2.1	Angles	569
7.2.2	Distances	572
7.2.3	Line Intersection	575
7.2.4	Circle Intersection	579
7.2.5	Circle Tangents	583
7.3	Polygons and Point Sets	584
7.3.1	Polygon Sorting and Area	584
7.3.2	Point-in-Polygon	587
7.3.3	Convex Hull and Diametral Pair	589
7.3.4	Convex Polygon Queries	591
7.3.5	Minimum Enclosing Circle	594
7.3.6	Closest Pair	596
7.3.7	Multiple Segment Intersection	599
7.3.8	Maximum Collinear Points	602
7.4	Advanced Planar Geometry	604
7.4.1	Convex Polygon Cut	604
7.4.2	Half-Plane Intersection	606
7.4.3	Minkowski Sum	608

7.4.4	Polygon Intersection and Union	610
7.4.5	Delaunay Triangulation (Simple)	614
7.4.6	Delaunay Triangulation (Guibas-Stolfi)	616
7.4.7	Manhattan MST	621
7.5	3D Geometry	623
7.5.1	3D Point	623
7.5.2	3D Convex Hull	624
7.5.3	Polyhedron Volume	626
8	Miscellany	628
8.1	Contest Utilities	628
8.2	Fast IO	631
8.3	Debugging	635
8.4	Timing and Benchmarking	637
8.5	Stress Testing	638
8.6	Policy-Based Data Structures	640
8.7	Multithreading	642
8.8	Memory Allocation	643

Chapter 1

Elementary Algorithms

1.1 Array Transformations

1.1.1 Sorting Algorithms

These functions are equivalent to `std::sort()`, taking random-access iterators as a range `[lo, hi)` to be sorted. Elements between `lo` and `hi` (including the element pointed to by `lo` but excluding the element pointed to by `hi`) will be sorted into ascending order after the function call. Optionally, a comparison function object specifying a strict weak ordering may be specified to replace the default `operator<`.

- `quicksort(lo, hi)` sorts the range using quicksort.
- `mergesort(lo, hi)` sorts the range using merge sort, which is stable.
- `heapsort(lo, hi)` sorts the range using heapsort.
- `insertion_sort(lo, hi)` sorts the range using insertion sort, which is stable.
- `combsort(lo, hi)` sorts the range using comb sort.
- `radix_sort(lo, hi)` sorts the range using least-significant-byte radix sort.

`radix_sort()` is the exception to the shared interface above: it takes no comparator and requires an integer value type. These functions are not meant to compete with standard library implementations in terms of speed. Instead, they are meant to demonstrate how common sorting algorithms can be concisely implemented in C++.

```
#include <algorithm>
#include <functional>
#include <iterator>
#include <type_traits>
#include <vector>
```

Quicksort repeatedly selects a pivot and partitions the range so that elements comparing less than the pivot precede it, elements comparing equal stay in the middle, and elements comparing greater follow it. Divide and conquer is then applied to the two outer ranges until the original range is sorted. Despite having a worst case of $O(n^2)$, quicksort is often faster in practice than merge sort and heapsort, which both have a worst case time complexity of $O(n \log n)$.

The pivot chosen in this implementation is always a middle element of the range to be sorted. To reduce the likelihood of encountering the worst case, the pivot can be chosen in better ways (e.g. randomly, or using the “median of three” technique).

Time Complexity (Average): $O(n \log n)$.

Time Complexity (Worst): $O(n^2)$.

Space Complexity: $O(\log n)$ auxiliary stack space.

Stable?: No.

```
template<typename It, typename Compare = std::less<>>
void quicksort(It lo, It hi, Compare comp = Compare()) {
    while (hi - lo >= 2) {
        auto pivot = *(lo + (hi - lo) / 2);
        It lt = lo, i = lo, gt = hi;
        while (i != gt) {
            if (comp(*i, pivot)) {
                std::iter_swap(lt++, i++);
            } else if (comp(pivot, *i)) {
                std::iter_swap(i, --gt);
            } else {
                ++i;
            }
        }
        if (lt - lo < hi - gt) {
            quicksort(lo, lt, comp);
            lo = gt;
        } else {
            quicksort(gt, hi, comp);
            hi = lt;
        }
    }
}
```

Merge sort first divides a list into n sublists of one element each, then recursively merges the sublists into sorted order until only a single sorted sublist remains. Merge sort is a stable sort, meaning that it preserves the relative order of elements which compare equal by `operator<` or the custom comparator given.

An analogous function in the C++ standard library is `std::stable_sort()`, except that the implementation here requires sufficient memory to be available. When $O(n)$ auxiliary memory is not available, `std::stable_sort()` falls back to a time complexity of $O(n \log^2 n)$ whereas the implementation here will simply fail.

Time Complexity (Average): $O(n \log n)$.

Time Complexity (Worst): $O(n \log n)$.

Space Complexity: $O(\log n)$ auxiliary stack space and $O(n)$ auxiliary heap space.

Stable?: Yes.

```
template<typename It, typename Compare = std::less<>>
void mergesort(It lo, It hi, Compare comp = Compare()) {
    if (hi - lo < 2) {
        return;
    }
    It mid = lo + (hi - lo - 1) / 2, a = lo, c = mid + 1;
    mergesort(lo, mid + 1, comp);
    mergesort(mid + 1, hi, comp);
    using T = typename std::iterator_traits<It>::value_type;
    std::vector<T> merged;
    merged.reserve(hi - lo);
    while (a <= mid && c < hi) {
        merged.push_back(comp(*c, *a) ? *c++ : *a++);
    }
    merged.insert(merged.end(), a, mid + 1);
    merged.insert(merged.end(), c, hi);
    std::copy(merged.begin(), merged.end(), lo);
}
```

Heapsort first rearranges an array to satisfy the max-heap property. Then, it repeatedly pops the max element of the heap (the left, unsorted subrange), moving it to the beginning of the right, sorted subrange until the entire

range is sorted. Heapsort has a better worst case time complexity than quicksort and also a better space complexity than merge sort.

The C++ standard library equivalent is calling `std::make_heap(lo, hi)`, followed by `std::sort_heap(lo, hi)`.

The loop below has two modes controlled by whether `i > lo`. While `i > lo`, it is heapifying (Floyd's $O(n)$ bottom-up heap build: sift down each non-leaf from right to left). Once `i == lo`, the heap is built, and the loop switches to extraction: repeatedly swap the root to the back of the unsorted region (`*j`) and sift down the new root.

Time Complexity (Average): $O(n \log n)$.

Time Complexity (Worst): $O(n \log n)$.

Space Complexity: $O(1)$ auxiliary.

Stable?: No.

```
template<typename It, typename Compare = std::less<>>
void heapsort(It lo, It hi, Compare comp = Compare()) {
    using T = typename std::iterator_traits<It>::value_type;
    T tmp;
    It i = lo + (hi - lo) / 2, j = hi, parent, child;
    while (true) {
        if (i <= lo) {
            if (--j == lo) {
                return;
            }
            tmp = *j;
            *j = *lo;
        } else {
            tmp = *(--i);
        }
        parent = i;
        child = lo + 2 * (i - lo) + 1;
        while (child < j) {
            if (child + 1 < j && comp(*child, *(child + 1))) {
                child++;
            }
            if (!comp(tmp, *child)) {
                break;
            }
            *parent = *child;
            parent = child;
            child = lo + 2 * (parent - lo) + 1;
        }
        *parent = tmp;
    }
}
```

Insertion sort builds the sorted range one element at a time. It scans left to right, and for each element shifts the larger elements of the already-sorted prefix one position to the right to open a slot where the element belongs. Although its average and worst cases are quadratic, it is simple, stable, in-place, and online (it can sort a stream as elements arrive).

Its strength is being adaptive: on an already-sorted or nearly-sorted range, each element travels only a short distance, giving $O(n)$ best-case time and $O(n + d)$ in general for d total inversions. This is why it outperforms the asymptotically faster sorts on small or nearly-sorted ranges, and why hybrid sorts such as introsort and Timsort fall back to it once a subrange is small enough.

The comparison `comp(key, *(j - 1))` is strict, so an element never moves past an earlier element it compares equal to, which keeps the sort stable.

Time Complexity (Best): $O(n)$ on an already-sorted range.

Time Complexity (Average): $O(n^2)$.

Time Complexity (Worst): $O(n^2)$.

Space Complexity: $O(1)$ auxiliary.

Stable?: Yes.

```
template<typename It, typename Compare = std::less<>>
void insertion_sort(It lo, It hi, Compare comp = Compare()) {
    using T = typename std::iterator_traits<It>::value_type;
    for (It i = lo; i != hi; ++i) {
        T key = *i;
        It j = i;
        while (j != lo && comp(key, *(j - 1))) {
            *j = *(j - 1);
            --j;
        }
        *j = key;
    }
}
```

Comb sort is an improved bubble sort. While bubble sort increments the gap between swapped elements for every inner loop iteration, comb sort fixes the gap size in the inner loop, decreasing it by a particular shrink factor in every iteration of the outer loop. The shrink factor of 1.3 is empirically determined to be the most effective.

Even though the worst case time complexity is $O(n^2)$, a well chosen shrink factor ensures that the gap sizes are co-prime, in turn requiring astronomically large n to make the algorithm exceed $O(n \log n)$ steps. On random arrays, comb sort is only 2-3 times slower than merge sort. Its short code length relative to its good performance makes it a worthwhile algorithm to remember.

Time Complexity (Worst): $O(n^2)$.

Space Complexity: $O(1)$ auxiliary.

Stable?: No.

```
template<typename It, typename Compare = std::less<>>
void combsort(It lo, It hi, Compare comp = Compare()) {
    int gap = static_cast<int>(hi - lo);
    bool swapped = true;
    while (gap > 1 || swapped) {
        if (gap > 1) {
            gap = gap * 10 / 13;
        }
        swapped = false;
        for (It it = lo; it + gap < hi; ++it) {
            if (comp(*(it + gap), *it)) {
                std::iter_swap(it, it + gap);
                swapped = true;
            }
        }
    }
}
```

Radix sort is used to sort integer elements with a constant number of bits in linear time. This implementation works on ranges pointing to any signed or unsigned integer primitive. Signed values are handled by sorting on a key that flips the sign bit, which maps the two's-complement order onto the unsigned order so the most negative value sorts first.

In this implementation, a power of two is chosen to be the base for the sort so that bitwise operations can be easily used to extract digits. This avoids the need to use modulo/exponentiation, which are much more expensive operations. In practice, it's been demonstrated that 2^8 is the best choice for sorting 32-bit integers (approximately 5 times faster than `std::sort()`, and typically 2-4 times faster than radix sort using any other power of two chosen as the base).

Time Complexity: $O(n \cdot w)$ for n integers of w bits each.

Space Complexity: $O(n + 2^b)$ auxiliary for a radix of b bits, i.e. $O(n)$ for constant b .

```
template<typename It>
void radix_sort(It lo, It hi) {
    if (hi - lo < 2) {
        return;
    }
    const int radix_bits = 8;
    const int radix_base = 1 << radix_bits; // e.g. 2^8 = 256
    const int radix_mask = radix_base - 1; // e.g. 2^8 - 1 = 0xFF
    using T = typename std::iterator_traits<It>::value_type;
    using U = typename std::make_unsigned<T>::type;
    int num_bits = 8 * sizeof(T); // 8 bits per byte
    // Sort on an unsigned key. For signed types, flipping the sign bit sends the most negative value
    // to 0, mapping the signed order onto the unsigned order; logical shifts then extract each digit.
    auto key = [](T x) -> U {
        U u = static_cast<U>(x);
        return std::is_signed<T>::value ? (u ^ (U(1) << (8 * sizeof(T) - 1))) : u;
    };
    std::vector<T> buf(hi - lo);
    for (int pos = 0; pos < num_bits; pos += radix_bits) {
        std::vector<int> count(radix_base);
        for (It it = lo; it != hi; ++it) {
            count[(key(*it) >> pos) & radix_mask]++;
        }
        std::vector<T*> bucket(radix_base);
        T *curr = buf.data();
        for (int i = 0; i < radix_base; curr += count[i++]) {
            bucket[i] = curr;
        }
        for (It it = lo; it != hi; ++it) {
            *bucket[(key(*it) >> pos) & radix_mask]++ = *it;
        }
        std::copy(buf.begin(), buf.end(), lo);
    }
}
```

Example Usage

```
#include <cassert>
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <random>
#include <vector>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

template<typename It>
bool sorted(It lo, It hi) {
    while (++lo != hi) {
        if (*lo < *(lo - 1)) {
            return false;
        }
    }
    return true;
}
```

```

}

bool compare_as_ints(double i, double j) {
    return static_cast<int>(i) < static_cast<int>(j);
}

int main() {
    { // Can be used to sort arrays like std::sort().
        vector<int> a{32, 71, 12, 45, 26, 80, 53, 33};
        quicksort(a.begin(), a.end());
        assert(sorted(a.begin(), a.end()));
    }
    { // STL containers work too.
        vector<int> a{32, 71, 12, 45, 26, 80, 53, 33};
        quicksort(a.begin(), a.end());
        assert(sorted(a.begin(), a.end()));
    }
    { // Reverse iterators work as expected.
        vector<int> a{32, 71, 12, 45, 26, 80, 53, 33};
        heapsort(a.rbegin(), a.rend());
        assert(sorted(a.rbegin(), a.rend()));
    }
    { // We can sort doubles just as well.
        vector<double> a{1.1, -5.0, 6.23, 4.123, 155.2};
        combsort(a.begin(), a.end());
        assert(sorted(a.begin(), a.end()));
    }
    { // Insertion sort is adaptive: an already-sorted range is left untouched in O(n).
        vector<int> a{12, 26, 32, 33, 45, 53, 71, 80};
        insertion_sort(a.begin(), a.end());
        assert(sorted(a.begin(), a.end()));
    }
    { // radix_sort() handles signed integers (including negatives), unlike a plain counting sort.
        vector<int> a{32, -71, 12, -45, 26, -80, 53, 33};
        radix_sort(a.begin(), a.end());
        assert(sorted(a.begin(), a.end()));
    }

    // Example from: http://www.cplusplus.com/reference/algorithm/stable\_sort
    const vector<double> a{3.14, 1.41, 2.72, 4.67, 1.73, 1.32, 1.62, 2.58};
    {
        vector<double> v(a);
        cout << "mergesort() with default comparisons: ";
        mergesort(v.begin(), v.end());
        print_range(v.begin(), v.end());
    }
    {
        vector<double> v(a);
        cout << "mergesort() with 'compare_as_ints()': ";
        mergesort(v.begin(), v.end(), compare_as_ints);
        print_range(v.begin(), v.end());
    }
    cout << "-----" << endl;

    std::mt19937 rng(std::random_device{}());
    vector<int> data;
    for (int i = 0; i < 5000000; i++) {
        data.push_back(static_cast<int>(rng()));
    }
    cout << "Sorting five million integers..." << endl;
    cout.precision(3);

    // Each sort is wrapped in a lambda so its name resolves at the call site (the bare names are
    // function templates, not values), then timed on a fresh copy of the same unsorted data.
    auto benchmark = [&](const string &name, auto sort) {
        vector<int> v = data;
        clock_t start = clock();
        sort(v.begin(), v.end());
    };

```

```

    double t = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
    cout << setw(14) << left << name + "(): " << fixed << t << "s" << endl;
    assert(sorted(v.begin(), v.end()));
};
benchmark("std::sort", [](auto lo, auto hi) { std::sort(lo, hi); });
benchmark("quicksort", [](auto lo, auto hi) { quicksort(lo, hi); });
benchmark("mergesort", [](auto lo, auto hi) { mergesort(lo, hi); });
benchmark("heapsort", [](auto lo, auto hi) { heapsort(lo, hi); });
benchmark("combsort", [](auto lo, auto hi) { combsort(lo, hi); });
benchmark("radix_sort", [](auto lo, auto hi) { radix_sort(lo, hi); });
return 0;
}

```

Example Output

```

mergesort() with default comparisons: 1.32 1.41 1.62 1.73 2.58 2.72 3.14 4.67
mergesort() with 'compare_as_ints()': 1.41 1.73 1.32 1.62 2.72 2.58 3.14 4.67
-----
Sorting five million integers...
std::sort(): 0.223s
quicksort(): 0.253s
mergesort(): 0.621s
heapsort(): 0.439s
combsort(): 0.365s
radix_sort(): 0.016s

```

1.1.2 Array Rotation

These functions are equivalent to `std::rotate()`, taking three iterators `lo`, `mid`, and `hi` ($lo \leq mid \leq hi$) to perform a left rotation on the range `[lo, hi)`. After the function call, `[lo, hi)` will consist of the concatenation of elements originally in `[mid, hi) + [lo, mid)`. That is, the range `[lo, hi)` will be rearranged in such a way that the element at `mid` becomes the first element of the new range, and the element before `mid` becomes the last element, all while preserving the relative ordering of elements within the two rotated subarrays.

All three versions below achieve the same result using in-place algorithms.

- `rotate1(lo, mid, hi)` uses a straightforward swapping algorithm requiring ForwardIterators.
- `rotate2(lo, mid, hi)` requires BidirectionalIterators, employing a well-known trick with three simple inversions.
- `rotate3(lo, mid, hi)` requires random-access iterators, applying a juggling algorithm which first divides the range into `gcd(hi - lo, mid - lo)` sets and then rotates the corresponding elements in each set.

Time Complexity: $O(n)$ per call to all versions, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary for all versions

```

#include <algorithm>
#include <numeric>

template<typename It>
void rotate1(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return;
    }
    It next = mid;
    while (lo != next) {
        std::iter_swap(lo++, next++);
        if (next == hi) {
            next = mid;
        } else if (lo == mid) {
            mid = next;
        }
    }
}

```

```

template<typename It>
void rotate2(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return;
    }
    std::reverse(lo, mid);
    std::reverse(mid, hi);
    std::reverse(lo, hi);
}

template<typename It>
void rotate3(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return;
    }
    int n = hi - lo, jump = mid - lo;
    int g = std::gcd(jump, n), cycle = n / g;
    for (int i = 0; i < g; i++) {
        int curr = i, next;
        for (int j = 0; j < cycle - 1; j++) {
            next = curr + jump;
            if (next >= n) {
                next -= n;
            }
            std::iter_swap(lo + curr, lo + next);
            curr = next;
        }
    }
}

```

Example Usage

```

#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> a0, a1, a2, a3;
    for (int i = 0; i < 10000; i++) {
        a0.push_back(i);
    }
    a1 = a2 = a3 = a0;
    int mid = 5678;
    std::rotate(a0.begin(), a0.begin() + mid, a0.end());
    rotate1(a1.begin(), a1.begin() + mid, a1.end());
    rotate2(a2.begin(), a2.begin() + mid, a2.end());
    rotate3(a3.begin(), a3.begin() + mid, a3.end());
    assert(a0 == a1 && a0 == a2 && a0 == a3);

    // Example from: http://en.cppreference.com/w/cpp/algorithm/rotate
    vector<int> a{2, 4, 2, 0, 5, 10, 7, 3, 7, 1};
    cout << "before sort: ";
    for (const auto &x : a) {
        cout << x << " ";
    }
    cout << endl;

    // Insertion sort.
    for (auto i = a.begin(); i != a.end(); ++i) {
        rotate1(std::upper_bound(a.begin(), i, *i), i, i + 1);
    }
    cout << "after sort: ";
    for (const auto &x : a) {

```

```

    cout << x << " ";
}
cout << endl;

// Simple rotation to the left.
rotate2(a.begin(), a.begin() + 1, a.end());
cout << "rotate left: ";
for (const auto &x : a) {
    cout << x << " ";
}
cout << endl;

// Simple rotation to the right.
rotate3(a.rbegin(), a.rbegin() + 1, a.rend());
cout << "rotate right: ";
for (const auto &x : a) {
    cout << x << " ";
}
cout << endl;
return 0;
}

```

Example Output

```

before sort:  2 4 2 0 5 10 7 3 7 1
after sort:   0 1 2 2 3 4 5 7 7 10
rotate left:  1 2 2 3 4 5 7 7 10 0
rotate right: 0 1 2 2 3 4 5 7 7 10

```

1.1.3 Counting Inversions

The number of inversions for a sequence `a` is the number of ordered pairs (i, j) such that $i < j$ and $a[i] > a[j]$. This is roughly how “close” an array is to being sorted, but is *not* the minimum number of swaps required to sort the array. If the array is sorted, then the inversion count is 0. If the array is sorted in decreasing order, then the inversion count is maximal. The following two functions are each techniques to efficiently count inversions. In the merge sort approach, whenever the merge step emits an element from the right half, that element jumps ahead of every unmerged left-half element, and exactly that many inversions are added to the count.

- `inversions(lo, hi)` uses merge sort to return the number of inversions given two random-access iterators as a range `[lo, hi)`. The input range will be sorted after the function call. Optionally, a comparison function object specifying a strict weak ordering may be specified to replace the default `operator<`.
- `inversions(a)` uses coordinate compression and a Fenwick tree to return the number of inversions in an integer vector without modifying it.

Time Complexity:

- $O(n \log n)$ per call to `inversions(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(n \log n)$ per call to `inversions(a)`.

Space Complexity:

- $O(n)$ auxiliary space and $O(\log n)$ stack space for `inversions(lo, hi)`.
- $O(n)$ auxiliary heap space for `inversions(a)`.

```

#include <algorithm>
#include <cstdint>
#include <functional>
#include <iterator>
#include <vector>

template<typename It, typename Compare = std::less<>>
int64_t inversions(It lo, It hi, Compare comp = Compare()) {
    if (hi - lo < 2) {

```

```

    return 0;
}
It mid = lo + (hi - lo - 1) / 2, a = lo, c = mid + 1;
int64_t res = 0;
res += inversions(lo, mid + 1, comp);
res += inversions(mid + 1, hi, comp);
using T = typename std::iterator_traits<It>::value_type;
std::vector<T> merged;
while (a <= mid && c < hi) {
    if (comp(*c, *a)) {
        merged.push_back(*(c++));
        res += (mid - a) + 1;
    } else {
        merged.push_back(*(a++));
    }
}
if (a > mid) {
    for (It it = c; it != hi; ++it) {
        merged.push_back(*it);
    }
} else {
    for (It it = a; it <= mid; ++it) {
        merged.push_back(*it);
    }
}
for (It it = lo; it != hi; ++it) {
    *it = merged[it - lo];
}
return res;
}

int64_t inversions(const std::vector<int> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> values(a.begin(), a.end());
    std::sort(values.begin(), values.end());
    values.erase(std::unique(values.begin(), values.end()), values.end());
    std::vector<int> bit(values.size() + 1, 0);
    int64_t res = 0;
    for (int i = n - 1; i >= 0; i--) {
        int id =
            static_cast<int>(std::lower_bound(values.begin(), values.end(), a[i]) - values.begin()) + 1;
        for (int j = id - 1; j > 0; j -= j & -j) {
            res += bit[j];
        }
        for (int j = id; j < static_cast<int>(bit.size()); j += j & -j) {
            bit[j]++;
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>

int main() {
    {
        std::vector<int> a{6, 9, 1, 14, 8, 12, 3, 2};
        assert(inversions(a.begin(), a.end()) == 16);
    }
    {
        std::vector<int> a{6, 9, 1, 14, 8, 12, 3, 2};
        assert(inversions(a) == 16);
    }
    return 0;
}

```

1.1.4 Coordinate Compression

Given a range of n numerical elements, reassign each element to an integer in the domain $[0, k)$, where k is the number of distinct elements in the original range, while preserving the initial relative ordering of elements. That is, if a is the array of original values and b is the array of compressed values, then every pair of indices i, j in $[0, n)$ shall satisfy $a[i] < a[j]$ if and only if $b[i] < b[j]$.

All implementations below take ranges as ForwardIterators and require `operator<` on the value type. The two `compress()` functions rewrite a range in place and then discard the mapping.

- `compress1(lo, hi)` performs the compression by sorting the array, removing duplicates, and binary searching for the position of each original value.
- `compress2(lo, hi)` achieves the same result by inserting all values into an `std::map`, which automatically removes duplicate values and supports efficient lookups of the compressed values.

`CoordinateCompressor` is a class version that retains the sorted table of distinct values so that arbitrary values can be mapped to and from compressed ranks long after construction (e.g. for offline queries arriving separately from the array being compressed).

- `CoordinateCompressor<T>(lo, hi)` constructs a compressor over the values of $[lo, hi)$, ready for querying. Alternatively, use the default constructor and register values with `add(x)`, then call `build()` once after all values are registered and before the first query.
- `size()` returns the number of distinct registered values k .
- `value(r)` returns the original value with rank r , inverting `rank()`.
- `rank(x)` returns the compressed value (rank) of x in $[0, k)$. x must have been registered.
- `contains(x)` returns whether x is a registered value.

Time Complexity:

- $O(n \log n)$ per call to `compress1(lo, hi)`, `compress2(lo, hi)`, and `CoordinateCompressor(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(1)$ amortized per call to `add(x)`, and $O(m \log m)$ per call to `build()`, where m is the total number of values registered.
- $O(\log k)$ per call to `rank(x)` and `contains(x)`, where k is the number of distinct values.
- $O(1)$ per call to `value(r)` and `size()`.

Space Complexity:

- $O(n)$ auxiliary heap space for `compress1()` and `compress2()`.
- $O(k)$ storage for `CoordinateCompressor`.

```
#include <algorithm>
#include <cassert>
#include <iterator>
#include <map>
#include <vector>

template<typename It>
void compress1(It lo, It hi) {
    using T = typename std::iterator_traits<It>::value_type;
    std::vector<T> v(lo, hi);
    std::sort(v.begin(), v.end());
    v.resize(std::unique(v.begin(), v.end()) - v.begin());
    for (It it = lo; it != hi; ++it) {
        *it = static_cast<int>(std::lower_bound(v.begin(), v.end(), *it) - v.begin());
    }
}

template<typename It>
void compress2(It lo, It hi) {
    using T = typename std::iterator_traits<It>::value_type;
    std::map<T, int> m;
    for (It it = lo; it != hi; ++it) {
```



```

    m[*it] = 0;
}
int id = 0;
for (auto &[key, val] : m) {
    val = id++;
}
for (It it = lo; it != hi; ++it) {
    *it = m[*it];
}
}

template<typename T>
class CoordinateCompressor {
    std::vector<T> values;

public:
    CoordinateCompressor() {}

    template<typename It>
    CoordinateCompressor(It lo, It hi) : values(lo, hi) {
        build();
    }

    int size() const { return static_cast<int>(values.size()); }
    const T &value(int r) const { return values[r]; }
    void add(const T &x) { values.push_back(x); }

    void build() {
        std::sort(values.begin(), values.end());
        values.resize(std::unique(values.begin(), values.end()) - values.begin());
    }

    int rank(const T &x) const {
        int r = static_cast<int>(std::lower_bound(values.begin(), values.end(), x) - values.begin());
        assert(r < size() && !(x < values[r])); // x must be registered.
        return r;
    }

    bool contains(const T &x) const { return std::binary_search(values.begin(), values.end(), x); }
};

```

Example Usage

```

#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

int main() {
    {
        vector<int> a{1, 30, 30, 7, 9, 8, 99, 99};
        compress1(a.begin(), a.end());
        print_range(a.begin(), a.end());
    }
    {
        vector<int> a{1, 30, 30, 7, 9, 8, 99, 99};
        compress2(a.begin(), a.end());
        print_range(a.begin(), a.end());
    }
    { // Non-integral types work too, as long as ints can be assigned to them.

```

```

vector<double> a{0.5, -1.0, 3, -1.0, 20, 0.5};
compress1(a.begin(), a.end());
print_range(a.begin(), a.end());
}
{
vector<int> a{10, 5, 30, 5, 20};
CoordinateCompressor<int> cc(a.begin(), a.end());
assert(cc.size() == 4);
assert(cc.rank(5) == 0);
assert(cc.rank(30) == 3);
assert(cc.value(2) == 20);
assert(cc.contains(10) && !cc.contains(7));
}
{
// Register values incrementally, then build once before querying.
CoordinateCompressor<double> cc;
cc.add(0.5);
cc.add(-1.0);
cc.add(0.5);
cc.build();
assert(cc.size() == 2);
assert(cc.rank(0.5) == 1);
assert(cc.value(0) == -1.0);
}
return 0;
}

```

Example Output

```

0 4 4 1 3 2 5 5
0 4 4 1 3 2 5 5
1 0 2 0 3 1

```

1.1.5 Selection (Quickselect)

Partially sort a range so that one chosen position ends up holding its final sorted element, like `std::nth_element()`.

- `nth_element2(lo, nth, hi)` rearranges the range `[lo, hi)` such that the value pointed to by `nth` is the element that would be there if the range were fully sorted. Furthermore, the range is partitioned such that no value in `[lo, nth)` compares greater than the value pointed to by `nth` and no value in `(nth, hi)` compares less. This requires `operator<` defined on the iterator's value type.

The pivot is chosen uniformly at random, and the partition step groups equal keys together so arrays with many duplicate values remain efficient. After each partition, the search continues in only the side that contains `nth`, shrinking the expected remaining work geometrically for a linear average.

Time Complexity: $O(n)$ on average per call to `nth_element2()`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```

#include <algorithm>
#include <iterator>
#include <random>

template<typename It>
void nth_element2(It lo, It nth, It hi) {
    static std::mt19937 rng(std::random_device{}());
    while (hi - lo > 1) {
        std::uniform_int_distribution<int> dist(0, hi - lo - 1);
        auto pivot = *(lo + dist(rng));
        It lt = lo, i = lo, gt = hi;
        while (i != gt) {
            if (*i < pivot) {
                std::iter_swap(lt++, i++);
            } else if (pivot < *i) {

```

```

        std::iter_swap(i, --gt);
    } else {
        ++i;
    }
}
if (nth < lt) {
    hi = lt;
} else if (nth >= gt) {
    lo = gt;
} else {
    return;
}
}
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> a{5, 6, 4, 3, 2, 6, 7, 9, 3};
    int n = static_cast<int>(a.size());
    nth_element2(a.begin(), a.begin() + n / 2, a.end());
    assert(a[n / 2] == 5);
    // Values left of the median are <= it and values right are >= it (the exact
    // order within each side is randomized, since the pivot is chosen at random).
    for (int i = 0; i < n / 2; i++) {
        assert(a[i] <= a[n / 2]);
    }
    for (int i = n / 2 + 1; i < n; i++) {
        assert(a[i] >= a[n / 2]);
    }
    return 0;
}

```

1.1.6 3-Way Partitioning (Dutch National Flag)

Partitions a range into three consecutive groups according to a pivot value: elements less than the pivot, elements equal to the pivot, and elements greater than the pivot. This is the Dutch National Flag algorithm, and is the core partitioning step used in three-way quicksort and arrays with many duplicate keys. A single pass maintains the three groups in-place using two boundary iterators, swapping each examined element into the region where it belongs.

- `partition_three_way(lo, hi, pivot)` rearranges the range `[lo, hi)` in-place and returns iterators `(mid_lo, mid_hi)`, where `[lo, mid_lo)` is less than `pivot`, `[mid_lo, mid_hi)` is equal to `pivot`, and `[mid_hi, hi)` is greater than `pivot`.
- `sort_012(lo, hi)` sorts a range consisting of only the values 0, 1, and 2.

Time Complexity: $O(n)$ per call, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```

#include <algorithm>
#include <utility>

template<typename It, typename T>
std::pair<It, It> partition_three_way(It lo, It hi, const T &pivot) {
    It lt = lo, i = lo, gt = hi;
    while (i != gt) {

```

```

    if (*i < pivot) {
        std::iter_swap(lt++, i++);
    } else if (pivot < *i) {
        std::iter_swap(i, --gt);
    } else {
        ++i;
    }
}
return {lt, gt};
}

template<typename It>
void sort_012(It lo, It hi) {
    partition_three_way(lo, hi, 1);
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> a{2, 0, 1, 2, 1, 0, 1};
    sort_012(a.begin(), a.end());
    for (int i = 0; i < 7; i++) {
        assert(a[i] == (i < 2 ? 0 : i < 5 ? 1 : 2));
    }

    vector<int> b{4, 2, 5, 3, 3, 1};
    auto [mid_lo, mid_hi] = partition_three_way(b.begin(), b.end(), 3);
    for (auto it = b.begin(); it != mid_lo; ++it) {
        assert(*it < 3);
    }
    for (auto it = mid_lo; it != mid_hi; ++it) {
        assert(*it == 3);
    }
    for (auto it = mid_hi; it != b.end(); ++it) {
        assert(*it > 3);
    }
    return 0;
}

```

1.2 Arrays and Dynamic Programming

1.2.1 Prefix Sums

Precomputes prefix sums so range-sum queries can be answered in constant time. This is one of the most common array preprocessing tools, and also appears as a building block for subarray sums, difference arrays, and two-dimensional grids. Each table entry stores the sum of all elements before it, so any range sum is the difference of two entries; in two dimensions, rectangle sums combine four entries by inclusion-exclusion.

- `prefix_sums(a)` returns array `pref` with `pref[0] = 0` and `pref[i + 1] = a[0] + ... + a[i]`.
- `range_sum(pref, lo, hi)` returns the sum of the inclusive range `[lo, hi]`.
- `prefix_sums_2d(a)` returns a two-dimensional prefix sum table for matrix `a`.
- `rectangle_sum(pref, r1, c1, r2, c2)` returns the sum of the rectangle with top left at `(r1, c1)` and bottom right at `(r2, c2)`.

Time Complexity:

- $O(n)$ per call to `prefix_sums(a)`, where n is the array size.
- $O(m \cdot n)$ per call to `prefix_sums_2d(a)`, where m and n are the number of rows and columns of `a`, respectively.
- $O(1)$ per range or rectangle query.

Space Complexity:

- $O(n)$ for `prefix_sums(a)`.
- $O(m \cdot n)$ for `prefix_sums_2d(a)`.
- $O(1)$ auxiliary for `range_sum()` and `rectangle_sum()`.

```
#include <cstdint>
#include <vector>

std::vector<int64_t> prefix_sums(const std::vector<int> &a) {
    std::vector<int64_t> pref(a.size() + 1, 0);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        pref[i + 1] = pref[i] + a[i];
    }
    return pref;
}

int64_t range_sum(const std::vector<int64_t> &pref, int lo, int hi) {
    return pref[hi + 1] - pref[lo];
}

std::vector<std::vector<int64_t>> prefix_sums_2d(const std::vector<std::vector<int>> &a) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    std::vector<std::vector<int64_t>> pref(rows + 1, std::vector<int64_t>(cols + 1, 0));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            pref[i + 1][j + 1] = a[i][j] + pref[i][j + 1] + pref[i + 1][j] - pref[i][j];
        }
    }
    return pref;
}

int64_t rectangle_sum(
    const std::vector<std::vector<int64_t>> &pref, int r1, int c1, int r2, int c2
) {
    return pref[r2 + 1][c2 + 1] - pref[r1][c2 + 1] - pref[r2 + 1][c1] + pref[r1][c1];
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{3, -1, 4, 1, 5};
    auto pref = prefix_sums(a);
    assert(range_sum(pref, 0, 4) == 12); // Whole array a[0..4]
    assert(range_sum(pref, 1, 3) == 4);  // -1 + 4 + 1

    vector<vector<int>> grid{
        {1, 2, 3},
        {4, 5, 6}
    };
    auto pre2 = prefix_sums_2d(grid);
    assert(rectangle_sum(pre2, 0, 1, 1, 2) == 16); // rows 0..1, cols 1..2
    return 0;
}
```

1.2.2 Difference Array

Applies many range-add updates to an array in linear total time using a difference array. Instead of immediately adding `delta` to every element of an inclusive range `[lo, hi]`, we can add `delta` at `diff[lo]` and subtract it at `diff[hi + 1]`; a final prefix sum reconstructs the updated values. This idea extends to two dimensions: a rectangle update touches just the four corner cells of a difference grid by inclusion-exclusion, and a final two-dimensional prefix sum reconstructs the grid.

- `DifferenceArray(n)` constructs an initially zero array of size `n`.
- `add(lo, hi, delta)` adds `delta` to every position in the inclusive range `[lo, hi]`.
- `build()` returns the final array after all range updates.
- `DifferenceArray2D(rows, cols)` constructs an initially zero grid with `rows` rows and `cols` columns.
- `add(r1, c1, r2, c2, delta)` adds `delta` to every cell of the rectangle with top left at `(r1, c1)` and bottom right at `(r2, c2)`, inclusive.
- `build()` returns the final grid after all rectangle updates.

Time Complexity:

- $O(1)$ per call to `add()` of either version.
- $O(n)$ per call to `build()` of `DifferenceArray`, where n is the array size.
- $O(m \cdot n)$ per call to `build()` of `DifferenceArray2D`, where m and n are the number of rows and columns, respectively.

Space Complexity:

- $O(n)$ for the one-dimensional difference array.
- $O(m \cdot n)$ for the two-dimensional difference grid.

```
#include <stdint>
#include <vector>

class DifferenceArray {
    std::vector<int64_t> diff;

public:
    explicit DifferenceArray(int n) : diff(n + 1, 0) {}

    void add(int lo, int hi, int64_t delta) {
        diff[lo] += delta;
        diff[hi + 1] -= delta;
    }

    std::vector<int64_t> build() const {
        std::vector<int64_t> res(static_cast<int>(diff.size()) - 1);
        int64_t cur = 0;
        for (int i = 0; i < static_cast<int>(res.size()); i++) {
            cur += diff[i];
            res[i] = cur;
        }
        return res;
    }
};

class DifferenceArray2D {
    std::vector<std::vector<int64_t>> diff;

public:
    DifferenceArray2D(int rows, int cols) : diff(rows + 1, std::vector<int64_t>(cols + 1, 0)) {}

    void add(int r1, int c1, int r2, int c2, int64_t delta) {
        diff[r1][c1] += delta;
        diff[r1][c2 + 1] -= delta;
        diff[r2 + 1][c1] -= delta;
        diff[r2 + 1][c2 + 1] += delta;
    }
};
```

```

std::vector<std::vector<int64_t>> build() const {
    int rows = static_cast<int>(diff.size()) - 1;
    int cols = static_cast<int>(diff[0].size()) - 1;
    std::vector<std::vector<int64_t>> res(rows, std::vector<int64_t>(cols));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            int64_t up = i > 0 ? res[i - 1][j] : 0;
            int64_t left = j > 0 ? res[i][j - 1] : 0;
            int64_t diag = i > 0 && j > 0 ? res[i - 1][j - 1] : 0;
            res[i][j] = diff[i][j] + up + left - diag;
        }
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    DifferenceArray d(5);
    d.add(0, 2, 2); // a[0..2] += 2
    d.add(1, 4, 4); // a[1..4] += 4
    d.add(2, 3, -1); // a[2..3] -= 1
    vector<int64_t> a = d.build();
    vector<int64_t> expected{2, 6, 5, 3, 4};
    for (int i = 0; i < static_cast<int>(expected.size()); i++) {
        assert(a[i] == expected[i]);
    }

    DifferenceArray2D d2(3, 4);
    d2.add(0, 0, 1, 2, 1); // Rows 0..1, cols 0..2 += 1
    d2.add(1, 1, 2, 3, 2); // Rows 1..2, cols 1..3 += 2
    vector<vector<int64_t>> g = d2.build();
    vector<vector<int64_t>> expected2{
        {1, 1, 1, 0},
        {1, 3, 3, 2},
        {0, 2, 2, 2}
    };
    assert(g == expected2);
    return 0;
}

```

1.2.3 Two Pointers

Uses two moving indices to process subarrays in linear time. This technique is most useful when expanding the right endpoint of a window monotonically and shrinking the left endpoint monotonically preserves enough state to test a condition or update an answer.

The examples below cover two common patterns: finding the shortest subarray with sum at least a target when all values are nonnegative, and finding the longest subarray with at most k distinct values.

- `min_length_at_least(a, target)` returns the minimum length of a contiguous subarray with sum at least `target`, or `-1` if none exists. Values in `a` must be nonnegative.
- `longest_at_most_k_distinct(a, k)` returns the maximum length of a contiguous subarray containing at most `k` distinct values.

Time Complexity: $O(n)$ expected per call, where n is the array size.

Space Complexity:

- $O(1)$ auxiliary for `min_length_at_least(a, target)`.
- $O(k)$ auxiliary heap space for `longest_at_most_k_distinct(a, k)`.

```
#include <algorithm>
#include <stdint>
#include <unordered_map>
#include <vector>

int min_length_at_least(const std::vector<int> &a, int64_t target) {
    int best = static_cast<int>(a.size()) + 1;
    int64_t sum = 0;
    for (int l = 0, r = 0; r < static_cast<int>(a.size()); r++) {
        sum += a[r];
        while (sum >= target) {
            best = std::min(best, r - l + 1);
            sum -= a[l++];
        }
    }
    return best == static_cast<int>(a.size()) + 1 ? -1 : best;
}

int longest_at_most_k_distinct(const std::vector<int> &a, int k) {
    if (k <= 0) {
        return 0;
    }
    std::unordered_map<int, int> count;
    int best = 0;
    for (int l = 0, r = 0; r < static_cast<int>(a.size()); r++) {
        count[a[r]]++;
        while (static_cast<int>(count.size()) > k) {
            if (--count[a[l]] == 0) {
                count.erase(a[l]);
            }
            l++;
        }
        best = std::max(best, r - l + 1);
    }
    return best;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{2, 3, 1, 2, 4, 3};
    assert(min_length_at_least(a, 7) == 2); // [4, 3].

    vector<int> b{1, 2, 1, 3, 4, 3, 5};
    assert(longest_at_most_k_distinct(b, 2) == 3); // [1, 2, 1].
    return 0;
}
```

1.2.4 Iterated Function Cycle Detection

Given a function f mapping a finite set to itself and a starting value x_0 , return the position and length of the cycle reached by repeatedly applying f . Formally, the sequence $x_0, x_1 = f(x_0), x_2 = f(x_1), \dots, x_n = f(x_{n-1}), \dots$ must eventually repeat some value. If $x_i = x_j$ for $i < j$, then the sequence from x_i to x_{j-1} repeats forever.

This is useful for detecting cycles in functional graphs, pseudo-random generators, Pollard rho style algorithms, degenerate linked lists, and arrays where each index points to the next index. For example, if `nums` has size $n + 1$ and all values are in $[1, n]$, then $f(i) = \text{nums}[i]$ forms a functional graph; the duplicate value is the entry point of the cycle reached from 0.

Floyd’s cycle-finding algorithm, a.k.a. the “tortoise and the hare algorithm”, is a space-efficient default choice: it moves two pointers through the sequence at different speeds, finds a meeting point inside the cycle, then locates the cycle start. Brent’s algorithm is a compact variant that usually calls f fewer times by only resetting the tortoise at powers of two; prefer it when evaluating f is expensive and constant factors matter. In the detection phase, Brent advances one pointer with one call to f per step, while Floyd advances the tortoise once and the hare twice, using three calls to f per step.

- `find_cycle_floyd(f, x0)` returns the reached cycle as a pair `(start, length)`, where `start` is the smallest index i such that x_i is in the cycle, and `length` is the number of distinct values in the cycle.
- `find_cycle_brent(f, x0)` does the same with fewer calls to f in many cases.

Time Complexity: $O(m + n)$ per call, where m is the first index in the cycle and n is the cycle length.

Space Complexity: $O(1)$ auxiliary.

```
#include <utility>

template<typename Fn, typename T>
std::pair<int, int> find_cycle_floyd(Fn f, T x0) {
    T tortoise = f(x0), hare = f(f(x0));
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(f(hare));
    }
    int start = 0;
    tortoise = x0;
    while (tortoise != hare) {
        tortoise = f(tortoise);
        hare = f(hare);
        start++;
    }
    int length = 1;
    hare = f(tortoise);
    while (tortoise != hare) {
        hare = f(hare);
        length++;
    }
    return {start, length};
}

template<typename Fn, typename T>
std::pair<int, int> find_cycle_brent(Fn f, T x0) {
    int power = 1, length = 1;
    T tortoise = x0, hare = f(x0);
    while (tortoise != hare) {
        if (power == length) {
            tortoise = hare;
            power *= 2;
            length = 0;
        }
        hare = f(hare);
        length++;
    }
    hare = x0;
    for (int i = 0; i < length; i++) {
        hare = f(hare);
    }
    int start = 0;
    tortoise = x0;
    while (tortoise != hare) {
        tortoise = f(tortoise);
```

```

    hare = f(hare);
    start++;
}
return {start, length};
}

```

Example Usage

```

#include <cassert>
#include <set>
#include <vector>
using namespace std;

int f(int x) {
    return (123 * x * x + 4567890) % 1337;
}

void verify(int x0, int start, int length) {
    set<int> s;
    int x = x0;
    for (int i = 0; i < start; i++) {
        assert(!s.count(x));
        s.insert(x);
        x = f(x);
    }
    int startx = x;
    s.clear();
    for (int i = 0; i < length; i++) {
        assert(!s.count(x));
        s.insert(x);
        x = f(x);
    }
    assert(startx == x);
}

// Given n + 1 integers within [1, n], where one value in [1, n] is present 2 or more times (but
// other values in [1, n] are present at most once), find the duplicate without modifying the array.
// Interpret each value as a pointer to the next index; the duplicate is the cycle entry.
int find_duplicate_integer(const vector<int> &nums) {
    auto [start, _] = find_cycle_floyd([&](int i) { return nums[i]; }, 0);
    int res = 0;
    for (int i = 0; i < start; i++) {
        res = nums[res];
    }
    return res;
}

int main() {
    int x0 = 0;
    auto [floyd_start, floyd_len] = find_cycle_floyd(f, x0);
    assert(floyd_start == 4 && floyd_len == 2);
    verify(x0, floyd_start, floyd_len);

    auto [brent_start, brent_len] = find_cycle_brent(f, x0);
    assert(brent_start == floyd_start && brent_len == floyd_len);

    assert(find_duplicate_integer({1, 3, 4, 2, 2}) == 2);
    assert(find_duplicate_integer({3, 1, 3, 4, 2}) == 3);
    return 0;
}

```

1.2.5 Sliding Window Maximum

Given an array and a fixed window length k , report the maximum (or minimum) of every contiguous window of that length as the window slides from left to right. A monotonic deque of indices answers all windows in linear total time: it holds the indices of candidates that could still become a window maximum, kept in decreasing order of value. Each new element pops every smaller element off the back, since those can never again be the maximum while the larger newer element is in the window, and indices that fall outside the window are dropped from the front. The front of the deque is always the maximum of the current window. This is the windowed counterpart of the monotonic stack: each index is pushed and popped at most once.

Both functions take a vector of n comparable values and a window length $k \in [1, n]$, returning a vector of $n - k + 1$ results, one per window in left-to-right order.

- `sliding_window_maximum(a, k)` returns the maximum of each window of length k .
- `sliding_window_minimum(a, k)` returns the minimum of each window of length k .

Time Complexity: $O(n)$ per call to both functions, where n is the size of the input.

Space Complexity: $O(n)$ auxiliary heap space for the result, plus $O(k)$ for the deque.

```
#include <deque>
#include <vector>

template<typename T>
std::vector<T> sliding_window_maximum(const std::vector<T> &a, int k) {
    int n = static_cast<int>(a.size());
    std::deque<int> window;
    std::vector<T> res;
    for (int i = 0; i < n; i++) {
        while (!window.empty() && a[window.back()] <= a[i]) {
            window.pop_back();
        }
        window.push_back(i);
        if (window.front() <= i - k) {
            window.pop_front();
        }
        if (i >= k - 1) {
            res.push_back(a[window.front()]);
        }
    }
    return res;
}

template<typename T>
std::vector<T> sliding_window_minimum(const std::vector<T> &a, int k) {
    int n = static_cast<int>(a.size());
    std::deque<int> window;
    std::vector<T> res;
    for (int i = 0; i < n; i++) {
        while (!window.empty() && a[window.back()] >= a[i]) {
            window.pop_back();
        }
        window.push_back(i);
        if (window.front() <= i - k) {
            window.pop_front();
        }
        if (i >= k - 1) {
            res.push_back(a[window.front()]);
        }
    }
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{1, 3, -1, -3, 5, 3, 6, 7};
    assert((sliding_window_maximum(a, 3) == vector<int>{3, 3, 5, 5, 6, 7}));
    assert((sliding_window_minimum(a, 3) == vector<int>{-1, -3, -3, -3, 3, 3}));
    return 0;
}
```

1.2.6 Aggregate Queue

Maintain a first-in, first-out queue that additionally answers, in $O(1)$ amortized time, the fold of an associative operation over all elements currently in the queue. This is the “sliding window aggregation” (SWAG) technique: by pushing new elements and popping old ones as a window advances, it reports an aggregate of every window in linear total time.

Unlike the monotonic deque used for the sliding-window maximum (see 1.2.4), the operation need only be associative, not order-based, so the same structure handles min, max, sum, gcd, matrix products, and other monoids. It is sometimes called a “monotonic queue,” but that term usually denotes the monotonic deque of 1.2.4 (which handles only min/max); the two are different structures. The queue is built from two stacks: a back stack receives pushes while a front stack serves pops, and each stack stores running folds so the two halves combine in $O(1)$. Whenever the front stack empties, the back stack is flipped onto it; this costs $O(1)$ amortized per element, since each element is moved between stacks at most once.

The fold is defined by an associative `combine(a, b)` function. The default code below returns the “min” of the elements; for “sum”, `combine(a, b)` should return `a + b`. The fold is taken in queue order (front to back), so non-commutative operations such as matrix products are also supported.

- `AggregateQueue<T>()` constructs an empty queue.
- `empty()` returns whether the queue is empty.
- `size()` returns the number of elements in the queue.
- `push(x)` appends `x` to the back of the queue.
- `pop()` removes the element at the front of the queue, which must be non-empty.
- `front()` returns the element at the front of the queue, which must be non-empty.
- `fold()` returns `combine()` applied to all elements in queue order; the queue must be non-empty.

Time Complexity:

- $O(1)$ per call to the constructor, `empty()`, `size()`, `front()`, and `fold()`.
- $O(1)$ amortized per call to `push()` and `pop()`.

Space Complexity:

- $O(n)$ for storage of the queue elements, where n is the number of elements.
- $O(1)$ auxiliary per operation, amortized (a `pop()` may flip the back stack onto the front stack).

```
#include <algorithm>
#include <utility>
#include <vector>

template<typename T>
class AggregateQueue {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

    // Each stack entry is (value, fold), where fold combines the value with all entries between it
    // and its end of the queue (the front for front_stack, the back for back_stack).
    std::vector<std::pair<T, T>> front_stack, back_stack;
```

```

public:
    bool empty() const { return front_stack.empty() && back_stack.empty(); }
    int size() const { return static_cast<int>(front_stack.size() + back_stack.size()); }

    void push(const T &x) {
        T acc = back_stack.empty() ? x : combine(back_stack.back().second, x);
        back_stack.emplace_back(x, acc);
    }

    void pop() {
        if (front_stack.empty()) {
            // Flip the back stack onto the front stack, reversing order so the oldest element ends up on
            // top, and recompute the front folds in queue order.
            while (!back_stack.empty()) {
                T x = back_stack.back().first;
                back_stack.pop_back();
                T acc = front_stack.empty() ? x : combine(x, front_stack.back().second);
                front_stack.emplace_back(x, acc);
            }
        }
        front_stack.pop_back();
    }

    T front() const {
        return front_stack.empty() ? back_stack.front().first : front_stack.back().first;
    }

    T fold() const {
        if (front_stack.empty()) {
            return back_stack.back().second;
        }
        if (back_stack.empty()) {
            return front_stack.back().second;
        }
        return combine(front_stack.back().second, back_stack.back().second);
    }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Used directly as a FIFO queue with an O(1) min query.
    AggregateQueue<int> q;
    q.push(5);
    q.push(3);
    q.push(8);
    assert(q.front() == 5 && q.size() == 3);
    assert(q.fold() == 3); // min(5, 3, 8)
    q.pop(); // Removes 5.
    assert(q.fold() == 3); // min(3, 8)
    q.pop(); // Removes 3.
    assert(q.fold() == 8);

    // Canonical use: the minimum of every width-3 window, advancing the window by push/pop.
    vector<int> a{4, 2, 5, 1, 3, 6};
    AggregateQueue<int> w;
    vector<int> mins;
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        w.push(a[i]);
        if (w.size() > 3) {
            w.pop();
        }
    }
}

```

```

    if (w.size() == 3) {
        mins.push_back(w.fold());
    }
}
assert((mins == vector<int>{2, 1, 1, 1})); // min of [4,2,5], [2,5,1], [5,1,3], [1,3,6]
return 0;
}

```

1.2.7 Monotonic Stack

A monotonic stack maintains its elements in sorted order as values are pushed, popping away any entries that would violate the ordering. Scanning an array once while keeping such a stack of indices answers, for every position, where the nearest smaller or larger neighbor lies. These “nearest smaller/greater” relationships underlie many array problems, the most famous being the largest rectangle in a histogram. Each index is pushed and popped at most once, so a full scan runs in linear time.

All functions below take a vector `a` of n comparable values and return a vector of n indices. A “less” query uses a strictly smaller neighbor and a “greater” query uses a strictly larger one; changing the comparison from strict to non-strict (e.g. `>=` to `>`) toggles how ties are handled.

- `previous_less(a)` returns, for each i , the largest index $j < i$ with $a[j] < a[i]$, or -1 if there’s no such index.
- `next_less(a)` returns, for each i , the smallest index $j > i$ with $a[j] < a[i]$, or n if there’s no such index.
- `previous_greater(a)` returns, for each i , the largest index $j < i$ with $a[j] > a[i]$, or -1 if there’s no such index.
- `next_greater(a)` returns, for each i , the smallest index $j > i$ with $a[j] > a[i]$, or n if there’s no such index.
- `largest_rectangle(heights)` returns the maximum area of an axis-aligned rectangle that fits under the given histogram, given an array of `heights` where each bar has width 1.

Time Complexity: $O(n)$ per call to all functions, where n is the size of the input.

Space Complexity: $O(n)$ auxiliary heap space for the stack and result.

```

#include <stack>
#include <vector>

template<typename T>
std::vector<int> previous_less(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && a[s.top()] >= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? -1 : s.top();
        s.push(i);
    }
    return res;
}

template<typename T>
std::vector<int> next_less(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = n - 1; i >= 0; i--) {
        while (!s.empty() && a[s.top()] >= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? n : s.top();
        s.push(i);
    }
    return res;
}

```

```

}

template<typename T>
std::vector<int> previous_greater(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = 0; i < n; i++) {
        while (!s.empty() && a[s.top()] <= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? -1 : s.top();
        s.push(i);
    }
    return res;
}

template<typename T>
std::vector<int> next_greater(const std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> res(n);
    std::stack<int> s;
    for (int i = n - 1; i >= 0; i--) {
        while (!s.empty() && a[s.top()] <= a[i]) {
            s.pop();
        }
        res[i] = s.empty() ? n : s.top();
        s.push(i);
    }
    return res;
}

template<typename T>
T largest_rectangle(const std::vector<T> &heights) {
    int n = static_cast<int>(heights.size());
    std::vector<int> left = previous_less(heights), right = next_less(heights);
    T best = 0;
    for (int i = 0; i < n; i++) {
        T area = heights[i] * (right[i] - left[i] - 1);
        if (area > best) {
            best = area;
        }
    }
    return best;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{2, 1, 4, 3};
    assert((previous_less(a) == vector<int>{-1, -1, 1, 1}));
    assert((next_less(a) == vector<int>{1, 4, 3, 4}));
    assert((previous_greater(a) == vector<int>{-1, 0, -1, 2}));
    assert((next_greater(a) == vector<int>{2, 2, 4, 4}));

    vector<int> hist{2, 1, 5, 6, 2, 3};
    assert(largest_rectangle(hist) == 10);
    return 0;
}

```

1.2.8 Minimum Excluded Value

Computes the minimum excluded nonnegative integer, commonly called the MEX. The MEX of a set is the smallest integer $x \geq 0$ that does not appear in the set. It appears frequently in array problems, game theory with Grundy numbers, and dynamic programming states. Since the MEX of n elements never exceeds n , the one-shot version simply marks which of the values 0 to n occur and scans for the first gap. The dynamic version keeps a count per value plus an ordered set of absent values, whose smallest member is always the current MEX.

- `mex(lo, hi)` returns the MEX of the values in $[lo, hi)$.
- `DynamicMex(n)` maintains counts of values in $[0, n]$, enough to track the MEX of a multiset of at most n relevant nonnegative values.
- `add(x)` inserts one copy of value x if $x \in [0, n]$.
- `remove(x)` removes one copy of value x if present.
- `get()` returns the current MEX.

Time Complexity:

- $O(n)$ per call to `mex(lo, hi)`, where n is the number of values.
- $O(\log n)$ per call to `add(x)`, `remove(x)`, and `get()` for `DynamicMex`.

Space Complexity: $O(n)$ auxiliary heap space for both approaches.

```
#include <set>
#include <vector>

template<typename It>
int mex(It lo, It hi) {
    int n = static_cast<int>(hi - lo);
    std::vector<bool> seen(n + 1, false);
    for (It it = lo; it != hi; ++it) {
        if (0 <= *it && *it <= n) {
            seen[*it] = true;
        }
    }
    for (int x = 0; x <= n; x++) {
        if (!seen[x]) {
            return x;
        }
    }
    return n + 1;
}

class DynamicMex {
    std::vector<int> count;
    std::set<int> missing;

public:
    explicit DynamicMex(int n) : count(n + 1, 0) {
        for (int x = 0; x <= n; x++) {
            missing.insert(x);
        }
    }

    void add(int x) {
        if (0 <= x && x < static_cast<int>(count.size()) && count[x]++ == 0) {
            missing.erase(x);
        }
    }

    void remove(int x) {
        if (0 <= x && x < static_cast<int>(count.size()) && count[x] > 0 && --count[x] == 0) {
            missing.insert(x);
        }
    }
}
```



```
int get() const { return *missing.begin(); }
};
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{0, 1, 4, 2, 1};
    assert(mex(a.begin(), a.end()) == 3);

    DynamicMex m(5);
    m.add(0);
    m.add(1);
    m.add(3);
    assert(m.get() == 2);
    m.add(2);
    assert(m.get() == 4);
    m.remove(1);
    assert(m.get() == 1);
    return 0;
}
```

1.2.9 Longest Increasing Subsequence

Given a range of elements, determine a longest subsequence of the range where all elements are in strictly ascending order. The subsequence is not necessarily contiguous or unique, so only one such answer will be found. This implementation requires random-access iterators and `operator<` on the value type.

The answer is computed by scanning left to right while maintaining, for each length, the smallest possible tail value of an increasing subsequence of that length. Each element extends or improves one of these tails, located in $O(\log n)$ by binary search; predecessor links then reconstruct the subsequence.

- `longest_increasing_subsequence(lo, hi)` returns a longest strictly increasing subsequence of $[lo, hi)$ as a `std::vector` of the selected values, in order.

Time Complexity: $O(n \log n)$ per call to `longest_increasing_subsequence()`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space for `longest_increasing_subsequence()`.

```
#include <algorithm>
#include <iterator>
#include <vector>

template<typename It>
auto longest_increasing_subsequence(It lo, It hi) {
    using T = typename std::iterator_traits<It>::value_type;
    int len = 0, n = static_cast<int>(hi - lo);
    if (n == 0) {
        return std::vector<T>();
    }
    // tail[i] = index (into [lo, hi)) of the last element of the best known LIS of length i+1.
    // prev[i] = index of the predecessor of element i in its LIS (or -1 if first).
    std::vector<int> prev(n), tail(n);
    for (int i = 0; i < n; i++) {
        // Find the tail to extend or improve. Comparing by value through the stored indices keeps
        // `tail` index-based for reconstruction; swap `<` for `<=` to allow a non-decreasing result.
        int pos = static_cast<int>(
            std::lower_bound(
                tail.begin(), tail.begin() + len, i, [&](int t, int x) { return *(lo + t) < *(lo + x); })
        );
    }
```

```

        ) -
        tail.begin()
    );
    if (pos == len) {
        len++;
    }
    prev[i] = pos > 0 ? tail[pos - 1] : -1;
    tail[pos] = i;
}
std::vector<T> res(len);
for (int i = tail[len - 1]; i != -1; i = prev[i]) {
    res[--len] = *(lo + i);
}
return res;
}

```

Example Usage

```

#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    while (lo != hi) {
        cout << *lo++ << " ";
    }
    cout << endl;
}

int main() {
    vector<int> a{-2, -5, 1, 9, 10, 8, 11, 10, 13, 11};
    vector<int> res = longest_increasing_subsequence(a.begin(), a.end());
    print_range(res.begin(), res.end());
    return 0;
}

```

Example Output

```
-5 1 9 10 11 13
```

1.2.10 0-1 Knapsack

Solves the 0-1 knapsack problem: given items with nonnegative integer weights and values, choose each item at most once to maximize total value without exceeding a capacity limit.

The one-dimensional dynamic programming array `dp[w]` stores the best value that can be achieved with total weight at most `w` after processing some prefix of the items. Iterating weights in decreasing order is what enforces the 0-1 constraint; it prevents the same item from being reused during the current item update.

- `knapsack_01(weight, value, capacity)` returns the maximum total value achievable by choosing a 0-1 subset of n items such that their total weight is at most `capacity`, where item `i` has weight `weight[i]` and value `value[i]`. All weights must be nonnegative integers. Items with weight 0 are allowed and can each be taken once.

Time Complexity: $O(n \cdot W)$, where n is the number of items and W is `capacity`.

Space Complexity: $O(W)$ auxiliary heap space.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

```

```
int64_t knapsack_01(
    const std::vector<int> &weight, const std::vector<int64_t> &value, int capacity
) {
    int n = static_cast<int>(weight.size());
    std::vector<int64_t> dp(capacity + 1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = capacity; w >= weight[i]; w--) {
            dp[w] = std::max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> weight{3, 4, 5, 9};
    vector<int64_t> value{4, 5, 7, 10};

    assert(knapsack_01(weight, value, 8) == 11); // Items of weight 3 and 5.
    assert(knapsack_01(weight, value, 10) == 12); // Items of weight 4 and 5.
    return 0;
}
```

1.2.11 Unbounded Knapsack and Coin Change

Solves two common unbounded knapsack variants, where each item may be used any number of times. In the maximum-value version, each item has a weight and value, and the goal is to maximize total value subject to a capacity limit. In the coin change version, each coin denomination may be used any number of times, and the goal is to count unordered ways to form a target sum.

For unbounded maximum-value knapsack, weights are iterated in increasing order so the current item may be reused. For unordered coin change, coins are the outer loop so each multiset of coins is counted once, independent of ordering.

- `unbounded_knapsack(weight, value, capacity)` returns the maximum value achievable with total weight at most `capacity`.
- `count_coin_change(coins, target)` returns the number of unordered ways to make sum `target` using the given coin denominations.
- All capacities must be nonnegative integers. Weights and coin denominations should be positive to avoid infinitely many uses of a zero-weight item.

Time Complexity:

- $O(n \cdot W)$ for `unbounded_knapsack(weight, value, capacity)`, where n is the number of items and W is `capacity`.
- $O(c \cdot t)$ for `count_coin_change(coins, target)`, where c is the number of coin denominations and t is `target`.

Space Complexity:

- $O(W)$ auxiliary heap space for `unbounded_knapsack(weight, value, capacity)`.
- $O(t)$ auxiliary heap space for `count_coin_change(coins, target)`.

```
#include <algorithm>
#include <cstdint>
#include <vector>
```

```
int64_t unbounded_knapsack(
```

```

    const std::vector<int> &weight, const std::vector<int64_t> &value, int capacity
) {
    int n = static_cast<int>(weight.size());
    std::vector<int64_t> dp(capacity + 1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = weight[i]; w <= capacity; w++) {
            dp[w] = std::max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}

int64_t count_coin_change(const std::vector<int> &coins, int target) {
    std::vector<int64_t> dp(target + 1, 0);
    dp[0] = 1;
    for (int coin : coins) {
        for (int sum = coin; sum <= target; sum++) {
            dp[sum] += dp[sum - coin];
        }
    }
    return dp[target];
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> weight{3, 4, 5};
    vector<int64_t> value{4, 5, 7};
    assert(unbounded_knapsack(weight, value, 10) == 14); // Two weight-5 items.

    vector<int> coins{1, 2, 5};
    assert(count_coin_change(coins, 5) == 4); // 5, 2+2+1, 2+1+1+1, 1*5.
    return 0;
}

```

1.2.12 Subset Sum

Given a collection of integers and a target v , the subset sum problem asks for the maximum sum of any subset that is at most v . Two algorithms below solve this problem in complementary regimes, and which one wins depends on whether the bottleneck is the number of items or the size of the weights.

The meet-in-the-middle method splits the collection in half, enumerates every subset sum for each half, sorts one side, and binary searches for the best compatible partner. It makes no assumption about the sign or size of the values, so it handles negative inputs, and it is the method of choice when the number of items n is small (up to roughly 40).

Pisinger’s algorithm targets the opposite regime: nonnegative weights whose maximum value is small, with possibly many items. It first greedily takes a prefix of items while they still fit, which leaves the optimum within one weight of the target. It then repairs that prefix by a balanced sweep, tracking only sums within $\max(\text{weight})$ of the target. The running time is linear in the number of items times the largest weight, with no dependence on the target itself beyond that window. This routine is sometimes called “fast knapsack,” a slight misnomer: it maximizes a subset sum rather than packing items with distinct values, so it is really the bounded-weight case of subset sum rather than the 0-1 value knapsack.

- `max_subset_sum_at_most(lo, hi, v)` returns the maximum sum of any subset of the range $[lo, hi)$ that does not exceed v . Values may be negative. The range is supplied as random-access iterators, and 64-bit integers are used in calculations to avoid overflow.

- `max_subset_sum_bounded(lo, hi, target)` returns the maximum sum of any subset of the range `[lo, hi)` that does not exceed `target`. All weights and `target` must be nonnegative integers. This is faster than meet-in-the-middle when the largest weight is small relative to $2^{n/2}$.

Time Complexity:

- $O(2^{n/2})$ per call to `max_subset_sum_at_most()`, where n is the distance between `lo` and `hi`.
- $O(n \cdot W)$ per call to `max_subset_sum_bounded()`, where n is the number of items and W is the largest weight.

Space Complexity:

- $O(2^{n/2})$ auxiliary heap space for `max_subset_sum_at_most()`.
- $O(W)$ auxiliary heap space for `max_subset_sum_bounded()`.

```
#include <algorithm>
#include <cstdint>
#include <vector>

template<typename It>
int64_t max_subset_sum_at_most(It lo, It hi, int64_t v) {
    int n = static_cast<int>(hi - lo);
    int64_t llen = 1LL << (n / 2), hlen = 1LL << (n - n / 2);
    std::vector<int64_t> lsum(llen), hsum(hlen);
    for (int64_t mask = 1; mask < llen; mask++) {
        int bit = __builtin_ctzll(mask);
        lsum[mask] = lsum[mask ^ (1LL << bit)] + *(lo + bit);
    }
    for (int64_t mask = 1; mask < hlen; mask++) {
        int bit = __builtin_ctzll(mask);
        hsum[mask] = hsum[mask ^ (1LL << bit)] + *(lo + n / 2 + bit);
    }
    std::sort(hsum.begin(), hsum.end());
    int64_t res = INT64_MIN;
    for (int64_t x : lsum) {
        auto it = std::upper_bound(hsum.begin(), hsum.end(), v - x);
        if (it != hsum.begin()) {
            res = std::max(res, x + *--it);
        }
    }
    return res;
}

template<typename It>
int64_t max_subset_sum_bounded(It lo, It hi, int target) {
    int n = static_cast<int>(hi - lo);
    // Greedily take a prefix while it still fits; the optimum then lies within one weight of target.
    int sum = 0, b = 0;
    while (b < n && sum + static_cast<int>(*(lo + b)) <= target) {
        sum += static_cast<int>(*(lo + b++));
    }
    if (b == n) {
        return sum; // Every item fits, so the whole collection is the best subset.
    }
    int m = 0;
    for (It it = lo; it != hi; ++it) {
        m = std::max(m, static_cast<int>(*it));
    }
    // reach[s - target + m] = largest prefix boundary that can realize a balanced subset of sum s.
    // Only sums within m of target are tracked, which is where the optimum must lie.
    std::vector<int> reach(2 * m, -1);
    reach[sum - target + m] = b;
    for (int i = b; i < n; i++) {
        std::vector<int> prev = reach;
        int wi = static_cast<int>(*(lo + i));
        for (int x = 0; x < m; x++) { // Add item i to sums currently below target.
            reach[x + wi] = std::max(reach[x + wi], prev[x]);
        }
    }
}
```

```

    for (int x = 2 * m - 1; x > m; x--) { // Drop an earlier item from sums above target.
        for (int j = std::max(0, prev[x]); j < reach[x]; j++) {
            int wj = static_cast<int>(*(lo + j));
            reach[x - wj] = std::max(reach[x - wj], j);
        }
    }
}
int s = target;
while (reach[s - target + m] < 0) {
    s--;
}
return s;
}

```

Example Usage

```

#include <cassert>

int main() {
    std::vector<int> a{9, 1, 5, 0, 1, 11, 5};
    assert(max_subset_sum_at_most(a.begin(), a.end(), 8) == 7);
    std::vector<int> b{-7, -3, -2, 5, 8};
    assert(max_subset_sum_at_most(b.begin(), b.end(), 0) == 0);

    // The bounded-weight method agrees with meet-in-the-middle on nonnegative inputs.
    assert(max_subset_sum_bounded(a.begin(), a.end(), 8) == 7);
    std::vector<int> c{3, 34, 4, 12, 5, 2};
    assert(max_subset_sum_bounded(c.begin(), c.end(), 9) == 9); // 4 + 5.
    assert(max_subset_sum_bounded(c.begin(), c.end(), 10) == 10); // 3 + 5 + 2.
    assert(max_subset_sum_bounded(c.begin(), c.end(), 100) == 60); // All items fit.
    return 0;
}

```

1.2.13 Largest Zero Submatrix

Given a rectangular matrix with m rows and n columns consisting of only 0's and 1's as a two-dimensional vector of `bool`, return the area of the largest rectangular submatrix consisting of only 0's. This solution sweeps the rows from top to bottom, maintaining for each column the height of the run of 0's ending at the current row. Each row then reduces to finding the maximum rectangular area under a histogram, which is solved in linear time with a monotonic stack.

- `max_zero_submatrix(a)` returns the area of the largest all-zero rectangular submatrix of `a`, a two-dimensional vector of `bool`.

Time Complexity: $O(m \cdot n)$ per call to `max_zero_submatrix()`, where m is the number of rows and n is the number of columns in the matrix.

Space Complexity: $O(n)$ auxiliary heap space, where n is the number of columns in the matrix.

```

#include <algorithm>
#include <stack>
#include <vector>

int max_zero_submatrix(const std::vector<std::vector<bool>> &a) {
    if (a.empty()) {
        return 0;
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    int res = 0;
    // last1[j] = row index of the most recent 1 in column j (-1 if none yet).
    // left_wall[j] = nearest col left of j whose last1 value >= last1[j] (exclusive left bound).
    // right_wall[j] = nearest col right of j whose last1 value >= last1[j] (exclusive right bound).

```

```

std::vector<int> last1(cols, -1), left_wall(cols), right_wall(cols);
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (a[i][j]) {
            last1[j] = i;
        }
    }
    std::stack<int> s;
    for (int j = 0; j < cols; j++) {
        while (!s.empty() && last1[s.top()] <= last1[j]) {
            s.pop();
        }
        left_wall[j] = s.empty() ? -1 : s.top();
        s.push(j);
    }
    while (!s.empty()) {
        s.pop();
    }
    for (int j = cols - 1; j >= 0; j--) {
        while (!s.empty() && last1[s.top()] <= last1[j]) {
            s.pop();
        }
        right_wall[j] = s.empty() ? cols : s.top();
        s.push(j);
    }
    for (int j = 0; j < cols; j++) {
        res = std::max(res, (i - last1[j]) * (right_wall[j] - left_wall[j] - 1));
    }
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<bool>> a{
        {1, 0, 1, 1, 0, 0},
        {1, 0, 0, 1, 0, 0},
        {0, 0, 0, 0, 0, 1},
        {1, 0, 0, 1, 0, 0},
        {1, 0, 1, 0, 0, 1}
    };
    assert(max_zero_submatrix(a) == 6);
    return 0;
}

```

1.3 Greedy and Scheduling

1.3.1 Interval Scheduling

Selects the maximum number of non-overlapping intervals using the classic greedy algorithm that sorts by earliest finish time. This is also known as activity selection.

Intervals are represented as half-open ranges `[start, finish)`, so two intervals are compatible if the next interval's `start` is at least the previous interval's `finish`.

- `interval_scheduling(intervals)` returns one maximum-size compatible subset of intervals, in the order they are selected, from an input vector of `Interval` with fields `start`, `finish`, and the original `id`.

Time Complexity: $O(n \log n)$ per call due to sorting, where n is the number of intervals.

Space Complexity: $O(n)$ auxiliary heap space for sorting and the answer.

```
#include <algorithm>
#include <climits>
#include <vector>

struct Interval {
    int start, finish, id;
};

std::vector<Interval> interval_scheduling(std::vector<Interval> intervals) {
    std::sort(intervals.begin(), intervals.end(), [](const Interval &a, const Interval &b) {
        return a.finish != b.finish ? a.finish < b.finish : a.start < b.start;
    });
    std::vector<Interval> res;
    int last_finish = INT_MIN;
    for (const auto &iv : intervals) {
        if (iv.start >= last_finish) {
            res.push_back(iv);
            last_finish = iv.finish;
        }
    }
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<Interval> intervals{{1, 4, 0}, {3, 5, 1}, {0, 6, 2}, {5, 7, 3}, {8, 9, 4}};
    vector<Interval> chosen = interval_scheduling(intervals);
    assert(chosen.size() == 3);
    assert(chosen[0].id == 0);
    assert(chosen[1].id == 3);
    assert(chosen[2].id == 4);
    return 0;
}
```

1.3.2 Weighted Interval Scheduling

Selects a maximum-weight subset of non-overlapping intervals using dynamic programming and binary search.

Unlike unweighted interval scheduling, the earliest finish-time greedy choice is not sufficient when intervals have weights. With intervals sorted by finish time, the best total for the first i intervals either skips interval i or adds its weight to the best total over intervals finishing no later than its start, with that predecessor located by binary search.

Intervals are represented as half-open ranges $[\text{start}, \text{finish})$, so two intervals are compatible if the next interval's `start` is at least the previous interval's `finish`.

- `weighted_interval_scheduling(intervals)` returns the maximum total weight of a compatible subset from an input vector of `WeightedInterval` with fields `start`, `finish`, and `weight`.

Time Complexity: $O(n \log n)$ per call due to sorting and binary searching compatible intervals.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <algorithm>
```



```

#include <cstdint>
#include <vector>

struct WeightedInterval {
    int start, finish;
    int64_t weight;
};

int64_t weighted_interval_scheduling(std::vector<WeightedInterval> intervals) {
    std::sort(
        intervals.begin(), intervals.end(), [](const WeightedInterval &a, const WeightedInterval &b) {
            return a.finish != b.finish ? a.finish < b.finish : a.start < b.start;
        }
    );
    int n = static_cast<int>(intervals.size());
    std::vector<int> finish;
    finish.reserve(n);
    for (const auto &iv : intervals) {
        finish.push_back(iv.finish);
    }
    std::vector<int64_t> dp(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int j = std::upper_bound(finish.begin(), finish.end(), intervals[i - 1].start) - finish.begin();
        dp[i] = std::max(dp[i - 1], dp[j] + intervals[i - 1].weight);
    }
    return dp[n];
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<WeightedInterval> intervals{{1, 3, 5}, {2, 5, 6}, {4, 6, 5}, {6, 7, 4}, {5, 8, 11}};
    assert(weighted_interval_scheduling(intervals) == 17);
    return 0;
}

```

1.3.3 Interval Partitioning

Assigns intervals to the minimum number of groups so that no two intervals in the same group overlap. This is the classic interval partitioning problem, also known as finding the minimum number of rooms needed for meetings.

The greedy algorithm sorts intervals by start time and reuses the group whose current finish time is earliest whenever possible. Otherwise, it creates a new group.

- `interval_partitioning(intervals)` returns a vector `room`, where `room[id]` is the assigned room for each input interval given as a vector of `PartitionInterval` with fields `start`, `finish`, and `id`. The number of rooms used in the returned assignment is $1 + \max(\text{room}[\text{id}])$, or 0 if there are no intervals.

Time Complexity: $O(n \log n)$ per call due to sorting and priority queue operations.

Space Complexity: $O(n)$ auxiliary heap space.

```

#include <algorithm>
#include <functional>
#include <queue>
#include <utility>
#include <vector>

struct PartitionInterval {

```

```

    int start, finish, id;
};

std::vector<int> interval_partitioning(std::vector<PartitionInterval> intervals) {
    std::sort(
        intervals.begin(), intervals.end(),
        [](const PartitionInterval &a, const PartitionInterval &b) {
            return a.start != b.start ? a.start < b.start : a.finish < b.finish;
        }
    );
    std::vector<int> room(intervals.size());
    std::priority_queue<
        std::pair<int, int>, std::vector<std::pair<int, int>>, std::greater<std::pair<int, int>>>
    > pq;
    int rooms = 0;
    for (const auto &[start, finish, id] : intervals) {
        if (!pq.empty() && pq.top().first <= start) {
            int r = pq.top().second;
            pq.pop();
            room[id] = r;
        } else {
            room[id] = rooms++;
        }
        pq.emplace(finish, room[id]);
    }
    return room;
}

```

Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    vector<PartitionInterval> intervals{{0, 30, 0}, {5, 10, 1}, {15, 20, 2}};
    vector<int> room = interval_partitioning(intervals);
    assert(1 + *max_element(room.begin(), room.end()) == 2);
    assert(room[1] == room[2]);
    assert(room[0] != room[1]);
    return 0;
}

```

1.3.4 Deadline Scheduling

Schedules unit-time jobs with deadlines and profits to maximize total profit. Each job takes one time slot and must be scheduled at or before its deadline. The greedy algorithm processes jobs in decreasing profit order and places each chosen job in the latest available slot before its deadline.

This is a compact scheduling pattern that combines sorting with a disjoint-set forest over available time slots.

- `schedule_deadline_jobs(jobs)` returns the maximum total profit for a vector of `Job` with fields `deadline` and `profit`. Deadlines are positive integer time slots.

Time Complexity: $O(n \log n)$ per call due to sorting, with near-constant disjoint-set operations.

Space Complexity: $O(n)$ auxiliary heap space.

```

#include <algorithm>
#include <cstdint>
#include <numeric>
#include <vector>

```

```

struct Job {
    int deadline;
    int64_t profit;
};

class SlotDSU {
    std::vector<int> root;

public:
    explicit SlotDSU(int n) : root(n + 1) { std::iota(root.begin(), root.end(), 0); }

    int find_root(int u) {
        if (root[u] != u) {
            root[u] = find_root(root[u]);
        }
        return root[u];
    }

    void occupy(int u) { root[u] = find_root(u - 1); }
};

int64_t schedule_deadline_jobs(std::vector<Job> jobs) {
    std::sort(jobs.begin(), jobs.end(), [](const Job &a, const Job &b) {
        return a.profit != b.profit ? a.profit > b.profit : a.deadline < b.deadline;
    });
    int max_deadline = 0;
    for (const auto &j : jobs) {
        max_deadline = std::max(max_deadline, j.deadline);
    }
    SlotDSU slots(max_deadline);
    int64_t res = 0;
    for (const auto &j : jobs) {
        int slot = slots.find_root(j.deadline);
        if (slot > 0) {
            res += j.profit;
            slots.occupy(slot);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<Job> jobs{{2, 100}, {1, 19}, {2, 27}, {1, 25}, {3, 15}};
    assert(schedule_deadline_jobs(jobs) == 142);
    return 0;
}

```

1.3.5 Minimizing Late Jobs (Moore-Hodgson)

Selects a maximum-size subset of jobs that can all be completed by their deadlines. Each job has a processing time and deadline, and all jobs are available at time 0. The Moore-Hodgson algorithm sorts by deadline and keeps the accepted jobs in a max-heap by processing time; whenever the schedule becomes late, it removes the longest accepted job.

- `maximize_on_time_jobs(jobs)` returns the maximum number of jobs that can finish by their deadlines for an input vector of `TimedJob` with fields `duration` and `deadline`.

Time Complexity: $O(n \log n)$ per call due to sorting and heap operations.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <algorithm>
#include <queue>
#include <vector>

struct TimedJob {
    int duration, deadline;
};

int maximize_on_time_jobs(std::vector<TimedJob> jobs) {
    std::sort(jobs.begin(), jobs.end(), [](const TimedJob &a, const TimedJob &b) {
        return a.deadline != b.deadline ? a.deadline < b.deadline : a.duration < b.duration;
    });
    std::priority_queue<int> durations;
    int time = 0;
    for (const auto &j : jobs) {
        time += j.duration;
        durations.push(j.duration);
        if (time > j.deadline) {
            time -= durations.top();
            durations.pop();
        }
    }
    return static_cast<int>(durations.size());
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<TimedJob> jobs{{3, 4}, {2, 3}, {1, 2}, {2, 7}};
    assert(maximize_on_time_jobs(jobs) == 3);
    return 0;
}
```

1.4 Streaming and Randomized Algorithms

1.4.1 Fisher-Yates Shuffle

Randomly permutes a range in-place using the Fisher-Yates shuffle. Each possible permutation is produced with equal probability, assuming the random number source is uniform.

The algorithm scans from right to left. At each position i , it chooses a random position j from $[0, i]$ and swaps the two elements. This fixes one uniformly random remaining element at a time.

- `fisher_yates_shuffle(lo, hi)` randomly shuffles the range `[lo, hi)`.

Time Complexity: $O(n)$ per call, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```
#include <algorithm>
#include <random>

template<typename It>
```

```

void fisher_yates_shuffle(It lo, It hi) {
    static std::mt19937 rng(std::random_device{}());
    int n = static_cast<int>(hi - lo);
    for (int i = n - 1; i > 0; i--) {
        std::uniform_int_distribution<int> dist(0, i);
        int j = dist(rng);
        std::iter_swap(lo + i, lo + j);
    }
}

```

Example Usage

```

#include <algorithm>
#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> a{1, 2, 3, 4, 5}, original(a);
    fisher_yates_shuffle(a.begin(), a.end());
    sort(a.begin(), a.end());
    assert(a == original);
    return 0;
}

```

1.4.2 Reservoir Sampling

Selects uniformly random samples from a stream whose length may be unknown in advance. Reservoir sampling keeps only the chosen sample or samples in memory while processing each element once. The first arrivals fill the reservoir, and the i -th element then replaces a uniformly chosen occupant with probability $1/i$ (or k/i when keeping k samples), which by induction leaves every element seen so far equally likely to be in the sample.

Each class maintains its reservoir incrementally; call `add(x)` once per stream element in any order, then `get()` to retrieve the result.

- `ReservoirSampleOne<T>()` constructs a single-element sampler.
- `ReservoirSampleK<T>(k)` constructs a k -element sampler.
- `add(x)` incorporates one more stream element.
- `get()` returns the current sample. For `ReservoirSampleOne`, `get()` requires at least one `add()` call; for `ReservoirSampleK`, returns the full reservoir (may be fewer than k elements if the stream was shorter).
- `count()` returns the number of stream elements seen so far.

Time Complexity:

- $O(1)$ per call to `add(x)`.
- $O(n)$ to process a range of n elements.

Space Complexity:

- $O(1)$ auxiliary for `ReservoirSampleOne`.
- $O(k)$ auxiliary heap space for `ReservoirSampleK`.

```

#include <cassert>
#include <random>
#include <vector>

int rand_int(int lo, int hi) {
    static std::mt19937 rng(std::random_device{}());
    return std::uniform_int_distribution<int>(lo, hi)(rng);
}

```

```

template<typename T>
class ReservoirSampleOne {
    T sample{};
    int seen;

public:
    ReservoirSampleOne() : seen(0) {}

    void add(const T &x) {
        if (rand_int(0, seen++) == 0) {
            sample = x;
        }
    }

    const T &get() const {
        assert(seen > 0);
        return sample;
    }

    int count() const { return seen; }
};

template<typename T>
class ReservoirSampleK {
    int k, seen;
    std::vector<T> reservoir;

public:
    explicit ReservoirSampleK(int k) : k(k), seen(0) {}

    void add(const T &x) {
        ++seen;
        if (static_cast<int>(reservoir.size()) < k) {
            reservoir.push_back(x);
        } else {
            int j = rand_int(0, seen - 1);
            if (j < k) {
                reservoir[j] = x;
            }
        }
    }

    const std::vector<T> &get() const { return reservoir; }
    int count() const { return seen; }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{10, 20, 30, 40, 50};

    // Streaming interface: feed elements one at a time.
    ReservoirSampleOne<int> s1;
    for (int x : a) {
        s1.add(x);
    }
    int one = s1.get();
    assert(one == 10 || one == 20 || one == 30 || one == 40 || one == 50);

    ReservoirSampleK<int> sk(3);
    for (int x : a) {
        sk.add(x);
    }
}

```

```

    assert(sk.get().size() == 3);
    return 0;
}

```

1.4.3 Maximum Subarray Sum (Kadane)

Given an array of numbers, determine the maximum possible sum of any contiguous subarray. Kadane's algorithm scans through the array, at each index computing the maximum nonnegative sum subarray ending there. Either this subarray is empty (in which case its sum is zero), or it consists of one more element than the maximum sequence ending at the previous position. This can be adapted to compute the maximal submatrix sum as well.

- `max_subarray_sum(lo, hi, &res_lo, &res_hi)` returns the maximal subarray sum for the range `[lo, hi)`, where `lo` and `hi` are random-access iterators to numeric types. This implementation requires operators `+` and `<` to be defined on the iterators' value type. Optionally, two `int` pointers may be passed to store the inclusive boundary indices `[res_lo, res_hi]` of the resulting subarray. By convention, the empty subarray is allowed, so an input range consisting of only negative values returns 0 with an empty result interval.
- `max_submatrix_sum(a, &r1, &c1, &r2, &c2)` returns the largest sum of any rectangular submatrix for a matrix `a` with m rows and n columns. This implementation requires operators `+` and `<` to be defined on the iterators' value type. Optionally, four `int` pointers may be passed to store the boundary indices of the resulting subarray, with `(r1, c1)` specifying the top-left index and `(r2, c2)` specifying the bottom-right index. By convention, the empty submatrix is allowed, so an input matrix consisting of only negative values returns 0 with an empty result interval.

Time Complexity:

- $O(n)$ per call to `max_subarray_sum()`, where n is the distance between `lo` and `hi`.
- $O(m \cdot n^2)$ per call to `max_submatrix_sum()`, where m is the number of rows and n is the number of columns in the matrix.

Space Complexity:

- $O(1)$ auxiliary for `max_subarray_sum()`.
- $O(m)$ auxiliary heap space for `max_submatrix_sum()`, where m is the number of rows in the matrix.

```

#include <algorithm>
#include <cstdint>
#include <iterator>
#include <vector>

template<typename It>
auto max_subarray_sum(It lo, It hi, int *res_lo = nullptr, int *res_hi = nullptr) {
    using T = typename std::iterator_traits<It>::value_type;
    if (lo == hi) {
        return T();
    }
    int curr_begin = 0, begin = 0, end = -1;
    T sum = 0, max_sum = 0;
    for (It it = lo; it != hi; ++it) {
        sum += *it;
        if (sum < 0) {
            sum = 0;
            curr_begin = (it - lo) + 1;
        } else if (max_sum < sum) {
            max_sum = sum;
            begin = curr_begin;
            end = it - lo;
        }
    }
    if (res_lo != nullptr && res_hi != nullptr) {
        *res_lo = begin;
        *res_hi = end;
    }
}

```

```

    }
    return max_sum;
}

template<typename T>
T max_submatrix_sum(
    const std::vector<std::vector<T>> &a, int *r1 = nullptr, int *c1 = nullptr, int *r2 = nullptr,
    int *c2 = nullptr
) {
    if (a.empty() || a[0].empty()) {
        return T();
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    T sum, max_sum = 0;
    for (int clo = 0; clo < cols; clo++) {
        std::vector<T> sums(rows, 0);
        for (int chi = clo; chi < cols; chi++) {
            for (int i = 0; i < rows; i++) {
                sums[i] += a[i][chi];
            }
            int rlo, rhi;
            sum = max_subarray_sum(sums.begin(), sums.end(), &rlo, &rhi);
            if (max_sum < sum) {
                max_sum = sum;
                if (r1 != nullptr && c1 != nullptr && r2 != nullptr && c2 != nullptr) {
                    *r1 = rlo;
                    *c1 = clo;
                    *r2 = rhi;
                    *c2 = chi;
                }
            }
        }
    }
    return max_sum;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    {
        vector<int> a{-2, -1, -3, 4, -1, 2, 1, -5, 4};
        int lo = 0, hi = 0;
        assert(max_subarray_sum(a.begin(), a.begin() + 3) == 0);
        assert(max_subarray_sum(a.begin(), a.end(), &lo, &hi) == 6);
        cout << "Maximal sum subarray:" << endl;
        for (int i = lo; i <= hi; i++) {
            cout << a[i] << " ";
        }
        cout << endl;
    }
    {
        vector<vector<int>> a{{0, -2, -7, 0, 5},
                           {9, 2, -6, 2, -4},
                           {-4, 1, -4, 1, 0},
                           {-1, 8, 0, -2, 3}};
        int r1 = 0, c1 = 0, r2 = 0, c2 = 0;
        assert(max_submatrix_sum(a, &r1, &c1, &r2, &c2) == 15);
        cout << "\nMaximal sum submatrix:" << endl;
        for (int i = r1; i <= r2; i++) {
            for (int j = c1; j <= c2; j++) {
                cout << a[i][j] << " ";
            }
        }
    }
}

```



```

        cout << endl;
    }
}
return 0;
}

```

Example Output

```

Maximal sum subarray:
4 -1 2 1

Maximal sum submatrix:
9 2
-4 1
-1 8

```

1.4.4 Majority Element (Boyer-Moore)

Given a range of n elements, find the majority element: an element that occurs strictly more than $\lfloor n/2 \rfloor$ times in the range.

The Boyer-Moore voting algorithm performs a first pass while maintaining a single candidate and a counter, incrementing it on a match and decrementing on a mismatch, replacing the candidate when the counter reaches zero; any majority element necessarily survives as the final candidate. The second pass counts the candidate's occurrences to confirm it truly holds a majority. The range is therefore traversed twice, so it only needs to be supplied as ForwardIterators rather than random-access.

If a majority element is guaranteed in advance to exist, the verification pass can be dropped and the candidate returned directly. The candidate phase alone is single-pass, so that variant could be written as a “true” streaming API with one element per call.

- `majority(lo, hi)` returns an iterator to the first occurrence of the majority element of the range $[lo, hi)$, or `hi` if no majority element exists. Requires `operator==` on the value type.

Time Complexity: $O(n)$ per call to `majority()`, where n is the size of the array.

Space Complexity: $O(1)$ auxiliary.

```

template<typename It>
It majority(It lo, It hi) {
    int count = 0, n = 0;
    It candidate = lo;
    for (It it = lo; it != hi; ++it) {
        n++; // Tally the size to avoid a O(n) std::distance() pass in case of ForwardIterators.
        if (count == 0) {
            candidate = it;
            count = 1;
        } else if (*it == *candidate) {
            count++;
        } else {
            count--;
        }
    }
    // Second pass: verify the candidate. Omit this if a majority is guaranteed to exist.
    count = 0;
    for (It it = lo; it != hi; ++it) {
        if (*it == *candidate) {
            count++;
        }
    }
    if (count <= n / 2) {
        return hi;
    }
    return candidate;
}

```

```
}

```

Example Usage

```
#include <cassert>
#include <vector>

int main() {
    std::vector<int> a{3, 2, 3, 1, 3};
    assert(*majority(a.begin(), a.end()) == 3);
    std::vector<int> b{2, 3, 3, 3, 2, 1};
    assert(majority(b.begin(), b.end()) == b.end());
    return 0;
}
```

1.4.5 Online Statistics (Welford)

Maintains the mean and variance of a stream of numbers in one pass using Welford's algorithm. This is more numerically stable than maintaining the sum of values and the sum of squares separately. Each new value shifts the running mean by its deviation divided by the new count, and accumulates the product of its deviations from the old and new means into a running sum of squared deviations, from which both population and sample variance follow directly.

- `OnlineStatistics()` constructs an empty summary.
- `add(x)` incorporates one more value `x` into the summary.
- `count()` returns the number of values seen so far.
- `mean()` returns the current arithmetic mean.
- `variance_population()` returns population variance, dividing by n .
- `variance_sample()` returns sample variance, dividing by $n - 1$.

Time Complexity: $O(1)$ per call to `add(x)` and each query.

Space Complexity: $O(1)$ auxiliary.

```
class OnlineStatistics {
    int n;
    double avg, m2;

public:
    OnlineStatistics() : n(0), avg(0), m2(0) {}

    void add(double x) {
        n++;
        double delta = x - avg;
        avg += delta / n;
        double delta2 = x - avg;
        m2 += delta * delta2;
    }

    int count() const { return n; }
    double mean() const { return avg; }
    double variance_population() const { return n == 0 ? 0 : m2 / n; }
    double variance_sample() const { return n <= 1 ? 0 : m2 / (n - 1); }
};
```

Example Usage

```
#include <cassert>
#include <cmath>
```

```

bool close(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    OnlineStatistics stats;
    for (int x = 1; x <= 5; x++) {
        stats.add(x);
    }
    assert(stats.count() == 5);
    assert(close(stats.mean(), 3.0));
    assert(close(stats.variance_population(), 2.0));
    assert(close(stats.variance_sample(), 2.5));
    return 0;
}

```

1.4.6 Frequent Elements (Misra-Gries)

Finds candidate frequent elements in a stream using the Misra-Gries algorithm. With parameter k , the algorithm keeps at most $k - 1$ counters and guarantees that every value occurring more than $\lfloor n/k \rfloor$ times appears among the returned candidates. Each element increments its counter if already tracked, claims a counter if one of the $k - 1$ slots is free, and otherwise decrements every counter, discarding those that reach zero. Since a decrement step cancels k distinct occurrences at once, it can happen at most $\lfloor n/k \rfloor$ times, so any value occurring more often must survive.

The candidates are not automatically verified, since the algorithm intentionally uses sublinear memory and does not retain the stream. If exact frequencies are needed, make a second pass over the input and count only the returned candidates.

- `misra_gries(lo, hi, k)` returns a hash table of candidate values to their residual counters. Use `k = 2` for the Boyer-Moore majority-candidate special case.

Time Complexity: $O(n)$ amortized per call on average: each element either hits an existing counter, taking $O(1)$, or triggers a full-table decrement, taking $O(k)$, but the table-full event can occur at most $n/(k - 1)$ times total, so the amortized cost per element is $O(1 + k/(k - 1)) = O(1)$. In the worst case, a single call is $O(n \cdot k)$, but that cannot be sustained across a long stream.

Space Complexity: $O(k)$ auxiliary heap space.

```

#include <unordered_map>
#include <vector>

template<typename T, typename It>
std::unordered_map<T, int> misra_gries(It lo, It hi, int k) {
    std::unordered_map<T, int> count;
    for (It it = lo; it != hi; ++it) {
        if (auto found = count.find(*it); found != count.end()) {
            found->second++;
        } else if (static_cast<int>(count.size()) < k - 1) {
            count[*it] = 1;
        } else {
            std::vector<T> erase;
            for (auto &[key, val] : count) {
                if (--val == 0) {
                    erase.push_back(key);
                }
            }
            for (const auto &key : erase) {
                count.erase(key);
            }
        }
    }
}

```

```

    }
    return count;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{1, 2, 1, 3, 1, 2, 1, 4, 2, 2, 2};
    unordered_map<int, int> candidates = misra_gries<int>(a.begin(), a.end(), 3);
    assert(candidates.count(1));
    assert(candidates.count(2));
    assert(!candidates.count(3));
    return 0;
}

```

1.5 Bit Manipulation

1.5.1 Bit Operations and Masks

Integers are all fixed-size sets of bits, where the i -th bit (counting from the least significant bit where $i = 0$) is “present” when set to 1. Treating an `int` as a bitmask makes set membership, insertion, deletion, and iteration into single machine instructions, which is why bitmasks are the backbone of subset dynamic programming and many low-level tricks. The helpers below operate on `uint32_t` masks; to use 64-bit masks, replace `uint32_t` with `uint64_t`, the literal `1u` with `1ull`, and the `1u << 31` inside `clz()` with `1ull << 63`. The loop implementations below exist for education only; production code should prefer the GCC built-ins named in the inline comments whenever available, or on C++20 systems the `<bit>` header, whose generic `constexpr` equivalents (also named in the comments) are additionally well-defined at 0.

- `test_bit(x, i)` returns whether the i -th bit of x is set.
- `set_bit(x, i)`, `clear_bit(x, i)`, and `toggle_bit(x, i)` return x with the i -th respectively forced to 1, forced to 0, or flipped.
- `lowest_set_bit(x)` returns the value of the lowest set bit of x (a power of two), or 0 if x is 0.
`clear_lowest_set_bit(x)` returns x with its lowest set bit removed.
- `popcount(x)` returns the number of set bits, analogous to `__builtin_popcount()` and C++20’s `std::popcount()`.
- `parity(x)` returns 1 if the number of set bits is odd and 0 otherwise, analogous to `__builtin_parity()`.
- `ctz(x)` returns the number of trailing 0-bits (undefined for 0), analogous to `__builtin_ctz()` and C++20’s `std::countr_zero()`.
- `ffs(x)` returns one plus the position of the lowest 1-bit of x , or 0 if x is 0, analogous to `__builtin_ffs()`.
- `clz(x)` returns the number of leading 0-bits (undefined for 0), analogous to `__builtin_clz()` and C++20’s `std::countl_zero()`.
- `is_pow2(x)` returns whether x has exactly one set bit, analogous to C++20’s `std::has_single_bit()`.
- `floor_pow2(x)` returns the largest power of two that is $\leq x$ (for $x > 0$), analogous to C++20’s `std::bit_floor()`.
- `ceil_pow2(x)` returns the smallest power of two that is $\geq x$ (for $x > 0$), analogous to C++20’s `std::bit_ceil()`.
- `for_each_set_bit(x, f)` calls `f(i)` once for each set bit position i of x , in increasing order.

Time Complexity:

- $O(b)$ worst case per call to `popcount(x)`, `parity(x)`, `ctz(x)`, `ffs(x)`, `clz(x)`, and `for_each_set_bit(x, f)`, where b is the bit width of the mask (32 here).
- $O(1)$ for all other operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <stdint>

bool test_bit(uint32_t x, int i) { return (x >> i) & 1u; }
uint32_t set_bit(uint32_t x, int i) { return x | (1u << i); }
uint32_t clear_bit(uint32_t x, int i) { return x & ~(1u << i); }
uint32_t toggle_bit(uint32_t x, int i) { return x ^ (1u << i); }
uint32_t lowest_set_bit(uint32_t x) { return x & (0u - x); }
uint32_t clear_lowest_set_bit(uint32_t x) { return x & (x - 1); }

// popcount(), parity(), ctz(), ffs(), and clz() are spelled out here for educational purposes.
// Production code should use the compiler intrinsics named in the comments below.

int popcount(uint32_t x) { // __builtin_popcount(x); std::popcount(x) in C++20.
    int count = 0;
    for (; x != 0; x = clear_lowest_set_bit(x)) {
        count++;
    }
    return count;
}

int parity(uint32_t x) { // __builtin_parity(x)
    return popcount(x) & 1;
}

int ctz(uint32_t x) { // __builtin_ctz(x); undefined when x == 0. std::countr_zero(x) in C++20.
    int count = 0;
    for (; (x & 1u) == 0; x >>= 1) {
        count++;
    }
    return count;
}

int ffs(uint32_t x) { // __builtin_ffs(x); returns 0 when x == 0.
    return x == 0 ? 0 : ctz(x) + 1;
}

int clz(uint32_t x) { // __builtin_clz(x); undefined when x == 0. std::countl_zero(x) in C++20.
    int count = 0;
    for (uint32_t mask = 1u << 31; (x & mask) == 0; mask >>= 1) {
        count++;
    }
    return count;
}

bool is_pow2(uint32_t x) { // std::has_single_bit(x) in C++20.
    return x != 0 && (x & (x - 1)) == 0;
}

uint32_t floor_pow2(uint32_t x) { // std::bit_floor(x) in C++20.
    return 1u << (31 - __builtin_clz(x));
}

uint32_t ceil_pow2(uint32_t x) { // std::bit_ceil(x) in C++20.
    return is_pow2(x) ? x : 1u << (32 - __builtin_clz(x));
}

template<typename Fn>
void for_each_set_bit(uint32_t x, Fn f) {
    while (x != 0) {
        f(ctz(x));
        x = clear_lowest_set_bit(x);
    }
}
```

Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

int main() {
    uint32_t x = 0b101100;
    assert(test_bit(x, 2) == true);
    assert(test_bit(x, 0) == false);
    assert(set_bit(x, 0) == 0b101101u);
    assert(clear_bit(x, 2) == 0b101000u);
    assert(toggle_bit(x, 3) == 0b100100u);

    assert(lowest_set_bit(x) == 0b100u);
    assert(clear_lowest_set_bit(x) == 0b101000u);
    assert(popcount(x) == 3);
    assert(parity(x) == 1);
    assert(parity(0b101101u) == 0);
    assert(ctz(x) == 2);
    assert(ffs(x) == 3);
    assert(ffs(0) == 0);
    assert(clz(x) == 26);

    assert(is_pow2(16) == true);
    assert(is_pow2(24) == false);
    assert(floor_pow2(20) == 16u);
    assert(ceil_pow2(20) == 32u);
    assert(ceil_pow2(16) == 16u);

    vector<int> bits;
    for_each_set_bit(x, [&](int b) { bits.push_back(b); });
    assert((bits == vector<int>{2, 3, 5}));
    return 0;
}
```

1.5.2 Subset and Submask Iteration

Many bitmask algorithms must visit related masks in a structured order: every submask of a fixed mask, every mask with a fixed number of set bits, or every mask while accumulating contributions from its submasks. Each pattern has a short, well-known idiom.

The classic submask loop `for (int s = m; s > 0; s = (s - 1) & m)` walks every nonzero submask of `m` in decreasing order; subtracting 1 borrows through the zero bits of `m`, and the `& m` masks them back off. Summed over all masks `m` of `n` bits, the total work is $\sum_k \binom{n}{k} 2^k = 3^n$, since each of the `n` bit positions is independently absent, present in `m` only, or present in both `m` and `s`.

- `submasks(m)` returns all submasks of `m`, including `m` itself and 0, in decreasing order.
- `next_popcount(x)` returns the smallest integer greater than `x` with the same number of set bits (Gosper’s hack), for `x > 0`.
- `masks_with_popcount(n, k)` returns all `k`-bit subsets of an `n`-bit universe as masks, in increasing order.
- `subset_sum_transform(f)` overwrites `f` (indexed by mask over `n` bits) so that `f[m]` becomes the sum of the original `f[s]` over all submasks `s` of `m`. This “sum over subsets” (SOS) DP is the bitmask analog of a prefix sum, accumulating one bit dimension at a time.

Time Complexity:

- $O(2^p)$ per call to `submasks(m)` where $p = \text{popcount}(m)$.
- $O(1)$ per call to `next_popcount(x)`.
- $O(\binom{n}{k})$ per call to `masks_with_popcount(n, k)`.
- $O(n \cdot 2^n)$ per call to `subset_sum_transform(f)`, where `f` has 2^n entries.

Space Complexity:

- $O(1)$ auxiliary for `next_popcount(x)` and `subset_sum_transform(f)`.
- $O(s)$ auxiliary for `submasks(m)` and `masks_with_popcount(n, k)`, where s is the result size.

```
#include <stdint>
#include <vector>

std::vector<uint32_t> submasks(uint32_t m) {
    std::vector<uint32_t> res;
    uint32_t s = m;
    while (true) {
        res.push_back(s);
        if (s == 0) {
            break;
        }
        s = (s - 1) & m;
    }
    return res;
}

uint32_t next_popcount(uint32_t x) {
    uint32_t c = x & (0u - x), r = x + c;
    return r | ((x ^ r) >> 2) / c;
}

std::vector<uint32_t> masks_with_popcount(int n, int k) {
    std::vector<uint32_t> res;
    if (k == 0) {
        return {0u};
    }
    uint32_t limit = 1u << n;
    for (uint32_t x = (1u << k) - 1; x < limit; x = next_popcount(x)) {
        res.push_back(x);
    }
    return res;
}

template<typename T>
void subset_sum_transform(std::vector<T> &f) {
    int n = __builtin_ctz(f.size()); // f.size() must be a power of two, 2^n.
    for (int b = 0; b < n; b++) {
        for (uint32_t m = 0; m < static_cast<uint32_t>(f.size()); m++) {
            if (m & (1u << b)) {
                f[m] += f[m ^ (1u << b)];
            }
        }
    }
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert((submasks(0b101) == vector<uint32_t>{0b101, 0b100, 0b001, 0b000}));

    assert(next_popcount(0b0110) == 0b1001u);
    assert((
        masks_with_popcount(4, 2) == vector<uint32_t>{0b0011, 0b0101, 0b0110, 0b1001, 0b1010, 0b1100}
    ));

    // f[m] = 1 for every mask; after the transform f[m] counts submasks of m = 2^popcount(m).
    vector<int> f(1 << 3, 1);
    subset_sum_transform(f);
```

```

assert(f[0b000] == 1);
assert(f[0b101] == 4);
assert(f[0b111] == 8);
return 0;
}

```

1.5.3 Gray Code

The binary reflected Gray code is an ordering of all 2^n bitmasks in which consecutive masks differ in exactly one bit. It is useful whenever the cost of moving between adjacent states depends on a single bit flip: enumerating subsets while incrementally maintaining a value, hardware where only one bit may toggle at a time, and constructions like Hamiltonian paths on the hypercube.

The k -th Gray code is obtained from the ordinary binary value k by $k \oplus (k \gg 1)$, a bijection on $[0, 2^n)$. Its inverse recovers the rank of a given Gray code, which tells how many flips into the sequence a mask sits.

- `gray_code(k)` returns the k -th mask in Gray code order.
- `inverse_gray_code(g)` returns the rank k such that `gray_code(k) == g`.
- `gray_sequence(n)` returns all 2^n masks in Gray code order; consecutive entries (and the last with the first) differ in exactly one bit.

Time Complexity:

- $O(1)$ per call to `gray_code(k)`.
- $O(\log g)$ per call to `inverse_gray_code(g)`.
- $O(2^n)$ per call to `gray_sequence(n)`.

Space Complexity:

- $O(1)$ auxiliary for `gray_code(k)` and `inverse_gray_code(g)`.
- $O(2^n)$ auxiliary for `gray_sequence(n)`.

```

#include <stdint>
#include <vector>

uint32_t gray_code(uint32_t k) {
    return k ^ (k >> 1);
}

uint32_t inverse_gray_code(uint32_t g) {
    for (uint32_t shift = 1; (g >> shift) != 0; shift <= 1) {
        g ^= g >> shift;
    }
    return g;
}

std::vector<uint32_t> gray_sequence(int n) {
    std::vector<uint32_t> res(1u << n);
    for (uint32_t k = 0; k < static_cast<uint32_t>(res.size()); k++) {
        res[k] = gray_code(k);
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(gray_code(0) == 0b000u);
}

```



```

assert(gray_code(1) == 0b001u);
assert(gray_code(2) == 0b011u);
assert(gray_code(3) == 0b010u);

for (uint32_t k = 0; k < 256; k++) {
    assert(inverse_gray_code(gray_code(k)) == k);
}

vector<uint32_t> seq = gray_sequence(3);
assert(seq.size() == 8);
for (int i = 0; i < static_cast<int>(seq.size()); i++) {
    uint32_t diff = seq[i] ^ seq[(i + 1) % seq.size()];
    assert(__builtin_popcount(diff) == 1); // Cyclically, one bit changes per step.
}
return 0;
}

```

1.5.4 XOR Basis

An XOR basis (also called a linear basis over the field $\text{GF}(2)$) is the analog of a vector space basis for integers under the bitwise XOR operation. Each integer is treated as a vector of bits, where XOR plays the role of vector addition. Inserting a set of integers one at a time and keeping only those that are linearly independent yields a basis whose XOR-combinations (subset XORs) span exactly the same set of values as the original integers. From a basis of size r , exactly 2^r distinct values are reachable, and queries such as the maximum attainable XOR become a simple greedy scan.

The basis is kept in reduced form: the i -th basis is either 0 or a value whose highest set bit is bit i , and no two basis elements share the same highest bit. Reducing a value means XORing it against the basis elements from the highest bit down, eliminating each set bit that a basis element covers.

- `insert(x)` adds `x` to the basis, returning `true` if `x` was linearly independent of the current basis (increasing its size), or `false` if `x` was already representable as a subset XOR.
- `contains(x)` returns whether `x` is representable as the XOR of some subset of the inserted values, i.e. whether it reduces to 0 against the basis.
- `max_xor(base)` returns the maximum value of `base` XORed with some subset XOR of the basis. With the default `base` of 0, this is the maximum subset XOR.
- `size()` returns the number of elements in the basis (its rank).

Time Complexity:

- $O(b)$ per call to `insert()`, `contains()`, and `max_xor()`, where b is the number of bits.
- $O(1)$ per call to `size()`.

Space Complexity:

- $O(b)$ for storage of the basis.
- $O(1)$ auxiliary for all operations.

```

#include <array>
#include <climits>
#include <cstdint>

class XorBasis {
    static const int BITS = sizeof(uint64_t) * CHAR_BIT;

    std::array<uint64_t, BITS> basis{}; // basis[i] != 0 has its highest set bit at bit i.
    int sz = 0;

public:
    bool insert(uint64_t x) {
        for (int i = BITS - 1; i >= 0; i--) {
            if ((x >> i) & 1) {

```

```

        continue;
    }
    if (basis[i] == 0) {
        basis[i] = x;
        sz++;
        return true;
    }
    x ^= basis[i];
}
return false;
}

bool contains(uint64_t x) const {
    for (int i = BITS - 1; i >= 0; i--) {
        if (((x >> i) & 1) && basis[i] != 0) {
            x ^= basis[i];
        }
    }
    return x == 0;
}

uint64_t max_xor(uint64_t base = 0) const {
    for (int i = BITS - 1; i >= 0; i--) {
        if ((base ^ basis[i]) > base) {
            base ^= basis[i];
        }
    }
    return base;
}

int size() const { return sz; }
};

```

Example Usage

```

#include <cassert>

int main() {
    XorBasis b;
    assert(b.insert(1));    // 001
    assert(b.insert(2));    // 010
    assert(!b.insert(3));   // 011 = 1 ^ 2, already representable.
    assert(b.size() == 2);

    assert(b.contains(3));
    assert(b.contains(0));
    assert(!b.contains(4));

    assert(b.max_xor() == 3); // Best subset XOR over {0, 1, 2, 3}.
    assert(b.max_xor(4) == 7); // 4 ^ 3.
    return 0;
}

```

1.5.5 Binary Trie

Maintain a multiset of nonnegative b -bit integers as a binary trie, storing each value as a root-to-leaf path of its bits from the most significant (bit $b - 1$) down to the least significant. Every node keeps a count of the stored values passing through it, which supports deletion and counting queries, and lets the trie answer XOR-extremal queries by a greedy walk down the bits.

The classic application is the maximum-XOR query: to maximize $\mathbf{x} \oplus \mathbf{y}$ over all stored $\mathbf{y}$, walk from the most significant bit. At each step, descend toward the child whose bit differs from that of $\mathbf{x}$ whenever such a branch

exists, since setting a higher bit of the result always dominates any combination of lower bits. The minimum-XOR query is the mirror image, preferring the matching bit.

This is a multiset (a value may be inserted more than once), and is distinct from the XOR basis: the basis spans subset XORs of a fixed set, whereas this answers XOR queries against the stored elements themselves with support for insertion and deletion.

Values must lie in $[0, 2^b)$ for bit width b ; the template parameters select the unsigned word type `U` and the bit width `BITS`, which must be less than the width of `U` (for example, 30 with `uint32_t` or 62 with `uint64_t`). Nodes are kept in a pool indexed by integer, where index 0 is the root and also serves as the null child.

- `BinaryTrie<U, BITS>()` constructs an empty multiset.
- `size()` returns the number of stored values, and `empty()` returns whether there are none.
- `insert(x)` adds one occurrence of x .
- `erase(x)` removes one occurrence of x , returning whether one was present.
- `count(x)` returns the number of occurrences of x .
- `max_xor(x)` and `min_xor(x)` return the maximum and minimum of $x \text{ XOR } y$ over all stored y . The trie must be non-empty.
- `count_xor_less(x, k)` returns the number of stored y (with multiplicity) such that $x \text{ XOR } y$ is strictly less than k .

Time Complexity:

- $O(b)$ per call to `insert()`, `erase()`, `count()`, `max_xor()`, `min_xor()`, and `count_xor_less()`, where b is the bit width `BITS`.
- $O(1)$ per call to `size()` and `empty()`.

Space Complexity: $O(n \cdot b)$ for storage, where n is the number of distinct values inserted so far.

```
#include <array>
#include <cstdint>
#include <vector>

template<typename U = uint32_t, int BITS = 30>
class BinaryTrie {
    std::vector<std::array<int, 2>> child; // child[node][bit], where 0 means no child.
    std::vector<int> cnt;                 // Number of stored values passing through each node.

    int new_node() {
        child.push_back({0, 0});
        cnt.push_back(0);
        return static_cast<int>(cnt.size()) - 1;
    }

public:
    BinaryTrie() { new_node(); } // Node 0 is the root (and doubles as the null child).

    int size() const { return cnt[0]; }
    bool empty() const { return cnt[0] == 0; }

    void insert(U x) {
        int node = 0;
        cnt[node]++;
        for (int b = BITS - 1; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (child[node][bit] == 0) {
                int next = new_node();
                child[node][bit] = next;
            }
            node = child[node][bit];
            cnt[node]++;
        }
    }

    int count(U x) const {
```

```

int node = 0;
for (int b = BITS - 1; b >= 0; b--) {
    int next = child[node][(x >> b) & 1];
    if (next == 0) {
        return 0;
    }
    node = next;
}
return cnt[node];
}

bool erase(U x) {
    if (count(x) == 0) {
        return false;
    }
    int node = 0;
    cnt[node]--;
    for (int b = BITS - 1; b >= 0; b--) {
        node = child[node][(x >> b) & 1];
        cnt[node]--;
    }
    return true;
}

U max_xor(U x) const {
    int node = 0;
    U res = 0;
    for (int b = BITS - 1; b >= 0; b--) {
        int bit = (x >> b) & 1;
        int opp = child[node][bit ^ 1];
        if (opp != 0 && cnt[opp] > 0) {
            res |= U(1) << b;
            node = opp;
        } else {
            node = child[node][bit];
        }
    }
    return res;
}

U min_xor(U x) const {
    int node = 0;
    U res = 0;
    for (int b = BITS - 1; b >= 0; b--) {
        int bit = (x >> b) & 1;
        int same = child[node][bit];
        if (same != 0 && cnt[same] > 0) {
            node = same;
        } else {
            res |= U(1) << b;
            node = child[node][bit ^ 1];
        }
    }
    return res;
}

int count_xor_less(U x, U k) const {
    if ((k >> BITS) != 0) {
        return cnt[0]; // Every XOR result is below 2^BITS, which is at most k.
    }
    int node = 0, res = 0;
    for (int b = BITS - 1; b >= 0; b--) {
        int xb = (x >> b) & 1, kb = (k >> b) & 1;
        if (kb == 1) {
            // Stored values matching x at bit b give XOR bit 0 here, hence are already below k.
            int below = child[node][xb];
            if (below != 0) {
                res += cnt[below];
            }
        }
    }
}

```

```

    }
    node = child[node][xb ^ 1]; // Stay tied with k by taking XOR bit 1.
  } else {
    node = child[node][xb]; // XOR bit must be 0 to remain below k.
  }
  if (node == 0) {
    break;
  }
}
return res;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    BinaryTrie<> trie;
    for (int x : {3, 10, 5, 25, 2}) {
        trie.insert(x);
    }
    assert(trie.size() == 5);

    // x XOR y over the stored y: {26, 19, 28, 0, 27} for x = 25.
    assert(trie.max_xor(25) == 28); // 25 XOR 5.
    assert(trie.min_xor(25) == 0); // 25 XOR 25.
    assert(trie.count_xor_less(25, 20) == 2); // XOR values 0 and 19 are below 20.
    assert(trie.count_xor_less(25, uint32_t(1) << 30) == 5); // Bound at/above 2^BITS counts all.

    assert(trie.count(10) == 1);
    trie.insert(10);
    assert(trie.count(10) == 2); // Multiset: two copies of 10.
    assert(trie.erase(10) && trie.count(10) == 1);
    assert(trie.erase(25) && trie.count(25) == 0);
    assert(!trie.erase(25)); // Already gone.
    assert(trie.max_xor(0) == 10); // Largest remaining value is 10.
    return 0;
}

```

1.5.6 Bitwise Convolution (FWHT)

Bitwise convolution combines two arrays indexed by masks, grouping pairs of masks by a bitwise operation instead of by ordinary addition. For arrays `a` and `b` over an n -bit universe, the XOR convolution `c[m]` is the sum of `a[x]*b[y]` over all $x \oplus y = m$. OR and AND convolution replace $x \oplus y$ with $x \mid y$ or $x \& y$. These appear in subset DP, counting pairs by bitwise result, and algebra over the Boolean hypercube.

The fast Walsh-Hadamard transform (FWHT) diagonalizes XOR convolution in the same way that the FFT diagonalizes polynomial convolution: transform both arrays, multiply pointwise, and transform back. The OR and AND versions are Zeta/Mobius transforms over the subset lattice, but they have the same butterfly-shaped implementation and the same $O(n \log n)$ runtime for arrays of length $n = 2^k$.

- `xor_transform(f, invert)` applies the XOR FWHT to `f` in place. The inverse transform uses `invert = true` and divides every coefficient by `f.size()`.
- `or_transform(f, invert)` applies the OR Zeta transform in place; the inverse is Mobius inversion.
- `and_transform(f, invert)` applies the AND Zeta transform in place; the inverse is Mobius inversion.
- `xor_convolve(a, b)` returns the XOR convolution of `a` and `b`.
- `or_convolve(a, b)` returns the OR convolution of `a` and `b`.
- `and_convolve(a, b)` returns the AND convolution of `a` and `b`.

Input arrays are padded with zeros to the next power of two. For exact integer XOR convolution, the inverse divisions are exact; for modular arithmetic, replace the division by multiplication with the modular inverse of the transform length.

Time Complexity: $O(n \log n)$ per transform or convolution, where n is the padded power-of-two length.

Space Complexity:

- $O(1)$ auxiliary for the in-place transforms.
- $O(n)$ auxiliary heap space for each convolution.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

int next_power_of_two(int n) {
    int res = 1;
    while (res < n) {
        res <<= 1;
    }
    return res;
}

template<typename T>
void xor_transform(std::vector<T> &f, bool invert) {
    int n = static_cast<int>(f.size());
    assert((n & (n - 1)) == 0);
    for (int len = 1; len < n; len <<= 1) {
        for (int i = 0; i < n; i += len << 1) {
            for (int j = 0; j < len; j++) {
                T u = f[i + j], v = f[i + j + len];
                f[i + j] = u + v;
                f[i + j + len] = u - v;
            }
        }
    }
    if (invert) {
        for (T &x : f) {
            x /= n;
        }
    }
}

template<typename T>
void or_transform(std::vector<T> &f, bool invert) {
    int n = static_cast<int>(f.size());
    assert((n & (n - 1)) == 0);
    for (int bit = 1; bit < n; bit <<= 1) {
        for (int mask = 0; mask < n; mask++) {
            if (mask & bit) {
                if (invert) {
                    f[mask] -= f[mask ^ bit];
                } else {
                    f[mask] += f[mask ^ bit];
                }
            }
        }
    }
}

template<typename T>
void and_transform(std::vector<T> &f, bool invert) {
    int n = static_cast<int>(f.size());
    assert((n & (n - 1)) == 0);
    for (int bit = 1; bit < n; bit <<= 1) {
        for (int mask = 0; mask < n; mask++) {
```

```

        if ((mask & bit) == 0) {
            if (invert) {
                f[mask] -= f[mask ^ bit];
            } else {
                f[mask] += f[mask ^ bit];
            }
        }
    }
}

template<typename T, typename Transform>
std::vector<T> bitwise_convolve(std::vector<T> a, std::vector<T> b, Transform transform) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int n = next_power_of_two(static_cast<int>(std::max(a.size(), b.size())));
    a.resize(n);
    b.resize(n);
    transform(a, false);
    transform(b, false);
    for (int i = 0; i < n; i++) {
        a[i] *= b[i];
    }
    transform(a, true);
    return a;
}

template<typename T>
std::vector<T> xor_convolve(std::vector<T> a, std::vector<T> b) {
    return bitwise_convolve(a, b, xor_transform<T>);
}

template<typename T>
std::vector<T> or_convolve(std::vector<T> a, std::vector<T> b) {
    return bitwise_convolve(a, b, or_transform<T>);
}

template<typename T>
std::vector<T> and_convolve(std::vector<T> a, std::vector<T> b) {
    return bitwise_convolve(a, b, and_transform<T>);
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int64_t> a{1, 2, 3, 4};
    vector<int64_t> b{5, 6, 7, 8};

    // XOR: c[m] sums a[x] * b[y] over pairs with x ^ y == m.
    assert((xor_convolve(a, b) == vector<int64_t>{70, 68, 62, 60}));

    // OR: c[m] sums a[x] * b[y] over pairs with x | y == m.
    assert((or_convolve(a, b) == vector<int64_t>{5, 28, 43, 184}));

    // AND: c[m] sums a[x] * b[y] over pairs with x & y == m.
    assert((and_convolve(a, b) == vector<int64_t>{103, 52, 73, 32}));

    vector<int64_t> one_mask{0, 1, 0, 0};
    vector<int64_t> two_mask{0, 0, 1};
    assert((xor_convolve(one_mask, two_mask) == vector<int64_t>{0, 0, 0, 1}));
    assert((or_convolve(one_mask, two_mask) == vector<int64_t>{0, 0, 0, 1}));
    assert((and_convolve(one_mask, two_mask) == vector<int64_t>{1, 0, 0, 0}));
}

```

```
    return 0;  
}
```


Chapter 2

Data Structures

2.1 Linked Lists

2.1.1 Singly Linked List

Implements common singly linked list operations using raw nodes. Linked lists are rarely the best contest choice compared with arrays, vectors, and `std::list`, but the pointer patterns are useful for interview-style problems and for understanding constant-time splicing.

For production C++, prefer containers such as `std::list`, `std::forward_list`, or `std::vector` when they fit the problem. Use the raw-node style below when the problem statement already gives nodes, or when manual pointer manipulation is the point of the exercise.

- `reverse_list(head)` reverses a singly linked list and returns the new head.
- `merge_sorted_lists(a, b)` merges two sorted lists using a dummy node and returns the merged head.
- `split_half(head, &second)` cuts a list into two halves, returning the first half and storing the second half in `second`.
- `splice_after(pos, before)` moves the node after `before` so it appears after `pos`.
- `splice_range_after(pos, before_first, last)` moves the half-open range (`before_first`, `last`) so it appears after `pos`.

The splicing helpers use the same “after” convention as `std::forward_list::splice_after()`. They work naturally with a dummy head node, which makes insertions and removals at the beginning of a list match all other positions. For doubly linked list splicing, use the next section.

For cycle detection, see the variant on iterated functions in chapter 1, which can be easily adapted for linked lists.

Time Complexity:

- $O(n)$ per call to `reverse_list(head)` and `split_half(head, &second)`.
- $O(n + m)$ per call to `merge_sorted_lists(a, b)`.
- $O(1)$ per call to `splice_after(pos, before)`.
- $O(k)$ per call to `splice_range_after(pos, before_first, last)`, where k is the number of moved nodes. The actual pointer splicing is $O(1)$, but this compact version walks to the end of the moved range.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <cstddef>

struct ListNode {
    int value;
    ListNode *next;
```

```

    explicit ListNode(int value = 0) : value(value), next(nullptr) {}
};

ListNode *reverse_list(ListNode *head) {
    ListNode *prev = nullptr;
    while (head != nullptr) {
        ListNode *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

ListNode *merge_sorted_lists(ListNode *a, ListNode *b) {
    ListNode dummy;
    ListNode *tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (b->value < a->value) {
            tail->next = b;
            b = b->next;
        } else {
            tail->next = a;
            a = a->next;
        }
        tail = tail->next;
    }
    tail->next = a != nullptr ? a : b;
    return dummy.next;
}

ListNode *split_half(ListNode *head, ListNode **second) {
    if (head == nullptr || head->next == nullptr) {
        *second = nullptr;
        return head;
    }
    ListNode *slow = head, *fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    *second = slow->next;
    slow->next = nullptr;
    return head;
}

void splice_after(ListNode *pos, ListNode *before) {
    ListNode *node = before->next;
    if (node == nullptr || pos == before || pos == node) {
        return;
    }
    before->next = node->next;
    node->next = pos->next;
    pos->next = node;
}

void splice_range_after(ListNode *pos, ListNode *before_first, ListNode *last) {
    ListNode *first = before_first->next;
    if (first == last) {
        return;
    }
    ListNode *tail = first;
    while (tail->next != last) {
        tail = tail->next;
    }
    before_first->next = last;
    tail->next = pos->next;
    pos->next = first;
}

```

```
}
```

Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

vector<int> values(ListNode *head) {
    vector<int> res;
    for (; head != nullptr; head = head->next) {
        res.push_back(head->value);
    }
    return res;
}

int main() {
    vector<ListNode> a{ListNode(1), ListNode(3), ListNode(5)};
    vector<ListNode> b{ListNode(2), ListNode(4), ListNode(6)};
    for (int i = 0; i + 1 < static_cast<int>(a.size()); i++) {
        a[i].next = &a[i + 1];
        b[i].next = &b[i + 1];
    }
    ListNode *merged = merge_sorted_lists(&a[0], &b[0]);
    vector<int> merged_values = values(merged);
    for (int i = 0; i < 6; i++) {
        assert(merged_values[i] == i + 1);
    }

    ListNode *second = nullptr;
    ListNode *first = split_half(merged, &second);
    assert(values(first).size() == 3);
    assert(values(second).size() == 3);
    assert(values(reverse_list(first))[0] == 3);

    ListNode dummy(0), w(1), x(2), y(3), z(4);
    dummy.next = &w;
    w.next = &x;
    x.next = &y;
    y.next = &z;
    splice_after(&dummy, &y); // Move 4 to the front: 4, 1, 2, 3.
    assert((values(dummy.next) == vector<int>{4, 1, 2, 3}));
    splice_range_after(&z, &w, nullptr); // Move 2, 3 after 4: 4, 2, 3, 1.
    assert((values(dummy.next) == vector<int>{4, 2, 3, 1}));
    return 0;
}
```

2.1.2 Doubly Linked List

Implements common doubly linked list operations using raw intrusive nodes. A node is intrusive when the `prev` and `next` pointers live inside the stored object itself, rather than inside a separate container allocation. This is useful when a problem already gives node pointers, when values must not be copied, or when a map needs to jump directly to a list node in $O(1)$ time, as in an LRU cache.

The functions below use a circular sentinel node. When the list is empty, `sentinel.next` and `sentinel.prev` both point back to `sentinel`. This removes special cases for inserting or erasing at the front and back of the list.

- `init_list(sentinel)` initializes an empty circular list around `sentinel`.
- `insert_after(pos, node)` inserts detached `node` immediately after `pos`.
- `insert_before(pos, node)` inserts detached `node` immediately before `pos`.
- `erase(node)` removes `node` from its current list and leaves it detached.

- `push_front(sentinel, node)` inserts detached `node` at the front of the list.
- `push_back(sentinel, node)` inserts detached `node` at the back of the list.
- `move_to_front(sentinel, node)` moves an already-linked `node` to the front of the list.
- `empty(sentinel)` returns whether the list has no data nodes.
- `splice(pos, node)` moves an already-linked `node` so it appears immediately before `pos`.
- `splice_range(pos, first, last)` moves the half-open range `[first, last)` so it appears immediately before `pos`.

The data nodes passed to insertion functions must be detached, meaning their `prev` and `next` pointers are both null. The node passed to `erase()` or `move_to_front()` must already be linked into a list and must not be the sentinel. For `splice_range(pos, first, last)`, the `pos` node must not lie inside the moved range.

Time Complexity: $O(1)$ per call to all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <cstddef>

struct DListNode {
    int value;
    DListNode *prev, *next;

    explicit DListNode(int value = 0) : value(value), prev(nullptr), next(nullptr) {}
};

void init_list(DListNode *sentinel) {
    sentinel->prev = sentinel;
    sentinel->next = sentinel;
}

bool empty(DListNode *sentinel) {
    return sentinel->next == sentinel;
}

void insert_after(DListNode *pos, DListNode *node) {
    node->prev = pos;
    node->next = pos->next;
    pos->next->prev = node;
    pos->next = node;
}

void insert_before(DListNode *pos, DListNode *node) {
    insert_after(pos->prev, node);
}

DListNode *erase(DListNode *node) {
    node->prev->next = node->next;
    node->next->prev = node->prev;
    node->prev = nullptr;
    node->next = nullptr;
    return node;
}

void push_front(DListNode *sentinel, DListNode *node) {
    insert_after(sentinel, node);
}

void push_back(DListNode *sentinel, DListNode *node) {
    insert_before(sentinel, node);
}

void move_to_front(DListNode *sentinel, DListNode *node) {
    push_front(sentinel, erase(node));
}

void splice(DListNode *pos, DListNode *node) {
```

```

    if (pos == node || pos == node->next) {
        return;
    }
    insert_before(pos, erase(node));
}

void splice_range(DListNode *pos, DListNode *first, DListNode *last) {
    if (first == last || pos == first) {
        return;
    }
    DListNode *before_first = first->prev;
    DListNode *tail = last->prev;
    before_first->next = last;
    last->prev = before_first;

    DListNode *before_pos = pos->prev;
    before_pos->next = first;
    first->prev = before_pos;
    tail->next = pos;
    pos->prev = tail;
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

// Returns the data values from front to back (walks the circular list until it loops to sentinel).
vector<int> values(DListNode *sentinel) {
    vector<int> res;
    for (DListNode *p = sentinel->next; p != sentinel; p = p->next) {
        res.push_back(p->value);
    }
    return res;
}

int main() {
    // `dummy` is the sentinel; a, b, c, d start detached. push_back/push_front link at back/front.
    DListNode dummy, a(1), b(2), c(3), d(4);
    init_list(&dummy);
    assert(empty(&dummy));

    push_back(&dummy, &a);
    push_back(&dummy, &b);
    push_back(&dummy, &c);
    push_front(&dummy, &d);
    assert((values(&dummy) == vector<int>{4, 1, 2, 3}));

    move_to_front(&dummy, &b); // list: 2 4 1 3
    assert((values(&dummy) == vector<int>{2, 4, 1, 3}));

    erase(&d); // detach d; list: 2 1 3
    assert((values(&dummy) == vector<int>{2, 1, 3}));

    insert_before(&dummy, &d); // before the sentinel == the back; list: 2 1 3 4
    assert((values(&dummy) == vector<int>{2, 1, 3, 4}));

    splice(&d, &a); // move a to just before d; list: 2 3 1 4
    assert((values(&dummy) == vector<int>{2, 3, 1, 4}));

    splice_range(&dummy, &a, &dummy); // range {a, d} already at the back: no-op
    assert((values(&dummy) == vector<int>{2, 3, 1, 4}));

    splice_range(&b, &a, &dummy); // move {a, d} to before b; list: 1 4 2 3
    assert((values(&dummy) == vector<int>{1, 4, 2, 3}));
}

```

```
    return 0;
}
```

2.1.3 LRU Cache

Maintains a least-recently-used cache with fixed capacity. The cache supports lookups and updates by key, evicting the least recently used entry whenever an insertion would exceed capacity.

This implementation combines a doubly linked list (`std::list`) with a map from keys to list iterators. The front of the list stores the most recently used item. The operation `items.splice(items.begin(), items, it)` is the standard-library way to move an existing list node to the front in $O(1)$ time.

To implement the same cache without `std::list`, use the intrusive doubly linked list operations from section 2.1.2. Store a `Node*` in the map instead of a list iterator, move hits to the front with `move_to_front(&sentinel, node)`, and evict `sentinel.prev`.

- `LRUCache<Key, Value>(capacity)` constructs an empty cache holding at most `capacity` keys.
- `get(key, &value)` returns whether `key` is present. If present, it stores the value in `value` and marks the key as most recently used.
- `put(key, value)` inserts or updates `key`, marking it as most recently used and evicting the least recently used key if needed.
- `size()` returns the number of keys currently stored.

Time Complexity: $O(1)$ amortized per call to `get(key, &value)` and `put(key, value)`.

Space Complexity: $O(n)$, where n is the cache capacity.

```
#include <list>
#include <unordered_map>
#include <utility>

// Replacing std::list with a manual doubly-linked list would look something like:
// struct DListNode { Key key; Value value; DListNode *prev, *next; };
// std::unordered_map<Key, DListNode*> where;
//
// On get:
// move_to_front(&sentinel, where[key]);
//
// On eviction:
// DListNode *old = sentinel.prev;
// erase(old);
// where.erase(old->key).

template<typename Key, typename Value>
class LRUCache {
    using ListIter = typename std::list<std::pair<Key, Value>>::iterator;

    int cap;
    std::list<std::pair<Key, Value>> items;
    std::unordered_map<Key, ListIter> where;

public:
    explicit LRUCache(int capacity) : cap(capacity) {}

    int size() const { return static_cast<int>(items.size()); }

    bool get(const Key &key, Value *value) {
        auto it = where.find(key);
        if (it == where.end()) {
            return false;
        }
        items.splice(items.begin(), items, it->second);
        it->second = items.begin();
    }
};
```

```

    *value = it->second->second;
    return true;
}

void put(const Key &key, const Value &value) {
    auto it = where.find(key);
    if (it != where.end()) {
        it->second->second = value;
        items.splice(items.begin(), items, it->second);
        it->second = items.begin();
        return;
    }
    if (cap == 0) {
        return;
    }
    if (static_cast<int>(items.size()) == cap) {
        where.erase(items.back().first);
        items.pop_back();
    }
    items.push_front({key, value});
    where[key] = items.begin();
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    LRUCache<int, int> cache(2);
    int value = 0;
    cache.put(1, 10);
    cache.put(2, 20);
    assert(cache.get(1, &value) && value == 10);
    cache.put(3, 30); // Evicts key 2.
    assert(!cache.get(2, &value));
    assert(cache.get(3, &value) && value == 30);
    cache.put(1, 11);
    assert(cache.get(1, &value) && value == 11);
    return 0;
}

```

2.2 Heaps and Priority Queues

2.2.1 Binary Heap

Maintain a min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum. This implementation requires an ordering on the set of possible elements defined by `operator<`. A binary min-heap implements a priority queue by inserting and deleting nodes into a binary tree such that the parent of any node is always less than its children. The tree is stored implicitly in an array, where the children of the node at index i live at indices $2i + 1$ and $2i + 2$. Insertion appends the new element and bubbles it up toward the root, while extraction moves the last element to the root and sifts it down.

- `BinaryHeap<T>()` constructs an empty priority queue.
- `BinaryHeap<T>(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range $[lo, hi)$.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.

- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$ per call to `push()` and `pop()`, where n is the number of elements in the priority queue.
- $O(n)$ per call to the second constructor on the distance between `lo` and `hi` (bottom-up heapify).

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <stdexcept>
#include <vector>

template<typename T>
class BinaryHeap {
    std::vector<T> heap;

    void sift_down(int i) {
        while (true) {
            int child = 2 * i + 1;
            if (child >= static_cast<int>(heap.size())) {
                break;
            }
            if (child + 1 < static_cast<int>(heap.size()) && heap[child + 1] < heap[child]) {
                child++;
            }
            if (heap[child] < heap[i]) {
                std::swap(heap[i], heap[child]);
                i = child;
            } else {
                break;
            }
        }
    }

public:
    BinaryHeap() {}

    template<typename It>
    BinaryHeap(It lo, It hi) : heap(lo, hi) {
        for (int i = static_cast<int>(heap.size()) / 2 - 1; i >= 0; i--) {
            sift_down(i);
        }
    }

    int size() const { return static_cast<int>(heap.size()); }
    bool empty() const { return heap.empty(); }

    void push(const T &v) {
        heap.push_back(v);
        int i = static_cast<int>(heap.size()) - 1;
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (!(heap[i] < heap[parent])) {
                break;
            }
            std::swap(heap[i], heap[parent]);
            i = parent;
        }
    }
}
```



```

void pop() {
    if (heap.empty()) {
        throw std::runtime_error("Cannot pop from empty heap.");
    }
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) {
        sift_down(0);
    }
}

T top() const {
    if (heap.empty()) {
        throw std::runtime_error("Cannot get top of empty heap.");
    }
    return heap[0];
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    vector<int> a{0, 5, -1, 12};
    BinaryHeap<int> h(a.begin(), a.end());
    h.push(10);
    while (!h.empty()) {
        cout << h.top() << endl;
        h.pop();
    }
    return 0;
}

```

Example Output

```

-1
0
5
10
12

```

2.2.2 Leftist Heap

Maintain a mergeable min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum as well as efficient merging with other instances. This implementation requires an ordering on the set of possible elements defined by `operator<`. A leftist heap stores at each node a rank (the length of the shortest path to a missing descendant) and keeps every left child's rank at least that of its right child, so the right spine has length $O(\log n)$. Merging walks only this right spine, swapping children where needed to restore the invariant. Every other operation reduces to this merge: insertion merges a one-node heap, and extraction of the minimum merges the root's two subtrees. Unlike the self-adjusting skew heap, the explicit rank bound makes every operation $O(\log n)$ in the worst case rather than amortized.

- `LeftistHeap<T>()` constructs an empty priority queue.
- `LeftistHeap<T>(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range $[lo, hi)$.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.

- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.
- `absorb(h)` inserts every value from `h` and sets `h` to the empty priority queue.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$ per call to `push()`, `pop()`, and `absorb()`, where n is the number of elements in the priority queue.
- $O(n)$ per call to the second constructor, where n is the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(\log n)$ auxiliary stack space for `push()`, `pop()`, and `absorb()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstdint>
#include <stdexcept>

template<typename T>
class LeftistHeap {
    struct Node {
        T value;
        int rank;
        Node *left, *right;

        explicit Node(const T &v) : value(v), rank(1), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;

    static int rank(Node *n) { return (n == nullptr) ? 0 : n->rank; }

    static Node *merge(Node *a, Node *b) {
        if (a == nullptr) {
            return b;
        }
        if (b == nullptr) {
            return a;
        }
        if (b->value < a->value) {
            std::swap(a, b);
        }
        a->right = merge(a->right, b);
        if (rank(a->left) < rank(a->right)) {
            std::swap(a->left, a->right);
        }
        a->rank = rank(a->right) + 1;
        return a;
    }

    static void clean_up(Node *n) {
        if (n != nullptr) {
            clean_up(n->left);
            clean_up(n->right);
            delete n;
        }
    }

public:
    LeftistHeap() : root(nullptr), num_nodes(0) {}

    template<typename It>
    LeftistHeap(It lo, It hi) : root(nullptr), num_nodes(0) {
        while (lo != hi) {
            push(*(lo++));
        }
    }
};
```

```

    }
}

~LeftistHeap() { clean_up(root); }
LeftistHeap(const LeftistHeap &) = delete;
LeftistHeap &operator=(const LeftistHeap &) = delete;
int size() const { return num_nodes; }
bool empty() const { return root == nullptr; }

void push(const T &v) {
    root = merge(root, new Node(v));
    num_nodes++;
}

void pop() {
    if (empty()) {
        throw std::runtime_error("Cannot pop from empty heap.");
    }
    Node *tmp = root;
    root = merge(root->left, root->right);
    delete tmp;
    num_nodes--;
}

T top() const {
    if (empty()) {
        throw std::runtime_error("Cannot get top of empty heap.");
    }
    return root->value;
}

void absorb(LeftistHeap &h) {
    root = merge(root, h.root);
    num_nodes += h.num_nodes;
    h.root = nullptr;
    h.num_nodes = 0;
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    LeftistHeap<int> h, h2;
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);
    while (!h.empty()) {
        cout << h.top() << endl;
        h.pop();
    }
    return 0;
}

```

Example Output

```

-1
0
5
10

```

2.2.3 Skew Heap

Maintain a mergeable min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum as well as efficient merging with other instances. This implementation requires an ordering on the set of possible elements defined by `operator<`. A skew heap attempts to maintain balance by unconditionally swapping all nodes in the merge path when merging. Every other operation reduces to this merge: insertion merges a one-node heap, and extraction of the minimum merges the root's two subtrees.

- `SkewHeap<T>()` constructs an empty priority queue.
- `SkewHeap<T>(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` into the priority queue.
- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.
- `absorb(h)` inserts every value from `h` and sets `h` to the empty priority queue.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, and `top()`.
- $O(\log n)$ amortized auxiliary per call to `push()`, `pop()`, and `absorb()`, where n is the number of elements in the priority queue.
- $O(n)$ per call to the second constructor, where n is the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(\log n)$ amortized auxiliary stack space for `push()`, `pop()`, and `absorb()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstdint>
#include <stdexcept>

template<typename T>
class SkewHeap {
    struct Node {
        T value;
        Node *left, *right;

        explicit Node(const T &v) : value(v), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;

    static Node *merge(Node *a, Node *b) {
        if (a == nullptr) {
            return b;
        }
        if (b == nullptr) {
            return a;
        }
        if (b->value < a->value) {
            std::swap(a, b);
        }
        std::swap(a->left, a->right);
        a->left = merge(b, a->left);
        return a;
    }
};
```

```

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
SkewHeap() : root(nullptr), num_nodes(0) {}

template<typename It>
SkewHeap(It lo, It hi) : root(nullptr), num_nodes(0) {
    while (lo != hi) {
        push(*(lo++));
    }
}

~SkewHeap() { clean_up(root); }
SkewHeap(const SkewHeap &) = delete;
SkewHeap &operator=(const SkewHeap &) = delete;
int size() const { return num_nodes; }
bool empty() const { return root == nullptr; }

void push(const T &v) {
    root = merge(root, new Node(v));
    num_nodes++;
}

void pop() {
    if (empty()) {
        throw std::runtime_error("Cannot pop from empty heap.");
    }
    Node *tmp = root;
    root = merge(root->left, root->right);
    delete tmp;
    num_nodes--;
}

T top() const {
    if (empty()) {
        throw std::runtime_error("Cannot get top of empty heap.");
    }
    return root->value;
}

void absorb(SkewHeap &h) {
    root = merge(root, h.root);
    num_nodes += h.num_nodes;
    h.root = nullptr;
    h.num_nodes = 0;
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    SkewHeap<int> h, h2;
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
}

```

```

h2.push(0);
h.absorb(h2);
while (!h.empty()) {
    cout << h.top() << endl;
    h.pop();
}
return 0;
}

```

Example Output

```

-1
0
5
10
12

```

2.2.4 Pairing Heap

Maintain a mergeable min-priority queue, that is, a collection of elements with support for querying and extraction of the minimum as well as efficient merging with other instances. This implementation requires an ordering on the set of possible elements defined by `operator<`. A pairing heap is a heap-ordered multi-way tree, using a two-pass merge to self-adjust during each deletion. Unlike the other heaps in this section, it also supports `decrease_key()` and `erase()` via handles returned by `push()`, the feature pairing heaps are best known for.

- `PairingHeap<T>()` constructs an empty priority queue.
- `PairingHeap<T>(lo, hi)` constructs a priority queue from two ForwardIterators, consisting of elements in the range `[lo, hi)`.
- `size()` returns the size of the priority queue.
- `empty()` returns whether the priority queue is empty.
- `push(v)` inserts the value `v` and returns a handle to the new element. The handle remains valid until that element is removed by `pop()` or `erase()`.
- `pop()` removes the minimum element from the priority queue.
- `top()` returns the minimum element in the priority queue.
- `absorb(h)` inserts every value from `h` and sets `h` to the empty priority queue.
- `decrease_key(h, v)` lowers the value of the element at handle `h` to `v`, which must not be greater than its current value.
- `erase(h)` removes the element at handle `h` from the priority queue.

In practice, `decrease_key()` runs in near-constant time, making the pairing heap the usual practical substitute for a Fibonacci heap. When handles are not needed, the lazy-deletion idiom (push updated entries and skip stale ones on `pop()`, as the Dijkstra section does) is simpler and often faster.

Time Complexity:

- $O(1)$ per call to the first constructor, `size()`, `empty()`, `top()`, `push()`, and `absorb()`.
- $O(\log n)$ amortized per call to `pop()` and `erase()`.
- $O(\log n)$ amortized per call to `decrease_key()`, a safe bound to budget for. The true amortized cost is sub-logarithmic, though the long-conjectured $O(1)$ bound is provably impossible.
- $O(n)$ per call to the second constructor on the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for storage of the priority queue elements.
- $O(\log n)$ amortized auxiliary stack space for `pop()`.
- $O(1)$ auxiliary for all other operations.

```

#include <cstdint>
#include <stdexcept>

```

```

template<typename T>
class PairingHeap {
    struct Node {
        T value;
        Node *left, *next, *prev;

        explicit Node(const T &v) : value(v), left(nullptr), next(nullptr), prev(nullptr) {}

        // `prev` points to the previous sibling, or to the parent when this is the first child, so any
        // node can be cut from its child list in O(1).
        void add_child(Node *n) {
            n->prev = this;
            n->next = left;
            if (left != nullptr) {
                left->prev = n;
            }
            left = n;
        }
    } *root;

    int num_nodes;

    static Node *merge(Node *a, Node *b) {
        if (a == nullptr) {
            return b;
        }
        if (b == nullptr) {
            return a;
        }
        if (a->value < b->value) {
            a->add_child(b);
            a->prev = a->next = nullptr;
            return a;
        }
        b->add_child(a);
        b->prev = b->next = nullptr;
        return b;
    }

    // Detaches non-root node `n` from its parent's child list.
    static void cut(Node *n) {
        if (n->prev->left == n) {
            n->prev->left = n->next;
        } else {
            n->prev->next = n->next;
        }
        if (n->next != nullptr) {
            n->next->prev = n->prev;
        }
        n->prev = n->next = nullptr;
    }

    static Node *merge_pairs(Node *n) {
        if (n == nullptr || n->next == nullptr) {
            return n;
        }
        Node *a = n, *b = n->next, *c = n->next->next;
        a->next = b->next = nullptr;
        return merge(merge(a, b), merge_pairs(c));
    }

    static void clean_up(Node *n) {
        if (n != nullptr) {
            clean_up(n->left);
            clean_up(n->next);
            delete n;
        }
    }
}

```

```

public:
    PairingHeap() : root(nullptr), num_nodes(0) {}

    template<typename It>
    PairingHeap(It lo, It hi) : root(nullptr), num_nodes(0) {
        while (lo != hi) {
            push(*(lo++));
        }
    }

    ~PairingHeap() { clean_up(root); }
    PairingHeap(const PairingHeap &) = delete;
    PairingHeap &operator=(const PairingHeap &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }

    Node *push(const T &v) {
        Node *n = new Node(v);
        root = merge(root, n);
        num_nodes++;
        return n;
    }

    void pop() {
        if (empty()) {
            throw std::runtime_error("Cannot pop from empty heap.");
        }
        Node *tmp = root;
        root = merge_pairs(root->left);
        if (root != nullptr) {
            root->prev = root->next = nullptr;
        }
        delete tmp;
        num_nodes--;
    }

    T top() const {
        if (empty()) {
            throw std::runtime_error("Cannot get top of empty heap.");
        }
        return root->value;
    }

    void absorb(PairingHeap &h) {
        root = merge(root, h.root);
        num_nodes += h.num_nodes;
        h.root = nullptr;
        h.num_nodes = 0;
    }

    void decrease_key(Node *n, const T &v) {
        if (n->value < v) {
            throw std::runtime_error("decrease_key cannot increase the value.");
        }
        n->value = v;
        if (n != root) {
            cut(n);
            root = merge(root, n);
        }
    }

    void erase(Node *n) {
        if (n == root) {
            pop();
            return;
        }
        cut(n);
    }

```



```

    root = merge(root, merge_pairs(n->left));
    delete n;
    num_nodes--;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    PairingHeap<int> h, h2;
    h.push(12);
    h.push(10);
    h2.push(5);
    h2.push(-1);
    h2.push(0);
    h.absorb(h2);

    auto handle = h.push(100);
    h.decrease_key(handle, -5); // 100 -> -5, the new minimum.
    assert(h.top() == -5);
    h.erase(handle); // Remove it again, restoring the original five elements.
    assert(h.size() == 5);

    while (!h.empty()) {
        cout << h.top() << endl;
        h.pop();
    }
    return 0;
}

```

Example Output

```

-1
0
5
10
12

```

2.3 Dictionaries and Ordered Sets

2.3.1 Treap

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A binary search tree (a.k.a. BST) implements this map by inserting and deleting keys into a binary tree while upholding the BST property: for each node in the BST, every node in its left subtree has a lesser key and every node in its right subtree has a greater key.

A treap is a binary search tree where each node also holds a randomly assigned priority. The tree satisfies the BST property on keys and the min-heap property on priorities (lower priority value is closer to root). Since priorities are random, this keeps the tree balanced with high probability, making insertions and deletions run in $O(\log n)$.

This implementation requires an ordering on the key type $\mathbb{K}$ defined by `operator<`.

- `Treap<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.

- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in ascending order of keys.
- `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` return the matching key-value entry, or `std::nullopt` if no such entry exists. These navigation routines depend only on the BST property and can be copied unchanged to the other BST variants later in this chapter.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ on average per call to `insert()`, `erase()`, `find()`, `min()`, `max()`, `lower_bound()`, `upper_bound()`, `prev()`, and `next()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space on average for `insert()`, `erase()`, and `entries()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstdlib>
#include <optional>
#include <utility>
#include <vector>

template<typename K, typename V>
class Treap {
    struct Node {
        static uint32_t rand32() {
            static uint32_t x = 123456789;
            x ^= x << 13;
            x ^= x >> 17;
            x ^= x << 5;
            return x;
        }

        K key;
        V value;
        uint32_t priority;
        Node *left, *right;

        Node(const K &k, const V &v)
            : key(k), value(v), priority(rand32()), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;

    static void rotate_left(Node *&n) {
        Node *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
    }

    static void rotate_right(Node *&n) {
        Node *tmp = n;
        n = n->left;
        tmp->left = n->right;
        n->right = tmp;
    }
};
```

```

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        num_nodes++;
        return true;
    }
    if (k < n->key && insert(n->left, k, v)) {
        if (n->left->priority < n->priority) {
            rotate_right(n);
        }
        return true;
    }
    if (n->key < k && insert(n->right, k, v)) {
        if (n->right->priority < n->priority) {
            rotate_left(n);
        }
        return true;
    }
    return false;
}

bool erase(Node *&n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    if (k < n->key) {
        return erase(n->left, k);
    } else if (n->key < k) {
        return erase(n->right, k);
    }
    if (n->left != nullptr && n->right != nullptr) {
        if (n->left->priority < n->right->priority) {
            rotate_right(n);
            return erase(n->right, k);
        }
        rotate_left(n);
        return erase(n->left, k);
    }
    Node *tmp = (n->left != nullptr) ? n->left : n->right;
    delete n;
    n = tmp;
    num_nodes--;
    return true;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static std::optional<std::pair<K, V>> entry(Node *n) {
    if (n == nullptr) {
        return std::nullopt;
    }
    return std::pair<K, V>{n->key, n->value};
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

```

```

public:
    Treap() : root(nullptr), num_nodes(0) {}

    ~Treap() { clean_up(root); }
    Treap(const Treap &) = delete;
    Treap &operator=(const Treap &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) const {
        Node *n = root;
        while (n != nullptr) {
            if (k < n->key) {
                n = n->left;
            } else if (n->key < k) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return nullptr;
    }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(num_nodes);
        collect_entries(root, res);
        return res;
    }

    std::optional<std::pair<K, V>> min() const {
        Node *n = root;
        if (n == nullptr) {
            return std::nullopt;
        }
        while (n->left != nullptr) {
            n = n->left;
        }
        return std::pair<K, V>{n->key, n->value};
    }

    std::optional<std::pair<K, V>> max() const {
        Node *n = root;
        if (n == nullptr) {
            return std::nullopt;
        }
        while (n->right != nullptr) {
            n = n->right;
        }
        return std::pair<K, V>{n->key, n->value};
    }

    std::optional<std::pair<K, V>> lower_bound(const K &k) const {
        Node *n = root, *best = nullptr;
        while (n != nullptr) {
            if (!(n->key < k)) {
                best = n;
                n = n->left;
            } else {
                n = n->right;
            }
        }
        return entry(best);
    }

    std::optional<std::pair<K, V>> upper_bound(const K &k) const {

```

```

    Node *n = root, *best = nullptr;
    while (n != nullptr) {
        if (k < n->key) {
            best = n;
            n = n->left;
        } else {
            n = n->right;
        }
    }
    return entry(best);
}

std::optional<std::pair<K, V>> prev(const K &k) const {
    Node *n = root, *best = nullptr;
    while (n != nullptr) {
        if (n->key < k) {
            best = n;
            n = n->right;
        } else {
            n = n->left;
        }
    }
    return entry(best);
}

std::optional<std::pair<K, V>> next(const K &k) const { return upper_bound(k); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    Treap<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    assert(t.min()->first == 1 && t.min()->second == 'a');
    assert(t.max()->first == 5 && t.max()->second == 'e');
    assert(t.lower_bound(4)->first == 4);
    assert(t.upper_bound(4)->first == 5);
    assert(t.prev(4)->first == 3);
    assert(t.next(4)->first == 5);
    assert(!t.prev(1));
    assert(!t.next(5));
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    return 0;
}

```

Example Output

```
abcde
bcde
```

2.3.2 AVL Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. An AVL tree is a binary search tree balanced by height, guaranteeing $O(\log n)$ worst-case running time in insertions and deletions by making sure that the heights of the left and right subtrees at every node differ by at most 1. Whenever an insertion or deletion breaks this invariant, it is repaired with one or two rotations at each affected node along the search path.

This implementation requires an ordering on the key type `K` defined by `operator<`.

- `AVLTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in ascending order of keys.

The navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.3.1 depend only on the BST property and may be copied here unchanged.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `insert()`, `erase()`, and `entries()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstdint>
#include <utility>
#include <vector>

template<typename K, typename V>
class AVLTree {
    struct Node {
        K key;
        V value;
        int height;
        Node *left, *right;

        Node(const K &k, const V &v) : key(k), value(v), height(1), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;

    static int height(Node *n) { return (n != nullptr) ? n->height : 0; }

    static void update_height(Node *n) {
        if (n != nullptr) {
```

```

    n->height = 1 + std::max(height(n->left), height(n->right));
}
}

static void rotate_left(Node *&n) {
    Node *tmp = n;
    n = n->right;
    tmp->right = n->left;
    n->left = tmp;
    update_height(tmp);
    update_height(n);
}

static void rotate_right(Node *&n) {
    Node *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
    update_height(tmp);
    update_height(n);
}

static int balance_factor(Node *n) {
    return (n != nullptr) ? (height(n->left) - height(n->right)) : 0;
}

static void rebalance(Node *&n) {
    if (n == nullptr) {
        return;
    }
    update_height(n);
    int bf = balance_factor(n);
    if (bf > 1 && balance_factor(n->left) >= 0) {
        rotate_right(n);
    } else if (bf > 1 && balance_factor(n->left) < 0) {
        rotate_left(n->left);
        rotate_right(n);
    } else if (bf < -1 && balance_factor(n->right) <= 0) {
        rotate_left(n);
    } else if (bf < -1 && balance_factor(n->right) > 0) {
        rotate_right(n->right);
        rotate_left(n);
    }
}

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        num_nodes++;
        return true;
    }
    if ((k < n->key && insert(n->left, k, v)) || (n->key < k && insert(n->right, k, v))) {
        rebalance(n);
        return true;
    }
    return false;
}

bool erase(Node *&n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    if (!(k < n->key || n->key < k)) {
        if (n->left != nullptr && n->right != nullptr) {
            Node *tmp = n->right, *parent = nullptr;
            while (tmp->left != nullptr) {
                parent = tmp;
                tmp = tmp->left;
            }

```

```

    }
    n->key = tmp->key;
    n->value = tmp->value;
    if (parent != nullptr) {
        if (!erase(parent->left, parent->left->key)) {
            return false;
        }
        else if (!erase(n->right, n->right->key)) {
            return false;
        }
    }
    else {
        Node *tmp = (n->left != nullptr) ? n->left : n->right;
        delete n;
        n = tmp;
        num_nodes--;
    }
    rebalance(n);
    return true;
}
if ((k < n->key && erase(n->left, k)) || (n->key < k && erase(n->right, k))) {
    rebalance(n);
    return true;
}
return false;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    AVLTree() : root(nullptr), num_nodes(0) {}

    ~AVLTree() { clean_up(root); }
    AVLTree(const AVLTree &) = delete;
    AVLTree &operator=(const AVLTree &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) const {
        Node *n = root;
        while (n != nullptr) {
            if (k < n->key) {
                n = n->left;
            }
            else if (n->key < k) {
                n = n->right;
            }
            else {
                return &(n->value);
            }
        }
        return nullptr;
    }

    std::vector<std::pair<K, V>> entries() const {

```



```

        std::vector<std::pair<K, V>> res;
        res.reserve(num_nodes);
        collect_entries(root, res);
        return res;
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    AVLTree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    return 0;
}

```

Example Output

```

abcde
bcde

```

2.3.3 Red-Black Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A red-black tree is a binary search tree balanced by coloring its nodes red or black, then constraining node colors on any simple path from the root to a leaf. Specifically, a red node may never have a red child, and every path from the root to a null leaf must pass through the same number of black nodes, which together bound the tree's height to $O(\log n)$. Insertions and deletions repair these invariants by recoloring nodes and performing rotations.

This implementation requires an ordering on the key type `K` defined by `operator<`.

- `RedBlackTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.

- `entries()` returns all key-value entries in ascending order of keys.

The navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.3.1 depend only on the BST property and may be copied here unchanged.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `entries()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstdlib>
#include <utility>
#include <vector>

template<typename K, typename V>
class RedBlackTree {
    enum Color { RED, BLACK };
    struct Node {
        K key;
        V value;
        Color color;
        Node *left, *right, *parent;

        Node(const K &k, const V &v, Color c)
            : key(k), value(v), color(c), left(nullptr), right(nullptr), parent(nullptr) {}
    } *root, *LEAF_NIL;

    int num_nodes;

    void rotate_left(Node *n) {
        Node *tmp = n->right;
        if ((n->right = tmp->left) != LEAF_NIL) {
            n->right->parent = n;
        }
        if ((tmp->parent = n->parent) == LEAF_NIL) {
            root = tmp;
        } else if (n->parent->left == n) {
            n->parent->left = tmp;
        } else {
            n->parent->right = tmp;
        }
        tmp->left = n;
        n->parent = tmp;
    }

    void rotate_right(Node *n) {
        Node *tmp = n->left;
        if ((n->left = tmp->right) != LEAF_NIL) {
            n->left->parent = n;
        }
        if ((tmp->parent = n->parent) == LEAF_NIL) {
            root = tmp;
        } else if (n->parent->right == n) {
            n->parent->right = tmp;
        } else {
            n->parent->left = tmp;
        }
        tmp->right = n;
        n->parent = tmp;
    }
};
```

```

}

void insert_fix(Node *n) {
    while (n->parent->color == RED) {
        Node *parent = n->parent;
        Node *grandparent = n->parent->parent;
        if (parent == grandparent->left) {
            Node *uncle = grandparent->right;
            if (uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                n = grandparent;
            } else {
                if (n == parent->right) {
                    rotate_left(parent);
                    n = parent;
                    parent = n->parent;
                }
                rotate_right(grandparent);
                std::swap(parent->color, grandparent->color);
                n = parent;
            }
        } else if (parent == grandparent->right) {
            Node *uncle = grandparent->left;
            if (uncle->color == RED) {
                grandparent->color = RED;
                parent->color = BLACK;
                uncle->color = BLACK;
                n = grandparent;
            } else {
                if (n == parent->left) {
                    rotate_right(parent);
                    n = parent;
                    parent = n->parent;
                }
                rotate_left(grandparent);
                std::swap(parent->color, grandparent->color);
                n = parent;
            }
        }
    }
    root->color = BLACK;
}

void replace(Node *n, Node *replacement) {
    if (n->parent == LEAF_NIL) {
        root = replacement;
    } else if (n == n->parent->left) {
        n->parent->left = replacement;
    } else {
        n->parent->right = replacement;
    }
    replacement->parent = n->parent;
}

void erase_fix(Node *n) {
    while (n != root && n->color == BLACK) {
        Node *parent = n->parent;
        if (n == parent->left) {
            Node *sibling = parent->right;
            if (sibling->color == RED) {
                sibling->color = BLACK;
                parent->color = RED;
                rotate_left(parent);
                sibling = parent->right;
            }
            if (sibling->left->color == BLACK && sibling->right->color == BLACK) {

```

```

        sibling->color = RED;
        n = parent;
    } else {
        if (sibling->right->color == BLACK) {
            sibling->left->color = BLACK;
            sibling->color = RED;
            rotate_right(sibling);
            sibling = parent->right;
        }
        sibling->color = parent->color;
        parent->color = BLACK;
        sibling->right->color = BLACK;
        rotate_left(parent);
        n = root;
    }
} else {
    Node *sibling = parent->left;
    if (sibling->color == RED) {
        sibling->color = BLACK;
        parent->color = RED;
        rotate_right(parent);
        sibling = parent->left;
    }
    if (sibling->left->color == BLACK && sibling->right->color == BLACK) {
        sibling->color = RED;
        n = parent;
    } else {
        if (sibling->left->color == BLACK) {
            sibling->right->color = BLACK;
            sibling->color = RED;
            rotate_left(sibling);
            sibling = parent->left;
        }
        sibling->color = parent->color;
        parent->color = BLACK;
        sibling->left->color = BLACK;
        rotate_right(parent);
        n = root;
    }
}
}
n->color = BLACK;
}

void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) const {
    if (n != LEAF_NIL) {
        collect_entries(n->left, res);
        res.push_back({n->key, n->value});
        collect_entries(n->right, res);
    }
}

void clean_up(Node *n) {
    if (n != LEAF_NIL) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
RedBlackTree() : num_nodes(0) { root = LEAF_NIL = new Node(K(), V(), BLACK); }

~RedBlackTree() {
    clean_up(root);
    delete LEAF_NIL;
}

```

```

RedBlackTree(const RedBlackTree &) = delete;
RedBlackTree &operator=(const RedBlackTree &) = delete;
int size() const { return num_nodes; }
bool empty() const { return num_nodes == 0; }

bool insert(const K &k, const V &v) {
    Node *curr = root, *prev = LEAF_NIL;
    while (curr != LEAF_NIL) {
        prev = curr;
        if (k < curr->key) {
            curr = curr->left;
        } else if (curr->key < k) {
            curr = curr->right;
        } else {
            return false;
        }
    }
    Node *n = new Node(k, v, RED);
    n->parent = prev;
    if (prev == LEAF_NIL) {
        root = n;
    } else if (k < prev->key) {
        prev->left = n;
    } else {
        prev->right = n;
    }
    n->left = n->right = LEAF_NIL;
    insert_fix(n);
    num_nodes++;
    return true;
}

bool erase(const K &k) {
    Node *n = root;
    while (n != LEAF_NIL) {
        if (k < n->key) {
            n = n->left;
        } else if (n->key < k) {
            n = n->right;
        } else {
            break;
        }
    }
    if (n == LEAF_NIL) {
        return false;
    }
    Color color = n->color;
    Node *replacement;
    if (n->left == LEAF_NIL) {
        replacement = n->right;
        replace(n, n->right);
    } else if (n->right == LEAF_NIL) {
        replacement = n->left;
        replace(n, n->left);
    } else {
        Node *tmp = n->right;
        while (tmp->left != LEAF_NIL) {
            tmp = tmp->left;
        }
        color = tmp->color;
        replacement = tmp->right;
        if (tmp->parent == n) {
            replacement->parent = tmp;
        } else {
            replace(tmp, tmp->right);
            tmp->right = n->right;
            tmp->right->parent = tmp;
        }
    }
}

```

```

        replace(n, tmp);
        tmp->left = n->left;
        tmp->left->parent = tmp;
        tmp->color = n->color;
    }
    delete n;
    if (color == BLACK) {
        erase_fix(replacement);
    }
    return true;
}

const V *find(const K &k) const {
    Node *n = root;
    while (n != LEAF_NIL) {
        if (k < n->key) {
            n = n->left;
        } else if (n->key < k) {
            n = n->right;
        } else {
            return &(n->value);
        }
    }
    return nullptr;
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_nodes);
    collect_entries(root, res);
    return res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    RedBlackTree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    return 0;
}

```

Example Output

```
abcde
bcde
```

2.3.4 Splay Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. A splay tree is a balanced binary search tree with the additional property that recently accessed elements are quick to access again. Every operation “splays” its target node up to the root through a series of rotations, reshaping the tree so that frequently accessed keys settle near the top. A single operation may degrade to $O(n)$, but any sequence of operations averages out to $O(\log n)$ amortized each.

This implementation requires an ordering on the key type `K` defined by `operator<`.

- `SplayTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `entries()` returns all key-value entries in ascending order of keys.

The navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.3.1 depend only on the BST property and may be copied here unchanged.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ amortized per call to `insert()`, `erase()`, and `find()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `insert()`, `erase()`, and `entries()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstddef>
#include <utility>
#include <vector>

template<typename K, typename V>
class SplayTree {
    struct Node {
        K key;
        V value;
        Node *left, *right;

        Node(const K &k, const V &v) : key(k), value(v), left(nullptr), right(nullptr) {}
    } *root;

    int num_nodes;

    static void rotate_left(Node *&n) {
        Node *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
    }
};
```

```

}

static void rotate_right(Node *&n) {
    Node *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
}

static void splay(Node *&n, const K &k) {
    if (n == nullptr) {
        return;
    }
    if (k < n->key && n->left != nullptr) {
        if (k < n->left->key) {
            splay(n->left->left, k);
            rotate_right(n);
        } else if (n->left->key < k) {
            splay(n->left->right, k);
            if (n->left->right != nullptr) {
                rotate_left(n->left);
            }
        }
        if (n->left != nullptr) {
            rotate_right(n);
        }
    } else if (n->key < k && n->right != nullptr) {
        if (k < n->right->key) {
            splay(n->right->left, k);
            if (n->right->left != nullptr) {
                rotate_right(n->right);
            }
        } else if (n->right->key < k) {
            splay(n->right->right, k);
            rotate_left(n);
        }
        if (n->right != nullptr) {
            rotate_left(n);
        }
    }
}

bool insert(Node *&n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        num_nodes++;
        return true;
    }
    splay(n, k);
    if (k < n->key) {
        Node *tmp = new Node(k, v);
        tmp->left = n->left;
        tmp->right = n;
        n->left = nullptr;
        n = tmp;
    } else if (n->key < k) {
        Node *tmp = new Node(k, v);
        tmp->left = n;
        tmp->right = n->right;
        n->right = nullptr;
        n = tmp;
    } else {
        return false;
    }
    return true;
}

bool erase(Node *&n, const K &k) {

```



```

    if (n == nullptr) {
        return false;
    }
    splay(n, k);
    if (k < n->key || n->key < k) {
        return false;
    }
    Node *tmp = n;
    if (n->left == nullptr) {
        n = n->right;
    } else {
        splay(n->left, k);
        n = n->left;
        n->right = tmp->right;
    }
    delete tmp;
    num_nodes--;
    return true;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    SplayTree() : root(nullptr), num_nodes(0) {}

    ~SplayTree() { clean_up(root); }
    SplayTree(const SplayTree &) = delete;
    SplayTree &operator=(const SplayTree &) = delete;
    int size() const { return num_nodes; }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }

    const V *find(const K &k) {
        splay(root, k);
        if (root == nullptr) {
            return nullptr;
        }
        return (k < root->key || root->key < k) ? nullptr : &(root->value);
    }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(num_nodes);
        collect_entries(root, res);
        return res;
    }
};

```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    SplayTree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    return 0;
}
```

Example Output

```
abcde
bcde
```

2.3.5 Size Balanced Tree

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. In addition, support queries for keys given their ranks as well as queries for the ranks of given keys. A size balanced tree augments each node with the size of its subtree, using it to maintain balance and compute order statistics. After each update, rotations restore the invariant that every subtree is at least as large as each of its sibling's child subtrees, keeping the height logarithmic.

This implementation requires an ordering on the key type `K` defined by `operator<`.

- `SbTree<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to a const value associated with key `k`, or `nullptr` if the key was not found.
- `select(r)` returns a key-value pair of the node with a key of 0-based rank `r` in the map, throwing an exception if the rank is not in the range `[0, size())`.
- `rank(k)` returns the 0-based rank of key `k` in the map, throwing an exception if the key was not found in the map.
- `entries()` returns all key-value entries in ascending order of keys.

The navigation routines `min()`, `max()`, `lower_bound(k)`, `upper_bound(k)`, `prev(k)`, and `next(k)` from the treap in 2.3.1 depend only on the BST property and may be copied here unchanged.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ per call to `insert()`, `erase()`, `find()`, `select()`, and `rank()`, where n is the number of entries currently in the map.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(\log n)$ auxiliary stack space for `insert()`, `erase()`, and `entries()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstddef>
#include <stdexcept>
#include <utility>
#include <vector>

template<typename K, typename V>
class SBTTree {
    struct Node {
        K key;
        V value;
        int size;
        Node *left, *right;

        Node(const K &k, const V &v) : key(k), value(v), size(1), left(nullptr), right(nullptr) {}

        inline Node *&child(int c) { return (c == 0) ? left : right; }

        void update() {
            size = 1;
            if (left != nullptr) {
                size += left->size;
            }
            if (right != nullptr) {
                size += right->size;
            }
        }
    } *root;

    static inline int size(Node *n) { return (n == nullptr) ? 0 : n->size; }

    static void rotate(Node *&n, int c) {
        Node *tmp = n->child(c);
        n->child(c) = tmp->child(!c);
        tmp->child(!c) = n;
        n->update();
        tmp->update();
        n = tmp;
    }

    static void maintain(Node *&n, int c) {
        if (n == nullptr || n->child(c) == nullptr) {
            return;
        }
        Node *&tmp = n->child(c);
        if (size(tmp->child(c)) > size(n->child(!c))) {
            rotate(n, c);
        } else if (size(tmp->child(!c)) > size(n->child(!c))) {
            rotate(tmp, !c);
            rotate(n, c);
        } else {
            return;
        }
        maintain(n->left, 0);
        maintain(n->right, 1);
    }
};
```

```

    maintain(n, 0);
    maintain(n, 1);
}

static bool insert(Node *n, const K &k, const V &v) {
    if (n == nullptr) {
        n = new Node(k, v);
        return true;
    }
    bool found;
    if (k < n->key) {
        found = insert(n->left, k, v);
        maintain(n, 0);
    } else if (n->key < k) {
        found = insert(n->right, k, v);
        maintain(n, 1);
    } else {
        return false;
    }
    n->update();
    return found;
}

static bool erase(Node *n, const K &k) {
    if (n == nullptr) {
        return false;
    }
    bool found;
    int c = (k < n->key);
    if (k < n->key) {
        found = erase(n->left, k);
    } else if (n->key < k) {
        found = erase(n->right, k);
    } else {
        if (n->right == nullptr || n->left == nullptr) {
            Node *tmp = n;
            n = (n->right == nullptr) ? n->left : n->right;
            delete tmp;
            return true;
        }
        Node *p = n->right;
        while (p->left != nullptr) {
            p = p->left;
        }
        n->key = p->key;
        found = erase(n->right, p->key);
    }
    maintain(n, c);
    n->update();
    return found;
}

static std::pair<K, V> select(Node *n, int r) {
    int rank = size(n->left);
    if (r < rank) {
        return select(n->left, r);
    } else if (r > rank) {
        return select(n->right, r - rank - 1);
    }
    return {n->key, n->value};
}

static int rank(Node *n, const K &k) {
    if (n == nullptr) {
        throw std::runtime_error("Cannot rank key that's not in tree.");
    }
    int r = size(n->left);
    if (k < n->key) {

```

```

        return rank(n->left, k);
    } else if (n->key < k) {
        return rank(n->right, k) + r + 1;
    }
    return r;
}

static void collect_entries(Node *n, std::vector<std::pair<K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->key, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    SBTTree() : root(nullptr) {}

    ~SBTTree() { clean_up(root); }
    SBTTree(const SBTTree &) = delete;
    SBTTree &operator=(const SBTTree &) = delete;
    int size() const { return size(root); }
    bool empty() const { return root == nullptr; }
    bool insert(const K &k, const V &v) { return insert(root, k, v); }
    bool erase(const K &k) { return erase(root, k); }
    int rank(const K &k) const { return rank(root, k); }

    const V *find(const K &k) const {
        Node *n = root;
        while (n != nullptr) {
            if (k < n->key) {
                n = n->left;
            } else if (n->key < k) {
                n = n->right;
            } else {
                return &(n->value);
            }
        }
        return nullptr;
    }

    std::pair<K, V> select(int r) const {
        if (r < 0 || r >= size(root)) {
            throw std::runtime_error("Select rank must be between 0 and size() - 1.");
        }
        return select(root, r);
    }

    std::vector<std::pair<K, V>> entries() const {
        std::vector<std::pair<K, V>> res;
        res.reserve(size(root));
        collect_entries(root, res);
        return res;
    }
};

```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    SBTTree<int, char> t;
    t.insert(2, 'b');
    t.insert(1, 'a');
    t.insert(3, 'c');
    t.insert(5, 'e');
    assert(t.insert(4, 'd'));
    assert(*t.find(4) == 'd');
    assert(!t.insert(4, 'd'));
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    assert(t.erase(1));
    assert(!t.erase(1));
    assert(t.find(1) == nullptr);
    for (const auto &[k, v] : t.entries()) {
        cout << v;
    }
    cout << endl;
    assert(t.rank(2) == 0);
    assert(t.rank(3) == 1);
    assert(t.rank(5) == 3);
    assert(t.select(0).first == 2);
    assert(t.select(1).first == 3);
    assert(t.select(2).first == 4);
    return 0;
}
```

Example Output

```
abcde
bcde
```

2.3.6 Interval Treap

Maintain an ordered map from closed, one-dimensional intervals to values while supporting efficient reporting of any or all entries that intersect with a given query interval. A treap is used to process the entries, where keys are compared lexicographically as pairs. Each node is additionally augmented with the maximum upper endpoint among the intervals in its subtree, letting intersection queries skip any subtree whose maximum falls below the query interval's lower bound.

This implementation uses `std::pair` to represent intervals, requiring operators `<` and `==` to be defined on the numeric key type `K`.

- `IntervalTreap<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(lo, hi, v)` adds an entry with key `[lo, hi]` and value `v` to the map, returning `true` if a new interval was added or `false` if the interval already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(lo, hi)` removes the entry with key `[lo, hi]` from the map, returning `true` if the removal was successful or `false` if the interval was not found.
- `find_key(lo, hi)` returns a pointer to a const `std::pair` representing the key of some interval in the map which intersects with `[lo, hi]`, or `nullptr` if no such entry was found.

- `find_value(lo, hi)` returns a pointer to a const value of some entry in the map with a key that intersects with $[lo, hi]$, or `nullptr` if no such entry was found.
- `find_all(lo, hi, f)` calls the function `f(lo, hi, v)` on each entry in the map that overlaps with $[lo, hi]$, in lexicographically ascending order of intervals.
- `entries()` returns all intervals and their values in lexicographically ascending order of intervals.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ on average per call to `insert()`, `erase()`, and `find_any()`, where n is the number of intervals currently in the set.
- $O(\log n + m)$ on average per call to `find_all()`, where m is the number of intersecting intervals that are reported.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(1)$ auxiliary for `size()` and `empty()`.
- $O(\log n)$ auxiliary stack space on average for all other operations.

```
#include <cstdlib>
#include <tuple>
#include <utility>
#include <vector>

template<typename K, typename V>
class IntervalTreap {
    using Interval = std::pair<K, K>;

    struct Node {
        static uint32_t rand32() {
            static uint32_t x = 123456789;
            x ^= x << 13;
            x ^= x >> 17;
            x ^= x << 5;
            return x;
        }

        Interval interval;
        V value;
        K max;
        uint32_t priority;
        Node *left, *right;

        Node(const Interval &i, const V &v)
            : interval(i), value(v), max(i.second), priority(rand32()), left(nullptr), right(nullptr) {}

        void update() {
            max = interval.second;
            if (left != nullptr && left->max > max) {
                max = left->max;
            }
            if (right != nullptr && right->max > max) {
                max = right->max;
            }
        }
    };

    *root;

    int num_nodes;

    static void rotate_left(Node *&n) {
        Node *tmp = n;
        n = n->right;
        tmp->right = n->left;
        n->left = tmp;
    }
};
```

```

    tmp->update();
}

static void rotate_right(Node *&n) {
    Node *tmp = n;
    n = n->left;
    tmp->left = n->right;
    n->right = tmp;
    tmp->update();
}

bool insert(Node *&n, const Interval &i, const V &v) {
    if (n == nullptr) {
        n = new Node(i, v);
        num_nodes++;
        return true;
    }
    if (i < n->interval && insert(n->left, i, v)) {
        if (n->left->priority < n->priority) {
            rotate_right(n);
        }
        n->update();
        return true;
    }
    if (i > n->interval && insert(n->right, i, v)) {
        if (n->right->priority < n->priority) {
            rotate_left(n);
        }
        n->update();
        return true;
    }
    return false;
}

bool erase(Node *&n, const Interval &i) {
    if (n == nullptr) {
        return false;
    }
    if (i < n->interval) {
        return erase(n->left, i);
    }
    if (i > n->interval) {
        return erase(n->right, i);
    }
    if (n->left != nullptr && n->right != nullptr) {
        bool res;
        if (n->left->priority < n->right->priority) {
            rotate_right(n);
            res = erase(n->right, i);
        } else {
            rotate_left(n);
            res = erase(n->left, i);
        }
        n->update();
        return res;
    }
    Node *tmp = (n->left != nullptr) ? n->left : n->right;
    delete n;
    n = tmp;
    num_nodes--;
    return true;
}

static Node *find_any(Node *n, const Interval &i) {
    if (n == nullptr) {
        return nullptr;
    }
    if (n->interval.first <= i.second && i.first <= n->interval.second) {

```



```

    return n;
}
if (n->left != nullptr && i.first <= n->left->max) {
    return find_any(n->left, i);
}
return find_any(n->right, i);
}

template<typename Fn>
static void find_all(Node *n, const Interval &i, Fn f) {
    if (n == nullptr || n->max < i.first) {
        return;
    }
    find_all(n->left, i, f);
    if (n->interval.first <= i.second && i.first <= n->interval.second) {
        f(n->interval.first, n->interval.second, n->value);
    }
    if (n->interval.first <= i.second) {
        find_all(n->right, i, f);
    }
}

static void collect_entries(Node *n, std::vector<std::tuple<K, K, V>> &res) {
    if (n != nullptr) {
        collect_entries(n->left, res);
        res.emplace_back(n->interval.first, n->interval.second, n->value);
        collect_entries(n->right, res);
    }
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
IntervalTreap() : root(nullptr), num_nodes(0) {}

~IntervalTreap() { clean_up(root); }
IntervalTreap(const IntervalTreap &) = delete;
IntervalTreap &operator=(const IntervalTreap &) = delete;
int size() const { return num_nodes; }
bool empty() const { return root == nullptr; }
bool insert(const K &lo, const K &hi, const V &v) { return insert(root, Interval{lo, hi}, v); }
bool erase(const K &lo, const K &hi) { return erase(root, Interval{lo, hi}); }

const Interval *find_key(const K &lo, const K &hi) const {
    Node *n = find_any(root, Interval{lo, hi});
    return (n == nullptr) ? nullptr : &(n->interval);
}

const V *find_value(const K &lo, const K &hi) const {
    Node *n = find_any(root, Interval{lo, hi});
    return (n == nullptr) ? nullptr : &(n->value);
}

template<typename Fn>
void find_all(const K &lo, const K &hi, Fn f) const {
    find_all(root, Interval{lo, hi}, f);
}

std::vector<std::tuple<K, K, V>> entries() const {
    std::vector<std::tuple<K, K, V>> res;
    res.reserve(num_nodes);
    collect_entries(root, res);
}

```

```

    return res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    IntervalTreap<int, char> t;
    t.insert(15, 20, 'a');
    t.insert(10, 30, 'b');
    t.insert(17, 19, 'c');
    t.insert(5, 20, 'd');
    t.insert(12, 15, 'e');
    t.insert(10, 40, 'f');
    assert(t.size() == 6);
    assert(!t.insert(5, 20, 'x'));
    t.erase(17, 19);
    assert(t.size() == 5);
    assert(*t.find_key(3, 9) == (pair<int, int>{5, 20}));
    assert(*t.find_value(3, 9) == 'd');
    cout << "Intervals intersecting [16, 20]:";
    auto print = [](int lo, int hi, char v) { cout << " [" << lo << ", " << hi << "]"; };
    t.find_all(16, 20, print);
    cout << "\nAll intervals:";
    for (const auto &[lo, hi, v] : t.entries()) {
        print(lo, hi, v);
    }
    cout << endl;
    return 0;
}

```

Example Output

```

Intervals intersecting [16, 20]: [5, 20] [10, 30] [10, 40] [15, 20]
All intervals: [5, 20] [10, 30] [10, 40] [12, 15] [15, 20]

```

2.3.7 Hash Table (Chaining)

Maintain an unordered map: a collection of key-value pairs in which each key appears at most once. Keys are hashed into buckets, and this implementation resolves collisions by chaining the entries in each bucket into a linked list. It requires `operator==` on the key type and a hash functor.

Compared with the open-addressing version (2.3.8), chaining keeps each entry at a stable address: a rehash never moves existing nodes, so pointers from `find()` and references from `operator[]` stay valid, and load factors above 1 are tolerated gracefully. The costs are a separate allocation per entry and cache-unfriendly pointer chasing during traversal.

- `ChainingHashMap<K, V, Hash>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to the value associated with key `k`, or `nullptr` if the key was not found.

- `operator[k]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `entries()` returns all key-value entries in no guaranteed order.

This is an educational implementation; in practice prefer one of the standard options:

- `std::unordered_map` is the portable standard-library hash map. It is also chaining-based, so this class is essentially a transparent version of it. Pass a custom hash (see 3.2.1) to harden it against adversarial inputs, as its default integer hash is effectively the identity.
- `__gnu_pbds::gp_hash_table` (from GCC's policy-based library) is an open-addressing table that is typically several times faster than `std::unordered_map`, but is a non-portable GNU extension. Prefer it on GCC-only judges when hashing is the bottleneck.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(1)$ amortized per call to `insert()`, `erase()`, `find()`, and `operator[]`.
- $O(n)$ per call to `entries()`, where n is the number of entries in the map.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(n)$ auxiliary heap space for `insert()`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstdint>
#include <list>
#include <utility>
#include <vector>

template<typename K, typename V, typename Hash>
class ChainingHashMap {
    struct HashNode {
        K key;
        V value;
        HashNode(const K &k, const V &v) : key(k), value(v) {}
    };

    std::vector<std::list<HashNode>>> table;
    int table_size, num_entries;

    void grow_and_rehash() {
        auto old = std::move(table);
        table_size = 2 * table_size;
        table.assign(table_size, std::list<HashNode>());
        num_entries = 0;
        for (auto &bucket : old) {
            for (auto &entry : bucket) {
                insert(entry.key, entry.value);
            }
        }
    }

public:
    explicit ChainingHashMap(int size = 128) : table(size), table_size(size), num_entries(0) {}

    int size() const { return num_entries; }
    bool empty() const { return num_entries == 0; }

    bool insert(const K &k, const V &v) {
        if (find(k) != nullptr) {
            return false;
        }
        if (num_entries >= table_size) {
            grow_and_rehash();
        }
    }
};
```

```

    uint32_t i = Hash()(k) % table_size;
    table[i].emplace_back(k, v);
    num_entries++;
    return true;
}

bool erase(const K &k) {
    uint32_t i = Hash()(k) % table_size;
    auto it = table[i].begin();
    while (it != table[i].end() && !(it->key == k)) {
        ++it;
    }
    if (it == table[i].end()) {
        return false;
    }
    table[i].erase(it);
    num_entries--;
    return true;
}

V *find(const K &k) {
    uint32_t i = Hash()(k) % table_size;
    auto it = table[i].begin();
    while (it != table[i].end() && !(it->key == k)) {
        ++it;
    }
    if (it == table[i].end()) {
        return nullptr;
    }
    return &(it->value);
}

V &operator[](const K &k) {
    if (V *ptr = find(k); ptr != nullptr) {
        return *ptr;
    }
    if (num_entries >= table_size) {
        grow_and_rehash();
    }
    uint32_t i = Hash()(k) % table_size;
    table[i].emplace_back(k, V());
    num_entries++;
    return table[i].back().value;
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_entries);
    for (int i = 0; i < table_size; i++) {
        for (const auto &[k, v] : table[i]) {
            res.emplace_back(k, v);
        }
    }
    return res;
}
};

```

Example Usage

```

#include <algorithm>
#include <cassert>
#include <chrono>
#include <string>
using namespace std;

// Example key hashers. For more hash algorithms and overloads, see 3.2.1. To defend against

```

```

// adversarially crafted collisions in open-hacking environments, a random seed is added to input
// keys before mixing (as 3.2.1's IntHasher does).
struct Hasher {
    inline static const uint64_t RAND_SEED =
        std::chrono::steady_clock::now().time_since_epoch().count();

    // Signed -> unsigned delegates.
    uint32_t operator()(int k) { return Hasher()(static_cast<uint32_t>(k)); }
    uint32_t operator()(int64_t k) { return Hasher()(static_cast<uint64_t>(k)); }

    // Knuth's multiplicative method. Fast, but affine: an additive RAND_SEED only shifts all buckets
    // uniformly and won't stop crafted collisions (unlike the non-linear hashers below). To harden
    // it, randomize the odd multiplier and take the high bits instead.
    uint32_t operator()(uint32_t k) {
        return k * 2654435761u; // Or just return k.
    }

    // SplitMix64 mixer (see 3.2.1's mix64).
    uint32_t operator()(uint64_t k) {
        k += RAND_SEED;
        k = (k ^ (k >> 30)) * 0xbf58476d1ce4e5b9ULL;
        k = (k ^ (k >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<uint32_t>(k ^ (k >> 31));
    }

    // Jenkins's one-at-a-time hash.
    uint32_t operator()(const std::string &k) {
        uint32_t hash = RAND_SEED;
        for (char c : k) {
            hash += ((hash + static_cast<unsigned char>(c)) << 10);
            hash ^= (hash >> 6);
        }
        hash += (hash << 3);
        hash ^= (hash >> 11);
        return hash + (hash << 15);
    }
};

int main() {
    ChainingHashMap<string, char, Hasher> m;
    m["foo"] = 'a';
    m.insert("bar", 'b');
    assert(m["foo"] == 'a');
    assert(m["bar"] == 'b');
    assert(m["baz"] == '\0');
    m["baz"] = 'c';

    string vals;
    for (const auto &[k, v] : m.entries()) {
        vals += v;
    }
    sort(vals.begin(), vals.end());
    assert(vals == "abc");
    assert(m.erase("foo"));
    assert(m.size() == 2);
    assert(m["foo"] == '\0');
    assert(m.size() == 3);
    return 0;
}

```

2.3.8 Hash Table (Open Addressing)

Maintain an unordered map: a collection of key-value pairs in which each key appears at most once. Keys are hashed into table slots, and this implementation resolves collisions using open addressing with linear probing.

Deletions leave tombstones behind so that probe chains remain searchable. It requires `operator==` on the key type and a hash functor.

Linear probing checks slots i , $i + 1$, $i + 2$, ... (modulo the table size) until the desired key or an empty slot is found. Other probe-step schemes include quadratic probing and double hashing, but these require additional table-sizing and coprimality conditions to guarantee correct coverage of the table. They are omitted to keep this implementation focused on core ideas of open addressing.

Compared with the chaining version (2.3.7), open addressing stores entries contiguously, so it is more cache-friendly and needs no per-entry allocation. The costs: deletions must leave tombstones, the load factor must stay well below 1, and, unlike chaining or `std::unordered_map`, a pointer from `find()` or reference from `operator[]` is invalidated as soon as a later insertion rehashes and relocates the entries.

- `ProbingHashMap<K, V, Hash>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to the value associated with key `k`, or `nullptr` if the key was not found.
- `operator[k]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `entries()` returns all key-value entries in no guaranteed order.

This is an educational implementation; in practice prefer one of the standard options:

- `std::unordered_map` is the portable standard-library hash map. Unlike this class, it is chaining-based rather than open-addressed. Pass a custom hash (see 3.2.1) to harden it against adversarial inputs, as its default integer hash is effectively the identity.
- `__gnu_pbds::gp_hash_table` (from GCC's policy-based library) is an open-addressing table that is typically several times faster than `std::unordered_map`, but is a non-portable GNU extension. Prefer it on GCC-only judges when hashing is the bottleneck.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(1)$ amortized per call to `insert()`, `erase()`, `find()`, and `operator[]`.
- $O(n)$ per call to `entries()`, where n is the number of entries in the map.

Space Complexity:

- $O(n)$ for storage of the map elements.
- $O(1)$ auxiliary heap space for `insert()` and `operator[]` amortized over a sequence of operations (excluding occasional table rehashes).
- $O(1)$ auxiliary for all other operations.

```
#include <cstdint>
#include <optional>
#include <utility>
#include <vector>

template<typename K, typename V, typename Hash>
class ProbingHashMap {
    enum class State { EMPTY, OCCUPIED, DELETED };

    struct HashNode {
        K key;
        V value;
        HashNode(const K &k, const V &v) : key(k), value(v) {}
    };
```

```

std::vector<std::optional<HashNode>> table;
std::vector<State> state;
int table_size, num_entries, num_tombstones;

int bucket(const K &k) const {
    return static_cast<int>(Hash()(k) % static_cast<uint32_t>(table_size));
}

void rehash(int new_size) {
    auto old_table = std::move(table);
    auto old_state = std::move(state);
    table_size = new_size;
    table.assign(table_size, std::nullopt);
    state.assign(table_size, State::EMPTY);
    num_entries = 0;
    num_tombstones = 0;
    for (int i = 0; i < static_cast<int>(old_table.size()); i++) {
        if (old_state[i] == State::OCCUPIED) {
            insert(old_table[i]->key, old_table[i]->value);
        }
    }
}

void grow_if_needed() {
    // Keep total non-empty slots below 50%. Tombstones count because they lengthen probe chains.
    if (2 * (num_entries + num_tombstones) >= table_size) {
        rehash(2 * table_size);
    }
}

public:
explicit ProbingHashMap(int size = 128)
    : table(size),
      state(size, State::EMPTY),
      table_size(size),
      num_entries(0),
      num_tombstones(0) {}

int size() const { return num_entries; }
bool empty() const { return num_entries == 0; }

bool insert(const K &k, const V &v) {
    grow_if_needed();
    int first_deleted = table_size;
    int i = bucket(k);
    for (int probes = 0; probes < table_size; probes++) {
        if (state[i] == State::OCCUPIED) {
            if (table[i]->key == k) {
                return false;
            }
        } else if (state[i] == State::DELETED) {
            if (first_deleted == table_size) {
                first_deleted = i;
            }
        } else {
            int dest = first_deleted == table_size ? i : first_deleted;
            table[dest].emplace(k, v);
            if (state[dest] == State::DELETED) {
                num_tombstones--;
            }
            state[dest] = State::OCCUPIED;
            num_entries++;
            return true;
        }
        i = (i + 1) % table_size;
    }
    // Should be rare because grow_if_needed keeps slack, but safe.

```

```

    rehash(2 * table_size);
    return insert(k, v);
}

bool erase(const K &k) {
    int i = bucket(k);
    for (int probes = 0; probes < table_size; probes++) {
        if (state[i] == State::EMPTY) {
            return false;
        }
        if (state[i] == State::OCCUPIED && table[i]->key == k) {
            table[i].reset();
            state[i] = State::DELETED;
            num_entries--;
            num_tombstones++;
            return true;
        }
        i = (i + 1) % table_size;
    }
    return false;
}

V *find(const K &k) {
    int i = bucket(k);
    for (int probes = 0; probes < table_size; probes++) {
        if (state[i] == State::EMPTY) {
            return nullptr;
        }
        if (state[i] == State::OCCUPIED && table[i]->key == k) {
            return &table[i]->value;
        }
        i = (i + 1) % table_size;
    }
    return nullptr;
}

V &operator[](const K &k) {
    if (V *ptr = find(k); ptr != nullptr) {
        return *ptr;
    }
    grow_if_needed();
    int first_deleted = table_size;
    int i = bucket(k);
    for (int probes = 0; probes < table_size; probes++) {
        if (state[i] == State::DELETED) {
            if (first_deleted == table_size) {
                first_deleted = i;
            }
        }
        else if (state[i] == State::EMPTY) {
            int dest = first_deleted == table_size ? i : first_deleted;
            table[dest].emplace(k, V());
            if (state[dest] == State::DELETED) {
                num_tombstones--;
            }
            state[dest] = State::OCCUPIED;
            num_entries++;
            return table[dest]->value;
        }
        i = (i + 1) % table_size;
    }
    rehash(2 * table_size);
    return (*this)[k];
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_entries);
    for (int i = 0; i < table_size; i++) {

```



```

        if (state[i] == State::OCCUPIED) {
            res.emplace_back(table[i]->key, table[i]->value);
        }
    }
    return res;
}
};

```

Example Usage

```

#include <algorithm>
#include <cassert>
#include <chrono>
#include <string>
using namespace std;

// Example key hashers. For more hash algorithms and overloads, see 3.2.1. To defend against
// adversarially crafted collisions in open-hacking environments, a random seed is added to input
// keys before mixing (as 3.2.1's IntHasher does).
struct Hasher {
    inline static const uint64_t RAND_SEED =
        std::chrono::steady_clock::now().time_since_epoch().count();

    // Signed -> unsigned delegates.
    uint32_t operator()(int k) { return Hasher()(static_cast<uint32_t>(k)); }
    uint32_t operator()(int64_t k) { return Hasher()(static_cast<uint64_t>(k)); }

    // Knuth's multiplicative method. Fast, but affine: an additive RAND_SEED only shifts all buckets
    // uniformly and won't stop crafted collisions (unlike the non-linear hashers below). To harden
    // it, randomize the odd multiplier and take the high bits instead.
    uint32_t operator()(uint32_t k) {
        return k * 2654435761u; // Or just return k.
    }

    // SplitMix64 mixer (see 3.2.1's mix64).
    uint32_t operator()(uint64_t k) {
        k += RAND_SEED;
        k = (k ^ (k >> 30)) * 0xbf58476d1ce4e5b9ULL;
        k = (k ^ (k >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<uint32_t>(k ^ (k >> 31));
    }

    // Jenkins's one-at-a-time hash.
    uint32_t operator()(const std::string &k) {
        uint32_t hash = RAND_SEED;
        for (char c : k) {
            hash += ((hash + static_cast<unsigned char>(c)) << 10);
            hash ^= (hash >> 6);
        }
        hash += (hash << 3);
        hash ^= (hash >> 11);
        return hash + (hash << 15);
    }
};

int main() {
    ProbingHashMap<string, char, Hasher> m;
    m["foo"] = 'a';
    m.insert("bar", 'b');
    assert(m["foo"] == 'a');
    assert(m["bar"] == 'b');
    assert(m["baz"] == '\0');
    m["baz"] = 'c';

    string vals;
    for (const auto &[k, v] : m.entries()) {

```

```

    vals += v;
}
sort(vals.begin(), vals.end());
assert(vals == "abc");
assert(m.erase("foo"));
assert(m.size() == 2);
assert(m["foo"] == '\0');
assert(m.size() == 3);
return 0;
}

```

2.3.9 Skip List

Maintain an ordered map, that is, an ordered collection of key-value pairs such that each possible key appears at most once in the collection. This implementation requires operators `<` and `==` to be defined on the key type. A skip list maintains a linked hierarchy of sorted subsequences with each successive subsequence skipping over fewer elements than the previous one. Each new node joins a number of levels decided by repeated coin flips, and a search starts at the sparsest level, moving forward until the next key would overshoot and then dropping down a level, which makes operations take $O(\log n)$ with high probability.

- `SkipList<K, V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(k, v)` adds an entry with key `k` and value `v` to the map, returning `true` if a new entry was added or `false` if the key already exists (in which case the map is unchanged and the old value associated with the key is preserved).
- `erase(k)` removes the entry with key `k` from the map, returning `true` if the removal was successful or `false` if the key to be removed was not found.
- `find(k)` returns a pointer to the value associated with key `k`, or `nullptr` if the key was not found.
- `operator[k]` returns a reference to key `k`'s associated value (which may be modified), or if necessary, inserts and returns a new entry with the default constructed value if key `k` was not originally found.
- `entries()` returns all key-value entries in ascending order of keys.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `empty()`.
- $O(\log n)$ on average per call to `insert()`, `erase()`, `find()`, and `operator[]`, where n is the number of entries currently in the map.
- $O(n)$ per call to `entries()`.

Space Complexity:

- $O(n)$ on average for storage of the map elements.
- $O(n)$ auxiliary heap space for `insert()` and `erase()`.
- $O(1)$ auxiliary for all other operations.

```

#include <random>
#include <utility>
#include <vector>

template<typename K, typename V>
class SkipList {
    static const int MAX_LEVELS = 32; // log2(max possible keys)

    struct Node {
        K key;
        V value;
        std::vector<Node *> next;

        Node(const K &k, const V &v, int levels) : key(k), value(v), next(levels, (Node *)nullptr) {}
    };
};

```

```

} *head;

int num_nodes;

static int random_level() {
    static std::mt19937 rng(std::random_device{}());
    static std::uniform_int_distribution<int> coin(0, 1);
    int level = 1;
    while (coin(rng) && level < MAX_LEVELS) {
        level++;
    }
    return level;
}

static int node_level(const std::vector<Node *> &v) {
    int i = 0;
    while (i < static_cast<int>(v.size()) && v[i] != nullptr) {
        i++;
    }
    return i + 1;
}

public:
Skiplist() : head(new Node(K(), V(), MAX_LEVELS)), num_nodes(0) {
    for (auto &ptr : head->next) {
        ptr = nullptr;
    }
}

~Skiplist() { delete head; }
Skiplist(const Skiplist &) = delete;
Skiplist &operator=(const Skiplist &) = delete;
int size() const { return num_nodes; }
bool empty() const { return num_nodes == 0; }

bool insert(const K &k, const V &v) {
    std::vector<Node *> update(head->next);
    int curr_level = node_level(update);
    Node *n = head;
    for (int i = curr_level; i-- > 0;) {
        while (n->next[i] != nullptr && n->next[i]->key < k) {
            n = n->next[i];
        }
        update[i] = n;
    }
    n = n->next[0];
    if (n != nullptr && n->key == k) {
        return false;
    }
    int new_level = random_level();
    if (new_level > curr_level) {
        for (int i = curr_level; i < new_level; i++) {
            update[i] = head;
        }
    }
    n = new Node(k, v, new_level);
    for (int i = 0; i < new_level; i++) {
        n->next[i] = update[i]->next[i];
        update[i]->next[i] = n;
    }
    num_nodes++;
    return true;
}

bool erase(const K &k) {
    std::vector<Node *> update(head->next);
    Node *n = head;
    for (int i = node_level(update); i-- > 0;) {

```

```

    while (n->next[i] != nullptr && n->next[i]->key < k) {
        n = n->next[i];
    }
    update[i] = n;
}
n = n->next[0];
if (n != nullptr && n->key == k) {
    for (int i = 0, levels = static_cast<int>(update.size()); i < levels; i++) {
        if (update[i]->next[i] != n) {
            break;
        }
        update[i]->next[i] = n->next[i];
    }
    delete n;
    num_nodes--;
    return true;
}
return false;
}

V *find(const K &k) const {
    Node *n = head;
    for (int i = node_level(n->next); i-- > 0;) {
        while (n->next[i] != nullptr && n->next[i]->key < k) {
            n = n->next[i];
        }
    }
    n = n->next[0];
    return (n != nullptr && n->key == k) ? &(n->value) : nullptr;
}

V &operator[](const K &k) {
    V *ptr = find(k);
    if (ptr != nullptr) {
        return *ptr;
    }
    insert(k, V());
    return *find(k);
}

std::vector<std::pair<K, V>> entries() const {
    std::vector<std::pair<K, V>> res;
    res.reserve(num_nodes);
    Node *n = head->next[0];
    while (n != nullptr) {
        res.emplace_back(n->key, n->value);
        n = n->next[0];
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    SkipList<int, char> l;
    l.insert(2, 'b');
    l.insert(1, 'a');
    l.insert(3, 'c');
    l.insert(5, 'e');
    assert(l.insert(4, 'd'));
    assert(*l.find(4) == 'd');
}

```

```

assert(!l.insert(4, 'd'));
for (const auto &[k, v] : l.entries()) {
    cout << v;
}
cout << endl;
assert(l.erase(1));
assert(!l.erase(1));
assert(l.find(1) == nullptr);
for (const auto &[k, v] : l.entries()) {
    cout << v;
}
cout << endl;
return 0;
}

```

Example Output

```

abcde
bcde

```

2.4 Range Queries in One Dimension

2.4.1 Sparse Table

Given a static array, precompute a table that answers range minimum queries in constant time. More generally, the sparse table works for any range query whose combining operation is both associative and idempotent, meaning `combine(x, x) == x`. The canonical examples are “min”, “max”, “gcd”, and the bitwise “and” and “or”.

For each level j , the entry `dp[j][i]` holds the result of `combine()` over the half-open range $[i, i + 2^j)$, built by combining two ranges of length 2^{j-1} . A query over $[lo, hi]$ is answered by combining the two overlapping ranges of length 2^j that are anchored at `lo` and at `hi`, where 2^j is the largest power of two not exceeding the length of the range. Because these two ranges overlap whenever the query length is not a power of two, the elements in the overlap are folded in twice; this is harmless only when `combine()` is idempotent. To support a non-idempotent but associative operation such as “sum” or “XOR” in constant time, use the disjoint sparse table in the next section instead, whose ranges meet at a center and never overlap.

The query operation is defined by an associative and idempotent function `combine(a, b)`. The default code below returns the “min” of the range; for “gcd”, `combine(a, b)` should return `gcd(a, b)`.

- `SparseTable<T>(lo, hi)` builds the table from two random-access iterators.
- `size()` returns the size of the array.
- `query(lo, hi)` returns `combine()` applied to all indices from `lo` to `hi`, inclusive.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()` and `query()`.

Space Complexity:

- $O(n \log n)$ for storage of the table.
- $O(1)$ auxiliary for `query()`.

```

#include <algorithm>
#include <cassert>
#include <vector>

template<typename T>
class SparseTable {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

```

```

int len;
std::vector<int> log2; // Can also use 31 - __builtin_clz(n) instead of precomputing log2[n].
std::vector<std::vector<T>> dp; // dp[j][i] = combine() over [i, i + 2^j).

public:
template<typename It>
SparseTable(It lo, It hi) {
    std::vector<T> a(lo, hi);
    len = static_cast<int>(a.size());
    log2.assign(len + 1, 0);
    for (int i = 2; i <= len; i++) {
        log2[i] = log2[i >> 1] + 1;
    }
    dp.assign(log2[len] + 1, std::vector<T>(len));
    dp[0] = std::move(a);
    for (int j = 1; (1 << j) <= len; j++) {
        for (int i = 0; i + (1 << j) <= len; i++) {
            dp[j][i] = combine(dp[j - 1][i], dp[j - 1][i + (1 << (j - 1))]);
        }
    }
}

int size() const { return len; }

T query(int lo, int hi) const {
    assert(0 <= lo && lo <= hi && hi < len);
    int j = log2[hi - lo + 1];
    return combine(dp[j][lo], dp[j][hi - (1 << j) + 1]);
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    SparseTable<int> t(a.begin(), a.end());
    assert(t.size() == 5);
    assert(t.query(0, 3) == -2);
    assert(t.query(2, 4) == 1);
    assert(t.query(3, 3) == 8);
    return 0;
}

```

2.4.2 Disjoint Sparse Table

Given a static array, precompute a table that answers range queries for any associative operation in constant time. Unlike the ordinary sparse table for range minimum queries, the disjoint sparse table does not require the operation to be idempotent, so it also handles sums, products, and matrix products where overlapping the two query halves would double-count.

The array is divided into blocks whose size doubles at each of the $O(\log n)$ levels. At the level whose block size is 2^{k+1} , every block is split at its center, and the table stores, for each position, the fold of the contiguous run from that position up to (or down to) the center. To answer a query $[l, r]$, find the level at which l and r first fall on opposite sides of a center: this is the position of the highest set bit of $l \oplus r$. At that level, l lies in the block's left half and r in its right half, so the answer is the stored suffix fold ending at the center combined with the stored prefix fold beginning at the center.

The query operation is defined by an associative function `combine(a, b)`. The default code below returns the “min” of the range; for “sum”, `combine(a, b)` should return `a + b`.

- `DisjointSparseTable<T>(lo, hi)` builds the table from two random-access iterators.
- `size()` returns the size of the array.
- `query(lo, hi)` returns `combine()` applied to all indices from `lo` to `hi`, inclusive.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()` and `query()`.

Space Complexity:

- $O(n \log n)$ for storage of the table.
- $O(1)$ auxiliary for `query()`.

```
#include <algorithm>
#include <cassert>
#include <cstdlib>
#include <vector>

template<typename T>
class DisjointSparseTable {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

    int len;
    std::vector<T> a;
    std::vector<std::vector<T>> fold;

public:
    template<typename It>
    DisjointSparseTable(It lo, It hi) : a(lo, hi) {
        len = static_cast<int>(a.size());
        int levels = 1;
        while ((1 << levels) < len) {
            levels++;
        }
        fold.assign(levels, std::vector<T>(len));
        for (int level = 0; level < levels; level++) {
            int range = 1 << (level + 1), half = 1 << level;
            for (int center = half; center < len + half; center += range) {
                int left = center - half, right = std::min(len, center + half);
                int top = std::min(center, len) - 1; // Suffix folds over [i, center).
                fold[level][top] = a[top];
                for (int i = top - 1; i >= left; i--) {
                    fold[level][i] = combine(a[i], fold[level][i + 1]);
                }
                if (center < len) { // Prefix folds over [center, i].
                    fold[level][center] = a[center];
                    for (int i = center + 1; i < right; i++) {
                        fold[level][i] = combine(fold[level][i - 1], a[i]);
                    }
                }
            }
        }
    }

    int size() const { return len; }

    T query(int lo, int hi) const {
        assert(0 <= lo && lo <= hi && hi < len);
        if (lo == hi) {
            return a[lo];
        }
        int level = 31 - __builtin_clz(static_cast<uint32_t>(lo ^ hi));
        return combine(fold[level][lo], fold[level][hi]);
    }
};
```

```
    }
};
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10, -5, 3};
    DisjointSparseTable<int> t(a.begin(), a.end());
    assert(t.size() == 7);
    assert(t.query(0, 6) == -5);
    assert(t.query(1, 3) == -2);
    assert(t.query(4, 4) == 10);
    assert(t.query(2, 4) == 1);
    return 0;
}
```

2.4.3 Square Root Decomposition

Maintain a fixed-size array while supporting point updates and contiguous range aggregate queries. Square root decomposition partitions the array into contiguous blocks of about $\sqrt{n}$ elements, each caching the aggregate of its block. A range query combines the cached aggregates of the whole blocks contained in the range with the individual elements of the partial blocks at either end, while a point update refreshes only the single block containing the changed index.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default definition below assumes a numerical array type, supporting queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at a single updated index. The default definition below supports updates that “set” the chosen array index to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

- `SqrtDecomposition<T>(n, v)` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `SqrtDecomposition<T>(lo, hi)` constructs an array from two random-access iterators, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.

The supported operations are identical to those of the point-update segment tree in this section.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\sqrt{n})$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <vector>
```



```

template<typename T>
class SqrtDecomposition {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d) { return d; }

    int len, blocklen;
    std::vector<T> value, block;

    void init() {
        blocklen = std::max(1, static_cast<int>(sqrt(len)));
        int nblocks = (len + blocklen - 1) / blocklen;
        for (int i = 0; i < nblocks; i++) {
            T blockval = value[i * blocklen];
            int blockhi = std::min(len, (i + 1) * blocklen);
            for (int j = i * blocklen + 1; j < blockhi; j++) {
                blockval = combine(blockval, value[j]);
            }
            block.push_back(blockval);
        }
    }

public:
    explicit SqrtDecomposition(int n, const T &v = T()) : len(n), value(n, v) { init(); }

    template<typename It>
    SqrtDecomposition(It lo, It hi) : len(hi - lo), value(lo, hi) {
        init();
    }

    int size() const { return len; }
    T at(int i) const { return query(i, i); }

    T query(int lo, int hi) const {
        int blocklo = ceil(static_cast<double>(lo) / blocklen), blockhi = (hi + 1) / blocklen - 1;
        if (blocklo > blockhi) {
            T res = value[lo];
            for (int i = lo + 1; i <= hi; i++) {
                res = combine(res, value[i]);
            }
            return res;
        }
        T res = block[blocklo];
        for (int i = blocklo + 1; i <= blockhi; i++) {
            res = combine(res, block[i]);
        }
        for (int i = lo; i < blocklo * blocklen; i++) {
            res = combine(res, value[i]);
        }
        for (int i = (blockhi + 1) * blocklen; i <= hi; i++) {
            res = combine(res, value[i]);
        }
        return res;
    }

    void update(int i, const T &d) {
        value[i] = apply_delta(value[i], d);
        int b = i / blocklen;
        int blockhi = std::min(len, (b + 1) * blocklen);
        block[b] = value[b * blocklen];
        for (int i = b * blocklen + 1; i < blockhi; i++) {
            block[b] = combine(block[b], value[i]);
        }
    }
};

```

Example Usage

```
#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    SqrtDecomposition<int> sd(a.begin(), a.end());
    sd.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < sd.size(); i++) {
        cout << " " << sd.at(i);
    }
    cout << endl;
    assert(sd.query(0, 3) == -2);
    return 0;
}
```

Example Output

Values: 6 -2 4 8 10

2.4.4 Mo's Algorithm

Mo's algorithm answers a batch of range queries offline by reordering them so that the current window [`cur_l`, `cur_r`] can be transformed into the next query's range with as few single-element insertions and deletions as possible. It applies whenever there is no fast direct formula, but extending or shrinking the window by one element updates the answer cheaply, as with the number of distinct values in a range or the sum of squares of element frequencies.

The trick is the query order. Sort queries by the block of their left endpoint, where each block spans about $n/\sqrt{q}$ indices, breaking ties by right endpoint. The left pointer then travels $O(n/\sqrt{q})$ per query, for $O(n\sqrt{q})$ moves in all, while within each block the right pointer sweeps monotonically across the array, contributing $O(n)$ per block over the $\sqrt{q}$ blocks, again $O(n\sqrt{q})$. So the pointers move $O(n\sqrt{q})$ times in total. Sorting the right endpoint in alternating directions between consecutive blocks (a “boustrophedon” order) avoids resetting the right pointer at every block boundary.

- `mos_algorithm(n, queries, add, remove, current)` answers the given inclusive 0-based ranges over an array of size `n`. The caller supplies three callbacks that maintain an arbitrary window state: `add(i)` and `remove(i)` insert or delete index `i` from the current window, and `current()` returns the answer for the window in its present state. The result is a vector of answers in the original query order, with element type matching the return type of `current()`.

Time Complexity: $O(n\sqrt{q})$ calls to `add()` and `remove()` in total, plus $O(q \log q)$ to sort, where n is the array size and q is the number of queries.

Space Complexity: $O(q)$ auxiliary, excluding whatever window state the callbacks maintain.

```
#include <algorithm>
#include <cmath>
#include <numeric>
#include <utility>
#include <vector>

template<typename AddFn, typename RemoveFn, typename CurrentFn>
std::vector<decltype(std::declval<CurrentFn>())> mos_algorithm(
    int n, const std::vector<std::pair<int, int>> &queries, AddFn add, RemoveFn remove,
    CurrentFn current
) {
    int q = static_cast<int>(queries.size());
    int block = std::max(1, static_cast<int>(n / std::max(1.0, std::sqrt(static_cast<double>(q)))));
```

```

std::vector<int> order(q);
std::iota(order.begin(), order.end(), 0);
std::sort(order.begin(), order.end(), [&](int x, int y) {
    int bx = queries[x].first / block, by = queries[y].first / block;
    if (bx != by) {
        return bx < by;
    }
    return (bx & 1) ? queries[x].second > queries[y].second : queries[x].second < queries[y].second;
});
std::vector<decltype(current())> res(q);
int cur_l = 0, cur_r = -1; // Window [cur_l, cur_r] is initially empty.
for (int idx : order) {
    int l = queries[idx].first, r = queries[idx].second;
    while (cur_r < r) add(++cur_r);
    while (cur_l > l) add(--cur_l);
    while (cur_r > r) remove(cur_r--);
    while (cur_l < l) remove(cur_l++);
    res[idx] = current();
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Answer "number of distinct values in [l, r]" for several ranges.
    vector<int> a{1, 1, 2, 1, 3, 2, 3};
    vector<int> freq(4, 0);
    int distinct = 0;
    auto add = [&](int i) {
        if (freq[a[i]]++ == 0) distinct++;
    };
    auto remove = [&](int i) {
        if (--freq[a[i]] == 0) distinct--;
    };
    auto current = [&]() { return distinct; };

    int n = static_cast<int>(a.size());
    vector<pair<int, int>> queries{{0, 6}, {0, 2}, {3, 5}, {1, 1}, {2, 4}};
    vector<int> ans = mos_algorithm(n, queries, add, remove, current);

    assert((ans == vector<int>{3, 2, 3, 1, 3}));
    return 0;
}

```

2.4.5 Segment Tree (Point Update)

Maintain a fixed-size array while supporting point updates and range aggregate queries. A segment tree stores one aggregate value for each recursively split interval, so an update only rebuilds the $O(\log n)$ intervals on the path from one leaf to the root.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at a single updated index. The default definition below supports updates that “set” the chosen array index to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

- `SegTree<T>(n, v)` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `SegTree<T>(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `find_first(lo, hi, pred)` returns the smallest index in `[lo, hi]` matching the search, or `-1` if none, in $O(\log n)$. `pred(v)` takes a node aggregate and must be monotone: if it is false, no element under that node qualifies (e.g. for the default min tree, `pred(v) = (v <= x)` finds the leftmost element `<= x`).
- `find_last(lo, hi, pred)` is the analogous query returning the largest such index.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `size()` if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `0` if the predicate remains true to the beginning.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()`, `update()`, `query()`, `find_first()`, `find_last()`, `max_right()`, and `min_left()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(\log n)$ auxiliary stack space for `update()` and `query()`.
- $O(1)$ auxiliary for `size()`.

```
#include <algorithm>
#include <optional>
#include <vector>

template<typename T>
class SegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d) { return d; }

    int len;
    std::vector<T> value;

    void build(int i, int lo, int hi, const T &v) {
        if (lo == hi) {
            value[i] = v;
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, v);
        build(i * 2 + 2, mid + 1, hi, v);
        value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
    }

    template<typename It>
    void build(int i, int lo, int hi, It arr) {
        if (lo == hi) {
            value[i] = *(arr + lo);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, arr);
        build(i * 2 + 2, mid + 1, hi, arr);
        value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
    }
};
```

```

T query(int i, int lo, int hi, int tgt_lo, int tgt_hi) const {
    if (lo == tgt_lo && hi == tgt_hi) {
        return value[i];
    }
    int mid = lo + (hi - lo) / 2;
    if (tgt_lo <= mid && mid < tgt_hi) {
        return combine(
            query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
            query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
        );
    }
    if (tgt_lo <= mid) {
        return query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid));
    }
    return query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
}

void update(int i, int lo, int hi, int target, const T &d) {
    if (target < lo || target > hi) {
        return;
    }
    if (lo == hi) {
        value[i] = apply_delta(value[i], d);
        return;
    }
    int mid = lo + (hi - lo) / 2;
    update(i * 2 + 1, lo, mid, target, d);
    update(i * 2 + 2, mid + 1, hi, target, d);
    value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
}

template<typename Pred>
int find_first(int i, int lo, int hi, int tgt_lo, int tgt_hi, const Pred &pred) const {
    if (tgt_hi < lo || hi < tgt_lo || !pred(value[i])) {
        return -1;
    }
    if (lo == hi) {
        return lo;
    }
    int mid = lo + (hi - lo) / 2;
    int res = find_first(i * 2 + 1, lo, mid, tgt_lo, tgt_hi, pred);
    return res != -1 ? res : find_first(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi, pred);
}

template<typename Pred>
int find_last(int i, int lo, int hi, int tgt_lo, int tgt_hi, const Pred &pred) const {
    if (tgt_hi < lo || hi < tgt_lo || !pred(value[i])) {
        return -1;
    }
    if (lo == hi) {
        return lo;
    }
    int mid = lo + (hi - lo) / 2;
    int res = find_last(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi, pred);
    return res != -1 ? res : find_last(i * 2 + 1, lo, mid, tgt_lo, tgt_hi, pred);
}

template<typename Pred>
int max_right(int i, int lo, int hi, int tgt_lo, const Pred &pred, std::optional<T> &acc) const {
    if (hi < tgt_lo) {
        return -1;
    }
    if (tgt_lo <= lo) {
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
        }
        return -1;
    }

```

```

    }
    if (lo == hi) {
        return lo;
    }
}
int mid = lo + (hi - lo) / 2;
int res = max_right(i * 2 + 1, lo, mid, tgt_lo, pred, acc);
return res != -1 ? res : max_right(i * 2 + 2, mid + 1, hi, tgt_lo, pred, acc);
}

template<typename Pred>
int min_left(int i, int lo, int hi, int tgt_hi, const Pred &pred, std::optional<T> &acc) const {
    if (tgt_hi <= lo) {
        return -1;
    }
    if (hi < tgt_hi) {
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
    }
    if (lo == hi) {
        return lo + 1;
    }
}
int mid = lo + (hi - lo) / 2;
int res = min_left(i * 2 + 2, mid + 1, hi, tgt_hi, pred, acc);
return res != -1 ? res : min_left(i * 2 + 1, lo, mid, tgt_hi, pred, acc);
}

public:
explicit SegTree(int n, const T &v = T()) : len(n), value(4 * len) {
    if (len > 0) {
        build(0, 0, len - 1, v);
    }
}

template<typename It>
SegTree(It lo, It hi) : len(hi - lo), value(4 * len) {
    if (len > 0) {
        build(0, 0, len - 1, lo);
    }
}

int size() const { return len; }
T at(int i) const { return query(i, i); }
T query(int lo, int hi) const { return query(0, 0, len - 1, lo, hi); }
void update(int i, const T &d) { update(0, 0, len - 1, i, d); }

template<typename Pred>
int find_first(int lo, int hi, const Pred &pred) const {
    return find_first(0, 0, len - 1, lo, hi, pred);
}

template<typename Pred>
int find_last(int lo, int hi, const Pred &pred) const {
    return find_last(0, 0, len - 1, lo, hi, pred);
}

template<typename Pred>
int max_right(int lo, const Pred &pred) const {
    std::optional<T> acc;
    int res = max_right(0, 0, len - 1, lo, pred, acc);
    return res == -1 ? len : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) const {

```

```

    std::optional<T> acc;
    int res = min_left(0, 0, len - 1, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    SegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    cout << "The minimum value in the range [0, 3] is " << t.query(0, 3) << "." << endl;
    assert(t.query(0, 3) == -2);

    // Segment-tree descent: leftmost/rightmost index in a range with value <= a threshold.
    auto at_most_4 = [](int v) { return v <= 4; };
    assert(t.find_first(0, 4, at_most_4) == 1); // value -2 at index 1
    assert(t.find_last(0, 4, at_most_4) == 2); // value 4 at index 2
    assert(t.find_first(0, 4, [](int v) { return v <= -5; }) == -1); // no value <= -5

    // Boundary search by accumulated aggregate: stop before including the -2 at index 1.
    assert(t.max_right(0, [](int mn) { return mn > -2; }) == 1);
    assert(t.min_left(5, [](int mn) { return mn > -2; }) == 2);
    return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
The minimum value in the range [0, 3] is -2.

```

2.4.6 Segment Tree (Range Update)

Maintain a fixed-size array while supporting range updates and range aggregate queries. A lazy segment tree stores one aggregate value for each recursively split interval, plus pending updates that are pushed to children only when needed.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `LazySegTree<T>(n, v)` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.

- `LazySegTree<T>(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `update(lo, hi, d)` modifies the value at each array index from `lo` to `hi`, inclusive, by applying the delta `d` to each value.
- `find_first(lo, hi, pred)` returns the smallest index in `[lo, hi]` matching the search, or `-1` if none, in $O(\log n)$. `pred(v)` takes a node aggregate and must be monotone: if it is false, no element under that node qualifies (e.g. for the default min tree, `pred(v) = (v <= x)` finds the leftmost element `<= x`).
- `find_last(lo, hi, pred)` is the analogous query returning the largest such index.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `size()` if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `0` if the predicate remains true to the beginning.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()`, `update()`, `query()`, `find_first()`, `find_last()`, `max_right()`, and `min_left()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(\log n)$ auxiliary stack space for `update()` and `query()`.
- $O(1)$ auxiliary for `size()`.

```
#include <algorithm>
#include <cstdint>
#include <optional>
#include <vector>

template<typename T>
class LazySegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    int len;
    std::vector<T> value, delta;
    std::vector<bool> pending;

    void build(int i, int lo, int hi, const T &v) {
        if (lo == hi) {
            value[i] = v;
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(i * 2 + 1, lo, mid, v);
        build(i * 2 + 2, mid + 1, hi, v);
        value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
    }

    template<typename It>
    void build(int i, int lo, int hi, It arr) {
        if (lo == hi) {
            value[i] = *(arr + lo);
            return;
        }
        int mid = lo + (hi - lo) / 2;
```



```

    build(i * 2 + 1, lo, mid, arr);
    build(i * 2 + 2, mid + 1, hi, arr);
    value[i] = combine(value[i * 2 + 1], value[i * 2 + 2]);
}

void push_delta(int i, int lo, int hi) {
    if (pending[i]) {
        value[i] = apply_delta(value[i], delta[i], hi - lo + 1);
        if (lo != hi) {
            int l = 2 * i + 1, r = 2 * i + 2;
            delta[l] = pending[l] ? compose_deltas(delta[l], delta[i]) : delta[i];
            delta[r] = pending[r] ? compose_deltas(delta[r], delta[i]) : delta[i];
            pending[l] = pending[r] = true;
        }
        pending[i] = false;
    }
}

T query(int i, int lo, int hi, int tgt_lo, int tgt_hi) {
    push_delta(i, lo, hi);
    if (lo == tgt_lo && hi == tgt_hi) {
        return value[i];
    }
    int mid = lo + (hi - lo) / 2;
    if (tgt_lo <= mid && mid < tgt_hi) {
        return combine(
            query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
            query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
        );
    }
    if (tgt_lo <= mid) {
        return query(i * 2 + 1, lo, mid, tgt_lo, std::min(tgt_hi, mid));
    }
    return query(i * 2 + 2, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
}

void update(int i, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
    push_delta(i, lo, hi);
    if (hi < tgt_lo || lo > tgt_hi) {
        return;
    }
    if (tgt_lo <= lo && hi <= tgt_hi) {
        delta[i] = d;
        pending[i] = true;
        push_delta(i, lo, hi);
        return;
    }
    update(2 * i + 1, lo, lo + (hi - lo) / 2, tgt_lo, tgt_hi, d);
    update(2 * i + 2, lo + (hi - lo) / 2 + 1, hi, tgt_lo, tgt_hi, d);
    value[i] = combine(value[2 * i + 1], value[2 * i + 2]);
}

template<typename Pred>
int find_first(int i, int lo, int hi, int tgt_lo, int tgt_hi, const Pred &pred) {
    push_delta(i, lo, hi);
    if (tgt_hi < lo || hi < tgt_lo || !pred(value[i])) {
        return -1;
    }
    if (lo == hi) {
        return lo;
    }
    int mid = lo + (hi - lo) / 2;
    int res = find_first(i * 2 + 1, lo, mid, tgt_lo, tgt_hi, pred);
    return res != -1 ? res : find_first(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi, pred);
}

template<typename Pred>
int find_last(int i, int lo, int hi, int tgt_lo, int tgt_hi, const Pred &pred) {

```

```

    push_delta(i, lo, hi);
    if (tgt_hi < lo || hi < tgt_lo || !pred(value[i])) {
        return -1;
    }
    if (lo == hi) {
        return lo;
    }
    int mid = lo + (hi - lo) / 2;
    int res = find_last(i * 2 + 2, mid + 1, hi, tgt_lo, tgt_hi, pred);
    return res != -1 ? res : find_last(i * 2 + 1, lo, mid, tgt_lo, tgt_hi, pred);
}

template<typename Pred>
int max_right(int i, int lo, int hi, int tgt_lo, const Pred &pred, std::optional<T> &acc) {
    push_delta(i, lo, hi);
    if (hi < tgt_lo) {
        return -1;
    }
    if (tgt_lo <= lo) {
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = max_right(i * 2 + 1, lo, mid, tgt_lo, pred, acc);
    return res != -1 ? res : max_right(i * 2 + 2, mid + 1, hi, tgt_lo, pred, acc);
}

template<typename Pred>
int min_left(int i, int lo, int hi, int tgt_hi, const Pred &pred, std::optional<T> &acc) {
    push_delta(i, lo, hi);
    if (tgt_hi <= lo) {
        return -1;
    }
    if (hi < tgt_hi) {
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo + 1;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = min_left(i * 2 + 2, mid + 1, hi, tgt_hi, pred, acc);
    return res != -1 ? res : min_left(i * 2 + 1, lo, mid, tgt_hi, pred, acc);
}

public:
    explicit LazySegTree(int n, const T &v = T())
        : len(n), value(4 * len), delta(4 * len), pending(4 * len, false) {
        if (len > 0) {
            build(0, 0, len - 1, v);
        }
    }

    template<typename It>
    LazySegTree(It lo, It hi)
        : len(hi - lo), value(4 * len), delta(4 * len), pending(4 * len, false) {
        if (len > 0) {
            build(0, 0, len - 1, lo);
        }
    }

```

```

}

int size() const { return len; }
T at(int i) { return query(i, i); }
T query(int lo, int hi) { return query(0, 0, len - 1, lo, hi); }
void update(int i, const T &d) { update(0, 0, len - 1, i, i, d); }
void update(int lo, int hi, const T &d) { update(0, 0, len - 1, lo, hi, d); }

template<typename Pred>
int find_first(int lo, int hi, const Pred &pred) {
    return find_first(0, 0, len - 1, lo, hi, pred);
}

template<typename Pred>
int find_last(int lo, int hi, const Pred &pred) {
    return find_last(0, 0, len - 1, lo, hi, pred);
}

template<typename Pred>
int max_right(int lo, const Pred &pred) {
    std::optional<T> acc;
    int res = max_right(0, 0, len - 1, lo, pred, acc);
    return res == -1 ? len : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) {
    std::optional<T> acc;
    int res = min_left(0, 0, len - 1, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    LazySegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == 1);

    // Segment-tree descent works through lazy updates too; values are now {5, 5, 5, 1, 5}.
    auto at_most_1 = [](int v) { return v <= 1; };
    assert(t.find_first(0, 4, at_most_1) == 3); // only index 3 (value 1) qualifies
    assert(t.find_last(0, 4, at_most_1) == 3);
    assert(t.find_first(0, 2, at_most_1) == -1); // no value <= 1 in [0, 2]
    assert(t.max_right(0, [](int mn) { return mn > 1; }) == 3);
    assert(t.min_left(5, [](int mn) { return mn > 1; }) == 4);
}

```

```

return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
Values: 5 5 5 1 5

```

2.4.7 Iterative Segment Tree (Point Update)

Maintain a fixed-size array while supporting point updates and range aggregate queries using an iterative segment tree. The leaves are stored starting at a power-of-two offset in a flat array and every internal node stores the aggregate of its two children, so point updates rebuild ancestors by walking upward instead of recursing. Unlike the recursive segment tree, this layout avoids recursive stack usage for ordinary queries and updates. It is especially convenient for fast point-update segment trees; lazy propagation is usually clearer in the recursive version.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at a single updated index. The default definition below supports updates that “set” the chosen array index to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

- `IterativeSegTree<T>(n, v)` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `IterativeSegTree<T>(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `find_first(lo, hi, pred)` returns the smallest index in `[lo, hi]` matching the search, or `-1` if none. `pred(v)` takes a node aggregate and must be monotone: if it is false, no element under that node qualifies.
- `find_last(lo, hi, pred)` is the analogous query returning the largest such index.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `size()` if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `0` if the predicate remains true to the beginning.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()` and `at()`.
- $O(\log n)$ per call to `update()`, `query()`, `find_first()`, `find_last()`, `max_right()`, and `min_left()`.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary space for `query()` and `update()`, and $O(\log n)$ for boundary searches.

```

#include <algorithm>
#include <optional>
#include <vector>

template<typename T>
class IterativeSegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

```

```

static T apply_delta(const T &v, const T &d) { return d; }

int len, base;
std::vector<T> value;

template<typename Pred>
int find_first(int i, int lo, int hi, int qlo, int qhi, const Pred &pred) const {
    if (hi <= qlo || qhi <= lo || len <= lo) {
        return -1;
    }
    if (qlo <= lo && hi <= qhi && hi <= len && !pred(value[i])) {
        return -1;
    }
    if (hi - lo == 1) {
        return lo;
    }
    int mid = (lo + hi) / 2;
    int res = find_first(i << 1, lo, mid, qlo, qhi, pred);
    return res != -1 ? res : find_first(i << 1 | 1, mid, hi, qlo, qhi, pred);
}

template<typename Pred>
int find_last(int i, int lo, int hi, int qlo, int qhi, const Pred &pred) const {
    if (hi <= qlo || qhi <= lo || len <= lo) {
        return -1;
    }
    if (qlo <= lo && hi <= qhi && hi <= len && !pred(value[i])) {
        return -1;
    }
    if (hi - lo == 1) {
        return lo;
    }
    int mid = (lo + hi) / 2;
    int res = find_last(i << 1 | 1, mid, hi, qlo, qhi, pred);
    return res != -1 ? res : find_last(i << 1, lo, mid, qlo, qhi, pred);
}

template<typename Pred>
int max_right(int i, int lo, int hi, int qlo, const Pred &pred, std::optional<T> &acc) const {
    if (hi <= qlo || len <= lo) {
        return -1;
    }
    if (qlo <= lo && hi <= len) {
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (hi - lo == 1) {
            return lo;
        }
    }
    int mid = (lo + hi) / 2;
    int res = max_right(i << 1, lo, mid, qlo, pred, acc);
    return res != -1 ? res : max_right(i << 1 | 1, mid, hi, qlo, pred, acc);
}

template<typename Pred>
int min_left(int i, int lo, int hi, int qhi, const Pred &pred, std::optional<T> &acc) const {
    if (qhi <= lo || len <= lo) {
        return -1;
    }
    if (hi <= qhi && hi <= len) {
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
    }
}

```

```

    if (hi - lo == 1) {
        return hi;
    }
}
int mid = (lo + hi) / 2;
int res = min_left(i << 1 | 1, mid, hi, qhi, pred, acc);
return res != -1 ? res : min_left(i << 1, lo, mid, qhi, pred, acc);
}

public:
explicit IterativeSegTree(int n, const T &v = T()) : len(n), base(1) {
    while (base < len) {
        base <<= 1;
    }
    value.assign(2 * base, T());
    for (int i = 0; i < len; i++) {
        value[base + i] = v;
    }
    for (int i = base - 1; i > 0; i--) {
        value[i] = combine(value[i << 1], value[i << 1 | 1]);
    }
}

template<typename It>
IterativeSegTree(It lo, It hi) : len(hi - lo), base(1) {
    while (base < len) {
        base <<= 1;
    }
    value.assign(2 * base, T());
    for (int i = 0; i < len; i++) {
        value[base + i] = *(lo + i);
    }
    for (int i = base - 1; i > 0; i--) {
        value[i] = combine(value[i << 1], value[i << 1 | 1]);
    }
}

int size() const { return len; }
T at(int i) const { return value[base + i]; }

T query(int lo, int hi) const {
    std::optional<T> left, right;
    for (lo += base, hi += base + 1; lo < hi; lo >>= 1, hi >>= 1) {
        if (lo & 1) {
            left = left ? combine(*left, value[lo++]) : value[lo++];
        }
        if (hi & 1) {
            right = right ? combine(value[--hi], *right) : value[--hi];
        }
    }
    if (!left) {
        return *right;
    }
    if (!right) {
        return *left;
    }
    return combine(*left, *right);
}

void update(int i, const T &d) {
    i += base;
    value[i] = apply_delta(value[i], d);
    for (i >>= 1; i > 0; i >>= 1) {
        value[i] = combine(value[i << 1], value[i << 1 | 1]);
    }
}

template<typename Pred>

```

```

int find_first(int lo, int hi, const Pred &pred) const {
    return find_first(1, 0, base, lo, hi + 1, pred);
}

template<typename Pred>
int find_last(int lo, int hi, const Pred &pred) const {
    return find_last(1, 0, base, lo, hi + 1, pred);
}

template<typename Pred>
int max_right(int lo, const Pred &pred) const {
    std::optional<T> acc;
    int res = max_right(1, 0, base, lo, pred, acc);
    return res == -1 ? len : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) const {
    std::optional<T> acc;
    int res = min_left(1, 0, base, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    IterativeSegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    cout << "The minimum value in the range [0, 3] is " << t.query(0, 3) << "." << endl;
    assert(t.query(0, 3) == -2);

    auto at_most_4 = [](int v) { return v <= 4; };
    assert(t.find_first(0, 4, at_most_4) == 1);
    assert(t.find_last(0, 4, at_most_4) == 2);
    assert(t.find_first(0, 4, [](int v) { return v <= -5; }) == -1);

    assert(t.max_right(0, [](int mn) { return mn > -2; }) == 1);
    assert(t.min_left(5, [](int mn) { return mn > -2; }) == 2);
    return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
The minimum value in the range [0, 3] is -2.

```

2.4.8 Iterative Segment Tree (Range Update)

Maintain a fixed-size array while supporting range updates and range aggregate queries using a bottom-up lazy segment tree. Like the point-update iterative tree, leaves are stored starting at a power-of-two offset in a flat array. Each internal node stores an aggregate value, plus an optional pending lazy delta that is pushed down only along paths that need to inspect children. Compared with the recursive lazy tree, this version keeps ordinary

range updates and queries in flat loops, which can reduce constant factors. The recursive version is usually easier to adapt when the lazy operation or search logic becomes unusual.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `IterativeLazySegTree<T>(n, v)` constructs an array of size `n` with indices `[0, n)`, and all values initialized to `v`.
- `IterativeLazySegTree<T>(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value at index `i` by applying the delta `d`.
- `update(lo, hi, d)` modifies the value at each array index from `lo` to `hi`, inclusive, by applying the delta `d` to each value.
- `find_first(lo, hi, pred)` returns the smallest index in `[lo, hi]` matching the search, or `-1` if none. `pred(v)` takes a node aggregate and must be monotone: if it is false, no element under that node qualifies.
- `find_last(lo, hi, pred)` is the analogous query returning the largest such index.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `size()` if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range `[lo, hi)` satisfies `pred`, or `0` if the predicate remains true to the beginning.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()`, `update()`, `query()`, `find_first()`, `find_last()`, `max_right()`, and `min_left()`.

Space Complexity:

- $O(n)$ for storage of the array elements and lazy deltas.
- $O(1)$ auxiliary space for `query()` and `update()`, and $O(\log n)$ for boundary searches.

```
#include <algorithm>
#include <cstdint>
#include <optional>
#include <vector>

template<typename T>
class IterativeLazySegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    int len, base, height;
    std::vector<T> value, delta;
    std::vector<int64_t> seg_len;
    std::vector<bool> pending;
```



```

void init_storage() {
    base = 1;
    height = 0;
    while (base < len) {
        base <<= 1;
        height++;
    }
    value.assign(2 * base, T());
    delta.assign(2 * base, T());
    pending.assign(2 * base, false);
    seg_len.assign(2 * base, 1);
    for (int i = base - 1; i > 0; i--) {
        seg_len[i] = seg_len[i << 1] + seg_len[i << 1 | 1];
    }
}

void pull(int i) { value[i] = combine(value[i << 1], value[i << 1 | 1]); }

void apply(int i, const T &d) {
    value[i] = apply_delta(value[i], d, seg_len[i]);
    if (i < base) {
        delta[i] = pending[i] ? compose_deltas(delta[i], d) : d;
        pending[i] = true;
    }
}

void push(int i) {
    if (pending[i]) {
        apply(i << 1, delta[i]);
        apply(i << 1 | 1, delta[i]);
        pending[i] = false;
    }
}

void push_path(int i) {
    for (int h = height; h > 0; h--) {
        push(i >> h);
    }
}

void pull_after_update(int l, int r) {
    for (int h = 1; h <= height; h++) {
        if (((l >> h) << h) != l) {
            pull(l >> h);
        }
        if (((r >> h) << h) != r) {
            pull((r - 1) >> h);
        }
    }
}

template<typename Pred>
int find_first(int i, int lo, int hi, int qlo, int qhi, const Pred &pred) {
    if (hi <= qlo || qhi <= lo || len <= lo) {
        return -1;
    }
    if (qlo <= lo && hi <= qhi && hi <= len && !pred(value[i])) {
        return -1;
    }
    if (hi - lo == 1) {
        return lo;
    }
    push(i);
    int mid = (lo + hi) / 2;
    int res = find_first(i << 1, lo, mid, qlo, qhi, pred);
    return res != -1 ? res : find_first(i << 1 | 1, mid, hi, qlo, qhi, pred);
}

```

```

template<typename Pred>
int find_last(int i, int lo, int hi, int qlo, int qhi, const Pred &pred) {
    if (hi <= qlo || qhi <= lo || len <= lo) {
        return -1;
    }
    if (qlo <= lo && hi <= qhi && hi <= len && !pred(value[i])) {
        return -1;
    }
    if (hi - lo == 1) {
        return lo;
    }
    push(i);
    int mid = (lo + hi) / 2;
    int res = find_last(i << 1 | 1, mid, hi, qlo, qhi, pred);
    return res != -1 ? res : find_last(i << 1, lo, mid, qlo, qhi, pred);
}

template<typename Pred>
int max_right(int i, int lo, int hi, int qlo, const Pred &pred, std::optional<T> &acc) {
    if (hi <= qlo || len <= lo) {
        return -1;
    }
    if (qlo <= lo && hi <= len) {
        T next = acc ? combine(*acc, value[i]) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (hi - lo == 1) {
            return lo;
        }
    }
    push(i);
    int mid = (lo + hi) / 2;
    int res = max_right(i << 1, lo, mid, qlo, pred, acc);
    return res != -1 ? res : max_right(i << 1 | 1, mid, hi, qlo, pred, acc);
}

template<typename Pred>
int min_left(int i, int lo, int hi, int qhi, const Pred &pred, std::optional<T> &acc) {
    if (qhi <= lo || len <= lo) {
        return -1;
    }
    if (hi <= qhi && hi <= len) {
        T next = acc ? combine(value[i], *acc) : value[i];
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (hi - lo == 1) {
            return hi;
        }
    }
    push(i);
    int mid = (lo + hi) / 2;
    int res = min_left(i << 1 | 1, mid, hi, qhi, pred, acc);
    return res != -1 ? res : min_left(i << 1, lo, mid, qhi, pred, acc);
}

public:
explicit IterativeLazySegTree(int n, const T &v = T()) : len(n) {
    init_storage();
    for (int i = 0; i < len; i++) {
        value[base + i] = v;
    }
    for (int i = base - 1; i > 0; i--) {
        pull(i);
    }
}

```

```

}

template<typename It>
IterativeLazySegTree(It lo, It hi) : len(hi - lo) {
    init_storage();
    for (int i = 0; i < len; i++) {
        value[base + i] = *(lo + i);
    }
    for (int i = base - 1; i > 0; i--) {
        pull(i);
    }
}

int size() const { return len; }

T at(int i) {
    i += base;
    push_path(i);
    return value[i];
}

T query(int lo, int hi) {
    int l = lo + base, r = hi + 1 + base;
    push_path(l);
    push_path(r - 1);
    std::optional<T> left, right;
    for (; l < r; l >>= 1, r >>= 1) {
        if (l & 1) {
            left = left ? combine(*left, value[l++]) : value[l++];
        }
        if (r & 1) {
            right = right ? combine(value[--r], *right) : value[--r];
        }
    }
    return !left ? *right : (!right ? *left : combine(*left, *right));
}

void update(int lo, int hi, const T &d) {
    int l = lo + base, r = hi + 1 + base;
    push_path(l);
    push_path(r - 1);
    int l0 = l, r0 = r;
    for (; l < r; l >>= 1, r >>= 1) {
        if (l & 1) {
            apply(l++, d);
        }
        if (r & 1) {
            apply(--r, d);
        }
    }
    pull_after_update(l0, r0);
}

void update(int i, const T &d) { update(i, i, d); }

template<typename Pred>
int find_first(int lo, int hi, const Pred &pred) {
    return find_first(1, 0, base, lo, hi + 1, pred);
}

template<typename Pred>
int find_last(int lo, int hi, const Pred &pred) {
    return find_last(1, 0, base, lo, hi + 1, pred);
}

template<typename Pred>
int max_right(int lo, const Pred &pred) {
    std::optional<T> acc;

```

```

    int res = max_right(1, 0, base, lo, pred, acc);
    return res == -1 ? len : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) {
    std::optional<T> acc;
    int res = min_left(1, 0, base, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<int> a{6, -2, 1, 8, 10};
    IterativeLazySegTree<int> t(a.begin(), a.end());
    t.update(2, 4);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == 1);

    // Segment-tree descent works through lazy updates too; values are now {5, 5, 5, 1, 5}.
    auto at_most_1 = [](int v) { return v <= 1; };
    assert(t.find_first(0, 4, at_most_1) == 3); // only index 3 (value 1) qualifies
    assert(t.find_last(0, 4, at_most_1) == 3);
    assert(t.find_first(0, 2, at_most_1) == -1); // no value <= 1 in [0, 2]
    assert(t.max_right(0, [](int mn) { return mn > 1; }) == 3);
    assert(t.min_left(5, [](int mn) { return mn > 1; }) == 4);
    return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
Values: 5 5 5 1 5

```

2.4.9 Segment Tree Beats

Maintain a fixed-size array under the range “chmin” update $a[i] = \min(a[i], t)$ for all i in a range, while answering range-sum and range-maximum queries. An ordinary lazy segment tree cannot represent a clamp like this with a single composable tag, because clamping affects only the entries that currently exceed t . Segment tree beats (the Ji Driver tree) resolves this by storing, for each node, the strict maximum, the second-largest distinct value, and a count of entries equal to the maximum. A clamp then has three cases at each node.

- If $t \geq \text{max1}$, the clamp changes nothing and the recursion stops (the prune case).

- If $\text{max2} < \text{t} < \text{max1}$, every entry equal to max1 drops to t and all others are untouched, so the node updates in $O(1)$: the sum falls by $(\text{max1} - \text{t}) * \text{count}$ and max1 becomes t (the break, or tag, case). This lazy tag pushes down to a child only when the child's maximum is larger.
- Otherwise ($\text{t} \leq \text{max2}$) the node is too coarse to update directly, so the clamp recurses into both children and re-pulls.

The break condition is what makes this efficient: a potential-function argument on the number of distinct values along root-to-leaf paths shows the total work over any sequence of clamps is amortized $O(\log^2 n)$ each, even though a single clamp can momentarily descend deep. The same scheme extends to range “chmax” by tracking the symmetric minimum trio, and to mixed clamp-and-add workloads by carrying an additional add tag; only the chmin / sum / max core is shown here. Unlike the generic lazy segment tree, this structure cannot be reduced to a pluggable `combine` and `apply`, because a clamp acts on only the maximal entries of a node rather than uniformly on all of them, so each operation set must be hand-written and its amortized bound argued separately.

- `SegTreeBeats<T>(n, v)` constructs an array of size `n`, indices `[0, n)`, all equal to `v`.
- `SegTreeBeats<T>(lo, hi)` constructs an array from two random-access iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `chmin(lo, hi, t)` replaces `a[i]` with `min(a[i], t)` for every `i` in `[lo, hi]`, inclusive.
- `query_sum(lo, hi)` returns the sum of `a[i]` over `i` in `[lo, hi]`, inclusive.
- `query_max(lo, hi)` returns the maximum of `a[i]` over `i` in `[lo, hi]`, inclusive.
- `at(i)` returns the value at index `i`.

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log^2 n)$ amortized per call to `chmin()`.
- $O(\log n)$ per call to `query_sum()`, `query_max()`, and `at()`.

Space Complexity:

- $O(n)$ for storage of the tree.
- $O(\log n)$ auxiliary stack space per `chmin()` and query call.

```
#include <algorithm>
#include <cstdint>
#include <limits>
#include <vector>

template<typename T>
class SegTreeBeats {
    static constexpr T NEG_INF = std::numeric_limits<T>::lowest();

    int len;
    std::vector<T> max1, max2, sum;
    std::vector<int> cnt; // Number of entries in the node equal to max1.

    void pull(int i) {
        int l = 2 * i + 1, r = 2 * i + 2;
        sum[i] = sum[l] + sum[r];
        if (max1[l] == max1[r]) {
            max1[i] = max1[l];
            cnt[i] = cnt[l] + cnt[r];
            max2[i] = std::max(max2[l], max2[r]);
        } else if (max1[l] > max1[r]) {
            max1[i] = max1[l];
            cnt[i] = cnt[l];
            max2[i] = std::max(max2[l], max1[r]);
        } else {
            max1[i] = max1[r];
            cnt[i] = cnt[r];
            max2[i] = std::max(max1[l], max2[r]);
        }
    }
};
```

```

    }
}

// Lower the node's maximum to t. Valid only when max2 < t < max1, so exactly the cnt entries
// equal to max1 change and the second maximum is left untouched.
void apply_chmin(int i, T t) {
    sum[i] -= (max1[i] - t) * static_cast<T>(cnt[i]);
    max1[i] = t;
}

void push(int i) {
    for (int c : {2 * i + 1, 2 * i + 2}) {
        if (max1[c] > max1[i]) {
            apply_chmin(c, max1[i]);
        }
    }
}

template<typename Gen>
void build(int i, int lo, int hi, const Gen &gen) {
    if (lo == hi) {
        max1[i] = sum[i] = gen(lo);
        max2[i] = NEG_INF;
        cnt[i] = 1;
        return;
    }
    int mid = lo + (hi - lo) / 2;
    build(i * 2 + 1, lo, mid, gen);
    build(i * 2 + 2, mid + 1, hi, gen);
    pull(i);
}

void chmin(int i, int lo, int hi, int tlo, int thi, T t) {
    if (thi < lo || hi < tlo || max1[i] <= t) {
        return; // Prune: outside the range, or every entry is already at most t.
    }
    if (tlo <= lo && hi <= thi && max2[i] < t) {
        apply_chmin(i, t); // Break: only the maximal entries are affected.
        return;
    }
    push(i);
    int mid = lo + (hi - lo) / 2;
    chmin(i * 2 + 1, lo, mid, tlo, thi, t);
    chmin(i * 2 + 2, mid + 1, hi, tlo, thi, t);
    pull(i);
}

T query_sum(int i, int lo, int hi, int tlo, int thi) {
    if (thi < lo || hi < tlo) {
        return 0;
    }
    if (tlo <= lo && hi <= thi) {
        return sum[i];
    }
    push(i);
    int mid = lo + (hi - lo) / 2;
    return query_sum(i * 2 + 1, lo, mid, tlo, thi) + query_sum(i * 2 + 2, mid + 1, hi, tlo, thi);
}

T query_max(int i, int lo, int hi, int tlo, int thi) {
    if (thi < lo || hi < tlo) {
        return NEG_INF;
    }
    if (tlo <= lo && hi <= thi) {
        return max1[i];
    }
    push(i);
    int mid = lo + (hi - lo) / 2;

```

```

    return std::max(
        query_max(i * 2 + 1, lo, mid, tlo, thi), query_max(i * 2 + 2, mid + 1, hi, tlo, thi)
    );
}

public:
explicit SegTreeBeats(int n, const T &v = T())
    : len(n), max1(4 * n), max2(4 * n), sum(4 * n), cnt(4 * n) {
    if (len > 0) {
        build(0, 0, len - 1, [&](int) { return v; });
    }
}

template<typename It>
SegTreeBeats(It lo, It hi)
    : len(static_cast<int>(hi - lo)), max1(4 * len), max2(4 * len), sum(4 * len), cnt(4 * len) {
    if (len > 0) {
        build(0, 0, len - 1, [&](int i) { return *(lo + i); });
    }
}

int size() const { return len; }
void chmin(int lo, int hi, const T &t) { chmin(0, 0, len - 1, lo, hi, t); }
T query_sum(int lo, int hi) { return query_sum(0, 0, len - 1, lo, hi); }
T query_max(int lo, int hi) { return query_max(0, 0, len - 1, lo, hi); }
T at(int i) { return query_sum(0, 0, len - 1, i, i); }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int64_t> a{5, 4, 3, 2, 1};
    SegTreeBeats<int64_t> t(a.begin(), a.end());
    assert(t.query_sum(0, 4) == 15);
    assert(t.query_max(0, 4) == 5);

    t.chmin(0, 2, 3); // {3, 3, 3, 2, 1}: only the 5 and 4 are clamped down to 3.
    assert(t.at(0) == 3);
    assert(t.at(1) == 3);
    assert(t.query_sum(0, 4) == 12);
    assert(t.query_max(0, 4) == 3);

    t.chmin(0, 4, 2); // {2, 2, 2, 2, 1}.
    assert(t.query_sum(0, 4) == 9);
    assert(t.query_max(0, 3) == 2);

    // Cross-check against a brute-force array over random clamps and queries.
    vector<int64_t> b{7, 1, 4, 9, 2, 6, 5, 8, 3, 0};
    SegTreeBeats<int64_t> s(b.begin(), b.end());
    int clamps[][3] = {{0, 9, 6}, {2, 5, 3}, {4, 8, 7}, {0, 4, 2}, {3, 9, 5}};
    for (auto &c : clamps) {
        for (int i = c[0]; i <= c[1]; i++) {
            b[i] = min(b[i], static_cast<int64_t>(c[2]));
        }
        s.chmin(c[0], c[1], c[2]);
        for (int lo = 0; lo < 10; lo++) {
            int64_t want_sum = 0, want_max = INT64_MIN;
            for (int hi = lo; hi < 10; hi++) {
                want_sum += b[hi];
                want_max = max(want_max, b[hi]);
                assert(s.query_sum(lo, hi) == want_sum);
                assert(s.query_max(lo, hi) == want_max);
            }
        }
    }
}

```

```

    }
}
return 0;
}

```

2.4.10 Sparse Segment Tree

A sparse segment tree (also commonly called a dynamic or implicit segment tree) maintains an array over a large index range while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique. This implementation uses lazy initialization of nodes to conserve memory: only the nodes covering touched indices are ever allocated, so indices up to N are supported without preallocating the whole tree.

The query operation is defined by an associative aggregate function `combine(a, b)`. Since untouched nodes are implicit, `repeat_value(v, len)` must return the aggregate summary of `len` copies of the initial value `v`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. For range-sum queries, `combine(a, b)` should return `a + b` and `repeat_value(v, len)` should return `v * len`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `SparseSegTree<T, N>(v)` constructs an array over indices $[0, N]$, with every value implicitly initialized to `v`. Nodes are allocated lazily as indices are touched.
- `at(i)` returns the value at index `i`, where `i` is between 0 and `N`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive. If `lo == hi`, then the single specified value is returned.
- `update(i, d)` assigns the value `v` at index `i` to `apply_delta(v, d)`.
- `update(lo, hi, d)` modifies the value at each array index from `lo` to `hi`, inclusive, by applying the delta `d` to each value.
- `find_first(lo, hi, pred)` returns the smallest index in $[lo, hi]$ matching the search, or -1 if none, in $O(\log N)$. `pred(v)` takes a node aggregate and must be monotone: if it is false, no element under that node qualifies (untouched ranges use the implicit `repeat_value(init, len)`). For the default min tree, `pred(v) = (v <= x)` finds the leftmost element `<= x`.
- `find_last(lo, hi, pred)` is the analogous query returning the largest such index.
- `max_right(lo, pred)` returns the largest boundary `hi` such that the aggregate over the half-open range $[lo, hi)$ satisfies `pred`, or `N + 1` if the predicate remains true to the end.
- `min_left(hi, pred)` returns the smallest boundary `lo` such that the aggregate over the half-open range $[lo, hi)$ satisfies `pred`, or 0 if the predicate remains true to the beginning.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\log N)$ per call to `at()`, `update()`, `query()`, `find_first()`, `find_last()`, `max_right()`, and `min_left()`.

Space Complexity:

- $O(k \log(N))$ for storage after k index updates.
- $O(\log N)$ auxiliary stack space for `update()` and `query()`.

```

#include <algorithm>
#include <cstdint>
#include <cstdint>
#include <optional>

```



```

template<typename T, int N = 1000000000>
class SparseSegTree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T repeat_value(const T &v, int64_t len) { return v; }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    struct Node {
        T value, delta;
        bool pending;
        Node *left, *right;

        explicit Node(const T &v) : value(v), pending(false), left(nullptr), right(nullptr) {}
    } *root;

    T init;

    void update_delta(Node *&n, const T &d, int64_t len) {
        if (n == nullptr) {
            n = new Node(repeat_value(init, len));
        }
        n->delta = n->pending ? compose_deltas(n->delta, d) : d;
        n->pending = true;
    }

    void push_delta(Node *n, int lo, int hi) {
        if (n == nullptr) {
            return;
        }
        if (n->pending) {
            n->value = apply_delta(n->value, n->delta, hi - lo + 1);
            if (lo != hi) {
                int mid = lo + (hi - lo) / 2;
                update_delta(n->left, n->delta, mid - lo + 1);
                update_delta(n->right, n->delta, hi - mid);
            }
        }
        n->pending = false;
    }

    T query(Node *n, int lo, int hi, int tgt_lo, int tgt_hi) {
        if (n == nullptr) {
            return repeat_value(init, hi - lo + 1);
        }
        push_delta(n, lo, hi);
        if (lo == tgt_lo && hi == tgt_hi) {
            return n->value;
        }
        int mid = lo + (hi - lo) / 2;
        if (tgt_lo <= mid && mid < tgt_hi) {
            return combine(
                query(n->left, lo, mid, tgt_lo, std::min(tgt_hi, mid)),
                query(n->right, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi)
            );
        }
        if (tgt_lo <= mid) {
            return query(n->left, lo, mid, tgt_lo, std::min(tgt_hi, mid));
        }
        return query(n->right, mid + 1, hi, std::max(tgt_lo, mid + 1), tgt_hi);
    }

    void update(Node *&n, int lo, int hi, int tgt_lo, int tgt_hi, const T &d) {
        if (n == nullptr) {
            if (hi < tgt_lo || lo > tgt_hi) {
                return;
            }
            n = new Node(repeat_value(init, hi - lo + 1));
        } else {

```

```

    push_delta(n, lo, hi);
}
if (hi < tgt_lo || lo > tgt_hi) {
    return;
}
if (tgt_lo <= lo && hi <= tgt_hi) {
    n->delta = d;
    n->pending = true;
    push_delta(n, lo, hi);
    return;
}
int mid = lo + (hi - lo) / 2;
update(n->left, lo, mid, tgt_lo, tgt_hi, d);
update(n->right, mid + 1, hi, tgt_lo, tgt_hi, d);
T left_value = (n->left != nullptr) ? n->left->value : repeat_value(init, mid - lo + 1);
T right_value = (n->right != nullptr) ? n->right->value : repeat_value(init, hi - mid);
n->value = combine(left_value, right_value);
}

template<typename Pred>
int find_first(Node *n, int lo, int hi, int tgt_lo, int tgt_hi, const Pred &pred) {
    if (hi < tgt_lo || lo > tgt_hi) {
        return -1;
    }
    if (n == nullptr) {
        // Untouched range of init values; if it qualifies, the leftmost in-range index does.
        return pred(repeat_value(init, hi - lo + 1)) ? std::max(lo, tgt_lo) : -1;
    }
    push_delta(n, lo, hi);
    if (!pred(n->value)) {
        return -1;
    }
    if (lo == hi) {
        return lo;
    }
    int mid = lo + (hi - lo) / 2;
    int res = find_first(n->left, lo, mid, tgt_lo, tgt_hi, pred);
    return res != -1 ? res : find_first(n->right, mid + 1, hi, tgt_lo, tgt_hi, pred);
}

template<typename Pred>
int find_last(Node *n, int lo, int hi, int tgt_lo, int tgt_hi, const Pred &pred) {
    if (hi < tgt_lo || lo > tgt_hi) {
        return -1;
    }
    if (n == nullptr) {
        return pred(repeat_value(init, hi - lo + 1)) ? std::min(hi, tgt_hi) : -1;
    }
    push_delta(n, lo, hi);
    if (!pred(n->value)) {
        return -1;
    }
    if (lo == hi) {
        return lo;
    }
    int mid = lo + (hi - lo) / 2;
    int res = find_last(n->right, mid + 1, hi, tgt_lo, tgt_hi, pred);
    return res != -1 ? res : find_last(n->left, lo, mid, tgt_lo, tgt_hi, pred);
}

template<typename Pred>
int max_right(Node *n, int lo, int hi, int tgt_lo, const Pred &pred, std::optional<T> &acc) {
    if (hi < tgt_lo) {
        return -1;
    }
    if (n != nullptr) {
        push_delta(n, lo, hi);
    }
}

```

```

    T node_value = n != nullptr ? n->value : repeat_value(init, hi - lo + 1);
    if (tgt_lo <= lo) {
        T next = acc ? combine(*acc, node_value) : node_value;
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = max_right(n == nullptr ? nullptr : n->left, lo, mid, tgt_lo, pred, acc);
    return res != -1 ? res
        : max_right(n == nullptr ? nullptr : n->right, mid + 1, hi, tgt_lo, pred, acc);
}

template<typename Pred>
int min_left(Node *n, int lo, int hi, int tgt_hi, const Pred &pred, std::optional<T> &acc) {
    if (tgt_hi <= lo) {
        return -1;
    }
    if (n != nullptr) {
        push_delta(n, lo, hi);
    }
    T node_value = n != nullptr ? n->value : repeat_value(init, hi - lo + 1);
    if (hi < tgt_hi) {
        T next = acc ? combine(node_value, *acc) : node_value;
        if (pred(next)) {
            acc = next;
            return -1;
        }
        if (lo == hi) {
            return lo + 1;
        }
    }
    int mid = lo + (hi - lo) / 2;
    int res = min_left(n == nullptr ? nullptr : n->right, mid + 1, hi, tgt_hi, pred, acc);
    return res != -1 ? res : min_left(n == nullptr ? nullptr : n->left, lo, mid, tgt_hi, pred, acc);
}

void clean_up(Node *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit SparseSegTree(const T &v = T()) : root(nullptr), init(v) {}

    ~SparseSegTree() { clean_up(root); }
    SparseSegTree(const SparseSegTree &) = delete;
    SparseSegTree &operator=(const SparseSegTree &) = delete;
    T at(int i) { return query(i, i); }
    T query(int lo, int hi) { return query(root, 0, N, lo, hi); }
    void update(int i, const T &d) { return update(i, i, d); }
    void update(int lo, int hi, const T &d) { return update(root, 0, N, lo, hi, d); }

    template<typename Pred>
    int find_first(int lo, int hi, const Pred &pred) {
        return find_first(root, 0, N, lo, hi, pred);
    }

    template<typename Pred>
    int find_last(int lo, int hi, const Pred &pred) {
        return find_last(root, 0, N, lo, hi, pred);
    }

```

```

}

template<typename Pred>
int max_right(int lo, const Pred &pred) {
    std::optional<T> acc;
    int res = max_right(root, 0, N, lo, pred, acc);
    return res == -1 ? N + 1 : res;
}

template<typename Pred>
int min_left(int hi, const Pred &pred) {
    std::optional<T> acc;
    int res = min_left(root, 0, N, hi, pred, acc);
    return res == -1 ? 0 : res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    SparseSegTree<int> t(0);
    t.update(0, 6);
    t.update(1, -2);
    t.update(2, 4);
    t.update(3, 8);
    t.update(4, 10);
    cout << "Values:";
    for (int i = 0; i < 5; i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == -2);
    t.update(0, 4, 5);
    t.update(3, 2);
    t.update(3, 1);
    cout << "Values:";
    for (int i = 0; i < 5; i++) {
        cout << " " << t.at(i);
    }
    cout << endl;
    assert(t.query(0, 3) == 1);

    // Segment-tree descent over the sparse domain; values at indices 0..4 are now {5, 5, 5, 1, 5}.
    auto at_most_1 = [](int v) { return v <= 1; };
    assert(t.find_first(0, 4, at_most_1) == 3); // only index 3 (value 1) qualifies
    assert(t.find_last(0, 4, at_most_1) == 3);
    assert(t.find_first(0, 2, at_most_1) == -1); // no value <= 1 in [0, 2]
    assert(t.max_right(0, [](int mn) { return mn > 1; }) == 3);
    assert(t.min_left(5, [](int mn) { return mn > 1; }) == 4);
    return 0;
}

```

Example Output

```

Values: 6 -2 4 8 10
Values: 5 5 5 1 5

```

2.4.11 Persistent Segment Tree

Maintains immutable versions of a segment tree under point updates. Each update creates a new root by copying only the nodes on the path from the root to the updated leaf, while all unchanged subtrees are shared with previous versions. Persistent segment trees are useful for offline order-statistics queries, rollback-like histories, and functional dynamic programming states.

- `PersistentSegTree(a)` constructs version 0 of an array with indices $[0, n)$ from an array `a` of length `n`.
- `update(version, index, value)` returns a new version where `a[index]` is set to `value`, leaving the old version unchanged.
- `query(version, lo, hi)` returns the sum of the inclusive range $[lo, hi]$ in the specified version.
- `versions()` returns the number of stored roots.

Time Complexity:

- $O(n)$ for construction.
- $O(\log n)$ per call to `update(version, index, value)` and `query(version, lo, hi)`.

Space Complexity:

- $O(n + q \log n)$ for q updates.
- $O(\log n)$ auxiliary stack space per update and query.

```
#include <cstdint>
#include <vector>

class PersistentSegTree {
    struct node {
        int64_t sum;
        int left, right;

        node(int64_t sum = 0, int left = -1, int right = -1) : sum(sum), left(left), right(right) {}
    };

    int n;
    std::vector<node> tree;
    std::vector<int> root;

    int build(const std::vector<int> &a, int lo, int hi) {
        if (lo == hi) {
            tree.emplace_back(a[lo]);
            return static_cast<int>(tree.size()) - 1;
        }
        int mid = lo + (hi - lo) / 2;
        int left = build(a, lo, mid);
        int right = build(a, mid + 1, hi);
        tree.emplace_back(tree[left].sum + tree[right].sum, left, right);
        return static_cast<int>(tree.size()) - 1;
    }

    int update(int cur, int lo, int hi, int index, int value) {
        if (lo == hi) {
            tree.emplace_back(value);
            return static_cast<int>(tree.size()) - 1;
        }
        int mid = lo + (hi - lo) / 2;
        int left = tree[cur].left, right = tree[cur].right;
        if (index <= mid) {
            left = update(left, lo, mid, index, value);
        } else {
            right = update(right, mid + 1, hi, index, value);
        }
        tree.emplace_back(tree[left].sum + tree[right].sum, left, right);
        return static_cast<int>(tree.size()) - 1;
    }
};
```

```

int64_t query(int cur, int lo, int hi, int qlo, int qhi) const {
    if (qlo <= lo && hi <= qhi) {
        return tree[cur].sum;
    }
    int mid = lo + (hi - lo) / 2;
    int64_t res = 0;
    if (qlo <= mid) {
        res += query(tree[cur].left, lo, mid, qlo, qhi);
    }
    if (mid < qhi) {
        res += query(tree[cur].right, mid + 1, hi, qlo, qhi);
    }
    return res;
}

public:
explicit PersistentSegTree(const std::vector<int> &a) : n(a.size()) {
    if (n > 0) {
        root.push_back(build(a, 0, n - 1));
    }
}

int versions() const { return static_cast<int>(root.size()); }

int update(int version, int index, int value) {
    root.push_back(update(root[version], 0, n - 1, index, value));
    return static_cast<int>(root.size() - 1);
}

int64_t query(int version, int lo, int hi) const {
    return query(root[version], 0, n - 1, lo, hi);
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{1, 2, 3, 4};
    PersistentSegTree t(a);
    int v1 = t.update(0, 1, 10);
    int v2 = t.update(v1, 3, -1);
    assert(t.query(0, 0, 3) == 10);
    assert(t.query(v1, 0, 3) == 18);
    assert(t.query(v2, 1, 3) == 12);
    assert(t.versions() == 3);
    return 0;
}

```

2.4.12 Merge Sort Tree

A merge sort tree is a static segment tree in which every node stores the sorted multiset of the values in its range. The nodes are exactly the ranges visited by a merge sort, and each node's sorted list is the merge of its two children's lists. This supports queries that count, within a range of indices, how many values fall below a threshold or inside a value interval: decompose the index range into $O(\log n)$ canonical nodes and binary search each node's sorted list.

The structure is built once and not modified afterward. All index ranges are inclusive $[l_0, h_1]$ with 0-based indices, and values may be of any comparable type.

- `MergeSortTree<T>(a)` builds the tree over the array `a`.
- `count_leq(lo, hi, x)` returns the number of indices `i` in `[lo, hi]` with `a[i] ≤ x`.
- `count_in(lo, hi, x, y)` returns the number of indices `i` in `[lo, hi]` with `x ≤ a[i] ≤ y`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the size of the array.
- $O(\log^2 n)$ per call to `count_leq()` and `count_in()`.

Space Complexity:

- $O(n \log n)$ for storage of the tree.
- $O(\log n)$ auxiliary stack space per query.

```
#include <algorithm>
#include <iterator>
#include <vector>

template<typename T>
class MergeSortTree {
    int len;
    std::vector<std::vector<T>> tree;

    void build(int node, int lo, int hi, const std::vector<T> &a) {
        if (lo == hi) {
            tree[node] = {a[lo]};
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(2 * node, lo, mid, a);
        build(2 * node + 1, mid + 1, hi, a);
        std::merge(
            tree[2 * node].begin(), tree[2 * node].end(), tree[2 * node + 1].begin(),
            tree[2 * node + 1].end(), std::back_inserter(tree[node])
        );
    }

    template<typename Fn>
    int query(int node, int lo, int hi, int tgt_lo, int tgt_hi, Fn count_node) const {
        if (tgt_hi < lo || hi < tgt_lo) {
            return 0;
        }
        if (tgt_lo <= lo && hi <= tgt_hi) {
            return count_node(tree[node]);
        }
        int mid = lo + (hi - lo) / 2;
        return query(2 * node, lo, mid, tgt_lo, tgt_hi, count_node) +
            query(2 * node + 1, mid + 1, hi, tgt_lo, tgt_hi, count_node);
    }

public:
    MergeSortTree(const std::vector<T> &a) : len(a.size()), tree(4 * a.size()) {
        if (len > 0) {
            build(1, 0, len - 1, a);
        }
    }

    int count_leq(int lo, int hi, const T &x) const {
        return query(1, 0, len - 1, lo, hi, [&](const std::vector<T> &v) {
            return static_cast<int>(std::upper_bound(v.begin(), v.end(), x) - v.begin());
        });
    }

    int count_in(int lo, int hi, const T &x, const T &y) const {
        return query(1, 0, len - 1, lo, hi, [&](const std::vector<T> &v) {
            return static_cast<int>(
                std::upper_bound(v.begin(), v.end(), y) - std::lower_bound(v.begin(), v.end(), x)
            );
        });
    }
};
```

```

    );
  });
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{5, 2, 8, 6, 1, 9, 3};
    MergeSortTree<int> t(a);

    assert(t.count_leq(0, 6, 5) == 4);    // 5, 2, 1, 3
    assert(t.count_leq(2, 4, 6) == 2);    // 6, 1
    assert(t.count_in(0, 6, 3, 8) == 4);   // 5, 8, 6, 3
    assert(t.count_in(1, 5, 2, 6) == 2);   // 2, 6 (8, 1, 9 excluded)
    return 0;
}

```

2.4.13 Wavelet Tree

A wavelet tree is a static structure over an integer array with values in a known range $[\text{min_val}, \text{max_val}]$. The root splits the value range at its midpoint, recording for every prefix of the array how many of its elements belong to the lower value half, then stably partitions the array so that those elements form the left child and the rest form the right child. Recursing on each half builds a balanced tree of height $O(\log \sigma)$, where σ is the size of the value range.

The recorded prefix counts let a query at any node translate a range of array positions into the corresponding range of positions in either child, so a single root-to-leaf descent answers order statistics and rank queries. This is strictly more powerful than a merge sort tree: besides counting values below a threshold, it reports the k -th smallest value of a range in $O(\log \sigma)$ time.

All position ranges are inclusive $[\text{lo}, \text{hi}]$ with 0-based indices. If the values are large or sparse, compress them to a small contiguous range first.

- `WaveletTree(a, min_val, max_val)` builds the tree over the array `a`, whose values must all lie in $[\text{min_val}, \text{max_val}]$.
- `kth_smallest(lo, hi, k)` returns the k -th smallest value among positions $[\text{lo}, \text{hi}]$, where k is 1-based (so `k == 1` returns the minimum).
- `count_leq(lo, hi, x)` returns the number of positions i in $[\text{lo}, \text{hi}]$ with `a[i] ≤ x`.
- `count_in(lo, hi, x, y)` returns the number of positions i in $[\text{lo}, \text{hi}]$ with `x ≤ a[i] ≤ y`.

Time Complexity:

- $O(n \log \sigma)$ per call to the constructor, where n is the size of the array and σ is the size of the value range.
- $O(\log \sigma)$ per call to `kth_smallest()`, `count_leq()`, and `count_in()`.

Space Complexity:

- $O(n \log \sigma)$ for storage of the tree.
- $O(\log \sigma)$ auxiliary stack space per query.

```

#include <algorithm>
#include <cassert>
#include <memory>
#include <vector>

class WaveletTree {

```



```

int min_val, max_val; // The value range this node covers.
std::unique_ptr<WaveletTree> left, right;
std::vector<int> b; // b[i] = number of the first i elements that go to the left child.

WaveletTree(std::vector<int>::iterator lo, std::vector<int>::iterator hi, int x, int y)
    : min_val(x), max_val(y) {
    if (lo >= hi) {
        return;
    }
    int mid = min_val + (max_val - min_val) / 2;
    auto go_left = [mid](int v) { return v <= mid; };
    b.reserve((hi - lo) + 1);
    b.push_back(0);
    for (auto it = lo; it != hi; ++it) {
        b.push_back(b.back() + (go_left(*it) ? 1 : 0));
    }
    if (min_val == max_val) {
        return;
    }
    auto pivot = std::stable_partition(lo, hi, go_left);
    left.reset(new WaveletTree(lo, pivot, min_val, mid));
    right.reset(new WaveletTree(pivot, hi, mid + 1, max_val));
}

// 1-based positions [lo, hi] for the recursion below.
int kth(int lo, int hi, int k) const {
    if (min_val == max_val) {
        return min_val;
    }
    int in_left = b[hi] - b[lo - 1];
    if (k <= in_left) {
        return left->kth(b[lo - 1] + 1, b[hi], k);
    }
    return right->kth(lo - b[lo - 1], hi - b[hi], k - in_left);
}

int leq(int lo, int hi, int x) const {
    if (lo > hi || x < min_val) {
        return 0;
    }
    if (max_val <= x) {
        return hi - lo + 1;
    }
    return left->leq(b[lo - 1] + 1, b[hi], x) + right->leq(lo - b[lo - 1], hi - b[hi], x);
}

public:
WaveletTree(std::vector<int> a, int min_val, int max_val)
    : WaveletTree(a.begin(), a.end(), min_val, max_val) {}

int kth_smallest(int lo, int hi, int k) const {
    int n = static_cast<int>(b.size()) - 1;
    assert(0 <= lo && lo <= hi && hi < n);
    assert(1 <= k && k <= hi - lo + 1);
    return kth(lo + 1, hi + 1, k);
}

int count_leq(int lo, int hi, int x) const {
    int n = static_cast<int>(b.size()) - 1;
    assert(0 <= lo && lo <= hi && hi < n);
    return leq(lo + 1, hi + 1, x);
}

int count_in(int lo, int hi, int x, int y) const {
    int n = static_cast<int>(b.size()) - 1;
    assert(0 <= lo && lo <= hi && hi < n);
    assert(x <= y);
    return leq(lo + 1, hi + 1, y) - leq(lo + 1, hi + 1, x - 1);
}

```

```

    }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{5, 2, 8, 6, 1, 9, 3};
    WaveletTree t(a, 1, 9);

    assert(t.kth_smallest(0, 6, 1) == 1); // Minimum of the whole array.
    assert(t.kth_smallest(0, 6, 4) == 5); // Median of the whole array.
    assert(t.kth_smallest(2, 4, 2) == 6); // {8, 6, 1} -> 6 is 2nd smallest.

    assert(t.count_leq(0, 6, 5) == 4); // 5, 2, 1, 3
    assert(t.count_in(0, 6, 3, 8) == 4); // 5, 8, 6, 3
    assert(t.count_in(1, 5, 2, 6) == 2); // 2, 6
    return 0;
}

```

2.4.14 Cartesian Tree

Builds the min Cartesian tree of an array in linear time. A Cartesian tree is a binary tree whose inorder traversal is the original array order and whose heap property is determined by array values. For a min Cartesian tree, each parent has value less than or equal to its children.

Cartesian trees connect arrays, stacks, range minimum queries, treaps, and largest rectangle style problems. Equal values are broken by position, so earlier equal values become ancestors of later equal values.

- `CartesianTree(a)` constructs the tree for array `a`.
- `root[]` stores the root index.
- `parent[]`, `left[]`, and `right[]` store neighboring node indices, or `-1` if absent.

Time Complexity: $O(n)$ construction time, where n is the array size.

Space Complexity: $O(n)$ for tree arrays and the monotone stack.

```

#include <vector>

struct CartesianTree {
    int root;
    std::vector<int> parent, left, right;

    explicit CartesianTree(const std::vector<int> &a)
        : root(-1), parent(a.size(), -1), left(a.size(), -1), right(a.size(), -1) {
        std::vector<int> st;
        for (int i = 0, n = static_cast<int>(a.size()); i < n; i++) {
            int last = -1;
            while (!st.empty() && a[i] < a[st.back()]) {
                last = st.back();
                st.pop_back();
            }
            if (!st.empty()) {
                right[st.back()] = i;
                parent[i] = st.back();
            }
            if (last != -1) {
                left[i] = last;
                parent[last] = i;
            }
        }
    }
}

```

```

        st.push_back(i);
    }
    root = st.empty() ? -1 : st[0];
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

void inorder(const CartesianTree &t, int u, vector<int> *order) {
    if (u == -1) {
        return;
    }
    inorder(t, t.left[u], order);
    order->push_back(u);
    inorder(t, t.right[u], order);
}

int main() {
    vector<int> a{3, 2, 6, 1, 9};
    CartesianTree t(a);
    assert(t.root == 3);
    assert(t.parent[1] == 3);
    assert(t.parent[4] == 3);

    vector<int> order;
    inorder(t, t.root, &order);
    for (int i = 0; i < 5; i++) {
        assert(order[i] == i);
    }
    return 0;
}

```

2.4.15 Implicit Treap

Maintain a dynamically-sized array using a balanced binary search tree while supporting both dynamic queries and updates of contiguous subarrays via the lazy propagation technique. A treap maintains a balanced binary tree structure by preserving the heap property on the randomly generated priority values of nodes, thereby making insertions and deletions run in $O(\log n)$ with high probability.

The query operation is defined by an associative aggregate function `combine(a, b)`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`.

Range updates are defined by `apply_delta(v, d, len)`, which applies an update delta `d` to an aggregate summary `v` representing `len` array values, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined segment must be equivalent to applying it to each child segment and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines range assignment. For range increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, len)` should return `v + d` for range-min/range-max queries, and `v + d * len` for range-sum queries.

- `ImplicitTreap<T>(n, v)` constructs an array of size `n`, with indices `[0, n)`, with all values initialized to `v`.
- `ImplicitTreap<T>(lo, hi)` constructs an array from two iterators as a range `[lo, hi)`, initialized to the elements of the range in the same order.
- `size()` returns the size of the array.
- `empty()` returns whether the array is empty.

- `at(i)` returns the value at index `i`.
- `query(lo, hi)` returns the result of `combine()` applied to all indices from `lo` to `hi`, inclusive.
- `update(lo, hi, d)` applies the delta `d` to every index from `lo` to `hi`, inclusive.
- `update(i, d)` applies the delta `d` to the single index `i`.
- `insert(i, v)` inserts a new value `v` before index `i`, shifting later elements one position right.
- `erase(i)` removes the element at index `i`, shifting later elements one position left.
- `push_back(v)` appends a new value `v` to the end of the array.
- `pop_back()` removes the last element of the array.

The query and update operations match those of the point- and range-update segment trees in this section; `insert()`, `erase()`, `push_back()`, and `pop_back()` are additionally analogous to those of `std::vector` (here, `insert()` and `erase()` take an index instead of an iterator).

Time Complexity:

- $O(n)$ per call to both constructors, where n is the size of the array.
- $O(1)$ per call to `size()` and `empty()`.
- $O(\log n)$ on average per call to all other operations.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for `size()` and `empty()`.
- $O(\log n)$ auxiliary stack space for all other operations.

```
#include <cstdint>

template<typename T>
class ImplicitTreap {
    static T combine(const T &a, const T &b) { return a < b ? a : b; }
    static T apply_delta(const T &v, const T &d, int64_t len) { return d; }
    static T compose_deltas(const T &d1, const T &d2) { return d2; }

    struct Node {
        static uint32_t rand32() {
            static uint32_t x = 123456789;
            x ^= x << 13;
            x ^= x >> 17;
            x ^= x << 5;
            return x;
        }

        T value, subtree_value, delta;
        bool pending;
        int size;
        uint32_t priority;
        Node *left, *right;

        explicit Node(const T &v)
            : value(v),
              subtree_value(v),
              pending(false),
              size(1),
              priority(rand32()),
              left(nullptr),
              right(nullptr) {}
    } *root;

    static int size(Node *n) { return (n == nullptr) ? 0 : n->size; }

    static void update_value(Node *n) {
        if (n == nullptr) {
            return;
        }
        n->subtree_value = n->value;
    }
};
```

```

    if (n->left != nullptr) {
        n->subtree_value = combine(n->subtree_value, n->left->subtree_value);
    }
    if (n->right != nullptr) {
        n->subtree_value = combine(n->subtree_value, n->right->subtree_value);
    }
    n->size = 1 + size(n->left) + size(n->right);
}

static void update_delta(Node *n, const T &d) {
    if (n != nullptr) {
        n->value = apply_delta(n->value, d, 1);
        n->subtree_value = apply_delta(n->subtree_value, d, n->size);
        n->delta = n->pending ? compose_deltas(n->delta, d) : d;
        n->pending = true;
    }
}

static void push_delta(Node *n) {
    if (n == nullptr || !n->pending) {
        return;
    }
    if (n->size > 1) {
        update_delta(n->left, n->delta);
        update_delta(n->right, n->delta);
    }
    n->pending = false;
}

static void merge(Node *&n, Node *left, Node *right) {
    push_delta(left);
    push_delta(right);
    if (left == nullptr) {
        n = right;
    } else if (right == nullptr) {
        n = left;
    } else if (left->priority < right->priority) {
        merge(left->right, left->right, right);
        n = left;
    } else {
        merge(right->left, left, right->left);
        n = right;
    }
    update_value(n);
}

static void split(Node *n, Node *&left, Node *&right, int i) {
    push_delta(n);
    if (n == nullptr) {
        left = right = nullptr;
    } else if (i <= size(n->left)) {
        split(n->left, left, n->left, i);
        right = n;
    } else {
        split(n->right, n->right, right, i - size(n->left) - 1);
        left = n;
    }
    update_value(n);
}

static void insert(Node *&n, Node *new_node, int i) {
    push_delta(n);
    if (n == nullptr) {
        n = new_node;
    } else if (new_node->priority < n->priority) {
        split(n, new_node->left, new_node->right, i);
        n = new_node;
    } else if (i <= size(n->left)) {

```

```

    insert(n->left, new_node, i);
} else {
    insert(n->right, new_node, i - size(n->left) - 1);
}
update_value(n);
}

static void erase(Node *&n, int i) {
    push_delta(n);
    if (i == size(n->left)) {
        Node *left = n->left, *right = n->right;
        delete n;
        merge(n, left, right);
    } else if (i < size(n->left)) {
        erase(n->left, i);
    } else {
        erase(n->right, i - size(n->left) - 1);
    }
    update_value(n);
}

static Node *select(Node *n, int i) {
    push_delta(n);
    if (i < size(n->left)) {
        return select(n->left, i);
    }
    if (i > size(n->left)) {
        return select(n->right, i - size(n->left) - 1);
    }
    return n;
}

void clean_up(Node *&n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit ImplicitTreap(int n = 0, const T &v = T()) : root(nullptr) {
        for (int i = 0; i < n; i++) {
            push_back(v);
        }
    }

    template<typename It>
    ImplicitTreap(It lo, It hi) : root(nullptr) {
        for (; lo != hi; ++lo) {
            push_back(*lo);
        }
    }

    ~ImplicitTreap() { clean_up(root); }
    ImplicitTreap(const ImplicitTreap &) = delete;
    ImplicitTreap &operator=(const ImplicitTreap &) = delete;
    int size() const { return size(root); }
    bool empty() const { return root == nullptr; }
    void insert(int i, const T &v) { insert(root, new Node(v), i); }
    void erase(int i) { erase(root, i); }
    void push_back(const T &v) { insert(size(), v); }
    void pop_back() { erase(size() - 1); }
    T at(int i) const { return select(root, i)->value; }

    T query(int lo, int hi) {
        Node *l1, *r1, *l2, *r2, *t;
        split(root, l1, r1, hi + 1);

```

```

    split(l1, l2, r2, lo);
    T res = r2->subtree_value;
    merge(t, l2, r2);
    merge(root, t, r1);
    return res;
}

void update(int lo, int hi, const T &d) {
    Node *l1, *r1, *l2, *r2, *t;
    split(root, l1, r1, hi + 1);
    split(l1, l2, r2, lo);
    update_delta(r2, d);
    merge(t, l2, r2);
    merge(root, t, r1);
}

void update(int i, const T &d) { update(i, i, d); }
};

```

Example Usage

```

#include <iostream>
#include <vector>
using namespace std;

void print(ImplicitTreap<int> &t) {
    cout << "Values:";
    for (int i = 0; i < t.size(); i++) {
        cout << " " << t.at(i);
    }
    cout << " (min: " << t.query(0, t.size() - 1) << ")" << endl;
}

int main() {
    vector<int> a{99, -2, 1, 8, 10};
    ImplicitTreap<int> t(a.begin(), a.end());
    t.push_back(11);
    t.push_back(12);
    t.pop_back();
    print(t);
    t.insert(0, 90);
    t.erase(1);
    print(t);
    t.update(0, 1, 2);
    print(t);
    return 0;
}

```

Example Output

```

Values: 99 -2 1 8 10 11 (min: -2)
Values: 90 -2 1 8 10 11 (min: -2)
Values: 2 2 1 8 10 11 (min: 1)

```

2.5 Range Queries in Two Dimensions

2.5.1 Quadtree (Point Update)

Maintain a two-dimensional array over a huge grid while supporting point updates and rectangle queries. This is a sparse (a.k.a. dynamic or implicit) quadtree: it recursively splits each rectangle into four quadrants, and lazily allocates only the nodes touched by updates or needed to summarize queried regions.

The query operation is defined by a commutative associative aggregate function `combine(a, b)`. Because untouched regions are implicit, `repeat_value(v, area)` must return the aggregate summary of `area` copies of the initial value `v`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. For rectangle-sum queries, `combine(a, b)` should return `a + b` and `repeat_value(v, area)` should return `v * area`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at one updated cell. The default code below defines updates that “set” the chosen cell to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

Choose this over a sparse 2D segment tree when the grid is huge and touched cells or queried rectangles are sparse or naturally align with large quadrants. Choose the sparse 2D segment tree instead when many thin or adversarial rectangles are expected and a predictable $O(\log(R) \cdot \log(C))$ bound matters more than lower constant factors on aligned regions.

- `Quadtree<T, R, C>(v)` constructs a two-dimensional array with rows from 0 to R (inclusive) and columns from 0 to C (inclusive). All array values are initialized to `v`.
- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `combine()` applied to every value in the rectangular region consisting of rows from `r1` to `r2`, inclusive, and columns from `c1` to `c2`, inclusive.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `apply_delta(v, d)`.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\log(\max(R, C)))$ per call to `at()` and `update()`, which descend a single root-to-cell path.
- $O(R + C)$ worst case per call to `query()`, attained when the queried rectangle straddles the quadrant boundaries at every level (for example a one-cell-thick full-width strip); rectangles that align with a few large quadrants are far cheaper.

Space Complexity:

- $O(\min(n \cdot \log(\max(R, C)), R \cdot C))$ for storage of quadtree nodes after n point updates. More precisely, since bounds are inclusive, the dense-grid ceiling is $(R + 1)(C + 1)$ cells.
- $O(\log(\max(R, C)))$ auxiliary stack space for `update()`, `query()`, and `at()`.

```
#include <algorithm>
#include <cstdint>
#include <cstdint>

template<typename T, int R = 1000000000, int C = 1000000000>
class Quadtree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T repeat_value(const T &v, int64_t area) { return v; }
    static T apply_delta(const T &v, const T &d) { return d; }

    struct Node {
        T value;
        Node *child[4];

        explicit Node(const T &v) : value(v) {
            for (auto &c : child) {
                c = nullptr;
            }
        }
    };

    Node *root;
    T init;

    // Helper variables for query().
    int tgt_r1, tgt_c1, tgt_r2, tgt_c2;
    T res;
    bool found;
```



```

static int64_t area(int r1, int c1, int r2, int c2) {
    return static_cast<int64_t>(r2 - r1 + 1) * (c2 - c1 + 1);
}

void append_result(const T &v) {
    res = found ? combine(res, v) : v;
    found = true;
}

T child_value(Node *n, int r1, int c1, int r2, int c2) {
    return (n != nullptr) ? n->value : repeat_value(init, area(r1, c1, r2, c2));
}

void query(Node *n, int r1, int c1, int r2, int c2) {
    if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
        return;
    }
    if (n == nullptr) {
        int rlen = std::min(r2, tgt_r2) - std::max(r1, tgt_r1) + 1;
        int clen = std::min(c2, tgt_c2) - std::max(c1, tgt_c1) + 1;
        append_result(repeat_value(init, static_cast<int64_t>(rlen) * clen));
        return;
    }
    if (tgt_r1 <= r1 && r2 <= tgt_r2 && tgt_c1 <= c1 && c2 <= tgt_c2) {
        append_result(n->value);
        return;
    }
    int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
    query(n->child[0], r1, c1, rmid, cmid);
    query(n->child[1], rmid + 1, c1, r2, cmid);
    query(n->child[2], r1, cmid + 1, rmid, c2);
    query(n->child[3], rmid + 1, cmid + 1, r2, c2);
}

// Helper variables for update().
int tgt_r, tgt_c;
T delta;

void update(Node *&n, int r1, int c1, int r2, int c2) {
    if (tgt_r < r1 || tgt_r > r2 || tgt_c < c1 || tgt_c > c2) {
        return;
    }
    if (n == nullptr) {
        n = new Node(repeat_value(init, area(r1, c1, r2, c2)));
    }
    if (r1 == r2 && c1 == c2) {
        n->value = apply_delta(n->value, delta);
        return;
    }
    int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
    update(n->child[0], r1, c1, rmid, cmid);
    update(n->child[1], rmid + 1, c1, r2, cmid);
    update(n->child[2], r1, cmid + 1, rmid, c2);
    update(n->child[3], rmid + 1, cmid + 1, r2, c2);
    n->value = combine(
        combine(
            child_value(n->child[0], r1, c1, rmid, cmid),
            child_value(n->child[1], rmid + 1, c1, r2, cmid)
        ),
        combine(
            child_value(n->child[2], r1, cmid + 1, rmid, c2),
            child_value(n->child[3], rmid + 1, cmid + 1, r2, c2)
        )
    );
}

static void clean_up(Node *n) {
    if (n != nullptr) {

```

```

        for (auto *c : n->child) {
            clean_up(c);
        }
        delete n;
    }
}

public:
    explicit Quadtree(const T &v = T()) : root(nullptr), init(v) {}

    ~Quadtree() { clean_up(root); }
    Quadtree(const Quadtree &) = delete;
    Quadtree &operator=(const Quadtree &) = delete;
    T at(int r, int c) { return query(r, c, r, c); }

    T query(int r1, int c1, int r2, int c2) {
        tgt_r1 = r1;
        tgt_c1 = c1;
        tgt_r2 = r2;
        tgt_c2 = c2;
        found = false;
        query(root, 0, 0, R, C);
        return found ? res : repeat_value(init, area(r1, c1, r2, c2));
    }

    void update(int r, int c, const T &d) {
        tgt_r = r;
        tgt_c = c;
        delta = d;
        update(root, 0, 0, R, C);
    }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    Quadtree<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.query(0, 0, 0, 1) == 6);
    assert(t.query(0, 0, 1, 0) == 5);
    assert(t.query(1, 1, 2, 2) == 0);
    assert(t.query(0, 0, 1000000000, 1000000000) == 0);
    t.update(500000000, 500000000, -100);
    assert(t.query(0, 0, 1000000000, 1000000000) == -100);
    return 0;
}

```

Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

2.5.2 Quadtree (Range Update)

Maintain a two-dimensional array over a huge grid while supporting rectangle updates and rectangle queries. This is a sparse (a.k.a. dynamic or implicit) quadtree: it recursively splits each rectangle into four quadrants, lazily allocates touched nodes, and stores pending rectangle updates with lazy propagation.

The query operation is defined by a commutative associative aggregate function `combine(a, b)`. Because untouched regions are implicit, `repeat_value(v, area)` must return the aggregate summary of `area` copies of the initial value `v`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. For rectangle-sum queries, `combine(a, b)` should return `a + b` and `repeat_value(v, area)` should return `v * area`.

Range updates are defined by `apply_delta(v, d, area)`, which applies an update delta `d` to an aggregate summary `v` representing `area` cells, and by `compose_deltas(old, d)`, which combines a pending older delta with a newer delta in that order. These functions do not support arbitrary combinations: applying a delta to a combined region must be equivalent to applying it to each child region and then combining the results, and composed deltas must be equivalent to performing their updates sequentially. The default code below defines rectangle assignment. For rectangle increment, `compose_deltas(old, d)` should return `old + d`; `apply_delta(v, d, area)` should return `v + d` for range-min/range-max queries, and `v + d * area` for range-sum queries.

Choose this over a sparse 2D segment tree when rectangle updates or queries often cover large aligned regions of a huge sparse grid. Choose a sparse 2D segment tree when worst-case rectangles may be long, thin, or otherwise poorly aligned with quadrant boundaries; the quadtree can visit many boundary nodes in those cases.

- `LazyQuadtree<T, R, C>(v)` constructs a 2D array with rows 0 to R (inclusive) and columns 0 to C (inclusive), where template parameters R and C . All array values are implicitly initialized to `v`. Nodes are allocated lazily as indices are touched.
- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `combine()` applied to every value in the rectangular region consisting of rows from `r1` to `r2` and columns from `c1` to `c2`, inclusive.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `apply_delta(v, d)`.
- `update(r1, c1, r2, c2)` modifies the value at each index of the rectangular region consisting of rows from `r1` to `r2` and columns from `c1` to `c2`, inclusive, by applying the delta `d` to each value.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\log(\max(R, C)))$ per call to `at()`, which descends a single root-to-cell path.
- $O(R + C)$ worst case per call to `update()` and `query()`, attained when the rectangle straddles quadrant boundaries at many levels; rectangles that align with a few large quadrants are far cheaper.

Space Complexity:

- $O(k)$ for storage, where k is the number of allocated quadtree nodes. Each rectangle update can allocate as many nodes as it visits; this may be large for thin or poorly aligned rectangles, but aligned rectangles and point updates allocate far fewer nodes.
- $O(\log(\max(R, C)))$ auxiliary stack space for `update()`, `query()`, and `at()`.

```

#include <algorithm>
#include <cstdint>
#include <cstdlib>

template<typename T, int R = 1000000000, int C = 1000000000>
class LazyQuadtree {
    static T combine(const T &a, const T &b) { return std::min(a, b); }

```

```

static T repeat_value(const T &v, int64_t area) { return v; }
static T apply_delta(const T &v, const T &d, int64_t area) { return d; }
static T compose_deltas(const T &d1, const T &d2) { return d2; }

struct Node {
    T value, delta;
    bool pending;
    Node *child[4];

    explicit Node(const T &v) : value(v), pending(false) {
        for (auto &c : child) {
            c = nullptr;
        }
    }
} *root;

T init;

static int64_t area(int r1, int c1, int r2, int c2) {
    return static_cast<int64_t>(r2 - r1 + 1) * (c2 - c1 + 1);
}

void update_delta(Node *&n, const T &d, int64_t area) {
    if (n == nullptr) {
        n = new Node(repeat_value(init, area));
    }
    n->delta = n->pending ? compose_deltas(n->delta, d) : d;
    n->pending = true;
}

void push_delta(Node *n, int r1, int c1, int r2, int c2) {
    if (n->pending) {
        int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
        int64_t cur_area = area(r1, c1, r2, c2);
        n->value = apply_delta(n->value, n->delta, cur_area);
        if (cur_area > 1) {
            int rlen = r2 - r1 + 1, clen = c2 - c1 + 1;
            int rlen1 = rmid - r1 + 1, rlen2 = rlen - rlen1;
            int clen1 = cmid - c1 + 1, clen2 = clen - clen1;
            update_delta(n->child[0], n->delta, static_cast<int64_t>(rlen1) * clen1);
            update_delta(n->child[1], n->delta, static_cast<int64_t>(rlen2) * clen1);
            update_delta(n->child[2], n->delta, static_cast<int64_t>(rlen1) * clen2);
            update_delta(n->child[3], n->delta, static_cast<int64_t>(rlen2) * clen2);
        }
        n->pending = false;
    }
}

// Helper variables for query() and update().
int tgt_r1, tgt_c1, tgt_r2, tgt_c2;
T res, delta;
bool found;

void append_result(const T &v) {
    res = found ? combine(res, v) : v;
    found = true;
}

T child_value(Node *n, int r1, int c1, int r2, int c2) {
    return (n != nullptr) ? n->value : repeat_value(init, area(r1, c1, r2, c2));
}

void query(Node *n, int r1, int c1, int r2, int c2) {
    if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
        return;
    }
    if (n == nullptr) {
        int rlen = std::min(r2, tgt_r2) - std::max(r1, tgt_r1) + 1;

```

```

    int clen = std::min(c2, tgt_c2) - std::max(c1, tgt_c1) + 1;
    append_result(repeat_value(init, static_cast<int64_t>(rlen) * clen));
    return;
}
push_delta(n, r1, c1, r2, c2);
if (tgt_r1 <= r1 && r2 <= tgt_r2 && tgt_c1 <= c1 && c2 <= tgt_c2) {
    append_result(n->value);
    return;
}
int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
query(n->child[0], r1, c1, rmid, cmid);
query(n->child[1], rmid + 1, c1, r2, cmid);
query(n->child[2], r1, cmid + 1, rmid, c2);
query(n->child[3], rmid + 1, cmid + 1, r2, c2);
}

void update(Node *&n, int r1, int c1, int r2, int c2) {
    if (n == nullptr) {
        if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
            return;
        }
        n = new Node(repeat_value(init, area(r1, c1, r2, c2)));
    } else {
        push_delta(n, r1, c1, r2, c2);
    }
    if (tgt_r2 < r1 || tgt_r1 > r2 || tgt_c2 < c1 || tgt_c1 > c2) {
        return;
    }
    if (tgt_r1 <= r1 && r2 <= tgt_r2 && tgt_c1 <= c1 && c2 <= tgt_c2) {
        n->delta = delta;
        n->pending = true;
        push_delta(n, r1, c1, r2, c2);
        return;
    }
    int rmid = r1 + (r2 - r1) / 2, cmid = c1 + (c2 - c1) / 2;
    update(n->child[0], r1, c1, rmid, cmid);
    update(n->child[1], rmid + 1, c1, r2, cmid);
    update(n->child[2], r1, cmid + 1, rmid, c2);
    update(n->child[3], rmid + 1, cmid + 1, r2, c2);
    n->value = combine(
        combine(
            child_value(n->child[0], r1, c1, rmid, cmid),
            child_value(n->child[1], rmid + 1, c1, r2, cmid)
        ),
        combine(
            child_value(n->child[2], r1, cmid + 1, rmid, c2),
            child_value(n->child[3], rmid + 1, cmid + 1, r2, c2)
        )
    );
}

static void clean_up(Node *n) {
    if (n != nullptr) {
        for (auto *c : n->child) {
            clean_up(c);
        }
        delete n;
    }
}

public:
explicit LazyQuadtree(const T &v = T()) : root(nullptr), init(v) {}

~LazyQuadtree() { clean_up(root); }
LazyQuadtree(const LazyQuadtree &) = delete;
LazyQuadtree &operator=(const LazyQuadtree &) = delete;
T at(int r, int c) { return query(r, c, r, c); }

```

```

T query(int r1, int c1, int r2, int c2) {
    tgt_r1 = r1;
    tgt_c1 = c1;
    tgt_r2 = r2;
    tgt_c2 = c2;
    found = false;
    query(root, 0, 0, R, C);
    return found ? res : repeat_value(init, area(r1, c1, r2, c2));
}

void update(int r, int c, const T &d) { update(r, c, r, c, d); }

void update(int r1, int c1, int r2, int c2, const T &d) {
    tgt_r1 = r1;
    tgt_c1 = c1;
    tgt_r2 = r2;
    tgt_c2 = c2;
    delta = d;
    update(root, 0, 0, R, C);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    LazyQuadtree<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
    cout << "Values:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << t.at(i, j) << " ";
        }
        cout << endl;
    }
    assert(t.query(0, 0, 0, 1) == 6);
    assert(t.query(0, 0, 1, 0) == 5);
    assert(t.query(1, 1, 2, 2) == 0);
    assert(t.query(0, 0, 1000000000, 1000000000) == 0);
    t.update(500000000, 500000000, -100);
    assert(t.query(0, 0, 1000000000, 1000000000) == -100);
    return 0;
}

```

Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

2.5.3 2D Segment Tree (Point Update)

Maintain a two-dimensional array over a huge grid while supporting dynamic queries of rectangular subarrays and dynamic updates of individual indices. This is a sparse (a.k.a. dynamic or implicit) 2D segment tree: row and

column nodes are allocated lazily as cells are touched, so large coordinate bounds are supported without allocating the full grid.

The query operation is defined by a commutative associative aggregate function `combine(a, b)`. Because untouched regions are implicit, `repeat_value(v, area)` must return the aggregate summary of `area` copies of the initial value `v`. The default code below assumes a numerical array type, defining queries for the “min” of the target range. For rectangle-sum queries, `combine(a, b)` should return `a + b` and `repeat_value(v, area)` should return `v * area`.

The point update operation is defined by `apply_delta(v, d)`, which returns the new value at one updated cell. The default code below defines updates that “set” the chosen cell to a new value. Another possible update operation is “increment”, in which case `apply_delta(v, d)` should return `v + d`.

Compared with the quadtrees in the previous sections, this structure splits rows and columns independently, so every rectangle query decomposes into $O(\log(R) \cdot \log(C))$ canonical rectangles. That makes it preferable when queries may be thin, off-center, or adversarially placed: a quadtree can degrade to visiting $O(R + C)$ boundary nodes for such rectangles. The trade-off is a larger tree and less benefit from queries that happen to align with big square quadrants.

For dense additive rectangle sums, prefer the simple 2D Fenwick tree in 2.6.5. A dense vector-backed 2D segment tree can be simpler and faster on modest grids with custom aggregates such as min/max, but it needs $O(R \cdot C)$ storage with large constants, so this sparse version is usually the safer codebook default.

- `SegTree2D<T, R, C>(v)` constructs a two-dimensional array with rows from 0 to R (inclusive) and columns from 0 to C (inclusive). All array values are implicitly initialized to `v`. Nodes are allocated lazily as indices are touched.
- `at(r, c)` returns the value at row `r`, column `c`.
- `query(r1, c1, r2, c2)` returns the result of `combine()` applied to every value in the rectangular region consisting of rows from `r1` to `r2`, inclusive, and columns from `c1` to `c2`, inclusive.
- `update(r, c, d)` assigns the value `v` at `(r, c)` to `apply_delta(v, d)`.

Time Complexity:

- $O(1)$ per call to the constructor.
- $O(\log(R) \cdot \log(C))$ per call to `at()`, `update()`, and `query()`.

Space Complexity:

- $O(n \log(R) \log(C))$ for storage after n point updates.
- $O(\log(R) + \log(C))$ auxiliary stack space for `update()`, `query()`, and `at()`.

```
#include <algorithm>
#include <cstdint>
#include <cstdint>
#include <optional>

template<typename T, int R = 1000000000, int C = 1000000000>
class SegTree2D {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T repeat_value(const T &v, int64_t area) { return v; }
    static T apply_delta(const T &v, const T &d) { return d; }

    struct InnerNode {
        T value;
        int low, high;
        InnerNode *left, *right;

        InnerNode(int lo, int hi, const T &v)
            : value(v), low(lo), high(hi), left(nullptr), right(nullptr) {}
    };

    struct OuterNode {
        InnerNode root;
        int low, high;
        OuterNode *left, *right;
    };
};
```

```

OuterNode(int lo, int hi, const T &v)
    : root(0, C, v), low(lo), high(hi), left(nullptr), right(nullptr) {}
} *root;

T init;

// Helper variables for query() and update().
int tgt_r1, tgt_c1, tgt_r2, tgt_c2;
int64_t width;

static int64_t length(int lo, int hi) { return hi - lo + 1LL; }

void append_result(std::optional<T> &res, const T &v) { res = res ? combine(*res, v) : v; }

T query(InnerNode *n, int qlo, int qhi, int64_t rows) {
    if (n == nullptr) {
        return repeat_value(init, rows * length(qlo, qhi));
    }
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    std::optional<T> res;
    if (qlo < lo) {
        int missing_hi = std::min(qhi, lo - 1);
        append_result(res, repeat_value(init, rows * length(qlo, missing_hi)));
    }
    int overlap_lo = std::max(qlo, lo), overlap_hi = std::min(qhi, hi);
    if (overlap_lo <= overlap_hi) {
        if (overlap_lo == lo && overlap_hi == hi) {
            append_result(res, n->value);
        } else {
            if (overlap_lo <= mid) {
                append_result(res, query(n->left, overlap_lo, std::min(overlap_hi, mid), rows));
            }
            if (mid < overlap_hi) {
                append_result(res, query(n->right, std::max(overlap_lo, mid + 1), overlap_hi, rows));
            }
        }
    }
    if (hi < qhi) {
        int missing_lo = std::max(qlo, hi + 1);
        append_result(res, repeat_value(init, rows * length(missing_lo, qhi)));
    }
    return *res;
}

T query(OuterNode *n, int qlo, int qhi) {
    if (n == nullptr) {
        return repeat_value(init, width * length(qlo, qhi));
    }
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    std::optional<T> res;
    if (qlo < lo) {
        int missing_hi = std::min(qhi, lo - 1);
        append_result(res, repeat_value(init, width * length(qlo, missing_hi)));
    }
    int overlap_lo = std::max(qlo, lo), overlap_hi = std::min(qhi, hi);
    if (overlap_lo <= overlap_hi) {
        if (overlap_lo == lo && overlap_hi == hi) {
            append_result(res, query(&(n->root), tgt_c1, tgt_c2, length(lo, hi)));
        } else {
            if (overlap_lo <= mid) {
                append_result(res, query(n->left, overlap_lo, std::min(overlap_hi, mid)));
            }
            if (mid < overlap_hi) {
                append_result(res, query(n->right, std::max(overlap_lo, mid + 1), overlap_hi));
            }
        }
    }
    return *res;
}

```



```

    if (hi < qhi) {
        int missing_lo = std::max(qlo, hi + 1);
        append_result(res, repeat_value(init, width * length(missing_lo, qhi)));
    }
    return *res;
}

void update(InnerNode *n, int c, const T &d, bool leaf_row, int64_t rows) {
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    if (lo == hi) {
        if (leaf_row) {
            n->value = apply_delta(n->value, d);
        } else {
            n->value = d;
        }
        return;
    }
    InnerNode *&target = (c <= mid) ? n->left : n->right;
    if (target == nullptr) {
        target = new InnerNode(c, c, repeat_value(init, rows));
    }
    if (target->low <= c && c <= target->high) {
        update(target, c, d, leaf_row, rows);
    } else {
        int split_lo = lo, split_hi = hi, split_mid = mid;
        do {
            if (c <= split_mid) {
                split_hi = split_mid;
            } else {
                split_lo = split_mid + 1;
            }
            split_mid = split_lo + (split_hi - split_lo) / 2;
        } while ((c <= split_mid) == (target->low <= split_mid));
        InnerNode *tmp =
            new InnerNode(split_lo, split_hi, repeat_value(init, rows * length(split_lo, split_hi)));
        if (target->low <= split_mid) {
            tmp->left = target;
        } else {
            tmp->right = target;
        }
        target = tmp;
        update(tmp, c, d, leaf_row, rows);
    }
    T left_value =
        (n->left != nullptr) ? n->left->value : repeat_value(init, rows * length(lo, mid));
    T right_value =
        (n->right != nullptr) ? n->right->value : repeat_value(init, rows * length(mid + 1, hi));
    n->value = combine(left_value, right_value);
}

void update(OuterNode *n, int r, int c, const T &d) {
    int lo = n->low, hi = n->high, mid = lo + (hi - lo) / 2;
    int64_t rows = length(lo, hi);
    if (lo == hi) {
        update(&(n->root), c, d, true, 1);
        return;
    }
    if (r <= mid) {
        if (n->left == nullptr) {
            n->left = new OuterNode(lo, mid, repeat_value(init, length(lo, mid) * length(0, C)));
        }
        update(n->left, r, c, d);
    } else {
        if (n->right == nullptr) {
            n->right =
                new OuterNode(mid + 1, hi, repeat_value(init, length(mid + 1, hi) * length(0, C)));
        }
        update(n->right, r, c, d);
    }
}

```

```

    }
    T value = repeat_value(init, rows);
    if (n->left != nullptr || n->right != nullptr) {
        tgt_c1 = tgt_c2 = c;
        T left_value = (n->left != nullptr) ? query(&(n->left->root), c, c, length(lo, mid))
                                              : repeat_value(init, length(lo, mid));
        T right_value = (n->right != nullptr) ? query(&(n->right->root), c, c, length(mid + 1, hi))
                                                : repeat_value(init, length(mid + 1, hi));
        value = combine(left_value, right_value);
    }
    update(&(n->root), c, value, false, rows);
}

static void clean_up(InnerNode *n) {
    if (n != nullptr) {
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

static void clean_up(OuterNode *n) {
    if (n != nullptr) {
        clean_up(n->root.left);
        clean_up(n->root.right);
        clean_up(n->left);
        clean_up(n->right);
        delete n;
    }
}

public:
    explicit SegTree2D(const T &v = T())
        : root(new OuterNode(0, R, repeat_value(v, length(0, R) * length(0, C))), init(v) {})

    ~SegTree2D() { clean_up(root); }
    SegTree2D(const SegTree2D &) = delete;
    SegTree2D &operator=(const SegTree2D &) = delete;
    T at(int r, int c) { return query(r, c, r, c); }

    T query(int r1, int c1, int r2, int c2) {
        tgt_r1 = r1;
        tgt_c1 = c1;
        tgt_r2 = r2;
        tgt_c2 = c2;
        width = length(c1, c2);
        return query(root, r1, r2);
    }

    void update(int r, int c, const T &d) { update(root, r, c, d); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    SegTree2D<int> t(0);
    t.update(0, 0, 7);
    t.update(0, 1, 6);
    t.update(1, 0, 5);
    t.update(1, 1, 4);
    t.update(2, 1, 1);
    t.update(2, 2, 9);
}

```

```

cout << "Values:" << endl;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        cout << t.at(i, j) << " ";
    }
    cout << endl;
}
assert(t.query(0, 0, 0, 1) == 6);
assert(t.query(0, 0, 1, 0) == 5);
assert(t.query(1, 1, 2, 2) == 0);
assert(t.query(0, 0, 1000000000, 1000000000) == 0);
t.update(500000000, 500000000, -100);
assert(t.query(0, 0, 1000000000, 1000000000) == -100);
return 0;
}

```

Example Output

```

Values:
7 6 0
5 4 0
0 1 9

```

2.5.4 2D Range Tree

Maintain a set of two-dimensional points while supporting queries for all points that fall inside given rectangular regions. This implementation uses `std::pair` to represent points, requiring operators `<` and `==` to be defined on the numeric template type. A balanced tree over the points sorted by x stores at each node its subrange of points sorted by y , merged from its children's lists as in a merge sort tree. A query decomposes the x -range into $O(\log n)$ nodes and binary searches each node's list for the matching y -range.

Use this for static point-reporting queries when guaranteed worst-case bounds are more important than memory. Compared with a range k-d tree, it uses more space but gives $O(\log^2(n) + m)$ query time regardless of point distribution; the k-d tree is lighter and often faster on typical inputs, but its pruning is more distribution-dependent.

- `RangeTree<T>(lo, hi)` constructs a set from two random-access iterators to `std::pair` as a range $[lo, hi)$ of points.
- `query(x1, y1, x2, y2, f)` calls the function `f(i, p)` on each point in the set that falls into the rectangular region consisting of rows from `x1` to `x2`, inclusive, and columns from `y1` to `y2`, inclusive. The first argument to `f` is the 0-based index of the point in the original range given to the constructor. The second argument is the point itself as an `std::pair`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of points.
- $O(\log^2(n) + m)$ per call to `query()`, where m is the number of points that are reported by the query.

Space Complexity:

- $O(n \log n)$ for storage of the points.
- $O(\log^2(n))$ auxiliary stack space for `query()`.

```

#include <algorithm>
#include <iterator>
#include <utility>
#include <vector>

template<typename T>
class RangeTree {
    using point = std::pair<T, T>;
    using colindex = std::pair<int, T>;

```

```

std::vector<point> points;
std::vector<std::vector<colindex>> columns;

static inline bool comp1(const colindex &a, const colindex &b) { return a.second < b.second; }
static inline bool comp2(const colindex &a, const T &v) { return a.second < v; }

void build(int n, int lo, int hi) {
    if (points[lo].first == points[hi].first) {
        for (int i = lo; i <= hi; i++) {
            columns[n].emplace_back(i, points[i].second);
        }
        return;
    }
    int l = n * 2 + 1, r = n * 2 + 2, mid = lo + (hi - lo) / 2;
    build(l, lo, mid);
    build(r, mid + 1, hi);
    columns[n].resize(columns[l].size() + columns[r].size());
    std::merge(
        columns[l].begin(), columns[l].end(), columns[r].begin(), columns[r].end(),
        columns[n].begin(), comp1
    );
}

// Helper variables for query().
T x1, y1, x2, y2;

template<typename Fn>
void query(int n, int lo, int hi, Fn f) {
    if (points[hi].first < x1 || x2 < points[lo].first) {
        return;
    }
    if (!(points[lo].first < x1 || x2 < points[hi].first)) {
        if (!columns[n].empty() && !(y2 < y1)) {
            auto it = std::lower_bound(columns[n].begin(), columns[n].end(), y1, comp2);
            for (; it != columns[n].end() && it->second <= y2; ++it) {
                f(it->first, points[it->first]);
            }
        }
    }
    else if (lo != hi) {
        int mid = lo + (hi - lo) / 2;
        query(n * 2 + 1, lo, mid, f);
        query(n * 2 + 2, mid + 1, hi, f);
    }
}

public:
template<typename It>
RangeTree(It lo, It hi) : points(lo, hi) {
    int n = std::distance(lo, hi);
    columns.resize(4 * n + 1);
    std::sort(points.begin(), points.end());
    build(0, 0, n - 1);
}

template<typename Fn>
void query(const T &x1, const T &y1, const T &x2, const T &y2, Fn f) {
    this->x1 = x1;
    this->y1 = y1;
    this->x2 = x2;
    this->y2 = y2;
    query(0, 0, points.size() - 1, f);
}
};

```

Example Usage

```
#include <iostream>
using namespace std;

void print(int i, const pair<int, int> &p) {
    cout << "(" << p.first << ", " << p.second << ") ";
}

int main() {
    vector<pair<int, int>> v{{1, 4}, {5, 4}, {2, 2}, {3, 1}, {6, -5},
                          {5, -1}, {3, -3}, {-1, -2}, {-1, -1}, {2, -1}};
    RangeTree<int> t(v.begin(), v.end());
    t.query(-1, -1, 2, 5, print);
    cout << endl;
    t.query(1, 1, 4, 8, print);
    cout << endl;
    return 0;
}
```

Example Output

```
(-1, -1) (2, -1) (2, 2) (1, 4)
(1, 4) (2, 2) (3, 1)
```

2.5.5 K-d Tree for 2D Range Query

Maintain a static set of two-dimensional points while supporting rectangle reporting queries. A k-d tree recursively splits points by alternating coordinates and stores a bounding box for each subtree, allowing whole subtrees to be accepted or pruned during a query.

This implementation uses `std::pair` to represent points, requiring operators `<` and `==` to be defined on the numeric template type.

Use this for static point-reporting queries when $O(n)$ space and good average performance are more important than a strict worst-case guarantee. Use the 2D range tree instead when adversarial point sets or query rectangles are expected and the extra $O(n \log n)$ space is acceptable.

- `RangeKDTree<T>(lo, hi)` constructs a set from two random-access iterators to `std::pair` as a range `[lo, hi)` of points.
- `query(x1, y1, x2, y2, f)` calls the function `f(i, p)` on each point in the set that falls into the rectangular region consisting of rows from `x1` to `x2`, inclusive, and columns from `y1` to `y2`, inclusive. The first argument to `f` is the 0-based index of the point in the original range given to the constructor. The second argument is the point itself as an `std::pair`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of points.
- $O(\log(n) + m)$ on average per call to `query()`, where m is the number of points that are reported by the query.

Space Complexity:

- $O(n)$ for storage of the points.
- $O(\log n)$ auxiliary stack space for `query()`.

```
#include <algorithm>
#include <utility>
#include <vector>

template<typename T>
class RangeKDTree {
```

```

using point = std::pair<T, T>;

static inline bool comp1(const point &a, const point &b) { return a.first < b.first; }
static inline bool comp2(const point &a, const point &b) { return a.second < b.second; }

std::vector<point> tree, minp, maxp;
std::vector<int> l_index, h_index;

void build(int lo, int hi, bool div_x) {
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;
    std::nth_element(
        tree.begin() + lo, tree.begin() + mid, tree.begin() + hi, div_x ? comp1 : comp2
    );
    l_index[mid] = lo;
    h_index[mid] = hi;
    minp[mid].first = maxp[mid].first = tree[lo].first;
    minp[mid].second = maxp[mid].second = tree[lo].second;
    for (int i = lo + 1; i < hi; i++) {
        minp[mid].first = std::min(minp[mid].first, tree[i].first);
        minp[mid].second = std::min(minp[mid].second, tree[i].second);
        maxp[mid].first = std::max(maxp[mid].first, tree[i].first);
        maxp[mid].second = std::max(maxp[mid].second, tree[i].second);
    }
    build(lo, mid, !div_x);
    build(mid + 1, hi, !div_x);
}

// Helper variables for query().
T x1, y1, x2, y2;

template<typename Fn>
void query(int lo, int hi, Fn f) {
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;
    T ax = minp[mid].first, ay = minp[mid].second;
    T bx = maxp[mid].first, by = maxp[mid].second;
    if (x2 < ax || bx < x1 || y2 < ay || by < y1) {
        return;
    }
    if (!(ax < x1 || x2 < bx || ay < y1 || y2 < by)) {
        for (int i = l_index[mid]; i < h_index[mid]; i++) {
            f(tree[i]);
        }
        return;
    }
    query(lo, mid, f);
    query(mid + 1, hi, f);
    if (tree[mid].first < x1 || x2 < tree[mid].first || tree[mid].second < y1 ||
        y2 < tree[mid].second) {
        return;
    }
    f(tree[mid]);
}

public:
template<typename It>
RangeKDTree(It lo, It hi) : tree(lo, hi) {
    int n = std::distance(lo, hi);
    l_index.resize(n);
    h_index.resize(n);
    minp.resize(n);
    maxp.resize(n);
    build(0, n, true);
}

```

```

}

template<typename Fn>
void query(const T &x1, const T &y1, const T &x2, const T &y2, Fn f) {
    this->x1 = x1;
    this->y1 = y1;
    this->x2 = x2;
    this->y2 = y2;
    query(0, static_cast<int>(tree.size()), f);
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

void print(const pair<int, int> &p) {
    cout << "(" << p.first << ", " << p.second << ") ";
}

int main() {
    vector<pair<int, int>> v{{1, 4}, {5, 4}, {2, 2}, {3, 1}, {6, -5},
                           {5, -1}, {3, -3}, {-1, -2}, {-1, -1}, {2, -1}};
    RangeKDTree<int> t(v.begin(), v.end());
    t.query(-1, -1, 2, 5, print);
    cout << endl;
    t.query(1, 1, 4, 8, print);
    cout << endl;
    return 0;
}

```

Example Output

```

(2, -1) (1, 4) (2, 2) (-1, -1)
(1, 4) (2, 2) (3, 1)

```

2.5.6 K-d Tree for Nearest Neighbor

Maintain a static set of two-dimensional points while supporting nearest-neighbor queries. A k-d tree recursively splits points by the coordinate with larger spread, searches the more promising side first, and only explores the other side when its splitting plane can still improve the answer.

This implementation uses `std::pair` to represent points, requiring operators `<`, `==`, `-`, and `long double` casting to be defined on the numeric template type.

Use this for static nearest-neighbor queries on point sets in low dimensions. It is a geometric search tree rather than an aggregate structure: for rectangle reporting use the range k-d tree or 2D range tree, and for grid cell updates or rectangle sums/minima use a Fenwick tree, quadtree, or 2D segment tree.

- `NearestKDTree<T>(lo, hi)` constructs a set from two random-access iterators to `std::pair` as a range `[lo, hi)` of points.
- `nearest(x, y, can_equal)` returns a point in the set that is closest to `(x, y)` by Euclidean distance. This may be equal to `(x, y)` only if `can_equal` is `true`.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of points.
- $O(\log n)$ on average per call to `nearest()`.

Space Complexity:

- $O(n)$ for storage of the points.
- $O(\log n)$ auxiliary stack space for `nearest()`.

```

#include <algorithm>
#include <cfloat>
#include <stdexcept>
#include <utility>
#include <vector>

template<typename T>
class NearestKDTree {
    using point = std::pair<T, T>;

    static inline bool comp1(const point &a, const point &b) { return a.first < b.first; }
    static inline bool comp2(const point &a, const point &b) { return a.second < b.second; }

    std::vector<point> tree;
    std::vector<bool> div_x;

    void build(int lo, int hi) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T minx, maxx, miny, maxy;
        minx = maxx = tree[lo].first;
        miny = maxy = tree[lo].second;
        for (int i = lo + 1; i < hi; i++) {
            minx = std::min(minx, tree[i].first);
            miny = std::min(miny, tree[i].second);
            maxx = std::max(maxx, tree[i].first);
            maxy = std::max(maxy, tree[i].second);
        }
        div_x[mid] = !((maxx - minx) < (maxy - miny));
        std::nth_element(
            tree.begin() + lo, tree.begin() + mid, tree.begin() + hi, div_x[mid] ? comp1 : comp2
        );
        if (lo + 1 == hi) {
            return;
        }
        build(lo, mid);
        build(mid + 1, hi);
    }

    // Helper variables for nearest().
    long double min_dist;
    int id;

    void nearest(int lo, int hi, const T &x, const T &y, bool can_equal) {
        if (lo >= hi) {
            return;
        }
        int mid = lo + (hi - lo) / 2;
        T dx = x - tree[mid].first, dy = y - tree[mid].second;
        long double d = dx * static_cast<long double>(dx) + dy * static_cast<long double>(dy);
        if (d < min_dist && (can_equal || d != 0)) {
            min_dist = d;
            id = mid;
        }
        if (lo + 1 == hi) {
            return;
        }
        d = static_cast<long double>((div_x[mid] ? dx : dy));
        int l1 = lo, r1 = mid, l2 = mid + 1, r2 = hi;
        if (d > 0) {
            std::swap(l1, l2);
        }
    }
};

```



```

        std::swap(r1, r2);
    }
    nearest(l1, r1, x, y, can_equal);
    if (d * static_cast<long double>(d) < min_dist) {
        nearest(l2, r2, x, y, can_equal);
    }
}

public:
    template<typename It>
    NearestKDTree(It lo, It hi) : tree(lo, hi) {
        int n = std::distance(lo, hi);
        if (n <= 1) {
            throw std::runtime_error("K-d tree must have at least 2 points.");
        }
        div_x.resize(n);
        build(0, n);
    }

    point nearest(const T &x, const T &y, bool can_equal = true) {
        min_dist = LDBL_MAX;
        nearest(0, static_cast<int>(tree.size()), x, y, can_equal);
        return tree[id];
    }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<pair<int, int>> p{{0, 2}, {0, 3}, {-1, 0}};
    NearestKDTree<int> t(p.begin(), p.end());
    assert(t.nearest(0, 2, true) == (pair<int, int>{0, 2}));
    assert(t.nearest(0, 2, false) == (pair<int, int>{0, 3}));
    assert(t.nearest(0, 0) == (pair<int, int>{-1, 0}));
    assert(t.nearest(-10000, 0) == (pair<int, int>{-1, 0}));
    return 0;
}

```

2.6 Fenwick Trees

2.6.1 Fenwick Tree (Point Update, Range Query)

Maintain a numerical array while supporting point increments and range-sum queries. A Fenwick tree (also called a binary indexed tree) stores partial prefix sums: each internal entry `tree[i]` holds the sum of a block of indices whose length is the lowest set bit of `i`. A point update touches $O(\log n)$ covering blocks, and a prefix-sum query combines $O(\log n)$ disjoint blocks.

When prefix sums are nondecreasing, `max_prefix(c)` answers prefix-boundary queries by binary lifting: it walks the implicit power-of-two block structure to find the longest prefix with sum at most `c`. This lets the same structure double as a dynamic multiset over a bounded integer domain, useful for coordinate-compressed k -th element lookups and online rank queries.

- `Fenwick<T>(n)` constructs an array with 0-based indices $[0, n)$, with all values initialized to 0.
- `size()` returns the size of the array.
- `add(i, x)` adds `x` to the value at index `i`.
- `set(i, x)` assigns the value at index `i` to `x`.

- `at(i)` returns the value at index `i`.
- `sum(hi)` returns the sum of all values at indices from 0 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.
- `max_prefix(c)` returns the largest boundary `hi` such that `sum(0, hi - 1) <= c`, assuming prefix sums are nondecreasing. It may return any value from 0 to `size()`.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `add()`, both `sum()` functions, `set()`, `at()`, and `max_prefix()`.

Space Complexity:

- $O(n)$ for storage of the Fenwick tree.
- $O(1)$ auxiliary for all operations.

```
#include <vector>

template<typename T>
class Fenwick {
    int len;
    std::vector<T> tree;

public:
    explicit Fenwick(int n) : len(n), tree(n + 1) {}

    int size() const { return len; }

    void add(int i, const T &x) {
        for (++i; i <= len; i += i & -i) {
            tree[i] += x;
        }
    }

    T sum(int hi) const {
        T res = 0;
        for (++hi; hi > 0; hi -= hi & -hi) {
            res += tree[hi];
        }
        return res;
    }

    void set(int i, const T &x) { add(i, x - at(i)); }
    T at(int i) const { return sum(i, i); }
    T sum(int lo, int hi) const { return sum(hi) - sum(lo - 1); }

    int max_prefix(T c) const {
        int pos = 0, pw = 1;
        while (pw * 2 <= len) {
            pw *= 2;
        }
        for (; pw > 0; pw >>= 1) {
            if (pos + pw <= len && tree[pos + pw] <= c) {
                c -= tree[pos + pw];
                pos += pw;
            }
        }
        return pos;
    }
};
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    vector<int> a{10, 1, 2, 3, 4};
    Fenwick<int> t(5);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        t.set(i, a[i]);
    }
    t.add(0, -5); // a is now {5, 1, 2, 3, 4}.
    assert(t.at(0) == 5);
    assert(t.sum(1, 3) == 6); // 1 + 2 + 3
    assert(t.sum(4) == 15); // 5 + 1 + 2 + 3 + 4

    // With nonnegative values, the tree doubles as a frequency table for order-statistic queries.
    Fenwick<int> freq(8);
    freq.add(1, 1); // One element of value 1.
    freq.add(3, 3); // Three elements of value 3.
    freq.add(6, 1); // One element of value 6.
    assert(freq.max_prefix(0) == 1); // The longest prefix with count <= 0 is [0, 1).
    assert(freq.max_prefix(3) == 3); // Prefix [0, 3) has count 1; [0, 4) has count 4.
    assert(freq.max_prefix(4) == 6); // Prefix [0, 6) has count 4; [0, 7) has count 5.
    return 0;
}
```

2.6.2 Fenwick Tree (Range Update, Point Query)

Maintain a numerical array while supporting range increments and point queries. This is done by storing the difference array in a Fenwick tree: adding x to $[lo, hi]$ adds $+x$ at lo and $-x$ after hi , and the value at index i is the prefix sum of those differences through i .

- `FenwickRUPQ<T>(n)` constructs an array with 0-based indices $[0, n)$, with values set to 0.
- `size()` returns the size of the array.
- `at(i)` returns the value at index i .
- `add(i, x)` adds x to the value at index i .
- `add(lo, hi, x)` adds x to the values at all indices from lo to hi , inclusive.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to `at()` and both `add()` functions.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
#include <vector>

template<typename T>
class FenwickRUPQ {
    int len;
    std::vector<T> tree;

public:
    explicit FenwickRUPQ(int n) : len(n), tree(n + 2) {}

    int size() const { return len; }
```

```

T at(int i) const {
    T res = 0;
    for (i++; i > 0; i -= i & -i) {
        res += tree[i];
    }
    return res;
}

void add(int i, const T &x) {
    for (i++; i <= len + 1; i += i & -i) {
        tree[i] += x;
    }
}

void add(int lo, int hi, const T &x) {
    add(lo, x);
    add(hi + 1, -x);
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    FenwickRUPQ<int> t(5);
    t.add(0, 1, 5);
    t.add(1, 2, 5);
    t.add(2, 4, 10);
    cout << "Values: ";
    for (int i = 0; i < t.size(); i++) {
        cout << t.at(i) << " ";
    }
    cout << endl;
    return 0;
}

```

Example Output

Values: 5 10 15 10 10

2.6.3 Fenwick Tree (Range Update, Range Query)

Maintain a numerical array while supporting range increments and range-sum queries. This uses two Fenwick trees to recover prefix sums after difference-array updates: if the difference array stores range additions, then the prefix sum through `hi` can be written as `hi*sum(t1, hi) - sum(t2, hi)`.

- `FenwickRUPQ<T>(n)` constructs an array with 0-based indices `[0, n)`, with values set to 0.
- `size()` returns the size of the array.
- `at(i)` returns the value at index `i`.
- `add(i, x)` increments the value at index `i` by `x`.
- `add(lo, hi, x)` adds `x` to the values at all indices from `lo` to `hi`, inclusive.
- `set(i, x)` assigns the value at index `i` to `x`.
- `sum(hi)` returns the sum of all values at indices from 0 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.
- `max_prefix(c)` returns the largest boundary `hi` such that `sum(0, hi - 1) <= c`, assuming prefix sums are nondecreasing. It may return any value from 0 to `size()`.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the size of the array.
- $O(1)$ per call to `size()`.
- $O(\log n)$ per call to all other operations.

Space Complexity:

- $O(n)$ for storage of the array elements.
- $O(1)$ auxiliary for all operations.

```
#include <vector>

template<typename T>
class FenwickRURQ {
    int len;
    std::vector<T> t1, t2;

    T sum(const std::vector<T> &tree, int i) {
        T res = 0;
        for (; i != 0; i -= i & -i) {
            res += tree[i];
        }
        return res;
    }

    void add(std::vector<T> &tree, int i, const T &x) {
        for (; i <= len + 1; i += i & -i) {
            tree[i] += x;
        }
    }

public:
    explicit FenwickRURQ(int n) : len(n), t1(n + 2), t2(n + 2) {}

    int size() const { return len; }

    void add(int lo, int hi, const T &x) {
        lo++;
        hi++;
        add(t1, lo, x);
        add(t1, hi + 1, -x);
        add(t2, lo, x * (lo - 1));
        add(t2, hi + 1, -x * hi);
    }

    void add(int i, const T &x) { return add(i, i, x); }
    void set(int i, const T &x) { add(i, x - at(i)); }

    T sum(int hi) {
        hi++;
        return hi * sum(t1, hi) - sum(t2, hi);
    }

    T sum(int lo, int hi) { return sum(hi) - sum(lo - 1); }
    T at(int i) { return sum(i, i); }

    int max_prefix(T c) {
        T s1 = 0, s2 = 0;
        int pos = 0, pw = 1;
        while (pw * 2 <= len) {
            pw *= 2;
        }
        for (; pw > 0; pw >>= 1) {
            int next = pos + pw;
            if (next <= len) {
                T ns1 = s1 + t1[next], ns2 = s2 + t2[next];
```

```

        if (next * ns1 - ns2 <= c) {
            s1 = ns1;
            s2 = ns2;
            pos = next;
        }
    }
    return pos;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<int> a{10, 1, 2, 3, 4};
    FenwickRURQ<int> t(5);
    for (int i = 0; i < t.size(); i++) {
        t.set(i, a[i]);
    }
    t.add(0, 2, 5);
    t.set(3, -5);
    cout << "Values: ";
    for (int i = 0; i < t.size(); i++) {
        cout << t.at(i) << " ";
    }
    cout << endl;
    assert(t.sum(0, 4) == 27);
    FenwickRURQ<int> freq(8);
    freq.add(1, 1, 1);
    freq.add(3, 3, 3);
    freq.add(6, 6, 1);
    assert(freq.max_prefix(0) == 1);
    assert(freq.max_prefix(3) == 3);
    assert(freq.max_prefix(4) == 6);
    return 0;
}

```

Example Output

Values: 15 6 7 -5 4

2.6.4 Sparse Fenwick Tree (Range Update, Range Query)

Maintain a numerical array over a huge index range while supporting range increments and range-sum queries. This is the sparse version of the two-tree Fenwick range-update/range-query trick: only Fenwick nodes reached by previous operations are stored, using `std::unordered_map` instead of dense vectors.

- `SparseFenwick<T, N>()` constructs an array with 0-based indices $[0, \mathbb{N})$, implicitly initialized to 0.
- `at(i)` returns the value at index `i`.
- `add(i, x)` adds `x` to the value at index `i`.
- `add(lo, hi, x)` adds `x` to the values at all indices from `lo` to `hi`, inclusive.
- `set(i, x)` assigns the value at index `i` to `x`.
- `sum(hi)` returns the sum of all values at indices from 0 to `hi`, inclusive.
- `sum(lo, hi)` returns the sum of all values at indices from `lo` to `hi`, inclusive.
- `max_prefix(c)` returns the largest boundary `hi` such that `sum(0, hi - 1) <= c`, assuming prefix sums are nondecreasing. It may return any value from 0 to `N`.

Time Complexity: $O(\log N)$ per call to all member functions.

Space Complexity:

- $O(n \log n)$ for storage of the array elements, where n is the number of distinct indices that have been accessed across all of the operations so far.
- $O(1)$ auxiliary for all operations.

```
#include <unordered_map>

template<typename T, int N = 1000000001>
class SparseFenwick {
    std::unordered_map<int, T> tmul, tadd;

    T get(const std::unordered_map<int, T> &tree, int i) const {
        if (auto it = tree.find(i); it != tree.end()) {
            return it->second;
        }
        return 0;
    }

    void add_helper(int at, const T &mul, const T &add) {
        for (int i = at; i <= N; i += i & -i) {
            tmul[i] += mul;
            tadd[i] += add;
        }
    }

public:
    void add(int lo, int hi, const T &x) {
        lo++;
        hi++;
        add_helper(lo, x, x * (lo - 1));
        add_helper(hi + 1, -x, -x * hi);
    }

    void add(int i, const T &x) { add(i, i, x); }
    void set(int i, const T &x) { add(i, x - at(i)); }

    T sum(int hi) {
        T mul = 0, add = 0;
        hi++;
        for (int i = hi; i > 0; i -= i & -i) {
            mul += get(tmul, i);
            add += get(tadd, i);
        }
        return mul * hi - add;
    }

    T sum(int lo, int hi) { return sum(hi) - sum(lo - 1); }
    T at(int i) { return sum(i, i); }

    int max_prefix(T c) {
        T mul = 0, add = 0;
        int pos = 0, pw = 1;
        while (pw * 2 <= N) {
            pw *= 2;
        }
        for (; pw > 0; pw >>= 1) {
            int next = pos + pw;
            if (next <= N) {
                int key = next;
                T nmul = mul + get(tmul, key), nadd = add + get(tadd, key);
                if (key * nmul - nadd <= c) {
                    mul = nmul;
                    add = nadd;
                    pos = next;
                }
            }
        }
    }
};
```

```

    }
  }
  return pos;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> a{10, 1, 2, 3, 4};
    SparseFenwick<int> t;
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        t.set(i, a[i]);
    }
    t.add(0, 2, 5);
    t.set(3, -5);
    cout << "Values: ";
    for (int i = 0; i < 5; i++) {
        cout << t.at(i) << " ";
    }
    cout << endl;
    assert(t.sum(0, 4) == 27);
    t.add(500000001, 500000010, 3);
    t.add(500000011, 500000015, 5);
    t.set(500000000, 10);
    assert(t.sum(0, 1000000000) == 92);

    SparseFenwick<long long> freq;
    freq.add(1, 1, 1);
    freq.add(3, 3, 3);
    freq.add(6, 6, 1);
    assert(freq.max_prefix(0) == 1);
    assert(freq.max_prefix(3) == 3);
    assert(freq.max_prefix(4) == 6);
    return 0;
}

```

Example Output

Values: 15 6 7 -5 4

2.6.5 2D Fenwick Tree

Maintain a two-dimensional numerical array while supporting point increments and rectangle-sum queries. This is the two-dimensional form of the standard Fenwick tree: each internal entry stores a rectangular block sum, and a prefix query combines $O(\log R \cdot \log C)$ disjoint blocks.

Choose among the three 2D Fenwick trees by coordinate range and update style. Use this dense version when the rows and columns are small enough to allocate the full grid: it has the simplest code and the best constants, supporting point updates and rectangle sums. Use the sparse 2D Fenwick tree (next section) when the coordinates are too large to store densely and the updates arrive online; it relies on hashing and is the only one of the three that also supports rectangle updates. Use the offline 2D Fenwick tree when the coordinates are large but every updated cell is known before the queries begin, which lets it coordinate-compress instead of hashing for markedly better constants than the sparse version. For non-additive aggregates such as min/max with custom updates, use a 2D segment tree or quadtree instead, since Fenwick-tree algebra relies on addition and subtraction.

- `Fenwick2D<T>(rows, cols)` constructs a `rows` by `cols` array with 0-based indices, with all values initialized to 0.
- `size_rows()` and `size_cols()` return the number of rows and columns.
- `add(r, c, x)` adds `x` to the value at index `(r, c)`.
- `set(r, c, x)` assigns `x` to the value at index `(r, c)`.
- `at(r, c)` returns the value at index `(r, c)`.
- `sum(r, c)` returns the sum of the rectangle with corners `(0, 0)` and `(r, c)`, inclusive.
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with upper-left corner `(r1, c1)` and lower-right corner `(r2, c2)`, inclusive.

Time Complexity:

- $O(R \cdot C)$ per call to the constructor, where R and C are the number of rows and columns.
- $O(1)$ per call to `size_rows()` and `size_cols()`.
- $O(\log(R) \cdot \log(C))$ per call to all other operations.

Space Complexity:

- $O(R \cdot C)$ for storage of the Fenwick tree.
- $O(1)$ auxiliary for all operations.

```
#include <vector>

template<typename T>
class Fenwick2D {
    int rows, cols;
    std::vector<std::vector<T>> tree;

public:
    Fenwick2D(int rows, int cols)
        : rows(rows), cols(cols), tree(rows + 1, std::vector<T>(cols + 1)) {}

    int size_rows() const { return rows; }
    int size_cols() const { return cols; }

    void add(int r, int c, const T &x) {
        for (int i = r + 1; i <= rows; i += i & -i) {
            for (int j = c + 1; j <= cols; j += j & -j) {
                tree[i][j] += x;
            }
        }
    }

    T sum(int r, int c) const {
        T res = 0;
        for (int i = r + 1; i > 0; i -= i & -i) {
            for (int j = c + 1; j > 0; j -= j & -j) {
                res += tree[i][j];
            }
        }
        return res;
    }

    T sum(int r1, int c1, int r2, int c2) const {
        return sum(r2, c2) - sum(r1 - 1, c2) - sum(r2, c1 - 1) + sum(r1 - 1, c1 - 1);
    }

    T at(int r, int c) const { return sum(r, c, r, c); }
    void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }
};
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    Fenwick2D<int> t(3, 3);
    t.set(0, 0, 5);
    t.set(0, 1, 6);
    t.set(1, 0, 7);
    t.add(2, 2, 9);
    t.add(1, 0, -4); // Value at (1, 0) is now 3.
    assert(t.at(1, 0) == 3);
    assert(t.sum(0, 0, 0, 1) == 11); // 5 + 6
    assert(t.sum(0, 0, 1, 0) == 8); // 5 + 3
    assert(t.sum(0, 0, 2, 2) == 23); // 5 + 6 + 3 + 9
    return 0;
}
```

2.6.6 2D Sparse Fenwick Tree

Maintain a 2D numerical array over a huge index range while supporting rectangle increments and rectangle-sum queries. This is the sparse two-dimensional analogue of range-update/range-query Fenwick trees: four sparse trees store the inclusion-exclusion coefficients needed to recover any prefix rectangle sum.

Choose this for huge sparse grids when the operation is additive: point or rectangle increments and rectangle sums. It avoids allocating the dense $R \times C$ table of the dense 2D Fenwick tree, and unlike the dense and offline variants it supports rectangle updates, not just point updates. Prefer it over the offline 2D Fenwick tree only when updates arrive online; if every updated cell is known in advance, that version compresses coordinates instead of hashing and runs with better constants. It is less general than the sparse 2D segment tree or quadtrees because Fenwick-tree algebra relies on addition and subtraction.

- `SparseFenwick2D<T, R, C>()` constructs a 2D array over rows 0 to $R - 1$ (inclusive) and columns 0 to $C - 1$ (inclusive). All values are implicitly initialized to 0, as nodes are allocated lazily as indices are touched.
- `add(r, c, x)` adds x to the value at index (r, c) .
- `add(r1, c1, r2, c2, x)` adds x to all indices in the rectangle with upper-left corner $(r1, c1)$ and lower-right corner $(r2, c2)$.
- `set(r, c, x)` assigns x to the value at index (r, c) .
- `sum(r, c)` returns the sum of the rectangle with upper-left corner $(0, 0)$ and lower-right corner (r, c) .
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with upper-left corner $(r1, c1)$ and lower-right corner $(r2, c2)$.
- `at(r, c)` returns the value at index (r, c) .

Time Complexity: $O(\log(R) \cdot \log(C))$ per call to all member functions.

Space Complexity:

- $O(n \cdot \log(R) \cdot \log(C))$ for storage of the array elements, where n is the number of distinct indices that have been accessed across all of the operations so far.
- $O(1)$ auxiliary for all operations.

```
#include <cstdint>
#include <unordered_map>
#include <utility>

template<typename T, int R = 1000000001, int C = 1000000001>
class SparseFenwick2D {
    std::unordered_map<int64_t, T> t1, t2, t3, t4;

    template<typename Map>
```

```

static T get(const Map &tree, int r, int c) {
    auto it = tree.find(static_cast<int64_t>(r) * (C + 1) + c);
    return it == tree.end() ? T() : it->second;
}

template<typename Map>
static void add(Map &tree, int r, int c, const T &x) {
    for (int i = r + 1; i <= R; i += i & -i) {
        for (int j = c + 1; j <= C; j += j & -j) {
            tree[static_cast<int64_t>(i) * (C + 1) + j] += x;
        }
    }
}

void add_helper(int r, int c, const T &x) {
    add(t1, 0, 0, x);
    add(t1, 0, c, -x);
    add(t2, 0, c, x * c);
    add(t1, r, 0, -x);
    add(t3, r, 0, x * r);
    add(t1, r, c, x);
    add(t2, r, c, -x * c);
    add(t3, r, c, -x * r);
    add(t4, r, c, x * r * c);
}

public:
void add(int r1, int c1, int r2, int c2, const T &x) {
    add_helper(r2 + 1, c2 + 1, x);
    add_helper(r1, c2 + 1, -x);
    add_helper(r2 + 1, c1, -x);
    add_helper(r1, c1, x);
}

void add(int r, int c, const T &x) { add(r, c, r, c, x); }
void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }

T sum(int r, int c) {
    r++;
    c++;
    T s1 = 0, s2 = 0, s3 = 0, s4 = 0;
    for (int i = r; i > 0; i -= i & -i) {
        for (int j = c; j > 0; j -= j & -j) {
            s1 += get(t1, i, j);
            s2 += get(t2, i, j);
            s3 += get(t3, i, j);
            s4 += get(t4, i, j);
        }
    }
    return s1 * r * c + s2 * r + s3 * c + s4;
}

T sum(int r1, int c1, int r2, int c2) {
    return sum(r2, c2) + sum(r1 - 1, c1 - 1) - sum(r1 - 1, c2) - sum(r2, c1 - 1);
}

T at(int r, int c) { return sum(r, c, r, c); }
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {

```

```

SparseFenwick2D<int> t;
t.set(0, 0, 5);
t.set(0, 1, 6);
t.set(1, 0, 7);
t.add(2, 2, 9);
t.add(1, 0, -4);
t.add(1, 1, 2, 2, 5);
cout << "Values:" << endl;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        cout << t.at(i, j) << " ";
    }
    cout << endl;
}
assert(t.sum(0, 0, 0, 1) == 11);
assert(t.sum(0, 0, 1, 0) == 8);
assert(t.sum(1, 1, 2, 2) == 29);
t.set(500000000, 500000000, 100);
assert(t.sum(0, 0, 999999999, 999999999) == 143);
return 0;
}

```

Example Output

```

Values:
5 6 0
3 5 5
0 5 14

```

2.6.7 Offline 2D Fenwick Tree

Maintain a 2D numerical array over an arbitrary, possibly huge index range while supporting point increments and rectangle-sum queries, when every coordinate that will ever be updated is known in advance. This offline restriction is what makes the structure cheap: both coordinates are coordinate-compressed against the update points, so no hashing is required and the storage is exactly the set of cells each Fenwick node can ever touch.

The outer dimension is an ordinary Fenwick tree over the sorted distinct update rows. Each outer node owns an inner Fenwick tree over only those columns that reach it along an update path, again stored in a sorted vector. A prefix query maps its row argument to the number of update rows at or below it, walks the outer tree, and within each visited node maps its column argument to a rank by binary search. Compared with the hashed sparse 2D Fenwick tree, this is deterministic and has markedly better constants, at the cost of requiring all update locations up front. Prefer the dense 2D Fenwick tree when the coordinates are small enough to allocate directly, and the sparse 2D Fenwick tree when update locations are not known ahead of time or rectangle updates are needed.

Usage is two-phase. First declare every cell that will be updated with `reserve()`; then call `build()` once; then issue `add()` updates and `sum()` / `at()` queries freely. Updates may only target reserved cells, but queries may use any coordinates.

- `OfflineFenwick2D<T>()` constructs an empty structure in the declaration phase.
- `reserve(r, c)` declares that the cell (r, c) may later be updated. Call before `build()`.
- `build()` finalizes the coordinate compression. Call exactly once, after all `reserve()` calls.
- `add(r, c, x)` adds `x` to the value at index (r, c) , which must have been reserved.
- `set(r, c, x)` assigns `x` to the value at index (r, c) , which must have been reserved.
- `sum(r, c)` returns the sum of the rectangle with corners $(0, 0)$ and (r, c) , inclusive.
- `sum(r1, c1, r2, c2)` returns the sum of the rectangle with upper-left corner $(r1, c1)$ and lower-right corner $(r2, c2)$, inclusive.
- `at(r, c)` returns the value at index (r, c) .

Time Complexity:

- $O(n \log n)$ per call to `build()`, where n is the number of reserved cells.

- $O(\log^2 n)$ per call to `add()`, `set()`, `sum()`, and `at()`.

Space Complexity:

- $O(n \log n)$ for storage, where n is the number of reserved cells.
- $O(1)$ auxiliary for all operations after `build()`.

```
#include <algorithm>
#include <cassert>
#include <utility>
#include <vector>

template<typename T>
class OfflineFenwick2D {
    std::vector<int> xs;           // Sorted distinct update rows.
    std::vector<std::vector<int>>> ys; // ys[p]: sorted distinct columns at outer node p.
    std::vector<std::vector<T>>> tree; // tree[p]: 1-indexed inner Fenwick over ys[p].
    std::vector<std::pair<int, int>>> pending; // Reserved (row, column) cells.
    bool built = false;

    // Number of update rows at most r; the outer Fenwick prefix bound for a query at row r.
    int row_rank(int r) const {
        return static_cast<int>(std::upper_bound(xs.begin(), xs.end(), r) - xs.begin());
    }

    // 1-indexed outer position of an update row r, which is guaranteed to be present in xs.
    int row_pos(int r) const {
        return static_cast<int>(std::lower_bound(xs.begin(), xs.end(), r) - xs.begin()) + 1;
    }

    // Number of columns at most c registered at outer node p; the inner prefix bound.
    int col_rank(int p, int c) const {
        return static_cast<int>(std::upper_bound(ys[p].begin(), ys[p].end(), c) - ys[p].begin());
    }

    bool is_reserved(int r, int c) const {
        int p = row_pos(r);
        return p < static_cast<int>(tree.size()) && p > 0 && xs[p - 1] == r &&
            std::binary_search(ys[p].begin(), ys[p].end(), c);
    }

public:
    OfflineFenwick2D() = default;

    void reserve(int r, int c) {
        assert(!built);
        pending.emplace_back(r, c);
    }

    void build() {
        assert(!built);
        built = true;
        for (const auto &u : pending) {
            xs.push_back(u.first);
        }
        std::sort(xs.begin(), xs.end());
        xs.erase(std::unique(xs.begin(), xs.end()), xs.end());
        int m = static_cast<int>(xs.size());
        ys.assign(m + 1, {});
        // Register each update's column at every outer node on its update path.
        for (const auto &u : pending) {
            for (int x = row_pos(u.first); x <= m; x += x & -x) {
                ys[x].push_back(u.second);
            }
        }
        tree.assign(m + 1, {});
        for (int x = 1; x <= m; x++) {
            std::sort(ys[x].begin(), ys[x].end());
        }
    }
};
```

```

        ys[x].erase(std::unique(ys[x].begin(), ys[x].end()), ys[x].end());
        tree[x].assign(ys[x].size() + 1, T());
    }
    pending.clear();
}

void add(int r, int c, const T &x) {
    assert(built);
    assert(is_reserved(r, c));
    for (int i = row_pos(r); i < static_cast<int>(tree.size()); i += i & -i) {
        for (int j = col_rank(i, c); j <= static_cast<int>(ys[i].size()); j += j & -j) {
            tree[i][j] += x;
        }
    }
}

T sum(int r, int c) const {
    T res = T();
    for (int i = row_rank(r); i > 0; i -= i & -i) {
        for (int j = col_rank(i, c); j > 0; j -= j & -j) {
            res += tree[i][j];
        }
    }
    return res;
}

T sum(int r1, int c1, int r2, int c2) const {
    return sum(r2, c2) - sum(r1 - 1, c2) - sum(r2, c1 - 1) + sum(r1 - 1, c1 - 1);
}

T at(int r, int c) const { return sum(r, c, r, c); }
void set(int r, int c, const T &x) { add(r, c, x - at(r, c)); }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    OfflineFenwick2D<int> t;
    // Declare every cell that will be updated, then finalize.
    t.reserve(0, 0);
    t.reserve(0, 1000000);
    t.reserve(1000000, 0);
    t.reserve(1000000, 1000000);
    t.reserve(500000, 500000);
    t.build();

    t.set(0, 0, 5);
    t.set(0, 1000000, 6);
    t.set(1000000, 0, 7);
    t.add(1000000, 1000000, 9);
    t.add(500000, 500000, 4);

    assert(t.at(1000000, 1000000) == 9);
    assert(t.sum(0, 0, 0, 1000000) == 11);           // 5 + 6.
    assert(t.sum(0, 0, 1000000, 0) == 12);           // 5 + 7.
    assert(t.sum(0, 0, 1000000, 1000000) == 31);      // 5 + 6 + 7 + 9 + 4.
    assert(t.sum(400000, 400000, 600000, 600000) == 4); // Only the center cell.
    return 0;
}

```

2.7 Disjoint Sets and Tree Structures

2.7.1 Disjoint Set Union

Maintain a set of elements partitioned into non-overlapping subsets. Each partition is assigned a unique representative known as the parent, or root. Each subset is stored as a tree whose nodes point toward its root: finding a representative follows parent pointers, redirecting visited nodes straight to the root along the way (path compression), while unions attach the smaller tree beneath the root of the larger one (union-by-size). Together these keep the trees nearly flat. Union-by-size is interchangeable with the union-by-rank strategy used by Sparse Disjoint Set Union and yields the same complexity bounds, but union-by-size additionally supports `set_size()` queries.

This version is simplified to work only on the fixed set of integer elements $[0, n)$, chosen at construction. For arbitrary element types, see Sparse Disjoint Set Union.

- `DSU(n)` constructs n singleton partitions over elements $[0, n)$.
- `find_root(u)` returns the unique representative of the partition containing `u`.
- `sets()` returns the current number of partitions.
- `set_size(u)` returns the number of elements in the partition containing `u`.
- `is_united(u, v)` returns whether elements `u` and `v` belong to the same partition.
- `unite(u, v)` replaces the partitions containing `u` and `v` with a single new partition consisting of the union of elements in the original partitions, returning `true` if a merge occurred or `false` if `u` and `v` already belonged to the same partition.

Time Complexity:

- $O(n)$ per call to `DSU(n)`, and $O(1)$ per call to `sets()`.
- $O(\alpha(n))$ per call to `find_root()`, `set_size()`, `is_united()`, and `unite()`, where n is the number of elements, and $\alpha(n)$ is the extremely slow growing inverse of the Ackermann function (effectively a very small constant for all practical values of n).

Space Complexity:

- $O(n)$ for storage of the disjoint set forest elements.
- $O(1)$ auxiliary for all operations.

```
#include <numeric>
#include <utility>
#include <vector>

class DSU {
    std::vector<int> root, size;
    int num_sets;

public:
    explicit DSU(int n) : root(n), size(n, 1), num_sets(n) { std::iota(root.begin(), root.end(), 0); }

    int find_root(int u) {
        if (root[u] != u) {
            root[u] = find_root(root[u]);
        }
        return root[u];
    }

    int sets() const { return num_sets; }
    int set_size(int u) { return size[find_root(u)]; }
    bool is_united(int u, int v) { return find_root(u) == find_root(v); }

    bool unite(int u, int v) {
        int ru = find_root(u), rv = find_root(v);
        if (ru == rv) {
            return false;
        }
    }
};
```

```

    }
    num_sets--;
    if (size[ru] < size[rv]) {
        std::swap(ru, rv);
    }
    root[rv] = ru;
    size[ru] += size[rv];
    return true;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    DSU dsu(7);
    assert(dsu.unite(0, 1));
    assert(dsu.unite(1, 5));
    assert(dsu.unite(3, 4));
    assert(dsu.unite(3, 6));
    assert(!dsu.unite(0, 5)); // Already united.
    assert(dsu.sets() == 3);
    assert(dsu.set_size(1) == 3);
    assert(dsu.set_size(2) == 1);
    assert(dsu.is_united(0, 1));
    assert(!dsu.is_united(0, 2));
    assert(!dsu.is_united(1, 6));
    assert(dsu.is_united(4, 6));
    return 0;
}

```

2.7.2 Sparse Disjoint Set Union

Maintain a set of elements partitioned into non-overlapping subsets using a collection of trees. Each partition is assigned a unique representative known as the parent, or root. Each subset is stored as a tree whose nodes point toward its root: finding a representative follows parent pointers, redirecting visited nodes straight to the root along the way (path compression), while unions attach the shallower tree beneath the root of the deeper one (union-by-rank). Union-by-rank is interchangeable with union-by-size as used by the plain Disjoint Set Union, yielding the same complexity bounds; rank stores only the tree-depth estimate, so it cannot answer set-size queries.

This version uses an `std::unordered_map` for storage and coordinate compression (thus, element types must meet the requirements of key types for `std::unordered_map`). The order of sets returned by `get_all_sets()` is unspecified.

- `SparseDSU<T>()` constructs an empty set.
- `make_set(u)` creates a new partition consisting of the single element `u`, which must not have been previously added to the data structure.
- `size()` returns the number of elements that have been added.
- `sets()` returns the current number of disjoint sets.
- `is_united(u, v)` returns whether elements `u` and `v` belong to the same partition.
- `unite(u, v)` replaces the partitions containing `u` and `v` with a single new partition consisting of the union of elements in the original partitions, returning `true` if a merge occurred or `false` if `u` and `v` already belonged to the same partition.
- `get_all_sets()` returns all current partitions as a vector of vectors.

A precondition to the last three operations is that `make_set()` must have been previously called on their arguments.

Time Complexity:

- $O(1)$ per call to the constructor.

- $O(1)$ per call to `size()` and `sets()`.
- $O(1)$ on average per call to `make_set()`, where n is the number of elements that have been added via `make_set()` so far.
- $O(\alpha(n))$ on average per call to `is_united()` and `unite()`, where n is the number of elements that have been added via `make_set()` so far, and $\alpha(n)$ is the extremely slow growing inverse of the Ackermann function (effectively a very small constant for all practical values of n).
- $O(n)$ per call to `get_all_sets()`.

Space Complexity:

- $O(n)$ for storage of the disjoint set forest elements.
- $O(n)$ auxiliary heap space for `get_all_sets()`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <unordered_map>
#include <vector>

template<typename T>
class SparseDSU {
    int num_elements, num_sets;
    std::unordered_map<T, int> id;
    std::vector<int> root, rank;

    int find_root(int u) {
        if (root[u] != u) {
            root[u] = find_root(root[u]);
        }
        return root[u];
    }

public:
    SparseDSU() : num_elements(0), num_sets(0) {}

    void make_set(const T &u) {
        if (id.find(u) != id.end()) {
            return;
        }
        id[u] = num_elements;
        root.push_back(num_elements++);
        rank.push_back(0);
        num_sets++;
    }

    int size() const { return num_elements; }
    int sets() const { return num_sets; }
    bool is_united(const T &u, const T &v) { return find_root(id[u]) == find_root(id[v]); }

    bool unite(const T &u, const T &v) {
        int ru = find_root(id[u]), rv = find_root(id[v]);
        if (ru == rv) {
            return false;
        }
        num_sets--;
        if (rank[ru] < rank[rv]) {
            root[ru] = rv;
        } else {
            root[rv] = ru;
            if (rank[ru] == rank[rv]) {
                rank[ru]++;
            }
        }
        return true;
    }

    std::vector<std::vector<T>> get_all_sets() {
```

```

    std::unordered_map<int, std::vector<T>> tmp;
    for (auto &[key, val] : id) {
        tmp[find_root(val)].push_back(key);
    }
    std::vector<std::vector<T>> res;
    for (auto &[key, val] : tmp) {
        res.push_back(val);
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    SparseDSU<char> dsu;
    for (char c = 'a'; c <= 'g'; c++) {
        dsu.make_set(c);
    }
    assert(dsu.unite('a', 'b'));
    assert(dsu.unite('b', 'f'));
    assert(dsu.unite('d', 'e'));
    assert(dsu.unite('d', 'g'));
    assert(!dsu.unite('a', 'f')); // Already united.
    assert(dsu.size() == 7);
    assert(dsu.sets() == 3);
    auto s = dsu.get_all_sets();
    for (auto &set : s) {
        sort(set.begin(), set.end());
    }
    sort(s.begin(), s.end());
    bool first_set = true;
    for (const auto &set : s) {
        cout << (first_set ? "[" : ", ");
        first_set = false;
        bool first_value = true;
        for (char value : set) {
            cout << (first_value ? "" : ", ") << value;
            first_value = false;
        }
        cout << "]";
    }
    cout << endl;
    return 0;
}

```

Example Output

```
[a, b, f], [c], [d, e, g]
```

2.7.3 Rollback Disjoint Set Union

Maintains disjoint sets while supporting rollback to previous snapshots. Rollback DSU is useful for offline dynamic connectivity, divide-and-conquer over time, and backtracking search where unions must be undone.

Path compression is intentionally not used, because it is hard to undo. Union by size keeps tree height logarithmic, and every successful union records enough information to restore the previous state.

- `RollbackDSU(n)` constructs `n` singleton sets over elements `[0, n)`.

- `count_sets()` returns the current number of disjoint sets.
- `find_root(u)` returns the representative of the set containing `u`.
- `is_united(u, v)` returns whether `u` and `v` are in the same set.
- `unite(u, v)` merges two sets and returns whether a merge occurred.
- `snapshot()` returns a token representing the current history size.
- `rollback(s)` undoes all changes made after snapshot `s`.

Time Complexity:

- $O(n)$ per call to `RollbackDSU(n)`.
- $O(1)$ per call to `count_sets()`.
- $O(\log n)$ worst-case per call to `find_root(u)`, `is_united(u, v)`, and `unite(u, v)`.
- $O(1)$ per undone union during `rollback(s)`.

Space Complexity: $O(n + q)$ for q successful unions stored in the rollback history.

```
#include <numeric>
#include <vector>

class RollbackDSU {
    struct Change {
        int child, parent, parent_size;

        Change(int child = -1, int parent = -1, int parent_size = 0)
            : child(child), parent(parent), parent_size(parent_size) {}
    };

    std::vector<int> root, size;
    std::vector<Change> history;
    int sets;

    int find_root(int u) const {
        while (root[u] != u) {
            u = root[u];
        }
        return u;
    }

public:
    explicit RollbackDSU(int n) {
        root.resize(n);
        size.assign(n, 1);
        history.clear();
        sets = n;
        std::iota(root.begin(), root.end(), 0);
    }

    int count_sets() const { return sets; }
    bool is_united(int u, int v) const { return find_root(u) == find_root(v); }

    bool unite(int u, int v) {
        u = find_root(u);
        v = find_root(v);
        if (u == v) {
            return false;
        }
        if (size[u] > size[v]) {
            int tmp = u;
            u = v;
            v = tmp;
        }
        history.emplace_back(u, v, size[v]);
        root[u] = v;
        size[v] += size[u];
        sets--;
        return true;
    }
};
```

```

}

int snapshot() const { return static_cast<int>(history.size()); }

void rollback(int snapshot) {
    while (static_cast<int>(history.size()) > snapshot) {
        Change c = history.back();
        history.pop_back();
        root[c.child] = c.child;
        size[c.parent] = c.parent_size;
        sets++;
    }
}
};

```

Example Usage

```

#include <cassert>

int main() {
    RollbackDSU dsu(5);
    dsu.unite(0, 1);
    int s = dsu.snapshot();
    dsu.unite(1, 2);
    dsu.unite(3, 4);
    assert(dsu.is_united(0, 2));
    assert(dsu.count_sets() == 2);
    dsu.rollback(s);
    assert(dsu.is_united(0, 1));
    assert(!dsu.is_united(0, 2));
    assert(!dsu.is_united(3, 4));
    assert(dsu.count_sets() == 4);
    return 0;
}

```

2.7.4 Lowest Common Ancestor (Sparse Table)

Given a rooted tree or forest, determine the lowest common ancestor of any two nodes in the same tree. The lowest common ancestor of two nodes u and v is the node that has the greatest depth while having both u and v as descendants. A node is considered to be a descendant of itself.

This implementation preprocesses binary ancestor jumps. During depth-first search, it records entry and exit times so ancestry can be tested in $O(1)$, records each node's depth and root, and stores `up[u][i]`, the 2^i -th ancestor of each node `u`. To answer `lca(u, v)`, it first handles the case where one node is already an ancestor of the other, then jumps `u` upward by decreasing powers of two until its parent is the lowest common ancestor.

- `SparseTableLCA(adj)` builds the structure over a forest represented by a bidirectional adjacency list `adj` of nodes numbered $[0, n)$, where `n` is `adj.size()`.
- `go_up(u, k)` returns the k -th ancestor of node `u`, stopping at that tree's root if `k` is larger than `u`'s depth.
- `lca(u, v)` returns the lowest common ancestor of nodes `u` and `v`, or -1 if they are in different trees.
- `is_ancestor(parent, child)` returns whether `parent` is an ancestor of `child`.
- `dist(u, v)` returns the number of edges on the path between nodes `u` and `v`, or -1 if they are in different trees.

Time Complexity:

- $O(n \log n)$ for construction, where n is the number of nodes.
- $O(\log n)$ per call to `go_up()`, `lca()`, and `dist()`.
- $O(1)$ per call to `is_ancestor()`.

Space Complexity:

- $O(n \log n)$ to store the binary ancestor table.
- $O(n)$ auxiliary stack space for construction.
- $O(1)$ auxiliary for `go_up()`, `lca()`, `is_ancestor()`, and `dist()`.

```
#include <algorithm>
#include <vector>

class SparseTableLCA {
    std::vector<std::vector<int>>> adj, up;
    std::vector<int> tin, tout, depth, root;
    int len, timer;

    void dfs(int u, int p, int r, int d) {
        tin[u] = timer++;
        depth[u] = d;
        root[u] = r;
        up[u][0] = p;
        for (int i = 1; i < len; i++) {
            up[u][i] = up[up[u][i - 1]][i - 1];
        }
        for (int v : adj[u]) {
            if (v != p) {
                dfs(v, u, r, d + 1);
            }
        }
        tout[u] = timer++;
    }

public:
    explicit SparseTableLCA(const std::vector<std::vector<int>>> &adj) : adj(adj), timer(0) {
        int n = static_cast<int>(adj.size());
        len = 1;
        while ((1 << len) <= std::max(1, n)) {
            len++;
        }
        up.assign(n, std::vector<int>(len));
        tin.assign(n, 0);
        tout.assign(n, 0);
        depth.assign(n, 0);
        root.assign(n, -1);
        for (int u = 0; u < n; u++) {
            if (root[u] == -1) {
                dfs(u, u, u, 0);
            }
        }
    }

    bool is_ancestor(int parent, int child) const {
        return root[parent] == root[child] && tin[parent] <= tin[child] && tout[child] <= tout[parent];
    }

    int go_up(int u, int k) const {
        k = std::min(k, depth[u]);
        for (int i = 0; i < len; i++) {
            if ((k & (1 << i)) != 0) {
                u = up[u][i];
            }
        }
        return u;
    }

    int lca(int u, int v) const {
        if (root[u] != root[v]) {
            return -1;
        }
    }
}
```

```

    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(up[u][i], v)) {
            u = up[u][i];
        }
    }
    return up[u][0];
}

int dist(int u, int v) const {
    int l = lca(u, v);
    return l == -1 ? -1 : depth[u] + depth[v] - 2 * depth[l];
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(vector<vector<int>> &adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    vector<vector<int>> adj(7);
    add_edge(adj, 0, 1);
    add_edge(adj, 0, 4);
    add_edge(adj, 1, 2);
    add_edge(adj, 1, 3);
    add_edge(adj, 5, 6);

    SparseTableLCA tree(adj);
    assert(tree.go_up(3, 1) == 1);
    assert(tree.go_up(3, 2) == 0);
    assert(tree.go_up(3, 10) == 0);
    assert(tree.lca(3, 2) == 1);
    assert(tree.lca(2, 4) == 0);
    assert(tree.lca(2, 6) == -1);
    assert(tree.lca(5, 6) == 5);
    assert(tree.dist(3, 2) == 2);    // 3-1-2.
    assert(tree.dist(2, 4) == 3);    // 2-1-0-4.
    assert(tree.dist(5, 6) == 1);    // 5-6.
    assert(tree.dist(2, 6) == -1);    // Different trees.
    return 0;
}

```

2.7.5 Lowest Common Ancestor (Segment Tree)

Given a rooted tree or forest, determine the lowest common ancestor of any two nodes in the same tree. The lowest common ancestor of two nodes u and v is the node that has the greatest depth while having both u and v as descendants. A node is considered to be a descendant of itself.

This reduces LCA to a range-minimum query. An Euler tour of the forest records, at each visit to a node, that node's depth; the lowest common ancestor of u and v in the same tree is the shallowest node visited between their

first occurrences in the tour, found by a range-minimum query over the depth sequence. This version answers those queries with a segment tree.

- `SegTreeLCA(adj)` builds the structure over a forest represented by a bidirectional adjacency list `adj` of nodes numbered $[0, n)$, where `n` is `adj.size()`.
- `lca(u, v)` returns the lowest common ancestor of nodes `u` and `v`, or -1 if they are in different trees.
- `dist(u, v)` returns the number of edges on the path between nodes `u` and `v`, or -1 if they are in different trees.

Time Complexity:

- $O(n \log n)$ for construction, where n is the number of nodes.
- $O(\log n)$ per call to `lca()` and `dist()`.

Space Complexity:

- $O(n)$ for storage of the segment tree.
- $O(n)$ auxiliary stack space for construction.
- $O(\log n)$ auxiliary stack space for `lca()`.

```
#include <algorithm>
#include <vector>

class SegTreeLCA {
    std::vector<std::vector<int>>> adj;
    std::vector<int> depth, order, first, root, minpos;
    int len;

    void dfs(int u, int r, int d) {
        root[u] = r;
        depth[u] = d;
        first[u] = static_cast<int>(order.size());
        order.push_back(u);
        for (int v : adj[u]) {
            if (depth[v] == -1) {
                dfs(v, r, d + 1);
                order.push_back(u);
            }
        }
    }

    int better(int a, int b) const { return depth[a] < depth[b] ? a : b; }

    void build(int node, int lo, int hi) {
        if (lo == hi) {
            minpos[node] = order[lo];
            return;
        }
        int mid = lo + (hi - lo) / 2;
        build(2 * node + 1, lo, mid);
        build(2 * node + 2, mid + 1, hi);
        minpos[node] = better(minpos[2 * node + 1], minpos[2 * node + 2]);
    }

    int get_minpos(int a, int b, int node, int lo, int hi) const {
        if (a == lo && b == hi) {
            return minpos[node];
        }
        int mid = lo + (hi - lo) / 2;
        if (b <= mid) {
            return get_minpos(a, b, 2 * node + 1, lo, mid);
        }
        if (mid < a) {
            return get_minpos(a, b, 2 * node + 2, mid + 1, hi);
        }
        return better(
            get_minpos(a, mid, 2 * node + 1, lo, mid), get_minpos(mid + 1, b, 2 * node + 2, mid + 1, hi)
        );
    }
};
```

```

    );
}

public:
explicit SegTreeLCA(const std::vector<std::vector<int>> &adj) : adj(adj) {
    int n = static_cast<int>(adj.size());
    depth.assign(n, -1);
    first.assign(n, -1);
    root.assign(n, -1);
    for (int u = 0; u < n; u++) {
        if (depth[u] == -1) {
            dfs(u, u, 0);
        }
    }
    len = static_cast<int>(order.size());
    minpos.assign(std::max(1, 4 * len), 0);
    if (len > 0) {
        build(0, 0, len - 1);
    }
}

int lca(int u, int v) const {
    if (root[u] != root[v]) {
        return -1;
    }
    return get_minpos(std::min(first[u], first[v]), std::max(first[u], first[v]), 0, 0, len - 1);
}

int dist(int u, int v) const {
    int l = lca(u, v);
    return l == -1 ? -1 : depth[u] + depth[v] - 2 * depth[l];
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(vector<vector<int>> &adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    vector<vector<int>> adj(7);
    add_edge(adj, 0, 1);
    add_edge(adj, 0, 4);
    add_edge(adj, 1, 2);
    add_edge(adj, 1, 3);
    add_edge(adj, 5, 6);

    SegTreeLCA tree(adj);
    assert(tree.lca(3, 2) == 1);
    assert(tree.lca(2, 4) == 0);
    assert(tree.lca(2, 6) == -1);
    assert(tree.lca(5, 6) == 5);
    assert(tree.dist(3, 2) == 2);    // 3-1-2.
    assert(tree.dist(2, 4) == 3);    // 2-1-0-4.
    assert(tree.dist(5, 6) == 1);    // 5-6.
    assert(tree.dist(2, 6) == -1);   // Different trees.
    return 0;
}

```


2.7.6 Virtual Tree

Given a rooted tree and a subset of its nodes, build the virtual tree (also called the auxiliary or compressed tree): the smallest tree that contains every node of the subset together with the lowest common ancestor of each pair, with chains of irrelevant degree-two ancestors compressed into single edges. Many problems ask repeated queries each restricted to a handful of “important” nodes; the virtual tree has $O(k)$ nodes for a subset of size k , so an algorithm can run over it instead of the whole tree, turning a per-query cost proportional to n into one proportional to the subset size.

The construction sorts the subset by depth-first entry time, which lists the nodes in the order a walk of the tree would meet them. The lowest common ancestor of any contiguous span of the subset is the minimum-depth node among the pairwise LCAs of adjacent entries, so it suffices to add the LCA of each adjacent pair, re-sort, and deduplicate. A single stack pass over the resulting nodes then links each one to its parent in the virtual tree: ancestors that are no longer on the path to the current node are popped, and the node on top becomes the parent. This implementation carries its own binary lifting so it is self-contained; given an existing lowest common ancestor structure, the same sort-and-stack logic applies directly.

- `VirtualTree(adj, root)` builds the structure over a tree rooted at `root`, given a bidirectional adjacency list `adj` of nodes numbered $[0, n)$, where `n` is `adj.size()`.
- `lca(u, v)` returns the lowest common ancestor of nodes `u` and `v`.
- `depth_of(u)` returns the depth of `u` in the original tree (the root has depth 0). The original path length of a virtual-tree edge (p, c) is `depth_of(c) - depth_of(p)`.
- `build(nodes)` returns the virtual tree induced by `nodes`. The result has `root` set to the topmost node (the common ancestor of `nodes`), `vertices` listing every original node ID in the virtual tree sorted by entry time, and `edges` listing each (p, c) pair in original IDs. Both the vertex set and the edges use the node numbering of the original tree. An empty input yields an empty result with `root` = `-1`.

Time Complexity:

- $O(n \log n)$ for construction, where n is the number of nodes.
- $O(\log n)$ per call to `lca()`.
- $O(k \log k)$ per call to `build()`, where k is the size of the input subset.

Space Complexity:

- $O(n \log n)$ to store the binary ancestor table.
- $O(k)$ auxiliary space and result size per call to `build()`.

```
#include <algorithm>
#include <vector>

class VirtualTree {
    std::vector<std::vector<int>>> up;
    std::vector<int> tin, tout, depth;
    int len, timer;

    void dfs(const std::vector<std::vector<int>>> &adj, int u, int p, int d) {
        tin[u] = timer++;
        depth[u] = d;
        up[u][0] = p;
        for (int i = 1; i < len; i++) {
            up[u][i] = up[up[u][i - 1]][i - 1];
        }
        for (int v : adj[u]) {
            if (v != p) {
                dfs(adj, v, u, d + 1);
            }
        }
        tout[u] = timer++;
    }

public:
    struct Tree {
```

```

    int root = -1;
    std::vector<int> vertices;
    std::vector<std::pair<int, int>> edges; // (parent, child) in original node ids.
};

VirtualTree(const std::vector<std::vector<int>> &adj, int root) {
    int n = static_cast<int>(adj.size());
    len = 1;
    while ((1 << len) <= std::max(1, n)) {
        len++;
    }
    up.assign(n, std::vector<int>(len));
    tin.assign(n, 0);
    tout.assign(n, 0);
    depth.assign(n, 0);
    timer = 0;
    dfs(adj, root, root, 0);
}

bool is_ancestor(int parent, int child) const {
    return tin[parent] <= tin[child] && tout[child] <= tout[parent];
}

// Depth of u in the original tree; depth(child) - depth(parent) is the length of the original
// path that a compressed virtual-tree edge stands for.
int depth_of(int u) const { return depth[u]; }

int lca(int u, int v) const {
    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(up[u][i], v)) {
            u = up[u][i];
        }
    }
    return up[u][0];
}

Tree build(std::vector<int> nodes) const {
    Tree res;
    if (nodes.empty()) {
        return res;
    }
    auto by_tin = [&](int a, int b) { return tin[a] < tin[b]; };
    std::sort(nodes.begin(), nodes.end(), by_tin);
    nodes.erase(std::unique(nodes.begin(), nodes.end()), nodes.end());
    int k = static_cast<int>(nodes.size());
    for (int i = 0; i + 1 < k; i++) {
        nodes.push_back(lca(nodes[i], nodes[i + 1]));
    }
    std::sort(nodes.begin(), nodes.end(), by_tin);
    nodes.erase(std::unique(nodes.begin(), nodes.end()), nodes.end());

    std::vector<int> stack;
    for (int v : nodes) {
        while (!stack.empty() && !is_ancestor(stack.back(), v)) {
            stack.pop_back();
        }
        if (!stack.empty()) {
            res.edges.emplace_back(stack.back(), v);
        }
        stack.push_back(v);
    }
    res.root = nodes.front();
}

```

```

    res.vertices = std::move(nodes);
    return res;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(vector<vector<int>> &adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    //      0
    //      / |
    //      1 2
    //      /| |
    //      3 4 5
    //      /  |
    //      6   7
    vector<vector<int>> adj(8);
    add_edge(adj, 0, 1);
    add_edge(adj, 0, 2);
    add_edge(adj, 1, 3);
    add_edge(adj, 1, 4);
    add_edge(adj, 2, 5);
    add_edge(adj, 3, 6);
    add_edge(adj, 5, 7);

    VirtualTree vt(adj, 0);
    assert(vt.lca(6, 4) == 1);
    assert(vt.lca(6, 7) == 0);
    assert(vt.depth_of(0) == 0 && vt.depth_of(6) == 3);
    // The compressed edge 0 -> 7 stands for an original path of length 3 (0-2-5-7).
    assert(vt.depth_of(7) - vt.depth_of(0) == 3);

    // Virtual tree of {6, 7}: their LCA is 0, so the tree is 0 -> {6, 7} with edges compressed.
    VirtualTree::Tree t = vt.build({6, 7});
    assert(t.root == 0);
    assert((t.vertices == vector<int>{0, 6, 7}));
    assert(t.edges.size() == 2);
    for (auto &e : t.edges) {
        assert(e.first == 0 && (e.second == 6 || e.second == 7));
    }

    // Virtual tree of {6, 4, 7}: node 1 becomes a branch point (LCA of 6 and 4).
    VirtualTree::Tree u = vt.build({6, 4, 7});
    assert(u.root == 0);
    assert((u.vertices == vector<int>{0, 1, 6, 4, 7}));
    // Expected edges: 0->1, 1->6, 1->4, 0->7.
    vector<pair<int, int>> want{{0, 1}, {1, 6}, {1, 4}, {0, 7}};
    auto got = u.edges;
    sort(got.begin(), got.end());
    sort(want.begin(), want.end());
    assert(got == want);

    // A single node is its own virtual tree with no edges.
    VirtualTree::Tree s = vt.build({4});
    assert(s.root == 4 && s.edges.empty() && s.vertices == vector<int>{4});
    return 0;
}

```

2.7.7 Weighted Tree Path Queries (Binary Lifting)

Given a rooted tree or forest with weighted edges, aggregate the edge weights along the path between any two nodes in the same tree. This builds on the same binary lifting table as LCA: alongside `up[u][i]`, the 2^i -th ancestor of `u`, it stores `agg[u][i]`, the combined weight of the 2^i edges traversed by that jump. A path query splits the path `u-v` at their lowest common ancestor and merges the two upward climbs.

The aggregate is a commutative monoid defined by `combine()` and its identity `identity()`. The two must be edited together; the defaults below compute the maximum edge weight on the path. For the minimum, use `std::min` with `std::numeric_limits<W>::max()`; for the sum (the path length), use `a + b` with `W()`. Commutativity is required because the two half-paths are merged without regard to orientation. For an invertible aggregate such as the sum, an alternative that avoids the `agg` table is to store each node's weighted depth `dw[u]` and return

```
dw[u] + dw[v] - 2*dw[lca].
```

- `WeightedTreePath<W>(adj)` builds the structure over a forest given by a weighted, bidirectional adjacency list `adj`, where `adj[u]` holds `{v, w}` pairs for each edge `u-v` of weight `w`, over nodes numbered `[0, n)`, where `n` is `adj.size()`.
- `lca(u, v)` returns the lowest common ancestor of `u` and `v`, or `-1` if they lie in different trees.
- `kth_ancestor(u, k)` returns the k -th ancestor of `u`, stopping at that tree's root if `k` exceeds `u`'s depth.
- `is_ancestor(parent, child)` returns whether `parent` is an ancestor of `child`.
- `path_query(u, v)` returns the combined weight of the edges on the path from `u` to `v`, or `identity()` if they lie in different trees. If `u` equals `v` then the path has no edges, so `identity()` is returned.

Time Complexity:

- $O(n \log n)$ for construction, where n is the number of nodes.
- $O(\log n)$ per call to `lca()`, `kth_ancestor()`, and `path_query()`.
- $O(1)$ per call to `is_ancestor()`.

Space Complexity:

- $O(n \log n)$ to store the binary ancestor and aggregate tables.
- $O(n)$ auxiliary stack space for construction.
- $O(1)$ auxiliary for `lca()`, `kth_ancestor()`, `is_ancestor()`, and `path_query()`.

```
#include <algorithm>
#include <limits>
#include <utility>
#include <vector>

template<typename W>
class WeightedTreePath {
    // Edit combine() and identity() together for the desired commutative monoid.
    // The defaults give the maximum edge weight on the path; for the minimum use std::min with
    // std::numeric_limits<W>::max(), and for the sum (path length) use a + b with W().
    static W combine(const W &a, const W &b) { return std::max(a, b); }
    static W identity() { return std::numeric_limits<W>::lowest(); }

    std::vector<std::vector<int>>> up;
    std::vector<std::vector<W>>> agg; // agg[u][i] = combined weight of the 2^i edges above u.
    std::vector<int> tin, tout, depth, root;
    int len, timer;

    void dfs(int u, int p, int r, int d, const std::vector<std::vector<std::pair<int, W>>> &adj) {
        tin[u] = timer++;
        depth[u] = d;
        root[u] = r;
        up[u][0] = p;
        for (int i = 1; i < len; i++) {
            up[u][i] = up[up[u][i - 1]][i - 1];
            agg[u][i] = combine(agg[u][i - 1], agg[up[u][i - 1]][i - 1]);
        }
        for (const auto &[v, w] : adj[u]) {
```

```

        if (v != p) {
            agg[v][0] = w; // The edge into a child is stored on the child.
            dfs(v, u, r, d + 1, adj);
        }
    }
    tout[u] = timer++;
}

// Combined weight of the k edges on the upward path of k steps from u.
W climb(int u, int k) const {
    W res = identity();
    for (int i = 0; i < len; i++) {
        if ((k & (1 << i)) != 0) {
            res = combine(res, agg[u][i]);
            u = up[u][i];
        }
    }
    return res;
}

public:
explicit WeightedTreePath(const std::vector<std::vector<std::pair<int, W>>> &adj) : timer(0) {
    int n = static_cast<int>(adj.size());
    len = 1;
    while ((1 << len) <= std::max(1, n)) {
        len++;
    }
    up.assign(n, std::vector<int>(len));
    agg.assign(n, std::vector<W>(len, identity()));
    tin.assign(n, 0);
    tout.assign(n, 0);
    depth.assign(n, 0);
    root.assign(n, -1);
    for (int u = 0; u < n; u++) {
        if (root[u] == -1) {
            dfs(u, u, u, 0, adj);
        }
    }
}

bool is_ancestor(int parent, int child) const {
    return root[parent] == root[child] && tin[parent] <= tin[child] && tout[child] <= tout[parent];
}

int kth_ancestor(int u, int k) const {
    k = std::min(k, depth[u]);
    for (int i = 0; i < len; i++) {
        if ((k & (1 << i)) != 0) {
            u = up[u][i];
        }
    }
    return u;
}

int lca(int u, int v) const {
    if (root[u] != root[v]) {
        return -1;
    }
    if (is_ancestor(u, v)) {
        return u;
    }
    if (is_ancestor(v, u)) {
        return v;
    }
    for (int i = len - 1; i >= 0; i--) {
        if (!is_ancestor(up[u][i], v)) {
            u = up[u][i];
        }
    }
}

```

```

    }
    return up[u][0];
}

W path_query(int u, int v) const {
    int l = lca(u, v);
    if (l == -1) {
        return identity();
    }
    return combine(climb(u, depth[u] - depth[l]), climb(v, depth[v] - depth[l]));
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(vector<vector<pair<int, int>>> &adj, int u, int v, int w) {
    adj[u].push_back({v, w});
    adj[v].push_back({u, w});
}

int main() {
    //      0
    //      (4) / \ (2)
    //      1   2
    //      (7) / | (1) | \ (3)
    //      3 4 (5) 5 6
    vector<vector<pair<int, int>>> adj(7);
    add_edge(adj, 0, 1, 4);
    add_edge(adj, 0, 2, 2);
    add_edge(adj, 1, 3, 7);
    add_edge(adj, 1, 4, 1);
    add_edge(adj, 2, 5, 5);
    add_edge(adj, 2, 6, 3);
    WeightedTreePath<int> tree(adj);

    assert(tree.lca(3, 4) == 1);
    assert(tree.lca(5, 6) == 2);
    assert(tree.lca(3, 6) == 0);

    assert(tree.kth_ancestor(3, 1) == 1);
    assert(tree.kth_ancestor(3, 2) == 0);
    assert(tree.kth_ancestor(3, 5) == 0); // Clamped to the root.

    // Default combine() reports the maximum edge weight on the path.
    assert(tree.path_query(3, 4) == 7); // 3-1 (7), 1-4 (1).
    assert(tree.path_query(5, 6) == 5); // 5-2 (5), 2-6 (3).
    assert(tree.path_query(4, 5) == 5); // 4-1 (1), 1-0 (4), 0-2 (2), 2-5 (5).
    assert(tree.path_query(0, 3) == 7); // 0-1 (4), 1-3 (7).

    vector<vector<pair<int, int>>> forest(2);
    WeightedTreePath<int> disconnected(forest);
    assert(disconnected.lca(0, 1) == -1);
    assert(disconnected.path_query(0, 1) == numeric_limits<int>::lowest());
    return 0;
}

```

2.7.8 Heavy Light Decomposition

Maintain a tree or forest with values associated with either edges or nodes, while supporting both dynamic queries and dynamic updates of all values on a given path between two nodes in the tree. Heavy-light decomposition

partitions the nodes of the tree into disjoint paths where all nodes have degree two, except the endpoints of a path which has degree one. A node stays on its parent's path when its subtree contains at least half of its parent's nodes, so walking from any node to the root crosses $O(\log n)$ different paths. Each path stores its values in its own lazy segment tree, decomposing any path query or update into $O(\log n)$ contiguous range operations.

The query operation is defined by an associative `combine()` function. The default code below assumes a numerical tree type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`. For direction-independent path queries, `combine()` should also be commutative; otherwise, store enough information in each aggregate to combine paths in the required order.

The update operation is defined by `apply_delta()` and `compose_deltas()`. A delta must act on an aggregate summary of a path of length `len`: `apply_delta(v, d, len)` returns the aggregate after applying update `d` to every element represented by aggregate `v`. Pending deltas are combined in chronological order by `compose_deltas(old, d)`, meaning “apply `old`, then apply `d`”. These hooks do not support arbitrary query/update pairings; the delta operation must distribute over `combine()`, and composed deltas must have the same effect as applying their updates sequentially. The default code below defines updates that “set” a path's edges or nodes to a new value. For range increment updates, `apply_delta(v, d, len)` would return `v + d` for min/max queries, or `v + d * len` for sum queries, and `compose_deltas(old, d)` would return `old + d`.

- `HeavyLight<T>(adj, v)` constructs a new heavy light decomposition on a forest defined by the adjacency list `adj`, with all values initialized to `v`. The adjacency list must consist of only the integers $[0, n)$, where `n` is `adj.size()`. No duplicate edges should exist.
- `query(u, v)` returns the result of `combine()` applied to all values on the path from node `u` to node `v`, or throws if `u` and `v` are in different trees.
- `update(u, v, d)` modifies all values on the path from node `u` to node `v` by respectively applying the delta `d`, or throws if `u` and `v` are in different trees.
- `for_each_path(u, v, include_lca, f)` decomposes the path from node `u` to node `v` into heavy path ranges and calls `f(path, lo, hi, up)` on each range, where `up` says the path segment is traversed upward from `u` toward the LCA. If values are stored on edges, pass `include_lca = false` to skip the LCA's vertex slot. Returns false if `u` and `v` are in different trees.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the number of nodes.
- $O(\log n)$ calls to `f()` per call to `for_each_path()`.
- $O(\log^2 n)$ per call to `query()` and `update()`.

Space Complexity:

- $O(n)$ for storage of the decomposition.
- $O(n)$ auxiliary stack space for the constructor.
- $O(1)$ auxiliary for `query()` and `update()`.

```
#include <algorithm>
#include <stdexcept>
#include <tuple>
#include <utility>
#include <vector>

template<typename T>
class HeavyLight {
    // Set this to true to store values on edges, false to store values on nodes.
    static const bool VALUES_ON_EDGES = true;

    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int len) { return d; }
    static T compose_deltas(const T &old, const T &d) { return d; }

    std::vector<std::vector<int>>> adj;
    std::vector<std::vector<T>>> value, delta;
    std::vector<std::vector<bool>>> pending;
    std::vector<std::vector<int>>> len;
```

```

std::vector<int> size, parent, root, tin, tout, path, pathlen, pathpos, pathroot;
int counter, paths;

void dfs(int u, int p, int r) {
    tin[u] = counter++;
    parent[u] = p;
    root[u] = r;
    size[u] = 1;
    for (int v : adj[u]) {
        if (v != p) {
            dfs(v, u, r);
            size[u] += size[v];
        }
    }
    tout[u] = counter++;
}

int new_path(int u) {
    pathroot[paths] = u;
    return paths++;
}

void build_paths(int u, int path) {
    this->path[u] = path;
    pathpos[u] = pathlen[path]++;
    for (int v : adj[u]) {
        if (v != parent[u]) {
            build_paths(v, (2 * size[v] >= size[u]) ? path : new_path(v));
        }
    }
}

inline T applied_value(int path, int i) {
    return pending[path][i] ? apply_delta(value[path][i], delta[path][i], len[path][i])
        : value[path][i];
}

void push_delta(int path, int i) {
    int d = 0;
    while ((i >> d) > 0) {
        d++;
    }
    for (d -= 2; d >= 0; d--) {
        int l = (i >> d), r = (1 ^ 1), n = 1 / 2;
        if (pending[path][n]) {
            value[path][n] = applied_value(path, n);
            delta[path][l] =
                pending[path][l] ? compose_deltas(delta[path][l], delta[path][n]) : delta[path][n];
            delta[path][r] =
                pending[path][r] ? compose_deltas(delta[path][r], delta[path][n]) : delta[path][n];
            pending[path][l] = pending[path][r] = true;
            pending[path][n] = false;
        }
    }
}

bool query(int path, int u, int v, T *res) {
    push_delta(path, u += static_cast<int>(value[path].size() / 2));
    push_delta(path, v += static_cast<int>(value[path].size() / 2));
    bool found = false;
    for (; u <= v; u = (u + 1) / 2, v = (v - 1) / 2) {
        if ((u & 1) != 0) {
            T value = applied_value(path, u);
            *res = found ? combine(*res, value) : value;
            found = true;
        }
        if ((v & 1) == 0) {
            T value = applied_value(path, v);

```



```

        *res = found ? combine(*res, value) : value;
        found = true;
    }
}
return found;
}

void update(int path, int u, int v, const T &d) {
    push_delta(path, u += static_cast<int>(value[path].size() / 2);
    push_delta(path, v += static_cast<int>(value[path].size() / 2);
    int tu = -1, tv = -1;
    for (; u <= v; u = (u + 1) / 2, v = (v - 1) / 2) {
        if ((u & 1) != 0) {
            delta[path][u] = pending[path][u] ? compose_deltas(delta[path][u], d) : d;
            pending[path][u] = true;
            if (tu == -1) {
                tu = u;
            }
        }
        if ((v & 1) == 0) {
            delta[path][v] = pending[path][v] ? compose_deltas(delta[path][v], d) : d;
            pending[path][v] = true;
            if (tv == -1) {
                tv = v;
            }
        }
    }
    for (int i = tu; i > 1; i /= 2) {
        value[path][i / 2] = combine(applied_value(path, i), applied_value(path, i ^ 1));
    }
    for (int i = tv; i > 1; i /= 2) {
        value[path][i / 2] = combine(applied_value(path, i), applied_value(path, i ^ 1));
    }
}

inline bool is_ancestor(int parent, int child) {
    return (tin[parent] <= tin[child]) && (tout[child] <= tout[parent]);
}

public:
explicit HeavyLight(const std::vector<std::vector<int>> &adj, const T &v = T())
    : adj(adj),
      size(adj.size()),
      parent(adj.size()),
      root(adj.size(), -1),
      tin(adj.size()),
      tout(adj.size()),
      path(adj.size()),
      pathlen(adj.size()),
      pathpos(adj.size()),
      pathroot(adj.size()),
      counter(0),
      paths(0) {
    for (int u = 0; u < static_cast<int>(adj.size()); u++) {
        if (root[u] == -1) {
            dfs(u, -1, u);
            build_paths(u, new_path(u));
        }
    }
    value.resize(paths);
    delta.resize(paths);
    pending.resize(paths);
    len.resize(paths);
    for (int i = 0; i < paths; i++) {
        int m = pathlen[i];
        value[i].assign(2 * m, v);
        delta[i].resize(2 * m);
        pending[i].assign(2 * m, false);
    }
}

```

```

    len[i].assign(2 * m, 1);
    for (int j = 2 * m - 1; j > 1; j -= 2) {
        value[i][j / 2] = combine(value[i][j], value[i][j ^ 1]);
        len[i][j / 2] = len[i][j] + len[i][j ^ 1];
    }
}

template<typename Fn>
bool for_each_path(int u, int v, bool include_lca, Fn f) {
    if (root[u] != root[v]) {
        return false;
    }
    int path_root;
    while (!is_ancestor(path_root = pathroot[path[u]], v)) {
        f(path[u], 0, pathpos[u], true);
        u = parent[path_root];
    }
    std::vector<std::tuple<int, int, int>> down_parts;
    while (!is_ancestor(path_root = pathroot[path[v]], u)) {
        down_parts.emplace_back(path[v], 0, pathpos[v]);
        v = parent[path_root];
    }
    int lo = std::min(pathpos[u], pathpos[v]) + static_cast<int>(!include_lca);
    int hi = std::max(pathpos[u], pathpos[v]);
    if (lo <= hi) {
        f(path[u], lo, hi, pathpos[u] > pathpos[v]);
    }
    for (int i = static_cast<int>(down_parts.size()) - 1; i >= 0; i--) {
        int path, lo, hi;
        std::tie(path, lo, hi) = down_parts[i];
        f(path, lo, hi, false);
    }
    return true;
}

T query(int u, int v) {
    if (root[u] != root[v]) {
        throw std::runtime_error("No path exists between nodes in different trees.");
    }
    if (VALUES_ON_EDGES && u == v) {
        throw std::runtime_error("No edge between u and v to be queried.");
    }
    bool found = false;
    T res = T(), value;
    for_each_path(u, v, !VALUES_ON_EDGES, [&](int path, int lo, int hi, bool) {
        if (query(path, lo, hi, &value)) {
            res = found ? combine(res, value) : value;
            found = true;
        }
    });
    if (!found) {
        throw std::runtime_error("Unexpected error: No values found.");
    }
    return res;
}

void update(int u, int v, const T &d) {
    if (root[u] != root[v]) {
        throw std::runtime_error("No path exists between nodes in different trees.");
    }
    if (VALUES_ON_EDGES && u == v) {
        return;
    }
    for_each_path(u, v, !VALUES_ON_EDGES, [&](int path, int lo, int hi, bool) {
        update(path, lo, hi, d);
    });
}

```

```
};
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    //   w=40   w=20   w=10
    // 0-----1-----2-----3   5-----6
    //               |
    //               -----4
    //               w=30
    vector<vector<int>> adj(7);
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(2);
    adj[2].push_back(1);
    adj[2].push_back(3);
    adj[3].push_back(2);
    adj[2].push_back(4);
    adj[4].push_back(2);
    adj[5].push_back(6);
    adj[6].push_back(5);
    HeavyLight<int> hld(adj, 0);
    hld.update(0, 1, 40);
    hld.update(1, 2, 20);
    hld.update(2, 3, 10);
    hld.update(2, 4, 30);
    assert(hld.query(0, 3) == 10);
    assert(hld.query(2, 4) == 30);
    hld.update(3, 4, 5);
    assert(hld.query(1, 4) == 5);
    hld.update(5, 6, 7);
    assert(hld.query(5, 6) == 7);
    assert(!hld.for_each_path(0, 5, true, [](int, int, int, bool) {}));
    return 0;
}
```

2.7.9 Euler Tour Tree

Maintain an undirected forest dynamically using Euler Tour Trees. Each tree is represented by a sequence of directed edge occurrences in an implicit treap: an undirected edge (u, v) contributes two nodes, $(u \rightarrow v)$ and $(v \rightarrow u)$. Rerooting a represented tree is a cyclic rotation of this sequence, while `link` concatenates two tours with the new edge occurrences and `cut` removes the two occurrences of an existing edge, splitting one tour into two.

This version supports dynamic connectivity, `link`, `cut`, rerooting, and listing the vertices in a connected component. It does not store vertex aggregates. To add component aggregates, a common extension is to insert one self-loop occurrence per vertex and maintain aggregate data in the treap.

- `EulerTourTree(n)` constructs a forest on vertices numbered $[0, n)$.
- `connected(u, v)` returns whether `u` and `v` are in the same tree.
- `link(u, v)` adds an edge between different trees and returns false if the edge would create a cycle or already exists.
- `cut(u, v)` removes an existing edge and returns false if it is not present.
- `reroot(u)` rotates the Euler tour of `u`'s tree so an occurrence leaving `u` is first.
- `component_vertices(u)` returns the distinct vertices in `u`'s tree.

Time Complexity:

- $O(\log m)$ expected per call to `connected`, `link`, `cut`, and `reroot`, where m is the number of edges currently in the forest.
- $O(k)$ expected per call to `component_vertices`, where k is the number of directed edge occurrences in the component.

Space Complexity:

- $O(n + m)$ for the forest.
- $O(\log m)$ auxiliary recursion stack for treap operations.

```
#include <algorithm>
#include <cassert>
#include <random>
#include <set>
#include <unordered_map>
#include <vector>

class EulerTourTree {
    struct Node {
        int u, v, priority, size;
        Node *left, *right, *parent;

        Node(int u, int v, int priority)
            : u(u), v(v), priority(priority), size(1), left(nullptr), right(nullptr), parent(nullptr) {}
    };

    int n;
    std::mt19937 rng;
    std::vector<std::unordered_map<int, Node *>> adj;
    std::vector<Node *> allocated;

    static int size(Node *t) { return t == nullptr ? 0 : t->size; }

    static void set_parent(Node *t, Node *p) {
        if (t != nullptr) {
            t->parent = p;
        }
    }

    static void update(Node *t) {
        if (t != nullptr) {
            t->size = 1 + size(t->left) + size(t->right);
            set_parent(t->left, t);
            set_parent(t->right, t);
        }
    }

    static Node *root(Node *t) {
        if (t == nullptr) {
            return nullptr;
        }
        while (t->parent != nullptr) {
            t = t->parent;
        }
        return t;
    }

    static int index(Node *t) {
        int res = size(t->left);
        while (t->parent != nullptr) {
            if (t == t->parent->right) {
                res += 1 + size(t->parent->left);
            }
            t = t->parent;
        }
    }
};
```

```

    return res;
}

static Node *merge(Node *a, Node *b) {
    if (a == nullptr) {
        return b;
    }
    if (b == nullptr) {
        return a;
    }
    if (a->priority > b->priority) {
        a->right = merge(a->right, b);
        update(a);
        a->parent = nullptr;
        return a;
    }
    b->left = merge(a, b->left);
    update(b);
    b->parent = nullptr;
    return b;
}

static std::pair<Node *, Node *> split(Node *t, int left_size) {
    if (t == nullptr) {
        return {nullptr, nullptr};
    }
    if (size(t->left) >= left_size) {
        auto [a, b] = split(t->left, left_size);
        t->left = b;
        update(t);
        set_parent(a, nullptr);
        t->parent = nullptr;
        return {a, t};
    }
    auto [a, b] = split(t->right, left_size - size(t->left) - 1);
    t->right = a;
    update(t);
    set_parent(b, nullptr);
    t->parent = nullptr;
    return {t, b};
}

static Node *make_first(Node *x) {
    auto [a, b] = split(root(x), index(x));
    return merge(b, a);
}

static Node *remove_first(Node *t) {
    auto [first, rest] = split(t, 1);
    if (first != nullptr) {
        first->left = first->right = first->parent = nullptr;
        first->size = 1;
    }
    return rest;
}

Node *new_node(int u, int v) {
    Node *node = new Node(u, v, std::uniform_int_distribution<int>()(rng));
    allocated.push_back(node);
    return node;
}

Node *any_occurrence(int u) const { return adj[u].empty() ? nullptr : adj[u].begin()->second; }

static void collect(Node *t, std::vector<int> &out) {
    if (t == nullptr) {
        return;
    }
}

```

```

    collect(t->left, out);
    out.push_back(t->u);
    collect(t->right, out);
}

public:
explicit EulerTourTree(int n) : n(n), rng(1234567), adj(n) {}

~EulerTourTree() {
    for (Node *node : allocated) {
        delete node;
    }
}

bool connected(int u, int v) const {
    assert(0 <= u && u < n && 0 <= v && v < n);
    if (u == v) {
        return true;
    }
    Node *a = any_occurrence(u), *b = any_occurrence(v);
    return a != nullptr && b != nullptr && root(a) == root(b);
}

void reroot(int u) {
    assert(0 <= u && u < n);
    Node *a = any_occurrence(u);
    if (a != nullptr) {
        make_first(a);
    }
}

bool link(int u, int v) {
    assert(0 <= u && u < n && 0 <= v && v < n);
    if (u == v || adj[u].count(v) || connected(u, v)) {
        return false;
    }
    Node *tu = any_occurrence(u), *tv = any_occurrence(v);
    if (tu != nullptr) {
        tu = make_first(tu);
    }
    if (tv != nullptr) {
        tv = make_first(tv);
    }
    Node *uv = new_node(u, v), *vu = new_node(v, u);
    adj[u][v] = uv;
    adj[v][u] = vu;
    merge(merge(tu, uv), merge(tv, vu));
    return true;
}

bool cut(int u, int v) {
    assert(0 <= u && u < n && 0 <= v && v < n);
    auto it = adj[u].find(v);
    if (it == adj[u].end()) {
        return false;
    }
    Node *uv = it->second, *vu = adj[v][u];
    make_first(uv);
    auto [left, right] = split(root(uv), index(vu));
    remove_first(left);    // Remove uv.
    remove_first(right);  // Remove vu.
    adj[u].erase(v);
    adj[v].erase(u);
    return true;
}

std::vector<int> component_vertices(int u) const {
    assert(0 <= u && u < n);

```

```

    Node *a = any_occurrence(u);
    if (a == nullptr) {
        return {u};
    }
    std::vector<int> occurrences, res;
    collect(root(a), occurrences);
    std::set<int> seen(occurrences.begin(), occurrences.end());
    res.assign(seen.begin(), seen.end());
    return res;
}
};

```

Example Usage

```

int main() {
    EulerTourTree ett(6);
    assert(!ett.connected(0, 1));
    assert(ett.connected(0, 0));

    assert(ett.link(0, 1));
    assert(ett.link(1, 2));
    assert(ett.link(3, 4));
    assert(ett.connected(0, 2));
    assert(!ett.connected(0, 4));
    assert(!ett.link(0, 2)); // Would create a cycle.

    auto c = ett.component_vertices(1);
    assert((c == std::vector<int>{0, 1, 2}));

    ett.reroot(2);
    assert(ett.connected(0, 2));
    assert(ett.cut(1, 2));
    assert(!ett.connected(0, 2));
    assert((ett.component_vertices(2) == std::vector<int>{2}));

    assert(ett.link(2, 3));
    assert(ett.connected(0, 4) == false);
    assert(ett.connected(2, 4));
    assert(!ett.cut(0, 5));
    return 0;
}

```

2.7.10 Link-Cut Tree

Maintain a forest of trees with values associated with its nodes, while supporting both dynamic queries and dynamic updates of all values on any path between two nodes in a given tree. In addition, support testing of whether two nodes are connected in the forest, as well as the merging and splitting of trees by adding or removing specific edges. Link/cut forests divide each of its trees into vertex-disjoint paths, each represented by a splay tree.

The query operation is defined by an associative `combine()` function. The default code below assumes a numerical forest type, defining queries for the “min” of the target range. Another possible query operation is “sum”, in which case `combine(a, b)` should return `a + b`. For direction-independent path queries, `combine()` should also be commutative; otherwise, store enough information in each aggregate to combine paths in the required order.

The update operation is defined by `apply_delta()` and `compose_deltas()`. A delta must act on an aggregate summary of a path of length `len`: `apply_delta(v, d, len)` returns the aggregate after applying update `d` to every element represented by aggregate `v`. Pending deltas are combined in chronological order by `compose_deltas(old, d)`, meaning “apply `old`, then apply `d`”. These hooks do not support arbitrary query/update pairings; the delta operation must distribute over `combine()`, and composed deltas must have the same effect as applying their updates sequentially. The default code below defines updates that “set” a path’s nodes to a new

value. For range increment updates, `apply_delta(v, d, len)` would return `v + d` for min/max queries, or `v + d * len` for sum queries, and `compose_deltas(old, d)` would return `old + d`.

- `LinkCut<T>()` constructs an empty forest.
- `size()` returns the number of nodes in the forest.
- `trees()` returns the number of trees in the forest.
- `add_node(i, v)` adds a new single-node tree to the forest, labeled with the integer `i` and with value initialized to `v`.
- `is_connected(a, b)` returns whether nodes `a` and `b` are connected.
- `link(a, b)` adds an edge between the nodes `a` and `b`, both of which must exist and not be connected.
- `cut(a, b)` removes the edge between the nodes `a` and `b`, both of which must exist and be connected.
- `query(a, b)` returns the result of `combine()` applied to all values on the path from the node `a` to node `b`.
- `update(a, b, d)` modifies all the values on the path from node `a` to node `b` by respectively applying the delta `d`.
- `reroot(i)` makes node `i` the root of its tree.
- `find_root(i)` returns the label of the root of the tree containing node `i`.
- `lca(a, b)` returns the lowest common ancestor of `a` and `b` relative to the tree's current root.

The forest is unrooted: a tree's root is whichever node was most recently established as such. `reroot(i)` sets it explicitly, while `link`, `query`, and `update` reroot implicitly (the latter two at their first argument). `find_root(i)` and `lca(a, b)` are therefore relative to the current root, so call `reroot(r)` first whenever a specific root `r` is intended.

Time Complexity:

- $O(1)$ per call to the constructor, `size()`, and `trees()`.
- $O(\log n)$ amortized per call to all other operations, where n is the number of nodes.

Space Complexity:

- $O(n)$ for storage of the forest, where n is the number of nodes.
- $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cstdint>
#include <stdexcept>
#include <unordered_map>

template<typename T>
class LinkCut {
    static T combine(const T &a, const T &b) { return std::min(a, b); }
    static T apply_delta(const T &v, const T &d, int len) { return d; }
    static T compose_deltas(const T &old, const T &d) { return d; }

    struct Node {
        int id;
        T value, subtree_value, delta;
        int size;
        bool rev, pending;
        Node *left, *right, *parent;

        Node(int id, const T &v)
            : id(id),
              value(v),
              subtree_value(v),
              size(1),
              rev(false),
              pending(false),
              left(nullptr),
              right(nullptr),
              parent(nullptr) {}

        inline bool is_root() const {
            return parent == nullptr || (parent->left != this && parent->right != this);
        }
    };
};
```



```

}

inline T get_subtree_value() const {
    return pending ? apply_delta(subtree_value, delta, size) : subtree_value;
}

void push() {
    if (rev) {
        rev = false;
        std::swap(left, right);
        if (left != nullptr) {
            left->rev = !left->rev;
        }
        if (right != nullptr) {
            right->rev = !right->rev;
        }
    }
    if (pending) {
        value = apply_delta(value, delta, 1);
        subtree_value = apply_delta(subtree_value, delta, size);
        if (left != nullptr) {
            left->delta = left->pending ? compose_deltas(left->delta, delta) : delta;
            left->pending = true;
        }
        if (right != nullptr) {
            right->delta = right->pending ? compose_deltas(right->delta, delta) : delta;
            right->pending = true;
        }
        pending = false;
    }
}

void update() {
    size = 1;
    subtree_value = value;
    if (left != nullptr) {
        // Combine in in-order (left, value, right) so non-commutative aggregates stay correct.
        subtree_value = combine(left->get_subtree_value(), subtree_value);
        size += left->size;
    }
    if (right != nullptr) {
        subtree_value = combine(subtree_value, right->get_subtree_value());
        size += right->size;
    }
}

};

int num_trees;
std::unordered_map<int, Node *> nodes;

static void connect(Node *child, Node *parent, bool is_left) {
    if (child != nullptr) {
        child->parent = parent;
    }
    if (is_left) {
        parent->left = child;
    } else {
        parent->right = child;
    }
}

static void rotate(Node *n) {
    Node *parent = n->parent, *grandparent = parent->parent;
    bool parent_is_root = parent->is_root(), is_left = (n == parent->left);
    connect(is_left ? n->right : n->left, parent, is_left);
    connect(parent, n, !is_left);
    if (parent_is_root) {
        if (n != nullptr) {

```

```

        n->parent = grandparent;
    }
} else {
    connect(n, grandparent, parent == grandparent->left);
}
parent->update();
}

static void splay(Node *n) {
    while (!n->is_root()) {
        Node *parent = n->parent, *grandparent = parent->parent;
        if (!parent->is_root()) {
            grandparent->push();
        }
        parent->push();
        n->push();
        if (!parent->is_root()) {
            if ((n == parent->left) == (parent == grandparent->left)) {
                rotate(parent);
            } else {
                rotate(n);
            }
        }
        rotate(n);
    }
    n->push();
    n->update();
}

// Convention: a node's left child leads toward the tree's root, so an exposed path is ordered by
// depth with the root as its leftmost node (matching the standard link/cut tree presentation).
// cut() and find_root() rely on this orientation.
static Node *expose(Node *n) {
    Node *prev = nullptr;
    for (Node *curr = n; curr != nullptr; curr = curr->parent) {
        splay(curr);
        curr->right = prev;
        curr->update();
        prev = curr;
    }
    splay(n);
    return prev;
}

// Helper variables.
Node *u, *v;

void get_uv(int a, int b) {
    auto it1 = nodes.find(a);
    auto it2 = nodes.find(b);
    if (it1 == nodes.end() || it2 == nodes.end()) {
        throw std::runtime_error("Queried node ID does not exist in forest.");
    }
    u = it1->second;
    v = it2->second;
}

public:
    LinkCut() : num_trees(0) {}

    ~LinkCut() {
        for (auto &[key, node] : nodes) {
            delete node;
        }
    }

    LinkCut(const LinkCut &) = delete;
    LinkCut &operator=(const LinkCut &) = delete;

```

```

int size() const { return static_cast<int>(nodes.size()); }
int trees() const { return num_trees; }

void add_node(int i, const T &v = T()) {
    if (auto it = nodes.find(i); it != nodes.end()) {
        throw std::runtime_error("Cannot add a node with an existing ID.");
    }
    Node *n = new Node(i, v);
    expose(n);
    n->rev = !n->rev;
    nodes[i] = n;
    num_trees++;
}

bool is_connected(int a, int b) {
    get_uv(a, b);
    if (a == b) {
        return true;
    }
    expose(u);
    expose(v);
    return u->parent != nullptr;
}

void link(int a, int b) {
    if (is_connected(a, b)) {
        throw std::runtime_error("Cannot link nodes that are already connected.");
    }
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    u->parent = v;
    num_trees--;
}

void cut(int a, int b) {
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    expose(v);
    if (v->left != u || u->right != nullptr) {
        throw std::runtime_error("Cannot cut edge that does not exist.");
    }
    v->left->parent = nullptr;
    v->left = nullptr;
    num_trees++;
}

T query(int a, int b) {
    if (!is_connected(a, b)) {
        throw std::runtime_error("Cannot query nodes that are not connected.");
    }
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    expose(v);
    return v->get_subtree_value();
}

void update(int a, int b, const T &d) {
    if (!is_connected(a, b)) {
        throw std::runtime_error("Cannot update nodes that are not connected.");
    }
    get_uv(a, b);
    expose(u);
    u->rev = !u->rev;
    expose(v);
    v->delta = v->pending ? compose_deltas(v->delta, d) : d;
}

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    LinkCut<int> lcf;
    lcf.add_node(0, 10);
    lcf.add_node(1, 40);
    lcf.add_node(2, 20);
    lcf.add_node(3, 10);
    lcf.add_node(4, 30);
    assert(lcf.size() == 5);
    assert(lcf.trees() == 5);
    lcf.link(0, 1);
    lcf.link(1, 2);
    lcf.link(2, 3);
    lcf.link(2, 4);
    assert(lcf.trees() == 1);

    // v=10          v=40          v=20          v=10
    // 0-----1-----2-----3
    //                |
    //                -----4
```

```

//                                     v=30
assert(lcf.query(1, 4) == 20);

// find_root and lca are relative to the current root, set explicitly via reroot.
lcf.reroot(0);
assert(lcf.find_root(3) == 0 && lcf.find_root(4) == 0);
assert(lcf.lca(3, 4) == 2); // Paths 3-2 and 4-2 meet at 2 under root 0.
assert(lcf.lca(1, 4) == 1); // 1 is an ancestor of 4 under root 0.
lcf.reroot(4);
assert(lcf.lca(0, 3) == 2); // Rerooting at 4 changes the LCA of 0 and 3.

lcf.update(1, 1, 100);
lcf.update(2, 4, 100);

// v=10      v=100      v=100      v=10
// 0-----1-----2-----3
//                               |
//                               -----4
//                               v=100
assert(lcf.query(4, 4) == 100);
assert(lcf.query(0, 4) == 10);
assert(lcf.query(3, 4) == 10);
lcf.cut(1, 2);

// v=10      v=100      v=100      v=0
// 0-----1      2-----3
//                               |
//                               -----4
//                               v=100
assert(lcf.trees() == 2);
assert(!lcf.is_connected(1, 2));
assert(!lcf.is_connected(0, 4));
assert(lcf.is_connected(2, 3));
return 0;
}

```

Chapter 3

Strings

3.1 String Utilities

Common string functions, many of which already have standard STL equivalents. Most of the following implementations are presented for educational purposes, and are not heavy optimized. They often depend on certain `std::string` functions that have unspecified complexity.

Time Complexity: $O(n)$ per call to most operations, where n is the length of the input string or total length of all input strings. Exceptions are noted with the individual functions below.

Space Complexity:

- $O(n)$ auxiliary heap space per call to operations that return a new string or vector of strings.
- $O(1)$ auxiliary space per call to in-place operations.

```
#include <cctype>
#include <regex>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
using std::string;
```

Integer Conversion:

- `to_str(i)` returns the string representation of integer `i`, much like `std::to_string()`.
- `to_int(s)` returns the integer representation of string `s`, much like `std::atoi()`, except here we handle special cases of overflow by throwing an exception.
- `itoa(value, &str, base)` implements the non-standard C function which converts `value` into a C string, storing it into pointer `str` in the given `base`. For more generalized base conversion, see the math utilities section.
- `is_integer(s)` returns whether `s` is a valid base-10 integer literal with an optional sign.
- `is_number(s)` returns whether `s` is a decimal number literal with optional sign, decimal point, and exponent.

```
template<typename Int>
string to_str(Int i) {
    std::ostringstream oss;
    oss << i;
    return oss.str();
}
```

```
int to_int(const string &s) {
    std::istringstream iss(s);
```

```

int res;
if (!(iss >> res)) {
    throw std::runtime_error("to_int failed");
}
return res;
}

char *itoa(int value, char *str, int base = 10) {
    if (base < 2 || base > 36) {
        *str = '\0';
        return str;
    }
    char *ptr = str, *ptr1 = str, tmp_c;
    int tmp_v;
    do {
        tmp_v = value;
        value /= base;
        *ptr++ =
            "zyxwvutsrqponmlkjihgfedcba9876543210123456789"
            "abcdefghijklmnopqrstuvwxyz"[35 + (tmp_v - value * base)];
    } while (value);
    if (tmp_v < 0) {
        *ptr++ = '-';
    }
    for (*ptr-- = '\0'; ptr1 < ptr; *ptr1++ = tmp_c) {
        tmp_c = *ptr;
        *ptr-- = *ptr1;
    }
    return str;
}

bool is_integer(const string &s) {
    int i = (s.empty() || (s[0] != '-' && s[0] != '+')) ? 0 : 1;
    if (i == static_cast<int>(s.size())) {
        return false;
    }
    for (; i < static_cast<int>(s.size()); i++) {
        if (!isdigit(static_cast<unsigned char>(s[i]))) {
            return false;
        }
    }
    return true;
}

bool is_number(const string &s) {
    int i = (s.empty() || (s[0] != '-' && s[0] != '+')) ? 0 : 1;
    bool seen_digit = false, seen_dot = false;
    for (; i < static_cast<int>(s.size()); i++) {
        unsigned char c = s[i];
        if (isdigit(c)) {
            seen_digit = true;
        } else if (c == '.' && !seen_dot) {
            seen_dot = true;
        } else {
            break;
        }
    }
    if (!seen_digit) {
        return false;
    }
    if (i < static_cast<int>(s.size()) && (s[i] == 'e' || s[i] == 'E')) {
        i++;
        if (i < static_cast<int>(s.size()) && (s[i] == '-' || s[i] == '+')) {
            i++;
        }
        bool seen_exp_digit = false;
        for (; i < static_cast<int>(s.size()); i++) {
            if (!isdigit(static_cast<unsigned char>(s[i]))) {

```

```

        return false;
    }
    seen_exp_digit = true;
}
return seen_exp_digit;
}
return i == static_cast<int>(s.size());
}

```

Case Conversion:

- `to_upper(s)` returns `s` with all alphabetical characters converted to uppercase.
- `to_lower(s)` returns `s` with all alphabetical characters converted to lowercase.
- `to_title(s)` returns the title case representation of string `s`, where the first letter of every word (consecutive alphabetical characters) is capitalized.

```

string to_upper(const string &s) {
    string res;
    for (char c : s) {
        res.push_back(static_cast<char>(toupper(c)));
    }
    return res;
}

string to_lower(const string &s) {
    string res;
    for (char c : s) {
        res.push_back(static_cast<char>(tolower(c)));
    }
    return res;
}

string to_title(const string &s) {
    string res;
    auto lower = [](unsigned char c) { return static_cast<char>(tolower(c)); };
    auto upper = [](unsigned char c) { return static_cast<char>(toupper(c)); };
    char prev = '\0';
    for (char c : s) {
        res.push_back(isalpha(static_cast<unsigned char>(prev)) ? lower(c) : upper(c));
        prev = res.back();
    }
    return res;
}

```

Stripping:

- `lstrip(s)` strips the left side of `s` in-place (that is, the input is modified) using the given delimiters and returns a reference to the stripped string.
- `rstrip(s)` strips the right side of `s` in-place using the given delimiters and returns a reference to the stripped string.
- `strip(s)` strips both sides of `s` in-place and returns a reference to the stripped string.
- `ltrimmed(s)`, `rtrimmed(s)`, and `trimmed(s)` do not modify `s` and returns stripped copies.

```

string &lstrip(string &s, const string &delim = " \n\t\v\f\r") {
    std::size_t pos = s.find_first_not_of(delim);
    if (pos != string::npos) {
        s.erase(0, pos);
    }
    return s;
}

string &rstrip(string &s, const string &delim = " \n\t\v\f\r") {
    std::size_t pos = s.find_last_not_of(delim);

```



```

    if (pos != string::npos) {
        s.erase(pos + 1);
    }
    return s;
}

string &strip(string &s, const string &delim = " \n\t\v\f\r") {
    return lstrip(rstrip(s));
}

string ltrimmed(string s, const string &delim = " \n\t\v\f\r") {
    return lstrip(s, delim);
}

string rtrimmed(string s, const string &delim = " \n\t\v\f\r") {
    return rstrip(s, delim);
}

string trimmed(string s, const string &delim = " \n\t\v\f\r") {
    return strip(s, delim);
}

```

Find and Replace:

- `starts_with(s, prefix)` returns whether `s` begins with `prefix`.
- `ends_with(s, suffix)` returns whether `s` ends with `suffix`.
- `contains(s, needle)` returns whether `needle` occurs in `s`.
- `find_all(haystack, needle)` returns a vector of all positions where the string `needle` appears in the string `haystack`. Matches may overlap; e.g. `find_all("aaa", "aa")` returns `{0, 1}`.
- `count(haystack, needle)` returns the number of non-overlapping occurrences of `needle` in `haystack`, like Python's `str.count`. This differs from `find_all().size()`, which counts overlapping matches; e.g. `count("aaa", "aa")` returns `1`, but `find_all("aaa", "aa")` finds two.
- `replace(s, old, replacement)` returns a copy of `s` with all occurrences of the string `old` replaced with the given `replacement`.
- `regex_find_all(s, pattern)` returns every substring of `s` matched by `pattern`.
- `regex_groups(s, pattern)` returns the capture groups of the first match of `pattern` in `s`, excluding the full match. If there is no match, returns an empty vector.
- `regex_replace_all(s, pattern, replacement)` returns a copy of `s` with every regex match replaced by `replacement`.

```

bool starts_with(const string &s, const string &prefix) {
    return s.size() >= prefix.size() && s.compare(0, prefix.size(), prefix) == 0;
}

bool ends_with(const string &s, const string &suffix) {
    return s.size() >= suffix.size() &&
        s.compare(s.size() - suffix.size(), suffix.size(), suffix) == 0;
}

bool contains(const string &s, const string &needle) {
    return s.find(needle) != string::npos;
}

std::vector<int> find_all(const string &haystack, const string &needle) {
    std::vector<int> res;
    std::size_t pos = haystack.find(needle, 0);
    while (pos != string::npos) {
        res.push_back(pos);
        pos = haystack.find(needle, pos + 1);
    }
    return res;
}

```

```

int count(const string &haystack, const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    int res = 0;
    std::size_t pos = haystack.find(needle, 0);
    while (pos != string::npos) {
        res++;
        pos = haystack.find(needle, pos + needle.size());
    }
    return res;
}

string replace(const string &s, const string &old, const string &replacement) {
    if (old.empty()) {
        return s;
    }
    string res(s);
    std::size_t pos = 0;
    while ((pos = res.find(old, pos)) != string::npos) {
        res.replace(pos, old.length(), replacement);
        pos += replacement.length();
    }
    return res;
}

std::vector<string> regex_find_all(const string &s, const string &pattern) {
    std::regex re(pattern);
    return {std::sregex_token_iterator(s.begin(), s.end(), re, 0), std::sregex_token_iterator()};
}

std::vector<string> regex_groups(const string &s, const string &pattern) {
    std::smatch match;
    if (!std::regex_search(s, match, std::regex(pattern))) {
        return {};
    }
    std::vector<string> res;
    for (int i = 1; i < static_cast<int>(match.size()); i++) {
        res.push_back(match[i].str());
    }
    return res;
}

string regex_replace_all(const string &s, const string &pattern, const string &replacement) {
    return std::regex_replace(s, std::regex(pattern), replacement);
}

```

Joining and Splitting:

- `join(v, delim)` returns the strings in vector `v` concatenated, separated by the given delimiter.
- `repeat(s, n)` returns `s` concatenated with itself `n` times, like Python's `s * n`. For a single repeated character, prefer `std::string(n, ch)`.
- `ljust(s, width, ch)` returns `s` left-justified (padded on the right with `ch`) to at least `width` characters, like Python's `str.ljust`.
- `rjust(s, width, ch)` returns `s` right-justified (padded on the left with `ch`) to at least `width` characters, like Python's `str.rjust`.
- `center(s, width, ch)` returns `s` centered in a field of at least `width` characters by padding both sides with `ch`, like Python's `str.center`. If the padding is uneven, the extra character goes on the right.
- `split(s, char delim)` returns a vector of tokens of `s`, split on a single character delimiter. Empty tokens are skipped, so `split("a::b", ':')` returns `{"a", "b"}`, not `{"a", "", "b"}`.
- `split(s, string delim)` returns a vector of tokens of `s`, split on a set of many possible single character delimiters. All characters of `delim` will be removed from `s`, and the remaining token(s) of `s` will be added

sequentially to a vector and returned. As with the first version, empty tokens are skipped. For example,

`split("a::b", ":")` returns `{"a", "b"}`, not `{"a", "", "b"}`.

- `explode(s, delim)` returns a vector of tokens of `s`, split on the entire delimiter string `delim`. Unlike the `split()` functions above, `delim` is treated as a contiguous boundary string, not merely a set of possible boundary characters. This will not skip empty tokens. For example, `explode("a:::b", ":::")` yields `{"a", "", "b"}`, not `{"a", "b"}`.
- `explode(s, char delim)` is the single-character delimiter version that also preserves empty tokens.

```
string join(const std::vector<string> &v, const string &delim = " ") {
    string res;
    res.reserve(v.size() + delim.size() * (v.size() - 1));
    for (int i = 0; i < static_cast<int>(v.size()); i++) {
        res += (i > 0 ? delim : "") + v[i];
    }
    return res;
}

string repeat(const string &s, int n) {
    string res;
    res.reserve(s.size() * n);
    for (int i = 0; i < n; i++) {
        res += s;
    }
    return res;
}

string ljust(const string &s, int width, char ch = ' ') {
    int pad = width - static_cast<int>(s.size());
    return pad > 0 ? s + string(pad, ch) : s;
}

string rjust(const string &s, int width, char ch = ' ') {
    int pad = width - static_cast<int>(s.size());
    return pad > 0 ? string(pad, ch) + s : s;
}

string center(const string &s, int width, char ch = ' ') {
    int pad = width - static_cast<int>(s.size());
    if (pad <= 0) {
        return s;
    }
    int left = pad / 2;
    return string(left, ch) + s + string(pad - left, ch);
}

std::vector<string> split(const string &s, char delim) {
    std::vector<string> res;
    std::stringstream ss(s);
    string curr;
    while (std::getline(ss, curr, delim)) {
        if (!curr.empty()) {
            res.push_back(curr);
        }
    }
    return res;
}

std::vector<string> split(const string &s, const string &delim = " \n\t\v\f\r") {
    std::vector<string> res;
    string curr;
    for (char c : s) {
        if (delim.find(c) == string::npos) {
            curr += c;
        } else if (!curr.empty()) {
            res.push_back(curr);
            curr = "";
        }
    }
    if (!curr.empty()) {
        res.push_back(curr);
    }
    return res;
}
```

```

    }
}
if (!curr.empty()) {
    res.push_back(curr);
}
return res;
}

std::vector<string> explode(const string &s, const string &delim) {
    std::vector<string> res;
    std::size_t last = 0, next = 0;
    while ((next = s.find(delim, last)) != string::npos) {
        res.push_back(s.substr(last, next - last));
        last = next + static_cast<int>(delim.size());
    }
    res.push_back(s.substr(last));
    return res;
}

std::vector<string> explode(const string &s, char delim) {
    std::vector<string> res;
    std::size_t last = 0, next = 0;
    while ((next = s.find(delim, last)) != string::npos) {
        res.push_back(s.substr(last, next - last));
        last = next + 1;
    }
    res.push_back(s.substr(last));
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(to_str(123) + "4" == "1234");
    assert(to_int("1234") == 1234);
    vector<char> buffer(50);
    assert(string(itoa(1750, buffer.data(), 10)) == "1750");
    assert(string(itoa(1750, buffer.data(), 16)) == "6d6");
    assert(string(itoa(1750, buffer.data(), 2)) == "11011010110");
    assert(is_integer("-123") && !is_integer("12.3"));
    assert(is_number("-.5e+2") && is_number("123.") && !is_number("1e"));

    assert(to_upper("Hello world") == "HELLO WORLD");
    assert(to_lower("Hello World") == "hello world");
    assert(to_title("hello world") == "Hello World");

    string s("  abc \n");
    string t = s;
    assert(lstrip(s) == "abc \n");
    assert(rstrip(s) == strip(t));
    assert(trimmed(" \t abc \n") == "abc");

    vector<int> pos;
    pos.push_back(0);
    pos.push_back(7);
    assert(starts_with("abracadabra", "abra"));
    assert(ends_with("abracadabra", "dabra"));
    assert(contains("abracadabra", "cad"));
    assert(find_all("abracadabra", "ab") == pos);
    assert(count("abracadabra", "ab") == 2);
    assert(count("aaa", "aa") == 1 && find_all("aaa", "aa").size() == 2);
    assert(replace("abcdabba", "ab", "00") == "00cd00ba");
    assert(join(regex_find_all("a12b345", "[0-9]+"), "|") == "12|345");
}

```

```

assert(join(regex_groups("abc-123", "([a-z]+)-([0-9]+)", "|") == "abc|123");
assert(regex_replace_all("a12b345", "[0-9]+", "#") == "a#b#");

assert(repeat("ab", 3) == "ababab");
assert(rjust("42", 5, '0') == "00042");
assert(ljust("hi", 4, '.') == "hi..");
assert(center("ab", 5, '*') == "*ab*");
assert(join(split("a\nb\ncde\nf", '\n'), "|") == "a|b|cde|f"); // split v1
assert(join(split("a::b", ':'), "|") == "a|b"); // split v1 skips empty tokens
assert(join(split("a::b,cde:,f", ":", "|"), "|") == "a|b|cde|f"); // split v2
assert(join(explode("a..b.cde...f", ".."), "|") == "a|b.cde|f");
assert(join(explode("a::b:", ':'), "|") == "a||b|");
return 0;
}

```

3.2 Hashing

3.2.1 Hash Functions

Provides a small library of non-cryptographic hash functions and hash functors for common contest keys. These functions are useful for fingerprinting values, combining keys, seeding randomized algorithms, and defining `std::unordered_map` keys for pairs, tuples, and vectors. They are not suitable for passwords, signatures, or adversarial security.

The standalone functions below are deterministic. The `IntHasher` functor adds a per-run random seed before mixing, which is useful for making integer-keyed hash tables harder to hack in open-test contests.

The composite hashers separate the mixing primitive (`mix64`/`hash_combine64`) from each type's traversal of its parts, so swapping in a different mixing function only requires editing one place.

FNV-1a (Fowler-Noll-Vo) is a tiny hash for variable-length byte sequences. Starting from a fixed offset basis, it XORs each byte into the accumulator and then multiplies by a fixed prime; the `1a` variant XORs before multiplying, which mixes slightly better than the original FNV-1. Reach for it when the key is a string or byte buffer, whereas `mix32` and `mix64` are mixers for a single fixed-width integer. Its appeal is simplicity: a few lines, no lookup tables, and no seed, with distribution good enough for the short keys common in contests. For very long inputs a block hash such as MurmurHash or xxHash is faster and mixes more thoroughly, and because FNV-1a is unseeded it offers no protection against adversarial inputs.

- `mix32(x)` mixes a 32-bit unsigned integer `x`, using MurmurHash3's `fmix32` mixer.
- `mix64(x)` mixes a 64-bit unsigned integer `x`, using the `SplitMix64` mixer.
- `hash32(x)` and `hash64(x)` hash any integer type that can be converted to `uint32_t` or `uint64_t`, respectively.
- `hash_combine32(h, x)` and `hash_combine64(h, x)` fold another already-hashed value into an existing hash accumulator.
- `hash_float(x)` and `hash_double(x)` hash floating-point bit patterns, normalizing `-0.0` to `0.0`. Different NaN representations may still hash differently.
- `hash_string_fnv1a32(s)` and `hash_string_fnv1a64(s)` compute FNV-1a hashes of string `s`.
- `hash_range32(first, last)` and `hash_range64(first, last)` hash a sequence of integer-like values.
- `PairIntHasher` is a self-contained hasher for `std::pair<int, int>` keys, written without any dependency on the templates below so that it can be copy-pasted on its own.
- `IntHasher<Int>`, `PairHasher<A, B>`, `TupleHasher<Ts...>`, and `VectorHasher<T>` are hash functors passed as the third template argument of `std::unordered_map` or `std::unordered_set`.
- `GenericHasher<T>` recursively handles integer, pair, tuple, and vector keys, and falls back to `std::hash<T>` for other hashable types.
- Specializations of `std::hash` for `std::pair` and `std::tuple` are also provided, letting those keys be used in `std::unordered_map`/`std::unordered_set` without an explicit hasher argument. Specializing `std::hash` for

purely standard types is technically nonstandard, so the functor forms above are the portable alternative. (Vector keys are left to the functors to avoid clashing with the standard `std::hash<std::vector<bool>>.`)

Time Complexity:

- $O(1)$ per call to the integer, combiner, and floating-point hash functions.
- $O(n)$ per call to string and range hashing functions, where n is the number of elements processed.
- $O(n)$ per call to `VectorHasher<T>::operator()`, where n is the vector size.

Space Complexity: $O(1)$ auxiliary.

```
#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <functional>
#include <string>
#include <tuple>
#include <type_traits>
#include <unordered_map>
#include <utility>
#include <vector>

// MurmurHash3's fmix32 mixer.
uint32_t mix32(uint32_t x) {
    x ^= x >> 16;
    x *= 0x85ebca6bU;
    x ^= x >> 13;
    x *= 0xc2b2ae35U;
    return x ^ (x >> 16);
}

// SplitMix64 mixer.
uint64_t mix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<typename Int>
uint32_t hash32(Int x) {
    return mix32(static_cast<uint32_t>(x));
}

template<typename Int>
uint64_t hash64(Int x) {
    return mix64(static_cast<uint64_t>(x));
}

uint32_t hash_combine32(uint32_t h, uint32_t x) {
    return mix32(h ^ (x + 0x9e3779b9U + (h << 6) + (h >> 2)));
}

uint64_t hash_combine64(uint64_t h, uint64_t x) {
    return mix64(h ^ (x + 0x9e3779b97f4a7c15ULL + (h << 6) + (h >> 2)));
}

uint32_t hash_float(float x) {
    if (x == 0.0f) {
        x = 0.0f; // Normalize -0.0.
    }
    uint32_t bits = 0;
    std::memcpy(&bits, &x, sizeof(bits));
    return mix32(bits);
}
```

```

uint64_t hash_double(double x) {
    if (x == 0.0) {
        x = 0.0; // Normalize -0.0.
    }
    uint64_t bits = 0;
    std::memcpy(&bits, &x, sizeof(bits));
    return mix64(bits);
}

uint32_t hash_string_fnv1a32(const std::string &s) {
    uint32_t h = 2166136261U;
    for (unsigned char c : s) {
        h ^= c;
        h *= 16777619U;
    }
    return h;
}

uint64_t hash_string_fnv1a64(const std::string &s) {
    uint64_t h = 14695981039346656037ULL;
    for (unsigned char c : s) {
        h ^= c;
        h *= 1099511628211ULL;
    }
    return h;
}

template<typename It>
uint32_t hash_range32(It first, It last) {
    uint32_t h = 0;
    for (It it = first; it != last; ++it) {
        h = hash_combine32(h, hash32(*it));
    }
    return h;
}

template<typename It>
uint64_t hash_range64(It first, It last) {
    uint64_t h = 0;
    for (It it = first; it != last; ++it) {
        h = hash_combine64(h, hash64(*it));
    }
    return h;
}

template<typename Int>
struct IntHasher {
    std::size_t operator()(Int x) const {
        static const uint64_t RAND_SEED = std::chrono::steady_clock::now().time_since_epoch().count();
        return static_cast<std::size_t>(mix64(static_cast<uint64_t>(x) + RAND_SEED));
    }
};

template<typename T>
struct GenericHasher;

// Use std::hash by default.
template<typename T, bool IsInteger>
struct ScalarHasher {
    std::size_t operator()(const T &x) const { return std::hash<T>()(x); }
};

// Optional: Use our seeded hasher for ints in open-hacking environments.
template<typename T>
struct ScalarHasher<T, true> {
    std::size_t operator()(T x) const { return IntHasher<T>{}(x); }
};

```

```

// A self-contained hasher for std::pair<int, int> keys: copy just this struct when you do not need
// the rest of this file. The two 32-bit halves pack losslessly into one 64-bit word, which the
// SplitMix64 mixer (the same one used by mix64 above) then scrambles. Add a random seed to `key`
// before mixing, as IntHasher does, if you need anti-hacking protection.
struct PairIntHasher {
    std::size_t operator()(const std::pair<int, int> &p) const {
        uint64_t key = (static_cast<uint64_t>(static_cast<uint32_t>(p.first)) << 32) |
            static_cast<uint32_t>(p.second);
        key += 0x9e3779b97f4a7c15ULL;
        key = (key ^ (key >> 30)) * 0xbf58476d1ce4e5b9ULL;
        key = (key ^ (key >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<std::size_t>(key ^ (key >> 31));
    }
};

template<typename A, typename B>
struct PairHasher {
    std::size_t operator()(const std::pair<A, B> &p) const {
        uint64_t h = 0;
        h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<A>()(p.first)));
        h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<B>()(p.second)));
        return static_cast<std::size_t>(h);
    }
};

template<typename T>
struct VectorHasher {
    std::size_t operator()(const std::vector<T> &v) const {
        uint64_t h = 0;
        for (const auto &x : v) {
            h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<T>()(x)));
        }
        h = hash_combine64(h, v.size()); // Mix in the length so different sizes (e.g. empty) separate.
        return static_cast<std::size_t>(h);
    }
};

template<class... Ts>
struct TupleHasher {
    std::size_t operator()(const std::tuple<Ts...> &t) const {
        uint64_t h = 0;
        std::apply(
            [&](const Ts &...xs) {
                (h = hash_combine64(h, static_cast<uint64_t>(GenericHasher<Ts>()(xs)))), ...;
            },
            t
        );
        return static_cast<std::size_t>(h);
    }
};

template<typename T>
struct GenericHasher : ScalarHasher<T, std::is_integral<T>::value> {};

template<typename A, typename B>
struct GenericHasher<std::pair<A, B>> : PairHasher<A, B> {};

template<class... Ts>
struct GenericHasher<std::tuple<Ts...>> : TupleHasher<Ts...> {};

template<typename T>
struct GenericHasher<std::vector<T>> : VectorHasher<T> {};

// Specializing std::hash lets pair and tuple keys be used directly in std::unordered_map and
// std::unordered_set, with no explicit hasher template argument. The C++ standard only sanctions
// specializing std::hash when a user-defined type is involved, so the functors above remain the
// portable choice; these specializations are a widely supported contest convenience. Vector keys
// are intentionally left out here to avoid clashing with the standard library's own

```



```
// std::hash<std::vector<bool>>; pass VectorHasher or GenericHasher explicitly for those.
namespace std {

template<typename A, typename B>
struct hash<pair<A, B>> {
    std::size_t operator()(const pair<A, B> &p) const { return ::GenericHasher<pair<A, B>>()(p); }
};

template<class... Ts>
struct hash<tuple<Ts...>> {
    std::size_t operator()(const tuple<Ts...> &t) const { return ::GenericHasher<tuple<Ts...>>()(t); }
};

} // namespace std
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(mix32(123U) == mix32(123U));
    assert(mix32(123U) != mix32(124U));
    assert(hash32(-1) == hash32(-1));
    assert(hash64(123) == mix64(123ULL));
    assert(hash_float(-0.0f) == hash_float(0.0f));
    assert(hash_double(-0.0) == hash_double(0.0));
    assert(hash_string_fnv1a32("abc") == hash_string_fnv1a32("abc"));
    assert(hash_string_fnv1a64("abc") != hash_string_fnv1a64("acb"));

    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    assert(hash_range32(v.begin(), v.end()) == hash_range32(v.begin(), v.end()));
    assert(hash_range64(v.begin(), v.end()) == hash_range64(v.begin(), v.end()));

    unordered_map<int64_t, int, IntHasher<int64_t>> count;
    count[1000000000000LL] = 7;
    assert(count[1000000000000LL] == 7);

    using Point = pair<int, int>;
    unordered_map<Point, int, PairHasher<int, int>> dist;
    dist[Point(3, 4)] = 5;
    assert(dist[Point(3, 4)] == 5);

    // The standalone PairIntHasher is the quick choice for pair<int, int> keys.
    unordered_map<Point, int, PairIntHasher> dist_fast;
    dist_fast[Point(3, 4)] = 5;
    assert(dist_fast[Point(3, 4)] == 5);

    unordered_map<vector<int>, int, VectorHasher<int>> seen;
    seen[v] = 1;
    assert(seen[v] == 1);

    using State = pair<vector<int>, double>;
    unordered_map<State, int, GenericHasher<State>> dp;
    dp[State(v, 4.5)] = 9;
    assert(dp[State(v, 4.5)] == 9);

    // Tuple keys via the explicit functor.
    using Triple = tuple<int, char, double>;
    unordered_map<Triple, int, GenericHasher<Triple>> grid;
    grid[Tuple(1, 'a', 3.14)] = 6;
    assert(grid[Tuple(1, 'a', 3.14)] == 6);
```

```
// With the std::hash specializations, pair and tuple keys need no hasher argument.
unordered_map<Point, int> dist_default;
dist_default[Point(3, 4)] = 5;
assert(dist_default[Point(3, 4)] == 5);

unordered_map<Triple, int> grid_default;
grid_default[Triple(1, 'a', 3.14)] = 6;
assert(grid_default[Triple(1, 'a', 3.14)] == 6);
return 0;
}
```

3.2.2 Rolling Hash

Computes polynomial rolling hashes for sequences, supporting $O(1)$ contiguous subsequence hash queries after preprocessing. This is useful for probabilistic string matching, substring equality checks, repeated subarray detection, and binary-searching over candidate lengths.

Given a sequence $a_0, a_1, \dots, a_{n-1}$ and a value hash $h(a_i)$ for each element, the hash is the polynomial $H(a) = \sum_{i=0}^{n-1} h(a_i) B^{n-1-i} \pmod{M}$, where B is `HASH_BASE` and M is `HASH_MOD`. Equivalently, it is built left-to-right by the recurrence `pref[i + 1] = pref[i] * HASH_BASE + h(a[i])`. A subsequence hash for `[l, r)` is obtained by subtracting away the prefix before l : `pref[r] - pref[l] * pow(HASH_BASE, r - l)`.

The implementation works modulo the Mersenne prime $2^{61} - 1$, using `__uint128_t` for the multiplication where available and falling back to a double-and-add modular multiply otherwise. The base `HASH_BASE` should be changed or chosen randomly for open-hacking environments. With a fixed base, hashing remains fast and practical, but it is still probabilistic and should not be used as proof of equality when exact verification is required.

By default, each sequence value is cast to `uint64_t` and mixed. For non-integer element types, pass a custom value hasher that maps each element to a stable nonzero value in `[1, HASH_MOD)`.

- `RollingHash<T, ValueHasher>(first, last)` constructs prefix hashes for any iterator range of values accepted by the value hasher.
- `RollingHash<T, ValueHasher>(v)` constructs prefix hashes for vector `v`.
- `get(l, r)` returns the hash of the half-open subsequence `[l, r)`.
- `hash(first, last)` returns the hash of an iterator range.
- `hash(v)` returns the hash of vector `v`.
- `concat(left, right, right_len)` returns the hash of the concatenation of a sequence with hash `left` and a sequence with hash `right` and length `right_len`.

Time Complexity:

- $O(n)$ per constructor call and whole-sequence hash, where n is the sequence length.
- $O(1)$ per call to `get(l, r)` and `concat(left, right, right_len)`.

Space Complexity:

- $O(n)$ for storage of prefix hashes and powers, where n is the maximum sequence length processed so far.
- $O(1)$ auxiliary per query.

```
#include <cstdint>
#include <string>
#include <utility>
#include <vector>

const uint64_t HASH_MOD = (1ULL << 61) - 1;
const uint64_t HASH_BASE = 911382323ULL; // Change or randomize.

uint64_t hash_add(uint64_t a, uint64_t b) {
    uint64_t c = a + b;
    return c >= HASH_MOD ? c - HASH_MOD : c;
}
```

```

uint64_t hash_sub(uint64_t a, uint64_t b) {
    return a >= b ? a - b : a + HASH_MOD - b;
}

uint64_t hash_mul(uint64_t a, uint64_t b) {
#if defined(__SIZEOF_INT128__)
    __uint128_t p = static_cast<__uint128_t>(a) * b;
    return hash_add(static_cast<uint64_t>(p) & HASH_MOD, static_cast<uint64_t>(p >> 61));
#else
    // Portable fallback when no 128-bit type exists: double-and-add modular multiply.
    uint64_t res = 0;
    for (a %= HASH_MOD; b > 0; b >>= 1) {
        if (b & 1) {
            res = hash_add(res, a);
        }
        a = hash_add(a, a);
    }
    return res;
#endif
}

// SplitMix64 mixer.
uint64_t mix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

template<typename T>
struct RollingValueHasher {
    // +1 ensures the result is in [1, HASH_MOD) and never zero; a zero element in a polynomial
    // hash is invisible and enables trivial collisions.
    uint64_t operator()(const T &x) const { return mix64((uint64_t)x) % (HASH_MOD - 1) + 1; }
};

template<typename T, typename ValueHasher = RollingValueHasher<T>>
class RollingHash {
    static std::vector<uint64_t> pow_base;
    std::vector<uint64_t> pref;
    ValueHasher value_hasher;

    static void ensure_powers(int n) {
        while (static_cast<int>(pow_base.size()) <= n) {
            pow_base.push_back(hash_mul(pow_base.back(), HASH_BASE));
        }
    }

    template<typename It>
    void build(It first, It last) {
        pref.clear();
        pref.push_back(0);
        for (; first != last; ++first) {
            pref.push_back(hash_add(hash_mul(pref.back(), HASH_BASE), value_hasher(*first)));
        }
        ensure_powers(static_cast<int>(pref.size()) - 1);
    }

public:
    explicit RollingHash(const ValueHasher &hasher = ValueHasher()) : value_hasher(hasher) {
        ensure_powers(0);
        pref.push_back(0);
    }

    template<typename It>
    RollingHash(It first, It last, const ValueHasher &hasher = ValueHasher()) : value_hasher(hasher) {
        build(first, last);
    }

```

```

}

explicit RollingHash(const std::vector<T> &v, const ValueHasher &hasher = ValueHasher())
    : value_hasher(hasher) {
    build(v.begin(), v.end());
}

int size() const { return static_cast<int>(pref.size()) - 1; }

uint64_t get(int lo, int hi) const {
    return hash_sub(pref[hi], hash_mul(pref[lo], pow_base[hi - lo]));
}

template<typename It>
static uint64_t hash(It first, It last, const ValueHasher &hasher = ValueHasher()) {
    RollingHash<T, ValueHasher> h(first, last, hasher);
    return h.get(0, h.size());
}

static uint64_t hash(const std::vector<T> &v, const ValueHasher &hasher = ValueHasher()) {
    RollingHash<T, ValueHasher> h(v, hasher);
    return h.get(0, h.size());
}

static uint64_t concat(uint64_t left, uint64_t right, int right_len) {
    ensure_powers(right_len);
    return hash_add(hash_mul(left, pow_base[right_len]), right);
}
};

template<typename T, typename ValueHasher>
std::vector<uint64_t> RollingHash<T, ValueHasher>::pow_base(1, 1);

```

Example Usage

```

#include <cassert>
using namespace std;

using PointI = pair<int, int>;

struct PointHasher {
    uint64_t operator()(const PointI &p) const {
        // Pack the two 32-bit coordinates losslessly into one 64-bit key before mixing, so distinct
        // points never collide pre-mix (a small-multiplier combine like a * C + b would).
        uint64_t key = (static_cast<uint64_t>(static_cast<uint32_t>(p.first)) << 32) |
            static_cast<uint32_t>(p.second);
        return mix64(key) % (HASH_MOD - 1) + 1;
    }
};

int main() {
    string s = "abracadabra";
    RollingHash<char> hs(s.begin(), s.end());
    assert(hs.get(0, 4) == hs.get(7, 11)); // "abra" == "abra"
    assert(hs.get(0, 4) != hs.get(3, 7));  // "abra" != "acad"

    string a = "abc", b = "def";
    uint64_t ab = RollingHash<char>::concat(
        RollingHash<char>::hash(a.begin(), a.end()), RollingHash<char>::hash(b.begin(), b.end()),
        b.size()
    );
    string c = a + b;
    assert(ab == RollingHash<char>::hash(c.begin(), c.end()));

    vector<int> v;
    v.push_back(1);
}

```

```

v.push_back(2);
v.push_back(3);
v.push_back(1);
v.push_back(2);
RollingHash<int> hv(v);
assert(hv.get(0, 2) == hv.get(3, 5)); // [1, 2] == [1, 2]
assert(hv.get(0, 3) == RollingHash<int>::hash(v.begin(), v.begin() + 3));

vector<PointI> poly{{1, 2}, {3, 4}};
RollingHash<PointI, PointHasher> hp(poly);
assert((hp.get(0, 2) == RollingHash<PointI, PointHasher>::hash(poly)));
return 0;
}

```

3.2.3 Rolling Hash Deque

Maintain a polynomial rolling hash over a sequence of elements of an arbitrary type $\mathbb{T}$, under insertion and deletion at either end, in $O(1)$ time per operation. Each element is first reduced to a nonnegative integer “digit” by a user-supplied hasher, so any comparable type (characters, integers, whole words, tuples) can be hashed. This is the deque generalization of the classic Rabin-Karp sliding window: pushing at the back and popping from the front is exactly a moving window, and this structure additionally supports pushing at the front and popping from the back.

The sequence of digits is hashed with the front element as the most significant digit, so a sequence of length n has hash $\sum_{i=0}^{n-1} a_i B^{n-1-i}$ (the front element a_0 carries the highest power). This makes the four end operations pure $O(1)$ updates of a running accumulator:

- `push_back(c)`: $H \rightarrow H \cdot B + c$ (Horner’s step; the existing weights all shift up).
- `pop_front()`: $H \rightarrow H - a_0 \cdot B^{n-1}$ (drop the most significant term).
- `push_front(c)`: $H \rightarrow c \cdot B^n + H$ (the new front becomes the most significant term).
- `pop_back()`: $H \rightarrow (H - a_{n-1}) \cdot B^{-1}$ (drop the constant term, then renormalize).

Only `pop_back()` needs a modular inverse of the base, which is precomputed once. The weight of the front element, B^{n-1} , is maintained incrementally (multiplied by B on a push, by B^{-1} on a pop), so no power table or length-dependent exponentiation is needed. Each element’s digit is stored when pushed, so popping never re-invokes the hasher. Because the hash depends only on the current contents, two deques with equal sequences hash equally regardless of how they were built, and a sliding window can be compared against a fixed target in $O(1)$ per step.

`MOD1` and `MOD2` are two distinct primes, each kept below 2^{31} so that the product of any two residues fits in a 64-bit integer; hashing modulo both and comparing the resulting pair makes a collision astronomically unlikely. `BASE1` and `BASE2` are the corresponding polynomial bases, which should exceed the largest digit value and may be randomized per run to resist adversarial inputs.

The hasher’s output must be nonnegative. As with any polynomial hash, comparing sequences of different lengths is only safe when digits are positive (for example, map an alphabet to $[1, B)$) so that a leading zero cannot make a shorter sequence collide with a longer one.

- `HashDeque<T, Hash>(hasher)` constructs an empty deque whose `hasher` maps each element to its nonnegative integer digit. For integer-like $\mathbb{T}$ the default hasher casts the element to `uint64_t`, so `HashDeque<char>` and `HashDeque<int>` work with no hasher supplied.
- `size()` returns the number of elements, and `empty()` whether there are none.
- `push_back(x)` and `push_front(x)` append `x` at the back or front.
- `pop_back()` and `pop_front()` remove the back or front element; the deque must be nonempty.
- `hash()` returns the pair of hashes of the current sequence.
- `a == b` (and `a != b`) compares two deques: it checks equal length, then equal hash pairs, so two deques hold equal sequences with high probability if and only if they compare equal.

Time Complexity: $O(1)$ per call to every operation, plus one hasher evaluation per `push_back()` / `push_front()`.

Space Complexity:

- $O(n)$ for storage, where n is the current number of elements.
- $O(1)$ auxiliary per operation.

```

#include <cassert>
#include <cstdint>
#include <deque>
#include <utility>

template<class T>
struct identity_digit {
    uint64_t operator()(const T &x) const { return static_cast<uint64_t>(x); }
};

template<class T = int, class Hash = identity_digit<T>>
class HashDeque {
    static const uint64_t MOD1 = 1000000007, MOD2 = 1000000009; // Distinct primes below 2^31.
    static const uint64_t BASE1 = 131, BASE2 = 137; // Should exceed the maximum digit value.

    Hash hasher; // Maps an element to its nonnegative integer digit.
    std::deque<uint64_t> digits; // Per-element digit values, retained so pops never re-hash.
    uint64_t h1 = 0, h2 = 0; // Hash of the current sequence modulo MOD1, MOD2.
    uint64_t top1, top2; // B^(len-1): weight of the front element (B^-1 while empty).
    uint64_t inv1, inv2; // Modular inverses of BASE1, BASE2, for pop_back().

    // Products and sums of residues below 2^30 stay under 2^60, so 64-bit arithmetic never overflows.
    static uint64_t powmod(uint64_t b, uint64_t e, uint64_t m) {
        uint64_t res = 1;
        for (b %= m; e > 0; e >>= 1) {
            if (e & 1) {
                res = res * b % m;
            }
            b = b * b % m;
        }
        return res;
    }

public:
    explicit HashDeque(Hash hasher = Hash()) : hasher(hasher) {
        inv1 = powmod(BASE1, MOD1 - 2, MOD1);
        inv2 = powmod(BASE2, MOD2 - 2, MOD2);
        top1 = inv1; // B^(0-1) = B^-1, so the first push lifts the front weight to B^0 = 1.
        top2 = inv2;
    }

    int size() const { return static_cast<int>(digits.size()); }
    bool empty() const { return digits.empty(); }
    std::pair<uint64_t, uint64_t> hash() const { return {h1, h2}; }

    void push_back(const T &x) {
        uint64_t d = hasher(x), c1 = d % MOD1, c2 = d % MOD2;
        h1 = (h1 * BASE1 + c1) % MOD1;
        h2 = (h2 * BASE2 + c2) % MOD2;
        top1 = top1 * BASE1 % MOD1;
        top2 = top2 * BASE2 % MOD2;
        digits.push_back(d);
    }

    void push_front(const T &x) {
        uint64_t d = hasher(x), c1 = d % MOD1, c2 = d % MOD2;
        top1 = top1 * BASE1 % MOD1; // Now B^len: the weight the new front element receives.
        top2 = top2 * BASE2 % MOD2;
        h1 = (c1 * top1 + h1) % MOD1;
        h2 = (c2 * top2 + h2) % MOD2;
        digits.push_front(d);
    }
};

```

```

void pop_front() {
    assert(!digits.empty());
    uint64_t d = digits.front(), c1 = d % MOD1, c2 = d % MOD2;
    h1 = (h1 + MOD1 - c1 * top1 % MOD1) % MOD1;
    h2 = (h2 + MOD2 - c2 * top2 % MOD2) % MOD2;
    top1 = top1 * inv1 % MOD1;
    top2 = top2 * inv2 % MOD2;
    digits.pop_front();
}

void pop_back() {
    assert(!digits.empty());
    uint64_t d = digits.back(), c1 = d % MOD1, c2 = d % MOD2;
    h1 = (h1 + MOD1 - c1) % MOD1 * inv1 % MOD1;
    h2 = (h2 + MOD2 - c2) % MOD2 * inv2 % MOD2;
    top1 = top1 * inv1 % MOD1;
    top2 = top2 * inv2 % MOD2;
    digits.pop_back();
}

// Compares lengths first, so sequences of different lengths never compare equal.
friend bool operator==(const HashDeque &a, const HashDeque &b) {
    return a.digits.size() == b.digits.size() && a.h1 == b.h1 && a.h2 == b.h2;
}

friend bool operator!=(const HashDeque &a, const HashDeque &b) { return !(a == b); }
};

```

Example Usage

```

#include <functional>
#include <string>
using namespace std;

int main() {
    // Default hasher: characters hash by their code point.
    HashDeque<char> hd, hdrev;
    for (char c : string("abc")) {
        hd.push_back(c);
    }
    for (char c : string("cba")) {
        hdrev.push_front(c); // Prepending c, b, a also yields "abc".
    }
    assert(hd.size() == 3 && hd == hdrev);

    // Slide a width-3 window across the text like Rabin-Karp: push_back + pop_front.
    string text = "Xabcab";
    auto target = hd.hash(); // Hash of "abc".
    HashDeque<char> window;
    for (int i = 0; i < 3; i++) {
        window.push_back(text[i]); // "Xab".
    }
    bool found = (window.hash() == target);
    for (int i = 3; i < static_cast<int>(text.size()); i++) {
        window.push_back(text[i]);
        window.pop_front();
        found = found || (window.hash() == target);
    }
    assert(found); // "abc" occurs in "Xabcab".

    // pop_back undoes a push_back, restoring an earlier state's hash.
    HashDeque<char> d;
    d.push_back('a');
    d.push_back('b');
    auto ab = d.hash();
    d.push_back('c');

```

```

d.pop_back();
assert(d.hash() == ab); // Back to "ab".

// An arbitrary element type via a custom hasher: a deque of whole words.
auto whash = [](const string &w) -> uint64_t { return std::hash<string>{}(w); };
HashDeque<string, decltype(whash)> sentence(whash), reversed(whash);
for (string w : {string("the"), string("quick"), string("fox")}) {
    sentence.push_back(w);
}
for (string w : {string("fox"), string("quick"), string("the")}) {
    reversed.push_front(w); // Same word sequence, assembled from the front.
}
assert(sentence == reversed);

sentence.pop_back(); // Drop "fox", leaving the words "the quick".
HashDeque<string, decltype(whash)> prefix(whash);
prefix.push_back("the");
prefix.push_back("quick");
assert(sentence == prefix);
return 0;
}

```

3.2.4 Dynamic Rolling Hash

Maintain a polynomial rolling hash over a nonnegative integer sequence, supporting point updates and $O(\log n)$ subsequence hash queries. Unlike the static rolling hash, which fixes its prefix hashes at construction, this structure allows any element to change between queries, which is useful for problems that compare substrings while characters are edited.

The key idea is to weight each element by a power of the base tied to its absolute position: storing $a_i B^i$ at index i turns the hash of any prefix into an ordinary prefix sum, which a Fenwick tree maintains under point updates. The raw sum over a half-open range $[l, h)$ is $\sum_i a_i B^i$, and multiplying by B^{-l} shifts it to the position-independent value $\sum_i a_i B^{i-l}$, so two ranges with equal contents receive equal hashes (the leftmost element carries the lowest power). A segment tree could replace the Fenwick tree if lazy range updates are needed.

`MOD1` and `MOD2` are two distinct primes, each kept below 2^{31} so that the product of any two residues fits in a 64-bit integer; hashing modulo both and comparing the resulting pair makes a collision astronomically unlikely. `BASE1` and `BASE2` are the corresponding polynomial bases, which should exceed the largest element value and may be randomized per run to resist adversarial inputs.

Element values must be nonnegative. As with any polynomial hash, comparing subsequences of different lengths is only safe when values are positive (for example, map an alphabet to $[1, B)$) so that trailing zeros cannot make a shorter sequence collide with a longer one.

- `DynamicHash(a)` builds prefix hashes over the integer sequence `a`.
- `size()` returns the length of the sequence.
- `set(i, x)` assigns the value at index `i` to `x`.
- `hash(lo, hi)` returns the pair of hashes of the half-open subsequence `[lo, hi)`. Two ranges are equal with high probability if and only if their `hash()` pairs are equal.

Time Complexity:

- $O(n \log n)$ for the constructor, where n is the length of the sequence.
- $O(\log n)$ per call to `set()` and `hash()`.
- $O(1)$ per call to `size()`.

Space Complexity:

- $O(n)$ for storage.
- $O(1)$ auxiliary per query.


```

#include <cstdint>
#include <utility>
#include <vector>

class DynamicHash {
    static const uint64_t MOD1 = 1000000007, MOD2 = 1000000009; // Distinct primes below 2^31.
    static const uint64_t BASE1 = 131, BASE2 = 137; // Should exceed the maximum element value.

    int len;
    std::vector<int> cur;
    std::vector<uint64_t> pw1, pw2, ip1, ip2; // Base powers and inverse base powers.
    std::vector<uint64_t> ft1, ft2; // Fenwick trees, 1-indexed of size len + 1.

    // Products and sums of residues below 2^30 stay under 2^60, so 64-bit arithmetic never overflows.
    static uint64_t powmod(uint64_t b, uint64_t e, uint64_t m) {
        uint64_t res = 1;
        for (b %= m; e > 0; e >>= 1) {
            if (e & 1) {
                res = res * b % m;
            }
            b = b * b % m;
        }
        return res;
    }

    void add(std::vector<uint64_t> &ft, int i, uint64_t delta, uint64_t m) {
        for (i++; i <= len; i += i & -i) {
            ft[i] = (ft[i] + delta) % m;
        }
    }

    uint64_t sum(const std::vector<uint64_t> &ft, int i, uint64_t m) const {
        uint64_t res = 0;
        for (i++; i > 0; i -= i & -i) {
            res = (res + ft[i]) % m;
        }
        return res;
    }

public:
    explicit DynamicHash(const std::vector<int> &a) : len(static_cast<int>(a.size())), cur(len, 0) {
        pw1.resize(len + 1);
        pw2.resize(len + 1);
        ip1.resize(len + 1);
        ip2.resize(len + 1);
        pw1[0] = pw2[0] = ip1[0] = ip2[0] = 1;
        uint64_t invb1 = powmod(BASE1, MOD1 - 2, MOD1), invb2 = powmod(BASE2, MOD2 - 2, MOD2);
        for (int i = 1; i <= len; i++) {
            pw1[i] = pw1[i - 1] * BASE1 % MOD1;
            pw2[i] = pw2[i - 1] * BASE2 % MOD2;
            ip1[i] = ip1[i - 1] * invb1 % MOD1;
            ip2[i] = ip2[i - 1] * invb2 % MOD2;
        }
        ft1.assign(len + 1, 0);
        ft2.assign(len + 1, 0);
        for (int i = 0; i < len; i++) {
            set(i, a[i]);
        }
    }

    int size() const { return len; }

    void set(int i, int x) {
        uint64_t old1 = static_cast<uint64_t>(cur[i]) % MOD1 * pw1[i] % MOD1;
        uint64_t old2 = static_cast<uint64_t>(cur[i]) % MOD2 * pw2[i] % MOD2;
        uint64_t new1 = static_cast<uint64_t>(x) % MOD1 * pw1[i] % MOD1;
        uint64_t new2 = static_cast<uint64_t>(x) % MOD2 * pw2[i] % MOD2;
        add(ft1, i, (new1 + MOD1 - old1) % MOD1, MOD1);

```

```

    add(ft2, i, (new2 + MOD2 - old2) % MOD2, MOD2);
    cur[i] = x;
}

std::pair<uint64_t, uint64_t> hash(int lo, int hi) const {
    uint64_t raw1 = (sum(ft1, hi - 1, MOD1) + MOD1 - sum(ft1, lo - 1, MOD1)) % MOD1;
    uint64_t raw2 = (sum(ft2, hi - 1, MOD2) + MOD2 - sum(ft2, lo - 1, MOD2)) % MOD2;
    return {raw1 * ip1[lo] % MOD1, raw2 * ip2[lo] % MOD2};
}
};

```

Example Usage

```

#include <cassert>
#include <string>
using namespace std;

int main() {
    string s = "abcabc";
    vector<int> a(s.begin(), s.end());
    DynamicHash h(a);
    assert(h.size() == 6);
    assert(h.hash(0, 3) == h.hash(3, 6)); // "abc" == "abc".

    h.set(0, 'x'); // "xabcabc".
    assert(!(h.hash(0, 3) == h.hash(3, 6)));
    h.set(3, 'x'); // "xabcxabc".
    assert(h.hash(0, 3) == h.hash(3, 6)); // "xabc" == "xabc".

    // A single element's hash is invariant under where an equal value sits.
    assert(h.hash(1, 2) == h.hash(4, 5)); // both 'b'.
    return 0;
}

```

3.2.5 Zobrist Hashing

Computes randomized XOR fingerprints for keys and sets of keys using Zobrist hashing. Each distinct key is assigned a pseudorandom 64-bit token, and a set is represented by the XOR of its members' tokens. This makes insertion and deletion the same operation: XOR the key's token once.

This technique is useful for probabilistic equality checks on sets, distinct-prefix queries, board-game states, and other unordered collections. It is not cryptographic, and collisions are possible with probability roughly $q^2/2^{64}$ over q compared fingerprints. For multisets, plain XOR only records element parity; use a summed hash or pair it with counts if multiplicity matters.

- `ZobristHash<T>(seed)` constructs a token generator with initial value `seed`.
- `get(x)` returns the stable token assigned to key `x`, creating it if needed.
- `toggle(h, x)` returns the hash obtained by inserting `x` into set hash `h` if absent, or removing `x` from `h` if present.
- `distinct_prefix_hashes(lo, hi)` returns prefix hashes where each distinct key in the range `[lo, hi)` contributes only on its first occurrence.

Time Complexity:

- $O(1)$ expected per call to `get(x)` and `toggle(h, x)`.
- $O(n)$ expected per call to `distinct_prefix_hashes(lo, hi)`, where n is the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ for stored key tokens, where n is the number of distinct keys seen so far.
- $O(n)$ auxiliary for `distinct_prefix_hashes(lo, hi)`.

```

#include <cstdint>
#include <unordered_map>
#include <unordered_set>
#include <vector>

template<typename T>
class ZobristHash {
    std::unordered_map<T, uint64_t> token;
    uint64_t state;

    // SplitMix64 mixer.
    static uint64_t mix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

public:
    explicit ZobristHash(uint64_t seed = 0) : state(seed) {}

    uint64_t get(const T &x) {
        if (auto it = token.find(x); it != token.end()) {
            return it->second;
        }
        state = mix64(state);
        token[x] = state;
        return state;
    }

    uint64_t toggle(uint64_t h, const T &x) { return h ^ get(x); }

    template<typename It>
    std::vector<uint64_t> distinct_prefix_hashes(It lo, It hi) {
        std::unordered_set<T> seen;
        std::vector<uint64_t> res(1, 0);
        uint64_t h = 0;
        for (It it = lo; it != hi; ++it) {
            if (seen.insert(*it).second) {
                h ^= get(*it);
            }
            res.push_back(h);
        }
        return res;
    }
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> a{1, 2, 1, 3};
    vector<int> b{2, 1, 3};
    ZobristHash<int> zh(123);
    vector<uint64_t> pa = zh.distinct_prefix_hashes(a.begin(), a.end());
    vector<uint64_t> pb = zh.distinct_prefix_hashes(b.begin(), b.end());

    assert(pa[2] == pb[2]); // Distinct sets {1, 2} and {2, 1}.
    assert(pa[4] == pb[3]); // Distinct sets {1, 2, 3} and {2, 1, 3}.

    uint64_t h = 0;
    h = zh.toggle(h, 5);
    assert(h != 0);
    h = zh.toggle(h, 5);
}

```

```

    assert(h == 0);
    return 0;
}

```

3.3 Pattern Matching

3.3.1 String Searching (KMP)

Given a single string (needle) and subsequent queries of texts (haystacks) to be searched, determine the first positions in which the needle occurs within the given haystacks in linear time using the Knuth-Morris-Pratt algorithm. In comparison, `std::string::find` runs in $O(n^2)$ in the worst case. KMP precomputes, for every prefix of the needle, the length of its longest proper border (a prefix that is also a suffix). On a mismatch, the needle falls back to that border instead of restarting, so the position in the haystack never moves backward.

- `KMP(needle)` constructs the partial match table for a string `needle` that is to be searched for subsequently in `haystack` queries.
- `find_in(haystack)` returns the first position that `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found. Note that the function can be modified to return all matches by simply letting the loop run and storing the results instead of returning early.

Time Complexity:

- $O(m)$ per call to the constructor, where m is the length of `needle`.
- $O(n)$ per call to `find_in(haystack)`, where n is the length of `haystack`.

Space Complexity:

- $O(m)$ for storage of the partial match table, where m is the length of `needle`.
- $O(1)$ auxiliary space per call to `find_in(haystack)`.

```

#include <string>
#include <vector>
using std::string;

class KMP {
    string needle;
    std::vector<int> table;

public:
    explicit KMP(const string &needle) : needle(needle), table(needle.size()) {
        for (int i = 1, j = 0; i < static_cast<int>(needle.size()); i++) {
            while (j > 0 && needle[i] != needle[j]) {
                j = table[j - 1];
            }
            if (needle[i] == needle[j]) {
                j++;
            }
            table[i] = j;
        }
    }

    std::size_t find_in(const string &haystack) {
        int m = static_cast<int>(needle.size());
        if (m == 0) {
            return 0;
        }
        for (int i = 0, j = 0; i < static_cast<int>(haystack.size()); i++) {
            while (j > 0 && needle[j] != haystack[i]) {
                j = table[j - 1];
            }
            if (needle[j] == haystack[i]) {

```

```

        j++;
    }
    if (j == m) {
        return i + 1 - m;
    }
}
return string::npos;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    assert(15 == KMP("ABCDABD").find_in("ABC ABCDAB ABCDABCDABDE"));
    return 0;
}

```

3.3.2 String Searching (Z Algorithm)

Given a single string (needle) and a single text (haystack) to be searched, determine the first position in which the needle occurs within the haystack in linear time using the Z algorithm. In comparison, `std::string::find` runs in quadratic time.

The Z array stores, for each position of a string, the length of the longest substring starting there that matches a prefix of the whole string. It is computed in linear time by maintaining the rightmost matched window and seeding each position from its mirror earlier in that window.

The `find()` function below calls the Z algorithm on the concatenation of `needle` and `haystack`, separated by a sentinel value that is guaranteed not to collide with any byte in either input. Any position whose Z value reaches the needle's length marks an occurrence.

- `z_array(s)` constructs the Z array for a needle string `s` that can be used for string searching. Each resulting value `z[i]` stores the length of the longest substring starting from `s[i]` which is also a prefix of the needle `s`.
- `find(haystack, needle)` returns the first position that `needle` occurs in `haystack`, or `std::string::npos` if it cannot be found. Note that the function can be modified to return all matches by simply letting the loop run and storing the results instead of returning early.

Time Complexity:

- $O(n)$ per call to `z_array(s)`, where n is the length of `s`.
- $O(n + m)$ per call to `find(haystack, needle)`, where n is the length of `haystack` and m is the length of `needle`.

Space Complexity:

- $O(n)$ auxiliary heap space for `z_array(s)`, where n is the length of `s`.
- $O(n + m)$ auxiliary heap space for `find(haystack, needle)` where n is the length of `haystack` and m is the length of `needle`.

```

#include <algorithm>
#include <string>
#include <vector>
using std::string;

template<typename Seq>
std::vector<int> z_array(const Seq &s) {
    std::vector<int> z(s.size());
    for (int i = 1, l = 0, r = 0; i < static_cast<int>(z.size()); i++) {

```

```

    if (i <= r) {
        z[i] = std::min(r - i + 1, z[i - 1]);
    }
    while (i + z[i] < static_cast<int>(z.size()) && s[z[i]] == s[i + z[i]]) {
        z[i]++;
    }
    if (r < i + z[i] - 1) {
        l = i;
        r = i + z[i] - 1;
    }
}
return z;
}

std::size_t find(const string &haystack, const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    std::vector<int> s;
    s.reserve(needle.size() + haystack.size() + 1);
    for (unsigned char c : needle) {
        s.push_back(c + 1);
    }
    s.push_back(0);
    for (unsigned char c : haystack) {
        s.push_back(c + 1);
    }
    auto z = z_array(s);
    int m = static_cast<int>(needle.size());
    for (int i = m + 1; i < static_cast<int>(z.size()); i++) {
        if (z[i] == m) {
            return i - m - 1;
        }
    }
    return string::npos;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(15 == find("ABC ABCDAB ABCDABCDABDE", "ABCDABD"));
    return 0;
}

```

3.3.3 String Searching (Aho-Corasick)

Given a set of strings (needles) and subsequent queries of texts (haystacks) to be searched, determine all positions in which needles occur within the given haystacks in linear time using the Aho-Corasick algorithm. The needles are arranged in a trie whose nodes carry failure links to the longest proper suffix that is also a trie node. Scanning the haystack once while following transitions and failure links then reports every match through each node's output set.

This implementation stores transitions in hash tables, which keeps lookups expected constant time without assuming a fixed alphabet size. The output tables are stored as ordered sets so matches ending at the same position are reported in deterministic needle order.

- `AhoCorasick(needles)` constructs the finite-state automaton for a set of needle strings that are to be searched for subsequently in haystack queries. Empty needles are ignored.

- `find_all_in(haystack, report_match)` calls the function `report_match(s, pos)` once on each occurrence of each needle that occurs in the `haystack`, where `pos` is the starting position in the `haystack` at which string `s` (a matched needle) occurs. The matches will be reported in increasing order of their ending positions within the `haystack`.

Time Complexity:

- $O(m \cdot l + z \log m)$ expected per call to the constructor, where m is the number of needles, l is the maximum length of any needle, and z is the total number of inherited output entries.
- $O(n + z)$ expected per call to `find_all_in(haystack, report_match)`, where n is the length of `haystack` and z is the number of matches.

Space Complexity:

- $O(m \cdot l)$ for storage of the automaton, where m is the number of needles and l is the maximum length for any needle.
- $O(1)$ auxiliary space per call to `find_all_in(haystack, report_match)`.

```
#include <queue>
#include <set>
#include <string>
#include <unordered_map>
#include <vector>
using std::string;

class AhoCorasick {
    std::vector<string> needles;
    std::vector<int> fail;
    std::vector<std::unordered_map<char, int>> adj;
    std::vector<std::set<int>> out;

    int next_state(int curr, char c) {
        int next = curr;
        while (next != 0 && adj[next].find(c) == adj[next].end()) {
            next = fail[next];
        }
        auto it = adj[next].find(c);
        return (it != adj[next].end()) ? it->second : 0;
    }

public:
    explicit AhoCorasick(const std::vector<string> &needles) : needles(needles) {
        int total_len = 0;
        for (const auto &needle : needles) {
            total_len += static_cast<int>(needle.size());
        }
        // The trie has at most total_len + 1 states: the root plus one per character when no prefixes
        // are shared. Sizing to total_len alone overflows for low-sharing needle sets (e.g. a single
        // needle, which still needs the root plus one node).
        int max_states = total_len + 1;
        fail.resize(max_states, -1);
        adj.resize(max_states);
        out.resize(max_states);
        int states = 1;
        for (int i = 0; i < static_cast<int>(needles.size()); i++) {
            if (needles[i].empty()) {
                continue;
            }
            int curr = 0;
            for (char c : needles[i]) {
                if (auto it = adj[curr].find(c); it != adj[curr].end()) {
                    curr = it->second;
                } else {
                    curr = adj[curr][c] = states++;
                }
            }
        }
    }
};
```

```

    out[curr].insert(i);
}
std::queue<int> q;
for (auto &[c, v] : adj[0]) {
    if (v != 0) {
        fail[v] = 0;
        q.push(v);
    }
}
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (auto &[c, v] : adj[u]) {
        int f = fail[u];
        while (f != 0 && adj[f].find(c) == adj[f].end()) {
            f = fail[f];
        }
        auto fit = adj[f].find(c);
        f = (fit != adj[f].end()) ? fit->second : 0;
        fail[v] = f;
        out[v].insert(out[f].begin(), out[f].end());
        q.push(v);
    }
}
}

template<typename Fn>
void find_all_in(const string &haystack, Fn report_match) {
    int state = 0;
    for (int i = 0; i < static_cast<int>(haystack.size()); i++) {
        state = next_state(state, haystack[i]);
        for (int idx : out[state]) {
            report_match(needles[idx], i - static_cast<int>(needles[idx].size()) + 1);
        }
    }
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    vector<string> needles;
    needles.push_back("a");
    needles.push_back("ab");
    needles.push_back("bab");
    needles.push_back("bc");
    needles.push_back("bca");
    needles.push_back("c");
    needles.push_back("caa");
    needles.push_back("abccab");

    AhoCorasick(needles).find_all_in("abccab", [](const string &needle, int pos) {
        cout << "Matched \"" << needle << "\" at position " << pos << "." << endl;
    });
    return 0;
}

```

Example Output

```

Matched "a" at position 0.
Matched "ab" at position 0.
Matched "bc" at position 1.

```



```

Matched "c" at position 2.
Matched "c" at position 3.
Matched "a" at position 4.
Matched "ab" at position 4.
Matched "abccab" at position 0.

```

3.3.4 Palindrome Searching (Manacher)

Computes all odd- and even-length palindromic radii of a string in linear time using Manacher's algorithm. These radii can be used to test whether any substring is a palindrome in $O(1)$, find the longest palindromic substring, or count all palindromic substrings. The algorithm tracks the rightmost known palindrome and seeds each new center with the radius of its mirror inside that window, so characters are only ever compared when a palindrome is extended past the window.

For odd palindromes, `odd[i]` is the radius centered at character `s[i]`, including the center character. Thus the palindrome spans `[i - odd[i] + 1, i + odd[i])`. For even palindromes, `even[i]` is the radius centered between `s[i - 1]` and `s[i]`. Thus the palindrome spans `[i - even[i], i + even[i])`.

- `Manacher(s)` constructs the palindromic radii arrays for string `s`.
- `is_palindrome(lo, hi)` returns whether substring `[lo, hi)` is a palindrome.
- `longest_palindrome()` returns one longest palindromic substring of `s`.
- `count_palindromes()` returns the total number of palindromic substrings of `s`, counted with multiplicity by position.

Time Complexity:

- $O(n)$ per constructor call, where n is the length of `s`.
- $O(1)$ per call to `is_palindrome(lo, r)`.
- $O(n)$ per call to `longest_palindrome()` and `count_palindromes()`.

Space Complexity:

- $O(n)$ for storage of the radii arrays.
- $O(1)$ auxiliary per query.

```

#include <algorithm>
#include <cstdint>
#include <string>
#include <vector>
using std::string;

class Manacher {
    string s;
    std::vector<int> odd, even;

public:
    explicit Manacher(const string &s) : s(s), odd(s.size()), even(s.size()) {
        int n = static_cast<int>(s.size());
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? 1 : std::min(odd[l + r - i], r - i + 1);
            while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
                k++;
            }
            odd[i] = k;
            if (i + k - 1 > r) {
                l = i - k + 1;
                r = i + k - 1;
            }
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = (i > r) ? 0 : std::min(even[l + r - i + 1], r - i + 1);
            while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
                k++;
            }
        }
    }
};

```

```

        even[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

bool is_palindrome(int lo, int hi) const {
    int len = hi - lo;
    if (len <= 0) {
        return true;
    }
    int mid = lo + (hi - lo) / 2;
    return (len % 2) ? odd[mid] >= len / 2 + 1 : even[mid] >= len / 2;
}

string longest_palindrome() const {
    int best_l = 0, best_len = 0;
    int n = static_cast<int>(s.size());
    for (int i = 0; i < n; i++) {
        int len = 2 * odd[i] - 1;
        if (len > best_len) {
            best_len = len;
            best_l = i - odd[i] + 1;
        }
        len = 2 * even[i];
        if (len > best_len) {
            best_len = len;
            best_l = i - even[i];
        }
    }
    return s.substr(best_l, best_len);
}

int64_t count_palindromes() const {
    int64_t res = 0;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        res += odd[i] + even[i];
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    Manacher m("abacaba");
    assert(m.is_palindrome(0, 7));
    assert(m.is_palindrome(1, 6));
    assert(!m.is_palindrome(0, 6));
    assert(m.longest_palindrome() == "abacaba");
    assert(m.count_palindromes() == 12);

    Manacher e("cabbad");
    assert(e.is_palindrome(1, 5));
    assert(e.longest_palindrome() == "abba");
    return 0;
}

```

3.3.5 Lyndon Factorization (Duval)

A Lyndon word is a nonempty string that is strictly smaller than all of its proper suffixes, or equivalently, strictly smaller than all of its nontrivial cyclic rotations. By the Chen-Fox-Lyndon theorem, every string has a unique factorization into a sequence of Lyndon words whose values are non-increasing: $s = w_1 w_2 \cdots w_k$ with $w_1 \geq w_2 \geq \cdots \geq w_k$. Duval's algorithm computes this factorization in linear time and constant auxiliary space by scanning the string while tracking the current candidate Lyndon prefix and the position it last matched.

The same scan, applied to the string viewed as a cycle, yields the starting position of its lexicographically least rotation. This is the standard way to canonicalize a necklace, for example to test whether two strings are cyclic rotations of one another.

Both functions below operate on any random-access sequence whose elements support `operator<`.

- `lyndon_factorization(s)` returns a vector of the starting indices of the Lyndon factors of `s`, in order. The factor at index `i` of the result spans the half-open range from `res[i]` to `res[i + 1]` (or to `s.size()` for the final factor).
- `min_rotation(s)` returns the 0-indexed position `p` such that the rotation beginning at `p`, that is, `s[p]`, `s[(p + 1) % n]`, ..., `s[(p + n - 1) % n]`, is lexicographically smallest. If several rotations tie, the smallest such `p` is returned.

Time Complexity: $O(n)$ per call to `lyndon_factorization(s)` and `min_rotation(s)`, where n is the length of `s`.

Space Complexity:

- $O(k)$ auxiliary heap space for `lyndon_factorization(s)`, where k is the number of factors.
- $O(1)$ auxiliary space for `min_rotation(s)`.

```
#include <vector>

template<typename Seq>
std::vector<int> lyndon_factorization(const Seq &s) {
    int n = static_cast<int>(s.size());
    std::vector<int> starts;
    int i = 0;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && !(s[j] < s[k])) {
            k = (s[k] < s[j]) ? i : k + 1;
            j++;
        }
        while (i <= k) {
            starts.push_back(i);
            i += j - k;
        }
    }
    return starts;
}

template<typename Seq>
int min_rotation(const Seq &s) {
    int n = static_cast<int>(s.size());
    if (n == 0) {
        return 0;
    }
    int i = 0, res = 0;
    while (i < n) {
        res = i;
        int j = i + 1, k = i;
        while (j < 2 * n && !(s[j % n] < s[k % n])) {
            k = (s[k % n] < s[j % n]) ? i : k + 1;
            j++;
        }
        while (i <= k) {
            i += j - k;
        }
    }
}
```

```

    }
}
return res;
}

```

Example Usage

```

#include <cassert>
#include <string>
using std::string;

int main() {
    // "banana" factors as "b" >= "an" >= "an" >= "a".
    assert((lyndon_factorization(string("banana")) == std::vector<int>{0, 1, 3, 5}));

    // Among the rotations of "baca", "abac" (starting at index 3) is smallest.
    assert(min_rotation(string("baca")) == 3);

    // Both routines work on any comparable sequence, not just strings.
    assert(min_rotation(std::vector<int>{3, 1, 2, 1, 3}) == 1);
    return 0;
}

```

3.3.6 De Bruijn Sequence

A De Bruijn sequence $B(k, n)$ is a cyclic string over the alphabet $[0, k)$ of length k^n in which every one of the k^n possible length- n strings appears exactly once as a (cyclic) substring. Reading a window of n symbols and advancing one position at a time therefore cycles through all length- n strings without repetition, which makes the sequence a compact way to probe every short pattern: cracking a rotating lock, addressing a shift register, or building the perfect-hash table that maps an isolated set bit to its index.

The construction here is the Fredricksen-Kessler-Maiorana (FKM) algorithm. It concatenates, in lexicographic order, the Lyndon words (see the Lyndon factorization section) whose length divides n . The result is the lexicographically smallest De Bruijn sequence. The recursion enumerates these necklace representatives directly, appending the first p symbols of each candidate whenever its period p divides n .

- `de_bruijn(k, n)` returns a vector of length k^n holding one period of $B(k, n)$. Each entry lies in $[0, k)$. Both k and n must be at least 1. Treat the result as cyclic: the length- n windows that wrap past the end are exactly the ones still needed to cover all strings.

Time Complexity: $O(k^n)$ per call to `de_bruijn(k, n)`, which is optimal since the output itself has length k^n .

Space Complexity: $O(k^n)$ heap space for the returned sequence, plus $O(n)$ auxiliary space for the recursion.

```

#include <vector>

void de_bruijn_gen(int t, int p, int k, int n, std::vector<int> &word, std::vector<int> &seq) {
    if (t > n) {
        if (n % p == 0) {
            for (int i = 1; i <= p; i++) {
                seq.push_back(word[i]);
            }
        }
        return;
    }
    word[t] = word[t - p];
    de_bruijn_gen(t + 1, p, k, n, word, seq);
    for (int j = word[t - p] + 1; j < k; j++) {
        word[t] = j;
        de_bruijn_gen(t + 1, t, k, n, word, seq);
    }
}

```

```

}

std::vector<int> de_bruijn(int k, int n) {
    std::vector<int> word(n + 1, 0), seq;
    de_bruijn_gen(1, 1, k, n, word, seq);
    return seq;
}

```

Example Usage

```

#include <cassert>
#include <cmath>
#include <set>
#include <vector>
using namespace std;

int main() {
    // B(2, 3) has length 8 and every 3-bit string appears once cyclically.
    vector<int> seq = de_bruijn(2, 3);
    assert(seq.size() == 8);

    for (int k = 2; k <= 4; k++) {
        for (int n = 1; n <= 4; n++) {
            vector<int> s = de_bruijn(k, n);
            int len = static_cast<int>(std::lround(std::pow(k, n)));
            assert(static_cast<int>(s.size()) == len);

            // Collect every length-n window, wrapping cyclically, and confirm all k^n are distinct.
            set<vector<int>> windows;
            for (int i = 0; i < len; i++) {
                vector<int> w(n);
                for (int j = 0; j < n; j++) {
                    w[j] = s[(i + j) % len];
                }
                windows.insert(w);
            }
            assert(static_cast<int>(windows.size()) == len);
        }
    }
    return 0;
}

```

3.4 Suffix Arrays and LCP

3.4.1 Suffix Array and LCP (Manber-Myers)

Given a string s , a suffix array is the array of the smallest starting positions for the sorted suffixes of s . That is, the i -th position of the suffix array stores the starting position of the i -th lexicographically smallest suffix of s . For example, $s = \text{"cab"}$ has the suffixes "cab" , "ab" , and "b" . When sorted, the indices of the suffixes are "ab" , "b" , and "cab" , so the suffix array (assuming 0-based indices) is $[1, 2, 0]$.

For a string s of length n , the longest common prefix (LCP) array of length $n - 1$ stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in s . For example, "baa" has the sorted suffixes "a" , "aa" , and "baa" , with an LCP array of $[1, 0]$.

The original Manber-Myers doubling algorithm comparison-sorts the suffixes by their first 2^k characters for increasing k : each round orders suffixes by their pair of ranks from the previous round, so a comparison costs $O(1)$ and the full order emerges after $O(\log n)$ rounds.

- `SuffixArrayManberMyers(s)` constructs a suffix array from string `s`.

- `get_sa()` returns the constructed suffix array.
- `get_lcp()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length m , this implementation uses an $O(m \log n)$ binary search, but can be optimized to $O(m + \log n)$ by first computing the LCP-LR array using the LCP array.

Time Complexity:

- $O(n \log^2 n)$ per call to the constructor, where n is the length of `s`.
- $O(1)$ per call to `get_sa()`.
- $O(n)$ per call to `get_lcp()`, where n is the length of `s`.
- $O(m \log n)$ per call to `find(needle)`, where m is the length of `needle` and n is the length of `s`.

Space Complexity:

- $O(n)$ auxiliary for storage of the suffix and LCP arrays.
- $O(n)$ auxiliary heap space for the constructor.
- $O(1)$ auxiliary space for all other operations.

```
#include <algorithm>
#include <numeric>
#include <string>
#include <utility>
#include <vector>
using std::string;

class SuffixArrayManberMyers {
    string s;
    std::vector<int> sa, rk;

public:
    explicit SuffixArrayManberMyers(const string &s) : s(s), sa(s.size()), rk(s.size()) {
        int n = static_cast<int>(s.size());
        std::iota(sa.begin(), sa.end(), 0);
        for (int i = 0; i < n; i++) {
            rk[i] = static_cast<unsigned char>(s[i]);
        }
        std::vector<std::pair<int, int>> rk2(n);
        for (int gap = 1; gap < n; gap *= 2) {
            for (int i = 0; i < n; i++) {
                rk2[i] = {rk[i], i + gap < n ? rk[i + gap] + 1 : 0};
            }
            std::sort(sa.begin(), sa.end(), [&](int i, int j) { return rk2[i] < rk2[j]; });
            for (int i = 0; i < n; i++) {
                rk[sa[i]] = (i > 0 && rk2[sa[i - 1]] == rk2[sa[i]]) ? rk[sa[i - 1]] : i;
            }
        }
    }

    const std::vector<int> &get_sa() const { return sa; }

    std::vector<int> get_lcp() const {
        int n = static_cast<int>(s.size());
        if (n == 0) {
            return {}; // Avoid constructing a vector of size (size_t)(-1).
        }
        std::vector<int> lcp(n - 1);
        for (int i = 0, k = 0; i < n; i++) {
            if (rk[i] < n - 1) {
                int j = sa[rk[i] + 1];
                while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
                    k++;
                }
                lcp[rk[i]] = k;
                if (k > 0) {
                    k--;
                }
            }
        }
    }
};
```

```

    }
}
}
return lcp;
}

std::size_t find(const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    int lo = 0, hi = static_cast<int>(s.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SuffixArrayManberMyers sa("banana");
    vector<int> sarr = sa.get_sa(), lcp = sa.get_lcp();
    vector<int> sarr_expected{5, 3, 1, 0, 4, 2};
    vector<int> lcp_expected{1, 3, 0, 0, 2};
    assert(sarr == sarr_expected);
    assert(lcp == lcp_expected);
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);
    return 0;
}

```

3.4.2 Suffix Array and LCP (Counting Sort)

Given a string s , a suffix array is the array of the smallest starting positions for the sorted suffixes of s . That is, the i -th position of the suffix array stores the starting position of the i -th lexicographically smallest suffix of s . For example, $s = \text{"cab"}$ has the suffixes "cab" , "ab" , and "b" . When sorted, the indices of the suffixes are "ab" , "b" , and "cab" , so the suffix array (assuming 0-based indices) is $[1, 2, 0]$.

For a string s of length n , the longest common prefix (LCP) array of length $n - 1$ stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in s . For example, "baa" has the sorted suffixes "a" , "aa" , and "baa" , with an LCP array of $[1, 0]$.

The original Manber-Myers doubling algorithm comparison-sorts the suffixes by their first 2^k characters for increasing k : each round orders suffixes by their pair of ranks from the previous round, so a comparison costs $O(1)$ and the full order emerges after $O(\log n)$ rounds. Replacing the comparison sort of each round with counting-sort-style rank updates removes a logarithmic factor.

- `SuffixArrayCountingSort(s)` constructs a suffix array from string `s`.
- `get_sa()` returns the constructed suffix array.

- `get_lcp()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length m , this implementation uses an $O(m \log n)$ binary search, but can be optimized to $O(m + \log n)$ by first computing the LCP-LR array using the LCP array.

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the length of `s`.
- $O(1)$ per call to `get_sa()`.
- $O(n)$ per call to `get_lcp()`, where n is the length of `s`.
- $O(m \log n)$ per call to `find(needle)`, where m is the length of `needle` and n is the length of `s`.

Space Complexity:

- $O(n)$ auxiliary for storage of the suffix and LCP arrays.
- $O(n)$ auxiliary heap space for the constructor.
- $O(1)$ auxiliary space for all other operations.

```
#include <algorithm>
#include <numeric>
#include <string>
#include <utility>
#include <vector>
using std::string;

class SuffixArrayCountingSort {
    string s;
    std::vector<int> sa, rk;

public:
    explicit SuffixArrayCountingSort(const string &s) : s(s), sa(s.size()), rk(s.size()) {
        int n = static_cast<int>(s.size());
        std::iota(sa.rbegin(), sa.rend(), 0);
        for (int i = 0; i < n; i++) {
            rk[i] = static_cast<unsigned char>(s[i]);
        }
        std::stable_sort(sa.begin(), sa.end(), [&](int i, int j) {
            return static_cast<unsigned char>(s[i]) < static_cast<unsigned char>(s[j]);
        });
        for (int gap = 1; gap < n; gap *= 2) {
            std::vector<int> prev_rk(rk), prev_sa(sa), cnt(n);
            std::iota(cnt.begin(), cnt.end(), 0);
            for (int i = 0; i < n; i++) {
                rk[sa[i]] = (i > 0 && prev_rk[sa[i - 1]] == prev_rk[sa[i]] && sa[i - 1] + gap < n &&
                    prev_rk[sa[i - 1] + gap / 2] == prev_rk[sa[i] + gap / 2])
                    ? rk[sa[i - 1]]
                    : i;
            }
            for (int i = 0; i < n; i++) {
                int s1 = prev_sa[i] - gap;
                if (s1 >= 0) {
                    sa[cnt[rk[s1]]++] = s1;
                }
            }
        }
    }

    const std::vector<int> &get_sa() const { return sa; }

    std::vector<int> get_lcp() const {
        int n = static_cast<int>(s.size());
        if (n == 0) {
            return {}; // Avoid constructing a vector of size (size_t)(-1).
        }
        std::vector<int> lcp(n - 1);
        for (int i = 0, k = 0; i < n; i++) {
```



```

    if (rk[i] < n - 1) {
        int j = sa[rk[i] + 1];
        while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
            k++;
        }
        lcp[rk[i]] = k;
        if (k > 0) {
            k--;
        }
    }
}
return lcp;
}

std::size_t find(const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    int lo = 0, hi = static_cast<int>(s.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SuffixArrayCountingSort sa("banana");
    vector<int> sarr = sa.get_sa(), lcp = sa.get_lcp();
    vector<int> sarr_expected{5, 3, 1, 0, 4, 2};
    vector<int> lcp_expected{1, 3, 0, 0, 2};
    assert(sarr == sarr_expected);
    assert(lcp == lcp_expected);
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);
    return 0;
}

```

3.4.3 Suffix Array and LCP (DC3)

Given a string s , a suffix array is the array of the smallest starting positions for the sorted suffixes of s . That is, the i -th position of the suffix array stores the starting position of the i -th lexicographically smallest suffix of s . For example, $s = \text{"cab"}$ has the suffixes "cab" , "ab" , and "b" . When sorted, the indices of the suffixes are "ab" , "b" , and "cab" , so the suffix array (assuming 0-based indices) is $[1, 2, 0]$.

For a string s of length n , the longest common prefix (LCP) array of length $n - 1$ stores the lengths of the longest common prefixes between all pairs of lexicographically adjacent suffixes in s . For example, "baa" has the sorted suffixes "a" , "aa" , and "baa" , with an LCP array of $[1, 0]$.

The DC3/skew algorithm recursively constructs the suffix array of the suffixes starting at positions not divisible by 3, uses that order to sort the remaining suffixes with radix sort, and merges the two sorted groups in linear time.

- `SuffixArrayDC3(s)` constructs a suffix array from string `s`.
- `get_sa()` returns the constructed suffix array.
- `get_lcp()` returns the corresponding LCP array for the suffix array.
- `find(needle)` returns one position that `needle` occurs in `s` (not necessarily the first), or `std::string::npos` if it cannot be found. For a `needle` of length m , this implementation uses an $O(m \log n)$ binary search, but can be optimized to $O(m + \log n)$ by first computing the LCP-LR array using the LCP array.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the length of `s`.
- $O(1)$ per call to `get_sa()`.
- $O(n)$ per call to `get_lcp()`, where n is the length of `s`.
- $O(m \log n)$ per call to `find(needle)`, where m is the length of `needle` and n is the length of `s`.

Space Complexity:

- $O(n)$ auxiliary for storage of the suffix and LCP arrays.
- $O(n)$ auxiliary heap space for the constructor.
- $O(1)$ auxiliary space for all other operations.

```
#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using std::string;

class SuffixArrayDC3 {
    static bool leq(int a1, int a2, int b1, int b2) { return (a1 < b1) || (a1 == b1 && a2 <= b2); }

    static bool leq(int a1, int a2, int a3, int b1, int b2, int b3) {
        return (a1 < b1) || (a1 == b1 && leq(a2, a3, b2, b3));
    }
}

template<typename It>
static void radix_pass(It a, It b, It r, int n, int K) {
    std::vector<int> cnt(K + 1);
    for (int i = 0; i < n; i++) {
        cnt[r[a[i]]]++;
    }
    for (int i = 1; i <= K; i++) {
        cnt[i] += cnt[i - 1];
    }
    for (int i = n - 1; i >= 0; i--) {
        b[--cnt[r[a[i]]]] = a[i];
    }
}

template<typename It>
static void suffix_array_dc3(It s, It sa, int n, int K) {
    int n0 = (n + 2) / 3, n1 = (n + 1) / 3, n2 = n / 3, n02 = n0 + n2;
    std::vector<int> s12(n02 + 3), sa12(n02 + 3), s0(n0), sa0(n0);
    s12[n02] = s12[n02 + 1] = s12[n02 + 2] = 0;
    sa12[n02] = sa12[n02 + 1] = sa12[n02 + 2] = 0;
    for (int i = 0, j = 0; i < n + n0 - n1; i++) {
        if (i % 3 != 0) {
            s12[j++] = i;
        }
    }
    radix_pass(s12.begin(), sa12.begin(), s + 2, n02, K);
    radix_pass(sa12.begin(), s12.begin(), s + 1, n02, K);
    radix_pass(s12.begin(), sa12.begin(), s, n02, K);
    int name = 0, c0 = -1, c1 = -1, c2 = -1;
    for (int i = 0; i < n02; i++) {
```

```

    if (s[sa12[i]] != c0 || s[sa12[i] + 1] != c1 || s[sa12[i] + 2] != c2) {
        name++;
        c0 = s[sa12[i]];
        c1 = s[sa12[i] + 1];
        c2 = s[sa12[i] + 2];
    }
    (sa12[i] % 3 == 1 ? s12[sa12[i] / 3] : s12[sa12[i] / 3 + n0]) = name;
}
if (name < n02) {
    suffix_array_dc3(s12.begin(), sa12.begin(), n02, name);
    for (int i = 0; i < n02; i++) {
        s12[sa12[i]] = i + 1;
    }
} else {
    for (int i = 0; i < n02; i++) {
        sa12[s12[i] - 1] = i;
    }
}
for (int i = 0, j = 0; i < n02; i++) {
    if (sa12[i] < n0) {
        s0[j++] = 3 * sa12[i];
    }
}
radix_pass(s0.begin(), sa0.begin(), s, n0, K);
for (int p = 0, t = n0 - n1, k = 0; k < n; k++) {
    int i = (sa12[t] < n0) ? 3 * sa12[t] + 1 : 3 * (sa12[t] - n0) + 2, j = sa0[p];
    if (sa12[t] < n0
        ? leq(s[i], s12[sa12[t] + n0], s[j], s12[j / 3])
        : leq(s[i], s[i + 1], s12[sa12[t] - n0 + 1], s[j], s[j + 1], s12[j / 3 + n0])) {
        sa[k] = i;
        if (++t == n02) {
            for (k++; p < n0; p++, k++) {
                sa[k] = sa0[p];
            }
        }
    } else {
        sa[k] = j;
        if (++p == n0) {
            for (k++; t < n02; t++, k++) {
                sa[k] = (sa12[t] < n0) ? 3 * sa12[t] + 1 : 3 * (sa12[t] - n0) + 2;
            }
        }
    }
}
}
}

string s;
std::vector<int> sa;

public:
explicit SuffixArrayDC3(const string &s) : s(s), sa(s.size() + 1) {
    int n = static_cast<int>(s.size());
    std::vector<int> scopy(n);
    for (int i = 0; i < n; i++) {
        scopy[i] = static_cast<unsigned char>(s[i]) + 1;
    }
    scopy.resize(n + 4);
    suffix_array_dc3(scopy.begin(), sa.begin(), n + 1, 256);
    sa.erase(sa.begin());
}

const std::vector<int> &get_sa() const { return sa; }

std::vector<int> get_lcp() const {
    int n = static_cast<int>(s.size());
    if (n == 0) {
        return {};
    }
}

```

```

std::vector<int> rank(n), lcp(n - 1);
for (int i = 0; i < n; i++) {
    rank[sa[i]] = i;
}
for (int i = 0, k = 0; i < n; i++) {
    if (rank[i] < n - 1) {
        int j = sa[rank[i] + 1];
        while (std::max(i, j) + k < n && s[i + k] == s[j + k]) {
            k++;
        }
        lcp[rank[i]] = k;
        if (k > 0) {
            k--;
        }
    }
}
return lcp;
}

std::size_t find(const string &needle) {
    if (needle.empty()) {
        return 0;
    }
    int lo = 0, hi = static_cast<int>(s.size()) - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = s.compare(sa[mid], needle.size(), needle);
        if (cmp < 0) {
            lo = mid + 1;
        } else if (cmp > 0) {
            hi = mid - 1;
        } else {
            return sa[mid];
        }
    }
    return string::npos;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SuffixArrayDC3 sa("banana");
    vector<int> sarr = sa.get_sa(), lcp = sa.get_lcp();
    vector<int> sarr_expected{5, 3, 1, 0, 4, 2};
    vector<int> lcp_expected{1, 3, 0, 0, 2};
    assert(sarr == sarr_expected);
    assert(lcp == lcp_expected);
    assert(sa.find("ana") == 1);
    assert(sa.find("x") == string::npos);
    return 0;
}

```

3.5 String Data Structures

3.5.1 Trie

Maintain a map of strings to values using a tree data structure. Each node corresponds to a character, and each inserted string corresponds to a path from the root to a node that is flagged as a terminal node. Children are stored in hash tables, and `walk()` sorts child labels as needed to preserve lexicographic traversal.

- `Trie<V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(s, v)` adds an entry with string key `s` and value `v` to the map, returning `true` if a new entry was added or `false` if the string already exists (in which case the map is unchanged and the old value associated with the string key is preserved).
- `erase(s)` removes the entry with string key `s` from the map, returning `true` if the removal was successful or `false` if the string to be removed was not found.
- `find(s)` returns a pointer to a const value associated with string key `s`, or `nullptr` if the key was not found.
- `count_prefix(s)` returns the number of keys currently in the map that have `s` as a prefix (a key equal to `s` counts as well). `count_prefix("")` therefore equals `size()`.
- `walk(f)` calls the function `f(s, v)` on each entry of the map, in lexicographically ascending order of the string keys.

For a small, fixed alphabet, the `std::unordered_map` children can be replaced by a fixed-size array of child pointers indexed by character. This removes the per-step hashing and lets `walk()` traverse in sorted order without sorting each node's labels, at the cost of restricting the alphabet and spending more memory per node.

Time Complexity:

- $O(n)$ expected per call to `insert(s, v)`, `erase(s)`, `find(s)`, and `count_prefix(s)`, where n is the length of `s`.
- $O(l)$ per call to `walk()`, where l is the total length of string keys that are currently in the map, treating the character alphabet as constant.
- $O(1)$ per call to all other operations.

Space Complexity:

- $O(l)$ for storage of the Trie, where l is the total length of string keys that are currently in the map.
- $O(n)$ auxiliary stack space for construction, destruction, `walk()`, where n is the maximum length of any string that has been inserted so far.
- $O(n)$ auxiliary stack space for `erase(s)`, where n is the length of `s`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstddef>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

template<typename V>
class Trie {
    struct Node {
        V value;
        bool is_terminal;
        int cnt; // Optional: maintain count of terminal keys in this subtree for count_prefix queries.
        std::unordered_map<char, Node *> children;

        Node() : is_terminal(false), cnt(0) {}
    } *root;

    static bool erase(Node *n, const string &s, int i) {
```

```

    if (i == static_cast<int>(s.size())) {
        if (!n->is_terminal) {
            return false;
        }
        n->is_terminal = false;
        n->cnt--;
        return true;
    }
    auto it = n->children.find(s[i]);
    if (it == n->children.end() || !erase(it->second, s, i + 1)) {
        return false;
    }
    n->cnt--;
    // Prune the child only if it is now a non-terminal leaf; a childless terminal is still a key.
    if (it->second->children.empty() && !it->second->is_terminal) {
        delete it->second;
        n->children.erase(it);
    }
    return true;
}

template<typename Fn>
static void walk(Node *n, string &s, Fn f) {
    if (n->is_terminal) {
        f(s, n->value);
    }
    std::vector<std::pair<char, Node *>> children(n->children.begin(), n->children.end());
    std::sort(children.begin(), children.end());
    for (auto &[c, child] : children) {
        s += c;
        walk(child, s, f);
        s.pop_back();
    }
}

static void clean_up(Node *n) {
    for (auto &[c, child] : n->children) {
        clean_up(child);
    }
    delete n;
}

int num_terminals;

public:
    Trie() : root(new Node()), num_terminals(0) {}

    ~Trie() { clean_up(root); }
    Trie(const Trie &) = delete;
    Trie &operator=(const Trie &) = delete;
    int size() const { return num_terminals; }
    bool empty() const { return num_terminals == 0; }

    bool insert(const string &s, const V &v) {
        Node *n = root;
        for (char c : s) {
            auto [it, inserted] = n->children.try_emplace(c, nullptr);
            if (inserted) {
                it->second = new Node();
            }
            n = it->second;
        }
        if (n->is_terminal) {
            return false;
        }
        num_terminals++;
        n->is_terminal = true;
        n->value = v;
    }

```

```

    // Re-walk from the root to bump the subtree count along the inserted path.
    n = root;
    n->cnt++;
    for (char c : s) {
        n = n->children[c];
        n->cnt++;
    }
    return true;
}

bool erase(const string &s) {
    if (erase(root, s, 0)) {
        num_terminals--;
        return true;
    }
    return false;
}

const V *find(const string &s) const {
    Node *n = root;
    for (char c : s) {
        if (auto it = n->children.find(c); it != n->children.end()) {
            n = it->second;
        } else {
            return nullptr;
        }
    }
    return n->is_terminal ? &(n->value) : nullptr;
}

int count_prefix(const string &s) const {
    Node *n = root;
    for (char c : s) {
        auto it = n->children.find(c);
        if (it == n->children.end()) {
            return 0;
        }
        n = it->second;
    }
    return n->cnt;
}

template<typename Fn>
void walk(Fn f) const {
    string s;
    walk(root, s, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    vector<string> s{"", "a", "to", "tea", "ted", "ten", "i", "in", "inn"};
    Trie<int> t;
    assert(t.empty());
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        assert(t.insert(s[i], i));
    }
    t.walk([](string k, int v) { cout << "\"" << k << "\", " << v << "\"" << endl; });
    assert(!t.empty());
    assert(t.size() == 9);
    assert(!t.insert(s[0], 2));
}

```

```

assert(t.size() == 9);
assert(*t.find("") == 0);
assert(*t.find("ten") == 5);
assert(t.count_prefix("") == 9);    // Every key has the empty prefix.
assert(t.count_prefix("te") == 3);  // "tea", "ted", "ten".
assert(t.count_prefix("in") == 2);  // "in", "inn".
assert(t.count_prefix("z") == 0);
assert(t.erase("tea"));
assert(t.size() == 8);
assert(t.find("tea") == nullptr);
assert(t.count_prefix("te") == 2);  // "ted", "ten" remain.
assert(t.erase(""));
assert(t.find("") == nullptr);
assert(t.count_prefix("") == 7);

// Erasing a key must not delete a shorter key sharing its path.
Trie<int> u;
u.insert("bc", 1);
u.insert("bca", 2);
assert(u.erase("bca"));
assert(u.find("bc") != nullptr && *u.find("bc") == 1);
assert(u.size() == 1 && u.count_prefix("bc") == 1);
return 0;
}

```

Example Output

```

("", 0)
("a", 1)
("i", 6)
("in", 7)
("inn", 8)
("tea", 3)
("ted", 4)
("ten", 5)
("to", 2)

```

3.5.2 Radix Tree

Maintain a map of strings to values using a compressed tree data structure. Each node corresponds to a substring of an inserted string, and each inserted string corresponds to a path from the root to a node that is flagged as a terminal node. Contrary to a regular trie, a radix tree is more space efficient as it combines chains of nodes with only a single child. Children are stored in hash tables, and `walk()` sorts child labels as needed to preserve lexicographic traversal.

- `RadixTree<V>()` constructs an empty map.
- `size()` returns the size of the map.
- `empty()` returns whether the map is empty.
- `insert(s, v)` adds an entry with string key `s` and value `v` to the map, returning `true` if a new entry was added or `false` if the string already exists (in which case the map is unchanged and the old value associated with the string key is preserved).
- `erase(s)` removes the entry with string key `s` from the map, returning `true` if the removal was successful or `false` if the string to be removed was not found.
- `find(s)` returns a pointer to a const value associated with string key `s`, or `nullptr` if the key was not found.
- `count_prefix(s)` returns the number of keys currently in the map that have `s` as a prefix (a key equal to `s` counts as well), including the case where `s` ends partway along a compressed edge. `count_prefix("")` therefore equals `size()`.
- `walk(f)` calls the function `f(s, v)` on each entry of the map, in lexicographically ascending order of the string keys.

Time Complexity:

- $O(n)$ expected per call to `insert(s, v)`, `erase(s)`, `find(s)`, and `count_prefix(s)`, where n is the length of `s`, assuming the number of outgoing compressed edges checked at each node is small.
- $O(l \log l)$ per call to `walk()`, where l is the total length of string keys that are currently in the map.
- $O(1)$ per call to all other operations.

Space Complexity:

- $O(l)$ for storage of the radix tree, where l is the total length of string keys that are currently in the map.
- $O(n)$ auxiliary stack space for construction, destruction, `walk()`, where n is the maximum length of any string that has been inserted so far.
- $O(n)$ auxiliary stack space for `erase(s)`, where n is the length of `s`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <cstdint>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

template<typename V>
class RadixTree {
    struct Node {
        V value;
        bool is_terminal;
        int cnt; // Optional: maintain count of terminal keys in this subtree for count_prefix queries.
        std::unordered_map<string, Node *> children;

        Node(const V &value = V(), bool is_terminal = false)
            : value(value), is_terminal(is_terminal), cnt(0) {}
    } *root;

    static int lcp_len(const string &s1, const string &s2, int s2start) {
        int i = 0;
        for (int j = s2start; i < static_cast<int>(s1.size()) && j < static_cast<int>(s2.size());
             i++, j++) {
            if (s1[i] != s2[j]) {
                break;
            }
        }
        return i;
    }

    static bool insert(Node *n, const string &s, int i, const V &v) {
        if (i == static_cast<int>(s.size())) {
            if (n->is_terminal) {
                return false;
            }
            n->value = v;
            n->is_terminal = true;
            n->cnt++;
            return true;
        }
        for (auto it = n->children.begin(); it != n->children.end(); ++it) {
            int len = lcp_len(it->first, s, i);
            if (len == 0) {
                continue;
            }
            if (len == static_cast<int>(it->first.size())) {
                if (!insert(it->second, s, i + len, v)) {
                    return false;
                }
            }
            n->cnt++;
        }
    }
};
```

```

    return true;
}
string left = it->first.substr(0, len);
string right = it->first.substr(len);
Node *tmp = new Node();
tmp->cnt = it->second->cnt; // The new split node heads the same subtree as the old child.
tmp->children[right] = it->second;
n->children.erase(it);
n->children[left] = tmp;
if (len == static_cast<int>(s.size()) - i) {
    tmp->value = v;
    tmp->is_terminal = true;
    tmp->cnt++;
} else {
    insert(tmp, s, i + len, v);
}
n->cnt++;
return true;
}
Node *leaf = new Node(v, true);
leaf->cnt = 1;
n->children[s.substr(i)] = leaf;
n->cnt++;
return true;
}

static bool erase(Node *n, const string &s, int i) {
    if (i == static_cast<int>(s.size())) {
        if (!n->is_terminal) {
            return false;
        }
        n->is_terminal = false;
        n->cnt--;
        return true;
    }
    for (auto it = n->children.begin(); it != n->children.end(); ++it) {
        int len = lcp_len(it->first, s, i);
        if (len == 0) {
            continue;
        }
        // The entire edge label must be consumed; a partial match means the key is not present, so
        // erasing must fail rather than descend (otherwise erase("te") would remove "tea").
        if (len < static_cast<int>(it->first.size())) {
            return false;
        }
        Node *child = it->second;
        if (!erase(child, s, i + len)) {
            return false;
        }
        n->cnt--;
        // The merge below leaves counts intact: a non-terminal node with one child already has the
        // same subtree count as that child, so reusing the node keeps its `cnt` correct.
        if (child->children.empty() && !child->is_terminal) {
            delete child;
            n->children.erase(it);
        } else if (child->children.size() == 1) {
            Node *grandchild = child->children.begin()->second;
            if (!child->is_terminal) {
                string merged_key(it->first + child->children.begin()->first);
                child->value = grandchild->value;
                child->is_terminal = grandchild->is_terminal;
                child->children = grandchild->children;
                delete grandchild;
                n->children.erase(it);
                n->children[merged_key] = child;
            }
        }
    }
    return true;
}

```

```

    }
    return false;
}

template<typename Fn>
static void walk(Node *n, string &s, Fn f) {
    if (n->is_terminal) {
        f(s, n->value);
    }
    std::vector<std::pair<string, Node *>> children(n->children.begin(), n->children.end());
    std::sort(children.begin(), children.end());
    for (auto &[key, child] : children) {
        s += key;
        walk(child, s, f);
        s.erase(s.size() - key.size());
    }
}

static void clean_up(Node *n) {
    for (auto &[key, child] : n->children) {
        clean_up(child);
    }
    delete n;
}

int num_terminals;

public:
    RadixTree() : root(new Node()), num_terminals(0) {}

    ~RadixTree() { clean_up(root); }
    RadixTree(const RadixTree &) = delete;
    RadixTree &operator=(const RadixTree &) = delete;
    int size() const { return num_terminals; }
    bool empty() const { return num_terminals == 0; }

    bool insert(const string &s, const V &v) {
        if (insert(root, s, 0, v)) {
            num_terminals++;
            return true;
        }
        return false;
    }

    bool erase(const string &s) {
        if (erase(root, s, 0)) {
            num_terminals--;
            return true;
        }
        return false;
    }

    const V *find(const string &s) const {
        Node *n = root;
        int i = 0;
        while (i < static_cast<int>(s.size())) {
            bool found = false;
            for (auto &[key, child] : n->children) {
                if (key[0] == s[i]) {
                    // The entire edge label must be consumed; a partial match (s ends mid-edge or diverges
                    // from it) means the key is not present.
                    if (lcp_len(key, s, i) < static_cast<int>(key.size())) {
                        return nullptr;
                    }
                    i += static_cast<int>(key.size());
                    n = child;
                    found = true;
                    break;
                }
            }
        }
    }

```

```

    }
  }
  if (!found) {
    return nullptr;
  }
}
return n->is_terminal ? &(n->value) : nullptr;
}

int count_prefix(const string &s) const {
  Node *n = root;
  int i = 0;
  while (i < static_cast<int>(s.size())) {
    bool found = false;
    for (auto &[key, child] : n->children) {
      if (key[0] == s[i]) {
        int len = lcp_len(key, s, i);
        if (i + len == static_cast<int>(s.size())) {
          // `s` ends within (or exactly at the end of) this edge: every key below qualifies.
          return child->cnt;
        }
        if (len < static_cast<int>(key.size())) {
          return 0; // `s` diverges partway along the edge.
        }
        i += len;
        n = child;
        found = true;
        break;
      }
    }
    if (!found) {
      return 0;
    }
  }
  return n->cnt;
}

template<typename Fn>
void walk(Fn f) const {
  string s;
  walk(root, s, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void print_entry(const string &k, int v) {
  cout << "(" << k << ", " << v << ")" << endl;
}

int main() {
  vector<string> s{"", "a", "to", "tea", "ted", "ten", "i", "in", "inn"};
  RadixTree<int> t;
  assert(t.empty());
  for (int i = 0; i < static_cast<int>(s.size()); i++) {
    assert(t.insert(s[i], i));
  }
  t.walk([&](string k, int v) { cout << "(" << k << ", " << v << ")" << endl; });
  assert(!t.empty());
  assert(t.size() == 9);
  assert(!t.insert(s[0], 2));
  assert(t.size() == 9);
}

```

```

assert(t.find("") && *t.find("") == 0);
assert(*t.find("ten") == 5);
assert(t.count_prefix("") == 9);    // Every key has the empty prefix.
assert(t.count_prefix("te") == 3);  // "tea", "ted", "ten" (prefix ends mid-edge).
assert(t.count_prefix("in") == 2);  // "in", "inn".
assert(t.count_prefix("z") == 0);
assert(t.erase("tea"));
assert(t.size() == 8);
assert(t.find("tea") == nullptr);
assert(t.count_prefix("te") == 2);  // "ted", "ten" remain.
assert(t.erase(""));
assert(t.find("") == nullptr);
assert(t.count_prefix("") == 7);

// Erasing a key must not delete a shorter key sharing its path.
RadixTree<int> u;
u.insert("bc", 1);
u.insert("bca", 2);
assert(u.erase("bca"));
assert(u.find("bc") != nullptr && *u.find("bc") == 1);
assert(u.size() == 1 && u.count_prefix("bc") == 1);
return 0;
}

```

Example Output

```

("", 0)
("a", 1)
("i", 6)
("in", 7)
("inn", 8)
("tea", 3)
("ted", 4)
("ten", 5)
("to", 2)

```

3.5.3 Suffix Tree (Ukkonen)

A suffix tree is the compressed trie of every suffix of a string: a rooted tree in which each edge is labeled with a substring, no two edges leaving a node start with the same character, and the concatenation of labels from the root to each leaf spells one suffix. Paths from the root therefore spell exactly the distinct substrings of the string, which makes the tree a powerful index for substring queries, repeated-substring problems, and many other tasks. It is the explicit-pointer counterpart of the suffix automaton; the two store the same information and either can be converted into the other.

This implementation is Ukkonen's algorithm, which builds the tree online in linear time by feeding one character at a time. It maintains an "active point" (a node, an edge, and a length along that edge) describing where the next suffix must be inserted, together with suffix links that let it hop between successive insertion points in amortized constant time. Edges to leaves are stored with an open-ended length that grows automatically as the string lengthens, so each phase touches only the suffixes that genuinely change. A unique terminal sentinel smaller than every real character is appended internally so that no suffix is a prefix of another and every suffix ends at its own leaf. The input must therefore not contain the character `'\1'`.

- `SuffixTree(s)` builds the suffix tree of string `s`.
- `contains(t)` returns whether string `t` occurs as a substring of `s`.
- `count_distinct_substrings()` returns the number of distinct nonempty substrings of `s`.
- `longest_repeated_substring()` returns one longest substring of `s` that occurs at least twice, or the empty string if every character is distinct.

Time Complexity:

- $O(n)$ to construct the tree for a string of length n (using a per-node ordered map, $O(n \log s)$ where s is the alphabet size).
- $O(m \log s)$ per call to `contains(t)`, where m is the length of `t`.
- $O(n)$ per call to `count_distinct_substrings()` and `longest_repeated_substring()`.

Space Complexity:

- $O(n)$ nodes and edges.
- $O(n)$ auxiliary stack space for the traversals.

```
#include <map>
#include <string>
#include <vector>

class SuffixTree {
    static const int OPEN = -1; // Marks a leaf edge whose right end follows the growing string.

    struct Node {
        int start, end; // Edge into this node labels s[start..end]; end == OPEN means leaf_end.
        int link;
        std::map<char, int> next;
        Node(int start, int end) : start(start), end(end), link(0) {}
    };

    std::string s;
    std::vector<Node> tree;
    int leaf_end;
    // Active point and bookkeeping for Ukkonen's algorithm.
    int active_node, active_edge, active_length, remaining, last_new;

    int edge_end(int v) const { return tree[v].end == OPEN ? leaf_end : tree[v].end; }
    int edge_len(int v) const { return edge_end(v) - tree[v].start + 1; }

    int make_node(int start, int end) {
        tree.emplace_back(start, end);
        return static_cast<int>(tree.size()) - 1;
    }

    // If the active length runs past the edge into v, descend and report that we moved.
    bool walk_down(int v) {
        if (active_length >= edge_len(v)) {
            active_edge += edge_len(v);
            active_length -= edge_len(v);
            active_node = v;
            return true;
        }
        return false;
    }

    void extend(int pos) {
        leaf_end = pos;
        remaining++;
        last_new = -1;
        while (remaining > 0) {
            if (active_length == 0) {
                active_edge = pos;
            }
            char c = s[active_edge];
            auto it = tree[active_node].next.find(c);
            if (it == tree[active_node].next.end()) {
                // Rule 2: no edge starts with c, so grow a new leaf from the active node.
                tree[active_node].next[c] = make_node(pos, OPEN);
                if (last_new != -1) {
                    tree[last_new].link = active_node;
                    last_new = -1;
                }
            } else {

```

```

    int v = it->second;
    if (walk_down(v)) {
        continue; // Re-evaluate the same suffix from the deeper node.
    }
    if (s[tree[v].start + active_length] == s[pos]) {
        // Rule 3: the character is already present; stop and lengthen the active point.
        if (last_new != -1) {
            tree[last_new].link = active_node;
        }
        active_length++;
        break;
    }
    // Rule 2 with a split: break edge v in two and attach a new leaf at the split point.
    int split = make_node(tree[v].start, tree[v].start + active_length - 1);
    tree[active_node].next[c] = split;
    tree[split].next[s[pos]] = make_node(pos, OPEN);
    tree[v].start += active_length;
    tree[split].next[s[tree[v].start]] = v;
    if (last_new != -1) {
        tree[last_new].link = split;
    }
    last_new = split;
}
remaining--;
if (active_node == 0 && active_length > 0) {
    active_length--;
    active_edge = pos - remaining + 1;
} else if (active_node != 0) {
    active_node = tree[active_node].link;
}
}
}

// String depth and right end (one past) of the deepest internal node, for repeated substrings.
void deepest_internal(int v, int depth, int &best_depth, int &best_end) const {
    bool internal = false;
    for (const auto &kv : tree[v].next) {
        internal = true;
        int w = kv.second;
        deepest_internal(w, depth + edge_len(w), best_depth, best_end);
    }
    if (internal && v != 0 && depth > best_depth) {
        best_depth = depth;
        best_end = edge_end(v) + 1;
    }
}

public:
explicit SuffixTree(const std::string &str) : s(str) {
    s.push_back('\1'); // Unique terminal sentinel, smaller than every real character.
    make_node(-1, -1); // Root, at index 0; its incoming edge is never read.
    active_node = 0;
    active_edge = 0;
    active_length = 0;
    remaining = 0;
    leaf_end = -1;
    last_new = -1;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        extend(i);
    }
}

bool contains(const std::string &t) const {
    int v = 0, i = 0, m = static_cast<int>(t.size());
    while (i < m) {
        auto it = tree[v].next.find(t[i]);
        if (it == tree[v].next.end()) {
            return false;
        }
    }
}

```

```

    }
    v = it->second;
    int len = edge_len(v);
    for (int j = 0; j < len && i < m; j++, i++) {
        if (s[tree[v].start + j] != t[i]) {
            return false;
        }
    }
}
return true;
}

int count_distinct_substrings() const {
    int total = 0;
    for (int v = 1; v < static_cast<int>(tree.size()); v++) {
        total += edge_len(v);
    }
    // Every leaf edge ends in the sentinel; the s.size() suffixes of the sentineled string are the
    // only substrings that contain it, so removing them leaves the count for the original string.
    return total - static_cast<int>(s.size());
}

std::string longest_repeated_substring() const {
    int best_depth = 0, best_end = 0;
    deepest_internal(0, 0, best_depth, best_end);
    return s.substr(best_end - best_depth, best_depth);
}
};

```

Example Usage

```

#include <cassert>
#include <set>
using namespace std;

int main() {
    SuffixTree st("banana");
    assert(st.contains("ana"));
    assert(st.contains("banana"));
    assert(st.contains(""));
    assert(!st.contains("anana_"));
    assert(!st.contains("nb"));
    assert(st.longest_repeated_substring() == "ana");

    // "banana" has 15 distinct substrings:
    // a, an, ana, anan, anana, b, ba, ban, bana, banan, banana, n, na, nan, nana.
    assert(st.count_distinct_substrings() == 15);

    // Cross-check both queries against brute force on a small string.
    string text = "mississippi";
    SuffixTree t(text);
    set<string> subs;
    int n = static_cast<int>(text.size());
    for (int i = 0; i < n; i++) {
        for (int len = 1; i + len <= n; len++) {
            subs.insert(text.substr(i, len));
        }
    }
    assert(t.count_distinct_substrings() == static_cast<int>(subs.size()));
    for (const string &sub : subs) {
        assert(t.contains(sub));
    }
    assert(!t.contains("issa"));
    assert(t.longest_repeated_substring() == "issi"); // "issi" appears at indices 1 and 4.
    return 0;
}

```


3.5.4 Suffix Automaton

Maintains a compact deterministic automaton representing all substrings of a string. A suffix automaton is useful for substring membership queries, counting distinct substrings, finding longest common substrings, and many other string dynamic programming tasks.

Each state represents an equivalence class of substrings with the same set of ending positions. The value `st[v].len` is the maximum length represented by state `v`, while `st[v].link` points to the state representing the longest proper suffix of that class.

- `SuffixAutomaton()` constructs an empty automaton.
- `SuffixAutomaton(s)` constructs the automaton for string `s`.
- `extend(c)` appends character `c` to the current string.
- `contains(t)` returns whether string `t` occurs as a substring.
- `first_occurrence(t)` returns the starting index of the first occurrence of `t`, or `-1` if absent.
- `count_distinct_substrings()` returns the number of distinct nonempty substrings.
- `longest_common_substring(t)` returns one longest substring common to the built string and string `t`.

Time Complexity:

- $O(n)$ expected to construct the automaton for a string of length n .
- $O(m)$ expected per call to `contains(t)`, `first_occurrence(t)`, or `longest_common_substring(t)`, where m is the length of `t`.
- $O(n)$ per call to `count_distinct_substrings()`.

Space Complexity:

- $O(n)$ transition storage and $O(n)$ states.
- $O(1)$ auxiliary per character processed.

```
#include <stdint>
#include <string>
#include <unordered_map>
#include <vector>
using std::string;

class SuffixAutomaton {
    struct State {
        int len, link, first_pos;
        std::unordered_map<char, int> next;

        State() : len(0), link(-1), first_pos(-1) {}
    };

    std::vector<State> st;
    int last;

public:
    SuffixAutomaton() : st(1), last(0) {}

    explicit SuffixAutomaton(const string &s) : st(1), last(0) {
        for (char c : s) {
            extend(c);
        }
    }

    void extend(char c) {
        int cur = static_cast<int>(st.size());
        st.emplace_back();
        st[cur].len = st[last].len + 1;
        st[cur].first_pos = st[cur].len - 1;
        int p = last;
        while (p != -1 && st[p].next.find(c) == st[p].next.end()) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
    }
};
```

```

    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = static_cast<int>(st.size());
            st.push_back(st[q]);
            st[clone].len = st[p].len + 1;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

bool contains(const string &t) const {
    int v = 0;
    for (char c : t) {
        if (auto it = st[v].next.find(c); it != st[v].next.end()) {
            v = it->second;
        } else {
            return false;
        }
    }
    return true;
}

// Returns the starting index of the first (leftmost) occurrence of `t` as a substring, or -1 if
// `t` does not occur. `first_pos` stores the end index of the first occurrence of each state's
// class, so subtracting the length of `t` recovers its start.
int first_occurrence(const string &t) const {
    int v = 0;
    for (char c : t) {
        auto it = st[v].next.find(c);
        if (it == st[v].next.end()) {
            return -1;
        }
        v = it->second;
    }
    return st[v].first_pos - static_cast<int>(t.size()) + 1;
}

int64_t count_distinct_substrings() const {
    int64_t res = 0;
    for (int v = 1; v < static_cast<int>(st.size()); v++) {
        res += st[v].len - st[st[v].link].len;
    }
    return res;
}

string longest_common_substring(const string &t) const {
    int v = 0, len = 0, best = 0, best_pos = -1;
    for (int i = 0; i < static_cast<int>(t.size()); i++) {
        while (v && !st[v].next.count(t[i])) {
            v = st[v].link;
            len = st[v].len;
        }
        if (auto it = st[v].next.find(t[i]); it != st[v].next.end()) {
            v = it->second;
            len++;
        } else {
            v = 0;
        }
    }

```

```

        len = 0;
    }
    if (len > best) {
        best = len;
        best_pos = i;
    }
}
return best == 0 ? "" : t.substr(best_pos - best + 1, best);
}
};

```

Example Usage

```

#include <cassert>

int main() {
    SuffixAutomaton sa("ababa");
    assert(sa.contains("aba"));
    assert(sa.contains("bab"));
    assert(!sa.contains("abba"));
    assert(sa.count_distinct_substrings() == 9);
    assert(sa.longest_common_substring("zzbabab") == "baba");
    assert(sa.first_occurrence("aba") == 0);
    assert(sa.first_occurrence("bab") == 1);
    assert(sa.first_occurrence("abba") == -1);

    SuffixAutomaton online;
    online.extend('a');
    online.extend('b');
    online.extend('c');
    assert(online.contains("bc"));
    return 0;
}

```

3.5.5 Eertree

Maintains all distinct palindromic substrings of a string online using an Eertree, also known as a palindromic tree. Each non-root node represents one distinct palindrome, and suffix links connect each palindrome to its longest proper palindromic suffix.

The structure has two roots: one of length -1 , which simplifies boundary cases, and one of length 0 , representing the empty palindrome. After each appended character, `last` points to the node for the longest palindromic suffix of the current string.

- `Eertree()` constructs an empty palindromic tree.
- `Eertree(s)` constructs the tree for string `s`.
- `add(c)` appends character `c` and returns `true` if this creates a new distinct palindrome.
- `count_distinct_palindromes()` returns the number of distinct nonempty palindromic substrings.
- `longest_suffix_length()` returns the length of the current longest palindromic suffix.
- `count_occurrences()` propagates occurrence counts through suffix links and returns `occ[v]` for each node `v`.

Time Complexity:

- $O(n)$ expected to build the tree for a string of length n .
- $O(1)$ expected amortized per call to `add(c)`.
- $O(n)$ per call to `count_occurrences()`.

Space Complexity:

- $O(n)$ transition storage and $O(n)$ nodes.
- $O(1)$ auxiliary per appended character.

```

#include <string>
#include <unordered_map>
#include <vector>
using std::string;

class Eertree {
    struct Node {
        int len, link, occ;
        std::unordered_map<char, int> next;

        Node(int len = 0) : len(len), link(0), occ(0) {}
    };

    std::vector<Node> tree;
    string s;
    int last;

    int get_suffix(int v, int pos, char c) const {
        while (pos - 1 - tree[v].len < 0 || s[pos - 1 - tree[v].len] != c) {
            v = tree[v].link;
        }
        return v;
    }

public:
    Eertree() : tree(), s(), last(1) {
        tree.emplace_back(-1);
        tree.emplace_back(0);
        tree[0].link = 0;
        tree[1].link = 0;
    }

    explicit Eertree(const string &text) : tree(), s(), last(1) {
        tree.emplace_back(-1);
        tree.emplace_back(0);
        tree[0].link = 0;
        tree[1].link = 0;
        for (char c : text) {
            add(c);
        }
    }

    bool add(char c) {
        s += c;
        int pos = static_cast<int>(s.size()) - 1;
        int cur = get_suffix(last, pos, c);
        auto it = tree[cur].next.find(c);
        if (it != tree[cur].next.end()) {
            last = it->second;
            tree[last].occ++;
            return false;
        }
        last = static_cast<int>(tree.size());
        tree.emplace_back(tree[cur].len + 2);
        tree[last].occ = 1;
        tree[cur].next[c] = last;
        if (tree[last].len == 1) {
            tree[last].link = 1;
            return true;
        }
        int link_parent = get_suffix(tree[cur].link, pos, c);
        tree[last].link = tree[link_parent].next[c];
        return true;
    }

    int count_distinct_palindromes() const { return static_cast<int>(tree.size()) - 2; }
    int longest_suffix_length() const { return tree[last].len; }
}

```

```

std::vector<int> count_occurrences() const {
    std::vector<int> occ(tree.size());
    for (int i = 0; i < static_cast<int>(tree.size()); i++) {
        occ[i] = tree[i].occ;
    }
    for (int i = static_cast<int>(tree.size()) - 1; i >= 2; i--) {
        occ[tree[i].link] += occ[i];
    }
    return occ;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    Eertree t("abacaba");
    assert(t.count_distinct_palindromes() == 7);
    assert(t.longest_suffix_length() == 7);

    Eertree online;
    assert(online.add('a'));
    assert(online.add('a'));
    assert(online.add('b'));
    assert(online.add('a'));
    assert(online.count_distinct_palindromes() == 4);
    assert(online.longest_suffix_length() == 3);

    Eertree duplicate;
    assert(duplicate.add('a'));
    assert(duplicate.add('b'));
    assert(duplicate.add('c'));
    assert(!duplicate.add('a'));

    std::vector<int> occ = t.count_occurrences();
    assert(static_cast<int>(occ.size()) == t.count_distinct_palindromes() + 2);
    return 0;
}

```

3.6 Sequence Dynamic Programming

3.6.1 Longest Common Substring

Given two strings, determine their longest common substring: a contiguous run of characters that appears in both. A dynamic program computes, for each pair of prefixes, the length of their longest common suffix; the largest value over all pairs locates a longest common substring. Only the previous row of the table is retained, so the auxiliary space is linear in the shorter string.

- `longest_common_substring(s1, s2)` returns a longest common substring of `s1` and `s2` (one such substring if several are tied in length), or the empty string if they have none in common.

Time Complexity: $O(n \cdot m)$ per call to `longest_common_substring(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

Space Complexity: $O(\min(n, m))$ auxiliary heap space, where n and m are the lengths of `s1` and `s2`, respectively.

```

#include <string>

```

```

#include <vector>
using std::string;

string longest_common_substring(const string &s1, const string &s2) {
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());
    if (n == 0 || m == 0) {
        return "";
    }
    if (n < m) {
        return longest_common_substring(s2, s1);
    }
    std::vector<int> curr(m), prev(m);
    int pos = 0, len = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (s1[i] == s2[j]) {
                curr[j] = (i > 0 && j > 0) ? prev[j - 1] + 1 : 1;
                if (len < curr[j]) {
                    len = curr[j];
                    pos = i - curr[j] + 1;
                }
            } else {
                curr[j] = 0;
            }
        }
        curr.swap(prev);
    }
    return s1.substr(pos, len);
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(longest_common_substring("bbbabca", "aababcd") == "babc");
    return 0;
}

```

3.6.2 Longest Common Subsequence

Given two strings, determine their longest common subsequence. A subsequence is a string that can be derived from the original string by deleting some elements without changing the order of the remaining elements (e.g. "ACE" is a subsequence of "ABCDE", but "BAE" is not).

- `longest_common_subsequence(s1, s2)` returns the longest common subsequence of strings `s1` and `s2` using a classic dynamic programming approach. This implementation computes `dp[i][j]` (the length of the longest common subsequence for the length i prefix of `s1` and the length j prefix of `s2`) before following the path backwards to construct the answer.
- `hirschberg_lcs(s1, s2)` returns the longest common subsequence of strings `s1` and `s2` using the more memory efficient Hirschberg's algorithm.

Time Complexity: $O(n \cdot m)$ per call to `longest_common_subsequence(s1, s2)` as well as `hirschberg_lcs(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

Space Complexity:

- $O(n \cdot m)$ auxiliary heap space for `longest_common_subsequence(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.
- $O(\log(\max(n, m)))$ auxiliary stack space and $O(\min(n, m))$ auxiliary heap space for `hirschberg_lcs(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

```

#include <algorithm>
#include <iterator>
#include <string>
#include <vector>
using std::string;

string longest_common_subsequence(const string &s1, const string &s2) {
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());
    std::vector<std::vector<int>>> dp(n + 1, std::vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = std::max(dp[i][j - 1], dp[i - 1][j]);
            }
        }
    }
    string res;
    for (int i = n, j = m; i > 0 && j > 0;) {
        if (s1[i - 1] == s2[j - 1]) {
            res += s1[i - 1];
            i--;
            j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    std::reverse(res.begin(), res.end());
    return res;
}

template<typename It>
std::vector<int> lcs_len(It lo1, It hi1, It lo2, It hi2) {
    std::vector<int> res(std::distance(lo2, hi2) + 1, prev(res));
    for (It it1 = lo1; it1 != hi1; ++it1) {
        res.swap(prev);
        int i = 0;
        for (It it2 = lo2; it2 != hi2; ++it2) {
            res[i + 1] = (*it1 == *it2) ? prev[i] + 1 : std::max(res[i], prev[i + 1]);
            i++;
        }
    }
    return res;
}

template<typename It>
void hirschberg_rec(It lo1, It hi1, It lo2, It hi2, string *res) {
    if (lo1 == hi1) {
        return;
    }
    if (lo1 + 1 == hi1) {
        if (std::find(lo2, hi2, *lo1) != hi2) {
            *res += *lo1;
        }
        return;
    }
    It mid1 = lo1 + (hi1 - lo1) / 2;
    std::reverse_iterator<It> rlo1(hi1), rmid1(mid1), rlo2(hi2), rhi2(lo2);
    std::vector<int> fwd = lcs_len(lo1, mid1, lo2, hi2);
    std::vector<int> rev = lcs_len(rlo1, rmid1, rlo2, rhi2);
    It mid2 = lo2;
    int maxlen = -1;
    for (int i = 0, j = static_cast<int>(rev.size()) - 1; i < static_cast<int>(fwd.size());
        i++, j--) {
        if (fwd[i] + rev[j] > maxlen) {

```

```

        maxlen = fwd[i] + rev[j];
        mid2 = lo2 + i;
    }
}
hirschberg_rec(lo1, mid1, lo2, mid2, res);
hirschberg_rec(mid1, hi1, mid2, hi2, res);
}

string hirschberg_lcs(const string &s1, const string &s2) {
    if (s1.size() < s2.size()) {
        return hirschberg_lcs(s2, s1);
    }
    string res;
    hirschberg_rec(s1.begin(), s1.end(), s2.begin(), s2.end(), &res);
    return res;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(longest_common_subsequence("xmjyauz", "mzjawxu") == "mjau");
    assert(hirschberg_lcs("xmjyauz", "mzjawxu") == "mjau");
    return 0;
}

```

3.6.3 Sequence Alignment (Needleman-Wunsch)

Given two strings, determine their minimum-cost alignment. An alignment of two strings is a transformation of both strings by inserting gap characters `_` in some way to make the final lengths equal. The total cost of an alignment given a `gap_cost` (insertion or deletion cost) and a `sub_cost` (substitution, i.e. mismatch cost) is `gap_cost` times the number of inserted gaps, plus `sub_cost` times the number of indices at which the two aligned strings differ.

- `align_sequences(s1, s2, gap_cost, sub_cost)` returns a pair of aligned strings for strings `s1` and `s2`, using a classic dynamic programming approach. This implementation first computes `dp[i][j]` (the cost of aligning the length i prefix of `s1` with the length j prefix of `s2`) before following the path backwards to construct the answer. For `gap_cost = sub_cost = 1`, `dp[n][m]` will be the Levenshtein edit distance, where n and m are the lengths of `s1` and `s2`, respectively.
- `hirschberg_align_sequences(s1, s2, gap_cost, sub_cost)` returns the sequence alignment of strings `s1` and `s2` using the more memory efficient Hirschberg's algorithm.

Time Complexity: $O(n \cdot m)$ per call to `align_sequences(s1, s2)` as well as `hirschberg_align_sequences(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

Space Complexity:

- $O(n \cdot m)$ auxiliary heap space for `align_sequences(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.
- $O(\log(\max(n, m)))$ auxiliary stack space and $O(\min(n, m))$ auxiliary heap space for `hirschberg_align_sequences(s1, s2)`, where n and m are the lengths of `s1` and `s2`, respectively.

```

#include <algorithm>
#include <string>
#include <utility>
#include <vector>
using std::string;

std::pair<string, string> align_sequences(

```



```

    const string &s1, const string &s2, int gap_cost = 1, int sub_cost = 1
) {
    int n = static_cast<int>(s1.size()), m = static_cast<int>(s2.size());
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) {
        dp[i][0] = i * gap_cost;
    }
    for (int j = 0; j <= m; j++) {
        dp[0][j] = j * gap_cost;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            dp[i][j] = (s1[i - 1] == s2[j - 1]) ? dp[i - 1][j - 1]
                : std::min(
                    dp[i - 1][j - 1] + sub_cost,
                    std::min(dp[i - 1][j], dp[i][j - 1]) + gap_cost
                );
        }
    }
    string res1, res2;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1] || dp[i][j] == dp[i - 1][j - 1] + sub_cost) {
            res1 += s1[--i];
            res2 += s2[--j];
        } else if (dp[i][j] == dp[i - 1][j] + gap_cost) {
            res1 += s1[--i];
            res2 += '_';
        } else if (dp[i][j] == dp[i][j - 1] + gap_cost) {
            res1 += '_';
            res2 += s2[--j];
        }
    }
    while (i > 0 || j > 0) {
        res1 += (i > 0) ? s1[--i] : '_';
        res2 += (j > 0) ? s2[--j] : '_';
    }
    std::reverse(res1.begin(), res1.end());
    std::reverse(res2.begin(), res2.end());
    return {res1, res2};
}

template<typename It>
std::vector<int> row_cost(It lo1, It hi1, It lo2, It hi2, int gap_cost, int sub_cost) {
    std::vector<int> res(std::distance(lo2, hi2) + 1, prev(res));
    for (It it1 = lo1; it1 != hi1; ++it1) {
        res.swap(prev);
        int i = 0;
        for (It it2 = lo2; it2 != hi2; ++it2) {
            res[i + 1] = (*it1 == *it2) ? prev[i] : std::min(prev[i] + sub_cost, res[i] + gap_cost);
            i++;
        }
    }
    return res;
}

template<typename It>
void hirschberg_rec(
    It lo1, It hi1, It lo2, It hi2, string *res1, string *res2, int gap_cost, int sub_cost
) {
    if (lo1 == hi1) {
        for (It it2 = lo2; it2 != hi2; ++it2) {
            *res1 += '_';
            *res2 += *it2;
        }
        return;
    }
    if (lo1 + 1 == hi1) {

```

```

    It pos = std::find(lo2, hi2, *lo1);
    bool insert = (pos == hi2) && (gap_cost * (hi2 - lo2 + 1) < sub_cost);
    if (lo2 == hi2 || insert) {
        *res1 += *lo1;
        *res2 += '_';
    }
    for (It it2 = lo2; it2 != hi2; ++it2) {
        *res1 += (pos == it2 || (!insert && it2 == lo2)) ? *lo1 : '_';
        *res2 += *it2;
    }
    return;
}
It mid1 = lo1 + (hi1 - lo1) / 2;
std::reverse_iterator<It> rlo1(hi1), rmid1(mid1), rlo2(hi2), rhi2(lo2);
std::vector<int> fwd = row_cost(lo1, mid1, lo2, hi2, gap_cost, sub_cost);
std::vector<int> rev = row_cost(rlo1, rmid1, rlo2, rhi2, gap_cost, sub_cost);
It mid2 = lo2;
int mincost = -1;
for (int i = 0, j = static_cast<int>(rev.size()) - 1; i < static_cast<int>(fwd.size());
    i++, j--) {
    if (mincost < 0 || fwd[i] + rev[j] < mincost) {
        mincost = fwd[i] + rev[j];
        mid2 = lo2 + i;
    }
}
hirschberg_rec(lo1, mid1, lo2, mid2, res1, res2, gap_cost, sub_cost);
hirschberg_rec(mid1, hi1, mid2, hi2, res1, res2, gap_cost, sub_cost);
}

std::pair<string, string> hirschberg_align_sequences(
    const string &s1, const string &s2, int gap_cost = 1, int sub_cost = 1
) {
    if (s1.size() < s2.size()) {
        return hirschberg_align_sequences(s2, s1, gap_cost, sub_cost);
    }
    string res1, res2;
    hirschberg_rec(s1.begin(), s1.end(), s2.begin(), s2.end(), &res1, &res2, gap_cost, sub_cost);
    return {res1, res2};
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(
        align_sequences("AGGGCT", "AGGCA", 2, 3) == (std::pair<string, string>{"AGGGCT", "A_GGCA"})
    );
    assert(
        hirschberg_align_sequences("AGGGCT", "AGGCA", 2, 3) ==
        (std::pair<string, string>{"AGGGCT", "A_GGCA"})
    );
    return 0;
}

```

3.7 Expression Parsing

3.7.1 Recursive Descent Parsing (Simple)

Evaluate an expression in accordance to the order of operations (parentheses, unary plus and minus signs, multiplication/division, addition/subtraction). The following is a minimalistic recursive descent implementation

using iterators. Each precedence level is parsed by its own function that obtains operands by calling the next-tighter level, so the call structure itself enforces the order of operations.

- `eval(s)` returns an evaluation of the arithmetic expression `s`.

Time Complexity: $O(n)$ per call to `eval(s)`, where n is the length of `s`.

Space Complexity: $O(n)$ auxiliary stack space for `eval(s)`, where n is the length of `s`.

```
#include <string>

template<typename It>
int eval(It &it, int prec) {
    if (prec == 0) {
        int sign = 1, res = 0;
        for (; *it == '-'; it++) {
            sign *= -1;
        }
        if (*it == '(') {
            res = eval(++it, 2);
            it++;
        } else
            while (*it >= '0' && *it <= '9') {
                res = 10 * res + (*it++) - '0';
            }
        return sign * res;
    }
    int num = eval(it, prec - 1);
    while (!(prec == 2 && *it != '+' && *it != '-' || (prec == 1 && *it != '*' && *it != '/'))) {
        switch (*it++) {
            case '+':
                num += eval(it, prec - 1);
                break;
            case '-':
                num -= eval(it, prec - 1);
                break;
            case '*':
                num *= eval(it, prec - 1);
                break;
            case '/':
                num /= eval(it, prec - 1);
                break;
        }
    }
    return num;
}

int eval(const std::string &s) {
    std::string expr = s + '\0';
    auto it = expr.begin();
    return eval(it, 2);
}
```

Example Usage

```
#include <cassert>

int main() {
    assert(eval("1++1") == 2);
    assert(eval("1+2*3*4+3*(2+2)-100") == -63);
    return 0;
}
```

3.7.2 Recursive Descent Parsing

Evaluate an expression with custom operand types, prefix unary operators, binary operators, and precedences. Evaluation is performed by recursive descent: the parser recursively evaluates tighter precedence levels before looser ones. Parentheses are supported, but multiplication by juxtaposition is not.

An arbitrary operand type is supported by changing the `Operand` alias and the static `is_operand()` and `eval_operand()` helpers. For maximum reliability, the string representation of operands should not use characters shared by any operator. For instance, the best practice instead of accepting `-1` as a valid operand (since the `-` sign may conflict with the identical binary operator), is to specify nonnegative numbers as operands alongside the unary operator `-`.

Operators may be non-empty strings of any length, but should not contain any parentheses or shared characters with the string representations of operands. Ideally, operators should not be prefixes or suffixes of one another, else the tokenization process may be ambiguous. For example, if `++` and `+` are both operators, then `++` may be split into either `["+"]`, `["+"]` or `["++"]` depending on the lexicographical ordering of conflicting operators.

- `RecursiveDescentParser(unary_op, binary_op)` initializes a parser with operators specified by hash tables `unary_op` (of operator to unary function object) and `binary_op` (of operator to pair of binary function object and operator precedence). Operator precedences should be numbered upwards starting at 0 (lowest precedence, evaluated last).
- `split(s)` returns a vector of tokens for the expression `s`, split on the given operators during construction. Each parenthesis, operator, and operand satisfying `is_operand()` will be split into a separate token. The algorithm is naive, matching operators lazily in the case of overlapping operators as mentioned above. Under these circumstances, the parse may not always succeed.
- `eval(lo, hi)` returns the evaluation of a range `[lo, hi)` of already split-up expression tokens, where `lo` and `hi` must be random-access iterators.
- `eval(s)` returns the evaluation of expression `s`, after first calling `split(s)` to obtain the tokens.

Time Complexity:

- $O(m)$ per call to the constructor, where m is the total number of operators.
- $O(n \cdot m \cdot k)$ per call to `split(s)`, where n is the length of `s`, m is the total number of operators defined for the parser instance, and k is the maximum length for any operator representation.
- $O(n)$ expected per call to `eval(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(n \cdot m \cdot k + n)$ expected per call to `eval(s)`, where n is the length of `s`, and m and k are as defined previously.

Space Complexity:

- $O(m \cdot k)$ for storage of the m operators, of maximum length k .
- $O(n)$ auxiliary stack space for `split(s)`, `eval(lo, hi)`, and `eval(s)`, where n is the length of the argument.

```
#include <algorithm>
#include <cctype>
#include <functional>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

// Define the custom operand type and representation below.
using Operand = double;
using UnaryOp = std::function<Operand(Operand)>;
using BinaryOp = std::function<Operand(Operand, Operand)>;

struct BinaryRule {
    BinaryOp op;
```

```

    int precedence;
};

class RecursiveDescentParser {
    using unary_op_map = std::unordered_map<string, UnaryOp>;
    using binary_op_map = std::unordered_map<string, BinaryRule>;
    unary_op_map unary_ops;
    binary_op_map binary_ops;
    std::set<string> op_tokens;
    int max_precedence;

    static bool is_operand(const string &s) {
        int npoints = 0;
        for (char c : s) {
            if (c == '.') {
                if (++npoints > 1) {
                    return false;
                }
            } else if (!isdigit(static_cast<unsigned char>(c))) {
                return false;
            }
        }
        return !s.empty();
    }

    static Operand eval_operand(const string &s) {
        Operand res;
        std::stringstream ss(s);
        ss >> res;
        return res;
    }

    template<typename StrIt>
    Operand eval_unary(StrIt &lo, StrIt hi) {
        if (is_operand(*lo)) {
            return eval_operand(++lo);
        }
        if (auto it = unary_ops.find(*lo); it != unary_ops.end()) {
            return (it->second)(eval_unary(++lo, hi));
        }
        if (*lo != "(") {
            throw std::runtime_error("Expected \"(\" during eval.");
        }
        Operand res = eval_binary(++lo, hi, 0);
        if (*lo != ")") {
            throw std::runtime_error("Expected \")\" during eval.");
        }
        ++lo;
        return res;
    }

    template<typename StrIt>
    Operand eval_binary(StrIt &lo, StrIt hi, int precedence) {
        if (precedence > max_precedence) {
            return eval_unary(lo, hi);
        }
        Operand v = eval_binary(lo, hi, precedence + 1);
        while (lo != hi) {
            auto it = binary_ops.find(*lo);
            if (it == binary_ops.end() || it->second.precedence != precedence) {
                return v;
            }
            v = (it->second.op)(v, eval_binary(++lo, hi, precedence + 1));
        }
        return v;
    }

    static string strip(string s) {

```

```

    auto not_space = [](unsigned char c) { return !std::isspace(c); };
    s.erase(s.begin(), std::find_if(s.begin(), s.end(), not_space));
    s.erase(std::find_if(s.rbegin(), s.rend(), not_space).base(), s.end());
    return s;
}

public:
RecursiveDescentParser(const unary_op_map &unary_ops, const binary_op_map &binary_ops)
    : unary_ops(unary_ops), binary_ops(binary_ops) {
    for (const auto &[op, _] : unary_ops) {
        op_tokens.insert(op);
    }
    max_precedence = 0;
    for (const auto &[op, fn_prec] : binary_ops) {
        op_tokens.insert(op);
        max_precedence = std::max(max_precedence, fn_prec.precedence);
    }
}

std::vector<string> split(const string &s) {
    std::vector<string> res;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        if (s[i] == ' ') {
            continue;
        }
        int next_paren = static_cast<int>(s.size());
        for (int j = i; j < static_cast<int>(s.size()); j++) {
            if (s[j] == '(' || s[j] == ')') {
                next_paren = j;
                break;
            }
        }
        while (i < next_paren) {
            int found = next_paren;
            string found_op;
            for (int j = i; j < next_paren && found == next_paren; j++) {
                for (const auto &op : op_tokens) {
                    if (s.substr(j, op.size()) == op) {
                        found = j;
                        found_op = op;
                        break;
                    }
                }
            }
            string term = strip(s.substr(i, found - i));
            if (!term.empty()) {
                res.push_back(term);
                if (!is_operand(term)) {
                    throw std::runtime_error("Failed to split term: \"" + term + "\".");
                }
            }
            if (found < next_paren) {
                res.push_back(found_op);
                i = found + static_cast<int>(found_op.size());
            } else {
                i = next_paren;
            }
        }
        if (next_paren < static_cast<int>(s.size())) {
            res.emplace_back(1, s[next_paren]);
        }
    }
    return res;
}

template<typename StrIt>
Operand eval(StrIt lo, StrIt hi) {
    Operand res = eval_binary(lo, hi, 0);

```

```

    if (lo != hi) {
        throw std::runtime_error("Eval failed at token " + *lo + ".");
    }
    return res;
}

Operand eval(const string &s) {
    std::vector<string> tokens = split(s);
    return eval(tokens.begin(), tokens.end());
}
};

```

Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-7;
}

int main() {
    unordered_map<string, UnaryOp> unary_ops;
    unary_ops["+"] = [](double x) { return +x; };
    unary_ops["-"] = [](double x) { return -x; };

    unordered_map<string, BinaryRule> binary_ops;
    binary_ops["+"] = {[](double a, double b) { return a + b; }, 0};
    binary_ops["-"] = {[](double a, double b) { return a - b; }, 0};
    binary_ops["*"] = {[](double a, double b) { return a * b; }, 1};
    binary_ops["/"] = {[](double a, double b) { return a / b; }, 1};
    binary_ops["^"] = {[](double a, double b) { return std::pow(a, b); }, 2};

    RecursiveDescentParser p(unary_ops, binary_ops);
    assert(EQ(p.eval("--+((--(+1)))"), -1));
    assert(EQ(p.eval("5*(3+3)-2-2"), 26));
    assert(EQ(p.eval("1+2*3*4+3*(+2)-100"), -69));
    assert(EQ(p.eval("3*3*3*3*3-2*2*2*2*2*2"), 473));
    assert(EQ(p.eval("3.14 + 3 * (7.7/9.8^32.9 )"), 3.14));
    assert(EQ(p.eval("5*(3+2)/-1*-2+(-2-2-2+3)-3-(-2)+15/2/2/2+(-2)"), 45.875));
    assert(EQ(p.eval("123456789./3/3/3*2*2*2+456/6-23/3"), 36579857.666666666667));
    assert(EQ(p.eval("10/3+10/4+10/5+10/6+10/7+10/8+10/9+10/10+15*23456"), 351854.28968253968));
    assert(
        EQ(p.eval(
            "-(5-(5-(5-(5-(5-2)))))+(3-(3-(3-(3-(3+3)))))*"
            "(7-(7-(7-(7-(7-7+4*5))))))"
        ),
        117
    );
    return 0;
}

```

3.7.3 Shunting Yard Parsing

Evaluate an expression with custom operand types, prefix unary operators, binary operators, and precedences. Evaluation is performed using the shunting yard algorithm, which maintains operator and value stacks while respecting precedence and parentheses. Multiplication by juxtaposition is not supported.

An arbitrary operand type is supported by changing the `Operand` alias and the static `is_operand()` and `eval_operand()` helpers. For maximum reliability, the string representation of operands should not use characters shared by any operator. For instance, the best practice instead of accepting `-1` as a valid operand (since the `-`

sign may conflict with the identical binary operator), is to specify nonnegative numbers as operands alongside the unary operator `-`.

Operators may be non-empty strings of any length, but should not contain any parentheses or shared characters with the string representations of operands. Ideally, operators should not be prefixes or suffixes of one another, else the tokenization process may be ambiguous. For example, if `++` and `+` are both operators, then `++` may be split into either `["+", "+"]` or `["++"]` depending on the lexicographical ordering of conflicting operators.

- `ShuntingYardParser(unary_op, binary_op)` initializes a parser with operators specified by hash tables `unary_op` (of operator to unary function object) and `binary_op` (of operator to pair of binary function object and operator precedence). Operator precedences should be numbered upwards starting at 0 (lowest precedence, evaluated last).
- `split(s)` returns a vector of tokens for the expression `s`, split on the given operators during construction. Each parenthesis, operator, and operand satisfying `is_operand()` will be split into a separate token. The algorithm is naive, matching operators lazily in the case of overlapping operators as mentioned above. Under these circumstances, the parse may not always succeed.
- `eval(lo, hi)` returns the evaluation of a range `[lo, hi)` of already split-up expression tokens, where `lo` and `hi` must be random-access iterators.
- `eval(s)` returns the evaluation of expression `s`, after first calling `split(s)` to obtain the tokens.

Time Complexity:

- $O(m)$ per call to the constructor, where m is the total number of operators.
- $O(n \cdot m \cdot k)$ per call to `split(s)`, where n is the length of `s`, m is the total number of operators defined for the parser instance, and k is the maximum length for any operator representation.
- $O(n)$ expected per call to `eval(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(n \cdot m \cdot k + n)$ expected per call to `eval(s)`, where n is the length of `s`, and m and k are as defined previously.

Space Complexity:

- $O(m \cdot k)$ for storage of the m operators, of maximum length k .
- $O(n)$ auxiliary stack space for `split(s)`, `eval(lo, hi)`, and `eval(s)`, where n is the length of the argument.

```
#include <algorithm>
#include <cctype>
#include <functional>
#include <set>
#include <sstream>
#include <stack>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>
using std::string;

// Define the custom operand type and representation below.
using Operand = double;
using UnaryOp = std::function<Operand(Operand)>;
using BinaryOp = std::function<Operand(Operand, Operand)>;

struct BinaryRule {
    BinaryOp op;
    int precedence;
};

class ShuntingYardParser {
    using UnaryOpMap = std::unordered_map<string, UnaryOp>;
    using BinaryOpMap = std::unordered_map<string, BinaryRule>;
    UnaryOpMap unary_ops;
    BinaryOpMap binary_ops;
    std::set<string> op_tokens;
```



```

static Operand eval_operand(const string &s) {
    Operand res;
    std::stringstream ss(s);
    ss >> res;
    return res;
}

static bool is_operand(const string &s) {
    int npoints = 0;
    for (char c : s) {
        if (c == '.') {
            if (++npoints > 1) {
                return false;
            }
        } else if (!isdigit(static_cast<unsigned char>(c))) {
            return false;
        }
    }
    return !s.empty();
}

static string strip(string s) {
    auto not_space = [](unsigned char c) { return !std::isspace(c); };
    s.erase(s.begin(), std::find_if(s.begin(), s.end(), not_space));
    s.erase(std::find_if(s.rbegin(), s.rend(), not_space).base(), s.end());
    return s;
}

public:
ShuntingYardParser(const UnaryOpMap &unary_ops, const BinaryOpMap &binary_ops)
    : unary_ops(unary_ops), binary_ops(binary_ops) {
    for (const auto &[op, fn] : unary_ops) {
        op_tokens.insert(op);
    }
    for (const auto &[op, fn_prec] : binary_ops) {
        op_tokens.insert(op);
    }
}

std::vector<string> split(const string &s) {
    std::vector<string> res;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        if (s[i] == ' ') {
            continue;
        }
        int next_paren = static_cast<int>(s.size());
        for (int j = i; j < static_cast<int>(s.size()); j++) {
            if (s[j] == '(' || s[j] == ')') {
                next_paren = j;
                break;
            }
        }
        while (i < next_paren) {
            int found = next_paren;
            string found_op;
            for (int j = i; j < next_paren && found == next_paren; j++) {
                for (const auto &op : op_tokens) {
                    if (s.substr(j, op.size()) == op) {
                        found = j;
                        found_op = op;
                        break;
                    }
                }
            }
            string term = strip(s.substr(i, found - i));
            if (!term.empty()) {
                res.push_back(term);
                if (!is_operand(term)) {

```

```

        throw std::runtime_error("Failed to split term: \"" + term + "\".");
    }
}
if (found < next_paren) {
    res.push_back(found_op);
    i = found + static_cast<int>(found_op.size());
} else {
    i = next_paren;
}
}
if (next_paren < static_cast<int>(s.size())) {
    res.emplace_back(1, s[next_paren]);
}
}
return res;
}

template<typename StrIt>
Operand eval(StrIt lo, StrIt hi) {
    std::stack<Operand> vals;
    std::stack<std::pair<string, bool>> op_stack;
    op_stack.emplace("(", false);
    StrIt prev = hi;
    do {
        string curr = (lo == hi) ? ")" : *lo;
        if (is_operand(curr)) {
            vals.push(eval_operand(curr));
        } else if (curr == "(") {
            op_stack.emplace(curr, false);
        } else if (
            unary_ops.find(curr) != unary_ops.end() &&
            (prev == hi || *prev == "(" || binary_ops.find(*prev) != binary_ops.end())
        ) {
            op_stack.emplace(curr, true);
        } else {
            while (true) {
                auto [op, is_unary] = op_stack.top();
                auto it1 = binary_ops.find(op);
                auto it2 = binary_ops.find(curr);
                if (!is_unary && (it1 == binary_ops.end() ? -1 : it1->second.precedence) <
                    (it2 == binary_ops.end() ? -1 : it2->second.precedence)) {
                    break;
                }
                op_stack.pop();
                if (op == "(") {
                    break;
                }
                Operand b = vals.top();
                vals.pop();
                if (is_unary) {
                    if (auto it = unary_ops.find(op); it != unary_ops.end()) {
                        vals.push((it->second)(b));
                    } else {
                        throw std::runtime_error("Failed to eval unary op: " + op);
                    }
                } else {
                    Operand a = vals.top();
                    vals.pop();
                    if (it1 == binary_ops.end()) {
                        throw std::runtime_error("Failed to eval binary op: " + op);
                    }
                    vals.push((it1->second.op)(a, b));
                }
            }
        }
        if (curr != ")") {
            op_stack.emplace(*lo, false);
        }
    }
}

```

```

    prev = lo;
  } while (lo++ != hi);
  return vals.top();
}

Operand eval(const string &s) {
  std::vector<string> tokens = split(s);
  return eval(tokens.begin(), tokens.end());
}
};

```

Example Usage

```

#include <cassert>
#include <cmath>
using namespace std;

bool EQ(double a, double b) {
  return fabs(a - b) < 1e-7;
}

int main() {
  unordered_map<string, UnaryOp> unary_ops;
  unary_ops["+"] = [](double x) { return +x; };
  unary_ops["-"] = [](double x) { return -x; };

  unordered_map<string, BinaryRule> binary_ops;
  binary_ops["+"] = {[](double a, double b) { return a + b; }, 0};
  binary_ops["-"] = {[](double a, double b) { return a - b; }, 0};
  binary_ops["*"] = {[](double a, double b) { return a * b; }, 1};
  binary_ops["/"] = {[](double a, double b) { return a / b; }, 1};
  binary_ops["^"] = {[](double a, double b) { return std::pow(a, b); }, 2};

  ShuntingYardParser p(unary_ops, binary_ops);
  assert(EQ(p.eval("-+-(--(+1))"), -1));
  assert(EQ(p.eval("5*(3+3)-2-2"), 26));
  assert(EQ(p.eval("1+2*3*4+3*(+2)-100"), -69));
  assert(EQ(p.eval("3*3*3*3*3-2*2*2*2*2*2"), 473));
  assert(EQ(p.eval("3.14 + 3 * (7.7/9.8^32.9 )"), 3.14));
  assert(EQ(p.eval("5*(3+2)/-1*-2+(-2-2-2+3)-3-(-2)+15/2/2/2+(-2)"), 45.875));
  assert(EQ(p.eval("123456789./3/3/3*2*2*2+456/6-23/3"), 36579857.666666666));
  assert(EQ(p.eval("10/3+10/4+10/5+10/6+10/7+10/8+10/9+10/10+15*23456"), 351854.28968253968));
  assert(
    EQ(p.eval(
      "-(5-(5-(5-(5-(5-2)))))+(3-(3-(3-(3-(3+3)))))*"
      "(7-(7-(7-(7-(7-7+4*5)))))"
    ),
    117
  );
  return 0;
}

```

3.8 Encoding and Compression

3.8.1 Entropy and Frequency Tables

Computes byte frequency tables and empirical information-theoretic quantities for strings. These primitives are useful when analyzing compression, building coding schemes such as Huffman coding, or comparing symbol distributions.

For a string with symbol probabilities p_i , empirical entropy is $H = -\sum_i p_i \log_2 p_i$ bits per symbol. This is a lower bound on the average number of bits per symbol achievable by any uniquely decodable code for the same independent symbol model.

- `byte_frequencies(s)` returns a length-256 table of byte counts in string `s`.
- `entropy(freq)` returns empirical entropy in bits per symbol for a frequency table.
- `entropy(s)` returns empirical entropy in bits per symbol for string `s`.
- `expected_code_length(freq, length)` returns the average encoded bits per symbol for code lengths `length[c]`.

Time Complexity:

- $O(n + m)$ per call to `entropy(s)`, where n is the string length and $m = 256$.
- $O(m)$ per call to `entropy(freq)` and `expected_code_length(freq, length)`.

Space Complexity:

- $O(m)$ auxiliary heap space for `byte_frequencies(s)`.
- $O(1)$ auxiliary for the other operations.

```
#include <cmath>
#include <string>
#include <vector>
using std::string;

std::vector<int> byte_frequencies(const string &s) {
    std::vector<int> freq(256, 0);
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        freq[static_cast<unsigned char>(s[i])]++;
    }
    return freq;
}

double entropy(const std::vector<int> &freq) {
    int total = 0;
    for (int i = 0; i < static_cast<int>(freq.size()); i++) {
        total += freq[i];
    }
    if (total == 0) {
        return 0;
    }
    double res = 0;
    for (int i = 0; i < static_cast<int>(freq.size()); i++) {
        if (freq[i] == 0) {
            continue;
        }
        double p = static_cast<double>(freq[i]) / total;
        res -= p * (log(p) / log(2.0));
    }
    return res;
}

double entropy(const string &s) {
    return entropy(byte_frequencies(s));
}

double expected_code_length(const std::vector<int> &freq, const std::vector<int> &length) {
    int total = 0;
    for (int i = 0; i < static_cast<int>(freq.size()); i++) {
        total += freq[i];
    }
    if (total == 0) {
        return 0;
    }
    double res = 0;
    for (int i = 0; i < static_cast<int>(freq.size()); i++) {
        res += static_cast<double>(freq[i]) * length[i] / total;
    }
}
```

```
    return res;
}
```

Example Usage

```
#include <cassert>
#include <cmath>
using namespace std;

bool close(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    vector<int> freq = byte_frequencies("aabb");
    assert(freq['a'] == 2);
    assert(freq['b'] == 2);
    assert(close(entropy(freq), 1.0));
    assert(close(entropy("aaaa"), 0.0));

    vector<int> length(256, 0);
    length['a'] = 1;
    length['b'] = 1;
    assert(close(expected_code_length(freq, length), 1.0));
    return 0;
}
```

3.8.2 Run-Length Encoding

Encodes consecutive equal characters as (character, count) runs. Run-length encoding is a simple lossless compression technique that works well when strings contain long repeated blocks and poorly when repetitions are rare.

The representation below stores runs in a vector of pairs rather than formatting them as a string. This avoids ambiguity when the original text contains digits or separator characters.

- `run_length_encode(s)` returns the sequence of runs in string `s`.
- `run_length_decode(runs)` reconstructs the original string from a sequence of runs.

Time Complexity: $O(n)$ per call, where n is the input or output length.

Space Complexity:

- $O(r)$ for encoded output, where r is the number of runs.
- $O(n)$ for decoded output.

```
#include <string>
#include <utility>
#include <vector>
using std::string;

std::vector<std::pair<char, int>> run_length_encode(const string &s) {
    std::vector<std::pair<char, int>> res;
    for (int i = 0; i < static_cast<int>(s.size()); i++) {
        if (res.empty() || res.back().first != s[i]) {
            res.emplace_back(s[i], 1);
        } else {
            res.back().second++;
        }
    }
    return res;
}
```

```
string run_length_decode(const std::vector<std::pair<char, int>> &runs) {
    string res;
    for (int i = 0; i < static_cast<int>(runs.size()); i++) {
        res.append(runs[i].second, runs[i].first);
    }
    return res;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    string s = "aaabcccd";
    vector<pair<char, int>> runs = run_length_encode(s);
    assert(runs.size() == 4);
    assert(runs[0] == (pair<char, int>{'a', 3}));
    assert(runs[2] == (pair<char, int>{'c', 4}));
    assert(run_length_decode(runs) == s);
    return 0;
}
```

3.8.3 Base64 Encoding

Encodes binary data as printable ASCII using Base64, and decodes Base64 text back to bytes. Base64 is an encoding, not encryption or compression: it preserves all input data exactly, but usually increases size by about one third. Each group of three input bytes is split into four 6-bit values, each selecting one character of a 64-symbol alphabet, with `=` padding marking a final group of only one or two bytes.

This implementation uses the standard alphabet with `+`, `/`, and `=` padding. Whitespace is not skipped by the decoder; clean the input first if line breaks or spaces may appear.

- `base64_encode(bytes)` returns the Base64 representation of string `bytes`.
- `base64_decode(text)` returns the decoded byte string. It assumes `text` is valid padded Base64.

Time Complexity: $O(n)$ per call, where n is the input length.

Space Complexity: $O(n)$ for the output string.

```
#include <string>
#include <vector>
using std::string;

const string BASE64_ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=";

string base64_encode(const string &bytes) {
    string res;
    int val = 0, bits = -6;
    for (int i = 0; i < static_cast<int>(bytes.size()); i++) {
        val = (val << 8) + static_cast<unsigned char>(bytes[i]);
        bits += 8;
        while (bits >= 0) {
            res.push_back(BASE64_ALPHABET[(val >> bits) & 63]);
            bits -= 6;
        }
    }
    if (bits > -6) {
        res.push_back(BASE64_ALPHABET[((val << 8) >> (bits + 8)) & 63]);
    }
    while (res.size() % 4) {
```

```

        res.push_back('=');
    }
    return res;
}

string base64_decode(const string &text) {
    std::vector<int> table(256, -1);
    for (int i = 0; i < 64; i++) {
        table[static_cast<unsigned char>(BASE64_ALPHABET[i])] = i;
    }
    string res;
    int val = 0, bits = -8;
    for (int i = 0; i < static_cast<int>(text.size()); i++) {
        if (text[i] == '=') {
            break;
        }
        int x = table[static_cast<unsigned char>(text[i])];
        if (x < 0) {
            break;
        }
        val = (val << 6) + x;
        bits += 6;
        if (bits >= 0) {
            res.push_back(static_cast<char>(((val >> bits) & 255)));
            bits -= 8;
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(base64_encode("hello") == "aGVsbG8=");
    assert(base64_decode("aGVsbG8=") == "hello");
    assert(base64_decode(base64_encode("abc123!?")) == "abc123!?");
    return 0;
}

```

3.8.4 Huffman Coding

Builds a Huffman code for a string and uses it to encode and decode that string. Huffman coding is a lossless prefix-code compression algorithm: more frequent characters receive shorter bit strings, and no code is a prefix of another code. The code tree is built greedily with a priority queue by repeatedly merging the two lowest-frequency subtrees until one tree remains; each character's bit string is then read off its root-to-leaf path.

The implementation below stores encoded bits as a string of `'0'` and `'1'` characters for clarity. For real compression, pack those bits into bytes. The tree is also needed to decode the bit string, so compressed data normally stores enough metadata to reconstruct the same tree.

- `HuffmanTree(text)` constructs a Huffman tree from the character frequencies in `text`.
- `codes()` returns a table mapping each byte value to its bit string.
- `encode(text)` returns the encoded bit string.
- `decode(bits)` returns the decoded text.

Time Complexity:

- $O(n + m \log m)$ to construct the tree, where n is the input length and m is the number of distinct characters.
- $O(n)$ to encode or decode, plus the number of produced or consumed bits.

Space Complexity:

- $O(m)$ for the tree and code table, where m is the number of distinct characters.
- $O(n)$ for the encoded or decoded output.

```

#include <queue>
#include <string>
#include <vector>
using std::string;

class HuffmanTree {
    struct Node {
        int freq;
        unsigned char ch;
        int left, right;

        Node(int freq = 0, unsigned char ch = 0, int left = -1, int right = -1)
            : freq(freq), ch(ch), left(left), right(right) {}
    };

    struct ByFrequency {
        const std::vector<Node> *nodes;
        explicit ByFrequency(const std::vector<Node> *nodes = nullptr) : nodes(nodes) {}
        bool operator()(int a, int b) const { return (*nodes)[a].freq > (*nodes)[b].freq; }
    };

    std::vector<Node> nodes;
    int root;
    std::vector<string> code;

    void build_codes(int u, const string &path) {
        if (nodes[u].left == -1 && nodes[u].right == -1) {
            code[nodes[u].ch] = path.empty() ? "0" : path;
            return;
        }
        build_codes(nodes[u].left, path + '0');
        build_codes(nodes[u].right, path + '1');
    }

public:
    explicit HuffmanTree(const string &text) : root(-1), code(256) {
        std::vector<int> freq(256, 0);
        for (int i = 0; i < static_cast<int>(text.size()); i++) {
            freq[static_cast<unsigned char>(text[i])]++;
        }
        ByFrequency cmp(&nodes);
        std::priority_queue<int, std::vector<int>, ByFrequency> pq(cmp);
        for (int c = 0; c < 256; c++) {
            if (freq[c] > 0) {
                nodes.emplace_back(freq[c], static_cast<unsigned char>(c));
                pq.push(static_cast<int>(nodes.size() - 1));
            }
        }
        if (pq.empty()) {
            return;
        }
        while (static_cast<int>(pq.size()) > 1) {
            int a = pq.top();
            pq.pop();
            int b = pq.top();
            pq.pop();
            nodes.emplace_back(nodes[a].freq + nodes[b].freq, 0, a, b);
            pq.push(static_cast<int>(nodes.size() - 1));
        }
        root = pq.top();
        build_codes(root, "");
    }
}

```



```

std::vector<string> codes() const { return code; }

string encode(const string &text) const {
    string bits;
    for (int i = 0; i < static_cast<int>(text.size()); i++) {
        bits += code[static_cast<unsigned char>(text[i])];
    }
    return bits;
}

string decode(const string &bits) const {
    string text;
    if (root == -1) {
        return text;
    }
    // Degenerate tree: the input had a single distinct character, so the root is itself a leaf and
    // each symbol was encoded as one bit. Walking children would dereference node -1.
    if (nodes[root].left == -1 && nodes[root].right == -1) {
        return string(bits.size(), static_cast<char>(nodes[root].ch));
    }
    int u = root;
    for (int i = 0; i < static_cast<int>(bits.size()); i++) {
        u = bits[i] == '0' ? nodes[u].left : nodes[u].right;
        if (nodes[u].left == -1 && nodes[u].right == -1) {
            text.push_back(static_cast<char>(nodes[u].ch));
            u = root;
        }
    }
    return text;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    string text = "banana bandana";
    HuffmanTree h(text);
    string bits = h.encode(text);
    assert(h.decode(bits) == text);
    assert(bits.size() < text.size() * 8);
    return 0;
}

```

3.8.5 Burrows-Wheeler Transform

The Burrows-Wheeler transform (BWT) permutes a string into one that tends to be far more compressible while remaining perfectly invertible. It writes down every cyclic rotation of the string, sorts them lexicographically, and reads off the last character of each sorted rotation. Characters that occur in similar contexts are brought together, producing long runs that run-length and move-to-front coders exploit; it is the heart of the bzip2 compressor and a building block of compressed-text indexes.

To make the permutation invertible, a unique sentinel character smaller than every real character is appended before transforming, so the rotations are all distinct and one of them is distinguished as the original. Sorting the rotations of the sentineled string is the same as sorting its suffixes, so the forward transform is computed with a prefix-doubling rotation sort. The inverse rebuilds the string with the last-to-first (LF) mapping: stably ordering the transformed characters yields, for each position of the sorted first column, the row that supplies the next character, and following that mapping from the sentinel row walks the original string out one character at a time.

The input must not contain the sentinel character `'\0'`.

- `bwt(s)` returns the BWT of `s`. The result has length `s.size() + 1` and contains the sentinel `'\0'` exactly once.
- `inverse_bwt(code)` returns the original string, inverting `bwt`. The argument must be a string produced by that function.

Time Complexity:

- $O(n \log^2 n)$ per call to `bwt(s)`, where n is the length of `s`.
- $O(n \log n)$ per call to `inverse_bwt(code)`.

Space Complexity: $O(n)$ auxiliary heap space for both functions.

```
#include <algorithm>
#include <numeric>
#include <string>
#include <vector>

std::string bwt(std::string s) {
    s.push_back('\0'); // Unique sentinel, smaller than every real character.
    int n = static_cast<int>(s.size());
    // Sort the indices of the cyclic rotations by prefix doubling; with the sentinel this matches
    // sorting the suffixes.
    std::vector<int> order(n), rank(n), tmp(n);
    for (int i = 0; i < n; i++) {
        order[i] = i;
        rank[i] = static_cast<unsigned char>(s[i]);
    }
    for (int k = 1; k < n; k <= 1) {
        auto cmp = [&](int a, int b) {
            if (rank[a] != rank[b]) {
                return rank[a] < rank[b];
            }
            return rank[(a + k) % n] < rank[(b + k) % n];
        };
        std::sort(order.begin(), order.end(), cmp);
        tmp[order[0]] = 0;
        for (int i = 1; i < n; i++) {
            tmp[order[i]] = tmp[order[i - 1]] + (cmp(order[i - 1], order[i]) ? 1 : 0);
        }
        rank = tmp;
        if (rank[order[n - 1]] == n - 1) {
            break;
        }
    }
    std::string code(n, '\0');
    for (int i = 0; i < n; i++) {
        code[i] = s[(order[i] + n - 1) % n]; // Last character of the i-th sorted rotation.
    }
    return code;
}

std::string inverse_bwt(const std::string &code) {
    int n = static_cast<int>(code.size());
    // first[k] is the row supplying the first column's k-th character; following it from row 0 (the
    // sentinel row, sorted first) reconstructs the sentinel followed by the original string.
    std::vector<int> first(n);
    std::iota(first.begin(), first.end(), 0);
    std::stable_sort(first.begin(), first.end(), [&](int a, int b) {
        return static_cast<unsigned char>(code[a]) < static_cast<unsigned char>(code[b]);
    });
    std::string res;
    res.reserve(n);
    for (int i = 0, row = 0; i < n; i++) {
        row = first[row];
        res.push_back(code[row]);
    }
    return res.substr(1); // Drop the leading sentinel.
}
```

```
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    string code = bwt("banana");
    assert(code.size() == 7); // "banana" plus the sentinel.
    assert(inverse_bwt(code) == "banana");

    // The transform clusters equal characters: "mississippi" yields long runs.
    string text = "mississippi";
    string t = bwt(text);
    assert(inverse_bwt(t) == text);

    // Round-trips on assorted strings, including repeats and a single character.
    for (const string &s :
        {string(""), string("a"), string("aaaa"), string("abracadabra"),
         string("the quick brown fox"), string("\1\2abc", 5)}) {
        assert(inverse_bwt(bwt(s)) == s);
    }
    return 0;
}
```

Chapter 4

Graphs

4.1 DFS and Tree Algorithms

4.1.1 Graph Class and DFS

A graph consists of a set of objects (a.k.a. vertices, or nodes) and a set of connections (a.k.a. edges) between pairs of said objects. A graph may be stored as an adjacency list, which is a space efficient representation that is also time-efficient for traversals.

The following class implements a simple graph using adjacency lists, along with depth-first search and a few other applications. The constructor takes a flag which specifies whether the instance is a directed or undirected graph. The nodes of the graph are identified by integer indices numbered consecutively starting from 0. The total number of nodes automatically increases based on the maximum node index passed to `add_edge()` so far.

- `Graph(directed)` constructs an empty graph, directed if `directed` is true (the default) and undirected otherwise.
- `nodes()` returns the current number of nodes.
- `operator[n]` returns a reference to the adjacency list (a `std::vector<int>`) of node `n`.
- `add_edge(u, v)` adds an edge from `u` to `v`, plus the reverse edge if the graph is undirected, growing the node count to accommodate the larger index if necessary.
- `dfs(start, f)` runs a depth-first search from node `start`, calling `f(n)` on each node `n` in the order it is first visited.
- `has_cycle()` returns whether the graph contains a cycle.
- `is_directed()` returns whether the graph is directed.
- `is_forest()` returns whether the graph is undirected and acyclic.
- `is_dag()` returns whether the graph is directed and acyclic.

For undirected graphs, `has_cycle()` assumes a simple graph; parallel edges are not distinguished from the tree edge back to the parent.

Time Complexity:

- $O(1)$ amortized per call to `add_edge()`, or $O(\max(n, m))$ for n calls where the maximum node index passed as an argument is m .
- $O(\max(n, m))$ per call for `dfs()`, `has_cycle()`, `is_forest()`, or `is_dag()`, where n is the number of nodes and m is the number of edges.
- $O(1)$ per call to all other public member functions.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `dfs()`, `has_cycle()`, `is_forest()`, and `is_dag()`.

- $O(1)$ auxiliary for all other public member functions.

```

#include <algorithm>
#include <vector>

class Graph {
    std::vector<std::vector<int>>> adj;
    bool directed;

    template<typename Fn>
    void dfs(int n, std::vector<bool> &visit, Fn f) const {
        f(n);
        visit[n] = true;
        for (int v : adj[n]) {
            if (!visit[v]) {
                dfs(v, visit, f);
            }
        }
    }

    bool has_cycle(int n, int prev, std::vector<bool> &visit, std::vector<bool> &onstack) const {
        visit[n] = true;
        onstack[n] = true;
        for (int v : adj[n]) {
            if (directed && onstack[v]) {
                return true;
            }
            if (!directed && visit[v] && v != prev) {
                return true;
            }
            if (!visit[v] && has_cycle(v, n, visit, onstack)) {
                return true;
            }
        }
        onstack[n] = false;
        return false;
    }

public:
    Graph(bool directed = true) : directed(directed) {}

    int nodes() const { return static_cast<int>(adj.size()); }
    std::vector<int> &operator[](int n) { return adj[n]; }

    void add_edge(int u, int v) {
        int n = static_cast<int>(adj.size());
        if (u >= n || v >= n) {
            adj.resize(std::max(u, v) + 1);
        }
        adj[u].push_back(v);
        if (!directed) {
            adj[v].push_back(u);
        }
    }

    bool has_cycle() const {
        int n = static_cast<int>(adj.size());
        std::vector<bool> visit(n, false), onstack(n, false);
        for (int i = 0; i < n; i++) {
            if (!visit[i] && has_cycle(i, -1, visit, onstack)) {
                return true;
            }
        }
        return false;
    }

    bool is_directed() const { return directed; }
    bool is_forest() const { return !directed && !has_cycle(); }

```

```

bool is_dag() const { return directed && !has_cycle(); }

template<typename Fn>
void dfs(int start, Fn f) const {
    std::vector<bool> visit(adj.size(), false);
    dfs(start, visit, f);
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    {
        Graph g;
        g.add_edge(0, 1);
        g.add_edge(0, 6);
        g.add_edge(0, 7);
        g.add_edge(1, 2);
        g.add_edge(1, 5);
        g.add_edge(2, 3);
        g.add_edge(2, 4);
        g.add_edge(7, 8);
        g.add_edge(7, 11);
        g.add_edge(8, 9);
        g.add_edge(8, 10);
        cout << "DFS order: ";
        g.dfs(0, [](int n) { cout << n << " "; });
        cout << endl;
        assert(g[0].size() == 3);
        assert(g.is_dag());
        assert(!g.has_cycle());
    }
    {
        Graph tree(false);
        tree.add_edge(0, 1);
        tree.add_edge(0, 2);
        tree.add_edge(1, 3);
        tree.add_edge(1, 4);
        assert(tree.is_forest());
        assert(!tree.is_dag());
        tree.add_edge(2, 3);
        assert(!tree.is_forest());
    }
    return 0;
}

```

Example Output

```
DFS order: 0 1 2 3 4 5 6 7 8 9 10 11
```

4.1.2 Topological Sort

Given a directed acyclic graph, find one of possibly many orderings of the nodes such that for every edge from node u to v , u comes before v in the ordering.

- `toposort_dfs()` returns a valid topological ordering using DFS post-order, or throws if the graph contains a cycle.
- `toposort_kahn()` returns a valid topological ordering using indegrees and a queue, or returns an empty vector if the graph contains a cycle.

Time Complexity: $O(\max(n, m))$ per call to either function, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `toposort_dfs()`.
- $O(n)$ auxiliary indegree and queue space for `toposort_kahn()`.

```
#include <algorithm>
#include <cassert>
#include <queue>
#include <stdexcept>
#include <vector>

std::vector<std::vector<int>> adj;

void dfs(int u, std::vector<bool> &visit, std::vector<bool> &done, std::vector<int> &res) {
    if (visit[u]) {
        throw std::runtime_error("Not a directed acyclic graph.");
    }
    if (done[u]) {
        return;
    }
    visit[u] = true;
    for (int v : adj[u]) {
        dfs(v, visit, done, res);
    }
    visit[u] = false;
    done[u] = true;
    res.push_back(u);
}

std::vector<int> toposort_dfs() {
    int n = static_cast<int>(adj.size());
    std::vector<bool> visit(n, false), done(n, false);
    std::vector<int> res;
    for (int i = 0; i < n; i++) {
        if (!done[i]) {
            dfs(i, visit, done, res);
        }
    }
    std::reverse(res.begin(), res.end());
    return res;
}

std::vector<int> toposort_kahn() {
    int n = static_cast<int>(adj.size());
    std::vector<int> indegree(n), res;
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            indegree[v]++;
        }
    }
    std::queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        res.push_back(u);
        for (int v : adj[u]) {
            if (--indegree[v] == 0) {
                q.push(v);
            }
        }
    }
}
```

```

    }
}
}
return static_cast<int>(res.size()) == n ? res : std::vector<int>();
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    adj.assign(8, {});
    adj[0].push_back(3);
    adj[0].push_back(4);
    adj[1].push_back(3);
    adj[2].push_back(4);
    adj[2].push_back(7);
    adj[3].push_back(5);
    adj[3].push_back(6);
    adj[3].push_back(7);
    adj[4].push_back(6);
    vector<int> dfs_res = toposort_dfs();
    cout << "The topological order:";
    for (int v : dfs_res) {
        cout << " " << v;
    }
    cout << endl;
    vector<int> kahn = toposort_kahn();
    vector<int> position(kahn.size());
    for (int i = 0; i < static_cast<int>(kahn.size()); i++) {
        position[kahn[i]] = i;
    }
    for (int u = 0; u < static_cast<int>(adj.size()); u++) {
        for (int v : adj[u]) {
            assert(position[u] < position[v]);
        }
    }
    return 0;
}

```

Example Output

The topological order: 2 1 0 4 3 7 6 5

4.1.3 Tree Centers and Diameter

An unweighted tree possesses a center, centroid, and diameter. The centers are found by repeatedly peeling away the leaves of the tree until one or two nodes remain. The diameter relies on the fact that the farthest node from any start is always one endpoint of a diameter, so a second traversal from that endpoint measures the full diameter. The following functions apply to a global, bidirectionally pre-populated adjacency list `adj` which must form a valid tree with nodes numbered $[0, n)$, where n is `adj.size()`.

- `find_centers()` returns a vector of either one or two tree Jordan centers. The Jordan center of a tree is the set of all nodes with minimum eccentricity, that is, the set of all nodes where the maximum distance to all other nodes in the tree is minimal.
- `find_centroid()` returns the node where all of its subtrees have a size less than or equal to $n/2$, where n is the number of nodes in the tree.
- `diameter()` returns the maximum distance between any two nodes in the tree, using a well-known double depth-first search technique.

Time Complexity: $O(\max(n, m))$ per call to `find_centers()`, `find_centroid()`, and `diameter()`, where n is the number of nodes and m is the number of edges.

Space Complexity: $O(n)$ auxiliary stack space for `find_centers()`, `find_centroid()`, and `diameter()`, where n is the number of nodes.

```
#include <algorithm>
#include <utility>
#include <vector>

std::vector<std::vector<int>> adj;

std::vector<int> find_centers() {
    int n = static_cast<int>(adj.size());
    std::vector<int> leaves, degree(n);
    for (int i = 0; i < n; i++) {
        degree[i] = static_cast<int>(adj[i].size());
        if (degree[i] <= 1) {
            leaves.push_back(i);
        }
    }
    int removed = static_cast<int>(leaves.size());
    while (removed < n) {
        std::vector<int> nleaves;
        for (int u : leaves) {
            for (int v : adj[u]) {
                if (--degree[v] == 1) {
                    nleaves.push_back(v);
                }
            }
        }
        leaves = nleaves;
        removed += static_cast<int>(leaves.size());
    }
    return leaves;
}

// Returns the centroid node index if found in this subtree, or -(subtree size) to propagate
// the size up to the parent so it can check the complementary component's size.
int find_centroid(int u = 0, int p = -1) {
    int n = static_cast<int>(adj.size());
    int count = 1;
    bool good_center = true;
    for (int v : adj[u]) {
        if (v == p) {
            continue;
        }
        int res = find_centroid(v, u);
        if (res >= 0) {
            return res;
        }
        int size = -res;
        good_center &= (size <= n / 2);
        count += size;
    }
    good_center &= (n - count <= n / 2);
    return good_center ? u : -count;
}

std::pair<int, int> dfs(int u, int p, int depth) {
    std::pair<int, int> res{depth, u};
    for (int v : adj[u]) {
        if (v != p) {
            res = std::max(res, dfs(v, u, depth + 1));
        }
    }
    return res;
}
```

```

}

int diameter() {
    int furthest_node = dfs(0, -1, 0).second;
    return dfs(furthest_node, -1, 0).first;
}

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    adj.assign(6, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(1, 4);
    add_edge(3, 4);
    add_edge(4, 5);
    vector<int> centers = find_centers();
    assert(centers.size() == 2 && centers[0] == 1 && centers[1] == 4);
    assert(find_centroid() == 4);
    assert(diameter() == 3);
    return 0;
}

```

4.1.4 Tree Rerooting DP

Given a tree DP that can answer one rooted query with a recursive `dfs(u, p)`, compute that same answer for every possible root. Rerooting DP first stores parent pointers, depths, preorder, subtree sizes, and entry/exit times. A bottom-up pass computes the usual child-to-parent DFS values, then a top-down pass uses prefix/suffix aggregation to compute the missing parent-to-child values. The prefix/suffix aggregation requires `combine` to be associative for a running time of $O(n)$, which is usually preferred, but a segment-tree/exclusive-combine variant can support a left-fold style interface in $O(n \log n)$.

- `RerootingTree(n)` constructs a tree with `n` nodes numbered `[0, n)`.
- `add_edge(u, v)` adds an undirected edge between nodes `u` and `v`.
- `build_rooted_tree(root)` populates `parent`, `depth`, `subtree_size`, `order`, `tin`, and `tout` for the tree rooted at `root` (default 0). This must be called only after the added edges define a connected tree of size `n`.
- `is_ancestor(u, v)` returns whether `u` is an ancestor of `v` in the last rooted tree.
- `rerooting_dp(identity, combine, finalize, lift)` returns the value that `dfs(root, -1)` would produce for every possible `root`. This must be called only after the added edges define a connected tree of size `n`.

The aggregation scheme for `rerooting_dp()` is defined by the following pieces:

- `Data` is the return type of a normal fixed-root DFS call `dfs(u, p)`. It summarizes the component containing `u` after removing edge `u-p`, with all values measured from node `u`.
- `identity` is the neutral value for an empty component, which for all aggregate values `a` must satisfy `combine(a, identity) = combine(identity, a) = a`.
- `combine(a, b)` merges two independent neighbor contributions into the same node `u`, like two different child DFS returns.
- `finalize(acc, u)` finalizes node `u` after all neighbor contributions have been merged. This is the last line of a normal `dfs(u, p)` before returning.

- `lift(acc, u, p)` converts a finished `dfs(u, p)` value into the contribution that node `u` gives to adjacent node `p`. Here `p` is the neighbor treated as the parent/excluded side of the directed DFS state, not necessarily `u`'s parent in the initial root-0 traversal.

This covers problems such as subtree sizes, sum/max of distances from every root, and rerooted polynomial/hash-like aggregates, as long as sibling contributions can be combined associatively. The usage example shows how to compute the sum of distances from every root: `Data` stores `sum`, the total distance from `u` to all nodes in `dfs(u, p)`, and `nodes`, the node count in that component. The node count is needed because `lift(acc, u, p)` crosses one edge, so every represented node becomes 1 farther from `p` and `sum` increases by `nodes`.

Time Complexity:

- $O(1)$ per call to `add_edge()`.
- $O(n)$ per call to `build_rooted_tree()`, where n is the number of nodes.
- $O(1)$ per call to `is_ancestor()`.
- $O(n)$ per call to `rerooting_dp()`, assuming $O(1)$ callback functions.

Space Complexity:

- $O(n)$ for the tree, metadata, and DP arrays.
- $O(n)$ auxiliary stack space for the recursive search.

```
#include <stdint>
#include <vector>

class RerootingTree {
    int timer;

    void dfs_rooted(int u, int p) {
        parent[u] = p;
        tin[u] = timer++;
        order.push_back(u);
        subtree_size[u] = 1;
        for (int v : adj[u]) {
            if (v == p) {
                continue;
            }
            depth[v] = depth[u] + 1;
            dfs_rooted(v, u);
            subtree_size[u] += subtree_size[v];
        }
        tout[u] = timer;
    }

public:
    std::vector<std::vector<int>>> adj;
    std::vector<int> parent, depth, subtree_size, order, tin, tout;

    explicit RerootingTree(int n) : timer(0), adj(n) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void build_rooted_tree(int root = 0) {
        int n = static_cast<int>(adj.size());
        parent.assign(n, -1);
        depth.assign(n, 0);
        subtree_size.assign(n, 0);
        order.clear();
        tin.assign(n, 0);
        tout.assign(n, 0);
        timer = 0;
        dfs_rooted(root, -1);
    }
}
```

```

bool is_ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

template<typename Data, typename Combine, typename Finalize, typename Lift>
std::vector<Data> rerooting_dp(
    const Data &identity, Combine combine, Finalize finalize, Lift lift
) {
    int n = static_cast<int>(adj.size());
    std::vector<Data> down(n, identity), outside(n, identity), answer(n, identity);
    build_rooted_tree();
    for (int i = n - 1; i >= 0; i--) {
        int u = order[i];
        Data acc = identity;
        for (int v : adj[u]) {
            if (parent[v] == u) {
                acc = combine(acc, lift(down[v], v, u));
            }
        }
        down[u] = finalize(acc, u);
    }
    for (int u : order) {
        std::vector<int> children;
        std::vector<Data> vals;
        for (int v : adj[u]) {
            if (parent[v] == u) {
                children.push_back(v);
                vals.push_back(lift(down[v], v, u));
            }
        }
        std::vector<Data> suffix(vals.size() + 1, identity);
        for (int i = static_cast<int>(vals.size()) - 1; i >= 0; i--) {
            suffix[i] = combine(vals[i], suffix[i + 1]);
        }
        Data prefix = outside[u];
        for (int i = 0; i < static_cast<int>(children.size()); i++) {
            Data without_child = finalize(combine(prefix, suffix[i + 1]), u);
            outside[children[i]] = lift(without_child, u, children[i]);
            prefix = combine(prefix, vals[i]);
        }
        answer[u] = finalize(prefix, u);
    }
    return answer;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

vector<int64_t> sum_distances_all_roots(RerootingTree &tree) {
    struct DistanceData {
        int64_t sum = 0;
        int nodes = 0;
    };
    auto combine = [](DistanceData a, DistanceData b) {
        return DistanceData{a.sum + b.sum, a.nodes + b.nodes};
    };
    auto finalize = [](DistanceData a, int) {
        a.nodes++;
        return a;
    };
    auto lift = [](DistanceData a, int, int) {
        a.sum += a.nodes;
        return a;
    };
};

```

```

vector<DistanceData> dp = tree.rerooting_dp(DistanceData{}, combine, finalize, lift);
vector<int64_t> answer(dp.size());
for (int i = 0; i < static_cast<int>(dp.size()); i++) {
    answer[i] = dp[i].sum;
}
return answer;
}

int main() {
    // 0--1--2
    //   |
    //   3--4
    RerootingTree tree(5);
    tree.add_edge(0, 1);
    tree.add_edge(1, 2);
    tree.add_edge(1, 3);
    tree.add_edge(3, 4);
    tree.build_rooted_tree();
    assert(tree.parent[4] == 3);
    assert(tree.depth[4] == 3);
    assert(tree.subtree_size[1] == 4);
    assert(tree.is_ancestor(1, 4));
    assert(!tree.is_ancestor(3, 2));
    vector<int64_t> distances = sum_distances_all_roots(tree);
    assert((distances == vector<int64_t>{8, 5, 8, 6, 9}));
    return 0;
}

```

4.1.5 Centroid Decomposition

Given a tree, build its centroid decomposition. A centroid is a node whose removal splits the current subtree into connected components of size at most half of that subtree. Recursively choosing centroids creates a decomposition tree of height $O(\log n)$, which is useful for queries based on distances to marked nodes, nearest special node, and other “path through a centroid” problems.

- `CentroidDecomposition(adj)` builds the centroid decomposition of the tree given by a bidirectional adjacency list `adj`, which must form a valid tree with `n` nodes numbered $[0, n)$, where `n` is `adj.size()`.
- `parent(u)` returns the parent of node `u` in the centroid tree, or -1 if `u` is the root centroid.

Time Complexity: $O(n \log n)$ for construction, where n is the number of nodes.

Space Complexity:

- $O(n)$ for storage of the centroid tree.
- $O(n)$ auxiliary stack space for the recursive searches.

```

#include <vector>

class CentroidDecomposition {
    std::vector<std::vector<int>>> adj;
    std::vector<int> subtree_size, par;
    std::vector<bool> removed;

    int get_size(int u, int p) {
        subtree_size[u] = 1;
        for (int v : adj[u]) {
            if (v != p && !removed[v]) {
                subtree_size[u] += get_size(v, u);
            }
        }
        return subtree_size[u];
    }

    int get_centroid(int u, int p, int total) {

```

```

    for (int v : adj[u]) {
        if (v != p && !removed[v] && subtree_size[v] > total / 2) {
            return get_centroid(v, u, total);
        }
    }
    return u;
}

void decompose(int entry, int parent) {
    int total = get_size(entry, -1);
    int centroid = get_centroid(entry, -1, total);
    par[centroid] = parent;
    removed[centroid] = true;
    for (int v : adj[centroid]) {
        if (!removed[v]) {
            decompose(v, centroid);
        }
    }
}

public:
    explicit CentroidDecomposition(const std::vector<std::vector<int>> &adj)
        : adj(adj), subtree_size(adj.size()), par(adj.size(), -1), removed(adj.size()) {
        if (adj.size() > 0) {
            decompose(0, -1);
        }
    }

    int parent(int u) const { return par[u]; }
};

```

Example Usage

```

#include <cassert>
using namespace std;

void add_edge(vector<vector<int>> &adj, int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    vector<vector<int>> adj(7);
    add_edge(adj, 0, 1);
    add_edge(adj, 1, 2);
    add_edge(adj, 1, 3);
    add_edge(adj, 3, 4);
    add_edge(adj, 3, 5);
    add_edge(adj, 5, 6);

    CentroidDecomposition tree(adj);
    assert(tree.parent(3) == -1);
    assert(tree.parent(1) == 3);
    assert(tree.parent(5) == 3);
    return 0;
}

```

4.1.6 Prufer Code

Encode and decode labeled trees using Prufer codes. A Prufer code is a sequence of length $n - 2$ that uniquely represents a labeled tree on nodes $0, 1, \dots, n - 1$. It is useful for counting labeled trees, generating test cases, and converting between trees and compact sequences. Encoding repeatedly removes a leaf and records its neighbor

until two nodes remain; decoding reverses this, deducing each removed leaf from the node degrees implied by the code. Both functions choose the smallest available leaf at each step. This makes the implementation deterministic and matches the usual textbook convention.

- `encode_prufer()` returns the prufer code for the global, bidirectionally pre-populated adjacency list `adj` which must form a valid tree with nodes numbered $[0, n)$, where `n` is `adj.size()`.
- `decode_prufer()` takes a Prufer code and returns the corresponding tree edges.

Time Complexity: $O(n \log n)$ per call to `encode_prufer()` or `decode_prufer()`, where n is the number of nodes in the tree.

Space Complexity: $O(n)$ auxiliary space per call to `encode_prufer()` or `decode_prufer()`.

```
#include <set>
#include <utility>
#include <vector>

std::vector<int> encode_prufer(const std::vector<std::vector<int>> &adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> degree(n), parent(n, -1), code;
    std::set<int> leaves;
    if (n <= 2) {
        return code;
    }
    // Root at node n - 1, which is guaranteed to survive smallest-leaf removal (it is never the
    // smallest leaf while other nodes remain). Rooting elsewhere risks recording parent[root] == -2
    // if the chosen root itself gets stripped to a leaf (e.g. when n - 1 happens to be a leaf).
    int root = n - 1;
    for (int u = 0; u < n; u++) {
        degree[u] = static_cast<int>(adj[u].size());
        if (degree[u] == 1) {
            leaves.insert(u);
        }
    }
    std::vector<int> stack(1, root);
    parent[root] = -2;
    while (!stack.empty()) {
        int u = stack.back();
        stack.pop_back();
        for (int v : adj[u]) {
            if (parent[v] == -1) {
                parent[v] = u;
                stack.push_back(v);
            }
        }
    }
    for (int i = 0; i < n - 2; i++) {
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());
        int p = parent[leaf];
        code.push_back(p);
        if (--degree[p] == 1) {
            leaves.insert(p);
        }
    }
    return code;
}

std::vector<std::pair<int, int>> decode_prufer(const std::vector<int> &code) {
    int n = static_cast<int>(code.size()) + 2;
    std::vector<int> degree(n, 1);
    std::set<int> leaves;
    std::vector<std::pair<int, int>> edges;
    for (int c : code) {
        degree[c]++;
    }
    for (int u = 0; u < n; u++) {
```

```

    if (degree[u] == 1) {
        leaves.insert(u);
    }
}
for (int u : code) {
    int leaf = *leaves.begin();
    leaves.erase(leaves.begin());
    edges.emplace_back(leaf, u);
    if (--degree[u] == 1) {
        leaves.insert(u);
    }
}
edges.emplace_back(*leaves.begin(), *leaves.rbegin());
return edges;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int>>> adj;
    auto add_edge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };
    adj.assign(5, {});
    add_edge(0, 3);
    add_edge(1, 3);
    add_edge(2, 3);
    add_edge(3, 4);
    vector<int> code = encode_prufer(adj);
    assert(code.size() == 3 && code[0] == 3 && code[1] == 3 && code[2] == 3);
    vector<pair<int, int>>> edges = decode_prufer(code);
    assert(edges.size() == 4);
    return 0;
}

```

4.1.7 Cycle Finding

Given a directed or undirected graph, find simple cycles encountered by a depth-first search. Cycles are returned as edge IDs rather than vertex lists, which makes the representation robust for multigraphs with parallel edges. In an undirected graph, the cycles found this way form a DFS cycle basis: one cycle for each non-tree edge. In a directed graph, this finds cycles created by back edges to vertices still on the recursion stack.

- `CycleFinder(n, directed)` constructs a graph of `n` nodes numbered `[0, n)`. The graph is directed if `directed` is true, or undirected otherwise.
- `add_edge(u, v)` adds an edge and returns its edge ID.
- `find_cycles(max_cycles, max_total_size)` returns simple cycles as vectors of edge IDs, stopping once either optional limit is reached.
- `cycle_vertices(edge_cycle)` converts one edge-ID cycle into a corresponding cyclic list of vertices.

Time Complexity:

- $O(\max(n, m + s))$ per call to `find_cycles()`, where n is the number of nodes, m is the number of edges, and s is the total size of the returned cycles.
- $O(k)$ per call to `cycle_vertices()`, where k is the number of edges in the cycle.

Space Complexity:

- $O(\max(n, m + s))$ for graph storage, DFS state, and returned cycles.
- $O(n)$ auxiliary stack space for the DFS.

```

#include <cassert>
#include <vector>

class CycleFinder {
    std::vector<std::pair<int, int>> edges;
    std::vector<std::vector<int>> adj;
    bool directed;

    void dfs_cycles(
        int u, int parent_edge, int max_cycles, int max_total_size, std::vector<int> &state,
        std::vector<int> &stack, std::vector<std::vector<int>> &cycles, int &total_size
    ) const {
        if (static_cast<int>(cycles.size()) >= max_cycles || total_size >= max_total_size) {
            return;
        }
        state[u] = static_cast<int>(stack.size());
        for (int id : adj[u]) {
            if (!directed && id == parent_edge) {
                continue;
            }
            const auto &[eu, ev] = edges[id];
            int v = directed ? ev : eu ^ ev ^ u;
            if (state[v] >= 0) {
                std::vector<int> cycle{id};
                for (int i = state[v]; i < static_cast<int>(stack.size()); i++) {
                    cycle.push_back(stack[i]);
                }
                cycles.push_back(cycle);
                total_size += static_cast<int>(cycle.size());
                if (static_cast<int>(cycles.size()) >= max_cycles || total_size >= max_total_size) {
                    return;
                }
            } else if (state[v] == -1) {
                stack.push_back(id);
                dfs_cycles(v, id, max_cycles, max_total_size, state, stack, cycles, total_size);
                stack.pop_back();
            }
        }
        state[u] = -2;
    }

public:
    CycleFinder(int n, bool directed) : adj(n), directed(directed) {}

    int add_edge(int u, int v) {
        int id = static_cast<int>(edges.size());
        edges.emplace_back(u, v);
        adj[u].push_back(id);
        if (!directed) {
            adj[v].push_back(id);
        }
        return id;
    }

    std::vector<std::vector<int>> find_cycles(
        int max_cycles = 1 << 30, int max_total_size = 1 << 30
    ) const {
        int n = static_cast<int>(adj.size());
        std::vector<int> state(n, -1), stack;
        std::vector<std::vector<int>> cycles;
        int total_size = 0;
        for (int u = 0; u < n; u++) {
            if (state[u] == -1) {

```

```

        dfs_cycles(u, -1, max_cycles, max_total_size, state, stack, cycles, total_size);
    }
}
return cycles;
}

std::vector<int> cycle_vertices(const std::vector<int> &edge_cycle) const {
    int size = static_cast<int>(edge_cycle.size());
    std::vector<int> vertices;
    if (size == 0) {
        return vertices;
    }
    if (size <= 2) {
        vertices.push_back(edges[edge_cycle[0]].first);
        if (size == 2) {
            vertices.push_back(edges[edge_cycle[0]].second);
        }
        return vertices;
    }
    for (int i = 0; i < size; i++) {
        const auto &[au, av] = edges[edge_cycle[i]];
        const auto &[bu, bv] = edges[edge_cycle[(i + 1) % size]];
        vertices.push_back((bu == au || bv == au) ? av : au);
    }
    return vertices;
}
};

```

Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    {
        CycleFinder g(4, false);
        int a = g.add_edge(0, 1);
        int b = g.add_edge(1, 2);
        int c = g.add_edge(2, 0);
        g.add_edge(2, 3);
        vector<vector<int>> cycles = g.find_cycles();
        assert(cycles.size() == 1);
        sort(cycles[0].begin(), cycles[0].end());
        assert((cycles[0] == vector<int>{a, b, c}));
        vector<int> vertices = g.cycle_vertices(cycles[0]);
        assert(vertices.size() == 3);
    }
    {
        CycleFinder g(2, false);
        int a = g.add_edge(0, 1);
        int b = g.add_edge(0, 1);
        vector<vector<int>> cycles = g.find_cycles();
        assert(cycles.size() == 1);
        sort(cycles[0].begin(), cycles[0].end());
        assert((cycles[0] == vector<int>{a, b}));
    }
    {
        CycleFinder g(3, true);
        g.add_edge(0, 1);
        g.add_edge(1, 2);
        g.add_edge(2, 0);
        assert(g.find_cycles(1).size() == 1);
    }
    return 0;
}

```

4.1.8 Eulerian Trails

A Eulerian trail is a path in a graph which contains every edge exactly once. A Eulerian cycle or circuit is an Eulerian trail which begins and ends on the same node. A directed graph has a Eulerian trail when all nonzero-degree nodes belong to one connected part of the underlying graph, and either every node has equal in-degree and out-degree or exactly one node has one extra outgoing edge and exactly one node has one extra incoming edge. An undirected graph has a Eulerian trail when all nonzero-degree nodes are connected and either zero or two nodes have odd degree.

Hierholzer's algorithm walks unused edges until stuck, then backtracks to splice each closed detour into the final trail. This implementation stores and returns edge IDs, which supports multigraphs: parallel edges are distinct because each edge receives its own ID.

- `EulerianGraph(n, directed)` constructs a graph of `n` nodes numbered `[0, n)`. The graph is directed if `directed` is true, or undirected otherwise.
- `add_edge(u, v)` adds an edge and returns its edge ID.
- `eulerian_path(start)` returns a trail using every edge exactly once, or a result with `start = -1` if no such trail exists. If `start = -1`, a valid start is chosen automatically.
- `EulerianTrail::is_cycle()` returns whether the trail begins and ends on the same node.

Time Complexity: $O(\max(n, m))$ per call to `eulerian_path()`, where n is the number of nodes and m is the number of edges.

Space Complexity: $O(\max(n, m))$ for graph storage and auxiliary arrays.

```
#include <algorithm>
#include <vector>

class EulerianGraph {
    std::vector<std::pair<int, int>>> edges;
    std::vector<std::vector<int>>> adj;
    bool directed;

    bool valid_degrees(int start) const {
        int n = static_cast<int>(adj.size());
        if (directed) {
            std::vector<int> indeg(n), outdeg(n);
            for (const auto &[eu, ev] : edges) {
                outdeg[eu]++;
                indeg[ev]++;
            }
            int source = -1, sink = -1;
            for (int u = 0; u < n; u++) {
                int diff = outdeg[u] - indeg[u];
                if (diff == 1) {
                    if (source != -1) {
                        return false;
                    }
                    source = u;
                } else if (diff == -1) {
                    if (sink != -1) {
                        return false;
                    }
                    sink = u;
                } else if (diff != 0) {
                    return false;
                }
            }
            if ((source == -1) != (sink == -1)) {
                return false;
            }
            if (start != -1) {
                return source == -1 ? outdeg[start] > 0 : start == source;
            }
            return true;
        }
    }
};
```

```

    }
    std::vector<int> degree(n);
    for (const auto &[eu, ev] : edges) {
        degree[eu]++;
        degree[ev]++;
    }
    std::vector<int> odd;
    for (int u = 0; u < n; u++) {
        if (degree[u] % 2 == 1) {
            odd.push_back(u);
        }
    }
    if (!odd.empty() && odd.size() != 2) {
        return false;
    }
    if (start != -1) {
        return odd.empty() ? degree[start] > 0 : (start == odd[0] || start == odd[1]);
    }
    return true;
}

int choose_start() const {
    int n = static_cast<int>(adj.size());
    if (directed) {
        std::vector<int> indeg(n), outdeg(n);
        for (const auto &[eu, ev] : edges) {
            outdeg[eu]++;
            indeg[ev]++;
        }
        for (int u = 0; u < n; u++) {
            if (outdeg[u] - indeg[u] == 1) {
                return u;
            }
        }
        for (int u = 0; u < n; u++) {
            if (outdeg[u] > 0) {
                return u;
            }
        }
    } else {
        std::vector<int> degree(n);
        for (const auto &[eu, ev] : edges) {
            degree[eu]++;
            degree[ev]++;
        }
        for (int u = 0; u < n; u++) {
            if (degree[u] % 2 == 1) {
                return u;
            }
        }
        for (int u = 0; u < n; u++) {
            if (degree[u] > 0) {
                return u;
            }
        }
    }
    return 0;
}

std::vector<int> build_vertices(int start, const std::vector<int> &trail_edges) const {
    std::vector<int> vertices{start};
    int u = start;
    for (int id : trail_edges) {
        const auto &[eu, ev] = edges[id];
        u = directed ? ev : eu ^ ev ^ u;
        vertices.push_back(u);
    }
    return vertices;
}

```

```

    }

public:
    struct EulerianTrail {
        int start = -1;
        std::vector<int> edges;
        std::vector<int> vertices;

        bool is_cycle() const {
            return start != -1 && vertices.size() > 1 && vertices.front() == vertices.back();
        }
    };

    EulerianGraph(int n, bool directed) : adj(n), directed(directed) {}

    int add_edge(int u, int v) {
        int id = static_cast<int>(edges.size());
        edges.emplace_back(u, v);
        adj[u].push_back(id);
        if (!directed) {
            adj[v].push_back(id);
        }
        return id;
    }

    EulerianTrail eulerian_path(int start = -1) const {
        int n = static_cast<int>(adj.size());
        int m = static_cast<int>(edges.size());
        if (m == 0) {
            int s = (start == -1 ? 0 : start);
            return EulerianTrail{s, {}, {s}};
        }
        if (!valid_degrees(start)) {
            return EulerianTrail{};
        }
        if (start == -1) {
            start = choose_start();
        }
        std::vector<bool> used(m, false);
        std::vector<int> ptr(n), vertex_stack{start}, edge_stack{-1}, trail_edges;
        while (!vertex_stack.empty()) {
            int u = vertex_stack.back();
            while (ptr[u] < static_cast<int>(adj[u].size()) && used[adj[u][ptr[u]]]) {
                ptr[u]++;
            }
            if (ptr[u] == static_cast<int>(adj[u].size())) {
                if (edge_stack.back() != -1) {
                    trail_edges.push_back(edge_stack.back());
                }
                vertex_stack.pop_back();
                edge_stack.pop_back();
            } else {
                int id = adj[u][ptr[u]++];
                used[id] = true;
                const auto &[eu, ev] = edges[id];
                int v = directed ? ev : eu ^ ev ^ u;
                vertex_stack.push_back(v);
                edge_stack.push_back(id);
            }
        }
        if (static_cast<int>(trail_edges.size()) != m) {
            return EulerianTrail{};
        }
        std::reverse(trail_edges.begin(), trail_edges.end());
        return EulerianTrail{start, trail_edges, build_vertices(start, trail_edges)};
    }
};

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    {
        EulerianGraph g(5, true);
        g.add_edge(0, 1);
        g.add_edge(1, 2);
        g.add_edge(2, 0);
        g.add_edge(1, 3);
        g.add_edge(3, 4);
        g.add_edge(4, 1);
        auto trail = g.eulerian_path(0);
        assert(trail.edges.size() == 6);
        assert(trail.vertices.front() == 0 && trail.vertices.back() == 0);
        assert(trail.is_cycle());
    }
    {
        EulerianGraph g(5, false);
        g.add_edge(0, 1);
        g.add_edge(1, 2);
        g.add_edge(2, 0);
        g.add_edge(1, 3);
        g.add_edge(3, 4);
        g.add_edge(4, 1);
        auto trail = g.eulerian_path();
        assert(trail.edges.size() == 6);
        assert(trail.vertices.front() == trail.vertices.back());
        assert(trail.is_cycle());
    }
    {
        EulerianGraph g(2, false);
        int a = g.add_edge(0, 1);
        int b = g.add_edge(0, 1);
        auto trail = g.eulerian_path(0);
        assert(trail.is_cycle());
        assert((trail.edges == vector<int>{a, b} || trail.edges == vector<int>{b, a}));
    }
    {
        EulerianGraph g(3, true);
        g.add_edge(0, 1);
        g.add_edge(1, 2);
        auto trail = g.eulerian_path();
        assert(trail.start == 0);
        assert((trail.vertices == vector<int>{0, 1, 2}));
        assert(!trail.is_cycle());
    }
    return 0;
}
```

4.2 Connectivity

4.2.1 Connected Components

Given an undirected graph, label every node with its connected component and collect the nodes in each component. This is the standard graph reachability primitive behind many larger routines: run one depth-first search from every unseen node, and all nodes visited by that search belong to the same component. For directed graphs, use a strongly connected components algorithm instead.

- `connected_components()` populates `comp_id[]` (component ID for each node) and `components[]` (nodes for each component ID) given a global, bidirectionally pre-populated adjacency list `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`.

Time Complexity: $O(\max(n, m))$ per call to `connected_components()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary stack space for `dfs()`.

```
#include <algorithm>
#include <vector>

std::vector<std::vector<int>> adj;
std::vector<std::vector<int>> components;
std::vector<int> comp_id;

void dfs(int u, int id) {
    comp_id[u] = id;
    components[id].push_back(u);
    for (int v : adj[u]) {
        if (comp_id[v] == -1) {
            dfs(v, id);
        }
    }
}

void connected_components() {
    int n = static_cast<int>(adj.size());
    components.clear();
    comp_id.assign(n, -1);
    for (int u = 0; u < n; u++) {
        if (comp_id[u] == -1) {
            components.push_back({});
            dfs(u, static_cast<int>(components.size() - 1));
        }
    }
}
```

Example Usage

```
#include <cassert>
using namespace std;

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main() {
    adj.assign(6, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(3, 4);
    connected_components();
    assert(components.size() == 3);
    assert(comp_id[0] == comp_id[2]);
    assert(comp_id[0] != comp_id[3]);
    assert(comp_id[5] == 2);
    return 0;
}
```

4.2.2 Strongly Connected Components (Kosaraju)

Given a directed graph, determine the strongly connected components (SCCs) using the Kosaraju-Sharir algorithm. A strongly connected component is a maximal set of vertices where every vertex can reach every other vertex. Condensing each SCC into one node produces a directed acyclic graph. The algorithm runs two passes of depth-first search: the first records the order in which nodes finish, and the second explores the transposed graph in reverse finish order, with each search collecting exactly one component.

- `KosarajuSCC(n)` constructs a directed graph of `n` nodes numbered $[0, n)$.
- `add_edge(u, v)` adds the directed edge from `u` to `v`.
- `build_scc()` populates `scc` with the strongly connected components and `component[v]` with the component ID containing vertex `v`. Component IDs are in topological order: for every edge from component `a` to a different component `b`, $a < b$.

Time Complexity: $O(\max(n, m))$ per call to `build_scc()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, reverse graph, SCCs, and component IDs.
- $O(n)$ auxiliary stack space.

```
#include <algorithm>
#include <vector>

struct KosarajuSCC {
    std::vector<std::vector<int>>> adj, rev, scc;
    std::vector<int> component;
    std::vector<bool> visited;

    KosarajuSCC(int n = 0) : adj(n), rev(n) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        rev[v].push_back(u);
    }

    void dfs(const std::vector<std::vector<int>>> &g, std::vector<int> &res, int u) {
        visited[u] = true;
        for (int v : g[u]) {
            if (!visited[v]) {
                dfs(g, res, v);
            }
        }
        res.push_back(u);
    }

    void dfs_component(int u, int id, std::vector<int> &res) {
        visited[u] = true;
        component[u] = id;
        res.push_back(u);
        for (int v : rev[u]) {
            if (!visited[v]) {
                dfs_component(v, id, res);
            }
        }
    }

    void build_scc() {
        int n = static_cast<int>(adj.size());
        visited.assign(n, false);
        std::vector<int> order;
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                dfs(adj, order, i);
            }
        }
    }
};
```



```

    }
}
std::reverse(order.begin(), order.end());
visited.assign(n, false);
component.assign(n, -1);
scc.clear();
for (int u : order) {
    if (visited[u]) {
        continue;
    }
    std::vector<int> component_vertices;
    dfs_component(u, static_cast<int>(scc.size()), component_vertices);
    scc.push_back(component_vertices);
}
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    KosarajuSCC g(8);
    g.add_edge(0, 1);
    g.add_edge(1, 2);
    g.add_edge(1, 4);
    g.add_edge(1, 5);
    g.add_edge(2, 3);
    g.add_edge(2, 6);
    g.add_edge(3, 2);
    g.add_edge(3, 7);
    g.add_edge(4, 0);
    g.add_edge(4, 5);
    g.add_edge(5, 6);
    g.add_edge(6, 5);
    g.add_edge(7, 3);
    g.add_edge(7, 6);
    g.build_scc();
    cout << "Components:" << endl;
    for (auto &component : g.scc) {
        for (int v : component) {
            cout << v << " ";
        }
        cout << endl;
    }
    return 0;
}

```

Example Output

```

Components:
1 4 0
7 3 2
5 6

```

4.2.3 Strongly Connected Components (Tarjan)

Given a directed graph, determine the strongly connected components (SCCs) using Tarjan's algorithm. A strongly connected component is a maximal set of vertices where every vertex can reach every other vertex. Condensing each SCC into one node produces a directed acyclic graph. A single depth-first search keeps visited nodes on a stack and tracks each node's low-link, the smallest entry time reachable from its subtree; a node whose low-link equals its own entry time roots a component, which is popped off the stack in one piece.

- `TarjanSCC(n)` constructs a directed graph of `n` nodes numbered $[0, n)$.
- `add_edge(u, v)` adds the directed edge from `u` to `v`.
- `build_scc()` populates `scc` with the strongly connected components and `component[v]` with the component ID containing vertex `v`. Component IDs are in reverse topological order: for every edge from component a to a different component b , $a > b$.

Time Complexity: $O(\max(n, m))$ per call to `build_scc()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, SCCs, and component IDs.
- $O(n)$ auxiliary stack space.

```
#include <algorithm>
#include <climits>
#include <vector>

struct TarjanSCC {
    static const int INF = INT_MAX / 2;
    std::vector<std::vector<int>>> adj, scc;
    std::vector<int> component, stack, lowlink;
    std::vector<bool> visited;
    int timer;

    TarjanSCC(int n = 0) : adj(n) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    void dfs(int u) {
        lowlink[u] = timer++;
        visited[u] = true;
        stack.push_back(u);
        bool is_component_root = true;
        for (int v : adj[u]) {
            if (!visited[v]) {
                dfs(v);
            }
            if (lowlink[u] > lowlink[v]) {
                lowlink[u] = lowlink[v];
                is_component_root = false;
            }
        }
        if (!is_component_root) {
            return;
        }
        std::vector<int> comp_nodes;
        int id = static_cast<int>(scc.size());
        int v;
        do {
            v = stack.back();
            stack.pop_back();
            lowlink[v] = INF; // marks v as removed from the stack
            component[v] = id;
            comp_nodes.push_back(v);
        } while (u != v);
        scc.push_back(comp_nodes);
    }

    void build_scc() {
        int n = static_cast<int>(adj.size());
        scc.clear();
        component.assign(n, -1);
        stack.clear();
        lowlink.assign(n, 0);
        visited.assign(n, false);
        timer = 0;
    }
};
```

```

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i);
        }
    }
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    TarjanSCC g(8);
    g.add_edge(0, 1);
    g.add_edge(1, 2);
    g.add_edge(1, 4);
    g.add_edge(1, 5);
    g.add_edge(2, 3);
    g.add_edge(2, 6);
    g.add_edge(3, 2);
    g.add_edge(3, 7);
    g.add_edge(4, 0);
    g.add_edge(4, 5);
    g.add_edge(5, 6);
    g.add_edge(6, 5);
    g.add_edge(7, 3);
    g.add_edge(7, 6);
    g.build_scc();
    cout << "Components:" << endl;
    for (auto &component : g.scc) {
        for (int v : component) {
            cout << v << " ";
        }
        cout << endl;
    }
    return 0;
}

```

Example Output

```

Components:
5 6
7 3 2
4 1 0

```

4.2.4 Dominator Tree (Lengauer-Tarjan)

Given a directed graph and a start node, find the immediate dominator of every node reachable from the start. A node u dominates node v if every path from the start to v passes through u . The immediate dominator $\text{idom}[v]$ is the closest strict dominator of v ; these parent links form the dominator tree rooted at the start. Dominator trees are useful in control-flow graphs, program analysis, and reachability problems with unavoidable checkpoints.

Lengauer-Tarjan's algorithm numbers vertices by DFS order, computes each reachable node's semidominator using a disjoint-set structure with path compression, and then resolves immediate dominators from semidominator buckets.

- `Dominators(n)` constructs a directed graph of n nodes numbered $[0, n)$.
- `add_edge(u, v)` adds a directed edge from u to v .
- `find_dominators(start)` returns a vector `idom` where `idom[start] = start`, `idom[v]` is the immediate dominator of reachable node v , and `idom[v] = -1` if v is unreachable.

Time Complexity: $O(\max(n, m) \log n)$ per call to `find_dominators()` in this path-compressed implementation, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for the graph and auxiliary arrays.
- $O(n)$ auxiliary stack space for the DFS.

```
#include <algorithm>
#include <vector>

class Dominators {
    std::vector<std::vector<int>>> adj, pred, bucket;
    std::vector<int> time, vertex, parent, sdом, idом_index, dsu, best;
    int timer;

    void dfs(int u) {
        time[u] = ++timer;
        vertex[timer] = u;
        sdом[timer] = dsu[timer] = best[timer] = timer;
        for (int v : adj[u]) {
            if (time[v] == 0) {
                dfs(v);
                parent[time[v]] = time[u];
            }
            if (time[v] != 0) {
                pred[time[v]].push_back(time[u]);
            }
        }
    }

    int find_best(int u, bool compress) {
        if (dsu[u] != u) {
            int v = find_best(dsu[u], true);
            if (sdом[v] < sdом[best[u]]) {
                best[u] = v;
            }
            if (compress) {
                dsu[u] = dsu[dsu[u]];
            }
        }
        return best[u];
    }

public:
    explicit Dominators(int n) : adj(n), timer(0) {}

    void add_edge(int u, int v) { adj[u].push_back(v); }

    std::vector<int> find_dominators(int start) {
        int n = static_cast<int>(adj.size());
        timer = 0;
        time.assign(n, 0);
        vertex.assign(n + 1, 0);
        parent.assign(n + 1, 0);
        sdом.assign(n + 1, 0);
        idом_index.assign(n + 1, 0);
        dsu.assign(n + 1, 0);
        best.assign(n + 1, 0);
        pred.assign(n + 1, {});
        bucket.assign(n + 1, {});
        dfs(start);
        for (int i = timer; i >= 1; i--) {
            for (int v : pred[i]) {
                sdом[i] = std::min(sdом[i], sdом[find_best(v, false)]);
            }
            if (i > 1) {
                bucket[sdom[i]].push_back(i);
            }
        }
    }
};
```

```

    }
    for (int v : bucket[i]) {
        int u = find_best(v, false);
        idom_index[v] = (sdom[u] == sdom[v]) ? sdom[v] : u;
    }
    if (i > 1) {
        dsu[i] = parent[i];
    }
}
std::vector<int> idom(adj.size(), -1);
idom[start] = start;
for (int i = 2; i <= timer; i++) {
    if (idom_index[i] != sdom[i]) {
        idom_index[i] = idom_index[idom_index[i]];
    }
    idom[vertex[i]] = vertex[idom_index[i]];
}
return idom;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Dominators g(7);
    // 0 branches through 1 and 2, then reconverges at 3 before reaching 4 and 5.
    g.add_edge(0, 1);
    g.add_edge(0, 2);
    g.add_edge(1, 3);
    g.add_edge(2, 3);
    g.add_edge(3, 4);
    g.add_edge(3, 5);
    g.add_edge(4, 6);
    g.add_edge(5, 6);
    vector<int> idom = g.find_dominators(0);
    assert(idom[0] == 0);
    assert(idom[1] == 0);
    assert(idom[2] == 0);
    assert(idom[3] == 0);
    assert(idom[4] == 3);
    assert(idom[5] == 3);
    assert(idom[6] == 3);
    assert(g.find_dominators(0) == idom);
    return 0;
}

```

4.2.5 Articulation Points, Biconnected Components, Block-Cut Forest

Given an undirected graph, compute articulation points, vertex-biconnected components (BCCs), and the block-cut forest using Tarjan's algorithm. An articulation point (a.k.a. cut vertex) is a vertex whose removal increases the number of connected components in the graph. A vertex-biconnected component is a maximal subgraph that cannot be disconnected by removing one vertex, with single-edge and isolated-vertex components handled as degenerate cases. A single depth-first search tracks each node's low-link, the earliest entry time reachable from its subtree: a node is an articulation point exactly when some child's subtree cannot reach above it, and the edges of each BCC are collected on a stack popped at that moment.

The block-cut forest is a bipartite forest with one node for each BCC and one node for each articulation point, with an edge whenever an articulation point belongs to a BCC. Note that this differs from a bridge forest: the

block-cut forest describes vertex connectivity, while a bridge forest describes edge connectivity after compressing 2-edge-connected components.

- `BiconnectedComponents(n)` constructs an undirected graph of `n` nodes numbered `[0, n)`.
- `add_edge(u, v)` adds the undirected edge `u-v`. Parallel edges are supported.
- `build_bcc()` populates `articulation_points` and `bccs`.
- `build_block_cut_forest()` populates `block_cut_forest` and `block_cut_id` using the results of the previous `build_bcc()` call. After the call, `block_cut_forest` nodes are numbered in the range `[0, bccs.size())`, and `block_cut_id[v]` stores the node ID of articulation point `v` in the block-cut forest, or `-1` if `v` is not an articulation point.

Time Complexity: $O(\max(n, m))$ per call to `build_bcc()` and `build_block_cut_forest()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, edge stack, BCCs, and block-cut forest.
- $O(n)$ auxiliary stack space for `build_bcc()`.

```
#include <algorithm>
#include <utility>
#include <vector>

struct BiconnectedComponents {
    std::vector<std::vector<int>>> adj, bccs, block_cut_forest;
    std::vector<int> lowlink, tin, block_cut_id, articulation_points;
    std::vector<bool> visited, is_articulation;
    std::vector<std::pair<int, int>> edges;
    std::vector<int> edge_stack;
    int timer;

    BiconnectedComponents(int n = 0) : adj(n) {}

    void add_edge(int u, int v) {
        int id = static_cast<int>(edges.size());
        adj[u].push_back(id);
        adj[v].push_back(id);
        edges.emplace_back(u, v);
    }

    int other(int id, int u) const { return edges[id].first ^ edges[id].second ^ u; }

    void add_component_until(int stop_id) {
        std::vector<int> component;
        while (true) {
            int id = edge_stack.back();
            edge_stack.pop_back();
            component.push_back(edges[id].first);
            component.push_back(edges[id].second);
            if (id == stop_id) {
                break;
            }
        }
        std::sort(component.begin(), component.end());
        component.erase(std::unique(component.begin(), component.end()), component.end());
        bccs.push_back(component);
    }

    void dfs(int u, int p) {
        visited[u] = true;
        lowlink[u] = tin[u] = timer++;
        int children = 0;
        for (int id : adj[u]) {
            if (id == p) {
                continue;
            }
        }
    }
};
```

```

    int v = other(id, u);
    if (visited[v]) {
        if (tin[v] < tin[u]) {
            edge_stack.push_back(id);
            lowlink[u] = std::min(lowlink[u], tin[v]);
        }
    } else {
        edge_stack.push_back(id);
        dfs(v, id);
        lowlink[u] = std::min(lowlink[u], lowlink[v]);
        children++;
        if (lowlink[v] >= tin[u]) {
            if (p != -1 || children > 1) {
                is_articulation[u] = true;
            }
            add_component_until(id);
        }
    }
}
}
if (p == -1 && children == 0) {
    bccs.push_back({u});
}
}

void build_bcc() {
    int n = static_cast<int>(adj.size());
    articulation_points.clear();
    bccs.clear();
    edge_stack.clear();
    lowlink.assign(n, 0);
    tin.assign(n, 0);
    visited.assign(n, false);
    is_articulation.assign(n, false);
    timer = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i, -1);
        }
    }
    for (int i = 0; i < n; i++) {
        if (is_articulation[i]) {
            articulation_points.push_back(i);
        }
    }
}

void build_block_cut_forest() {
    int n = static_cast<int>(adj.size());
    int blocks = static_cast<int>(bccs.size());
    block_cut_id.assign(n, -1);
    int total = blocks;
    for (int v : articulation_points) {
        block_cut_id[v] = total++;
    }
    block_cut_forest.assign(total, {});
    for (int b = 0; b < blocks; b++) {
        for (int v : bccs[b]) {
            if (block_cut_id[v] != -1) {
                block_cut_forest[b].push_back(block_cut_id[v]);
                block_cut_forest[block_cut_id[v]].push_back(b);
            }
        }
    }
}
};

```

Example Usage

```
#include <iostream>
using namespace std;

int main() {
    BiconnectedComponents g(8);
    g.add_edge(0, 1);
    g.add_edge(0, 5);
    g.add_edge(1, 2);
    g.add_edge(1, 5);
    g.add_edge(3, 7);
    g.add_edge(4, 5);
    g.build_bcc();
    cout << "Articulation points:";
    for (int v : g.articulation_points) {
        cout << " " << v;
    }
    cout << endl << "Biconnected components:" << endl;
    for (auto &component : g.bccs) {
        for (int v : component) {
            cout << v << " ";
        }
        cout << endl;
    }
    g.build_block_cut_forest();
    cout << "Adjacency List for Block-Cut Forest:" << endl;
    for (int i = 0, n = static_cast<int>(g.block_cut_forest.size()); i < n; i++) {
        cout << i << " =>";
        for (int v : g.block_cut_forest[i]) {
            cout << " " << v;
        }
        cout << endl;
    }
    return 0;
}
```

Example Output

```
Articulation points: 1 5
Biconnected components:
1 2
4 5
0 1 5
3 7
6
Adjacency List for Block-Cut Forest:
0 => 5
1 => 6
2 => 5 6
3 =>
4 =>
5 => 0 2
6 => 1 2
```

4.2.6 Bridges, 2-Edge-Connected Components, Bridge Forest

Given an undirected graph, compute bridges, 2-edge-connected components, and the bridge forest using Tarjan's algorithm. A bridge is an edge whose removal increases the number of connected components in the graph. A 2-edge-connected component is a maximal set of vertices connected without crossing any bridge. A single depth-first search tracks each node's low-link, the earliest entry time reachable from its subtree: a tree edge is a bridge exactly when the child's subtree cannot reach back to the parent's side by any other route.

After each 2-edge-connected component is condensed into one node, the original bridges connect those nodes into a bridge tree, or a bridge forest when the original graph is disconnected. This differs from a block-cut forest: the bridge forest describes edge connectivity, while the block-cut forest describes vertex connectivity using articulation points and vertex-biconnected components.

- `BridgeDecomposition(n)` constructs an undirected graph of `n` nodes numbered `[0, n)`.
- `add_edge(u, v)` adds the undirected edge `u-v`. Parallel edges are supported.
- `build_bridges()` populates `bridges`.
- `build_bridge_forest()` populates `component`, `two_edge_components`, and `bridge_forest` using the results of the previous `build_bridges()` call. After the call, `component[u]` stores the 2-edge-connected component ID containing vertex `u`.

Time Complexity: $O(\max(n, m))$ per call to `build_bridges()` followed by `build_bridge_forest()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, bridges, 2-edge-connected components, and bridge forest.
- $O(n)$ auxiliary stack space for the DFS calls.

```
#include <algorithm>
#include <utility>
#include <vector>

struct BridgeDecomposition {
    std::vector<std::vector<int>>> adj, two_edge_components, bridge_forest;
    std::vector<int> lowlink, tin, component;
    std::vector<bool> visited, is_bridge_edge;
    std::vector<std::pair<int, int>> edges, bridges;
    int timer;

    BridgeDecomposition(int n = 0) : adj(n) {}

    void add_edge(int u, int v) {
        int id = static_cast<int>(edges.size());
        adj[u].push_back(id);
        adj[v].push_back(id);
        edges.emplace_back(u, v);
    }

    int other(int id, int u) const { return edges[id].first ^ edges[id].second ^ u; }

    void dfs_bridges(int u, int p) {
        visited[u] = true;
        lowlink[u] = tin[u] = timer++;
        for (int id : adj[u]) {
            if (id == p) {
                continue;
            }
            int v = other(id, u);
            if (visited[v]) {
                lowlink[u] = std::min(lowlink[u], tin[v]);
            } else {
                dfs_bridges(v, id);
                lowlink[u] = std::min(lowlink[u], lowlink[v]);
                if (lowlink[v] > tin[u]) {
                    bridges.emplace_back(u, v);
                    is_bridge_edge[id] = true;
                }
            }
        }
    }

    void build_bridges() {
        int n = static_cast<int>(adj.size());
        bridges.clear();
    }
};
```

```

    lowlink.assign(n, 0);
    tin.assign(n, 0);
    visited.assign(n, false);
    is_bridge_edge.assign(edges.size(), false);
    timer = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs_bridges(i, -1);
        }
    }
}

void dfs_component(int u, int id) {
    component[u] = id;
    two_edge_components[id].push_back(u);
    for (int edge_id : adj[u]) {
        int v = other(edge_id, u);
        if (component[v] == -1 && !is_bridge_edge[edge_id]) {
            dfs_component(v, id);
        }
    }
}

void build_bridge_forest() {
    int n = static_cast<int>(adj.size());
    component.assign(n, -1);
    two_edge_components.clear();
    for (int i = 0; i < n; i++) {
        if (component[i] == -1) {
            two_edge_components.push_back({});
            dfs_component(i, static_cast<int>(two_edge_components.size() - 1));
        }
    }
    bridge_forest.assign(two_edge_components.size(), {});
    for (auto &[u, v] : bridges) {
        int cu = component[u], cv = component[v];
        bridge_forest[cu].push_back(cv);
        bridge_forest[cv].push_back(cu);
    }
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    BridgeDecomposition g(8);
    g.add_edge(0, 1);
    g.add_edge(0, 5);
    g.add_edge(1, 2);
    g.add_edge(1, 5);
    g.add_edge(3, 7);
    g.add_edge(4, 5);
    g.build_bridges();
    g.build_bridge_forest();
    cout << "Bridges:" << endl;
    for (auto &[u, v] : g.bridges) {
        cout << u << " " << v << endl;
    }
    cout << "2-edge-connected components:" << endl;
    for (auto &component : g.two_edge_components) {
        for (int v : component) {
            cout << v << " ";
        }
    }
}

```

```

    cout << endl;
}
cout << "Adjacency List for Bridge Forest:" << endl;
for (int i = 0, n = static_cast<int>(g.bridge_forest.size()); i < n; i++) {
    cout << i << " =>";
    for (int v : g.bridge_forest[i]) {
        cout << " " << v;
    }
    cout << endl;
}
return 0;
}

```

Example Output

```

Bridges:
1 2
5 4
3 7
2-edge-connected components:
0 1 5
2
3
4
6
7
Adjacency List for Bridge Forest:
0 => 1 3
1 => 0
2 => 5
3 => 0
4 =>
5 => 2

```

4.2.7 Online Bridge Finding

Maintain the number of bridges in an undirected graph while edges are inserted one at a time, along with the 2-edge-connected components and ordinary connected components. When an inserted edge joins two different connected components, it creates a new bridge. When it joins two nodes already in the same connected component, it creates a cycle and every bridge on the path between the two endpoints stops being a bridge.

- `OnlineBridges(n)` constructs a graph with `n` isolated nodes numbered `[0, n)`.
- `add_edge(u, v)` adds the undirected edge `u-v` and updates: `bridges` with the current number of bridges, and the disjoint-set parent arrays `dsu_2ecc[]` and `dsu_cc[]`, which partition the nodes into 2-edge-connected components and ordinary connected components respectively.
- `find_2ecc(u)` returns node `u`'s representative in the 2-edge-connected components partition.
- `find_cc(u)` returns node `u`'s representative in the ordinary connected components partition.

Parallel edges are supported. Adding a second edge between two nodes that are already in the same connected component dissolves every bridge on the path between them, merging those nodes into the same 2-edge-connected component.

Time Complexity: $O(\log n)$ amortized per call to `add_edge()` in this implementation, where n is the number of nodes.

Space Complexity: $O(n)$ for the disjoint-set arrays and auxiliary parent links.

```

#include <algorithm>
#include <vector>

struct OnlineBridges {
    std::vector<int> dsu_2ecc, dsu_cc, dsu_cc_size, parent, last_visit;
    int lca_iteration, bridges;

```

```

OnlineBridges(int n = 0) {
    dsu_2ecc.resize(n);
    dsu_cc.resize(n);
    dsu_cc_size.assign(n, 1);
    parent.assign(n, -1);
    last_visit.assign(n, 0);
    lca_iteration = 0;
    bridges = 0;
    for (int i = 0; i < n; i++) {
        dsu_2ecc[i] = dsu_cc[i] = i;
    }
}

int find_2ecc(int u) {
    if (u == -1) {
        return -1;
    }
    if (dsu_2ecc[u] == u) {
        return u;
    }
    return dsu_2ecc[u] = find_2ecc(dsu_2ecc[u]);
}

int find_cc(int u) {
    u = find_2ecc(u);
    if (dsu_cc[u] == u) {
        return u;
    }
    return dsu_cc[u] = find_cc(dsu_cc[u]);
}

void make_root(int u) {
    u = find_2ecc(u);
    int root = u, child = -1;
    while (u != -1) {
        int p = find_2ecc(parent[u]);
        parent[u] = child;
        dsu_cc[u] = root;
        child = u;
        u = p;
    }
    dsu_cc_size[root] = dsu_cc_size[child];
}

void merge_path(int u, int v) {
    lca_iteration++;
    std::vector<int> path_a, path_b;
    int lca = -1;
    while (lca == -1) {
        if (u != -1) {
            u = find_2ecc(u);
            path_a.push_back(u);
            if (last_visit[u] == lca_iteration) {
                lca = u;
                break;
            }
            last_visit[u] = lca_iteration;
            u = parent[u];
        }
        if (v != -1) {
            v = find_2ecc(v);
            path_b.push_back(v);
            if (last_visit[v] == lca_iteration) {
                lca = v;
                break;
            }
            last_visit[v] = lca_iteration;
        }
    }
}

```

```

        v = parent[v];
    }
}
for (int a : path_a) {
    dsu_2ecc[a] = lca;
    if (a == lca) {
        break;
    }
    bridges--;
}
for (int b : path_b) {
    dsu_2ecc[b] = lca;
    if (b == lca) {
        break;
    }
    bridges--;
}
}
}

void add_edge(int u, int v) {
    u = find_2ecc(u);
    v = find_2ecc(v);
    if (u == v) {
        return;
    }
    int ca = find_cc(u), cb = find_cc(v);
    if (ca != cb) {
        bridges++;
        if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
            std::swap(u, v);
            std::swap(ca, cb);
        }
        make_root(u);
        parent[u] = v;
        dsu_cc[u] = v;
        dsu_cc_size[cb] += dsu_cc_size[u];
    } else {
        merge_path(u, v);
    }
}
};

```

Example Usage

```

#include <cassert>

int main() {
    OnlineBridges g(4);
    g.add_edge(0, 1);
    assert(g.bridges == 1);
    g.add_edge(1, 2);
    assert(g.bridges == 2);
    g.add_edge(2, 0);
    assert(g.bridges == 0);
    g.add_edge(2, 3);
    assert(g.bridges == 1);
    return 0;
}

```

4.2.8 2-SAT

Solve a Boolean formula in 2-CNF, where each clause contains at most two literals. A clause like $(a \vee b)$ is represented by the implications $\neg a \rightarrow b$ and $\neg b \rightarrow a$. The formula is satisfiable if and only if no variable and its negation belong to the same strongly connected component of the implication graph.

Variables are numbered from 0 to $n - 1$. A literal is represented by `literal(variable, value)`, where `value == true` means the variable itself and `value == false` means its negation.

- `TwoSAT(n)` constructs an empty formula over n variables.
- `literal(variable, value)` returns the integer ID for a variable or its negation.
- `add_implication(a, b)` adds the implication $a \rightarrow b$.
- `add_or(a, b)` adds the clause $(a \vee b)$.
- `add_true(a)` forces literal a to be true.
- `add_false(a)` forces literal a to be false.
- `add(variable, value)` forces a variable to equal `value`.
- `add(x, xval, y, yval)` adds the clause $(x = xval \vee y = yval)$.
- `satisfiable()` returns whether all added clauses can be satisfied and stores one valid assignment in `answer`.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the number of variables.
- $O(1)$ per call to `literal()`, `add_implication()`, `add_or()`, `add_true()`, `add_false()`, and `add()`.
- $O(\max(n, m))$ per call to `satisfiable()`, where n is the number of variables and m is the number of clauses.

Space Complexity: $O(\max(n, m))$ for the implication graph and DFS stacks.

```
#include <algorithm>
#include <vector>

struct TwoSAT {
    int variables;
    std::vector<std::vector<int>>> adj, rev;
    std::vector<int> order, component, answer;

    TwoSAT(int n = 0) : variables(n), adj(2 * n), rev(2 * n), answer(n, false) {}

    int literal(int variable, bool value) { return 2 * variable + (value ? 0 : 1); }
    int neg(int x) { return x ^ 1; }

    void add_implication(int a, int b) {
        adj[a].push_back(b);
        rev[b].push_back(a);
    }

    void add_or(int a, int b) {
        add_implication(neg(a), b);
        add_implication(neg(b), a);
    }

    void add_true(int a) { add_implication(neg(a), a); }
    void add_false(int a) { add_implication(a, neg(a)); }
    void add(int variable, bool value) { add_true(literal(variable, value)); }
    void add(int x, bool xval, int y, bool yval) { add_or(literal(x, xval), literal(y, yval)); }

    void dfs_order(int u, std::vector<bool> &visited) {
        visited[u] = true;
        for (int v : adj[u]) {
            if (!visited[v]) {
                dfs_order(v, visited);
            }
        }
        order.push_back(u);
    }
}
```

```

void dfs_component(int u, int id) {
    component[u] = id;
    for (int v : rev[u]) {
        if (component[v] == -1) {
            dfs_component(v, id);
        }
    }
}

bool satisfiable() {
    order.clear();
    component.assign(2 * variables, -1);
    std::vector<bool> visited(2 * variables, false);
    for (int i = 0; i < 2 * variables; i++) {
        if (!visited[i]) {
            dfs_order(i, visited);
        }
    }
    std::reverse(order.begin(), order.end());
    int id = 0;
    for (int u : order) {
        if (component[u] == -1) {
            dfs_component(u, id++);
        }
    }
    for (int i = 0; i < variables; i++) {
        if (component[2 * i] == component[2 * i + 1]) {
            return false;
        }
        answer[i] = component[2 * i] > component[2 * i + 1];
    }
    return true;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    TwoSAT solver(3);
    int x0 = solver.literal(0, true);
    int not_x0 = solver.literal(0, false);
    int x1 = solver.literal(1, true);
    int x2 = solver.literal(2, true);
    solver.add_or(x0, x1);
    solver.add_or(not_x0, x2);
    solver.add_true(x0);
    solver.add(1, true, 2, false);
    assert(solver.satisfiable());
    assert(solver.answer[0]);
    return 0;
}

```

4.2.9 Functional Graph

A functional graph is a directed graph in which every node has exactly one outgoing edge, given by a successor function f , where $f[i]$ is the node reached in one step from node i . Following the edges from any node eventually enters a cycle, so each weakly connected component consists of one directed cycle with trees of nodes feeding into it. This structure underlies problems about iterating a function: where a sequence $i, f[i], f[f[i]], \dots$ ends up, and how far it is from repeating.

Building the structure peels away nodes of in-degree zero to expose the cycles (the survivors), then labels each cycle and walks the remaining tail nodes to record their distance to the cycle. A binary lifting table stores f applied 2^k times, so any number of steps can be taken in logarithmic time. Together these answer, for any node, which cycle it eventually reaches, how many steps until it gets there, and exactly where it lands after a huge number of steps.

- `FunctionalGraph(f)` builds the structure for the successor function f over nodes numbered $[0, n)$, where n is `f.size()`.
- `on_cycle(i)` returns whether node i lies on a cycle.
- `dist_to_cycle(i)` returns the number of steps from i to the first cycle node it reaches (0 if i is itself on a cycle).
- `cycle_id(i)` returns an identifier for the cycle that i eventually reaches; two nodes share an identifier exactly when they funnel into the same cycle.
- `cycle_length(id)` returns the length of the cycle with the given identifier.
- `num_cycles()` returns the number of distinct cycles.
- `successor(i, steps)` returns the node reached from i after applying f exactly `steps` times, for any nonnegative `steps` (which may be astronomically large).

Time Complexity:

- $O(n \log n)$ per call to the constructor, where n is the number of nodes.
- $O(\log n)$ per call to `successor()`.
- $O(1)$ per call to all other member functions.

Space Complexity: $O(n \log n)$ for storage of the binary lifting table.

```
#include <cstdint>
#include <vector>

class FunctionalGraph {
    int n, LOG;
    std::vector<std::vector<int>>> up; // up[k][i] = f applied 2^k times to i.
    std::vector<bool> on_cyc;
    std::vector<int> to_cyc; // Steps from i to the first cycle node.
    std::vector<int> comp; // Identifier of the cycle that i reaches.
    std::vector<int> clen; // clen[c] = length of cycle c.

    int jump(int i, int64_t steps) const {
        for (int k = 0; steps > 0; k++, steps >>= 1) {
            if (steps & 1) {
                i = up[k][i];
            }
        }
        return i;
    }

public:
    FunctionalGraph(const std::vector<int> &f) : n(f.size()) {
        LOG = 1;
        while ((1 << LOG) < n) {
            LOG++;
        }
        up.assign(LOG + 1, std::vector<int>(n));
        up[0] = f;
        for (int k = 1; k <= LOG; k++) {
            for (int i = 0; i < n; i++) {
                up[k][i] = up[k - 1][up[k - 1][i]];
            }
        }
        // Peel nodes of in-degree zero; the survivors are exactly the cycle nodes.
        std::vector<int> indeg(n, 0);
        for (int i = 0; i < n; i++) {
            indeg[f[i]]++;
        }
    }
};
```



```

std::vector<bool> removed(n, false);
std::vector<int> stack;
for (int i = 0; i < n; i++) {
    if (indeg[i] == 0) {
        stack.push_back(i);
    }
}
while (!stack.empty()) {
    int u = stack.back();
    stack.pop_back();
    removed[u] = true;
    if (--indeg[f[u]] == 0) {
        stack.push_back(f[u]);
    }
}
on_cyc.assign(n, false);
for (int i = 0; i < n; i++) {
    on_cyc[i] = !removed[i];
}
// Label each cycle and record its length.
comp.assign(n, -1);
to_cyc.assign(n, 0);
for (int i = 0; i < n; i++) {
    if (on_cyc[i] && comp[i] == -1) {
        int len = 0;
        for (int u = i; comp[u] == -1; u = f[u]) {
            comp[u] = static_cast<int>(clen.size());
            len++;
        }
        clen.push_back(len);
    }
}
// Walk each tail to the cycle, assigning its component and distance.
for (int i = 0; i < n; i++) {
    if (comp[i] != -1) {
        continue;
    }
    std::vector<int> path;
    int u = i;
    while (comp[u] == -1) {
        path.push_back(u);
        u = f[u];
    }
    for (int j = static_cast<int>(path.size()) - 1, step = 1; j >= 0; j--, step++) {
        comp[path[j]] = comp[u];
        to_cyc[path[j]] = to_cyc[u] + step;
    }
}

bool on_cycle(int i) const { return on_cyc[i]; }
int dist_to_cycle(int i) const { return to_cyc[i]; }
int cycle_id(int i) const { return comp[i]; }
int cycle_length(int id) const { return clen[id]; }
int num_cycles() const { return static_cast<int>(clen.size()); }

int successor(int i, int64_t steps) const {
    if (steps <= to_cyc[i]) {
        return jump(i, steps);
    }
    steps -= to_cyc[i];
    i = jump(i, to_cyc[i]); // Advance to the cycle entry node.
    steps %= clen[comp[i]];
    return jump(i, steps);
}
};

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Cycle 0 -> 1 -> 2 -> 0, with a tail 5 -> 4 -> 3 -> 1 and 6 -> 2.
    vector<int> f{1, 2, 0, 1, 3, 4, 2};
    FunctionalGraph g(f);

    assert(g.on_cycle(0) && g.on_cycle(1) && g.on_cycle(2));
    assert(!g.on_cycle(3) && !g.on_cycle(5));
    assert(g.num_cycles() == 1);
    assert(g.cycle_length(0) == 3);

    assert(g.dist_to_cycle(2) == 0);
    assert(g.dist_to_cycle(5) == 3);
    assert(g.dist_to_cycle(6) == 1);
    assert(g.cycle_id(5) == g.cycle_id(0));

    assert(g.successor(5, 1) == 4);
    assert(g.successor(5, 3) == 1);    // 5 -> 4 -> 3 -> 1.
    assert(g.successor(5, 4) == 2);    // One more step around into the cycle.
    assert(g.successor(0, 100) == 1);  // 100 mod 3 == 1 step around the cycle.
    return 0;
}
```

4.2.10 Offline Dynamic Connectivity

Maintains the number of connected components of an undirected graph as edges are added and removed over time, answering queries offline once the entire sequence of operations is known. Each edge is alive over a contiguous interval of time, from when it is added until it is removed. These intervals are placed into a segment tree over the time axis, so that for T operations, each edge appears in $O(\log T)$ nodes whose ranges it spans. A depth-first traversal of this segment tree unites the edges stored at each node on the way down and undoes them on the way back up, so at each leaf the disjoint set forest reflects exactly the edges alive at that moment.

Undoing unions requires a disjoint set forest with rollback (see the disjoint sets section): it joins by size or rank without path compression, recording each change on a stack so it can be reversed. Path compression is incompatible with rollback, so finding a representative costs $O(\log n)$ rather than near-constant time.

- `OfflineDynamicConnectivity(n)` creates a structure of n nodes numbered $[0, n)$, initially with no edges.
- `add_edge(u, v)` and `remove_edge(u, v)` record adding or removing the undirected edge $u-v$ at the current time, then advance the time by one step. Each edge may be present at most once at a time: `add_edge` must not name an edge that is already present, and `remove_edge` must name one that is. Both preconditions are checked with `assert` during `solve()`.
- `count_components()` records a query at the current time and advances the time by one step.
- `solve()` processes all recorded operations and returns a vector holding, for each `count_components()` query in the order issued, the number of connected components at that time.

Time Complexity:

- $O((T + m \log T) \log n)$ for `solve()`, where T is the total number of operations (`add_edge`, `remove_edge`, and `count_components` combined), m is the number of `add_edge` operations, and n is the number of vertices.
- $O(1)$ amortized per call to `add_edge()`, `remove_edge()`, and `count_components()`.

Space Complexity: $O(n + m \log T)$ auxiliary heap space.

```
#include <algorithm>
#include <cassert>
#include <map>
#include <numeric>
```

```

#include <utility>
#include <vector>

class OfflineDynamicConnectivity {
    static const int ADD = 0, REMOVE = 1, QUERY = 2;

    struct Op {
        int type, u, v;
    };

    int n, comps;
    std::vector<Op> ops;
    std::vector<int> par, rnk;
    std::vector<std::pair<int, int>> undo; // (child root, whether its parent's rank increased)
    std::vector<std::vector<std::pair<int, int>>> seg;
    std::vector<int> answers;

    int find(int x) const {
        while (par[x] != x) {
            x = par[x];
        }
        return x;
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) {
            undo.emplace_back(-1, 0);
            return;
        }
        if (rnk[a] < rnk[b]) {
            std::swap(a, b);
        }
        undo.emplace_back(b, rnk[a] == rnk[b] ? 1 : 0);
        par[b] = a;
        if (rnk[a] == rnk[b]) {
            rnk[a]++;
        }
        comps--;
    }

    void rollback() {
        auto [b, raised] = undo.back();
        undo.pop_back();
        if (b == -1) {
            return;
        }
        rnk[par[b]] -= raised;
        par[b] = b;
        comps++;
    }

    void add_interval(int node, int lo, int hi, int l, int r, const std::pair<int, int> &e) {
        if (r < lo || hi < l) {
            return;
        }
        if (l <= lo && hi <= r) {
            seg[node].push_back(e);
            return;
        }
        int mid = lo + (hi - lo) / 2;
        add_interval(2 * node, lo, mid, l, r, e);
        add_interval(2 * node + 1, mid + 1, hi, l, r, e);
    }

    void dfs(int node, int lo, int hi) {
        int applied = 0;

```

```

    for (const auto &[eu, ev] : seg[node]) {
        unite(eu, ev);
        applied++;
    }
    if (lo == hi) {
        if (ops[lo].type == QUERY) {
            answers.push_back(comps);
        }
    } else {
        int mid = lo + (hi - lo) / 2;
        dfs(2 * node, lo, mid);
        dfs(2 * node + 1, mid + 1, hi);
    }
    while (applied-- > 0) {
        rollback();
    }
}

public:
OfflineDynamicConnectivity(int n) : n(n) {}

void add_edge(int u, int v) { ops.push_back({ADD, std::min(u, v), std::max(u, v)}); }
void remove_edge(int u, int v) { ops.push_back({REMOVE, std::min(u, v), std::max(u, v)}); }
void count_components() { ops.push_back({QUERY, 0, 0}); }

std::vector<int> solve() {
    int t = static_cast<int>(ops.size());
    answers.clear();
    if (t == 0) {
        return answers;
    }
    seg.assign(4 * t, {});
    std::map<std::pair<int, int>, int> active;
    for (int i = 0; i < t; i++) {
        std::pair<int, int> key(ops[i].u, ops[i].v);
        if (ops[i].type == ADD) {
            assert(active.find(key) == active.end()); // The edge must not already be present.
            active[key] = i;
        } else if (ops[i].type == REMOVE) {
            auto it = active.find(key);
            assert(it != active.end()); // The edge must currently be present.
            add_interval(1, 0, t - 1, it->second, i - 1, {ops[i].u, ops[i].v});
            active.erase(it);
        }
    }
    for (const auto &[key, start] : active) {
        add_interval(1, 0, t - 1, start, t - 1, {key.first, key.second});
    }
    par.resize(n);
    std::iota(par.begin(), par.end(), 0);
    rnk.assign(n, 0);
    comps = n;
    undo.clear();
    dfs(1, 0, t - 1);
    return answers;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    OfflineDynamicConnectivity dc(4);
    dc.count_components(); // 4 isolated vertices.
}

```

```

dc.add_edge(0, 1);
dc.add_edge(2, 3);
dc.count_components(); // {0,1} {2,3} -> 2 components.
dc.add_edge(1, 2);
dc.count_components(); // All four connected -> 1 component.
dc.remove_edge(1, 2);
dc.count_components(); // Back to 2 components.

assert((dc.solve() == vector<int>{4, 2, 1, 2}));
return 0;
}

```

4.3 Shortest Paths

4.3.1 Unweighted Shortest Path (BFS)

Given a starting node in an unweighted, directed graph, visit every connected node and determine the minimum distance to each such node. Breadth-first search explores nodes in order of increasing distance using a queue, so the first time a node is reached is via a shortest path. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor vector `pred`.

- `bfs(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

Time Complexity: $O(\max(n, m))$ per call to `bfs()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary heap space for `bfs()`.

```

#include <algorithm>
#include <climits>
#include <queue>
#include <vector>

const int INF = INT_MAX / 2;
std::vector<std::vector<int>> adj;
std::vector<int> dist, pred;

void bfs(int start) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    std::queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (dist[v] != INF) {
                continue;
            }
            dist[v] = dist[u] + 1;
            pred[v] = u;
        }
    }
}

```

```

    q.push(v);
  }
}
}

```

Example Usage

```

#include <iostream>
using namespace std;

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int nodes = 4, start = 0, dest = 3;
    adj.assign(nodes, {});
    adj[0].push_back(1);
    adj[1].push_back(2);
    adj[1].push_back(3);
    adj[2].push_back(3);
    bfs(start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << "."
         << endl;
    print_path(dest);
    return 0;
}

```

Example Output

```

The shortest distance from 0 to 3 is 2.
Take the path: 0->1->3.

```

4.3.2 0-1 Shortest Path (Deque BFS)

Given a starting node in a weighted graph whose edge weights are only 0 or 1, compute the shortest distance to every reachable node. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred`.

0-1 BFS is a specialized version of Dijkstra's algorithm. Because every relaxation changes the distance by either 0 or 1, a deque maintains nodes in nondecreasing distance order: push weight-0 relaxations to the front and weight-1 relaxations to the back.

- `bfs_zero_one(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`. Each edge is stored as `(neighbor, weight)`, where `weight` is either 0 or 1.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

Time Complexity: $O(\max(n, m))$ per call to `bfs_zero_one()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary deque space for `bfs_zero_one()`.

```
#include <algorithm>
#include <climits>
#include <deque>
#include <utility>
#include <vector>

const int INF = INT_MAX / 2;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int> dist, pred;

void bfs_zero_one(int start) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    std::deque<int> dq;
    dist[start] = 0;
    dq.push_front(start);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto &[v, w] : adj[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pred[v] = u;
                if (w == 0) {
                    dq.push_front(v);
                } else {
                    dq.push_back(v);
                }
            }
        }
    }
}
```

Example Usage

```
#include <iostream>
using namespace std;

void add_edge(int u, int v, int w) {
    adj[u].emplace_back(v, w);
}

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 3;
    adj.assign(4, {});
    add_edge(0, 1, 1);
    add_edge(0, 2, 0);
}
```

```

add_edge(2, 1, 0);
add_edge(1, 3, 1);
add_edge(2, 3, 1);
bfs_zero_one(start);
cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << ".\n";
print_path(dest);
return 0;
}

```

Example Output

The shortest distance from 0 to 3 is 1.
Take the path: 0->2->3.

4.3.3 Shortest Path (Dijkstra)

Given a starting node in a weighted, directed graph with nonnegative weights only, visit every connected node and determine the minimum distance to each such node. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred`.

Dijkstra's algorithm repeatedly selects the unvisited node of smallest tentative distance using a priority queue and relaxes its outgoing edges. Dijkstra's algorithm requires nonnegative edge weights. Use Bellman-Ford or SPFA instead when negative edges are present. Because the weights are nonnegative, a node's distance is final the first time it is removed from the queue.

- `dijkstra(start, adj, dist, pred)` returns a pair of vectors `dist` and `pred` for an adjacency list `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`. Each edge is stored as `(neighbor, weight)`, where `weight` is nonnegative.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

Time Complexity: $O(m \log n)$ for `dijkstra()`, where m is the number of edges and n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(m)$ auxiliary heap space for `dijkstra()`.

```

#include <cstdint>
#include <functional>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

template<typename T>
std::pair<std::vector<T>, std::vector<int>>> dijkstra(
    const std::vector<std::vector<std::pair<int, T>>> &adj, int start
) {
    const T INF = std::numeric_limits<T>::max() / 4;
    int n = static_cast<int>(adj.size());
    std::vector<T> dist(n, INF);
    std::vector<int> pred(n, -1);
    dist[start] = 0;
    std::priority_queue<
        std::pair<T, int>, std::vector<std::pair<T, int>>, std::greater<std::pair<T, int>>>
    > pq;
    pq.emplace(0, start);
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
    }
}

```



```

    if (du != dist[u]) {
        continue;
    }
    for (auto &[v, w] : adj[u]) {
        if (dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            pred[v] = u;
            pq.emplace(dist[v], v);
        }
    }
}
return {dist, pred};
}

```

Example Usage

```

#include <iostream>
using namespace std;

vector<vector<pair<int, int64_t>>> adj;

void print_path(const vector<int> &pred, int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 3;
    adj.assign(4, {});
    adj[0].emplace_back(1, 2);
    adj[0].emplace_back(3, 8);
    adj[1].emplace_back(2, 2);
    adj[1].emplace_back(3, 4);
    adj[2].emplace_back(3, 1);
    auto [dist, pred] = dijkstra(adj, start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << ".\n";
    print_path(pred, dest);
    return 0;
}

```

Example Output

```

The shortest distance from 0 to 3 is 5.
Take the path: 0->1->2->3.

```

4.3.4 Shortest Path (Bellman-Ford)

Given a starting node in a weighted, directed graph with possibly negative weights, visit every connected node and determine the minimum distance to each such node. Optionally, output the shortest path to a specific destination node using the shortest-path tree from the predecessor array `pred`.

Bellman-Ford relaxes every edge in the graph $n - 1$ times. Since any shortest path uses at most $n - 1$ edges, all distances are correct after these passes unless a negative-weight cycle keeps reducing them, which a further pass detects. Bellman-Ford will also detect whether the graph contains a negative-weight cycle reachable from the start

node, in which case the affected shortest paths are undefined and an error will be thrown. (To detect a negative cycle anywhere in the graph, add a virtual source with zero-weight edges to every node and start from it.)

- `bellman_ford(n, start)` populates `dist` and `pred` for a global, pre-populated edge list `edges` whose endpoints must be numbered $[0, n)$.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

Time Complexity: $O(n \cdot m)$ per call to `bellman_ford()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary heap space for `bellman_ford()`, where n is the number of nodes.

```
#include <stdint>
#include <stdexcept>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::tuple<int, int, int>> edges; // (u, v, w)
std::vector<int64_t> dist;
std::vector<int> pred;

void bellman_ford(int n, int start) {
    dist.assign(n, INF);
    pred.assign(n, -1);
    dist[start] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (auto &[u, v, w] : edges) {
            // The dist[u] != INF guard avoids relaxing out of unreachable nodes: a negative edge from an
            // unreachable u would otherwise give v a bogus finite distance (INF + w < INF).
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pred[v] = u;
            }
        }
    }
    // Optional: report a negative-weight cycle reachable from the start node.
    for (auto &[u, v, w] : edges) {
        if (dist[u] != INF && dist[v] > dist[u] + w) {
            throw std::runtime_error("Negative-weight cycle found.");
        }
    }
}
```

Example Usage

```
#include <iostream>
using namespace std;

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}
```

```

}

int main() {
    edges.emplace_back(0, 1, 1);
    edges.emplace_back(1, 2, 2);
    edges.emplace_back(0, 2, 5);
    int start = 0, dest = 2;
    bellman_ford(3, start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << ".\n";
    print_path(dest);
    return 0;
}

```

Example Output

The shortest distance from 0 to 2 is 3.
Take the path: 0->1->2.

4.3.5 Shortest Path (SPFA)

Given a starting node in a weighted directed graph, compute shortest paths even when some edge weights are negative. The Shortest Path Faster Algorithm (SPFA) is a queue-based optimization of Bellman-Ford: instead of relaxing every edge in every round, it keeps a queue of nodes whose distances have improved and relaxes only their outgoing edges. It is often fast on benign inputs, but it still has Bellman-Ford's worst-case behavior and can be forced to run in $O(n \cdot m)$. Prefer Dijkstra for nonnegative weights, and use SPFA mainly when negative edges are present and the input is not adversarial.

- `spfa(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`. Each edge is stored as `(neighbor, weight)`. The function returns `false` if it detects a reachable negative cycle, and returns `true` otherwise.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

Time Complexity: $O(n \cdot m)$ in the worst case for `spfa()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary queue space for `spfa()`.

```

#include <algorithm>
#include <cstdint>
#include <queue>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int64_t> dist;
std::vector<int> pred;

bool spfa(int start) {
    int n = static_cast<int>(adj.size());
    dist.assign(n, INF);
    pred.assign(n, -1);
    std::vector<int> relax_count(n, 0);
    std::vector<bool> in_queue(n, false);
    in_queue.assign(n, false);
    std::queue<int> q;
    dist[start] = 0;

```

```

q.push(start);
in_queue[start] = true;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    in_queue[u] = false;
    for (auto &[v, w] : adj[u]) {
        if (dist[u] != INF && dist[v] > dist[u] + w) {
            dist[v] = dist[u] + w;
            pred[v] = u;
            if (!in_queue[v]) {
                q.push(v);
                in_queue[v] = true;
                if (++relax_count[v] >= n) {
                    return false;
                }
            }
        }
    }
}
return true;
}

```

Example Usage

```

#include <iostream>
using namespace std;

void add_edge(int u, int v, int w) {
    adj[u].emplace_back(v, w);
}

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 3;
    adj.assign(4, {});
    add_edge(0, 1, 4);
    add_edge(0, 2, 5);
    add_edge(1, 2, -2);
    add_edge(2, 3, 3);
    spfa(start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << ".\n";
    print_path(dest);
    return 0;
}

```

Example Output

The shortest distance from 0 to 3 is 5.
 Take the path: 0->1->2->3.

4.3.6 DAG Shortest Path (Topological DFS)

Given a starting node in a weighted, directed acyclic graph (DAG), determine the minimum distance to every reachable node. Because the graph is acyclic, its nodes admit a topological ordering in which every edge points forward. Relaxing edges in this order settles each node's distance in a single pass, with no priority queue and no restriction on edge signs. This makes it both faster than Dijkstra's algorithm and able to handle negative edge weights, as long as the graph has no cycles. The same pass computes longest paths if the relaxation comparison is reversed, which is the standard way to find the critical path in a schedule of dependent tasks.

- `dag_shortest_path(start)` populates `dist` and `pred` for a global, pre-populated adjacency list `adj`, whose nodes are numbered $[0, n)$, where `n` is `adj.size()`. Each edge is stored as `(neighbor, weight)` and may have any sign. `dist[v]` is set to `INF` for nodes not reachable from `start`, and `pred` stores the shortest-path tree for path reconstruction.

For path reconstruction, `pred[v]` stores the node immediately before `v` on the shortest path from `start` to `v`, or `-1` if `v` is `start` or unreachable. Follow `pred` backward from the destination to `start`, then reverse that sequence to recover the path.

Time Complexity: $O(n + m)$ per call to `dag_shortest_path()`, where n is the number of nodes and m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph.
- $O(n)$ auxiliary heap space for `dag_shortest_path()`.

```
#include <algorithm>
#include <cstdint>
#include <utility>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<std::pair<int, int>>> adj;
std::vector<int64_t> dist;
std::vector<int> pred;

void topological_dfs(int u, std::vector<bool> &visit, std::vector<int> &order) {
    visit[u] = true;
    for (auto &[v, w] : adj[u]) {
        if (!visit[v]) {
            topological_dfs(v, visit, order);
        }
    }
    order.push_back(u);
}

void dag_shortest_path(int start) {
    int n = static_cast<int>(adj.size());
    std::vector<bool> visit(n, false);
    std::vector<int> order;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            topological_dfs(i, visit, order);
        }
    }
    std::reverse(order.begin(), order.end());
    dist.assign(n, INF);
    pred.assign(n, -1);
    dist[start] = 0;
    for (int u : order) {
        if (dist[u] == INF) {
            continue;
        }
        for (auto &[v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
```

```

        dist[v] = dist[u] + w;
        pred[v] = u;
    }
}
}
}

```

Example Usage

```

#include <iostream>
using namespace std;

void print_path(int dest) {
    vector<int> path;
    for (int j = dest; pred[j] != -1; j = pred[j]) {
        path.push_back(pred[j]);
    }
    cout << "Take the path: ";
    while (!path.empty()) {
        cout << path.back() << "->";
        path.pop_back();
    }
    cout << dest << "." << endl;
}

int main() {
    int start = 0, dest = 4;
    adj.assign(5, {});
    adj[0] = {{1, 1}, {2, 5}};
    adj[1] = {{2, 1}, {3, -2}}; // A negative edge, which Dijkstra could not handle.
    adj[2] = {{3, 1}};
    adj[3] = {{4, 3}};
    dag_shortest_path(start);
    cout << "The shortest distance from " << start << " to " << dest << " is " << dist[dest] << ".\n";
    print_path(dest);
    return 0;
}

```

Example Output

The shortest distance from 0 to 4 is 2.
Take the path: 0->1->3->4.

4.3.7 All-Pairs Shortest Path (Floyd-Warshall)

Given a weighted, directed graph with possibly negative weights, determine the minimum distance between all pairs of start and destination nodes in the graph. Optionally, output the shortest path between two nodes using the next-hop matrix precomputed `next_node`.

Floyd-Warshall's algorithm is a dynamic program over intermediate nodes: for each node k in turn, every pair (i, j) is relaxed by considering a path through k , so once all k have been processed the matrix holds the all-pairs shortest distances.

- `init_floyd(n)` initializes `dist` and `next_node` for a graph of n nodes numbered $[0, n)$.
- `floyd_warshall()` updates the global adjacency matrix `dist` so `dist[u][v]` stores the shortest path from u to v , and updates `next_node` for path reconstruction. If the graph contains negative-weighted cycles, there is no shortest path and an error will be thrown.

For path reconstruction, `next_node[i][j]` stores the next node to visit after i on a current shortest path from i to j . It is initialized to j for every pair and, when a shorter route $i \rightarrow k \rightarrow j$ is found, becomes `next_node[i][k]`. Repeatedly setting i to `next_node[i][j]` therefore walks the path from source to destination.

Time Complexity:

- $O(n^2)$ per call to `init_floyd()`, where n is the number of nodes.
- $O(n^3)$ per call to `floyd_warshall()`.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(n^2)$ auxiliary heap space for `init_floyd()` and `floyd_warshall()`.

```
#include <stdint>
#include <stdexcept>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<int64_t>>> dist;
std::vector<std::vector<int>>> next_node;

void init_floyd(int n) {
    dist.assign(n, std::vector<int64_t>(n));
    next_node.assign(n, std::vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = (i == j) ? 0 : INF;
            next_node[i][j] = j;
        }
    }
}

void floyd_warshall() {
    int n = static_cast<int>(dist.size());
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                // The INF guards avoid relaxing through an unreachable intermediate: with a negative edge,
                // INF + w < INF would otherwise give an unreachable pair a bogus finite distance.
                if (dist[i][k] != INF && dist[k][j] != INF && dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    next_node[i][j] = next_node[i][k];
                }
            }
        }
    }
    // Optional: Report negative-weighted cycles.
    for (int i = 0; i < n; i++) {
        if (dist[i][i] < 0) {
            throw std::runtime_error("Negative-weight cycle found.");
        }
    }
}
```

Example Usage

```
#include <iostream>
using namespace std;

void print_path(int u, int v) {
    cout << "Take the path: " << u;
    while (u != v) {
        u = next_node[u][v];
        cout << "->" << u;
    }
    cout << "." << endl;
}

int main() {
```

```

init_floyd(3);
dist[0][1] = 1;
dist[1][2] = 2;
dist[0][2] = 5;
floyd_warshall();
int start = 0, dest = 2;
cout << "The shortest distance from " << start << " to " << dest << " is " << dist[start][dest]
    << ".\n";
print_path(start, dest);
return 0;
}

```

Example Output

The shortest distance from 0 to 2 is 3.
Take the path: 0->1->2.

4.4 Spanning Trees

4.4.1 Minimum Spanning Tree (Prim)

Given a connected, undirected, weighted graph with possibly negative weights, its minimum spanning tree (MST) is a subgraph which is a tree that connects all nodes with a subset of its edges such that their total weight is minimized. If the input graph is not connected, then this implementation will find the minimum spanning forest.

Prim's algorithm grows the tree from an arbitrary start node, repeatedly adding the minimum-weight edge that joins a new node to the current tree, with a priority queue supplying the cheapest such edge at each step.

- `prim_mst()` populates `mst` with the edge IDs in the minimum spanning tree (returning the total MST weight) for a global, bidirectionally pre-populated adjacency list `adj` which must consist of nodes numbered `[0, n)`, where `n` is `adj.size()`. Edges are stored as `(neighbor, weight, edge_id)`.

The priority queue stores candidate edges as `(weight, from, to, edge_id)` and uses `std::greater` to make it a min-heap. To find a maximum spanning tree instead, use the default max-heap ordering. Multigraphs are supported; parallel edges are stored as separate adjacency entries and the algorithm automatically selects the minimum-weight one to each unvisited node.

Time Complexity: $O(m \log n)$ per call to `prim_mst()`, where m is the number of edges and n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(m)$ auxiliary heap space for `prim_mst()`.

```

#include <functional>
#include <queue>
#include <tuple>
#include <utility>
#include <vector>

std::vector<std::vector<std::tuple<int, int, int>>> adj; // adj[u] = {(v, weight, edge_id), ...}
std::vector<int> mst;

int prim_mst() {
    int n = static_cast<int>(adj.size());
    mst.clear();
    std::vector<bool> visit(n);
    int total_dist = 0;
    for (int i = 0; i < n; i++) {
        if (visit[i]) {
            continue;

```



```

    }
    visit[i] = true;
    using qnode = std::tuple<int, int, int, int>; // (weight, u, v, edge_id)
    std::priority_queue<qnode, std::vector<qnode>, std::greater<qnode>> pq;
    for (auto &[v, w, id] : adj[i]) {
        pq.emplace(w, i, v, id);
    }
    while (!pq.empty()) {
        auto [w, u, v, id] = pq.top();
        pq.pop();
        if (visit[u] && !visit[v]) {
            visit[v] = true;
            if (v != i) {
                mst.push_back(id);
                total_dist += w;
            }
            for (auto &[nb, ew, eid] : adj[v]) {
                pq.emplace(ew, v, nb, eid);
            }
        }
    }
}
}
return total_dist;
}

```

Example Usage

```

#include <iostream>
using namespace std;

vector<pair<int, int>> edge_endpoints;

void add_edge(int u, int v, int w) {
    int id = static_cast<int>(edge_endpoints.size());
    edge_endpoints.emplace_back(u, v);
    adj[u].emplace_back(v, w, id);
    adj[v].emplace_back(u, w, id);
}

int main() {
    int nodes = 7;
    adj.assign(nodes, {});
    add_edge(0, 1, 4);
    add_edge(1, 2, 6);
    add_edge(2, 0, 3);
    add_edge(3, 4, 1);
    add_edge(4, 5, 2);
    add_edge(5, 6, 3);
    add_edge(6, 4, 4);
    cout << "Total distance: " << prim_mst() << endl;
    for (int id : mst) {
        auto &[u, v] = edge_endpoints[id];
        cout << u << " <-> " << v << endl;
    }
    return 0;
}

```

Example Output

```

Total distance: 13
0 <-> 2
0 <-> 1
3 <-> 4
4 <-> 5
5 <-> 6

```

4.4.2 Minimum Spanning Tree (Kruskal)

Given a connected, undirected, weighted graph with possibly negative weights, its minimum spanning tree (MST) is a subgraph which is a tree that connects all nodes with a subset of its edges such that their total weight is minimized. If the input graph is not connected, then this implementation will find the minimum spanning forest.

Kruskal's algorithm scans the edges in nondecreasing weight order, adding each edge whose endpoints lie in different components (tracked with a disjoint-set structure) and skipping any edge that would form a cycle.

- `kruskal_mst(n)` populates `mst` with the edge IDs in the minimum spanning tree (returning the total MST weight) for a global, pre-populated edge list `edges` whose endpoints must be numbered $[0, n)$. Each edge is stored as `(weight, u, v)`.

Multigraphs are supported; parallel edges appear as separate entries in `edges` and any redundant ones are skipped once the cheaper edge has already connected the two components.

Time Complexity: $O(m \log n)$ per call to `kruskal_mst()`, where m is the number of edges and n is the number of nodes. The internal DSU uses path compression and union by size, making each DSU operation $O(\alpha(n))$ amortized.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges
- $O(n)$ auxiliary stack space for `kruskal_mst()`.

```
#include <algorithm>
#include <numeric>
#include <tuple>
#include <utility>
#include <vector>

std::vector<std::tuple<int, int, int>> edges; // (weight, u, v)
std::vector<int> dsu_root, dsu_size, mst;

int find_root(int u) {
    if (dsu_root[u] != u) {
        dsu_root[u] = find_root(dsu_root[u]);
    }
    return dsu_root[u];
}

int kruskal_mst(int n) {
    mst.clear();
    std::vector<int> order(edges.size());
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [](int a, int b) {
        return std::get<0>(edges[a]) < std::get<0>(edges[b]);
    });
    int total_dist = 0;
    dsu_root.assign(n, 0);
    dsu_size.assign(n, 1);
    std::iota(dsu_root.begin(), dsu_root.end(), 0);
    for (int id : order) {
        auto &[w, a, b] = edges[id];
        int u = find_root(a), v = find_root(b);
        if (u != v) {
            if (dsu_size[u] < dsu_size[v]) {
                std::swap(u, v);
            }
            dsu_root[v] = u;
            dsu_size[u] += dsu_size[v];
            mst.push_back(id);
            total_dist += w;
        }
    }
    return total_dist;
}
```

```
}
```

Example Usage

```
#include <iostream>
using namespace std;

void add_edge(int u, int v, int w) {
    edges.emplace_back(w, u, v);
}

int main() {
    add_edge(0, 1, 4);
    add_edge(1, 2, 6);
    add_edge(2, 0, 3);
    add_edge(3, 4, 1);
    add_edge(4, 5, 2);
    add_edge(5, 6, 3);
    add_edge(6, 4, 4);
    cout << "Total distance: " << kruskal_mst(7) << endl;
    for (int id : mst) {
        auto &[w, u, v] = edges[id];
        cout << u << " <-> " << v << endl;
    }
    return 0;
}
```

Example Output

```
Total distance: 13
3 <-> 4
4 <-> 5
2 <-> 0
5 <-> 6
0 <-> 1
```

4.4.3 Minimum Spanning Arborescence (Edmonds)

A spanning arborescence rooted at a node `root` of a weighted, directed graph is a set of edges forming a tree in which every other node is reachable from the root along directed edges, that is, every node except the root has exactly one incoming edge. The minimum spanning arborescence minimizes the total edge weight, and is the directed analog of the minimum spanning tree.

Edmonds' (a.k.a. the Chu-Liu/Edmonds) algorithm computes the arborescence by first selecting, for each non-root node, its cheapest incoming edge. If these choices form no cycle, they already constitute the answer. Otherwise each cycle is contracted into a single supernode, and every edge entering the cycle has its weight reduced by the cost of the edge it would replace inside the cycle. Solving the problem on the contracted graph and expanding the cycles back yields the optimum. The implementation below repeats this contraction in rounds, accumulating the selected weights, until no cycle remains.

- `directed_mst(n, root, edges)` returns the total weight of the minimum spanning arborescence rooted at `root` over `n` nodes numbered `[0, n)`, or `-1` if some node is unreachable from the root (no arborescence exists). Edges are given as `(from, to, weight)` triples; parallel edges and self-loops are allowed, with self-loops simply ignored.

Time Complexity: $O(n \cdot m)$ per call, where n is the number of nodes and m is the number of edges.

Space Complexity: $O(n + m)$ auxiliary heap space.

```
#include <cstdint>
#include <vector>
```

```

struct Edge {
    int u, v;
    int64_t weight;
};

int64_t directed_mst(int n, int root, std::vector<Edge> edges) {
    const int64_t INF = INT64_MAX / 4;
    int64_t result = 0;
    while (true) {
        std::vector<int64_t> min_in(n, INF);
        std::vector<int> pre(n, -1);
        for (const Edge &e : edges) {
            if (e.u != e.v && e.weight < min_in[e.v]) {
                min_in[e.v] = e.weight;
                pre[e.v] = e.u;
            }
        }
        for (int v = 0; v < n; v++) {
            if (v != root && min_in[v] == INF) {
                return -1; // Node v has no incoming edge, so it is unreachable.
            }
        }
        // Identify cycles formed by the chosen incoming edges, labeling each with an ID in `comp`.
        int cycles = 0;
        std::vector<int> comp(n, -1), seen(n, -1);
        min_in[root] = 0;
        for (int v = 0; v < n; v++) {
            result += min_in[v];
            int u = v;
            while (seen[u] != v && comp[u] == -1 && u != root) {
                seen[u] = v;
                u = pre[u];
            }
            if (u != root && comp[u] == -1) {
                for (int w = pre[u]; w != u; w = pre[w]) {
                    comp[w] = cycles;
                }
                comp[u] = cycles++;
            }
        }
        if (cycles == 0) {
            break; // No cycle remains; the selected edges form the arborescence.
        }
        // Contract each cycle into a supernode and reduce the weights of edges entering it.
        for (int v = 0; v < n; v++) {
            if (comp[v] == -1) {
                comp[v] = cycles++;
            }
        }
        for (Edge &e : edges) {
            int u = comp[e.u], v = comp[e.v];
            if (u != v) {
                e.weight -= min_in[e.v];
            }
            e.u = u;
            e.v = v;
        }
        n = cycles;
        root = comp[root];
    }
    return result;
}

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Root 0. Cheapest incoming edges 1<-0 (1), 2<-1 (2), 3<-2 (3) form a tree of weight 6.
    vector<Edge> edges{
        {0, 1, 1}, {0, 2, 5}, {1, 2, 2}, {2, 3, 3}, {3, 1, 4},
    };
    assert(directed_mst(4, 0, edges) == 6);

    // Node 3 is unreachable from root 0.
    vector<Edge> disconnected{{0, 1, 1}, {1, 2, 1}};
    assert(directed_mst(4, 0, disconnected) == -1);
    return 0;
}
```

4.5 Flows and Cuts

4.5.1 Maximum Flow (Ford-Fulkerson)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

The Ford-Fulkerson method repeatedly finds any augmenting path from source to sink in the residual graph (here by depth-first search) and pushes as much flow as the path allows, until no augmenting path remains. This algorithm should only be used on graphs with integer capacities, as there exists certain real-valued flow inputs for which the algorithm never terminates. The Edmonds-Karp algorithm is an improvement using breadth-first search, addressing this problem.

- `ford_fulkerson()` uses global `source` and `sink`, modifies the global residual capacity matrix `cap`, and returns maximum flow. Nodes are numbered $[0, n)$, where `n` is `cap.size()`.

Time Complexity: $O(n^2 \cdot f)$ per call to `ford_fulkerson()`, where n is the number of nodes and f is the maximum flow.

Space Complexity:

- $O(n^2)$ for storage of the flow network, where n is the number of nodes.
- $O(n)$ auxiliary stack space for `ford_fulkerson()`.

```
#include <algorithm>
#include <climits>
#include <cstdint>
#include <vector>

const int INF = INT_MAX / 2;
std::vector<std::vector<int>>> cap;
std::vector<bool> visit;

int dfs(int u, int f, int sink) {
    int n = static_cast<int>(cap.size());
    if (u == sink) {
        return f;
    }
    visit[u] = true;
    for (int v = 0; v < n; v++) {
        if (!visit[v] && cap[u][v] > 0) {
```

```

    int flow = dfs(v, std::min(f, cap[u][v]), sink);
    if (flow > 0) {
        cap[u][v] -= flow;
        cap[v][u] += flow;
        return flow;
    }
}
}
return 0;
}

int64_t ford_fulkerson(int source, int sink) {
    int n = static_cast<int>(cap.size());
    int64_t max_flow = 0;
    while (true) {
        visit.assign(n, false);
        int flow = dfs(source, INF, sink);
        if (flow == 0) {
            break;
        }
        max_flow += flow;
    }
    return max_flow;
}

```

Example Usage

```

#include <cassert>

int main() {
    int nodes = 6;
    cap.assign(nodes, std::vector<int>(nodes));
    // Example graph after max flow, with each edge labeled flow/capacity:
    //      2/2
    //      1 -----> 3
    //      / \      |
    // 3/4 /   \ 1/1 | 2/2
    //   /     v     v
    //  0      4 ----> 5
    //   \     ^   3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    cap[0][1] = 4;
    cap[0][2] = 3;
    cap[1][3] = 2;
    cap[1][4] = 1;
    cap[2][4] = 2;
    cap[3][5] = 2;
    cap[4][5] = 3;
    assert(ford_fulkerson(0, 5) == 5);
    return 0;
}

```

4.5.2 Maximum Flow (Edmonds-Karp)

Given a flow network with integer or floating-point capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Edmonds-Karp is the Ford-Fulkerson method with the augmenting path always chosen as a shortest one (fewest edges) via breadth-first search. This bounds the number of augmentations at $O(n \cdot m)$, independent of the capacity magnitudes.

- `EdmondsKarp<T>(n)` constructs an empty flow network with nodes numbered $[0, n)$.
- `add_edge(u, v, cap)` adds a directed residual-network edge from `u` to `v`.
- `max_flow(source, sink)` modifies the residual network and returns maximum flow.
- `clear_flow()` resets all edge flows to zero, allowing a fresh recomputation on the same graph.
- `min_cut(source)` returns the source side of a minimum cut after `max_flow()` has been called.

Repeated calls to `max_flow()` continue augmenting from the current flow. This is useful after adding new edges or increasing capacities; call `clear_flow()` first to recompute from zero. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Time Complexity:

- $O(\min(n \cdot m^2, m \cdot f))$ per call to `max_flow()`, where n is the number of nodes, m is the number of edges, and f is the maximum flow.
- $O(n + m)$ per call to `min_cut()`.

Space Complexity: $O(\max(n, m))$ for storage of the flow network, where n is the number of nodes and m is the number of edges.

```
#include <algorithm>
#include <limits>
#include <queue>
#include <vector>

template<typename T>
class EdmondsKarp {
    struct Edge {
        int u, v, rev;
        T cap, flow;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    T flow;
    std::vector<std::vector<Edge>> adj;

public:
    explicit EdmondsKarp(int n) : nodes(n), flow(0), adj(n) {}

    void add_edge(int u, int v, T cap) {
        adj[u].push_back(Edge{u, v, static_cast<int>(adj[v].size()), cap, 0});
        adj[v].push_back(Edge{v, u, static_cast<int>(adj[u].size()) - 1, 0, 0});
    }

    void clear_flow() {
        for (auto &edges : adj) {
            for (Edge &e : edges) {
                e.flow = 0;
            }
        }
        flow = 0;
    }

    T max_flow(int source, int sink) {
        while (true) {
            std::vector<Edge *> pred(nodes, nullptr);
            std::queue<int> q;
            q.push(source);
            while (!q.empty() && pred[sink] == nullptr) {
                int u = q.front();
```

```

    q.pop();
    for (Edge &e : adj[u]) {
        if (e.v != source && pred[e.v] == nullptr && residual(e)) {
            pred[e.v] = &e;
            q.push(e.v);
        }
    }
}
if (pred[sink] == nullptr) {
    break;
}
T flow = std::numeric_limits<T>::max();
for (int u = sink; u != source; u = pred[u]->u) {
    flow = std::min(flow, pred[u]->cap - pred[u]->flow);
}
for (int u = sink; u != source; u = pred[u]->u) {
    pred[u]->flow += flow;
    adj[pred[u]->v][pred[u]->rev].flow -= flow;
}
this->flow += flow;
}
return flow;
}

std::vector<bool> min_cut(int source) const {
    std::vector<bool> reachable(nodes);
    std::queue<int> q;
    reachable[source] = true;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const Edge &e : adj[u]) {
            if (!reachable[e.v] && residual(e)) {
                reachable[e.v] = true;
                q.push(e.v);
            }
        }
    }
    return reachable;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    EdmondsKarp<int> g(6);
    // Example graph after max flow, with each edge labeled flow/capacity:
    //      2/2
    //      1 -----> 3
    //      / \      |
    // 3/4 /   \ 1/1 | 2/2
    //   /     v     v
    //  0      4 ----> 5
    //   \     ^     3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    g.add_edge(0, 1, 4);
    g.add_edge(0, 2, 3);
    g.add_edge(1, 3, 2);
    g.add_edge(1, 4, 1);
    g.add_edge(2, 4, 2);
    g.add_edge(3, 5, 2);
}

```



```

g.add_edge(4, 5, 3);
assert(g.max_flow(0, 5) == 5);
assert(g.max_flow(0, 5) == 5);
std::vector<bool> cut = g.min_cut(0);
assert(cut[0] && !cut[5]);
g.clear_flow();
assert(g.max_flow(0, 5) == 5);
return 0;
}

```

4.5.3 Maximum Flow (Dinic)

Given a flow network with integer or floating-point capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Dinic's algorithm proceeds in phases. Each phase runs a breadth-first search to build a level graph of shortest residual distances from the source, then finds a blocking flow that saturates that level graph using depth-first search, before repeating on the new residual graph.

- `Dinic<T>(n)` constructs an empty flow network with nodes numbered $[0, n)$.
- `add_edge(u, v, cap)` adds a directed residual-network edge from `u` to `v`.
- `max_flow(source, sink)` modifies the residual network and returns maximum flow.
- `clear_flow()` resets all edge flows to zero, allowing a fresh recomputation on the same graph.
- `min_cut(source)` returns the source side of a minimum cut after `max_flow()` has been called.

Repeated calls to `max_flow()` continue augmenting from the current flow. This is useful after adding new edges or increasing capacities; call `clear_flow()` first to recompute from zero. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Time Complexity:

- $O(n^2 \cdot m)$ per call to `max_flow()`, where n is the number of nodes and m is the number of edges.
- $O(n + m)$ per call to `min_cut()`.

Space Complexity:

- $O(\max(n, m))$ for storage of the flow network.
- $O(n)$ auxiliary stack and heap space for `max_flow()`.

```

#include <algorithm>
#include <limits>
#include <queue>
#include <vector>

template<typename T>
class Dinic {
    struct Edge {
        int v, rev;
        T cap, flow;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    T flow;
    std::vector<std::vector<Edge>> adj;
    std::vector<int> dist, ptr;

    bool bfs(int source, int sink) {
        dist.assign(nodes, -1);
        dist[source] = 0;

```

```

std::queue<int> q;
q.push(source);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (const Edge &e : adj[u]) {
        if (dist[e.v] < 0 && residual(e)) {
            dist[e.v] = dist[u] + 1;
            q.push(e.v);
        }
    }
}
return dist[sink] >= 0;
}

T dfs(int u, T pushed, int sink) {
    if (u == sink || pushed <= EPS) {
        return pushed;
    }
    for (; ptr[u] < static_cast<int>(adj[u].size()); ptr[u]++) {
        Edge &e = adj[u][ptr[u]];
        if (dist[e.v] == dist[u] + 1 && residual(e)) {
            T pushed_next = dfs(e.v, std::min(pushed, e.cap - e.flow), sink);
            if (pushed_next > EPS) {
                e.flow += pushed_next;
                adj[e.v][e.rev].flow -= pushed_next;
                return pushed_next;
            }
        }
    }
    return 0;
}

public:
explicit Dinic(int n) : nodes(n), flow(0), adj(n), dist(n), ptr(n) {}

void add_edge(int u, int v, T cap) {
    adj[u].push_back(Edge{v, static_cast<int>(adj[v].size()), cap, 0});
    adj[v].push_back(Edge{u, static_cast<int>(adj[u].size()) - 1, 0, 0});
}

void clear_flow() {
    for (auto &edges : adj) {
        for (Edge &e : edges) {
            e.flow = 0;
        }
    }
    flow = 0;
}

T max_flow(int source, int sink) {
    while (bfs(source, sink)) {
        ptr.assign(nodes, 0);
        while (true) {
            T pushed = dfs(source, std::numeric_limits<T>::max(), sink);
            if (pushed <= EPS) {
                break;
            }
            flow += pushed;
        }
    }
    return flow;
}

std::vector<bool> min_cut(int source) const {
    std::vector<bool> reachable(nodes);
    std::queue<int> q;
    reachable[source] = true;

```

```

    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const Edge &e : adj[u]) {
            if (!reachable[e.v] && residual(e)) {
                reachable[e.v] = true;
                q.push(e.v);
            }
        }
    }
    return reachable;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    Dinic<int> g(6);
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //           1 -----> 3
    //          / \      |
    // 3/4 /   \ 1/1   | 2/2
    //   /       v      v
    //  0       4 ----> 5
    //   \      ^   3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    g.add_edge(0, 1, 4);
    g.add_edge(0, 2, 3);
    g.add_edge(1, 3, 2);
    g.add_edge(1, 4, 1);
    g.add_edge(2, 4, 2);
    g.add_edge(3, 5, 2);
    g.add_edge(4, 5, 3);
    assert(g.max_flow(0, 5) == 5);
    assert(g.max_flow(0, 5) == 5);
    std::vector<bool> cut = g.min_cut(0);
    assert(cut[0] && !cut[5]);
    g.clear_flow();
    assert(g.max_flow(0, 5) == 5);
    return 0;
}

```

4.5.4 Maximum Flow (Dinic with Decomposition)

Given a flow network with integer or floating-point capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Dinic's algorithm proceeds in phases. Each phase runs a breadth-first search to build a level graph of shortest residual distances from the source, then finds a blocking flow that saturates that level graph using depth-first search, before repeating on the new residual graph. This variant stores residual edges in a flat array so callers can inspect edge flows and decompose the final flow into source-to-sink paths.

- `Dinic<T>(n)` constructs an empty flow network with nodes numbered `[0, n)`.
- `add_edge(u, v, cap)` adds a directed residual-network edge from `u` to `v` and returns its edge ID.
- `edge_flow(id)` returns the flow through a previously added edge.

- `max_flow(source, sink)` modifies the residual network and returns maximum flow.
- `clear_flow()` resets all edge flows to zero, allowing a fresh recomputation on the same graph.
- `min_cut(source)` returns the source side of a minimum cut after `max_flow()` has been called.
- `decompose(source, sink)` decomposes the current flow into paths from `source` to `sink`.

Repeated calls to `max_flow()` continue augmenting from the current flow. This is useful after adding new edges or increasing capacities; call `clear_flow()` first to recompute from zero. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Time Complexity:

- $O(n^2 \cdot m)$ per call to `max_flow()`, where n is the number of nodes and m is the number of edges.
- $O(n + m)$ per call to `min_cut()`.
- $O(p \cdot m)$ per call to `decompose()`, where p is the number of paths returned.

Space Complexity:

- $O(\max(n, m))$ for storage of the flow network.
- $O(n)$ auxiliary stack and heap space for `max_flow()`.
- $O(n + m)$ auxiliary heap space for `decompose()`, not counting the returned paths.

```
#include <algorithm>
#include <functional>
#include <limits>
#include <queue>
#include <vector>

template<typename T>
class Dinic {
    struct Edge {
        int u, v;
        T cap, flow;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    T flow;
    std::vector<Edge> edges;
    std::vector<std::vector<int>>> adj;
    std::vector<int> dist, ptr;

    bool bfs(int source, int sink) {
        dist.assign(nodes, -1);
        dist[source] = 0;
        std::queue<int> q;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (dist[e.v] < 0 && residual(e)) {
                    dist[e.v] = dist[u] + 1;
                    q.push(e.v);
                }
            }
        }
        return dist[sink] >= 0;
    }

    T dfs(int u, T pushed, int sink) {
        if (u == sink || pushed <= EPS) {
            return pushed;
        }
    }
```

```

    for (; ptr[u] < static_cast<int>(adj[u].size()); ptr[u]++) {
        int id = adj[u][ptr[u]];
        Edge &e = edges[id];
        if (dist[e.v] == dist[u] + 1 && residual(e)) {
            T flow = dfs(e.v, std::min(pushed, e.cap - e.flow), sink);
            if (flow > EPS) {
                e.flow += flow;
                edges[id ^ 1].flow -= flow;
                return flow;
            }
        }
    }
    return 0;
}

public:
    struct FlowPath {
        T flow;
        std::vector<int> vertices;
        std::vector<int> edges;
    };

    explicit Dinic(int n) : nodes(n), flow(0), adj(n), dist(n), ptr(n) {}

    int add_edge(int u, int v, T cap) {
        int id = static_cast<int>(edges.size());
        adj[u].push_back(id);
        edges.push_back(Edge{u, v, cap, 0});
        adj[v].push_back(id + 1);
        edges.push_back(Edge{v, u, 0, 0});
        return id;
    }

    T edge_flow(int id) const { return edges[id].flow; }

    void clear_flow() {
        for (Edge &e : edges) {
            e.flow = 0;
        }
        flow = 0;
    }

    T max_flow(int source, int sink) {
        while (bfs(source, sink)) {
            ptr.assign(nodes, 0);
            while (true) {
                T flow = dfs(source, std::numeric_limits<T>::max(), sink);
                if (flow <= EPS) {
                    break;
                }
                this->flow += flow;
            }
        }
        return flow;
    }

    std::vector<bool> min_cut(int source) const {
        std::vector<bool> reachable(nodes);
        std::queue<int> q;
        reachable[source] = true;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (!reachable[e.v] && residual(e)) {
                    reachable[e.v] = true;
                }
            }
        }
    }

```

```

        q.push(e.v);
    }
}
}
return reachable;
}

std::vector<FlowPath> decompose(int source, int sink) const {
    std::vector<T> rem(edges.size());
    for (int id = 0; id < static_cast<int>(edges.size()); id++) {
        rem[id] = edges[id].flow;
    }
    std::vector<FlowPath> res;
    while (true) {
        FlowPath path{std::numeric_limits<T>::max(), {source}, {}};
        std::vector<bool> seen(nodes);
        std::function<bool(int)> find_path = [&](int u) {
            if (u == sink) {
                return true;
            }
            seen[u] = true;
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (rem[id] > EPS && !seen[e.v]) {
                    path.edges.push_back(id);
                    path.vertices.push_back(e.v);
                    if (find_path(e.v)) {
                        return true;
                    }
                    path.vertices.pop_back();
                    path.edges.pop_back();
                }
            }
            return false;
        };
        if (!find_path(source)) {
            break;
        }
        for (int id : path.edges) {
            path.flow = std::min(path.flow, rem[id]);
        }
        for (int id : path.edges) {
            rem[id] -= path.flow;
        }
        res.push_back(path);
    }
    return res;
}
};

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    Dinic<int> g(6);
    // Example graph after max flow, with each edge labeled flow/capacity:
    //      2/2
    //      1 -----> 3
    //      / \      |
    // 3/4 /   \ 1/1 | 2/2
    //    /     v     v
    // 0      4 ----> 5
    //    \     ^     3/3

```

```

// 2/3 \   / 2/2
//      v /
//      2
int id01 = g.add_edge(0, 1, 4);
int id02 = g.add_edge(0, 2, 3);
int id13 = g.add_edge(1, 3, 2);
int id14 = g.add_edge(1, 4, 1);
int id24 = g.add_edge(2, 4, 2);
int id35 = g.add_edge(3, 5, 2);
int id45 = g.add_edge(4, 5, 3);
int flow = g.max_flow(0, 5);
cout << "Max flow from 0 to 5: " << flow << '\n';
assert(flow == 5);
assert(g.max_flow(0, 5) == 5);
assert(g.edge_flow(id01) == 3);
assert(g.edge_flow(id02) == 2);
assert(g.edge_flow(id13) == 2);
assert(g.edge_flow(id14) == 1);
assert(g.edge_flow(id24) == 2);
assert(g.edge_flow(id35) == 2);
assert(g.edge_flow(id45) == 3);
vector<bool> cut = g.min_cut(0);
assert(cut[0] && !cut[5]);
vector<Dinic<int>::FlowPath> paths = g.decompose(0, 5);
cout << "Flow decomposition:\n";
int total = 0;
for (const auto &path : paths) {
    cout << path.flow << ": ";
    for (int i = 0; i < static_cast<int>(path.vertices.size()); i++) {
        if (i > 0) {
            cout << " -> ";
        }
        cout << path.vertices[i];
    }
    cout << '\n';
    assert(path.vertices.front() == 0 && path.vertices.back() == 5);
    assert(path.edges.size() + 1 == path.vertices.size());
    total += path.flow;
}
assert(total == 5);
g.clear_flow();
assert(g.max_flow(0, 5) == 5);
return 0;
}

```

Example Output

```

Max flow from 0 to 5: 5
Flow decomposition:
2: 0 -> 1 -> 3 -> 5
1: 0 -> 1 -> 4 -> 5
2: 0 -> 2 -> 4 -> 5

```

4.5.5 Maximum Flow (Push-Relabel)

Given a flow network with integer capacities, find the maximum flow from a given source node to a given sink node. The flow along each edge may not exceed its capacity, and flow is conserved at every node other than the source and sink.

Rather than augmenting along whole paths, the push-relabel algorithm maintains a preflow and a height label on each node, repeatedly pushing excess flow to lower-labeled neighbors and relabeling (raising) a node whose excess cannot yet be pushed, until no node other than the source and sink holds excess.

- `push_relabel(source, sink)` returns maximum flow for a global capacity matrix `cap` whose nodes are numbered $[0, n)$, where `n` is `cap.size()`.
- `min_cut(source)` returns the source side of a minimum cut after `push_relabel()` has been called.

Although the push-relabel algorithm is considered one of the most efficient maximum flow algorithms, it cannot take advantage of the magnitude of the maximum flow being less than n^3 (in which case the Ford-Fulkerson or Edmonds-Karp algorithms may be more efficient).

Time Complexity:

- $O(n^3)$ per call to `push_relabel()`, where n is the number of nodes.
- $O(n^2)$ per call to `min_cut()`.

Space Complexity:

- $O(n^2)$ for storage of the flow network, where n is the number of nodes.
- $O(n)$ auxiliary heap space for `push_relabel()`.

```
#include <algorithm>
#include <climits>
#include <cstdint>
#include <queue>
#include <vector>

const int INF = INT_MAX / 2;
std::vector<std::vector<int>>> cap;
std::vector<std::vector<int>>> f;

int64_t push_relabel(int source, int sink) {
    int n = static_cast<int>(cap.size());
    f.assign(n, std::vector<int>(n, 0));
    std::vector<int> e(n, 0), h(n, 0), maxh(n, 0);
    h[source] = n - 1;
    for (int i = 0; i < n; i++) {
        f[source][i] = cap[source][i];
        f[i][source] = -f[source][i];
        e[i] = cap[source][i];
    }
    int size = 0;
    while (true) {
        if (size == 0) {
            for (int i = 0; i < n; i++) {
                if (i != source && i != sink && e[i] > 0) {
                    if (size != 0 && h[i] > h[maxh[0]]) {
                        size = 0;
                    }
                    maxh[size++] = i;
                }
            }
        }
        if (size == 0) {
            break;
        }
        while (size != 0) {
            int i = maxh[size - 1];
            bool pushed = false;
            for (int j = 0; j < n && e[i] != 0; j++) {
                if (h[i] == h[j] + 1 && cap[i][j] - f[i][j] > 0) {
                    int df = std::min(cap[i][j] - f[i][j], e[i]);
                    f[i][j] += df;
                    f[j][i] -= df;
                    e[i] -= df;
                    e[j] += df;
                    if (e[i] == 0) {
                        size--;
                    }
                    pushed = true;
                }
            }
        }
    }
}
```



```

    }
}
if (pushed) {
    continue;
}
h[i] = INF;
for (int j = 0; j < n; j++) {
    if (h[i] > h[j] + 1 && cap[i][j] - f[i][j] > 0) {
        h[i] = h[j] + 1;
    }
}
if (h[i] > h[maxh[0]]) {
    size = 0;
    break;
}
}
}
int64_t max_flow = 0;
for (int i = 0; i < n; i++) {
    max_flow += f[source][i];
}
return max_flow;
}

std::vector<bool> min_cut(int source) {
    int n = static_cast<int>(cap.size());
    std::vector<bool> reachable(n);
    std::queue<int> q;
    reachable[source] = true;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v = 0; v < n; v++) {
            if (!reachable[v] && cap[u][v] - f[u][v] > 0) {
                reachable[v] = true;
                q.push(v);
            }
        }
    }
    return reachable;
}
}

```

Example Usage

```

#include <cassert>

int main() {
    int nodes = 6;
    cap.assign(nodes, std::vector<int>(nodes));
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //           1 -----> 3
    //          / \      |
    // 3/4 /   \ 1/1   | 2/2
    //   /     v     v
    //  0      4 -----> 5
    //   \     ^     3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    cap[0][1] = 4;
    cap[0][2] = 3;
    cap[1][3] = 2;
    cap[1][4] = 1;
    cap[2][4] = 2;
}

```

```

cap[3][5] = 2;
cap[4][5] = 3;
assert(push_relabel(0, 5) == 5);
std::vector<bool> cut = min_cut(0);
assert(cut[0] && !cut[5]);
return 0;
}

```

4.5.6 Minimum-Cost Maximum-Flow (SPFA)

Given a flow network with capacities and edge costs, find a minimum-cost flow from a source node to a sink node. This is the successive shortest path method: repeatedly find a minimum-cost augmenting path from source to sink in the residual graph and push as much flow as it allows, until the flow target is met or no path remains. The path search uses SPFA (queue-based Bellman-Ford), since residual edges carry negated costs.

- `MinCostMaxFlow<T, C>(n)` constructs an empty residual network with nodes numbered $[0, n)$.
- `add_edge(u, v, cap, cost, rev_cap = 0)` adds a directed residual-network edge and returns its edge ID.
- `edge_flow(id)` returns the flow through a previously added edge.
- `clear_flow()` resets all edge flows to zero.
- `min_cost_flow(source, sink, target_flow)` sends up to `target_flow` additional units of flow and returns (flow, cost) for the flow sent by that call. If the returned flow is smaller than `target_flow`, the residual network cannot carry the requested amount.

This implementation uses shortest augmenting paths with SPFA, so it supports negative edge costs as long as the residual graph has no reachable negative-cost cycle. For dense or adversarial inputs, a potential-based Dijkstra version is usually faster when reduced costs are nonnegative. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the number of nodes.
- $O(1)$ per call to `add_edge()`.
- $O(f \cdot n \cdot m)$ per call to `min_cost_flow()`, where f is the amount of flow sent, n is the number of nodes, and m is the number of edges.

Space Complexity: $O(\max(n, m))$ for storage of the residual network and auxiliary arrays.

```

#include <algorithm>
#include <cstdint>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

template<typename T, typename C = int64_t>
class MinCostMaxFlow {
    struct Edge {
        int u, v;
        T cap, flow;
        C cost;
    };

    static constexpr T EPS = static_cast<T>(1e-9);

    int nodes;
    std::vector<Edge> edges;
    std::vector<std::vector<int>>> adj;

public:
    explicit MinCostMaxFlow(int n) : nodes(n), adj(n) {}

```

```

int add_edge(int u, int v, T cap, C cost, T rev_cap = 0) {
    int id = static_cast<int>(edges.size());
    adj[u].push_back(id);
    edges.push_back(Edge{u, v, cap, 0, cost});
    adj[v].push_back(id + 1);
    edges.push_back(Edge{v, u, rev_cap, 0, -cost});
    return id;
}

T edge_flow(int id) const { return edges[id].flow; }

void clear_flow() {
    for (Edge &e : edges) {
        e.flow = 0;
    }
}

std::pair<T, C> min_cost_flow(int source, int sink, T target_flow) {
    static const C INF_COST = std::numeric_limits<C>::max() / 4;
    T flow = 0;
    C cost = 0;
    std::vector<C> dist(nodes);
    std::vector<int> parent_edge(nodes);
    std::vector<bool> in_queue(nodes);
    while (flow < target_flow) {
        std::fill(dist.begin(), dist.end(), INF_COST);
        std::fill(in_queue.begin(), in_queue.end(), false);
        std::queue<int> q;
        dist[source] = 0;
        q.push(source);
        in_queue[source] = true;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            in_queue[u] = false;
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (e.cap - e.flow > EPS && dist[u] + e.cost < dist[e.v]) {
                    dist[e.v] = dist[u] + e.cost;
                    parent_edge[e.v] = id;
                    if (!in_queue[e.v]) {
                        q.push(e.v);
                        in_queue[e.v] = true;
                    }
                }
            }
        }
        if (dist[sink] == INF_COST) {
            break;
        }
        T aug = target_flow - flow;
        for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
            const Edge &e = edges[parent_edge[v]];
            aug = std::min(aug, e.cap - e.flow);
        }
        for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
            int id = parent_edge[v];
            Edge &e = edges[id];
            e.flow += aug;
            edges[id ^ 1].flow -= aug;
            cost += aug * e.cost;
        }
        flow += aug;
    }
    return {flow, cost};
}
};

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    MinCostMaxFlow<int> g(6);
    // Example graph after max flow, with each edge labeled flow/capacity:
    //           2/2
    //           1 -----> 3
    //          / \      |
    // 3/4 /   \ 1/1   | 2/2
    //   /       v      v
    //  0        4 -----> 5
    //   \       ^      3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    int id01 = g.add_edge(0, 1, 4, 1);
    g.add_edge(0, 2, 3, 1);
    g.add_edge(1, 3, 2, 1);
    g.add_edge(1, 4, 1, 2);
    g.add_edge(2, 4, 2, 1);
    g.add_edge(3, 5, 2, 1);
    g.add_edge(4, 5, 3, 1);
    auto [flow, cost] = g.min_cost_flow(0, 5, 5);
    assert(flow == 5);
    assert(cost == 16);
    assert(g.edge_flow(id01) == 3);
    g.clear_flow();
    auto [flow2, cost2] = g.min_cost_flow(0, 5, 5);
    assert(flow2 == 5);
    assert(cost2 == 16);
    return 0;
}
```

4.5.7 Minimum-Cost Maximum-Flow (Potentials)

Given a flow network with capacities and edge costs, find a minimum-cost flow from a source node to a sink node. This is the successive shortest path method with Johnson potentials. A node potential $h[v]$ transforms each edge cost $c(u, v)$ into a reduced cost $c(u, v) + h[u] - h[v]$; when all reduced costs are nonnegative, Dijkstra's algorithm can find the shortest augmenting path safely. After each augmentation, h is updated with the new shortest-path distances so that reduced costs remain nonnegative on the updated residual graph.

- `MinCostMaxFlow<T, C>(n)` constructs an empty residual network with nodes numbered $[0, n)$.
- `add_edge(u, v, cap, cost, rev_cap = 0)` adds a directed residual-network edge and returns its edge ID.
- `edge_flow(id)` returns the flow through a previously added edge.
- `clear_flow()` resets all edge flows to zero.
- `min_cost_flow(source, sink, target_flow)` sends up to `target_flow` additional units of flow and returns (flow, cost) for the flow sent by that call. If the returned flow is smaller than `target_flow`, the residual network cannot carry the requested amount.

If all initial edge costs are nonnegative, the initial potentials can be zero and the first shortest path search is already Dijkstra-safe. If negative costs are present, this implementation first runs SPFA to compute valid initial potentials; it assumes there is no reachable negative-cost cycle. The capacity type `T` should be signed, since reverse residual edges store negative flow.

Time Complexity:

- $O(n)$ per call to the constructor, where n is the number of nodes.
- $O(1)$ per call to `add_edge()`.

- $O(m)$ to check whether initial potentials are needed, plus $O(n \cdot m)$ to compute them when negative residual costs are reachable.
- $O(f \cdot m \cdot \log n)$ for the augmenting paths, where f is the amount of flow sent, n is the number of nodes, and m is the number of edges.

Space Complexity: $O(\max(n, m))$ for storage of the residual network and auxiliary arrays.

```
#include <algorithm>
#include <cstdint>
#include <functional>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

template<typename T, typename C = int64_t>
class MinCostMaxFlow {
    struct Edge {
        int u, v;
        T cap, flow;
        C cost;
    };

    static constexpr T EPS = static_cast<T>(1e-9);
    static bool residual(const Edge &e) { return e.cap - e.flow > EPS; }

    int nodes;
    std::vector<Edge> edges;
    std::vector<std::vector<int>>> adj;
    std::vector<C> potential;

    void init_potentials(int source) {
        static const C INF_COST = std::numeric_limits<C>::max() / 4;
        std::fill(potential.begin(), potential.end(), 0);
        bool has_negative_cost = false;
        for (const Edge &e : edges) {
            has_negative_cost |= residual(e) && e.cost < 0;
        }
        if (!has_negative_cost) {
            return;
        }
        std::fill(potential.begin(), potential.end(), INF_COST);
        std::vector<bool> in_queue(nodes);
        std::queue<int> q;
        potential[source] = 0;
        q.push(source);
        in_queue[source] = true;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            in_queue[u] = false;
            for (int id : adj[u]) {
                const Edge &e = edges[id];
                if (residual(e) && potential[u] + e.cost < potential[e.v]) {
                    potential[e.v] = potential[u] + e.cost;
                    if (!in_queue[e.v]) {
                        q.push(e.v);
                        in_queue[e.v] = true;
                    }
                }
            }
        }
    }

    for (C &p : potential) {
        if (p == INF_COST) {
            p = 0;
        }
    }
}
```

```

}

public:
    explicit MinCostMaxFlow(int n) : nodes(n), adj(n), potential(n, 0) {}

    int add_edge(int u, int v, T cap, C cost, T rev_cap = 0) {
        int id = static_cast<int>(edges.size());
        adj[u].push_back(id);
        edges.push_back(Edge{u, v, cap, 0, cost});
        adj[v].push_back(id + 1);
        edges.push_back(Edge{v, u, rev_cap, 0, -cost});
        return id;
    }

    T edge_flow(int id) const { return edges[id].flow; }

    void clear_flow() {
        for (Edge &e : edges) {
            e.flow = 0;
        }
        std::fill(potential.begin(), potential.end(), 0);
    }

    std::pair<T, C> min_cost_flow(int source, int sink, T target_flow) {
        static const C INF_COST = std::numeric_limits<C>::max() / 4;
        init_potentials(source);
        T flow = 0;
        C cost = 0;
        std::vector<C> dist(nodes);
        std::vector<int> parent_edge(nodes);
        while (flow < target_flow) {
            std::fill(dist.begin(), dist.end(), INF_COST);
            std::priority_queue<
                std::pair<C, int>, std::vector<std::pair<C, int>>, std::greater<std::pair<C, int>>>
            > pq;
            dist[source] = 0;
            pq.push({0, source});
            while (!pq.empty()) {
                auto [du, u] = pq.top();
                pq.pop();
                if (du != dist[u]) {
                    continue;
                }
                for (int id : adj[u]) {
                    const Edge &e = edges[id];
                    C reduced_cost = e.cost + potential[u] - potential[e.v];
                    if (residual(e) && dist[u] + reduced_cost < dist[e.v]) {
                        dist[e.v] = dist[u] + reduced_cost;
                        parent_edge[e.v] = id;
                        pq.push({dist[e.v], e.v});
                    }
                }
            }
            if (dist[sink] == INF_COST) {
                break;
            }
            for (int i = 0; i < nodes; i++) {
                if (dist[i] < INF_COST) {
                    potential[i] += dist[i];
                }
            }
            T aug = target_flow - flow;
            for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
                const Edge &e = edges[parent_edge[v]];
                aug = std::min(aug, e.cap - e.flow);
            }
            for (int v = sink; v != source; v = edges[parent_edge[v]].u) {
                int id = parent_edge[v];

```

```

    Edge &e = edges[id];
    e.flow += aug;
    edges[id ^ 1].flow -= aug;
    cost += aug * e.cost;
}
flow += aug;
}
return {flow, cost};
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    MinCostMaxFlow<int> g(6);
    // Example graph after max flow, with each edge labeled flow/capacity:
    //      2/2
    //      1 -----> 3
    //      / \      |
    // 3/4 /   \ 1/1 | 2/2
    //      v     v
    // 0      4 ----> 5
    //      \   ^   3/3
    // 2/3 \   / 2/2
    //      v /
    //      2
    int id01 = g.add_edge(0, 1, 4, 1);
    g.add_edge(0, 2, 3, 1);
    g.add_edge(1, 3, 2, 1);
    g.add_edge(1, 4, 1, 2);
    g.add_edge(2, 4, 2, 1);
    g.add_edge(3, 5, 2, 1);
    g.add_edge(4, 5, 3, 1);
    auto [flow, cost] = g.min_cost_flow(0, 5, 5);
    assert(flow == 5);
    assert(cost == 16);
    assert(g.edge_flow(id01) == 3);
    g.clear_flow();
    auto [flow2, cost2] = g.min_cost_flow(0, 5, 5);
    assert(flow2 == 5);
    assert(cost2 == 16);
    return 0;
}

```

4.5.8 Gomory-Hu Tree

Given an undirected graph with nonnegative edge weights, build a Gomory-Hu tree: a weighted tree on the same vertices that represents every pairwise minimum s - t cut value. For any two vertices u and v , the minimum cut value between them in the original graph is the minimum edge weight on the path from u to v in the Gomory-Hu tree.

The algorithm starts with every vertex attached to vertex 0, then performs $n - 1$ max-flow/min-cut computations. After computing the minimum cut between vertex s and its current parent, vertices on the s side of that cut are reparented under s , gradually refining the cut-equivalent tree.

- `gomory_hu(n, edges)` returns the $n - 1$ edges of a Gomory-Hu tree as `(u, v, weight)` triples for an undirected, weighted graph with `n` nodes and `edges` of the same shape.

- `min_cut_value(n, tree, source, sink)` returns the minimum cut value between two vertices using a Gomory-Hu tree. Note that for many pairwise cut queries on the same tree, it's more efficient to prebuild the tree adjacency once and answer minimum edge-on-path queries with LCA/binary lifting instead of calling `min_cut_value()` each time.

Time Complexity:

- $O(n)$ calls to maximum flow. With the included Dinic implementation, this is $O(n^3 \cdot m)$ in the worst case, where n is the number of vertices and m is the number of undirected edges.
- $O(n)$ per call to `min_cut_value()`.

Space Complexity: $O(\max(n, m))$ for the residual network and auxiliary arrays.

```
#include <algorithm>
#include <cassert>
#include <limits>
#include <queue>
#include <tuple>
#include <utility>
#include <vector>

template<typename T>
class Dinic {
    int nodes;
    std::vector<std::vector<std::tuple<int, int, T>>> adj; // (v, rev, cap)
    std::vector<int> dist, ptr;

    bool bfs(int source, int sink) {
        dist.assign(nodes, -1);
        dist[source] = 0;
        std::queue<int> q;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const auto &[v, rev, cap] : adj[u]) {
                if (dist[v] < 0 && cap > 0) {
                    dist[v] = dist[u] + 1;
                    q.push(v);
                }
            }
        }
        return dist[sink] >= 0;
    }

    T dfs(int u, int sink, T pushed) {
        if (u == sink || pushed == 0) {
            return pushed;
        }
        for (; ptr[u] < static_cast<int>(adj[u].size()); ptr[u]++) {
            auto &[v, rev, cap] = adj[u][ptr[u]];
            if (dist[v] == dist[u] + 1 && cap > 0) {
                T pushed_next = dfs(v, sink, std::min(pushed, cap));
                if (pushed_next > 0) {
                    cap -= pushed_next;
                    std::get<2>(adj[v][rev]) += pushed_next;
                    return pushed_next;
                }
            }
        }
        return 0;
    }

public:
    explicit Dinic(int n) : nodes(n), adj(n), dist(n), ptr(n) {}

    void add_edge(int u, int v, T cap) {
```



```

    adj[u].emplace_back(v, static_cast<int>(adj[v].size()), cap);
    adj[v].emplace_back(u, static_cast<int>(adj[u].size()) - 1, 0);
}

T max_flow(int source, int sink) {
    T flow = 0;
    while (bfs(source, sink)) {
        ptr.assign(nodes, 0);
        while (true) {
            T pushed = dfs(source, sink, std::numeric_limits<T>::max());
            if (pushed == 0) {
                break;
            }
            flow += pushed;
        }
    }
    return flow;
}

std::vector<bool> min_cut(int source) const {
    std::vector<bool> reachable(nodes);
    std::queue<int> q;
    reachable[source] = true;
    q.push(source);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (const auto &[v, rev, cap] : adj[u]) {
            if (!reachable[v] && cap > 0) {
                reachable[v] = true;
                q.push(v);
            }
        }
    }
    return reachable;
}

};

template<typename T>
using Edge = std::tuple<int, int, T>;

template<typename T>
std::vector<Edge<T>> gomory_hu(int n, const std::vector<Edge<T>> &edges) {
    assert(n >= 1);
    std::vector<int> parent(n, 0);
    std::vector<T> cut_value(n);
    for (int s = 1; s < n; s++) {
        int t = parent[s];
        Dinic<T> mf(n);
        for (const auto &[u, v, w] : edges) {
            mf.add_edge(u, v, w);
            mf.add_edge(v, u, w);
        }
        T flow = mf.max_flow(s, t);
        std::vector<bool> cut = mf.min_cut(s);
        for (int v = s + 1; v < n; v++) {
            if (parent[v] == t && cut[v] == cut[s]) {
                parent[v] = s;
            }
        }
        if (cut[parent[t]]) {
            parent[s] = parent[t];
            parent[t] = s;
            cut_value[s] = cut_value[t];
            cut_value[t] = flow;
        } else {
            cut_value[s] = flow;
        }
    }
}

```

```

    }
    std::vector<Edge<T>> tree;
    for (int v = 1; v < n; v++) {
        tree.emplace_back(v, parent[v], cut_value[v]);
    }
    return tree;
}

template<typename T>
T min_cut_value_dfs(
    const std::vector<std::vector<std::pair<int, T>>> &adj, std::vector<bool> &seen, int u,
    int sink, T best
) {
    if (u == sink) {
        return best;
    }
    seen[u] = true;
    for (auto [v, w] : adj[u]) {
        if (!seen[v]) {
            T res = min_cut_value_dfs(adj, seen, v, sink, std::min(best, w));
            if (res != -1) {
                return res;
            }
        }
    }
    return -1;
}

template<typename T>
T min_cut_value(int n, const std::vector<Edge<T>> &tree, int source, int sink) {
    assert(0 <= source && source < n && 0 <= sink && sink < n && source != sink);
    // For many queries, prebuild this adjacency and add LCA/binary lifting for min edge on path.
    std::vector<std::vector<std::pair<int, T>>> adj(n);
    for (const auto &[u, v, w] : tree) {
        adj[u].emplace_back(v, w);
        adj[v].emplace_back(u, w);
    }
    std::vector<bool> seen(n);
    return min_cut_value_dfs(adj, seen, source, sink, std::numeric_limits<T>::max());
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<Edge<int>> edges{
        {0, 1, 3}, {0, 2, 2}, {1, 2, 1}, {1, 3, 2}, {2, 3, 4},
    };
    auto tree = gomory_hu(4, edges);
    assert(tree.size() == 3);
    assert(min_cut_value(4, tree, 0, 1) == 5);
    assert(min_cut_value(4, tree, 0, 2) == 5);
    assert(min_cut_value(4, tree, 0, 3) == 5);
    assert(min_cut_value(4, tree, 2, 3) == 6);
    return 0;
}

```

4.5.9 Global Minimum Cut (Stoer-Wagner)

Given a connected, undirected graph with nonnegative edge weights, the global minimum cut is the partition of the vertices into two nonempty sets that minimizes the total weight of edges crossing between them. Unlike a

minimum s-t cut, no source or sink is fixed; the goal is the cheapest way to split the graph in two. The Stoer-Wagner algorithm finds it without any maximum flow computation by repeatedly running a “minimum cut phase”.

A phase grows a set, starting from an arbitrary vertex, by repeatedly adding the most tightly connected outside vertex (the one with the greatest total weight to the set so far), exactly as in Prim’s algorithm. The last vertex added, together with its connection weight, yields one candidate cut, the “cut of the phase”. The last two vertices added are then merged into a single supernode and the next phase begins on the smaller graph. After $n - 1$ phases every candidate has been considered, and the smallest is the global minimum cut.

- `stoer_wagner(adj)` returns a pair `(weight, side)` for a graph as a symmetric adjacency matrix `adj` (with 0 for the diagonal and absent edges), where `weight` is the total weight of the global minimum cut and `side` lists the vertices on one side of that cut. The graph must have at least two vertices.

Parallel edges should be pre-summed into the adjacency matrix before calling: `adj[u][v]` holds a single combined weight, so the sum of all parallel edge weights between `u` and `v` should be stored there.

Time Complexity: $O(n^3)$ per call, where n is the number of vertices.

Space Complexity: $O(n^2)$ auxiliary heap space.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <limits>
#include <utility>
#include <vector>

std::pair<int64_t, std::vector<int>> stoer_wagner(std::vector<std::vector<int64_t>> adj) {
    int n = static_cast<int>(adj.size());
    assert(n >= 2);
    for (int i = 0; i < n; i++) {
        assert(static_cast<int>(adj[i].size()) == n);
        for (int j = i + 1; j < n; j++) {
            assert(adj[i][j] == adj[j][i]);
        }
    }
    std::pair<int64_t, std::vector<int>> best = {INT64_MAX, {}};
    std::vector<std::vector<int>> groups(n);
    std::vector<int64_t> weight(n);
    std::vector<bool> exists(n, true), added(n);
    for (int i = 0; i < n; i++) {
        groups[i] = {i};
    }
    for (int phase = 1; phase < n; phase++) {
        std::fill(weight.begin(), weight.end(), 0);
        std::fill(added.begin(), added.end(), false);
        int s = -1;
        for (int it = 0; it <= n - phase; it++) {
            int sel = -1;
            for (int i = 0; i < n; i++) {
                if (exists[i] && !added[i] && (sel == -1 || weight[i] > weight[sel])) {
                    sel = i;
                }
            }
            if (it == n - phase) {
                if (weight[sel] < best.first) {
                    best = {weight[sel], groups[sel]};
                }
                groups[s].insert(groups[s].end(), groups[sel].begin(), groups[sel].end());
                for (int i = 0; i < n; i++) {
                    adj[s][i] += adj[sel][i];
                    adj[i][s] = adj[s][i];
                }
                exists[sel] = false;
                break;
            }
        }
    }
}
```

```

    }
    added[sel] = true;
    for (int i = 0; i < n; i++) {
        weight[i] += adj[sel][i];
    }
    s = sel;
}
}
return best;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Triangle with edge weights 0-1: 2, 1-2: 3, 0-2: 1.
    // Cuts: {0}|{1,2} = 3, {1}|{0,2} = 5, {2}|{0,1} = 4. Minimum is 3.
    vector<vector<int64_t>> adj{
        {0, 2, 1},
        {2, 0, 3},
        {1, 3, 0},
    };
    auto [weight, side] = stoer_wagner(adj);
    assert(weight == 3);
    assert((side == vector<int>{1, 2})); // The shore {1, 2}, opposite the single vertex 0.
    return 0;
}

```

4.6 Matching and Assignment

4.6.1 Maximum Bipartite Matching (Kuhn)

Given two sets of nodes $A = \{0, 1, \dots, n_1 - 1\}$ and $B = \{0, 1, \dots, n_2 - 1\}$, as well as a set of edges E mapping nodes from set A to set B , find the largest possible subset of E containing no edges that share the same node.

Kuhn's algorithm builds the matching one left node at a time. For each, a depth-first search looks for an augmenting path: an alternating sequence of unmatched and matched edges that reaches a free right node. Flipping the edges along such a path enlarges the matching by one.

- `kuhn(n2)` populates `match_left` and `match_right`, then returns maximum matching size for a global, pre-populated adjacency list `adj` whose left-side nodes are numbered $[0, n)$ and whose right-side neighbors are numbered $[0, n2)$, where `n` is `adj.size()`.

Time Complexity: $O(m \cdot (n_1 + n_2))$ per call to `kuhn()`, where m is the number of edges.

Space Complexity: $O(n_1 + n_2)$ auxiliary stack space for `kuhn()`.

```

#include <vector>

std::vector<int> match_left, match_right;
std::vector<int> visit;
std::vector<std::vector<int>> adj;
int visit_time;

bool dfs(int u) {
    visit[u] = visit_time;
    for (int nb : adj[u]) {
        if (match_right[nb] == -1) {

```

```

        match_left[u] = nb;
        match_right[nb] = u;
        return true;
    }
}
for (int nb : adj[u]) {
    int v = match_right[nb];
    if (visit[v] != visit_time && dfs(v)) {
        match_left[u] = nb;
        match_right[nb] = u;
        return true;
    }
}
return false;
}

int kuhn(int n2) {
    int n1 = static_cast<int>(adj.size());
    match_left.assign(n1, -1);
    match_right.assign(n2, -1);
    visit.assign(n1, 0);
    visit_time = 0;
    int matches = 0;
    for (int i = 0; i < n1; i++) {
        visit_time++;
        if (dfs(i)) {
            matches++;
        }
    }
    return matches;
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    int n1 = 3, n2 = 4;
    adj.assign(n1, {});
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
    cout << "Matched " << kuhn(n2) << " pair(s):" << endl;
    for (int i = 0; i < n2; i++) {
        if (match_right[i] != -1) {
            cout << match_right[i] << " " << i << endl;
        }
    }
    return 0;
}

```

Example Output

```

Matched 3 pair(s):
1 0
0 1
2 2

```

4.6.2 Maximum Bipartite Matching (Hopcroft-Karp)

Given two sets of nodes $A = \{0, 1, \dots, n_1 - 1\}$ and $B = \{0, 1, \dots, n_2 - 1\}$, as well as a set of edges E mapping nodes from set A to set B , find the largest possible subset of E containing no edges that share the same node.

Hopcroft-Karp augments along many shortest augmenting paths per phase. Each phase first runs a breadth-first search to layer the graph by distance, then a depth-first search to find a maximal set of vertex-disjoint shortest augmenting paths to flip at once, for $O(m\sqrt{n})$ running time overall.

- `hopcroft_karp(n2)` populates `match` and returns maximum matching size for a global, pre-populated adjacency list `adj` whose left-side nodes are numbered $[0, \text{n})$ and whose right-side neighbors are numbered $[0, \text{n2})$, where `n` is `adj.size()`.

Time Complexity: $O(m \cdot \sqrt{n_1 + n_2})$ per call to `hopcroft_karp()`, where m is the number of edges.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n_1 + n_2)$ auxiliary stack and heap space for `hopcroft_karp()`.

```
#include <algorithm>
#include <queue>
#include <vector>

std::vector<std::vector<int>>> adj;
std::vector<bool> used, visit;
std::vector<int> match, dist;

void bfs() {
    int n1 = static_cast<int>(adj.size());
    dist.assign(n1, -1);
    std::queue<int> q;
    for (int u = 0; u < n1; u++) {
        if (!used[u]) {
            q.push(u);
            dist[u] = 0;
        }
    }
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int nb : adj[u]) {
            int v = match[nb];
            if (v >= 0 && dist[v] < 0) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

bool dfs(int u) {
    visit[u] = true;
    for (int nb : adj[u]) {
        int v = match[nb];
        if (v < 0 || (!visit[v] && dist[v] == dist[u] + 1 && dfs(v))) {
            match[nb] = u;
            used[u] = true;
            return true;
        }
    }
    return false;
}

int hopcroft_karp(int n2) {
    int n1 = static_cast<int>(adj.size());
    match.assign(n2, -1);
```

```

used.assign(n1, false);
int res = 0;
while (true) {
    bfs();
    visit.assign(n1, false);
    int f = 0;
    for (int u = 0; u < n1; u++) {
        if (!used[u] && dfs(u)) {
            f++;
        }
    }
    if (f == 0) {
        return res;
    }
    res += f;
}
return res;
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    int n1 = 3, n2 = 4;
    adj.assign(n1, {});
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);
    cout << "Matched " << hopcroft_karp(n2) << " pair(s):" << endl;
    for (int i = 0; i < n2; i++) {
        if (match[i] != -1) {
            cout << match[i] << " " << i << endl;
        }
    }
    return 0;
}

```

Example Output

```

Matched 3 pair(s):
1 0
0 1
2 2

```

4.6.3 Minimum Vertex Cover (Konig)

Given a bipartite graph with left nodes $A = \{0, \dots, n_1 - 1\}$ and right nodes $B = \{0, \dots, n_2 - 1\}$, find a minimum vertex cover: a smallest set of nodes such that every edge has at least one endpoint in the set. By Koenig's theorem, in a bipartite graph the size of a minimum vertex cover equals the size of a maximum matching, and the nodes left uncovered form a maximum independent set.

The construction first computes a maximum matching with Kuhn's algorithm. It then marks every left node reachable from an unmatched left node by an alternating path (following unmatched edges from left to right and matched edges from right to left). Koenig's theorem says the minimum vertex cover is exactly the left nodes that are not reachable together with the right nodes that are reachable; each matched edge contributes exactly one endpoint, so the cover size equals the matching size.

- `min_vertex_cover(n2)` returns the nodes of a minimum vertex cover for the global adjacency list `adj` (left nodes $[0, n_1)$, right neighbors $[0, n_2)$), where `n1` is `adj.size()`. Each returned value is a left node `i` in $[0, n_1)$, or a right node `j` encoded as `n1 + j`.

The adjacency list `adj` is shared with the matching routine: `adj[u]` lists the right-side neighbors of left node `u`. Cover nodes are reported with right node `j` encoded as `n1 + j` so the two sides share one index space.

Time Complexity: $O(m \cdot (n_1 + n_2))$ per call to `min_vertex_cover()`, dominated by the matching, where m is the number of edges.

Space Complexity: $O(n_1 + n_2)$ auxiliary space.

```
#include <vector>

std::vector<int> match_left, match_right;
std::vector<int> visit;
std::vector<std::vector<int>> adj;
int visit_time;

bool dfs(int u) {
    visit[u] = visit_time;
    for (int nb : adj[u]) {
        if (match_right[nb] == -1) {
            match_left[u] = nb;
            match_right[nb] = u;
            return true;
        }
    }
    for (int nb : adj[u]) {
        int v = match_right[nb];
        if (visit[v] != visit_time && dfs(v)) {
            match_left[u] = nb;
            match_right[nb] = u;
            return true;
        }
    }
    return false;
}

int kuhn(int n2) {
    int n1 = static_cast<int>(adj.size());
    match_left.assign(n1, -1);
    match_right.assign(n2, -1);
    visit.assign(n1, 0);
    visit_time = 0;
    int matches = 0;
    for (int i = 0; i < n1; i++) {
        visit_time++;
        if (dfs(i)) {
            matches++;
        }
    }
    return matches;
}

std::vector<int> min_vertex_cover(int n2) {
    int n1 = static_cast<int>(adj.size());
    kuhn(n2);
    // reachable_left[u] becomes true for left nodes on an alternating path from an unmatched left
    // node; seen_right[e] for the right nodes on those paths.
    std::vector<bool> reachable_left(n1, true), seen_right(n2, false);
    for (int u = 0; u < n1; u++) {
        if (match_left[u] != -1) {
            reachable_left[u] = false;
        }
    }
    std::vector<int> stack;
```



```

for (int u = 0; u < n1; u++) {
    if (reachable_left[u]) {
        stack.push_back(u);
    }
}
while (!stack.empty()) {
    int u = stack.back();
    stack.pop_back();
    reachable_left[u] = true;
    for (int e : adj[u]) {
        if (!seen_right[e] && match_right[e] != -1) {
            seen_right[e] = true;
            stack.push_back(match_right[e]); // Cross the matched edge back to the left.
        }
    }
}
std::vector<int> cover;
for (int u = 0; u < n1; u++) {
    if (!reachable_left[u]) {
        cover.push_back(u);
    }
}
for (int e = 0; e < n2; e++) {
    if (seen_right[e]) {
        cover.push_back(n1 + e);
    }
}
return cover;
}

```

Example Usage

```

#include <algorithm>
#include <cassert>
using namespace std;

int main() {
    // Left {0,1,2} and right {0,1,2,3}; a maximum matching has size 3.
    int n1 = 3, n2 = 4;
    adj.assign(n1, {});
    adj[0].push_back(1);
    adj[1].push_back(0);
    adj[1].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(2);
    adj[2].push_back(3);

    vector<int> cover = min_vertex_cover(n2);
    assert(cover.size() == 3); // Equals the maximum matching size, by Koenig's theorem.

    // The cover is valid: every edge has at least one endpoint in it.
    for (int u = 0; u < n1; u++) {
        for (int v : adj[u]) {
            bool left_in = find(cover.begin(), cover.end(), u) != cover.end();
            bool right_in = find(cover.begin(), cover.end(), n1 + v) != cover.end();
            assert(left_in || right_in);
        }
    }
    return 0;
}

```

4.6.4 Maximum Graph Matching (Blossom)

Given an undirected, unweighted graph, determine a maximum matching: a maximum subset of its edges such that no node is shared between different edges in the resulting subset.

Edmonds' blossom algorithm extends augmenting-path search to general (non-bipartite) graphs, where odd-length cycles called blossoms can conceal augmenting paths. Each blossom is contracted into a single node so the search can proceed, and is expanded afterward to recover the matching. For maximum-weight matching in bipartite graphs, use Hungarian or min-cost max-flow; for maximum-weight matching in general graphs, use the weighted blossom algorithm in the next section.

The graph must be simple; parallel edges and self-loops are not supported.

- `blossom()` returns a matching for a global, bidirectionally pre-populated adjacency list `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`. The returned vector `match` has `match[u] == v` and `match[v] == u` when `u` and `v` are matched, or `match[u] == -1` when `u` is unmatched.

Time Complexity: $O(n^3)$ per call to `blossom()`, where n is the number of nodes.

Space Complexity:

- $O(\max(n, m))$ for storage of the graph, where n is the number of nodes and m is the number of edges.
- $O(n)$ auxiliary heap space for `blossom()`, where n is the number of nodes.

```
#include <algorithm>
#include <numeric>
#include <queue>
#include <vector>

std::vector<int> blossom(const std::vector<std::vector<int>> &adj) {
    int n = static_cast<int>(adj.size());
    std::vector<int> match(n, -1), label(n), parent(n), base(n), aux(n, -1);
    std::queue<int> q;
    int aux_time = -1;
    auto lca = [&](int u, int v) {
        aux_time++;
        while (true) {
            if (u != -1) {
                u = base[u];
                if (aux[u] == aux_time) {
                    return u;
                }
                aux[u] = aux_time;
                u = (match[u] == -1) ? -1 : parent[match[u]];
            }
            std::swap(u, v);
        }
    };
    auto mark_blossom = [&](int u, int v, int b) {
        while (base[u] != b) {
            parent[u] = v;
            v = match[u];
            if (label[v] == 1) {
                label[v] = 0;
                q.push(v);
            }
            base[u] = base[v] = b;
            u = parent[v];
        }
    };
    auto augment = [&](int u) {
        while (u != -1) {
            int p = parent[u];
            int next = match[p];
            match[u] = p;
            match[p] = u;
        }
    };
}
```

```

    u = next;
}
};
auto find_path = [&](int root) {
    std::fill(label.begin(), label.end(), -1);
    std::fill(parent.begin(), parent.end(), -1);
    std::iota(base.begin(), base.end(), 0);
    while (!q.empty()) {
        q.pop();
    }
    label[root] = 0;
    q.push(root);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (base[u] == base[v] || match[u] == v) {
                continue;
            }
            if (label[v] == -1) {
                label[v] = 1;
                parent[v] = u;
                if (match[v] == -1) {
                    augment(v);
                    return true;
                }
                label[match[v]] = 0;
                q.push(match[v]);
            } else if (label[v] == 0) {
                int b = lca(u, v);
                mark_blossom(u, v, b);
                mark_blossom(v, u, b);
            }
        }
    }
    return false;
};
for (int i = 0; i < n; i++) {
    if (match[i] == -1) {
        for (int v : adj[i]) {
            if (i != v && match[v] == -1) {
                match[i] = v;
                match[v] = i;
                break;
            }
        }
    }
}
for (int i = 0; i < n; i++) {
    if (match[i] == -1) {
        find_path(i);
    }
}
return match;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

vector<vector<int>> adj;

void add_edge(int u, int v) {
    adj[u].push_back(v);
}

```

```

    adj[v].push_back(u);
};

int matching_size(const vector<int> &match) {
    return std::count_if(match.begin(), match.end(), [](int v) { return v != -1; }) / 2;
};

int main() {
    int nodes = 4;
    adj.assign(nodes, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(2, 3);
    add_edge(3, 0);
    vector<int> match = blossom(adj);
    cout << "Matched " << matching_size(match) << " pair(s):" << endl;
    for (int i = 0; i < nodes; i++) {
        if (match[i] != -1 && i < match[i]) {
            cout << i << " " << match[i] << endl;
        }
    }
    nodes = 5;
    adj.assign(nodes, {});
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(2, 0);
    add_edge(0, 3);
    add_edge(1, 4);
    assert(matching_size(blossom(adj)) == 2);
    return 0;
}

```

Example Output

```

Matched 2 pair(s):
0 1
2 3

```

4.6.5 Maximum Weight Graph Matching (Weighted Blossom)

Given an undirected graph with positive edge weights, determine a maximum-weight matching: a subset of edges, no two sharing a node, whose total weight is as large as possible. Unlike the bipartite case (which Hungarian or min-cost max-flow solves), general graphs may contain odd cycles, so this is the weighted generalization of Edmonds' blossom algorithm.

The algorithm runs a primal-dual method. Each node carries a dual label, and an edge is “tight” when its label sum equals twice its weight; the search uses only tight edges, exactly as unweighted blossom matching does, contracting odd cycles into blossoms. When no tight augmenting structure remains, the labels are adjusted by the largest step that keeps every dual constraint satisfied, which makes a new edge tight or lets a blossom expand, and the search resumes. Because the matching is over the implicit complete graph with absent edges weighted 0, a node may be left effectively unmatched whenever staying unmatched is at least as good as any incident edge.

All edge weights must be positive; an absent edge is treated as weight 0. The node count should be kept modest, as the algorithm uses dense storage.

- `WeightedGeneralMatching(n)` creates a graph of `n` nodes numbered `[0, n)`.
- `add_edge(u, v, w)` adds an undirected edge of positive weight `w` between `u` and `v`. If several edges join the same pair, the last weight set is used.
- `solve()` computes a maximum-weight matching and returns its total weight.
- `partner(u)` returns the node matched with `u`, or `-1` if `u` is unmatched. Valid after `solve()`.

Time Complexity: $O(n^3)$ per call to `solve()`, where n is the number of nodes.

Space Complexity: $O(n^2)$ for the dense edge and blossom storage.

```

#include <algorithm>
#include <cstdlib>
#include <queue>
#include <vector>

class WeightedGeneralMatching {
    struct Edge {
        int u, v;
        int64_t w;
    };

    int n, nx; // n original nodes (1-based internally); nx includes blossom nodes.
    std::vector<std::vector<Edge>> g;
    std::vector<std::vector<int>> flo, flo_from;
    std::vector<int64_t> lab;
    std::vector<int> match_, slack, st, par, s_, aux_;
    std::queue<int> q;
    int aux_clock;

    int64_t e_delta(const Edge &e) const { return lab[e.u] + lab[e.v] - 2 * e.w; }

    void update_slack(int u, int x) {
        if (slack[x] == 0 || e_delta(g[u][x]) < e_delta(g[slack[x]][x])) {
            slack[x] = u;
        }
    }

    void set_slack(int x) {
        slack[x] = 0;
        for (int u = 1; u <= n; u++) {
            if (g[u][x].w > 0 && st[u] != x && s_[st[u]] == 0) {
                update_slack(u, x);
            }
        }
    }

    void queue_push(int x) {
        if (x <= n) {
            q.push(x);
        } else {
            for (int t : flo[x]) {
                queue_push(t);
            }
        }
    }

    void set_st(int x, int b) {
        st[x] = b;
        if (x > n) {
            for (int t : flo[x]) {
                set_st(t, b);
            }
        }
    }

    int get_pr(int b, int xr) {
        int pr = static_cast<int>(std::find(flo[b].begin(), flo[b].end(), xr) - flo[b].begin());
        if (pr & 1) {
            std::reverse(flo[b].begin() + 1, flo[b].end());
            return static_cast<int>(flo[b].size()) - pr;
        }
        return pr;
    }

    void set_match(int u, int v) {
        match_[u] = g[u][v].v;
    }

```

```

    if (u <= n) {
        return;
    }
    Edge e = g[u][v];
    int xr = flo_from[u][e.u], pr = get_pr(u, xr);
    for (int i = 0; i < pr; i++) {
        set_match(flo[u][i], flo[u][i ^ 1]);
    }
    set_match(xr, v);
    std::rotate(flo[u].begin(), flo[u].begin() + pr, flo[u].end());
}

void augment(int u, int v) {
    while (true) {
        int xnv = st[match_[u]];
        set_match(u, v);
        if (xnv == 0) {
            return;
        }
        set_match(xnv, st[par[xnv]]);
        u = st[par[xnv]];
        v = xnv;
    }
}

int lca(int u, int v) {
    aux_clock++;
    while (u != 0 || v != 0) {
        if (u == 0) {
            std::swap(u, v);
            continue;
        }
        if (aux_[u] == aux_clock) {
            return u;
        }
        aux_[u] = aux_clock;
        u = st[match_[u]];
        if (u != 0) {
            u = st[par[u]];
        }
        std::swap(u, v);
    }
    return 0;
}

void add_blossom(int u, int anc, int v) {
    int b = n + 1;
    while (b <= nx && st[b] != 0) {
        b++;
    }
    if (b > nx) {
        nx++;
    }
    lab[b] = 0;
    s_[b] = 0;
    match_[b] = match_[anc];
    flo[b] = {anc};
    auto build = [&](int x) {
        for (int y; x != anc; x = st[par[y]]) {
            flo[b].push_back(x);
            y = st[match_[x]];
            flo[b].push_back(y);
            queue_push(y);
        }
    };
    build(u);
    std::reverse(flo[b].begin() + 1, flo[b].end());
    build(v);
}

```

```

    set_st(b, b);
    for (int x = 1; x <= nx; x++) {
        g[b][x].w = g[x][b].w = 0;
    }
    for (int x = 1; x <= n; x++) {
        flo_from[b][x] = 0;
    }
    for (int xs : flo[b]) {
        for (int x = 1; x <= nx; x++) {
            if (g[b][x].w == 0 || e_delta(g[xs][x]) < e_delta(g[b][x])) {
                g[b][x] = g[xs][x];
                g[x][b] = g[x][xs];
            }
        }
        for (int x = 1; x <= n; x++) {
            if (flo_from[xs][x] != 0) {
                flo_from[b][x] = xs;
            }
        }
    }
    set_slack(b);
}

void expand_blossom(int b) {
    for (int t : flo[b]) {
        set_st(t, t);
    }
    int xr = flo_from[b][g[b][par[b]].u], pr = get_pr(b, xr);
    for (int i = 0; i < pr; i += 2) {
        int xs = flo[b][i], xns = flo[b][i + 1];
        par[xs] = g[xns][xs].u;
        s_[xs] = 1;
        s_[xns] = 0;
        slack[xs] = 0;
        set_slack(xns);
        queue_push(xns);
    }
    s_[xr] = 1;
    par[xr] = par[b];
    for (int i = pr + 1; i < static_cast<int>(flo[b].size()); i++) {
        int xs = flo[b][i];
        s_[xs] = -1;
        set_slack(xs);
    }
    st[b] = 0;
}

bool on_found_edge(const Edge &e) {
    int u = st[e.u], v = st[e.v];
    if (s_[v] == -1) {
        par[v] = e.u;
        s_[v] = 1;
        int nu = st[match_[v]];
        slack[v] = slack[nu] = 0;
        s_[nu] = 0;
        queue_push(nu);
    } else if (s_[v] == 0) {
        int anc = lca(u, v);
        if (anc == 0) {
            augment(u, v);
            augment(v, u);
            return true;
        }
        add_blossom(u, anc, v);
    }
    return false;
}

```

```

bool matching() {
    std::fill(s_.begin(), s_.begin() + nx + 1, -1);
    std::fill(slack.begin(), slack.begin() + nx + 1, 0);
    q = std::queue<int>();
    for (int x = 1; x <= nx; x++) {
        if (st[x] == x && match_[x] == 0) {
            par[x] = s_[x] = 0;
            queue_push(x);
        }
    }
    if (q.empty()) {
        return false;
    }
    while (true) {
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (s_[st[u]] == 1) {
                continue;
            }
            for (int v = 1; v <= n; v++) {
                if (g[u][v].w > 0 && st[u] != st[v]) {
                    if (e_delta(g[u][v]) == 0) {
                        if (on_found_edge(g[u][v])) {
                            return true;
                        }
                    } else {
                        update_slack(u, st[v]);
                    }
                }
            }
        }
        int64_t d = INT64_MAX;
        for (int b = n + 1; b <= nx; b++) {
            if (st[b] == b && s_[b] == 1) {
                d = std::min(d, lab[b] / 2);
            }
        }
        for (int x = 1; x <= nx; x++) {
            if (st[x] == x && slack[x] != 0) {
                if (s_[x] == -1) {
                    d = std::min(d, e_delta(g[slack[x]][x]));
                } else if (s_[x] == 0) {
                    d = std::min(d, e_delta(g[slack[x]][x]) / 2);
                }
            }
        }
        for (int u = 1; u <= n; u++) {
            if (s_[st[u]] == 0) {
                if (lab[u] <= d) {
                    return false;
                }
                lab[u] -= d;
            } else if (s_[st[u]] == 1) {
                lab[u] += d;
            }
        }
        for (int b = n + 1; b <= nx; b++) {
            if (st[b] == b && s_[b] != -1) {
                lab[b] += (s_[b] == 0 ? 1 : -1) * d * 2;
            }
        }
        q = std::queue<int>();
        for (int x = 1; x <= nx; x++) {
            if (st[x] == x && slack[x] != 0 && st[slack[x]] != x && e_delta(g[slack[x]][x]) == 0) {
                if (on_found_edge(g[slack[x]][x])) {
                    return true;
                }
            }
        }
    }
}

```



```

    }
}
for (int b = n + 1; b <= nx; b++) {
    if (st[b] == b && s_[b] == 1 && lab[b] == 0) {
        expand_blossom(b);
    }
}
}
}

public:
explicit WeightedGeneralMatching(int n)
: n(n),
  nx(n),
  g(2 * n + 1, std::vector<Edge>(2 * n + 1)),
  flo(2 * n + 1),
  flo_from(2 * n + 1, std::vector<int>(n + 1, 0)),
  lab(2 * n + 1, 0),
  match_(2 * n + 1, 0),
  slack(2 * n + 1, 0),
  st(2 * n + 1, 0),
  par(2 * n + 1, 0),
  s_(2 * n + 1, 0),
  aux_(2 * n + 1, 0),
  aux_clock(0) {
    for (int u = 1; u <= n; u++) {
        for (int v = 1; v <= n; v++) {
            g[u][v] = {u, v, 0};
        }
    }
}

void add_edge(int u, int v, int64_t w) { g[u + 1][v + 1].w = g[v + 1][u + 1].w = w; }

int64_t solve() {
    nx = n;
    for (int u = 1; u <= n; u++) {
        st[u] = u;
        match_[u] = 0;
        flo[u].clear();
    }
    int64_t w_max = 0;
    for (int u = 1; u <= n; u++) {
        for (int v = 1; v <= n; v++) {
            flo_from[u][v] = (u == v ? u : 0);
            w_max = std::max(w_max, g[u][v].w);
        }
    }
    for (int u = 1; u <= n; u++) {
        lab[u] = w_max;
    }
    while (matching()) {
    }
    int64_t weight = 0;
    for (int u = 1; u <= n; u++) {
        if (match_[u] != 0 && match_[u] < u) {
            weight += g[u][match_[u]].w;
        }
    }
    return weight;
}

int partner(int u) const {
    int v = match_[u + 1];
    // A node "matched" through an absent (weight 0) edge is really unmatched.
    if (v == 0 || g[u + 1][v].w == 0) {
        return -1;
    }
}

```

```

    return v - 1;
}
};

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // A 5-cycle 0-1-2-3-4-0; the heaviest matching picks two disjoint edges.
    WeightedGeneralMatching m(5);
    m.add_edge(0, 1, 5);
    m.add_edge(1, 2, 6);
    m.add_edge(2, 3, 5);
    m.add_edge(3, 4, 6);
    m.add_edge(4, 0, 5);
    assert(m.solve() == 12); // Edges (1,2) and (3,4) with weights 6 + 6.
    assert(m.partner(1) == 2 && m.partner(2) == 1);
    assert(m.partner(3) == 4 && m.partner(4) == 3);
    assert(m.partner(0) == -1); // Node 0 is left unmatched.

    // A triangle plus a pendant edge: taking the pendant can beat a single triangle edge.
    WeightedGeneralMatching t(4);
    t.add_edge(0, 1, 3);
    t.add_edge(1, 2, 3);
    t.add_edge(0, 2, 3);
    t.add_edge(2, 3, 5);
    assert(t.solve() == 8); // (0,1) weight 3 and (2,3) weight 5.
    return 0;
}

```

4.6.6 Assignment (Hungarian Algorithm)

Solves the minimum-cost assignment problem for a rectangular cost matrix. Given n workers and m jobs (with $n \leq m$) where assigning worker `i` to complete job `j` would cost `cost[i][j]`, choose a distinct job for each worker so that the total assignment cost is minimized.

The classical Hungarian algorithm using potentials transforms a cost matrix using the principle that adding or subtracting constants from rows or columns does not change the optimal assignment. Workers are assigned one row at a time: each new row triggers a search for a least-cost augmenting path over zero-slack edges, with the potentials adjusted along the way to expose new zero-slack edges until the path reaches a free job. Note that while the input matrix is 0-based here, the internal calculations are 1-based.

- `hungarian(cost, &assignment)` returns the minimum total assignment cost for a 0-based `cost` matrix, while populating `assignment` with n values where `assignment[i]` is the chosen job for worker `i`, using 0-based job numbers.

Time Complexity: $O(n^2 \cdot m)$, where `cost` has n rows and m columns.

Space Complexity: $O(n + m)$ auxiliary heap space, not counting the input matrix and returned assignment.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <limits>
#include <vector>

template<typename T>
T hungarian(const std::vector<std::vector<T>> &cost, std::vector<int> *assignment) {
    static const T INF = std::numeric_limits<T>::max() / 4;

```

```

int n = static_cast<int>(cost.size());
int m = cost.empty() ? 0 : static_cast<int>(cost[0].size());
assert(n <= m);
std::vector<T> u(n + 1), v(m + 1), minv(m + 1);
std::vector<int> p(m + 1), way(m + 1);
std::vector<bool> used(m + 1);
for (int i = 1; i <= n; i++) {
    std::fill(minv.begin(), minv.end(), INF);
    std::fill(used.begin(), used.end(), false);
    p[0] = i;
    int j0 = 0;
    do {
        used[j0] = true;
        int i0 = p[j0], j1 = 0;
        T delta = INF;
        for (int j = 1; j <= m; j++) {
            if (used[j]) {
                continue;
            }
            T cur = cost[i0 - 1][j - 1] - u[i0] - v[j];
            if (cur < minv[j]) {
                minv[j] = cur;
                way[j] = j0;
            }
            if (minv[j] < delta) {
                delta = minv[j];
                j1 = j;
            }
        }
        for (int j = 0; j <= m; j++) {
            if (used[j]) {
                u[p[j]] += delta;
                v[j] -= delta;
            } else {
                minv[j] -= delta;
            }
        }
        j0 = j1;
    } while (p[j0] != 0);
    do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
    } while (j0 != 0);
}
assignment->assign(n, -1);
for (int j = 1; j <= m; j++) {
    if (p[j] != 0) {
        (*assignment)[p[j] - 1] = j - 1;
    }
}
return -v[0];
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int64_t>> cost{
        {9, 2, 7, 8},
        {6, 4, 3, 7},
        {5, 8, 1, 8},
    };
    vector<int> assignment;
}

```

```

assert(hungarian(cost, &assignment) == 9);
assert(assignment[0] == 1);
assert(assignment[1] == 0);
assert(assignment[2] == 2);
return 0;
}

```

4.6.7 Stable Marriage (Gale-Shapley)

The stable marriage problem pairs n men with n women, given that every person ranks all members of the opposite group in strict order of preference. A perfect matching is stable if no man and woman both prefer each other to their assigned partners; such a pair would otherwise abandon their matches. Unlike maximum matching, the goal is not the size of the matching (it is always perfect) but its stability with respect to the preference lists.

The Gale-Shapley algorithm produces a stable matching, and a stable matching always exists. Each free man proposes to the most preferred woman who has not yet rejected him. A woman provisionally accepts her best proposal so far and rejects the rest, possibly breaking a previous engagement when a man she prefers proposes later. Because every man descends his list at most once and each proposal is constant work after precomputing the women's rankings, the algorithm runs in quadratic time. The result is man-optimal: every man receives the best partner he could have in any stable matching.

- `stable_marriage(men_pref, women_pref)` returns a vector `wife` where `wife[m]` is the woman matched to man `m` in the man-optimal stable matching, given the input preference lists each as n by n matrices. `men_pref[m]` lists the women in `m`'s order of preference, most preferred first, and `women_pref[w]` lists the men in `w`'s order of preference.

Time Complexity: $O(n^2)$ per call, where n is the number of men (equivalently, women).

Space Complexity: $O(n^2)$ auxiliary heap space for the precomputed rankings.

```

#include <queue>
#include <vector>

std::vector<int> stable_marriage(
    const std::vector<std::vector<int>>> &men_pref, const std::vector<std::vector<int>>> &women_pref
) {
    int n = static_cast<int>(men_pref.size());
    std::vector<std::vector<int>>> rank(n, std::vector<int>(n));
    for (int w = 0; w < n; w++) {
        for (int i = 0; i < n; i++) {
            rank[w][women_pref[w][i]] = i; // How woman w ranks each man (smaller is better).
        }
    }
    std::vector<int> next_proposal(n, 0), husband(n, -1), wife(n, -1);
    std::queue<int> free_men;
    for (int m = 0; m < n; m++) {
        free_men.push(m);
    }
    while (!free_men.empty()) {
        int m = free_men.front();
        free_men.pop();
        int w = men_pref[m][next_proposal[m]++];
        if (husband[w] == -1) {
            husband[w] = m;
            wife[m] = w;
        } else if (rank[w][m] < rank[w][husband[w]]) {
            wife[husband[w]] = -1;
            free_men.push(husband[w]);
            husband[w] = m;
            wife[m] = w;
        } else {
            free_men.push(m); // Rejected; m remains free and will propose to the next woman.
        }
    }
}

```

```

    }
}
return wife;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // All three men prefer w0 > w1 > w2; all three women prefer m2 > m1 > m0.
    vector<vector<int>> men_pref{{0, 1, 2}, {0, 1, 2}, {0, 1, 2}};
    vector<vector<int>> women_pref{{2, 1, 0}, {2, 1, 0}, {2, 1, 0}};

    // Cascading proposals leave m2 with w0, m1 with w1, m0 with w2.
    assert((stable_marriage(men_pref, women_pref) == vector<int>{2, 1, 0}));
    return 0;
}

```

4.7 Exponential Graph Problems

4.7.1 Maximum Clique (Bron-Kerbosch)

Given an undirected graph, determine a maximum clique: a largest subset of nodes in which every pair is connected by an edge. A weighted variant instead seeks the clique of maximum total node weight, given a weight for each node.

The Bron-Kerbosch algorithm recursively extends a growing clique, tracking the set of candidate nodes that may still be added and the set of nodes already excluded. Choosing a pivot node to avoid branching on its neighbors prunes large parts of the search, keeping it efficient on most graphs.

- `max_clique()` returns the maximum clique size for a global, bidirectionally pre-populated adjacency matrix `adj` which must consist of nodes numbered $[0, n)$, where `n` is `adj.size()`.
- `max_clique_weighted()` additionally uses global `w` and returns the maximum clique weight.

These implementations use bitmasks of unsigned 64-bit integers, so the number of nodes must be less than 64.

Time Complexity: $O(3^{n/3})$ per call to `max_clique()` and `max_clique_weighted()`, where n is the number of nodes.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(n)$ auxiliary stack space for `max_clique()` and `max_clique_weighted()`.

```

#include <algorithm>
#include <cstdint>
#include <vector>

std::vector<std::vector<bool>> adj;
std::vector<int> w;

std::vector<uint64_t> build_mask_graph() {
    int n = static_cast<int>(adj.size());
    std::vector<uint64_t> g(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j]) {
                g[i] |= 1ULL << j;
            }
        }
    }
}

```

```

    }
}
return g;
}

int max_clique_rec(const std::vector<uint64_t> &g, uint64_t curr, uint64_t pool, uint64_t excl) {
    if (pool == 0) {
        return __builtin_popcountll(curr);
    }
    int res = 0;
    int pivot = __builtin_ctzll(pool | excl);
    uint64_t candidates = pool & ~g[pivot];
    while (candidates != 0) {
        int u = __builtin_ctzll(candidates);
        res = std::max(res, max_clique_rec(g, curr | (1ULL << u), pool & g[u], excl & g[u]));
        pool ^= 1ULL << u;
        excl |= 1ULL << u;
        candidates &= candidates - 1;
    }
    return res;
}

int max_clique() {
    int n = static_cast<int>(adj.size());
    std::vector<uint64_t> g = build_mask_graph();
    return max_clique_rec(g, 0, (1ULL << n) - 1, 0);
}

int max_clique_weighted_rec(
    const std::vector<uint64_t> &g, uint64_t curr, uint64_t pool, uint64_t excl
) {
    if (pool == 0) {
        int res = 0;
        while (curr != 0) {
            int u = __builtin_ctzll(curr);
            res += w[u];
            curr &= curr - 1;
        }
        return res;
    }
    int res = -1, pivot = __builtin_ctzll(pool | excl);
    uint64_t z = pool & ~g[pivot];
    while (z != 0) {
        int u = __builtin_ctzll(z);
        res = std::max(res, max_clique_weighted_rec(g, curr | (1ULL << u), pool & g[u], excl & g[u]));
        pool ^= 1ULL << u;
        excl |= 1ULL << u;
        z &= z - 1;
    }
    return res;
}

int max_clique_weighted() {
    int n = static_cast<int>(adj.size());
    std::vector<uint64_t> g = build_mask_graph();
    return max_clique_weighted_rec(g, 0, (1ULL << n) - 1, 0);
}

```

Example Usage

```

#include <cassert>

void add_edge(int u, int v) {
    adj[u][v] = true;
    adj[v][u] = true;
}

```

```

int main() {
    int nodes = 5;
    adj.assign(nodes, std::vector<bool>(nodes));
    w.assign(nodes, 0);
    add_edge(0, 1);
    add_edge(0, 2);
    add_edge(0, 3);
    add_edge(1, 2);
    add_edge(1, 3);
    add_edge(2, 3);
    add_edge(3, 4);
    add_edge(4, 2);
    w[0] = 10;
    w[1] = 20;
    w[2] = 30;
    w[3] = 40;
    w[4] = 50;
    assert(max_clique() == 4);
    assert(max_clique_weighted() == 120);
    return 0;
}

```

4.7.2 Graph Coloring (Backtracking)

Given an undirected graph, assign a color to every node such that no pair of adjacent nodes have the same color, and that the total number of colors used is minimized.

This implementation finds the chromatic number by backtracking: nodes (ordered by degree to prune sooner) are colored one at a time, each tried with every color already in use plus one new color, while the fewest colors seen in a complete coloring so far bounds the search and cuts off any branch at least as costly.

- `color_graph()` populates `color` and returns minimum color count for a global, bidirectionally pre-populated adjacency matrix `adj` which must consist of nodes numbered `[0, n)`, where `n` is `adj.size()`.

Time Complexity: Exponential on the number of nodes per call to `color_graph()`.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(n)$ auxiliary stack and heap space for `color_graph()`.

```

#include <algorithm>
#include <vector>

std::vector<std::vector<bool>> adj;
int min_colors;
std::vector<int> color, curr, id, degree;

void rec(int lo, int hi, int n, int used_colors) {
    if (used_colors >= min_colors) {
        return;
    }
    if (n == hi) {
        for (int i = lo; i < hi; i++) {
            color[id[i]] = curr[i];
        }
        min_colors = used_colors;
        return;
    }
    std::vector<bool> used(used_colors + 1);
    for (int i = 0; i < n; i++) {
        if (adj[id[n]][id[i]]) {
            used[curr[i]] = true;
        }
    }
}

```

```

    }
}
for (int i = 0; i <= used_colors; i++) {
    if (!used[i]) {
        int tmp = curr[n];
        curr[n] = i;
        rec(lo, hi, n + 1, std::max(used_colors, i + 1));
        curr[n] = tmp;
    }
}
}

int color_graph() {
    int n = static_cast<int>(adj.size());
    if (n == 0) {
        color.clear();
        return 0;
    }
    color.assign(n, 0);
    curr.assign(n, 0);
    id.assign(n + 1, 0);
    degree.assign(n + 1, 0);
    for (int i = 0; i <= n; i++) {
        id[i] = i;
        degree[i] = 0;
    }
    int res = 1, lo = 0;
    for (int hi = 1; hi <= n; hi++) {
        int best = hi;
        for (int i = hi; i < n; i++) {
            if (adj[id[hi] - 1][id[i]]) {
                degree[id[i]]++;
            }
            if (degree[id[best]] < degree[id[i]]) {
                best = i;
            }
        }
        std::swap(id[hi], id[best]);
        if (degree[id[hi]] == 0) {
            min_colors = n + 1;
            curr.assign(n, 0);
            rec(lo, hi, lo, 0);
            lo = hi;
            res = std::max(res, min_colors);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

void add_edge(int u, int v) {
    adj[u][v] = true;
    adj[v][u] = true;
}

int main() {
    int nodes = 5;
    adj.assign(nodes, std::vector<bool>(nodes));
    add_edge(0, 1);
    add_edge(0, 4);
    add_edge(1, 3);
}

```



```

add_edge(1, 4);
add_edge(2, 3);
add_edge(2, 4);
add_edge(3, 4);
int colors = color_graph();
cout << "Colored using " << colors << " color(s):" << endl;
for (int i = 0; i < colors; i++) {
    cout << "Color " << i + 1 << ":";
    for (int j = 0; j < nodes; j++) {
        if (color[j] == i) {
            cout << " " << j;
        }
    }
    cout << endl;
}
return 0;
}

```

Example Output

```

Colored using 3 color(s):
Color 1: 0 3
Color 2: 1 2
Color 3: 4

```

4.7.3 Edge Coloring (Misra-Gries)

Given a simple undirected graph with maximum degree D , assign a color to every edge so that no two edges sharing an endpoint receive the same color. Vizing's theorem guarantees that $D + 1$ colors always suffice for a simple graph (while D colors are clearly necessary), and the Misra-Gries algorithm constructs such a coloring. Deciding between D and $D + 1$ colors is NP-hard in general, though bipartite graphs can always be edge-colored with exactly D colors.

The algorithm colors edges one at a time. To color an edge (u, v) it grows a maximal “fan” of u : a sequence of neighbors starting at v in which each successive edge's color is free at the previous fan vertex. It then picks a color c free at u and a color d free at the fan's last vertex, and flips the colors along the maximal alternating c/d path leaving u . This frees d at u , after which rotating the fan shifts colors over by one and assigns d to the new edge. Each step keeps the partial coloring proper and never introduces a $(D + 2)$ -th color.

The graph must be simple: the implementation stores one color per ordered node pair in a dense matrix, so self-loops and parallel edges between the same pair cannot be represented.

- `edge_coloring(n, edges)` returns a vector of colors, one per entry of `edges`, using colors in the range $[0, D]$ where D is the maximum degree. Nodes are numbered $[0, n)$, and `edges[i]` is an undirected edge $u-v$.

Time Complexity: $O(n^2 \cdot m)$ in the worst case for `edge_coloring()`, where n is the number of nodes and m the number of edges; substantially faster in practice.

Space Complexity: $O(n^2)$ to store the per-edge colors.

```

#include <utility>
#include <vector>

class EdgeColoring {
    int n, max_deg;
    std::vector<std::vector<int>>> color; // color[u][v] is the color of edge (u, v), or 0 if none.

    bool is_free(int x, int c) const {
        for (int w = 0; w < n; w++) {
            if (color[x][w] == c) {
                return false;
            }
        }
    }
}

```

```

    return true;
}

int free_color(int x) const {
    int c = 1;
    while (!is_free(x, c)) {
        c++;
    }
    return c;
}

void set_color(int u, int v, int c) {
    color[u][v] = c;
    color[v][u] = c;
}

// Flip colors c and d along the maximal alternating path of those colors starting at x.
void invert_path(int x, int d, int c) {
    for (int w = 0; w < n; w++) {
        if (color[x][w] == d) {
            set_color(x, w, 0);
            invert_path(w, c, d);
            set_color(x, w, c);
            return;
        }
    }
}

public:
EdgeColoring(int n, const std::vector<std::pair<int, int>> &edges)
    : n(n), max_deg(0), color(n, std::vector<int>(n, 0)) {
    std::vector<int> deg(n, 0);
    for (const auto &e : edges) {
        max_deg = std::max(max_deg, std::max(++deg[e.first], ++deg[e.second]));
    }
    for (const auto &e : edges) {
        add_edge(e.first, e.second);
    }
}

void add_edge(int u, int v) {
    // Build a maximal fan of u starting at v: fan[0] = v, and color(u, fan[i+1]) is free at fan[i].
    std::vector<int> fan{v};
    std::vector<bool> used(n, false);
    used[v] = true;
    while (true) {
        int last = fan.back(), next = -1;
        for (int w = 0; w < n; w++) {
            if (!used[w] && color[u][w] != 0 && is_free(last, color[u][w])) {
                next = w;
                break;
            }
        }
        if (next == -1) {
            break;
        }
        fan.push_back(next);
        used[next] = true;
    }
    int c = free_color(u), d = free_color(fan.back());
    invert_path(u, d, c);
    // Find the prefix of the fan to rotate: the first vertex on which d is now free.
    int k = 0;
    while (k < static_cast<int>(fan.size()) && color[u][fan[k]] != d && !is_free(fan[k], d)) {
        k++;
    }
    for (int j = 0; j < k; j++) {
        set_color(u, fan[j], color[u][fan[j + 1]]);
    }
}

```

```

    }
    set_color(u, fan[k], d);
}

std::vector<int> coloring(const std::vector<std::pair<int, int>> &edges) const {
    std::vector<int> result;
    result.reserve(edges.size());
    for (const auto &e : edges) {
        result.push_back(color[e.first][e.second] - 1); // Shift internal 1..D+1 colors to 0..D.
    }
    return result;
}
};

std::vector<int> edge_coloring(int n, const std::vector<std::pair<int, int>> &edges) {
    return EdgeColoring(n, edges).coloring(edges);
}

```

Example Usage

```

#include <algorithm>
#include <cassert>
#include <set>
using namespace std;

int main() {
    // A 5-cycle has maximum degree 2 but is odd, so it needs 3 colors (D + 1).
    vector<pair<int, int>> cycle{{0, 1}, {1, 2}, {2, 3}, {3, 4}, {4, 0}};
    vector<int> col = edge_coloring(5, cycle);
    set<int> palette(col.begin(), col.end());
    assert(palette.size() == 3);

    // Verify the coloring is proper: edges sharing an endpoint differ in color.
    int m = static_cast<int>(cycle.size());
    for (int i = 0; i < m; i++) {
        for (int j = i + 1; j < m; j++) {
            bool share = cycle[i].first == cycle[j].first || cycle[i].first == cycle[j].second ||
                         cycle[i].second == cycle[j].first || cycle[i].second == cycle[j].second;
            if (share) {
                assert(col[i] != col[j]);
            }
        }
    }

    // A bipartite graph (here K_{2,3}) is edge-colored with exactly D colors.
    vector<pair<int, int>> bip{{0, 2}, {0, 3}, {0, 4}, {1, 2}, {1, 3}, {1, 4}};
    vector<int> bcol = edge_coloring(5, bip);
    set<int> bpal(bcol.begin(), bcol.end());
    assert(static_cast<int>(bpal.size()) == 3); // Max degree is 3.
    return 0;
}

```

4.7.4 Shortest Hamiltonian Path and Cycle (Held-Karp)

Given a complete, weighted, directed graph, find a minimum-length Hamiltonian path or cycle. A Hamiltonian path visits each node exactly once. A Hamiltonian cycle additionally returns to its starting node; this cycle version is the traveling salesman problem (TSP).

Both functions use the Held-Karp subset dynamic program, where `dp[s][i]` is the shortest path visiting exactly the nodes in `s` and ending at node `i`.

- `shortest_hamiltonian_path()` populates `path` and returns the minimum Hamiltonian path length for a global, pre-populated adjacency matrix `adj`.
- `shortest_hamiltonian_cycle()` fixes the start at node 0, populates `path`, and returns the minimum Hamiltonian cycle length for a global, pre-populated adjacency matrix `adj`.

Since this implementation uses bitmasks with signed 32-bit integers, the maximum number of nodes must be less than 31.

Time Complexity: $O(2^n \cdot n^2)$ per call, where n is the number of nodes.

Space Complexity:

- $O(n^2)$ for storage of the graph, where n is the number of nodes.
- $O(2^n \cdot n)$ auxiliary heap space.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

const int64_t INF = INT64_MAX / 4;
std::vector<std::vector<int64_t>> adj, dp;
std::vector<int> path;

int64_t shortest_hamiltonian_path() {
    int n = static_cast<int>(adj.size());
    assert(n >= 1);
    int max_mask = (1 << n) - 1;
    dp.assign(max_mask + 1, std::vector<int64_t>(n, INF));
    path.assign(n, 0);
    for (int i = 0; i < n; i++) {
        dp[1 << i][i] = 0;
    }
    for (int mask = 1; mask <= max_mask; mask++) {
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                for (int j = 0; j < n; j++) {
                    if ((mask & (1 << j)) != 0 && dp[mask ^ (1 << i)][j] != INF) {
                        dp[mask][i] = std::min(dp[mask][i], dp[mask ^ (1 << i)][j] + adj[j][i]);
                    }
                }
            }
        }
    }
    int64_t res = INF;
    for (int i = 0; i < n; i++) {
        res = std::min(res, dp[max_mask][i]);
    }
    int mask = max_mask, old = -1;
    for (int i = n - 1; i >= 0; i--) {
        int best = -1;
        for (int j = 0; j < n; j++) {
            if ((mask & (1 << j)) != 0 &&
                (best == -1 || dp[mask][best] + (old == -1 ? 0 : adj[best][old]) >
                 dp[mask][j] + (old == -1 ? 0 : adj[j][old]))) {
                best = j;
            }
        }
        path[i] = best;
        mask ^= 1 << best;
        old = best;
    }
    return res;
}

int64_t shortest_hamiltonian_cycle() {
    int n = static_cast<int>(adj.size());
```

```

assert(n >= 1);
if (n == 1) {
    path = {0};
    return 0;
}
int max_mask = (1 << n) - 1;
dp.assign(max_mask + 1, std::vector<int64_t>(n, INF));
path.assign(n, 0);
dp[1][0] = 0;
for (int mask = 1; mask <= max_mask; mask += 2) {
    for (int i = 1; i < n; i++) {
        if ((mask & (1 << i)) != 0) {
            for (int j = 0; j < n; j++) {
                if ((mask & (1 << j)) != 0 && dp[mask ^ (1 << i)][j] != INF) {
                    dp[mask][i] = std::min(dp[mask][i], dp[mask ^ (1 << i)][j] + adj[j][i]);
                }
            }
        }
    }
}
int64_t res = INF;
for (int i = 1; i < n; i++) {
    res = std::min(res, dp[max_mask][i] + adj[i][0]);
}
int mask = max_mask, old = 0;
for (int i = n - 1; i >= 1; i--) {
    int best = -1;
    for (int j = 1; j < n; j++) {
        if ((mask & (1 << j)) != 0 &&
            (best == -1 || dp[mask][best] + adj[best][old] > dp[mask][j] + adj[j][old])) {
            best = j;
        }
    }
    path[i] = best;
    mask ^= 1 << best;
    old = best;
}
return res;
}

```

Example Usage

```

#include <iostream>
using namespace std;

int main() {
    int nodes = 3;
    adj.assign(nodes, std::vector<int64_t>(nodes));
    adj[0][1] = 1;
    adj[0][2] = 1;
    adj[1][0] = 7;
    adj[1][2] = 2;
    adj[2][0] = 3;
    adj[2][1] = 5;

    cout << "Shortest Hamiltonian path length: " << shortest_hamiltonian_path() << endl;
    cout << "Path: " << path[0];
    for (int i = 1; i < nodes; i++) {
        cout << "->" << path[i];
    }
    cout << "\n\n";
    cout << "Shortest Hamiltonian cycle length: " << shortest_hamiltonian_cycle() << endl;
    cout << "Cycle: ";
    for (int i = 0; i < nodes; i++) {
        cout << path[i] << "->";
    }
}

```

```
    cout << path[0] << endl;  
    return 0;  
}
```

Example Output

Shortest Hamiltonian path length: 3
Path: 0->1->2

Shortest Hamiltonian cycle length: 6
Cycle: 0->1->2->0

Chapter 5

Optimization

5.1 Monotone Predicate Search

5.1.1 Binary Search

Binary search can be generally used to find the input value corresponding to any output value of a monotonic (strictly non-increasing or strictly non-decreasing) function in $O(\log n)$ time with respect to the domain size. This is a special case of finding the exact point at which any given monotonic Boolean function changes from true to false or vice versa. Each step evaluates the midpoint of the current interval and keeps the half that still contains the transition point, halving the search space until it closes on the answer. Unlike searching through an array, discrete binary search is not restricted by available memory, making it useful for handling infinitely large search spaces such as real number intervals.

- `binary_search_first_true(lo, hi, pred)` takes integer boundaries for the search space $[lo, hi)$ (i.e. including `lo`, but excluding `hi`) and returns the smallest integer `k` in $[lo, hi)$ for which the predicate `pred(k)` tests true. If `pred(k)` tests false for the entire input range, then `hi` is returned. The caller must ensure `pred` is monotonic on the input range, i.e. returning all false for some (possibly empty) prefix, followed by all true in some (possibly empty) suffix. E.g., patterns "01", "00", and "11" are allowed, but "10" is disallowed.
- `binary_search_last_true(lo, hi, pred)` takes integer boundaries for the search space $[lo, hi)$ (i.e. including `lo`, but excluding `hi`) and returns the largest integer `k` in $[lo, hi)$ for which the predicate `pred(k)` tests true. If `pred(k)` tests false for the entire input range, then `hi` is returned. The caller must ensure `pred` is monotonic on the input range, i.e. returning all true for some (possibly empty) prefix, followed by all false in some (possibly empty) suffix. E.g., patterns "10", "00", and "11" are allowed, but "01" is disallowed.
- `fbinary_search(lo, hi, pred)` is the equivalent of `binary_search_first_true()` on floating point predicates. Since any interval of real numbers is dense, the exact target cannot be found due to floating point error. Instead, a value that is very close to the border between false and true is returned. The precision of the answer depends on the number of repetitions the function performs. Since each repetition bisects the search space, the absolute error of the answer is $1/(2^n)$ times the distance between `lo` and `hi` after n repetitions. Although it is possible to control the error by looping until the distance shrinks to an arbitrary epsilon, it is simpler to let the loop run for a desired number of iterations until floating point arithmetic breaks down. 100 iterations is usually sufficient, since the search space will be reduced to 2^{-100} (roughly 10^{-30}) times its original size. This implementation can be modified to find the "last true" point in the range by simply interchanging the assignments of `lo` and `hi` in the if-else statements.

Time Complexity:

- $O(\log n)$ calls will be made to `pred()` in `binary_search_first_true()` and `binary_search_last_true()`, where n is the distance between `lo` and `hi`.
- $O(n)$ calls will be made to `pred()` in `fbinary_search()`, where n is the number of iterations performed.

Space Complexity: $O(1)$ auxiliary for all operations.

```

template<typename Int, typename Pred>
Int binary_search_first_true(Int lo, Int hi, Pred pred) { // 000[1]11
    while (lo < hi) {
        Int mid = lo + (hi - lo) / 2;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}

template<typename Int, typename Pred>
Int binary_search_last_true(Int lo, Int hi, Pred pred) { // 11[1]000
    Int _hi = hi;
    if (lo == hi) {
        return _hi;
    }
    hi--;
    while (lo < hi) {
        Int mid = lo + (hi - lo + 1) / 2;
        if (pred(mid)) {
            lo = mid;
        } else {
            hi = mid - 1;
        }
    }
    if (!pred(lo)) {
        return _hi; // All false.
    }
    return lo;
}

template<typename Pred>
double fbinary_search(double lo, double hi, Pred pred) { // 000[1]11
    double mid;
    for (int i = 0; i < 100; i++) {
        mid = (lo + hi) / 2.0;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid;
        }
    }
    return lo;
}

```

Example Usage

```

#include <cassert>
#include <cmath>

int main() {
    assert(binary_search_first_true(0, 7, [](int x) { return x >= 3; }) == 3);
    assert(binary_search_first_true(0, 7, [](int x) { return false; }) == 7);

    assert(binary_search_last_true(0, 7, [](int x) { return x <= 5; }) == 5);
    assert(binary_search_last_true(0, 7, [](int x) { return true; }) == 6);

    double res = fbinary_search(-10.0, 10.0, [](double x) { return x >= 1.2345; });
    assert(fabs(res - 1.2345) < 1e-15);
    return 0;
}

```


5.1.2 Exponential Search

Finds the first value where a monotone Boolean predicate becomes true when an upper bound is not known in advance. Exponential search first grows the search range by powers of two until it brackets the transition point, then finishes with ordinary binary search.

This is useful for answer-search problems over unbounded or very large integer domains. The predicate must eventually become true, or the search may overflow or run forever unless an explicit limit is added.

- `exponential_search_first_true(lo, pred)` returns the smallest integer `x` greater than or equal to `lo` such that `pred(x)` is true.

Time Complexity: $O(\log n)$ calls to `pred()`, where n is the distance from `lo` to the first true value.

Space Complexity: $O(1)$ auxiliary.

```
#include <cstdint>

template<typename Int, typename Pred>
Int exponential_search_first_true(Int lo, Pred pred) { // 000[1]11
    if (pred(lo)) {
        return lo;
    }
    Int step = 1;
    while (!pred(lo + step)) {
        step *= 2;
    }
    Int l = lo + step / 2, r = lo + step;
    while (l < r) {
        Int mid = l + (r - l) / 2;
        if (pred(mid)) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}
```

Example Usage

```
#include <cassert>

bool at_least_1000(int x) {
    return x >= 1000;
}

bool square_large_enough(int64_t x) {
    return x * x >= 123456789LL;
}

int main() {
    assert(exponential_search_first_true(0, at_least_1000) == 1000);
    assert(exponential_search_first_true(5, at_least_1000) == 1000);
    assert(exponential_search_first_true(0LL, square_large_enough) == 11112);
    return 0;
}
```

5.2 Unimodal and Continuous Search

5.2.1 Ternary and Golden Section Search

Given a unimodal function f , find the position of its global minimum or maximum: a value x such that the function strictly decreases up to x and strictly increases after it (for a minimum), or the reverse (for a maximum). All three searches below exploit the same idea: evaluate two interior points, and the comparison reveals one side of the interval that cannot contain the optimum, so it can be discarded. They differ only in how the probes are placed and whether the domain is continuous or discrete. Each maximum routine is the minimum routine applied to the negated function, since the location of a maximum of f is the location of a minimum of $-f$.

Ternary search splits the interval into thirds. It is the simplest to reason about, but it recomputes both probe points each step, spending two function evaluations per iteration while the interval shrinks to two-thirds.

Golden-section search places the probes at the golden ratio so that one probe is reused as an interior point of the next, smaller interval. It therefore performs only one new evaluation per iteration while shrinking the interval by a factor of $1/\varphi$ (about 0.618), and reaches a given accuracy in roughly 2.4 times fewer evaluations than ternary search. Prefer it when the function is expensive to evaluate.

The discrete version operates on an integer domain, narrowing by thirds until at most three candidates remain and then scanning them. It returns the index of the optimum rather than a real coordinate. As with ternary search on reals, the function must be strictly unimodal.

- `ternary_search_min(lo, hi, f)` and `ternary_search_max(lo, hi, f)` return a `double x` in $[lo, hi]$ optimizing the continuous unimodal function `f`, to within the optional absolute error `EPS`.
- `golden_section_min(lo, hi, f)` and `golden_section_max(lo, hi, f)` do the same with one function evaluation per iteration.
- `discrete_ternary_min(lo, hi, f)` and `discrete_ternary_max(lo, hi, f)` return the integer index in the inclusive range $[lo, hi]$ optimizing the unimodal function `f`, breaking ties toward the smaller index.

Time Complexity:

- $O(\log(n/\text{EPS}))$ calls to `f()` for the continuous searches, where n is the distance between `lo` and `hi`. Golden-section makes about half as many per iteration as ternary search.
- $O(\log n)$ calls to `f()` for the discrete searches.

Space Complexity: $O(1)$ auxiliary for all operations.

```
template<typename Fn>
double ternary_search_min(double lo, double hi, Fn f, double EPS = 1e-12) {
    while (hi - lo > EPS) {
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) {
            hi = m2;
        } else {
            lo = m1;
        }
    }
    return (lo + hi) / 2;
}
```

```
template<typename Fn>
double ternary_search_max(double lo, double hi, Fn f, double EPS = 1e-12) {
    return ternary_search_min(lo, hi, [&](double x) { return -f(x); }, EPS);
}
```

```
template<typename Fn>
double golden_section_min(double lo, double hi, Fn f, double EPS = 1e-12) {
    const double r = 0.6180339887498949; // 1 / phi = (sqrt(5) - 1) / 2.
    double m1 = hi - r * (hi - lo), m2 = lo + r * (hi - lo);
    double f1 = f(m1), f2 = f(m2);
    while (hi - lo > EPS) {
```

```

    if (f1 < f2) {
        hi = m2;
        m2 = m1;
        f2 = f1;
        m1 = hi - r * (hi - lo);
        f1 = f(m1);
    } else {
        lo = m1;
        m1 = m2;
        f1 = f2;
        m2 = lo + r * (hi - lo);
        f2 = f(m2);
    }
}
return (lo + hi) / 2;
}

template<typename Fn>
double golden_section_max(double lo, double hi, Fn f, double EPS = 1e-12) {
    return golden_section_min(lo, hi, [&](double x) { return -f(x); }, EPS);
}

template<typename Int, typename Fn>
Int discrete_ternary_min(Int lo, Int hi, Fn f) {
    while (hi - lo > 2) {
        Int m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) {
            hi = m2 - 1;
        } else {
            lo = m1 + 1;
        }
    }
    Int best = lo;
    for (Int k = lo + 1; k <= hi; k++) {
        if (f(k) < f(best)) {
            best = k;
        }
    }
    return best;
}

template<typename Int, typename Fn>
Int discrete_ternary_max(Int lo, Int hi, Fn f) {
    return discrete_ternary_min(lo, hi, [&](Int x) { return -f(x); });
}

```

Example Usage

```

#include <cassert>
#include <cmath>
#include <vector>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-6;
}

int main() {
    // Parabola opening up with vertex at x = -2.
    auto up = [](double x) { return 3 * x * x + 12 * x - 12; };
    // Parabola opening down with vertex at x = 2/19.
    auto down = [](double x) { return -5.7 * x * x + 1.2 * x + 88; };

    assert(EQ(ternary_search_min(-1000, 1000, up), -2));
    assert(EQ(golden_section_min(-1000, 1000, up), -2));
    assert(EQ(ternary_search_max(-1000, 1000, down), 2.0 / 19));
}

```

```

assert(EQ(golden_section_max(-1000, 1000, down), 2.0 / 19));

// Discrete unimodal arrays: a valley and a peak.
vector<int> valley{5, 3, 1, 2, 4, 6};
vector<int> peak{1, 3, 7, 6, 2};
auto v = [&](int i) { return valley[i]; };
auto p = [&](int i) { return peak[i]; };
assert(discrete_ternary_min(0, static_cast<int>(valley.size()) - 1, v) == 2); // Value 1.
assert(discrete_ternary_max(0, static_cast<int>(peak.size()) - 1, p) == 2);   // Value 7.
return 0;
}

```

5.2.2 Hill Climbing

Given a continuous function $f(x, y)$ returning a real number and a (possibly arbitrary) starting guess (x_0, y_0) , search for a global minimum using the hill-climbing heuristic.

Hill-climbing is a heuristic which starts at the guess, then considers taking a single step in each of a fixed number of directions. The direction with the best (in this case, minimum) value is chosen, and further steps are taken in it until the answer no longer improves. When this happens, the step size is reduced and the same process repeats until a desired absolute error is reached. The technique's success heavily depends on the behavior of f and the initial guess. Therefore, the result is not guaranteed to be the global minimum.

- `find_min(f, x0, y0, &critical_x, &critical_y)` returns a candidate global minimum value of function `f` reached by hill-climbing from the starting guess `(x0, y0)`. If the optional pointers `critical_x` and `critical_y` are supplied, the point attaining the returned value is stored through them. The step-size bounds and number of directions sampled are additional optional parameters.

Time Complexity: $O(d \log n)$ calls will be made to `f()`, where d is the number of directions considered at each position and n is the search space that is approximately proportional to the maximum possible step size divided by the minimum possible step size.

Space Complexity: $O(1)$ auxiliary.

```

#include <cmath>
#include <cstdint>

template<typename Fn>
double find_min(
    Fn f, double x0, double y0, double *critical_x = nullptr, double *critical_y = nullptr,
    const double STEP_MIN = 1e-9, const double STEP_MAX = 1e6, const int NUM_DIRECTIONS = 6
) {
    static const double PI = acos(-1.0);
    double x = x0, y = y0, res = f(x0, y0);
    for (double step = STEP_MAX; step > STEP_MIN;) {
        double best = res, best_x = x, best_y = y;
        bool found = false;
        for (int i = 0; i < NUM_DIRECTIONS; i++) {
            double a = 2.0 * PI * i / NUM_DIRECTIONS;
            double x2 = x + step * cos(a), y2 = y + step * sin(a);
            double value = f(x2, y2);
            if (best > value) {
                best_x = x2;
                best_y = y2;
                best = value;
                found = true;
            }
        }
        if (!found) {
            step /= 2.0;
        } else {
            x = best_x;
            y = best_y;
        }
    }
}

```

```

        res = best;
    }
}
if (critical_x != nullptr && critical_y != nullptr) {
    *critical_x = x;
    *critical_y = y;
}
return res;
}

```

Example Usage

```

#include <cassert>
#include <cmath>

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

// Paraboloid with global minimum at f(2, 3) = 0.
double f(double x, double y) {
    return (x - 2) * (x - 2) + (y - 3) * (y - 3);
}

int main() {
    double x, y;
    assert(EQ(find_min(f, 0, 0, &x, &y), 0));
    assert(EQ(x, 2) && EQ(y, 3));
    return 0;
}

```

5.2.3 Simulated Annealing

Given a function $f(x, y)$ to minimize over two real variables, return a good approximate minimum found by simulated annealing. This is a randomized heuristic for difficult search spaces where gradients, convexity, and unimodality are not available.

At each temperature, the algorithm proposes a random nearby point. Better moves are always accepted, while worse moves are accepted with probability $\exp((\text{current} - \text{candidate}) / \text{temperature})$. This lets the search sometimes escape local minima early on, then become greedier as the temperature decreases.

Simulated annealing is not a proof of optimality. Its quality depends heavily on the objective function, the starting point, the initial temperature, the cooling rate, and the number of independent restarts.

- `anneal_min(f, x0, y0, &best_x, &best_y)` returns a small value found by the randomized search starting from `(x0, y0)`. If the optional pointers `best_x` and `best_y` are supplied, the point attaining the returned value is stored through them.
- `TEMPERATURE_START` controls the initial move scale and willingness to accept worse states.
- `TEMPERATURE_END` controls when the search stops.
- `COOLING_RATE` controls how quickly the temperature decreases.

Time Complexity: $O(r \log n)$ calls to `f()`, where r is the number of restarts and n is roughly `TEMPERATURE_START / TEMPERATURE_END`.

Space Complexity: $O(1)$ auxiliary.

```

#include <cmath>
#include <random>

const double TEMPERATURE_START = 10.0;
const double TEMPERATURE_END = 1e-7;

```

```

const double COOLING_RATE = 0.997;
const int NUM_RESTARTS = 8;

template<typename Fn>
double anneal_min(Fn f, double x0, double y0, double *best_x = nullptr, double *best_y = nullptr) {
    auto rand_unit = []() {
        static std::mt19937 rng(1234567); // Set your own seed, or use std::random_device{}()
        return std::uniform_real_distribution<double>(0.0, 1.0)(rng);
    };
    double sol_x = x0, sol_y = y0, best = f(x0, y0);
    for (int restart = 0; restart < NUM_RESTARTS; restart++) {
        double x = x0, y = y0, cur = f(x, y);
        double temperature = TEMPERATURE_START;
        while (temperature > TEMPERATURE_END) {
            double nx = x + (2.0 * rand_unit() - 1.0) * temperature;
            double ny = y + (2.0 * rand_unit() - 1.0) * temperature;
            double next = f(nx, ny);
            double probability = exp((cur - next) / temperature);
            if (next < cur || rand_unit() < probability) {
                x = nx;
                y = ny;
                cur = next;
                if (cur < best) {
                    best = cur;
                    sol_x = x;
                    sol_y = y;
                }
            }
            temperature *= COOLING_RATE;
        }
    }

    if (best_x != nullptr) {
        *best_x = sol_x;
    }
    if (best_y != nullptr) {
        *best_y = sol_y;
    }
    return best;
}

```

Example Usage

```

#include <cassert>

bool close(double a, double b) {
    return fabs(a - b) < 1e-2;
}

// Smooth bowl with global minimum at f(2, -3) = 0.
double f(double x, double y) {
    return (x - 2) * (x - 2) + (y + 3) * (y + 3);
}

int main() {
    double x, y;
    double value = anneal_min(f, 0, 0, &x, &y);
    assert(value < 1e-4);
    // value < 1e-4 already forces |x-2|, |y+3| < 1e-2 = sqrt(1e-4); these confirm the out-params.
    assert(close(x, 2));
    assert(close(y, -3));
    return 0;
}

```

5.3 Dynamic Programming Optimization

5.3.1 Monotone Queue DP Optimization

Maintains the minimum or maximum value in a sliding window using a monotone queue. This is useful for dynamic programming recurrences where each transition may only come from one of the last w states, such as `dp[i] = a[i] + min(dp[j])` for $j \in [i - w, i - 1]$.

The queue stores candidate (index, value) pairs in monotone order. Expired indices are removed from the front, and dominated values are removed from the back before inserting a new candidate.

- `MonotoneQueue<Compare>()` constructs an empty queue. Use `std::less<T>` for minimum queries and `std::greater<T>` for maximum queries.
- `push(index, value)` inserts candidate `value` at position `index`, removing dominated candidates.
- `expire(first_valid)` removes candidates with index less than `first_valid`.
- `top()` returns the best active (index, value) pair.
- `empty()` returns whether the queue has no active candidates.

Time Complexity:

- $O(n)$ for any sequence of n calls to `push(index, value)` and `expire(first_valid)`.
- $O(1)$ amortized per call to `push(index, value)` and `expire(first_valid)`.
- $O(1)$ worst-case per call to `top()` and `empty()`.

Space Complexity: $O(w)$ for a sliding window of width w .

```
#include <deque>
#include <functional>
#include <utility>

template<typename T, typename Compare>
class MonotoneQueue {
    std::deque<std::pair<int, T>> q;
    Compare better;

public:
    bool empty() const { return q.empty(); }

    void push(int index, const T &value) {
        while (!q.empty() && !better(q.back().second, value)) {
            q.pop_back();
        }
        q.emplace_back(index, value);
    }

    void expire(int first_valid) {
        while (!q.empty() && q.front().first < first_valid) {
            q.pop_front();
        }
    }

    std::pair<int, T> top() const { return q.front(); }
};
```

Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

int main() {
    vector<int> a{4, 2, 7, 1, 3, 6};
```

```

MonotoneQueue<int, less<int>> minq;
vector<int> window_min;
for (int i = 0; i < static_cast<int>(a.size()); i++) {
    minq.push(i, a[i]);
    minq.expire(i - 2);
    if (i >= 2) {
        window_min.push_back(minq.top().second);
    }
}
assert(window_min[0] == 2);
assert(window_min[1] == 1);
assert(window_min[2] == 1);
assert(window_min[3] == 1);

// dp[i] = a[i] + min(dp[j]) over max(0, i - 2) <= j < i.
vector<int> dp(a.size());
MonotoneQueue<int, less<int>> best;
dp[0] = a[0];
best.push(0, dp[0]);
for (int i = 1; i < static_cast<int>(a.size()); i++) {
    best.expire(i - 2);
    dp[i] = a[i] + best.top().second;
    best.push(i, dp[i]);
}
assert(dp[5] == 13);
return 0;
}

```

5.3.2 Divide-and-Conquer DP Optimization

Computes one layer of a dynamic programming recurrence whose optimal transition indices are monotone. This optimization applies to recurrences of the form $dp_cur[i] = \min(dp_prev[k] + cost(k, i))$, where the minimizing index for i never decreases as i increases.

The function below computes $dp_cur[l..r]$ by evaluating the midpoint first, then recursing on the left half with a restricted upper bound on the optimal k , and on the right half with a restricted lower bound. The caller is responsible for checking that the recurrence satisfies the required monotonicity condition.

- `compute_dc_layer(dp_prev, dp_cur, l, r, opt_l, opt_r, cost)` fills $dp_cur[l..r]$ using candidate transition indices in $[opt_l, opt_r]$. The template parameter `cost` must be callable such that `cost(k, i)` returns the transition cost from previous state k to current state i .

Time Complexity: $O(n \log n)$ calls to `cost(k, i)` per layer when each state has $O(n)$ possible transitions.

Space Complexity: $O(\log n)$ auxiliary stack space.

```

#include <algorithm>
#include <cstdint>
#include <vector>

const int64_t INF = (1LL << 62);

template<typename Cost>
void compute_dc_layer(
    const std::vector<int64_t> &dp_prev, std::vector<int64_t> &dp_cur, int l, int r, int opt_l,
    int opt_r, Cost cost
) {
    if (l > r) {
        return;
    }
    int mid = l + (r - l) / 2;
    int64_t best = INF;
    int best_k = opt_l;

```



```

int upper = std::min(mid, opt_r);
for (int k = opt_l; k <= upper; k++) {
    int64_t candidate = dp_prev[k] + cost(k, mid);
    if (candidate < best) {
        best = candidate;
        best_k = k;
    }
}
dp_cur[mid] = best;
compute_dc_layer(dp_prev, dp_cur, l, mid - 1, opt_l, best_k, cost);
compute_dc_layer(dp_prev, dp_cur, mid + 1, r, best_k, opt_r, cost);
}

```

Example Usage

```

#include <cassert>
using namespace std;

struct SquareSegmentCost {
    vector<int64_t> prefix;

    explicit SquareSegmentCost(const vector<int> &a) : prefix(a.size() + 1) {
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            prefix[i + 1] = prefix[i] + a[i];
        }
    }

    int64_t operator()(int k, int i) const {
        int64_t sum = prefix[i] - prefix[k];
        return sum * sum;
    }
};

int main() {
    vector<int> a{1, 2, 3, 4};
    int n = static_cast<int>(a.size());
    SquareSegmentCost cost(a);

    vector<int64_t> dp_prev(n + 1, INF), dp_cur(n + 1, INF);
    dp_prev[0] = 0;
    compute_dc_layer(dp_prev, dp_cur, 1, n, 0, n - 1, cost);
    assert(dp_cur[4] == 100); // One group: (1 + 2 + 3 + 4)^2.

    vector<int64_t> dp_two(n + 1, INF);
    compute_dc_layer(dp_cur, dp_two, 1, n, 0, n - 1, cost);
    assert(dp_two[4] == 52); // Split as [1, 2] and [3, 4].
    return 0;
}

```

5.3.3 SMAWK Row Minima

Finds the position of the minimum in every row of a totally monotone matrix in time linear in its dimensions, without ever materializing the matrix. A matrix is totally monotone (for row minima) when the column index of each row's leftmost minimum never decreases as the row index increases, and this holds not just for the whole matrix but for every submatrix. Monge matrices, where $M_{i,j} + M_{i+1,j+1} \leq M_{i,j+1} + M_{i+1,j}$, are the most common source of such matrices and arise throughout dynamic programming on intervals and partitions.

The SMAWK algorithm interleaves two steps. REDUCE discards columns that cannot hold the minimum of any remaining row, leaving at most as many candidate columns as active rows. INTERPOLATE recurses on the odd-indexed active rows, then recovers each even-indexed active row's minimum by scanning only the candidate-column range bounded by its already-solved neighbors; total monotonicity guarantees those scans

advance monotonically and cost $O(R + C)$ overall, where R is the number of rows and C is the number of columns in the original matrix. This removes the logarithmic factor of divide-and-conquer DP optimization, but unlike that technique it is purely offline: every matrix entry must be available on demand from the oracle, so it cannot be used when an entry depends on a row minimum computed earlier in the same pass.

- `smawk_row_minima(R, C, get)` returns a vector `arg` of length `R`, where `arg[r]` is the column index minimizing `get(r, c)` over $0 \leq c < C$ (on ties, the smallest qualifying column is chosen). The input matrix is supplied as a totally monotone function `get(r, c)` with bounds $R \geq 0$ and $C \geq 1$.

Time Complexity: $O(R + C)$ evaluations of `get` per call to `smawk_row_minima(R, C, get)`.

Space Complexity: $O(R + C)$ auxiliary heap space.

```
#include <vector>

template<typename Get>
std::vector<int> smawk_rec(
    const Get &get, const std::vector<int> &active_rows, const std::vector<int> &candidate_cols
) {
    int n = static_cast<int>(active_rows.size());
    std::vector<int> arg(n, -1);
    if (n == 0) {
        return arg;
    }
    // Base case: a single row or column is cheap to scan directly.
    if (n <= 2 || candidate_cols.size() <= 2) {
        for (int i = 0; i < n; i++) {
            for (int c : candidate_cols) {
                if (arg[i] == -1 || get(active_rows[i], c) < get(active_rows[i], arg[i])) {
                    arg[i] = c;
                }
            }
        }
        return arg;
    }
    // REDUCE: keep only columns that can be the minimum of some row.
    std::vector<int> kept;
    if (static_cast<int>(candidate_cols.size()) > n) {
        for (int c : candidate_cols) {
            while (!kept.empty()) {
                int r = active_rows[static_cast<int>(kept.size()) - 1];
                if (get(r, kept.back()) <= get(r, c)) {
                    break;
                }
                kept.pop_back();
            }
            kept.push_back(c);
        }
        if (static_cast<int>(kept.size()) < n) {
            kept.push_back(c);
        }
    }
    else {
        kept = candidate_cols;
    }
    // INTERPOLATE: solve the odd rows recursively, then fill the even rows between known bounds.
    std::vector<int> odd_rows;
    for (int i = 1; i < n; i += 2) {
        odd_rows.push_back(active_rows[i]);
    }
    std::vector<int> odd_arg = smawk_rec(get, odd_rows, kept);
    for (int i = 0; i < static_cast<int>(odd_arg.size()); i++) {
        arg[2 * i + 1] = odd_arg[i];
    }
    int k = 0;
    for (int i = 0; i < n; i += 2) {
        int hi = (i + 1 < n) ? arg[i + 1] : kept.back();
        int r = active_rows[i];
        while (true) {
```

```

    int c = kept[k];
    if (arg[i] == -1 || get(r, c) < get(r, arg[i])) {
        arg[i] = c;
    }
    if (c == hi) {
        break; // Stop at the upper bound; the next even row starts scanning from here.
    }
    k++;
}
}
return arg;
}

template<typename Get>
std::vector<int> smawk_row_minima(int R, int C, const Get &get) {
    std::vector<int> row_ids(R), col_ids(C);
    for (int i = 0; i < R; i++) {
        row_ids[i] = i;
    }
    for (int j = 0; j < C; j++) {
        col_ids[j] = j;
    }
    return smawk_rec(get, row_ids, col_ids);
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

int main() {
    // An explicit totally monotone matrix; row minima sit at non-decreasing columns.
    vector<vector<int>>> m{
        {10, 17, 13, 28, 23}, {17, 22, 16, 29, 23}, {24, 28, 22, 34, 24},
        {11, 13, 6, 17, 7},    {45, 44, 32, 37, 23},
    };
    auto get = [&](int r, int c) { return m[r][c]; };
    vector<int> arg = smawk_row_minima(5, 5, get);

    // Cross-check each reported column against a brute-force row scan.
    for (int r = 0; r < 5; r++) {
        int best = 0;
        for (int c = 1; c < 5; c++) {
            if (m[r][c] < m[r][best]) {
                best = c;
            }
        }
        assert(m[r][arg[r]] == m[r][best]); // Same minimum value (smallest index on ties).
    }
    return 0;
}

```

5.3.4 Knuth DP Optimization

Computes interval dynamic programming recurrences whose optimal split points are monotone. Knuth optimization applies to recurrences of the form $dp[l][r] = \min(dp[l][k] + dp[k][r]) + cost(l, r)$, where the optimal split $opt[l][r]$ satisfies $opt[l][r - 1] \leq opt[l][r] \leq opt[l + 1][r]$.

Common applications include optimal binary search trees and some range merging problems. The caller is responsible for verifying the quadrangle inequality and monotonicity assumptions for the chosen $cost(l, r)$.

- `knuth_interval_dp(n, cost, &opt)` computes minimum costs for all half-open intervals $[1, r)$ over n items. The template parameter `cost` must be callable such that `cost(l, r)` returns the interval cost added after choosing the best split. If the optional pointer `opt` is supplied, it is filled with the chosen split points.

Time Complexity: $O(n^2)$ calls to `cost(l, r)` and $O(n^2)$ candidate split checks.

Space Complexity: $O(n^2)$ for the `dp` and `opt` tables.

```
#include <algorithm>
#include <cstdint>
#include <cstdint>
#include <vector>

const int64_t INF = (1LL << 62);

template<typename Cost>
std::vector<std::vector<int64_t>> knuth_interval_dp(
    int n, Cost cost, std::vector<std::vector<int>> *out = nullptr
) {
    std::vector<std::vector<int64_t>> dp(n + 1, std::vector<int64_t>(n + 1, 0));
    std::vector<std::vector<int>> opt(n + 1, std::vector<int>(n + 1, 0));
    for (int i = 0; i <= n; i++) {
        opt[i][i] = i;
        if (i < n) {
            opt[i][i + 1] = i + 1;
        }
    }
    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len;
            dp[l][r] = INF;
            int start = std::max(opt[l][r - 1], l + 1);
            int finish = std::min(opt[l + 1][r], r - 1);
            for (int k = start; k <= finish; k++) {
                int64_t candidate = dp[l][k] + dp[k][r] + cost(l, r);
                if (candidate < dp[l][r]) {
                    dp[l][r] = candidate;
                    opt[l][r] = k;
                }
            }
        }
    }
    if (out != nullptr) {
        *out = opt;
    }
    return dp;
}
```

Example Usage

```
#include <cassert>
using namespace std;

struct MergeCost {
    vector<int64_t> prefix;

    explicit MergeCost(const vector<int> &a) : prefix(a.size() + 1) {
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            prefix[i + 1] = prefix[i] + a[i];
        }
    }

    int64_t operator()(int l, int r) const { return prefix[r] - prefix[l]; }
};

int main() {
```

```

vector<int> a{1, 2, 3, 4};
vector<vector<int64_t>> dp = knuth_interval_dp(a.size(), MergeCost(a));
assert(dp[0][4] == 19);
return 0;
}

```

5.3.5 Convex Hull Trick (Semi-Dynamic)

Given a set of pairs (m, b) specifying lines of the form $y = mx + b$, process a set of x -coordinate queries each asking to find the minimum y -value when any of the given lines are evaluated at the specified x . This is useful for dynamic programming recurrences of the form $dp[i] = \min(m[j] * x[i] + b[j])$. Only the lower envelope of the lines can ever answer a query, so each added line pops previously stored lines that it renders useless, and ascending queries advance a pointer along the envelope so every line is visited at most twice.

The following implementation is a concise, semi-dynamic version of the convex hull optimization technique. It supports an interlaced sequence of `add_line()` and `query()` calls, as long as the preconditions of descending `m` and ascending `x` are satisfied. As a result, it may be necessary to sort the lines and queries before calling the functions. In that case, the overall time complexity will be dominated by the sorting step.

- `SemiDynamicCHT()` constructs an empty hull.
- `add_line(m, b)` inserts the line $y = mx + b$. The caller must ensure that slope `m` is less than or equal to the slope of every line added so far.
- `query(x)` returns the minimum y -value among all inserted lines at coordinate `x`. The caller must ensure that query coordinates are nondecreasing across calls.

Overflow warning: `add_line()` compares intersections by cross-multiplying slope and intercept differences, a product on the order of the squared coefficient magnitude. For large `m/b` (roughly beyond 10^9 with `int64_t`), cast that comparison to `__int128`. `query()` only forms the single product `m * x`, which simply needs to fit in `int64_t`.

Time Complexity: $O(n)$ for any interlaced sequence of `add_line()` and `query()` calls, where n is the number of lines added. This is because the overall number of steps taken by `add_line()` and `query()` are respectively bounded by the number of lines. Thus a single call to either `add_line()` or `query()` will have an amortized $O(1)$ running time.

Space Complexity:

- $O(n)$ for storage of the lines.
- $O(1)$ auxiliary for `add_line()` and `query()`.

```

#include <cstdlib>
#include <vector>

struct SemiDynamicCHT {
    std::vector<int64_t> M, B;
    int ptr = 0;

    void add_line(int64_t m, int64_t b) {
        int len = static_cast<int>(M.size());
        if (len > 0 && M.back() == m) {
            if (B.back() <= b) {
                return;
            }
            M.pop_back();
            B.pop_back();
            len--;
            if (ptr > len) {
                ptr = len;
            }
        }
    }
};

// Overflow risk: this cross-multiplication is ~O(coeff^2); cast to __int128 for large m/b.
while (len > 1 && (B[len - 2] - B[len - 1]) * (m - M[len - 1]) >=

```

```

        (B[len - 1] - b) * (M[len - 1] - M[len - 2])) {
    len--;
}
M.resize(len);
B.resize(len);
M.push_back(m);
B.push_back(b);
}

int64_t query(int64_t x) {
    if (ptr >= static_cast<int>(M.size())) {
        ptr = static_cast<int>(M.size()) - 1;
    }
    while (ptr + 1 < static_cast<int>(M.size()) &&
           M[ptr + 1] * x + B[ptr + 1] <= M[ptr] * x + B[ptr]) {
        ptr++;
    }
    return M[ptr] * x + B[ptr];
}
};

```

Example Usage

```

#include <cassert>

int main() {
    SemiDynamicCHT cht;
    cht.add_line(3, 0);
    cht.add_line(2, 1);
    cht.add_line(1, 2);
    cht.add_line(0, 6);
    assert(cht.query(0) == 0);
    assert(cht.query(1) == 3);
    assert(cht.query(2) == 4);
    assert(cht.query(3) == 5);
    return 0;
}

```

5.3.6 Convex Hull Trick (Fully-Dynamic)

Given a set of pairs (m, b) specifying lines of the form $y = mx + b$, process a set of x -coordinate queries each asking to find the minimum y -value when any of the given lines are evaluated at the specified x . To instead have the queries optimize for maximum y -value, call the constructor with `query_max = true`. This is useful for dynamic programming recurrences of the form `dp[i] = min(m[j] * x[i] + b[j])` when line slopes and query coordinates are not monotone.

The following implementation is a fully dynamic variant of the convex hull optimization technique, using a self-balancing binary search tree (`std::set`) to support the ability to call `add_line()` and `query()` in any desired order. The tree stores the lower envelope of the lines sorted by slope: inserting a line removes any neighbors it dominates, and each line records the interval of queries for which it is the best, so a query is a single tree lookup.

- `HullOptimizer(query_max)` constructs an empty hull. By default, `query(x)` minimizes; if `query_max` is true, `query(x)` maximizes.
- `add_line(m, b)` inserts line $y = mx + b$ (can be called in any order).
- `query(x)` returns the best y -value among all inserted lines at coordinate x . At least one line must have been inserted.

Time Complexity: $O(\log n)$ amortized per call to `add_line()` and $O(\log n)$ per call to `query()`, where n is the number of lines added.

Space Complexity:

- $O(n)$ for storage of the lines.
- $O(1)$ auxiliary for `add_line()` and `query()`.

```

#include <cassert>
#include <cstdint>
#include <set>

class HullOptimizer {
    struct Line {
        int64_t m, b;
        mutable int64_t xhi;
        bool is_query;

        Line(int64_t m, int64_t b, int64_t xhi = 0, bool is_query = false)
            : m(m), b(b), xhi(xhi), is_query(is_query) {}

        bool operator< (const Line &l) const { return l.is_query ? xhi < l.xhi : m < l.m; }
    };

    std::multiset<Line> hull;
    bool query_max;

    using hulliter = std::multiset<Line>::iterator;

    static int64_t div_floor(int64_t a, int64_t b) { return a / b - ((a ^ b) < 0 && a % b); }

    bool update_border(hulliter x, hulliter y) {
        if (y == hull.end()) {
            x->xhi = INT64_MAX;
            return false;
        }
        if (x->m == y->m) {
            x->xhi = (x->b > y->b) ? INT64_MAX : INT64_MIN;
        } else {
            x->xhi = div_floor(y->b - x->b, x->m - y->m);
        }
        return x->xhi >= y->xhi;
    }

public:
    explicit HullOptimizer(bool query_max = false) : query_max(query_max) {}

    void add_line(int64_t m, int64_t b) {
        if (!query_max) {
            m = -m;
            b = -b;
        }
        auto z = hull.insert(Line(m, b));
        auto y = z++;
        auto x = y;
        while (update_border(y, z)) {
            z = hull.erase(z);
        }
        if (x != hull.begin() && update_border(--x, y)) {
            update_border(x, y = hull.erase(y));
        }
        while ((y = x) != hull.begin() && (--x)->xhi >= y->xhi) {
            update_border(x, hull.erase(y));
        }
    }

    int64_t query(int64_t x) const {
        assert(!hull.empty());
        Line q(0, 0, x, true);
        auto it = hull.lower_bound(q);
        int64_t res = it->m * x + it->b;
    }
};

```

```

    return query_max ? res : -res;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    HullOptimizer h;
    h.add_line(3, 0);
    h.add_line(0, 6);
    h.add_line(1, 2);
    h.add_line(2, 1);
    assert(h.query(0) == 0);
    assert(h.query(2) == 4);
    assert(h.query(1) == 3);
    assert(h.query(3) == 5);
    return 0;
}

```

5.3.7 Li Chao Tree

Maintains a dynamic set of lines and answers minimum value queries at points in a fixed integer domain. A Li Chao tree is useful for dynamic programming recurrences of the form $dp[i] = \min(m[j] * x[i] + b[j])$, especially when line slopes and query points arrive in arbitrary order. Each node of a segment tree over the domain keeps the line that wins at its segment's midpoint; inserting a line keeps the midpoint winner and recurses only into the half where the losing line may still win, and a query takes the best line along one root-to-leaf path.

This implementation stores the lower envelope of lines over the inclusive domain $[lo, hi]$. It supports adding arbitrary lines and querying arbitrary integer x -coordinates in logarithmic time with respect to the domain size.

- `LiChaoTree(lo, hi)` constructs an empty tree over integer domain $[lo, hi]$.
- `add_line(m, b)` inserts line $y = mx + b$ (can be called in any order).
- `query(x)` returns the minimum y -value among all inserted lines at coordinate x . At least one line must have been inserted.

Time Complexity: $O(\log d)$ per call to `add_line(m, b)` and `query(x)`, where d is the distance between `lo` and `hi`.

Space Complexity:

- $O(n \log d)$ worst-case node storage for n line inserted, usually much less.
- $O(\log d)$ auxiliary stack space per operation.

```

#include <algorithm>
#include <cassert>
#include <cstdint>

const int64_t INF = INT64_MAX / 4;

class LiChaoTree {
    struct Line {
        int64_t m, b;
        Line(int64_t m = 0, int64_t b = INF) : m(m), b(b) {}
        int64_t eval(int64_t x) const { return m * x + b; }
    };

    struct Node {
        Line f;
        Node *left, *right;
    };

```



```

    explicit Node(const Line &f) : f(f), left(nullptr), right(nullptr) {}
};

Node *root;
int64_t lo, hi;

static void clean_up(Node *n) {
    if (n == nullptr) {
        return;
    }
    clean_up(n->left);
    clean_up(n->right);
    delete n;
}

static void add_line(Node *&n, int64_t l, int64_t r, Line f) {
    if (n == nullptr) {
        n = new Node(f);
        return;
    }
    int64_t mid = l + (r - l) / 2;
    bool left_better = f.eval(l) < n->f.eval(l);
    bool mid_better = f.eval(mid) < n->f.eval(mid);
    if (mid_better) {
        std::swap(f, n->f);
    }
    if (l == r) {
        return;
    }
    if (left_better != mid_better) {
        add_line(n->left, l, mid, f);
    } else {
        add_line(n->right, mid + 1, r, f);
    }
}

static int64_t query(Node *n, int64_t l, int64_t r, int64_t x) {
    if (n == nullptr) {
        return INF;
    }
    int64_t res = n->f.eval(x);
    if (l == r) {
        return res;
    }
    int64_t mid = l + (r - l) / 2;
    if (x <= mid) {
        return std::min(res, query(n->left, l, mid, x));
    }
    return std::min(res, query(n->right, mid + 1, r, x));
}

public:
LiChaoTree(int64_t lo, int64_t hi) : root(nullptr), lo(lo), hi(hi) {}

~LiChaoTree() { clean_up(root); }
LiChaoTree(const LiChaoTree &) = delete;
LiChaoTree &operator=(const LiChaoTree &) = delete;
void add_line(int64_t m, int64_t b) { add_line(root, lo, hi, Line(m, b)); }

int64_t query(int64_t x) const {
    assert(lo <= x && x <= hi);
    return query(root, lo, hi, x);
}
};

```

Example Usage

```
#include <cassert>

int main() {
    LiChaoTree cht(-10, 10);
    cht.add_line(3, 0);
    cht.add_line(1, 2);
    cht.add_line(-1, 10);
    assert(cht.query(0) == 0);
    assert(cht.query(2) == 4);
    assert(cht.query(10) == 0);
    return 0;
}
```

5.4 Numerical Methods

5.4.1 Root Finding (Bracketing)

Finds an x in an interval $[a, b]$ for a continuous function f such that $f(x) = 0$. By the intermediate value theorem, a root must exist in $[a, b]$ if the signs of $f(a)$ and $f(b)$ differ. Each bisection step evaluates the midpoint of the interval and keeps the half whose endpoints still differ in sign, so a root always remains bracketed. The answer is found with an absolute error of roughly $1/2^n$, where n is the number of iterations. Although it is possible to control the error by looping while $b - a$ is greater than an arbitrary epsilon, it is simpler to let the loop run for a desired number of iterations until floating point arithmetic breaks down. 100 iterations is usually sufficient, since the search space will be reduced to 2^{-100} (roughly 10^{-30}) times its original size.

- `bisection_root(f, a, b)` returns a root in an interval $[a, b]$ for a continuous function f where $\text{sgn}(f(a)) \neq \text{sgn}(f(b))$, using the bisection method.
- `falsi_illinois_root(f, a, b)` returns a root in an interval $[a, b]$ for a continuous function f where $\text{sgn}(f(a)) \neq \text{sgn}(f(b))$, using the Illinois algorithm variant of the false position (a.k.a. regula falsi) method.

Time Complexity: $O(n)$ calls will be made to `f()` in `bisection_root()` and `falsi_illinois_root()`, where n is the number of iterations performed.

Space Complexity: $O(1)$ auxiliary space for both operations.

```
#include <stdexcept>

template<typename Fn>
double bisection_root(Fn f, double a, double b, const int ITERATIONS = 100) {
    if (a > b || f(a) * f(b) > 0) {
        throw std::runtime_error("Must give [a, b] where sgn(f(a)) != sgn(f(b)).");
    }
    double m;
    for (int i = 0; i < ITERATIONS; i++) {
        m = a + (b - a) / 2;
        if (f(a) * f(m) >= 0) {
            a = m;
        } else {
            b = m;
        }
    }
    return m;
}

template<typename Fn>
double falsi_illinois_root(Fn f, double a, double b, const int ITERATIONS = 100) {
    if (a > b || f(a) * f(b) > 0) {
        throw std::runtime_error("Must give [a, b] where sgn(f(a)) != sgn(f(b)).");
    }
    double m;
```

```

}
double m, fm, fa = f(a), fb = f(b);
int side = 0;
for (int i = 0; i < ITERATIONS; i++) {
    m = (fa * b - fb * a) / (fa - fb);
    fm = f(m);
    if (fb * fm > 0) {
        b = m;
        fb = fm;
        if (side < 0) {
            fa /= 2;
        }
        side = -1;
    } else if (fa * fm > 0) {
        a = m;
        fa = fm;
        if (side > 0) {
            fb /= 2;
        }
        side = 1;
    } else {
        break;
    }
}
return m;
}

```

Example Usage

```

#include <cassert>
#include <cmath>

double f(double x) {
    return x * x - 4 * sin(x);
}

int main() {
    assert(fabs(f(bisection_root(f, 1, 3))) < 1e-10);
    assert(fabs(f(falsi_illinois_root(f, 1, 3))) < 1e-10);
    return 0;
}

```

5.4.2 Root Finding (Iteration)

Finds an x for a continuous function f such that $f(x) = 0$ using iterative approximation by an initial guess that is close to the answer. Each step follows the tangent line at the current guess down to its x -intercept, that is, $x \leftarrow x - f(x)/f'(x)$. Newton's method requires an explicit definition of the function's derivative while the secant method starts with two initial guesses and approximates the derivative using the secant slope from the previous iteration. For n iterations and a good initial guess, the methods below compute approximately 2^n digits of precision, with the secant method converging approximately 1.6 times slower than Newton's.

- `newton_root(f, fprime, x0)` returns a root x for a function `f` with derivative `fprime` using an initial guess `x0` which should be relatively close to x .
- `secant_root(f, x0, x1)` returns a root x for a function `f` using two initial guesses `x0` and `x1` which should be relatively close to x .

Time Complexity: $O(n)$ calls will be made to `f()` in `newton_root()` and `secant_root()`, where n is the number of iterations performed.

Space Complexity: $O(1)$ auxiliary space for both operations.

```

#include <cmath>
#include <stdexcept>

template<typename Fn>
double newton_root(
    Fn f, Fn fprime, double x0, const double EPS = 1e-15, const int ITERATIONS = 100
) {
    double x = x0, error = EPS + 1;
    for (int i = 0; error > EPS && i < ITERATIONS; i++) {
        double xnew = x - f(x) / fprime(x);
        error = fabs(xnew - x);
        x = xnew;
    }
    if (error > EPS) {
        throw std::runtime_error("Newton's method failed to converge.");
    }
    return x;
}

template<typename Fn>
double secant_root(
    Fn f, double x0, double x1, const double EPS = 1e-15, const int ITERATIONS = 100
) {
    double xold = x0, fxold = f(x0), x = x1, error = EPS + 1;
    for (int i = 0; error > EPS && i < ITERATIONS; i++) {
        double fx = f(x);
        double xnew = x - fx * ((x - xold) / (fx - fxold));
        xold = x;
        fxold = fx;
        error = fabs(xnew - x);
        x = xnew;
    }
    if (error > EPS) {
        throw std::runtime_error("Secant method failed to converge.");
    }
    return x;
}

```

Example Usage

```

#include <cassert>

double f(double x) {
    return x * x - 4 * sin(x);
}

double fprime(double x) {
    return 2 * x - 4 * cos(x);
}

int main() {
    assert(fabs(f(newton_root(f, fprime, 3))) < 1e-10);
    assert(fabs(f(secant_root(f, 3, 2))) < 1e-10);
    return 0;
}

```

5.4.3 Polynomial Root Finding (Differentiation)

Finds every root x for a polynomial p such that $p(x) = 0$ by differentiation. Each adjacent pair of local extrema is searched using the bisection method, where local extrema are recursively found by finding the root of the derivative.

- `horner_eval(p, x)` evaluates the polynomial `p` of degree d (represented as a vector of size $d + 1$ where `p[i]` stores the coefficient for the x^i term) at `x`, using Horner's method.
- `find_one_root(p, a, b, EPS)` returns a root in the interval `[a, b]` for a polynomial `p` where $\text{sgn}(f(a)) \neq \text{sgn}(f(b))$, using the bisection method. If this precondition is not satisfied, then `NaN` is returned. The root is found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).
- `find_all_roots(p, a, b, EPS)` returns a vector of all roots in the interval `[a, b]` for a polynomial `p` using the bisection method. The roots are found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).

Time Complexity:

- $O(n)$ per call to `horner_eval()`, where n is the degree of the polynomial.
- $O(n \log d)$ per call to `find_one_root()`, where n is the degree of the polynomial and $d = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.
- $O(n^3 \log d)$ per call to `find_all_roots()`, where n is the degree of the polynomial and $d = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.

Space Complexity:

- $O(1)$ auxiliary space for `horner_eval()` and `find_one_root()`.
- $O(n^2)$ auxiliary heap and $O(n)$ auxiliary stack space for `find_all_roots()`, where n is the degree of the polynomial.

```
#include <cmath>
#include <limits>
#include <utility>
#include <vector>

double horner_eval(const std::vector<double> &p, double x) {
    double res = p.back();
    for (int i = static_cast<int>(p.size()) - 2; i >= 0; i--) {
        res = res * x + p[i];
    }
    return res;
}

double find_one_root(const std::vector<double> &p, double a, double b, const double EPS = 1e-15) {
    double pa = horner_eval(p, a), pb = horner_eval(p, b);
    bool paneg = pa < 0, pbneg = pb < 0;
    if (paneg == pbneg) {
        return std::numeric_limits<double>::quiet_NaN();
    }
    while (b - a > EPS && a * (1 + EPS) < b && a < b * (1 + EPS)) {
        double m = a + (b - a) / 2;
        if ((horner_eval(p, m) < 0) == paneg) {
            a = m;
        } else {
            b = m;
        }
    }
    return a;
}

std::vector<double> find_all_roots(
    const std::vector<double> &p, double a = -1e20, double b = 1e20, const double EPS = 1e-15
) {
    std::vector<double> pprime;
    pprime.reserve(p.size() > 0 ? p.size() - 1 : 0);
    for (int i = 1; i < static_cast<int>(p.size()); i++) {
        pprime.push_back(p[i] * i);
    }
    if (pprime.empty()) {
        return {};
    }
    std::vector<double> res, r = find_all_roots(pprime, a, b, EPS);
```

```

r.push_back(b);
for (int i = 0; i < static_cast<int>(r.size()); i++) {
    double root = find_one_root(p, i == 0 ? a : r[i - 1], r[i], EPS);
    if (!std::isnan(root) && (res.empty() || root != res.back())) {
        res.push_back(root);
    }
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    { // -1 + 2x - 6x^2 + 2x^3
        vector<double> p{-1.0, 2.0, -6.0, 2.0};
        vector<double> roots = find_all_roots(p);
        assert(roots.size() == 1 && fabs(horner_eval(p, roots[0])) < 1e-10);
    }
    { // -20 + 4x + 3x^2
        vector<double> p{-20.0, 4.0, 3.0};
        vector<double> roots = find_all_roots(p);
        assert(roots.size() == 2);
        assert(fabs(horner_eval(p, roots[0])) < 1e-10);
        assert(fabs(horner_eval(p, roots[1])) < 1e-10);
    }
    return 0;
}

```

5.4.4 Polynomial Root Finding (Laguerre)

Finds every complex root x for a polynomial p with complex coefficients such that $p(x) = 0$. Laguerre's method is a root-finding iteration with reliable, near-cubic convergence from almost any starting point. Once a root is found it is removed by polynomial deflation, and the iteration repeats on the lower-degree quotient until every root has been extracted.

- `horner_eval(p, x)` evaluates a complex polynomial p of degree d (represented as a vector of size $d + 1$ where `p[i]` stores the complex coefficient for the x^i term) at `x`, using Horner's method, returning a pair where the first value is the final result $p(x)$ and the second value is the quotient polynomial q from the identity $p(t) = (t - x)q(t) + p(x)$.
- `find_one_root(p, x0)` returns a complex root x for a polynomial `p` (represented as a vector of size $d + 1$ where `p[i]` stores the complex coefficient for the x^i term) using an initial guess `x0` which should be relatively close to x . The root is found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).
- `find_all_roots(p)` returns a vector of all complex roots for a complex polynomial `p`. The roots are found to a tolerance of `EPS` in absolute or relative error (whichever is reached first).

Time Complexity:

- $O(n)$ per call to `horner_eval()`, where n is the degree of the polynomial.
- $O(n \log d)$ per call to `find_one_root()`, where n is the degree of the polynomial and $d = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.
- $O(n^2 \log d)$ per call to `find_all_roots()`, where n is the degree of the polynomial and $d = -\log_{10}(\text{EPS})$ is the number of digits of absolute or relative precision that is desired.

Space Complexity:

- $O(n)$ auxiliary heap space and $O(1)$ auxiliary stack space for `horner_eval()` and `find_one_root()`, where n is the degree of the polynomial.

- $O(n)$ auxiliary heap and $O(1)$ auxiliary stack space for `find_one_root()` and `find_all_roots()`, where n is the degree of the polynomial.

```

#include <complex>
#include <random>
#include <vector>

using cdouble = std::complex<double>;
using cpoly = std::vector<cdouble>;

std::pair<cdouble, cpoly> horner_eval(const cpoly &p, const cdouble &x) {
    int n = static_cast<int>(p.size());
    cpoly b(std::max(1, n - 1));
    for (int i = n - 1; i > 0; i--) {
        b[i - 1] = p[i] + (i < n - 1 ? b[i] * x : 0);
    }
    return {p[0] + b[0] * x, b};
}

cpoly derivative(const cpoly &p) {
    int n = static_cast<int>(p.size());
    cpoly res(std::max(1, n - 1));
    for (int i = 1; i < n; i++) {
        res[i - 1] = p[i] * cdouble(i);
    }
    return res;
}

int comp(const cdouble &a, const cdouble &b, const double EPS = 1e-15) {
    double diff = std::abs(a) - std::abs(b);
    return (diff < -EPS) ? -1 : (diff > EPS ? 1 : 0);
}

cdouble find_one_root(
    const cpoly &p, const cdouble &x0, const double EPS = 1e-15, const int ITERATIONS = 10000
) {
    cdouble x = x0;
    int n = static_cast<int>(p.size()) - 1;
    cpoly p1 = derivative(p), p2 = derivative(p1);
    for (int i = 0; i < ITERATIONS; i++) {
        cdouble y0 = horner_eval(p, x).first;
        if (comp(y0, 0, EPS) == 0) {
            break;
        }
        cdouble g = horner_eval(p1, x).first / y0;
        cdouble h = g * g - horner_eval(p2, x).first / y0;
        cdouble r = std::sqrt(cdouble(n - 1) * (h * cdouble(n) - g * g));
        cdouble d1 = g + r, d2 = g - r;
        cdouble a = cdouble(n) / (comp(d1, d2, EPS) > 0 ? d1 : d2);
        x -= a;
        if (comp(a, 0, EPS) == 0) {
            break;
        }
    }
    return x;
}

std::vector<cdouble> find_all_roots(
    const cpoly &p, const double EPS = 1e-15, const int ITERATIONS = 10000
) {
    std::vector<cdouble> res;
    cpoly q = p;
    static std::mt19937 rng(std::random_device{}());
    std::uniform_real_distribution<double> unit(0.0, 1.0);
    while (q.size() > 2) {
        cdouble z(unit(rng), unit(rng));
        z = find_one_root(q, find_one_root(q, z, EPS, ITERATIONS), EPS, ITERATIONS);
    }
}

```

```

    q = horner_eval(q, z).second;
    res.push_back(z);
}
res.push_back(-q[0] / q[1]);
return res;
}

```

Example Usage

```

#include <algorithm>
#include <cstdio>
#include <iostream>
using namespace std;

void print_roots(vector<cdouble> x) {
    sort(x.begin(), x.end(), [](const cdouble &a, const cdouble &b) {
        return abs(a.real() - b.real()) > 0.5e-5 ? a.real() < b.real() : a.imag() < b.imag();
    });
    for (const auto &z : x) {
        double real = abs(z.real()) < 0.5e-5 ? 0 : z.real();
        double imag = abs(z.imag()) < 0.5e-5 ? 0 : z.imag();
        printf("(%.5lf, %.5lf)\n", real, imag);
    }
}

int main() {
    { // 140 - 13x - 8x^2 + x^3 = (x + 4)(x - 5)(x - 7)
        printf("Roots of 140 - 13x - 8x^2 + x^3:\n");
        cpoly p;
        p.push_back(140);
        p.push_back(-13);
        p.push_back(-8);
        p.push_back(1);
        print_roots(find_all_roots(p));
    }
    { // (-24+36i) + (-26+12i)x + (-30+40i)x^2 + (-26+12i)x^3 + (-6+4i)x^4
      // = ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
        printf("Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):\n");
        cpoly p;
        p.emplace_back(-24, 36);
        p.emplace_back(-26, 12);
        p.emplace_back(-30, 40);
        p.emplace_back(-26, 12);
        p.emplace_back(-6, 4);
        print_roots(find_all_roots(p));
    }
    return 0;
}

```

Example Output

```

Roots of 140 - 13x - 8x^2 + x^3:
(-4.00000, 0.00000)
(5.00000, 0.00000)
(7.00000, 0.00000)
Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
(-3.00000, -2.00000)
(-0.92308, 1.38462)
(0.00000, -1.00000)
(0.00000, 1.00000)

```


5.4.5 Polynomial Root Finding (Ehrlich-Aberth)

Finds every complex root x for a polynomial p such that $p(x) = 0$. The Ehrlich-Aberth method is a simultaneous iteration: every root estimate is updated using both the Newton correction and a repulsion term from the other estimates.

This routine is intended for well-scaled polynomials when all complex roots are wanted. If only real roots are needed, a real-root isolator such as derivative recursion with bisection is usually more reliable. Multiple or tightly clustered roots may converge slowly, and very large or very small root scales may require rescaling the input or using multiprecision arithmetic.

- `eval_with_derivative(p, x)` returns a pair $(p(x), p'(x))$ for a polynomial p given as a vector `p` where `p[i]` stores the coefficient for the x^i term.
- `find_all_roots(p, EPS, ITERATIONS)` returns a vector of all complex roots for a complex polynomial given by the vector of coefficients `p`. A `vector<LD>` overload is provided for polynomials with real coefficients. The roots are found to a tolerance of `EPS` in absolute or relative error (whichever is reached first), and zero roots are removed exactly before the simultaneous iteration starts.

Time Complexity:

- $O(n)$ per call to `eval_with_derivative(p, x)`, where n is the degree of the polynomial.
- $O(n^2t)$ per call to `find_all_roots(p, EPS, ITERATIONS)`, where n is the degree of the polynomial and t is the number of iterations required to reach the desired precision.

Space Complexity:

- $O(1)$ auxiliary space for `eval_with_derivative(p, x)`.
- $O(n)$ auxiliary heap space for `find_all_roots(p, EPS, ITERATIONS)`, where n is the degree of the polynomial.

```
#include <algorithm>
#include <cmath>
#include <complex>
#include <vector>

using LD = long double;
using cdouble = std::complex<LD>;
using cpoly = std::vector<cdouble>;

const LD PI = acosl(-1.0L);
const LD ZERO_EPS = 1e-30L; // Treat coefficients and denominators this small as zero.
const LD ROOT_EPS = 1e-18L; // Stop once root updates are this small relative to the root.
const LD CHECK_EPS = 1e-12L; // Residual tolerance used by the example assertions.

bool is_zero(const cdouble &z, const LD EPS = ZERO_EPS) {
    return std::abs(z) <= EPS;
}

bool is_finite(const cdouble &z) {
    return std::isfinite(static_cast<double>(z.real())) &&
        std::isfinite(static_cast<double>(z.imag()));
}

std::pair<cdouble, cdouble> eval_with_derivative(const cpoly &p, const cdouble &x) {
    cdouble value = p.back(), derivative = 0;
    for (int i = static_cast<int>(p.size()) - 2; i >= 0; i--) {
        derivative = derivative * x + value;
        value = value * x + p[i];
    }
    return {value, derivative};
}

LD root_bound(const cpoly &p) {
    LD res = 0;
    int n = static_cast<int>(p.size()) - 1;
    for (int i = 0; i < n; i++) {
```

```

    LD ratio = std::abs(p[i] / p.back());
    if (ratio > 0) {
        res = std::max(res, powl(ratio, 1.0L / (n - i)));
    }
}
return 2 * std::max((LD)1, res);
}

bool root_less(const cdouble &a, const cdouble &b) {
    if (a.real() != b.real()) {
        return a.real() < b.real();
    }
    return a.imag() < b.imag();
}

cpoly find_all_roots(cpoly p, const LD EPS = ROOT_EPS, const int ITERATIONS = 2000) {
    while (!p.empty() && is_zero(p.back())) {
        p.pop_back();
    }
    cpoly roots;
    while (p.size() > 1 && is_zero(p[0])) {
        roots.push_back(0);
        p.erase(p.begin());
    }
    if (p.size() <= 1) {
        return roots;
    }
    LD scale = 0;
    for (int i = 0; i < static_cast<int>(p.size()); i++) {
        scale = std::max(scale, std::abs(p[i]));
    }
    for (int i = 0; i < static_cast<int>(p.size()); i++) {
        p[i] /= scale;
    }
    int n = static_cast<int>(p.size()) - 1;
    if (n == 1) {
        roots.push_back(-p[0] / p[1]);
        return roots;
    }
    cpoly z(n);
    LD radius = root_bound(p, offset = PI / (2 * n));
    LD max_radius = 2 * radius;
    for (int i = 0; i < n; i++) {
        LD angle = offset + 2 * PI * i / n;
        z[i] = radius * cdouble(cosl(angle), sinl(angle));
    }
    for (int it = 0; it < ITERATIONS; it++) {
        bool done = true;
        cpoly next = z;
        for (int i = 0; i < n; i++) {
            auto [fx, dfx] = eval_with_derivative(p, z[i]);
            if (std::abs(fx) <= EPS) {
                continue;
            }
            cdouble repulsion = 0;
            for (int j = 0; j < n; j++) {
                if (i != j) {
                    cdouble diff = z[i] - z[j];
                    if (std::abs(diff) <= EPS * EPS) {
                        LD angle = 2 * PI * (i + 1) / (n + 1);
                        diff = EPS * (1 + std::abs(z[i])) * cdouble(cosl(angle), sinl(angle));
                    }
                    repulsion += cdouble(1) / diff;
                }
            }
            cdouble denom = dfx - fx * repulsion;
            if (is_zero(denom)) {
                continue;
            }

```

```

    }
    cdouble step = fx / denom;
    if (!is_finite(step) && !is_zero(dfx)) {
        step = fx / dfx;
    }
    if (!is_finite(step)) {
        continue;
    }
    LD limit = 2 * max_radius;
    if (std::abs(step) > limit) {
        step *= limit / std::abs(step);
    }
    next[i] = z[i] - step;
    if (!is_finite(next[i])) {
        next[i] = z[i];
        continue;
    }
    if (std::abs(next[i]) > max_radius) {
        next[i] *= max_radius / std::abs(next[i]);
    }
    if (std::abs(step) > EPS * (1 + std::abs(next[i]))) {
        done = false;
    }
}
z = next;
if (done) {
    break;
}
}
for (int i = 0; i < n; i++) {
    roots.emplace_back(z[i]);
}
std::sort(roots.begin(), roots.end(), root_less);
return roots;
}

std::vector<cdouble> find_all_roots(
    const std::vector<LD> &p, const LD EPS = ROOT_EPS, const int ITERATIONS = 2000
) {
    cpoly q(p.begin(), p.end());
    return find_all_roots(q, EPS, ITERATIONS);
}

```

Example Usage

```

#include <cassert>
#include <cstdio>
using namespace std;

cdouble eval(const cpoly &p, const cdouble &x) {
    return eval_with_derivative(p, x).first;
}

void print_roots(vector<cdouble> x) {
    sort(x.begin(), x.end(), [](const cdouble &a, const cdouble &b) {
        return abs(a.real() - b.real()) > 0.5e-5 ? a.real() < b.real() : a.imag() < b.imag();
    });
    for (const auto &z : x) {
        LD real = abs(z.real()) < 0.5e-5 ? 0 : z.real();
        LD imag = abs(z.imag()) < 0.5e-5 ? 0 : z.imag();
        printf("%.5Lf, %.5Lf\n", real, imag);
    }
}

int main() {
    { // -1 + 2x - 6x^2 + 2x^3

```

```

printf("Roots of -1 + 2x - 6x^2 + 2x^3:\n");
vector<LD> p{-1.0, 2.0, -6.0, 2.0};
vector<cdouble> roots = find_all_roots(p);
assert(roots.size() == 3);
for (const auto &root : roots) {
    assert(abs(eval(cpoly(p.begin(), p.end()), root)) < CHECK_EPS);
}
print_roots(roots);
}
{ // (-24+36i) + (-26+12i)x + (-30+40i)x^2 + (-26+12i)x^3 + (-6+4i)x^4
  // = ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
  printf("Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):\n");
  cpoly p;
  p.emplace_back(-24, 36);
  p.emplace_back(-26, 12);
  p.emplace_back(-30, 40);
  p.emplace_back(-26, 12);
  p.emplace_back(-6, 4);
  vector<cdouble> roots = find_all_roots(p);
  assert(roots.size() == 4);
  for (const auto &root : roots) {
      assert(abs(eval(p, root)) < CHECK_EPS);
  }
  print_roots(roots);
}
return 0;
}

```

Example Output

```

Roots of -1 + 2x - 6x^2 + 2x^3:
(0.15098, -0.40314)
(0.15098, 0.40314)
(2.69805, 0.00000)
Roots of ((2 + 3i)x + 6)(x + i)(2x + (6 + 4i))(xi + 1):
(-3.00000, -2.00000)
(-0.92308, 1.38462)
(0.00000, -1.00000)
(0.00000, 1.00000)

```

5.4.6 Integration (Simpson)

Computes the definite integral from a to b for a continuous function f using the composite Simpson's rule: the interval is split into n equal subintervals and each consecutive pair is approximated by a parabola, giving the approximation $\int_a^b f(x) dx \approx h/3 \cdot [f(x_0) + 4f(x_1) + 2f(x_2) + \dots + 4f(x_{n-1}) + f(x_n)]$ with step $h = (b - a)/n$.

- `integrate(f, a, b, n)` returns the definite integral of a function `f` from `a` to `b` using `n` subintervals, where `n` must be even. Larger `n` trades runtime for accuracy: the error is $O((b - a) \cdot h^4)$, so for a smooth `f` a value of `n` on the order of 10^6 reaches near double precision.

Unlike adaptive quadrature, the step size here is uniform, so a feature narrower than `h` may be under-resolved. The benefit is a simple, non-recursive routine with predictable cost that is not fooled by the premature-termination heuristic that adaptive Simpson's rule can suffer on functions whose coarse sample points happen to fit a parabola.

Time Complexity: $O(n)$ per call to `integrate()`, requiring $n + 1$ evaluations of f .

Space Complexity: $O(1)$ auxiliary.

```

template<typename Fn>
double integrate(Fn f, double a, double b, int n = 1000000) {
    double h = (b - a) / n, s = f(a) + f(b);
    for (int i = 1; i < n; i++) {
        s += f(a + h * i) * (i & 1 ? 4 : 2);
    }
}

```

```
    return s * h / 3;
}
```

Example Usage

```
#include <cassert>
#include <cmath>
#include <cstdio>
using namespace std;

double f(double x) {
    return sin(x);
}

int main() {
    const double PI = acos(-1.0);
    assert(fabs(integrate(f, 0.0, PI / 2) - 1) < 1e-10);
    return 0;
}
```

5.4.7 Linear Programming (Simplex)

Solves a linear programming problem using Dantzig's simplex algorithm. The canonical form of a linear programming problem is to maximize (or minimize) the dot product cx , subject to $ax \leq b$ and $x \geq 0$, where x is a vector of unknowns to be solved, c is a vector of coefficients, a is a matrix of linear equation coefficients, and b is a vector of boundary coefficients.

The simplex algorithm walks between vertices of the feasible polytope, at each step pivoting to an adjacent vertex that improves the objective, until no improving move remains (an optimum is reached) or an unbounded improving direction is found.

The implementation uses a two-phase simplex tableau. Phase 1 introduces an artificial variable to find a feasible starting basis, which is essential when some constraint bounds are negative. Phase 2 then optimizes the original objective. The `basis` and `nonbasis` arrays track which variable each tableau row or column currently represents, making solution recovery independent of pivot order.

- `simplex_solve(a, b, c, &x)` solves the linear programming problem for an m by n matrix `a` of real values, a length m vector `b`, and a length n vector `c`, returning 0 if an optimum was found, -1 if there are no feasible solutions, or 1 if the objective is unbounded. If an optimum is found, then the vector pointed to by `x` is populated with a dense solution vector.

Time Complexity: Polynomial (average) on the number of equations and unknowns, but exponential in the worst case.

Space Complexity: $O(m \cdot n)$ auxiliary heap space.

```
#include <algorithm>
#include <cmath>
#include <vector>

int simplex_solve(
    const std::vector<std::vector<double>> &a, const std::vector<double> &b,
    const std::vector<double> &c, std::vector<double> *x, const bool MAXIMIZE = true,
    const double EPS = 1e-10
) {
    int m = static_cast<int>(a.size()), n = static_cast<int>(c.size());
    std::vector<int> basis(m), nonbasis(n + 1);
    std::vector<std::vector<double>> tab(m + 2, std::vector<double>(n + 2));
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
```

```

    tab[i][j] = a[i][j];
}
basis[i] = n + i;
tab[i][n] = -1;
tab[i][n + 1] = b[i];
}
for (int j = 0; j < n; j++) {
    nonbasis[j] = j;
    tab[m][j] = MAXIMIZE ? -c[j] : c[j];
}
nonbasis[n] = -1;
tab[m + 1][n] = 1;
auto pivot = [&](int row, int col) {
    double inv = 1.0 / tab[row][col];
    for (int i = 0; i < m + 2; i++) {
        if (i == row || fabs(tab[i][col]) <= EPS) {
            continue;
        }
        double factor = tab[i][col] * inv;
        for (int j = 0; j < n + 2; j++) {
            if (j != col) {
                tab[i][j] -= factor * tab[row][j];
            }
        }
        tab[i][col] = -factor;
    }
    for (int j = 0; j < n + 2; j++) {
        if (j != col) {
            tab[row][j] *= inv;
        }
    }
    tab[row][col] = inv;
    std::swap(basis[row], nonbasis[col]);
};
auto simplex = [&](int phase) {
    int objective = (phase == 1 ? m + 1 : m);
    while (true) {
        int col = -1;
        for (int j = 0; j <= n; j++) {
            if (phase == 2 && nonbasis[j] == -1) {
                continue;
            }
            if (col == -1 || tab[objective][j] < tab[objective][col] - EPS ||
                (fabs(tab[objective][j] - tab[objective][col]) <= EPS && nonbasis[j] < nonbasis[col])) {
                col = j;
            }
        }
        if (tab[objective][col] >= -EPS) {
            return true;
        }
        int row = -1;
        for (int i = 0; i < m; i++) {
            if (tab[i][col] <= EPS) {
                continue;
            }
            double cur = tab[i][n + 1] / tab[i][col];
            double best = row == -1 ? 0 : tab[row][n + 1] / tab[row][col];
            if (row == -1 || cur < best - EPS || (fabs(cur - best) <= EPS && basis[i] < basis[row])) {
                row = i;
            }
        }
        if (row == -1) {
            return false;
        }
        pivot(row, col);
    }
};
int row = 0;

```

```

for (int i = 1; i < m; i++) {
    if (tab[i][n + 1] < tab[row][n + 1]) {
        row = i;
    }
}
if (m > 0 && tab[row][n + 1] < -EPS) {
    pivot(row, n);
    if (!simplex(1) || fabs(tab[m + 1][n + 1]) > EPS) {
        return -1;
    }
    for (int i = 0; i < m; i++) {
        if (basis[i] != -1) {
            continue;
        }
        int col = -1;
        for (int j = 0; j <= n; j++) {
            if (fabs(tab[i][j]) > EPS && (col == -1 || nonbasis[j] < nonbasis[col])) {
                col = j;
            }
        }
        if (col != -1) {
            pivot(i, col);
        }
    }
}
if (!simplex(2)) {
    return 1;
}
x->assign(n, 0);
for (int i = 0; i < m; i++) {
    if (basis[i] < n) {
        (*x)[basis[i]] = tab[i][n + 1];
    }
}
return 0;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    // Solve [x, y] that maximizes 3x + 4y, subject to x, y >= 0 and:
    // -2x + 1y <= 0
    // 1x + 0.85y <= 9
    // 1x + 2y <= 14
    vector<vector<double>> va{{-2, 1}, {1, 0.85}, {1, 2}};
    vector<double> vb{0, 9, 14};
    vector<double> vc{3, 4};
    vector<double> x;
    assert(simplex_solve(va, vb, vc, &x) == 0);
    double maxval = 0;
    for (int i = 0; i < static_cast<int>(x.size()); i++) {
        maxval += vc[i] * x[i];
    }
    cout << "Solution = " << maxval << " at (" << x[0];
    for (int i = 1; i < static_cast<int>(x.size()); i++) {
        cout << ", " << x[i];
    }
    cout << ")." << endl;

    // Feasible even though the first RHS is negative: x >= 1 and x <= 3.
    vector<vector<double>> negative_rhs{{-1}, {1}};
    vector<double> negative_bounds{-1, 3};
}

```

```
vector<double> one_var{1};
assert(simplex_solve(negative_rhs, negative_bounds, one_var, &x) == 0);
assert(fabs(x[0] - 3) < 1e-9);

// Infeasible: x <= -1 contradicts x >= 0.
vector<vector<double>> infeasible{{1}};
vector<double> bad_bounds{-1};
assert(simplex_solve(infeasible, bad_bounds, one_var, &x) == -1);

// Unbounded: maximize x subject only to x >= 0.
vector<vector<double>> unbounded{{-1}};
vector<double> lower_bound{0};
assert(simplex_solve(unbounded, lower_bound, one_var, &x) == 1);
return 0;
}
```

Example Output

Solution = 33.3043 at (5.30435, 4.34783).

Chapter 6

Mathematics

6.1 Math Utilities

Common mathematic constants and functions, most of which already have standard STL equivalents:

`std::signbit()`, `std::copysign()`, `std::trunc()`, `std::round()`, `std::erf()`, `std::tgamma()`, `std::lgamma()`. The implementations below are for educational purposes only, and may not be as heavily optimized as their standard library counterparts.

Time Complexity:

- $O(1)$ for most operations.
- $O(1)$ per call to `mulmod()` with `__uint128_t`, or $O(\log n)$ with the portable fallback, where n is the second argument.
- $O(\log n)$ calls to `mulmod()` per call to `powmod(x, n, m)`.
- $O(d \cdot e)$ for `convert_base(d, a, b)`, where d is the number of input digits and e is the number of output digits.
- $O(\log_b(x + 1) + 1)$ for `to_base(x, b)`.
- $O(x/1000 + 1)$ for `to_roman(x)`, due to the repeated `M` prefix.

Space Complexity:

- $O(1)$ auxiliary for most operations.
- $O(d + e)$ auxiliary heap space for `convert_base(d, a, b)`, where d is the number of input digits and e is the number of output digits.
- $O(e)$ auxiliary heap space for `to_base(x, b)`, where e is the number of output digits.
- $O(x/1000 + 1)$ auxiliary heap space for `to_roman(x)`.

```
#include <algorithm>
#include <cfloat>
#include <climits>
#include <cmath>
#include <cstdint>
#include <limits>
#include <random>
#include <string>
#include <type_traits>
#include <vector>

#ifndef M_PI
const double M_PI = acos(-1.0); // or std::numbers::pi and std::numbers::pi_v<> in C++20 and later
#endif
#ifndef M_E
const double M_E = exp(1.0); // or std::numbers::e and std::numbers::e_v<> in C++20 and later
#endif
const double M_PHI = (1.0 + sqrt(5.0)) / 2.0;
```

```
const double M_INF = std::numeric_limits<double>::infinity();
const double M_NAN = std::numeric_limits<double>::quiet_NaN();
```

Epsilon Comparisons:

- `EQ()`, `NE()`, `LT()`, `GT()`, `LE()`, and `GE()` relationally compare two values x and y . Arguments may be of different types; the common type governs behavior. If the common type is integral, exact comparison is used (`==`, `<`, etc.). Otherwise, absolute-error epsilon comparison is used: values within `EPS` of each other are considered equal, and `LT`/`GT`/`LE`/`GE` shift the boundary by `EPS` accordingly.
- `rEQ(ref, val)` returns whether `val` equals reference `ref` within relative error `EPS`. The tolerance scales with `|ref|`, so `rEQ(ref, val)` is NOT the same as `rEQ(val, ref)`. Use this when one argument is a known exact value and the other is a computed approximation. Degenerates to exact comparison when `ref` = 0, since the tolerance collapses to 0; use `EQ` near zero.
- `rEQ_sym(x, y)` is the symmetric (commutative) variant: tolerance scales with `max(|x|, |y|)`, so the result is the same regardless of argument order. Still degenerates near zero when both arguments are close to 0.

```
const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}
template<typename T, typename U> bool NE(T a, U b) { return !EQ(a, b); }
template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}
template<typename T, typename U> bool GT(T a, U b) { return LT(b, a); }
template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }
template<typename T, typename U> bool GE(T a, U b) { return !LT(a, b); }

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool rEQ(T ref, U val) {
    return std::fabs(C(ref) - C(val)) <= EPS * std::fabs(C(ref));
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool rEQ_sym(T x, U y) {
    return std::fabs(C(x) - C(y)) <= EPS * std::max(std::fabs(C(x)), std::fabs(C(y)));
}
```

Sign Functions:

- `sgn(x)` returns -1 (if $x < 0$), 0 (if $x = 0$), or 1 (if $x > 0$). Unlike `std::signbit()` or `std::copysign()`, this does not handle the sign of `NaN`.
- `signbit_x(x)` is analogous to `std::signbit()`, returning whether the sign bit of the floating point number is set to true. If so, then `x` is considered “negative.” Note that this works as expected on `+0.0`, `-0.0`, `Inf`, `-Inf`, `NaN`, as well as `-NaN`. Warning: This assumes that the sign bit is the leading (most significant) bit in the internal representation of the IEEE floating point value.
- `copysign_x(x, y)` is analogous to `std::copysign()`, returning a number with the magnitude of `x` but the sign of `y`.

```
template<typename T>
int sgn(const T &x) {
    return (T(0) < x) - (x < T(0));
}

template<typename Double>
bool signbit_(Double x) {
    return (((unsigned char *)&x)[sizeof(x) - 1] >> (CHAR_BIT - 1)) & 1;
}
```

```
template<typename Double>
Double copysign_(Double x, Double y) {
    return signbit_(y) ? -fabs(x) : fabs(x);
}
```

Rounding Functions:

- `floor0(x)` returns `x` rounded down, symmetrically towards zero. This function is analogous to `std::trunc()`.
- `ceil0(x)` returns `x` rounded up, symmetrically away from zero. This function is analogous to `std::round()`.
- `round_half_up(x)` returns `x` rounded half up, towards positive infinity.
- `round_half_down(x)` returns `x` rounded half down, towards negative infinity.
- `round_half_to0(x)` returns `x` rounded half down, symmetrically towards zero.
- `round_half_from0(x)` returns `x` rounded half up, symmetrically away from zero.
- `round_half_even(x)` returns `x` rounded half to even, using banker's rounding.
- `round_half_alterate(x)` returns `x` rounded, where ties are broken by alternating rounds towards positive and negative infinity.
- `round_half_alterate0(x)` returns `x` rounded, where ties are broken by alternating symmetric rounds towards and away from zero.
- `round_half_random(x)` returns `x` rounded, where ties are broken randomly.
- `round_n_places(x, n, f)` returns `x` rounded to `n` digits after the decimal, using the specified rounding function `f(x)`.

```
template<typename Double>
Double floor0(const Double &x) {
    Double res = floor(fabs(x));
    return (x < 0.0) ? -res : res;
}
```

```
template<typename Double>
Double ceil0(const Double &x) {
    Double res = ceil(fabs(x));
    return (x < 0.0) ? -res : res;
}
```

```
template<typename Double>
Double round_half_up(const Double &x) {
    return floor(x + 0.5);
}
```

```
template<typename Double>
Double round_half_down(const Double &x) {
    return ceil(x - 0.5);
}
```

```
template<typename Double>
Double round_half_to0(const Double &x) {
    Double res = round_half_down(fabs(x));
    return (x < 0.0) ? -res : res;
}
```

```
template<typename Double>
Double round_half_from0(const Double &x) {
    Double res = round_half_up(fabs(x));
    return (x < 0.0) ? -res : res;
}
```

```
template<typename Double>
Double round_half_even(const Double &x, const Double &EPS = 1e-9) {
    if (x < 0.0) {
        return -round_half_even(-x, EPS);
    }
    Double ipart;
```

```

    modf(x, &ipart);
    if (fabs(x - (ipart + 0.5)) < EPS) { // exactly halfway: break the tie towards even
        return (fmod(ipart, 2.0) < EPS) ? ipart : ceil0(ipart + 0.5);
    }
    return round_half_from0(x);
}

template<typename Double>
Double round_half_alterate(const Double &x) {
    static bool up = true;
    return (up = !up) ? round_half_up(x) : round_half_down(x);
}

template<typename Double>
Double round_half_alterate0(const Double &x) {
    static bool up = true;
    return (up = !up) ? round_half_from0(x) : round_half_to0(x);
}

template<typename Double>
Double round_half_random(const Double &x) {
    static std::mt19937 rng(std::random_device{}());
    return (rng() % 2 == 0) ? round_half_from0(x) : round_half_to0(x);
}

template<typename Double, typename RoundingFunction>
Double round_n_places(const Double &x, unsigned int n, RoundingFunction f) {
    return f(x * pow(10, n)) / pow(10, n);
}

```

Error Function:

- `erf_(x)` returns the error encountered in integrating the normal distribution. Its value is $2/\sqrt{\pi} \int_0^x e^{-t^2} dt$. This function is analogous to `std::erf(x)`.
- `erfc_(x)` returns the error function complement, that is, $1 - \text{erf}_-(x)$. This function is analogous to `std::erfc(x)`.

```

#define ERF_EPS 1e-14

double erfc_(double x);

double erf_(double x) {
    if (signbit_(x)) {
        return -erf_(-x);
    }
    if (fabs(x) > 2.2) {
        return 1.0 - erfc_(x);
    }
    double sum = x, term = x, xx = x * x;
    int j = 1;
    do {
        term *= xx / j;
        sum -= term / (2 * (j++) + 1);
        term *= xx / j;
        sum += term / (2 * (j++) + 1);
    } while (fabs(term) > sum * ERF_EPS);
    return 2 / sqrt(M_PI) * sum;
}

double erfc_(double x) {
    if (fabs(x) < 2.2) {
        return 1.0 - erf_(x);
    }
    if (signbit_(x)) {
        return 2.0 - erfc_(-x);
    }
}

```

```
double a = 1, b = x, c = x, d = x * x + 0.5, q1, q2 = 0, n = 1.0, t;
do {
    t = a * n + b * x;
    a = b;
    b = t;
    t = c * n + d * x;
    c = d;
    d = t;
    n += 0.5;
    q1 = q2;
    q2 = b / d;
} while (fabs(q1 - q2) > q2 * ERF_EPS);
return 1 / sqrt(M_PI) * exp(-x * x) * q2;
}

#undef ERF_EPS
```

Gamma Functions:

- `tgamma(x)` returns the gamma function of `x`. Unlike `std::tgamma()`, this version only supports positive `x`, returning `NaN` if `x` ≤ 0 .
- `lgamma(x)` returns the natural logarithm of the absolute value of the gamma function of `x`. Unlike `std::lgamma()`, this version only supports positive `x`, returning `NaN` if `x` ≤ 0 .

```
double lgamma(double x);

double tgamma(double x) {
    if (x <= 0) {
        return M_NAN;
    }
    if (x < 1e-3) {
        return 1.0 / (x * (1.0 + 0.57721566490153286060651209 * x));
    }
    if (x < 12) {
        double y = x;
        int n = 0;
        bool arg_was_less_than_one = (y < 1);
        if (arg_was_less_than_one) {
            y += 1;
        } else {
            n = static_cast<int>(floor(y)) - 1;
            y -= n;
        }
        static const double p[] = {-1.71618513886549492533811e+0, 2.47656508055759199108314e+1,
                                     -3.79804256470945635097577e+2, 6.29331155312818442661052e+2,
                                     8.66966202790413211295064e+2, -3.14512729688483675254357e+4,
                                     -3.61444134186911729807069e+4, 6.64561438202405440627855e+4};
        static const double q[] = {-3.08402300119738975254353e+1, 3.15350626979604161529144e+2,
                                     -1.01515636749021914166146e+3, -3.10777167157231109440444e+3,
                                     2.25381184209801510330112e+4, 4.75584627752788110767815e+3,
                                     -1.34659959864969306392456e+5, -1.15132259675553483497211e+5};
        double num = 0, den = 1, z = y - 1;
        for (int i = 0; i < 8; i++) {
            num = (num + p[i]) * z;
            den = den * z + q[i];
        }
        double result = num / den + 1;
        if (arg_was_less_than_one) {
            result /= (y - 1);
        } else {
            for (int i = 0; i < n; i++) {
                result *= y++;
            }
        }
        return result;
    }
}
```

```

    return (x > 171.624) ? 2 * DBL_MAX : exp(lgamma(x));
}

double lgamma_(double x) {
    if (x <= 0) {
        return M_NAN;
    }
    if (x < 12) {
        return log(fabs(tgamma_(x)));
    }
    static const double c[] = {1.0 / 12,    -1.0 / 360,    1.0 / 1260, -1.0 / 1680,
                               1.0 / 1188, -691.0 / 360360, 1.0 / 156,  -3617.0 / 122400};
    double z = 1.0 / (x * x), sum = c[7];
    for (int i = 6; i >= 0; i--) {
        sum = sum * z + c[i];
    }
    return (x - 0.5) * log(x) - x + 0.91893853320467274178032973640562 + sum / x;
}

```

Modular Arithmetic:

- `addmod(a, b, m)` returns $(a + b) \bmod m$ and `submod(a, b, m)` returns $(a - b) \bmod m$, each in $[0, m)$. Both operands may be negative or unreduced; the result is normalized into range. These use signed arithmetic, so `m` (and the reduced operands) must be small enough that `a + b` does not overflow `int64_t`, i.e. `m` up to roughly 2^{62} , which covers all common contest moduli.
- `mulmod(x, n, m)` returns `x` multiplied by `n`, modulo `m`. This is done in a way to avoid overflow: on compilers with `__uint128_t` it uses one wide product, while the portable fallback uses double-and-add multiplication. The fallback is slower by a logarithmic factor, but avoids relying on nonstandard 128-bit integers. Unlike `addmod`/`submod`, this takes unsigned operands and supports a full 64-bit modulus.
- `powmod(x, n, m)` returns `x` raised to the power `n`, modulo `m`.

These are lightweight standalone helpers for the occasional modular computation. For a full value type that wraps a fixed modulus with overloaded operators, modular inverses, and combinatorics, see the `Modular` (“Mint”) class in section 6.3.2.

```

int64_t addmod(int64_t a, int64_t b, int64_t m) {
    return ((a + b) % m + m) % m;
}

int64_t submod(int64_t a, int64_t b, int64_t m) {
    return ((a - b) % m + m) % m;
}

uint64_t mulmod(uint64_t x, uint64_t n, uint64_t m) {
    #if defined(__SIZEOF_INT128__)
        return static_cast<uint64_t>((static_cast<__uint128_t>(x) * n % m));
    #else
        uint64_t a = 0, b = x % m;
        for (; n > 0; n >>= 1) {
            if (n & 1) {
                a = a >= m - b ? a - (m - b) : a + b;
            }
            b = b >= m - b ? b - (m - b) : b + b;
        }
        return a;
    #endif
}

uint64_t powmod(uint64_t x, uint64_t n, uint64_t m) {
    uint64_t a = 1, b = x;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = mulmod(a, b, m);
        }
    }
}

```

```

    b = mulmod(b, b, m);
}
return a % m;
}

```

Base Conversion:

- `to_base(x, b)` returns the digits of the unsigned integer `x` in base `b`, where index 0 of the result stores the least significant digit.
- `to_roman(x)` returns the Roman numeral representation of the unsigned integer `x` as a string.
- `convert_base(d, a, b)` converts an integer in base `a` as a vector `d` of digits (where `d[0]` is the least significant digit) to base `b` as a vector of digits (again with index 0 holding the least significant digit). This uses repeated long division, so the value itself does not need to fit in a machine integer. Each intermediate `rem*a + digit` must fit in `uint64_t`.

```

std::vector<int> to_base(unsigned int x, int b = 10) {
    std::vector<int> res;
    do { // do-while so that a value of 0 yields the single digit {0}
        res.push_back(x % b);
        x /= b;
    } while (x != 0);
    return res;
}

std::string to_roman(unsigned int x) {
    static const std::string h[] = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
    static const std::string t[] = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
    static const std::string o[] = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};
    std::string prefix(x / 1000, 'M');
    x %= 1000;
    return prefix + h[x / 100] + t[x / 10 % 10] + o[x % 10];
}

std::vector<int> convert_base(const std::vector<int> &d, int a, int b) {
    std::vector<int> cur = d, res;
    auto trim = [](std::vector<int> &v) {
        while (v.size() > 1 && v.back() == 0) {
            v.pop_back();
        }
    };
    trim(cur);
    if (cur.empty() || (cur.size() == 1 && cur[0] == 0)) {
        return {0};
    }
    while (!(cur.size() == 1 && cur[0] == 0)) {
        std::vector<int> q(cur.size());
        uint64_t rem = 0;
        for (int i = static_cast<int>(cur.size()) - 1; i >= 0; i--) {
            uint64_t x = rem * static_cast<uint64_t>(a) + static_cast<uint64_t>(cur[i]);
            q[i] = static_cast<int>(x / static_cast<uint64_t>(b));
            rem = x % static_cast<uint64_t>(b);
        }
        res.push_back(static_cast<int>(rem));
        trim(q);
        cur = std::move(q);
    }
    return res;
}

```

Example Usage

```

#include <cassert>
#include <iostream>

```

```

using namespace std;

int main() {
    assert(EQ(M_PI, 3.14159265359));
    assert(EQ(M_E, 2.718281828459));
    assert(EQ(M_PHI, 1.61803398875));

    double x = -12345.6789;
    assert((-M_INF < x) && (x < M_INF));
    assert((M_INF + x == M_INF) && (M_INF - x == M_INF));
    assert((M_INF + M_INF == M_INF) && (-M_INF - M_INF == -M_INF));
    assert((M_NAN != x) && (M_NAN != M_INF) && (M_NAN != M_NAN));
    assert(!(M_NAN < x) && !(M_NAN > x) && !(M_NAN <= x) && !(M_NAN >= x));
    assert(isnan(0.0 * M_INF) && isnan(0.0 * -M_INF) && isnan(M_INF / -M_INF));
    assert(isnan(M_NAN) && isnan(-M_NAN) && isnan(M_INF - M_INF));

    assert(sgn(x) == -1 && sgn(0.0) == 0 && sgn(5678) == 1);
    assert(signbit(x) && !signbit(0.0) && signbit(-0.0));
    assert(!signbit(M_INF) && signbit(-M_INF));
    assert(!signbit(M_NAN) && signbit(-M_NAN));
    assert(copysign(1.0, +2.0) == +1.0 && copysign(M_INF, -2.0) == -M_INF);
    assert(copysign(1.0, -2.0) == -1.0 && std::signbit(copysign(M_NAN, -2.0)));

    assert(EQ(floor0(1.5), 1.0) && EQ(ceil0(1.5), 2.0));
    assert(EQ(floor0(-1.5), -1.0) && EQ(ceil0(-1.5), -2.0));
    assert(EQ(round_half_up(+1.5), +2) && EQ(round_half_down(+1.5), +1));
    assert(EQ(round_half_up(-1.5), -1) && EQ(round_half_down(-1.5), -2));
    assert(EQ(round_half_to0(+1.5), +1) && EQ(round_half_from0(+1.5), +2));
    assert(EQ(round_half_to0(-1.5), -1) && EQ(round_half_from0(-1.5), -2));
    assert(EQ(round_half_even(+1.5), +2) && EQ(round_half_even(-1.5), -2));
    assert(EQ(round_half_even(3.1), 3) && EQ(round_half_even(3.4), 3)); // non-ties round normally
    assert(NE(round_half_alterate(+1.5), round_half_alterate(+1.5)));
    assert(NE(round_half_alterate0(-1.5), round_half_alterate0(-1.5)));
    assert(EQ(round_n_places(-1.23456, 3, round_half_to0<double>), -1.235));

    assert(EQ(erf_(1.0), 0.8427007929) && EQ(erf_(-1.0), -0.8427007929));
    assert(EQ(tgamma_(0.5), 1.7724538509) && EQ(tgamma_(1.0), 1.0));
    assert(EQ(lgamma_(0.5), 0.5723649429) && EQ(lgamma_(1.0), 0.0));

    assert(addmod(7, 8, 10) == 5 && submod(2, 5, 10) == 7);
    // Negative and unreduced operands are normalized into [0, m).
    assert(addmod(-3, -4, 10) == 3 && submod(-3, 4, 10) == 3);
    assert(addmod(25, -7, 10) == 8);

    assert(powmod(2, 10, 1000000007) == 1024);
    assert(powmod(2, 62, 1000000) == 387904);
    assert(powmod(10001, 10001, 100000) == 10001);

    assert(to_roman(1234) == "MCCXXXIV");
    assert(to_roman(5678) == "MMMMDCLXXVIII");
    std::vector<int> digits{6, 5, 4, 3, 2, 1};
    std::vector<int> base20 = to_base(123456, 20);
    assert(convert_base(base20, 20, 10) == digits);
    assert(convert_base(std::vector<int>{0, 0, 0}, 10, 2) == std::vector<int>{0});

    std::vector<int> big_decimal(30, 9); // 10^30 - 1, larger than uint64_t.
    std::vector<int> big_binary = convert_base(big_decimal, 10, 2);
    assert(convert_base(big_binary, 2, 10) == big_decimal);
    return 0;
}

```


6.2 Combinatorics

6.2.1 Combinatorial Calculations

The following functions implement common operations in combinatorics. All inputs must be nonnegative. All return values and table entries are computed as 64-bit integers modulo an input argument m or p . Modular products use ordinary `int64_t` multiplication, so the square of the chosen modulus must fit in `int64_t`.

- `factorial(n, m)` returns $n! \bmod m$.
- `factorialp(n, p)` returns $n! \bmod p$, where p is prime.
- `binomial_table(n, m)` returns rows 0 to n of Pascal's triangle as a two-dimensional vector t such that $t[i][j] = \binom{i}{j} \bmod m$.
- `permute(n, k, m)` returns $(n \text{ permute } k) \bmod m$.
- `choose(n, k, p)` returns $\binom{n}{k} \bmod p$, where p is prime.
- `multichoose(n, k, p)` returns $(n \text{ multichoose } k) \bmod p$, where p is prime.
- `catalan(n, p)` returns the n -th Catalan number mod p , where p is prime.
- `partitions(n, m)` returns the number of partitions of n , mod m .
- `partitions(n, k, m)` returns the number of partitions of n into k parts, mod m .
- `stirling1(n, k, m)` returns the (n, k) unsigned Stirling number of the 1st kind mod m .
- `stirling2(n, k, m)` returns the (n, k) Stirling number of the 2nd kind mod m .
- `eulerian1(n, k, m)` returns the (n, k) Eulerian number of the 1st kind mod m , where $n > k$.
- `eulerian2(n, k, m)` returns the (n, k) Eulerian number of the 2nd kind mod m , where $n > k$.

Time Complexity:

- $O(n)$ for `factorial(n, m)`.
- $O(p \log n)$ for `factorialp(n, p)`.
- $O(n^2)$ for `binomial_table(n, m)`.
- $O(k)$ for `permute(n, k, m)`.
- $O(\min(k, n - k))$ for `choose(n, k, p)`.
- $O(k)$ for `multichoose(n, k, p)`.
- $O(n)$ for `catalan(n, p)`.
- $O(n^2)$ for `partitions(n, m)`.
- $O(n \cdot k)$ for `partitions(n, k, m)`, `stirling1(n, k, m)`, `stirling2(n, k, m)`, `eulerian1(n, k, m)`, and `eulerian2(n, k, m)`.

Space Complexity:

- $O(n^2)$ auxiliary heap space for `binomial_table(n, m)`.
- $O(n \cdot k)$ auxiliary heap space for `partitions(n, k, m)`, `stirling1(n, k, m)`, `stirling2(n, k, m)`, `eulerian1(n, k, m)`, and `eulerian2(n, k, m)`.
- $O(1)$ auxiliary for all other operations.

```
#include <cstdint>
#include <vector>

int64_t factorial(int n, int m = 1000000007) {
    int64_t res = 1;
    for (int i = 2; i <= n; i++) {
        res = (res * i) % m;
    }
    return res % m;
}

int64_t factorialp(int64_t n, int64_t p = 1000000007) {
    int64_t res = 1;
    while (n > 1) {
        if (n / p % 2 == 1) {
            res = res * (p - 1) % p;
        }
        n /= p;
    }
    return res;
}
```

```

    }
    int max = n % p;
    for (int i = 2; i <= max; i++) {
        res = (res * i) % p;
    }
    n /= p;
}
return res % p;
}

std::vector<std::vector<int64_t>> binomial_table(int n, int64_t m = 1000000007) {
    std::vector<std::vector<int64_t>> t(n + 1);
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= i; j++) {
            if (i < 2 || j == 0 || i == j) {
                t[i].push_back(1);
            } else {
                t[i].push_back((t[i - 1][j - 1] + t[i - 1][j]) % m);
            }
        }
    }
    return t;
}

int64_t permute(int n, int k, int64_t m = 1000000007) {
    if (n < k) {
        return 0;
    }
    int64_t res = 1;
    for (int i = 0; i < k; i++) {
        res = res * (n - i) % m;
    }
    return res % m;
}

int64_t powmod(int64_t x, int64_t n, int64_t m) {
    int64_t a = 1, b = x % m;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = a * b % m;
        }
        b = b * b % m;
    }
    return a % m;
}

int64_t choose(int n, int k, int64_t p = 1000000007) {
    if (n < k) {
        return 0;
    }
    if (k > n - k) {
        k = n - k;
    }
    int64_t num = 1, den = 1;
    for (int i = 0; i < k; i++) {
        num = num * (n - i) % p;
    }
    for (int i = 1; i <= k; i++) {
        den = den * i % p;
    }
    return num * powmod(den, p - 2, p) % p;
}

int64_t multichoose(int n, int k, int64_t p = 1000000007) {
    return choose(n + k - 1, k, p);
}

int64_t catalan(int n, int64_t p = 1000000007) {

```

```

    return choose(2 * n, n, p) * powmod(n + 1, p - 2, p) % p;
}

int64_t partitions(int n, int64_t m = 1000000007) {
    std::vector<int64_t> t(n + 1, 0);
    t[0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = i; j <= n; j++) {
            t[j] = (t[j] + t[j - i]) % m;
        }
    }
    return t[n] % m;
}

int64_t partitions(int n, int k, int64_t m = 1000000007) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 0));
    t[0][1] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1, h = k < i ? k : i; j <= h; j++) {
            t[i][j] = (t[i - 1][j - 1] + t[i - j][j]) % m;
        }
    }
    return t[n][k] % m;
}

int64_t stirling1(int n, int k, int64_t m = 1000000007) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 0));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = (i - 1) * t[i - 1][j] % m;
            t[i][j] = (t[i][j] + t[i - 1][j - 1]) % m;
        }
    }
    return t[n][k] % m;
}

int64_t stirling2(int n, int k, int64_t m = 1000000007) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 0));
    t[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = j * t[i - 1][j] % m;
            t[i][j] = (t[i][j] + t[i - 1][j - 1]) % m;
        }
    }
    return t[n][k] % m;
}

int64_t eulerian1(int n, int k, int64_t m = 1000000007) {
    if (k > n - 1 - k) {
        k = n - 1 - k;
    }
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 1));
    for (int j = 1; j <= k; j++) {
        t[0][j] = 0;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            t[i][j] = (i - j) * t[i - 1][j - 1] % m;
            t[i][j] = (t[i][j] + ((j + 1) * t[i - 1][j] % m)) % m;
        }
    }
    return t[n][k] % m;
}

int64_t eulerian2(int n, int k, int64_t m = 1000000007) {
    std::vector<std::vector<int64_t>> t(n + 1, std::vector<int64_t>(k + 1, 1));

```

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= k; j++) {
        if (i == j) {
            t[i][j] = 0;
        } else {
            t[i][j] = (j + 1) * t[i - 1][j] % m;
            t[i][j] = ((2 * i - 1 - j) * t[i - 1][j - 1] % m + t[i][j]) % m;
        }
    }
}
return t[n][k] % m;
}

```

Example Usage

```

#include <cassert>

int main() {
    auto t = binomial_table(10);
    for (int i = 0; i < static_cast<int>(t.size()); i++) {
        for (int j = 0; j < static_cast<int>(t[i].size()); j++) {
            assert(t[i][j] == choose(i, j));
        }
    }
    assert(factorial(10) == 3628800);
    assert(factorialp(123456) == 639390503);
    assert(permute(10, 4) == 5040);
    assert(choose(20, 7) == 77520);
    assert(multichoose(20, 7) == 657800);
    assert(catalan(10) == 16796);
    assert(partitions(4) == 5);
    assert(partitions(100, 5) == 38225);
    assert(stirling1(4, 2) == 11);
    assert(stirling2(4, 3) == 6);
    assert(eulerian1(9, 5) == 88234);
    assert(eulerian2(8, 3) == 195800);
    return 0;
}

```

6.2.2 Enumerating Arrangements

For the purposes of this section, we define a “size k arrangement of n ” to be a permutation of a size k subset of the integers from 0 to $n - 1$, for $0 \leq k \leq n$. There are n permute k possible arrangements, but n^k possible arrangements if repeated values are allowed.

- `next_arrangement(n, a)` tries to rearrange `a` to the next lexicographically greater arrangement, returning true if such an arrangement exists or false if the array is already in descending order (in which case `a` is unchanged). The input `a` must consist of distinct integers in the range $[0, n)$.
- `arrangement_by_rank(n, k, r)` returns the size k arrangement of n which is lexicographically ranked r out of all size k arrangements of n , where r is a 0-based rank in the range $[0, n \text{ permute } k)$.
- `rank_by_arrangement(n, a)` returns an integer representing the 0-based rank of arrangement `a`, which must consist of distinct integers in the range $[0, n)$.
- `next_arrangement_with_repeats(n, a)` tries to rearrange `a` to the next lexicographically greater arrangement with repeats, returning true if such an arrangement exists or false if the array is already in descending order (in which case `a` is unchanged). The input `a` must consist of integers in the range $[0, n)$. If `a` were interpreted as a k digit integer in base n , this function could be thought of as incrementing the integer.

Time Complexity:

- $O(n \cdot k)$ for `next_arrangement(n, a)`, `arrangement_by_rank(n, k, r)`, and `rank_by_arrangement(n, a)`, where k is the size of `a`.
- $O(k)$ for `next_arrangement_with_repeats(n, a)`.

Space Complexity:

- $O(n)$ auxiliary heap space for `next_arrangement()`, `arrangement_by_rank()`, and `rank_by_arrangement()`.
- $O(1)$ auxiliary for `next_arrangement_with_repeats()`.

```
#include <algorithm>
#include <cstdint>
#include <numeric>
#include <vector>

bool next_arrangement(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    std::vector<bool> used(n);
    for (int i = 0; i < k; i++) {
        used[a[i]] = true;
    }
    for (int i = k - 1; i >= 0; i--) {
        used[a[i]] = false;
        for (int j = a[i] + 1; j < n; j++) {
            if (!used[j]) {
                a[i++] = j;
                used[j] = true;
                for (int x = 0; x < k; x++) {
                    if (!used[x]) {
                        a[i++] = x;
                    }
                }
                return true;
            }
        }
    }
    return false;
}

int64_t n_permute_k(int n, int k) {
    int64_t res = 1;
    for (int i = 0; i < k; i++) {
        res *= n - i;
    }
    return res;
}

std::vector<int> arrangement_by_rank(int n, int k, int64_t r) {
    std::vector<int> values(n), res(k);
    std::iota(values.begin(), values.end(), 0);
    for (int i = 0; i < k; i++) {
        int64_t count = n_permute_k(n - 1 - i, k - 1 - i);
        int pos = r / count;
        res[i] = values[pos];
        std::copy(values.begin() + pos + 1, values.end(), values.begin() + pos);
        r %= count;
    }
    return res;
}

int64_t rank_by_arrangement(int n, const std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    int64_t res = 0;
    std::vector<bool> used(n);
    for (int i = 0; i < k; i++) {
        int count = 0;
        for (int j = 0; j < a[i]; j++) {
            if (!used[j]) {

```

```

        count++;
    }
}
res += count * n_permute_k(n - i - 1, k - i - 1);
used[a[i]] = true;
}
return res;
}

bool next_arrangement_with_repeats(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - 1) {
            a[i]++;
            std::fill(a.begin() + i + 1, a.end(), 0);
            return true;
        }
    }
    return false;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : "," );
    }
    cout << "} ";
}

int main() {
    {
        int n = 4, k = 3;
        vector<int> a{0, 1, 2};
        cout << n << " permute " << k << " arrangements:" << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            auto b = arrangement_by_rank(n, k, count);
            assert(a == b);
            assert(rank_by_arrangement(n, a) == count);
            count++;
            if (count == 10 || count == 20) {
                cout << endl;
            }
        } while (next_arrangement(n, a));
        cout << endl;
    }
    {
        int n = 4, k = 2;
        vector<int> a{0, 0};
        cout << endl << n << "~" << k << " arrangements with repeats:" << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            count++;
            if (count == 13) {
                cout << endl;
            }
        } while (next_arrangement_with_repeats(n, a));
    }
}

```

```

    cout << endl;
}
return 0;
}

```

Example Output

```

4 permute 3 arrangements:
{0,1,2} {0,1,3} {0,2,1} {0,2,3} {0,3,1} {0,3,2} {1,0,2} {1,0,3} {1,2,0} {1,2,3}
{1,3,0} {1,3,2} {2,0,1} {2,0,3} {2,1,0} {2,1,3} {2,3,0} {2,3,1} {3,0,1} {3,0,2}
{3,1,0} {3,1,2} {3,2,0} {3,2,1}

4^2 arrangements with repeats:
{0,0} {0,1} {0,2} {0,3} {1,0} {1,1} {1,2} {1,3} {2,0} {2,1} {2,2} {2,3} {3,0}
{3,1} {3,2} {3,3}

```

6.2.3 Enumerating Permutations

A permutation is an ordered list consisting of n (not necessarily distinct) elements.

- `next_permutation(lo, hi)` is analogous to `std::next_permutation(lo, hi)`, taking two BidirectionalIterators as a range `[lo, hi)` for which the function tries to rearrange to the next lexicographically greater permutation. The function returns true if such a permutation exists, or false if the range is already in descending order (in which case the values are unchanged). This implementation requires an ordering on the set of possible elements defined by the `<` operator on the iterator's value type.
- `next_permutation(a)` is analogous to `next_permutation()`, except that it takes a vector instead of a range.
- `next_permutation(x)` returns the next lexicographically greater permutation of the binary digits of the integer `x`, that is, the lowest integer greater than `x` with the same number of 1-bits. This can be used to generate combinations of a set of n items by treating each 1 bit as whether to “take” the item at the corresponding position.
- `permutation_by_rank(n, r)` returns the permutation of the integers in the range $[0, n)$ which is lexicographically ranked r , where r is a 0-based rank in the range $[0, n!)$.
- `rank_by_permutation(a)` returns an integer representing the 0-based rank of permutation `a`, which must be a permutation of the integers $[0, n)$.
- `permutation_cycles(a)` returns the decomposition of the permutation `a` into cycles. A permutation cycle is a subset of a permutation whose elements are consecutively swapped, relative to a sorted set. For example, `{3, 1, 0, 2}` decomposes to `{0, 3, 2}` and `{1}`, meaning that starting from the sorted order `{0, 1, 2, 3}`, the 0-th value is replaced by the 3rd, the 3rd by the 2nd, and the 2nd by the 0-th ($0 \rightarrow 3 \rightarrow 2 \rightarrow 0$).

Time Complexity:

- $O(n^2)$ per call to `next_permutation(lo, hi)`, where n is the distance between `lo` and `hi`.
- $O(n^2)$ per call to `next_permutation(a)`, `permutation_by_rank(n, r)`, and `rank_by_permutation(a)`.
- $O(1)$ per call to `next_permutation(x)`.
- $O(n)$ per call to `permutation_cycles()`.

Space Complexity:

- $O(1)$ auxiliary for `next_permutation_()` and `next_permutation()`.
- $O(n)$ auxiliary heap space for `permutation_by_rank()`, `rank_by_permutation()`, and `permutation_cycles()`.

```

#include <algorithm>
#include <cstdint>
#include <numeric>
#include <vector>

template<typename It>
bool next_permutation_(It lo, It hi) {
    if (lo == hi) {
        return false;
    }
}

```

```

}
It i = lo;
if (++i == hi) {
    return false;
}
i = hi;
--i;
while (true) {
    It j = i;
    if (*--i < *j) {
        It k = hi;
        while (*(i < *--k)) {
        }
        std::iter_swap(i, k);
        std::reverse(j, hi);
        return true;
    }
    if (i == lo) {
        std::reverse(lo, hi);
        return false;
    }
}
}

template<typename T>
bool next_permutation(std::vector<T> &a) {
    int n = static_cast<int>(a.size());
    for (int i = n - 2; i >= 0; i--) {
        if (a[i] < a[i + 1]) {
            for (int j = n - 1; j-- > i) {
                if (a[i] < a[j]) {
                    std::swap(a[i++], a[j]);
                    for (j = n - 1; i < j; i++, j--) {
                        std::swap(a[i], a[j]);
                    }
                    return true;
                }
            }
        }
    }
    return false;
}

int64_t next_permutation(int64_t x) {
    int64_t s = x & -x, r = x + s;
    return r | ((x ^ r) >> 2) / s;
}

std::vector<int> permutation_by_rank(int n, int64_t x) {
    std::vector<int64_t> factorial(n);
    std::vector<int> values(n), res(n);
    factorial[0] = 1;
    for (int i = 1; i < n; i++) {
        factorial[i] = i * factorial[i - 1];
    }
    std::iota(values.begin(), values.end(), 0);
    for (int i = 0; i < n; i++) {
        int pos = x / factorial[n - 1 - i];
        res[i] = values[pos];
        std::copy(values.begin() + pos + 1, values.end(), values.begin() + pos);
        x %= factorial[n - 1 - i];
    }
    return res;
}

int64_t rank_by_permutation(const std::vector<int> &a) {
    int n = static_cast<int>(a.size());
    std::vector<int64_t> factorial(n);

```



```

factorial[0] = 1;
for (int i = 1; i < n; i++) {
    factorial[i] = i * factorial[i - 1];
}
int64_t res = 0;
for (int i = 0; i < n; i++) {
    int v = a[i];
    for (int j = 0; j < i; j++) {
        if (a[j] < a[i]) {
            v--;
        }
    }
    res += v * factorial[n - 1 - i];
}
return res;
}

using cycles = std::vector<std::vector<int>>;

cycles permutation_cycles(const std::vector<int> &a) {
    int n = static_cast<int>(a.size());
    std::vector<bool> visit(n);
    cycles res;
    for (int i = 0; i < n; i++) {
        if (!visit[i]) {
            int j = i;
            std::vector<int> curr;
            do {
                curr.push_back(j);
                visit[j] = true;
                j = a[j];
            } while (j != i);
            res.push_back(std::move(curr));
        }
    }
    return res;
}

```

Example Usage

```

#include <bitset>
#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? " " : ",");
    }
    cout << "} ";
}

int main() {
    {
        const int n = 4;
        vector<int> a{0, 1, 2, 3}, b = a, c = a;
        cout << "Permutations of [0, " << n << "]: " << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            assert(b == a);
            assert(c == a);
            vector<int> d = permutation_by_rank(n, count);
            assert(d == a);

```

```

    assert(rank_by_permutation(a) == count);
    count++;
    if (count == 8 || count == 16) {
        cout << endl;
    }
    std::next_permutation(b.begin(), b.end());
    next_permutation(c);
} while (next_permutation(a));
cout << endl;
}
{ // Permutations of binary digits.
    const int n = 5;
    cout << "\nPermutations of 2 zeros and 3 ones:" << endl;
    int lo = bitset<5>(string("00111")).to_ulong();
    int hi = bitset<6>(string("100011")).to_ulong();
    do {
        cout << bitset<n>(lo).to_string() << " ";
    } while ((lo = next_permutation(lo)) != hi);
    cout << endl;
}
{ // Decomposition into cycles.
    vector<int> a{3, 1, 0, 2};
    cout << "\nDecomposition of {3,1,0,2} into cycles:" << endl;
    cycles c = permutation_cycles(a);
    for (const auto &cycle : c) {
        print_range(cycle.begin(), cycle.end());
    }
    cout << endl;
}
return 0;
}

```

Example Output

```

Permutations of [0, 4):
{0,1,2,3} {0,1,3,2} {0,2,1,3} {0,2,3,1} {0,3,1,2} {0,3,2,1} {1,0,2,3} {1,0,3,2}
{1,2,0,3} {1,2,3,0} {1,3,0,2} {1,3,2,0} {2,0,1,3} {2,0,3,1} {2,1,0,3} {2,1,3,0}
{2,3,0,1} {2,3,1,0} {3,0,1,2} {3,0,2,1} {3,1,0,2} {3,1,2,0} {3,2,0,1} {3,2,1,0}

Permutations of 2 zeros and 3 ones:
00111 01011 01101 01110 10011 10101 10110 11001 11010 11100

Decomposition of {3,1,0,2} into cycles:
{0,3,2} {1}

```

6.2.4 Enumerating Combinations

A combination is a subset of size k chosen from a total of n (not necessarily distinct) elements, where order does not matter.

- `next_combination(lo, mid, hi)` takes random-access iterators `lo`, `mid`, and `hi` as a range $[lo, hi)$ of n elements for which the function will rearrange such that the k elements in $[lo, mid)$ become the next lexicographically greater combination. The function returns true if such a combination exists, or false if $[lo, mid)$ already consists of the lexicographically greatest combination of the elements in $[lo, hi)$ (in which case the values are unchanged). This implementation requires an ordering on the set of possible elements defined by `operator<` on the iterator's value type.
- `next_combination(n, a)` rearranges `a` to become the next lexicographically greater combination of distinct integers in the range $[0, n)$. The vector `a` must be sorted and contain distinct integers in the range $[0, n)$.
- `next_combination_mask(x)` interprets the bits of an integer `x` as a mask with 1-bits specifying the chosen items for a combination and returns the mask of the next lexicographically greater combination (that is, the lowest integer greater than `x` with the same number of 1 bits). Note that this does not generate

combinations in the same order as `next_combination()`, nor does it work if the corresponding n items are not distinct (in that case, duplicate combinations will be generated).

- `combination_by_rank(n, k, r)` returns the combination of k distinct integers in the range $[0, n)$ that is lexicographically ranked r , where r is a 0-based rank in the range $[0, \binom{n}{k})$.
- `rank_by_combination(n, a)` returns an integer representing the 0-based rank of combination `a`, which must contain sorted distinct integers in $[0, n)$.
- `next_combination_with_repeats(n, a)` rearranges `a` to become the next lexicographically greater combination of not necessarily distinct integers in the range $[0, n)$. The vector `a` must be sorted. Note that there is a total of n multichoose k combinations if repetition is allowed, where n multichoose $k = \binom{n+k-1}{k}$.

Time Complexity:

- $O(n)$ per call to `next_combination(lo, mid, hi)`, where n is the distance between `lo` and `hi`.
- $O(k)$ per call to `next_combination(n, a)` and `next_combination_with_repeats(n, a)`, where k is the size of `a`.
- $O(1)$ per call to `next_combination_mask(x)`.
- $O(n \cdot k)$ per call to `combination_by_rank()` and `rank_by_combination()`.

Space Complexity:

- $O(k)$ auxiliary heap space for `combination_by_rank(n, k, r)`.
- $O(1)$ auxiliary for all other operations.

```
#include <algorithm>
#include <stdint>
#include <iterator>
#include <vector>

template<typename It>
bool next_combination(It lo, It mid, It hi) {
    if (lo == mid || mid == hi) {
        return false;
    }
    It l = mid - 1, h = hi - 1;
    int len1 = 1, len2 = 1;
    while (l != lo && !(l < *h)) {
        --l;
        ++len1;
    }
    if (l == lo && !(l < *h)) {
        std::rotate(lo, mid, hi);
        return false;
    }
    for (; mid < h; ++len2) {
        if (!(l < *--h)) {
            ++h;
            break;
        }
    }
    if (len1 == 1 || len2 == 1) {
        std::iter_swap(l, h);
    } else if (len1 == len2) {
        std::swap_ranges(l, mid, h);
    } else {
        std::iter_swap(l++, h++);
        int total = (--len1) + (--len2), gcd = total;
        for (int i = len1; i != 0; i--) {
            std::swap(gcd %= i, i);
        }
        int skip = total / gcd - 1;
        for (int i = 0; i < gcd; i++) {
            It curr = (i < len1) ? (l + i) : (h + (i - len1));
            int k = i;
            auto prev = *curr;
            for (int j = 0; j < skip; j++) {
                k = (k + len1) % total;
            }
        }
    }
}
```

```

        It next = (k < len1) ? (1 + k) : (h + (k - len1));
        *curr = *next;
        curr = next;
    }
    *curr = prev;
}
}
return true;
}

bool next_combination(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i) {
            a[i]++;
            while (++i < k) {
                a[i] = a[i - 1] + 1;
            }
            return true;
        }
    }
    return false;
}

int64_t next_combination_mask(int64_t x) {
    if (x == 0) {
        return 0;
    }
    int64_t s = x & -x, r = x + s;
    return r | (((x ^ r) >> 2) / s);
}

int64_t n_choose_k(int64_t n, int64_t k) {
    if (k > n - k) {
        k = n - k;
    }
    int64_t res = 1;
    for (int i = 0; i < k; i++) {
        res = res * (n - i) / (i + 1);
    }
    return res;
}

std::vector<int> combination_by_rank(int n, int k, int64_t r) {
    std::vector<int> res(k);
    int count = n;
    for (int i = 0; i < k; i++) {
        int j = 1;
        for (;;) {
            int64_t am = n_choose_k(count - j, k - 1 - i);
            if (r < am) {
                break;
            }
            r -= am;
        }
        res[i] = (i > 0) ? (res[i - 1] + j) : (j - 1);
        count -= j;
    }
    return res;
}

int64_t rank_by_combination(int n, const std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    int64_t res = 0;
    int prev = -1;
    for (int i = 0; i < k; i++) {
        for (int j = prev + 1; j < a[i]; j++) {
            res += n_choose_k(n - 1 - j, k - 1 - i);
        }
    }
}

```

```

    }
    prev = a[i];
}
return res;
}

bool next_combination_with_repeats(int n, std::vector<int> &a) {
    int k = static_cast<int>(a.size());
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - 1) {
            for (++a[i]; ++i < k;) {
                a[i] = a[i - 1];
            }
            return true;
        }
    }
    return false;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : ",");
    }
    cout << "} ";
}

int main() {
    {
        int k = 3;
        string s = "11234";
        cout << "\" " << s << "\" choose " << k << ":" << endl;
        do {
            cout << s.substr(0, k) << " ";
        } while (next_combination(s.begin(), s.begin() + k, s.end()));
        cout << endl;
    }
    { // Unordered combinations using masks.
        int n = 5, k = 3;
        string char_set = "abcde"; // Must be distinct.
        cout << "\"\n\" " << char_set << "\" choose " << k << " with masks:" << endl;
        int64_t mask = 0, dest = 0;
        for (int i = 0; i < k; i++) {
            mask |= (1 << i);
        }
        for (int i = k - 1; i < n; i++) {
            dest |= (1 << i);
        }
        do {
            for (int i = 0; i < n; i++) {
                if ((mask >> i) & 1) {
                    cout << char_set[i];
                }
            }
            cout << " ";
            mask = next_combination_mask(mask);
        } while (mask != dest);
        cout << endl;
    }
}

```

```

{ // Combinations of distinct integers from 0 to n - 1.
  int n = 5, k = 3;
  vector<int> a{0, 1, 2};
  cout << "\n" << n << " choose " << k << ":" << endl;
  int count = 0;
  do {
    print_range(a.begin(), a.end());
    vector<int> b = combination_by_rank(n, k, count);
    assert(a == b);
    assert(rank_by_combination(n, a) == count);
    count++;
  } while (next_combination(n, a));
  cout << endl;
}
{ // Combinations with repeats.
  int n = 3, k = 2;
  vector<int> a{0, 0};
  cout << "\n" << n << " multichoose " << k << ":" << endl;
  do {
    print_range(a.begin(), a.end());
  } while (next_combination_with_repeats(n, a));
  cout << endl;
}
return 0;
}

```

Example Output

```

"11234" choose 3:
112 113 114 123 124 134 234

"abcde" choose 3 with masks:
abc abd acd bcd abe ace bce ade bde

5 choose 3:
{0,1,2} {0,1,3} {0,1,4} {0,2,3} {0,2,4} {0,3,4} {1,2,3} {1,2,4} {1,3,4} {2,3,4}

3 multichoose 2:
{0,0} {0,1} {0,2} {1,1} {1,2} {2,2}

```

6.2.5 Enumerating Partitions

A partition of a natural number n is a way to write n as a sum of positive integers where the order of the addends does not matter.

- `next_partition(p)` takes a reference to a vector `p` of positive integers as a partition of n for which the function will re-assign to become the next lexicographically greater partition. The function returns true if such a partition exists, or false if `p` already consists of the lexicographically greatest partition (i.e. the single integer n).
- `partition_by_rank(n, r)` returns the partition of n that is lexicographically ranked r if addends in each partition were sorted in non-increasing order, where r is a 0-based rank in the range $[0, \text{partitions}(n))$.
- `rank_by_partition(p)` returns an integer representing the 0-based rank of the partition specified by vector `p`, which must consist of positive integers sorted in non-increasing order.
- `generate_increasing_partitions(n, f)` calls the function `f(lo, hi)` on strictly increasing partitions of n in lexicographically increasing order of partition, where `lo` and `hi` are random-access iterators to a range $[\text{lo}, \text{hi})$ of integers. Note that non-strictly increasing partitions like $\{1, 1, 1, 1\}$ are skipped.

Time Complexity:

- $O(n)$ per call to `next_partition()`.
- $O(n^2)$ per call to `partition_by_rank(n, r)` and `rank_by_partition(p)`.
- $O(p(n))$ per call to `generate_increasing_partitions(n, f)`, where $p(n)$ is the number of partitions of n .

Space Complexity:

- $O(1)$ auxiliary for `next_partition()`.
- $O(n^2)$ auxiliary heap space for `partition_function()`, `partition_by_rank()`, and `rank_by_partition()`.
- $O(n)$ auxiliary stack space for `generate_increasing_partitions()`.

```

#include <cstdint>
#include <vector>

bool next_partition(std::vector<int> &p) {
    int n = static_cast<int>(p.size());
    if (n <= 1) {
        return false;
    }
    int s = p[n - 1] - 1, i = n - 2;
    p.pop_back();
    for (; i > 0 && p[i] == p[i - 1]; i--) {
        s += p[i];
        p.pop_back();
    }
    for (p[i]++; s > 0; s--) {
        p.push_back(1);
    }
    return true;
}

int64_t partition_function(int a, int b) {
    static std::vector<std::vector<int64_t>> p(1, std::vector<int64_t>(1, 1));
    if (a >= static_cast<int>(p.size())) {
        int old = static_cast<int>(p.size());
        p.resize(a + 1);
        p[0].resize(a + 1);
        for (int i = 1; i <= a; i++) {
            p[i].resize(a + 1);
            for (int j = old; j <= i; j++) {
                p[i][j] = p[i - 1][j - 1] + p[i - 1][j];
            }
        }
    }
    return p[a][b];
}

std::vector<int> partition_by_rank(int n, int64_t r) {
    std::vector<int> res;
    for (int i = n, j; i > 0; i -= j) {
        for (j = 1; j++) {
            int64_t count = partition_function(i, j);
            if (r < count) {
                break;
            }
            r -= count;
        }
        res.push_back(j);
    }
    return res;
}

int64_t rank_by_partition(const std::vector<int> &p) {
    int64_t res = 0;
    int sum = 0;
    for (int x : p) {
        sum += x;
    }
    for (int x : p) {
        for (int j = 0; j < x; j++) {
            res += partition_function(sum, j);
        }
    }
}

```

```

    sum -= x;
}
return res;
}

using Fn = void (*)(std::vector<int>::iterator, std::vector<int>::iterator);

void generate_increasing_partitions(int left, int prev, int i, std::vector<int> &p, Fn f) {
    if (left == 0) {
        f(p.begin(), p.begin() + i);
        return;
    }
    for (p[i] = prev + 1; p[i] <= left; p[i]++) {
        generate_increasing_partitions(left - p[i], p[i], i + 1, p, f);
    }
}

void generate_increasing_partitions(int n, Fn f) {
    std::vector<int> p(n, 0);
    generate_increasing_partitions(n, 0, 0, p, f);
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : "," );
    }
    cout << "} ";
}

int main() {
    {
        int n = 4;
        vector<int> a(n, 1);
        cout << "Partitions of " << n << ":" << endl;
        int count = 0;
        do {
            print_range(a.begin(), a.end());
            vector<int> b = partition_by_rank(n, count);
            assert(equal(a.begin(), a.end(), b.begin()));
            assert(rank_by_partition(a) == count);
            count++;
        } while (next_partition(a));
        cout << endl;
    }
    {
        int n = 8;
        cout << "\nIncreasing partitions of " << n << ":" << endl;
        generate_increasing_partitions(n, print_range);
        cout << endl;
    }
    return 0;
}

```


Example Output

Partitions of 4:

{1,1,1,1} {2,1,1} {2,2} {3,1} {4}

Increasing partitions of 8:

{1,2,5} {1,3,4} {1,7} {2,6} {3,5} {8}

6.2.6 Enumerating Generic Combinatorial Sequences

Enumerate combinatorial sequences by inheriting an abstract class. Child classes of `AbstractEnumerator` must implement the `count()` function which should return the number of combinatorial sequences starting with the given prefix.

- `to_rank(a)` returns an integer representing the 0-based rank of the combinatorial sequence `a`.
- `from_rank(r)` returns a combinatorial sequence of integers that is lexicographically ranked `r`, where `r` is a 0-based rank in the range $[0, \text{total_count}())$.
- `enumerate(f)` calls the function `f(lo, hi)` on every specified combinatorial sequence in lexicographically increasing order, where `lo` and `hi` are two random-access iterators to a range $[\text{lo}, \text{hi})$ of integers.

Time Complexity: $O(n^2)$ calls will be made to `count()` per call to all operations, where n is the length of the combinatorial sequence.

Space Complexity: $O(n)$ auxiliary heap space per call to all operations.

```
#include <stdint>
#include <vector>

class AbstractEnumerator {
protected:
    int range, length;

    AbstractEnumerator(int r, int l) : range(r), length(l) {}
    virtual ~AbstractEnumerator() = default;
    virtual int64_t count(const std::vector<int> &prefix) { return 0; }
    std::vector<int> next(std::vector<int> &a) { return from_rank(to_rank(a) + 1); }
    int64_t total_count() { return count(std::vector<int>(0)); }

public:
    int64_t to_rank(const std::vector<int> &a) {
        int64_t res = 0;
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            std::vector<int> prefix(a.begin(), a.end());
            prefix.resize(i + 1);
            for (prefix[i] = 0; prefix[i] < a[i]; prefix[i]++) {
                res += count(prefix);
            }
        }
        return res;
    }

    std::vector<int> from_rank(int64_t r) {
        std::vector<int> a(length);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            std::vector<int> prefix(a.begin(), a.end());
            prefix.resize(i + 1);
            for (prefix[i] = 0; prefix[i] < range; ++prefix[i]) {
                int64_t curr = count(prefix);
                if (r < curr) {
                    break;
                }
                r -= curr;
            }
            a[i] = prefix[i];
        }
    }
}
```

```

    return a;
}

// Accepts any callable f(lo, hi), including capturing lambdas and functors.
template<typename Fn>
void enumerate(Fn f) {
    int64_t total = total_count();
    for (int64_t i = 0; i < total; i++) {
        std::vector<int> curr = from_rank(i);
        f(curr.begin(), curr.end());
    }
}

};

class ArrangementEnumerator : public AbstractEnumerator {
public:
    ArrangementEnumerator(int n, int k) : AbstractEnumerator(n, k) {}

    int64_t count(const std::vector<int> &prefix) {
        int n = static_cast<int>(prefix.size());
        for (int i = 0; i < n - 1; i++) {
            if (prefix[i] == prefix[n - 1]) {
                return 0;
            }
        }
        int64_t res = 1;
        for (int i = 0; i < length - n; i++) {
            res *= range - n - i;
        }
        return res;
    }
};

class PermutationEnumerator : public ArrangementEnumerator {
public:
    PermutationEnumerator(int n) : ArrangementEnumerator(n, n) {}
};

class CombinationEnumerator : public AbstractEnumerator {
    std::vector<std::vector<int64_t>> table;

public:
    CombinationEnumerator(int n, int k)
        : AbstractEnumerator(n, k), table(n + 1, std::vector<int64_t>(n + 1)) {
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j <= i; j++) {
                table[i][j] = (j == 0) ? 1 : table[i - 1][j - 1] + table[i - 1][j];
            }
        }
    }

    int64_t count(const std::vector<int> &prefix) {
        int n = static_cast<int>(prefix.size());
        if (n >= 2 && prefix[n - 1] <= prefix[n - 2]) {
            return 0;
        }
        if (n == 0) {
            return table[range][length - n];
        }
        return table[range - prefix[n - 1] - 1][length - n];
    }
};

class PartitionEnumerator : public AbstractEnumerator {
    std::vector<std::vector<int64_t>> table;

public:
    PartitionEnumerator(int n)

```

```

        : AbstractEnumerator(n + 1, n), table(n + 1, std::vector<int64_t>(n + 1)) {
    std::vector<std::vector<int64_t>> tmp(table);
    tmp[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            tmp[i][j] = tmp[i - 1][j - 1] + tmp[i - j][j];
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            table[i][j] = tmp[i][j] + table[i][j - 1];
        }
    }
}

int64_t count(const std::vector<int> &prefix) {
    int n = static_cast<int>(prefix.size()), sum = 0;
    for (int x : prefix) {
        sum += x;
    }
    if (sum == range - 1) {
        return 1;
    }
    if (sum > range - 1 || (n > 0 && prefix[n - 1] == 0) ||
        (n >= 2 && prefix[n - 1] > prefix[n - 2])) {
        return 0;
    }
    if (n == 0) {
        return table[range - sum - 1][range - 1];
    }
    return table[range - sum - 1][prefix[n - 1]];
}
};

```

Example Usage

```

#include <iostream>
using namespace std;

template<typename It>
void print_range(It lo, It hi) {
    cout << "{";
    for (; lo != hi; ++lo) {
        cout << *lo << (lo == hi - 1 ? "" : ",");
    }
    cout << "} ";
}

int main() {
    {
        cout << "3 permute 2 arrangements:" << endl;
        ArrangementEnumerator arr(3, 2);
        arr.enumerate([](auto lo, auto hi) { print_range(lo, hi); });
        cout << endl;
    }
    {
        cout << "\nPermutations of [0, 3):" << endl;
        PermutationEnumerator perm(3);
        perm.enumerate([](auto lo, auto hi) { print_range(lo, hi); });
        cout << endl;
    }
    {
        cout << "\n4 choose 3 combinations:" << endl;
        CombinationEnumerator comb(4, 3);
        comb.enumerate([](auto lo, auto hi) { print_range(lo, hi); });
        cout << endl;
    }
}

```

```

}
{
    cout << "\nPartition of 4:" << endl;
    PartitionEnumerator part(4);
    part.enumerate([](auto lo, auto hi) { print_range(lo, hi); });
    cout << endl;
}
return 0;
}

```

Example Output

```

3 permute 2 arrangements:
{0,1} {0,2} {1,0} {1,2} {2,0} {2,1}

Permutations of [0, 3]:
{0,1,2} {0,2,1} {1,0,2} {1,2,0} {2,0,1} {2,1,0}

4 choose 3 combinations:
{0,1,2} {0,1,3} {0,2,3} {1,2,3}

Partition of 4:
{1,1,1,1} {2,1,1,0} {2,2,0,0} {3,1,0,0} {4,0,0,0}

```

6.3 Number Theory

6.3.1 GCD, LCM, Mod Inverse, Chinese Remainder

Common number theory operations relating to modular arithmetic.

- `gcd(a, b)` returns the greatest common divisor of `a` and `b` using the Euclidean algorithm. This is mainly for educational purposes, as `std::gcd(a, b)` from `<numeric>` is available as of C++17 (`__gcd(a, b)` from `<algorithm>` in C++14 and earlier).
- `lcm(a, b)` returns the lowest common multiple of `a` and `b`. This implementation is mainly for educational purposes, as `std::lcm(a, b)` from `<numeric>` is available as of C++17.
- `extended_euclid(a, b)` returns a pair (x, y) of integers such that $\text{gcd}(a, b) = ax + by$.
- `diophantine(a, b, c, &x, &y, &g)` solves the linear Diophantine equation $ax + by = c$, returning whether a solution exists (one does if and only if $\text{gcd}(a, b)$ divides c), and on success sets `g` to $\text{gcd}(a, b)$ and (x, y) to a particular solution bounded by $\max(|a|, |b|, |c|)$ in magnitude. A 128-bit intermediate is used where available to keep the scaling step overflow-free.
- `mod(a, b)` returns the value of $a \bmod b$ under the true Euclidean definition of modulo, that is, the smallest nonnegative integer m satisfying $a + bn = m$ for some integer n . Note that this is identical to the remainder operator `%` in C++ for nonnegative operands `a` and `b`, but the result will differ when an operand is negative.
- `mod_inverse(a, m)` returns an integer x such that $ax \equiv 1 \pmod{m}$, where the arguments must satisfy $m > 0$ and $\text{gcd}(a, m) = 1$.
- `generate_inverse(p)` returns a vector `v` of integers where for each index i in the vector, $i \cdot v[i] \equiv 1 \pmod{p}$, where the argument p is prime.
- `crt(r1, m1, r2, m2, r, m)` merges the two congruences $x \equiv r_1 \pmod{m_1}$ and $x \equiv r_2 \pmod{m_2}$ for arbitrary moduli (not necessarily coprime). It returns whether the system is consistent, and on success sets `m` to `lcm(m1, m2)` and `r` to the unique solution in $[0, m)$. Fold it pairwise to merge more than two congruences.
- `garner_restore(a, p)` returns the smallest nonnegative solution x for the system of simultaneous congruences $x \equiv a[i] \pmod{p[i]}$ for all indices i in $[0, n)$, where `p` consists of pairwise coprime integers (unlike `crt`, which allows shared factors). The exact solution is unique modulo the product of all moduli, so that product and the final answer must fit in `int64_t`.
- `garner_restore_mod(a, p, m)` returns that same CRT solution modulo `m`. This is the right variant when the product of the moduli is too large to fit in `int64_t`.

Time Complexity:

- $O(\log(a + b))$ per call to `gcd(a, b)`, `lcm(a, b)`, `extended_euclid(a, b)`, `mod_inverse(a, b)`, `diophantine(a, b, c, ...)`, and `crt(...)`.
- $O(1)$ for `mod(a, b)`.
- $O(p)$ for `generate_inverse(p)`.
- $O(n^2)$ for `garner_restore(a, p)` and `garner_restore_mod(a, p, m)`.

Space Complexity:

- $O(p)$ auxiliary heap space for `generate_inverse(p)`.
- $O(n)$ auxiliary heap space for `garner_restore(a, p)` and `garner_restore_mod(a, p, m)`.
- $O(1)$ auxiliary space for all other operations.

```
#include <cstdint>
#include <type_traits>
#include <utility>
#include <vector>

template<typename Int>
Int gcd(Int a, Int b) {
    while (b != 0) {
        Int t = b;
        b = a % b;
        a = t;
    }
    return (a < 0 ? -a : a);
}

template<typename Int>
Int lcm(Int a, Int b) {
    if (a == 0 || b == 0) {
        return 0;
    }
    Int res = a / gcd(a, b) * b;
    return (res < 0 ? -res : res);
}

template<typename Int>
std::pair<Int, Int> extended_euclid(Int a, Int b) {
    Int x = 1, y = 0, x1 = 0, y1 = 1;
    while (b != 0) {
        Int q = a / b, prev_x1 = x1, prev_y1 = y1, prev_b = b;
        x1 = x - q * x1;
        y1 = y - q * y1;
        b = a - q * b;
        x = prev_x1;
        y = prev_y1;
        a = prev_b;
    }
    return (a > 0) ? std::pair<Int, Int>{x, y} : std::pair<Int, Int>{-x, -y};
}

template<typename Int>
bool diophantine(Int a, Int b, Int c, Int &x, Int &y, Int &g) {
    if (a == 0 && b == 0) {
        x = y = g = 0;
        return c == 0;
    }
    if (a == 0) {
        g = (b < 0) ? -b : b;
        if (c % b != 0) {
            return false;
        }
        x = 0;
        y = c / b;
        return true;
    }

```

```

}
if (b == 0) {
    g = (a < 0) ? -a : a;
    if (c % a != 0) {
        return false;
    }
    x = c / a;
    y = 0;
    return true;
}
g = gcd(a, b);
if (c % g != 0) {
    return false;
}
// Absorb the bulk of c into a*dx + b*dy, then scale the base solution by the small remainder so
// the reported (x, y) stay bounded by max(|a|, |b|, |c|). The scaled product is taken modulo b
// (resp. a) through a 128-bit intermediate where available, to avoid overflow.
std::pair<Int, Int> base = extended_euclid(a, b);
Int dx = c / a;
c -= dx * a;
Int dy = c / b;
c -= dy * b;
Int f = c / g;
#ifdef __SIZEOF_INT128__
    __extension__ typedef std::conditional_t<sizeof(Int) <= 4, int64_t, __int128> wide_t;
#else
    using wide_t = int64_t;
#endif
x = dx + static_cast<Int>(static_cast<wide_t>(base.first) * f % b);
y = dy + static_cast<Int>(static_cast<wide_t>(base.second) * f % a);
return true;
}

template<typename Int>
Int mod(Int a, Int m) {
    Int r = a % m;
    return (r >= 0) ? r : (r + m);
}

template<typename Int>
Int mod_inverse(Int a, Int m) {
    return mod(extended_euclid(a, m).first, m);
}

std::vector<int> generate_inverse(int p) {
    std::vector<int> res(p);
    res[1] = 1;
    for (int i = 2; i < p; i++) {
        res[i] = (p - (p / i) * res[p % i] % p) % p;
    }
    return res;
}

bool crt(int64_t r1, int64_t m1, int64_t r2, int64_t m2, int64_t &r, int64_t &m) {
    r1 = mod(r1, m1);
    r2 = mod(r2, m2);
    int64_t x, y, g;
    if (!diophantine(m1, -m2, r2 - r1, x, y, g)) {
        return false;
    }
    m = m1 / g * m2;
    r = mod(r1 + m1 * mod(x, m2 / g), m);
    return true;
}

std::vector<int64_t> garner_digits(const std::vector<int> &a, const std::vector<int> &p) {
    int n = static_cast<int>(a.size());
    std::vector<int64_t> x(a.begin(), a.end());

```

```

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            // Reduce mod p[i] each step; otherwise x[i] compounds to ~p^i and overflows int64_t.
            x[i] = mod_inverse(static_cast<int64_t>(p[j]), static_cast<int64_t>(p[i])) * (x[i] - x[j]);
            x[i] = (x[i] % p[i] + p[i]) % p[i];
        }
    }
    return x;
}

int64_t garner_restore(const std::vector<int> &a, const std::vector<int> &p) {
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return 0;
    }
    std::vector<int64_t> x = garner_digits(a, p);
    int64_t res = x[0], m = 1;
    for (int i = 1; i < n; i++) {
        m *= p[i - 1];
        res += x[i] * m;
    }
    return res;
}

int64_t garner_restore_mod(const std::vector<int> &a, const std::vector<int> &p, int64_t m) {
    int n = static_cast<int>(a.size());
    if (n == 0) {
        return 0;
    }
    std::vector<int64_t> x = garner_digits(a, p);
    int64_t res = 0;
    for (int i = n - 1; i >= 0; i--) {
#ifdef __SIZEOF_INT128__
        __extension__ typedef __int128 int128_t;
        res = static_cast<int64_t>((static_cast<int128_t>(res) * p[i] + x[i]) % m);
#else
        res = (res * p[i] + x[i]) % m; // Requires the intermediate product to fit without int128.
#endif
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    {
        for (int a = -20; a <= 20; a++) {
            for (int b = -20; b <= 20; b++) {
                int g = gcd(a, b);
                auto [x, y] = extended_euclid(a, b);
                assert(g == a * x + b * y);
                if (g == 1 && b > 1) {
                    int inv = mod_inverse(a, b);
                    assert(mod(a * inv, b) == 1);
                }
            }
        }
    }
    {
        int p = 17;
        auto res = generate_inverse(p);
        for (int i = 0; i < p; i++) {
            if (i > 0) {

```

```

        assert(mod(i * res[i], p) == 1);
    }
}
}
{
    vector<int> a{2, 3, 1}, m{3, 4, 5};
    int n = static_cast<int>(a.size());
    int x = garner_restore(a, m);
    assert(x == garner_restore_mod(a, m, 1000000007));
    for (int i = 0; i < n; i++) {
        assert(mod(x, m[i]) == a[i]);
    }
    assert(x == 11);
}
#ifdef __SIZEOF_INT128__
{ // Garner modulo another number works even when the product of CRT moduli is too large.
    vector<int> p{1000000007, 1000000009, 1000000033};
    __extension__ typedef __int128 int128_t;
    int128_t x = (static_cast<int128_t>(1) << 80) + 123456789;
    vector<int> a;
    for (int m : p) {
        a.push_back(static_cast<int>(x % m));
    }
    int64_t m = 998244353;
    assert(garner_restore_mod(a, p, m) == static_cast<int64_t>(x % m));
}
#endif
{ // Diophantine: exhaustively verify solvability and that solutions satisfy a*x + b*y == c.
    for (int a = -8; a <= 8; a++) {
        for (int b = -8; b <= 8; b++) {
            for (int c = -8; c <= 8; c++) {
                int x, y, g, gg = gcd(a, b);
                bool ok = diophantine(a, b, c, x, y, g);
                assert(ok == (gg == 0 ? c == 0 : c % gg == 0));
                if (ok) {
                    assert(a * x + b * y == c);
                }
            }
        }
    }
}
{ // CRT: merge two congruences, including non-coprime and inconsistent moduli.
    for (int m1 = 1; m1 <= 12; m1++) {
        for (int m2 = 1; m2 <= 12; m2++) {
            for (int r1 = 0; r1 < m1; r1++) {
                for (int r2 = 0; r2 < m2; r2++) {
                    int64_t r, m;
                    bool ok = crt(r1, m1, r2, m2, r, m);
                    int limit = lcm(m1, m2), want = -1;
                    for (int v = 0; v < limit && want < 0; v++) {
                        if (v % m1 == r1 && v % m2 == r2) {
                            want = v;
                        }
                    }
                    assert(ok == (want >= 0));
                    if (ok) {
                        assert(m == limit && r == want);
                    }
                }
            }
        }
    }
}
return 0;
}

```


6.3.2 Modular Arithmetic

Wraps arithmetic modulo a compile-time constant in a small value type. This is a common contest helper for dynamic programming, combinatorics, polynomial operations, and any calculation where all answers are taken modulo some number p such as $10^9 + 7$.

The implementation is intentionally close to common contest “Mint” templates: normalization happens at construction, arithmetic operators are overloaded, mixed integer operations are supported through implicit construction, and combinations can be computed from lazy factorial tables. The operators use the hidden friend idiom: each `friend` defined inside the class is a non-template function, so mixed expressions like `2 + Mint(3)` convert the integer operand through the implicit constructor. (Free function template operators cannot do this - template argument deduction ignores implicit conversions - which is why some contest templates carry per-operator `Modular op U` and `U op Modular` overloads instead.)

The modulus is supplied directly as the `auto` template argument, so `Modular<1000000007>` and `Modular<998244353>` are all that is needed. Its literal type determines the storage type, so a 64-bit modulus (e.g. `Modular<(1LL << 61) - 1>`) is stored in 64 bits without any extra annotation. The modulus must be a compile-time constant; a runtime-chosen modulus is not supported by this design.

Two integer types are at play: the type of the modulus stores the representative, while construction, multiplication, and inverses widen through an intermediate type chosen automatically from the storage width - `int64_t` for 32-bit storage, or `__int128` (a GCC/Clang extension, in line with this book’s use of `__builtin` functions) for 64-bit storage; on compilers without `__int128`, only 32-bit moduli are supported. The requirements are that $2p$ fits the storage type (for addition) and $(p - 1)^2$ fits the intermediate type (for the widening multiply), so any modulus up to half the storage type’s range works out of the box; 64-bit moduli simply pay the cost of slower 128-bit multiplies. (Some contest templates instead estimate the multiply’s quotient in `long double` to avoid `__int128`; beware that this requires x86’s 80-bit `long double` and silently breaks for large moduli on ARM, where `long double` is only 64-bit.)

Division uses the extended Euclidean algorithm, so the divisor only needs to be coprime to the modulus. For the factorial-table combination helper, the usual contest assumption is that p is prime and the requested factorials are invertible modulo p .

- `Modular<MOD>(x)` constructs the residue class of integer `x` modulo `MOD`.
- `value()` and `operator()()` return the stored representative in $[0, \text{MOD})$.
- `pow(n)` returns this value raised to nonnegative integer exponent `n`.
- `inv()` returns the multiplicative inverse, asserting it exists.
- Operators `+`, `-`, `*`, `/`, comparison, increment, decrement, and stream I/O are overloaded.
- `ModCombinatorics<Mint>::factorial(n)` returns $n!$ using a lazy factorial table.
- `ModCombinatorics<Mint>::choose(n, k)` returns $\binom{n}{k}$ using lazy factorial and inverse-factorial tables.
- `ModCombinatorics<Mint>::permute(n, k)` returns the number of ordered selections of `k` distinct elements from `n`.
- `ModCombinatorics<Mint>::multichoose(n, k)` returns the number of size-`k` multisets drawn from `n` types.

Time Complexity:

- $O(1)$ per addition, subtraction, multiplication, comparison, and stream output.
- $O(\log n)$ per call to `pow(n)`.
- $O(\log \text{MOD})$ per call to `inv()` and division.
- $O(n)$ total table growth to answer factorials and combinations up to size `n`.

Space Complexity:

- $O(1)$ auxiliary for Modular arithmetic.
- $O(n)$ for factorial and inverse-factorial tables grown through `choose(n, k)`.

```
#include <cassert>
#include <cstdint>
#include <iostream>
#include <type_traits>
```

```

#include <utility>
#include <vector>

template<typename T>
T inverse(T a, T m) {
    T original_m = m;
    T u = 0, v = 1;
    while (a != 0) {
        T q = m / a;
        m -= q * a;
        std::swap(a, m);
        u -= q * v;
        std::swap(u, v);
    }
    assert(m == 1);
    u %= original_m;
    return u < 0 ? u + original_m : u;
}

template<auto MOD>
class Modular {
    using value_t = decltype(MOD);

    // Widen through int64_t for 32-bit storage, or __int128 for 64-bit storage where it exists
    // (__extension__ silences -pedantic). Without __int128, only 32-bit moduli are supported.
    #if defined(__SIZEOF_INT128__)
        __extension__ typedef std::conditional_t<sizeof(value_t) <= 4, int64_t, __int128> wide_t;
    #else
        using wide_t = int64_t;
        static_assert(sizeof(value_t) <= 4, "64-bit moduli require __int128, which is unavailable");
    #endif

    value_t v;

    static value_t normalize(wide_t x) {
        if (-static_cast<wide_t>(mod()) <= x && x < static_cast<wide_t>(mod())) {
            value_t y = static_cast<value_t>(x);
            return y < 0 ? y + mod() : y;
        }
        x %= mod();
        if (x < 0) {
            x += mod();
        }
        return static_cast<value_t>(x);
    }

public:
    Modular(wide_t x = 0) { v = normalize(x); }

    static value_t mod() { return MOD; }
    value_t value() const { return v; }
    value_t operator()() const { return v; }

    Modular pow(int64_t n) const {
        assert(n >= 0);
        Modular base = *this, res = 1;
        while (n > 0) {
            if (n & 1) {
                res *= base;
            }
            base *= base;
            n >>= 1;
        }
        return res;
    }

    Modular inv() const {
        assert(v != 0);

```

```

    return Modular(inverse(static_cast<wide_t>(v), static_cast<wide_t>(mod())));
}

Modular &operator+=(const Modular &other) {
    v += other.v;
    if (v >= mod()) {
        v -= mod();
    }
    return *this;
}

Modular &operator--(const Modular &other) {
    v -= other.v;
    if (v < 0) {
        v += mod();
    }
    return *this;
}

Modular &operator*=(const Modular &other) {
    v = normalize(static_cast<wide_t>(v) * other.v);
    return *this;
}

Modular operator-(const Modular &other) { return Modular(-v); }
Modular &operator++() { return *this += 1; }
Modular &operator--() { return *this -= 1; }
Modular &operator/=(const Modular &other) { return *this *= other.inv(); }
Modular operator++(int) { Modular res = *this; ++*this; return res; }
Modular operator--(int) { Modular res = *this; --*this; return res; }
friend Modular operator+(Modular a, const Modular &b) { return a += b; }
friend Modular operator-(Modular a, const Modular &b) { return a -= b; }
friend Modular operator*(Modular a, const Modular &b) { return a *= b; }
friend Modular operator/(Modular a, const Modular &b) { return a /= b; }
friend bool operator==(const Modular &a, const Modular &b) { return a.v == b.v; }
friend bool operator!=(const Modular &a, const Modular &b) { return !(a == b); }
friend bool operator<(const Modular &a, const Modular &b) { return a.v < b.v; }
friend std::ostream &operator<<(std::ostream &os, const Modular &x) { return os << x.v; }

friend std::istream &operator>>(std::istream &is, Modular &x) {
    int64_t value;
    is >> value;
    x = Modular(value);
    return is;
}
};

template<typename Mint>
class ModCombinatorics {
    std::vector<Mint> fact, inv_fact;

    void ensure(int n) {
        auto old = static_cast<int>(fact.size());
        if (old > n) {
            return;
        }
        fact.resize(n + 1);
        inv_fact.resize(n + 1);
        for (int i = old; i <= n; i++) {
            fact[i] = fact[i - 1] * i;
        }
        inv_fact[n] = fact[n].inv();
        for (int i = n; i > old; i--) {
            inv_fact[i - 1] = inv_fact[i] * i;
        }
    }

public:

```

```

ModCombinatorics() : fact(1, Mint(1)), inv_fact(1, Mint(1)) {}

Mint factorial(int n) {
    ensure(n);
    return fact[n];
}

Mint choose(int n, int k) {
    if (k < 0 || k > n) {
        return 0;
    }
    ensure(n);
    return fact[n] * inv_fact[k] * inv_fact[n - k];
}

Mint permute(int n, int k) {
    if (k < 0 || k > n) {
        return 0;
    }
    ensure(n);
    return fact[n] * inv_fact[n - k];
}

Mint multichoose(int n, int k) { return choose(n + k - 1, k); }
};

using Mint = Modular<1000000007>;

```

Example Usage

```

#include <cassert>
#include <sstream>
using namespace std;

int main() {
    Mint a = 1000000008LL;
    Mint b = -2;
    assert(a.value() == 1);
    assert(b() == Mint::mod() - 2);
    assert(a + b == -1);
    assert(2 + Mint(3) == 5);
    assert(Mint(2) * 3 == 6);
    assert(Mint(2).pow(10) == 1024);
    assert(Mint(5) / 5 == 1);

    Mint x = 0;
    x++;
    ++x;
    assert(x == 2);
    stringstream ss("1000000008");
    ss >> x;
    assert(x == 1);

    ModCombinatorics<Mint> comb;
    assert(comb.factorial(5) == 120);
    assert(comb.choose(5, 2) == 10);
    assert(comb.permute(5, 2) == 20);
    assert(comb.multichoose(3, 2) == 6);

    // Each distinct modulus is just another instantiation.
    using Mint2 = Modular<998244353>;
    assert(Mint2(2).pow(10) == 1024);
    assert(Mint2(-1) == Mint2::mod() - 1);

    // A 64-bit modulus (deduced as long long) automatically widens through __int128.
    using Mint3 = Modular<(1LL << 61) - 1>;

```

```

assert(Mint3(2).pow(62) == 2); // 2^62 mod (2^61 - 1)
assert(Mint3(3).inv() * 3 == 1);
return 0;
}

```

6.3.3 Primitive Roots

A primitive root modulo m is a number g whose powers generate every invertible residue modulo m . Equivalently, the multiplicative order of g modulo m is exactly $\phi(m)$. Primitive roots are useful for constructing generators of finite multiplicative groups, choosing roots for number theoretic transforms, discrete logarithm setups, and randomized modular hashing or testing schemes that need a full-period multiplicative generator.

Primitive roots do not exist for every m . They exist exactly for $m \in \{1, 2, 4, p^k, 2p^k\}$, where p is an odd prime and $k \geq 1$. This implementation first checks that classification by factoring m , then factors $\phi(m)$. A candidate g is primitive exactly when $g^{\phi(m)} \equiv 1 \pmod{m}$ and $g^{\phi(m)/q} \not\equiv 1 \pmod{m}$ for every prime factor q of $\phi(m)$.

- `primitive_root(m)` returns the smallest primitive root modulo `m`, or `-1` if no primitive root exists.

For a prime modulus p , this reduces to the common contest test: factor $p - 1$, then scan for the first g such that $g^{(p-1)/q} \not\equiv 1 \pmod{p}$ for every prime factor q of $p - 1$.

Modular multiplication uses `__uint128_t` when available for speed, with a slower portable double-and-add fallback to avoid overflow on compilers without 128-bit integers.

Time Complexity: $O(\sqrt{m} + g \cdot k \cdot \log m)$, where `g` is the answer candidate tested and k is the number of distinct prime factors of $\phi(m)$.

Space Complexity: $O(k)$ auxiliary heap space.

```

#include <cstdint>
#include <numeric>
#include <set>
#include <utility>
#include <vector>

int64_t mulmod(int64_t a, int64_t b, int64_t m) {
    #if defined(__SIZEOF_INT128__)
        return static_cast<int64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        int64_t res = 0;
        for (a %= m; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - a ? res - (m - a) : res + a;
            }
            a = a >= m - a ? a - (m - a) : a + a;
        }
        return res;
    #endif
}

int64_t powmod(int64_t a, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (a %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = mulmod(res, a, m);
        }
        a = mulmod(a, a, m);
    }
    return res;
}

std::vector<std::pair<int64_t, int>> factorize(int64_t n) {
    std::vector<std::pair<int64_t, int>> res;

```

```

for (int64_t p = 2; p <= n / p; p += (p == 2 ? 1 : 2)) {
    if (n % p != 0) {
        continue;
    }
    int count = 0;
    while (n % p == 0) {
        n /= p;
        count++;
    }
    res.emplace_back(p, count);
}
if (n > 1) {
    res.emplace_back(n, 1);
}
return res;
}

int64_t primitive_root(int64_t m) {
    if (m <= 0) {
        return -1;
    }
    if (m == 1 || m == 2 || m == 4) {
        return m - 1;
    }
    std::vector<std::pair<int64_t, int>> factors = factorize(m);
    bool has_root = false;
    if (factors[0].first == 2) {
        has_root = factors[0].second == 1 && factors.size() == 2;
    } else {
        has_root = factors.size() == 1;
    }
    if (!has_root) {
        return -1;
    }
    int64_t phi = m;
    std::set<int64_t> phi_factors;
    for (auto [p, e] : factors) {
        phi = phi / p * (p - 1);
        if (e > 1) {
            phi_factors.insert(p);
        }
        for (auto [q, count] : factorize(p - 1)) {
            phi_factors.insert(q);
        }
    }
    for (int64_t g = 1; g < m; g++) {
        if (std::gcd(g, m) != 1 || powmod(g, phi, m) != 1) {
            continue;
        }
        bool ok = true;
        for (int64_t q : phi_factors) {
            if (powmod(g, phi / q, m) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) {
            return g;
        }
    }
    return -1;
}

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    assert(primitive_root(1) == 0);
    assert(primitive_root(2) == 1);
    assert(primitive_root(4) == 3);

    assert(primitive_root(17) == 3);           // 3 has order phi(17) = 16.
    assert(primitive_root(998244353) == 3);    // Common NTT prime.
    assert(primitive_root(9) == 2);           // Odd prime power.
    assert(primitive_root(18) == 5);           // Twice an odd prime power.

    assert(primitive_root(8) == -1);
    assert(primitive_root(12) == -1);
    return 0;
}
```

6.3.4 Discrete Logarithm (Baby-Step Giant-Step)

Given integers a , b , and a modulus m , the discrete logarithm problem asks for the smallest nonnegative integer x with a^x congruent to b modulo m . This is the modular analogue of a logarithm and underlies Diffie-Hellman key exchange, order computations in the multiplicative group, and many number-theory reductions.

The baby-step giant-step method writes $x = np - q$ for a block size n near the square root of the modulus. It tabulates the “baby steps” $b \cdot a^q$ for q in $[0, n]$, then walks the “giant steps” a^{np} for increasing p until one lands in the table, yielding $x = np - q$. This trades the linear scan of all exponents for $O(\sqrt{m})$ work and memory. A short reduction loop first strips common factors between a and m , so the routine works for any modulus, not only when a and m are coprime.

- `discrete_log(a, b, m)` returns the smallest nonnegative `x` with `ax` congruent to `b` modulo `m`, or `-1` if no such `x` exists. Requires `m ≥ 1`; `a` and `b` are reduced modulo `m` internally.

Time Complexity: $O(\sqrt{m} \cdot \log(m))$ per call to `discrete_log()`, dominated by the hashed baby-step table.

Space Complexity: $O(\sqrt{m})$ auxiliary heap space for the table.

```
#include <cmath>
#include <cstdint>
#include <numeric>
#include <unordered_map>

int64_t mulmod(int64_t a, int64_t b, int64_t m) {
    #if defined(__SIZEOF_INT128__)
        return static_cast<int64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        int64_t res = 0;
        for (a %= m; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - a ? res - (m - a) : res + a;
            }
            a = a >= m - a ? a - (m - a) : a + a;
        }
        return res;
    #endif
}

int64_t discrete_log(int64_t a, int64_t b, int64_t m) {
    a %= m;
    b %= m;
    if (a < 0) {
```

```

    a += m;
}
if (b < 0) {
    b += m;
}
// Strip common factors of a and m so that a becomes invertible; add counts the exponents removed.
int64_t k = 1 % m, add = 0, g;
while ((g = std::gcd(a, m)) > 1) {
    if (b == k) {
        return add;
    }
    if (b % g != 0) {
        return -1;
    }
    b /= g;
    m /= g;
    add++;
    k = mulmod(k, a / g, m);
}
int64_t n = static_cast<int64_t>(std::sqrt(static_cast<double>(m))) + 1;
int64_t an = 1;
for (int64_t i = 0; i < n; i++) {
    an = mulmod(an, a, m); // an = a^n, the giant step.
}
std::unordered_map<int64_t, int64_t> baby;
int64_t cur = b;
for (int64_t q = 0; q <= n; q++) {
    baby[cur] = q; // Keep the largest q for each value to make the answer minimal.
    cur = mulmod(cur, a, m);
}
cur = k;
for (int64_t p = 1; p <= n; p++) {
    cur = mulmod(cur, an, m);
    auto it = baby.find(cur);
    if (it != baby.end()) {
        return n * p - it->second + add;
    }
}
return -1;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // 2^x = 9 (mod 11): 2^6 = 64 = 9, and 6 is the smallest such exponent.
    assert(discrete_log(2, 9, 11) == 6);

    assert(discrete_log(3, 1, 7) == 0); // a^0 = 1 always.
    assert(discrete_log(2, 3, 5) == 3); // 2^3 = 8 = 3 (mod 5).
    assert(discrete_log(2, 1, 6) == 0); // Works for composite, non-coprime moduli.
    assert(discrete_log(4, 6, 8) == -1); // No power of 4 is 6 (mod 8).
    assert(discrete_log(5, 33, 58) == 9); // 5^9 = 33 (mod 58).
    return 0;
}

```


6.3.5 Modular Square Root (Tonelli-Shanks)

Given an integer a and an odd prime p , find a modular square root: an x with x^2 congruent to a modulo p .

Exactly half of the nonzero residues modulo a prime are squares (quadratic residues), so a solution exists only for those; when it does, the two roots are x and $p - x$.

The Tonelli-Shanks algorithm handles the general case. Euler's criterion, $a^{(p-1)/2}$ modulo p , first tests whether a is a residue at all. When p is congruent to 3 modulo 4 the root is simply $a^{(p+1)/4}$. Otherwise the algorithm factors $p - 1$ as $s \cdot 2^e$, picks any quadratic non-residue to generate the 2-power part of the group, and repeatedly squares a running value to find the order of the current error term, correcting the candidate root until the error becomes 1.

- `mod_sqrt(a, p)` returns the smaller of the two square roots of `a` modulo the odd prime `p`, or `-1` if `a` is not a quadratic residue. The value 0 maps to 0. The argument `a` is reduced modulo `p` internally. The modulus `p` must be an odd prime.

Time Complexity: $O(\log^2 p)$ per call to `mod_sqrt()` in the worst case, from the squaring loop nested inside the exponentiations.

Space Complexity: $O(1)$ auxiliary.

```
#include <stdint>

int64_t mulmod(int64_t a, int64_t b, int64_t m) {
    #if defined(__SIZEOF_INT128__)
        return static_cast<int64_t>(static_cast<__uint128_t>(a) * b % m);
    #else
        int64_t res = 0;
        for (a %= m; b > 0; b >>= 1) {
            if (b & 1) {
                res = res >= m - a ? res - (m - a) : res + a;
            }
            a = a >= m - a ? a - (m - a) : a + a;
        }
        return res;
    #endif
}

int64_t powmod(int64_t a, int64_t n, int64_t m) {
    int64_t res = 1 % m;
    for (a %= m; n > 0; n >>= 1) {
        if (n & 1) {
            res = mulmod(res, a, m);
        }
        a = mulmod(a, a, m);
    }
    return res;
}

int64_t mod_sqrt(int64_t a, int64_t p) {
    a %= p;
    if (a < 0) {
        a += p;
    }
    if (a == 0) {
        return 0;
    }
    if (powmod(a, (p - 1) / 2, p) != 1) {
        return -1; // a is a quadratic non-residue.
    }
    if (p % 4 == 3) {
        int64_t x = powmod(a, (p + 1) / 4, p);
        return x < p - x ? x : p - x;
    }
    // Write p - 1 = s * 2^e with s odd.
    int64_t s = p - 1, e = 0;
    while (s % 2 == 0) {
```

```

    s /= 2;
    e++;
}
// Any quadratic non-residue generates the 2-power part of the group.
int64_t n = 2;
while (powmod(n, (p - 1) / 2, p) != p - 1) {
    n++;
}
int64_t x = powmod(a, (s + 1) / 2, p); // Candidate root, off by a 2-power factor.
int64_t b = powmod(a, s, p);         // Error term, a power of two in order.
int64_t g = powmod(n, s, p);         // Generator of the relevant 2-group.
int64_t r = e;
while (true) {
    int64_t t = b, order = 0;
    while (order < r && t != 1) {
        t = mulmod(t, t, p);
        order++;
    }
    if (order == 0) {
        return x < p - x ? x : p - x;
    }
    int64_t gs = powmod(g, static_cast<int64_t>(1) << (r - order - 1), p);
    g = mulmod(gs, gs, p);
    x = mulmod(x, gs, p);
    b = mulmod(b, g, p);
    r = order;
}
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    int64_t r = mod_sqrt(2, 113);
    assert(r != -1 && mulmod(r, r, 113) == 2);
    assert(r == 51); // 51^2 = 2601 = 2 (mod 113); the other root is 62.

    assert(mod_sqrt(0, 7) == 0);
    assert(mod_sqrt(5, 41) != -1); // p = 1 (mod 4) exercises the full algorithm.
    assert(mod_sqrt(4, 7) == 2);   // p = 3 (mod 4) fast path.
    assert(mod_sqrt(3, 7) == -1);  // 3 is a non-residue modulo 7.

    // Every residue's reported root squares back to it.
    for (int64_t a = 0; a < 41; a++) {
        int64_t x = mod_sqrt(a, 41);
        if (x != -1) {
            assert(mulmod(x, x, 41) == a);
        }
    }
    return 0;
}

```

6.3.6 Jacobi Symbol

The Jacobi symbol (a/n) , defined for an odd positive integer n , generalizes the Legendre symbol. For an odd prime p the Legendre symbol (a/p) is 0 when p divides a , 1 when a is a nonzero quadratic residue modulo p , and -1 when a is a non-residue. The Jacobi symbol extends this to composite n by multiplying the Legendre symbols of the prime factors of n , counted with multiplicity. It is computed without factoring n , using the law of quadratic reciprocity together with the supplementary rules for -1 and 2 , in the style of the binary GCD.

The symbol is fully multiplicative in a and in n . For prime n it exactly decides quadratic residuosity, so it is the fast way to test whether a modular square root exists. For composite n , beware that $(a/n) = 1$ does not guarantee that a is a quadratic residue; this asymmetry is what the Solovay-Strassen primality test exploits.

- `jacobi(a, n)` returns the Jacobi symbol (a / n) as one of -1 , 0 , or 1 . The modulus `n` must be a positive odd integer; `a` is reduced modulo `n` internally and may be negative. The result is 0 exactly when `a` and `n` share a common factor.

Time Complexity: $O(\log a \cdot \log n)$ per call to `jacobi()`.

Space Complexity: $O(1)$ auxiliary.

```
#include <stdint>
#include <utility>

int jacobi(int64_t a, int64_t n) {
    a %= n;
    if (a < 0) {
        a += n;
    }
    int result = 1;
    while (a != 0) {
        while (a % 2 == 0) {
            a /= 2;
            // (2 / n) is -1 when n is 3 or 5 modulo 8.
            if (n % 8 == 3 || n % 8 == 5) {
                result = -result;
            }
        }
        std::swap(a, n);
        // Quadratic reciprocity flips the sign when both are 3 modulo 4.
        if (a % 4 == 3 && n % 4 == 3) {
            result = -result;
        }
        a %= n;
    }
    return n == 1 ? result : 0;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int64_t legendre_via_euler(int64_t a, int64_t p) {
    int64_t r = 1 % p;
    for (int64_t b = ((a % p) + p) % p, e = (p - 1) / 2; e > 0; e >= 1) {
        if (e & 1) {
            r = r * b % p;
        }
        b = b * b % p;
    }
    return r == p - 1 ? -1 : r; // r is 0, 1, or p - 1.
}

int main() {
    assert(jacobi(2, 15) == 1); // (2/15) = (2/3)(2/5) = (-1)(-1).
    assert(jacobi(7, 15) == -1);
    assert(jacobi(3, 15) == 0); // gcd(3, 15) = 3.
    assert(jacobi(1001, 9907) == -1);

    // For odd primes the Jacobi symbol equals the Legendre symbol from Euler's criterion.
    for (int64_t p : {3, 5, 7, 11, 13, 101, 9973}) {
        for (int64_t a = 0; a < 50; a++) {
            assert(jacobi(a, p) == legendre_via_euler(a, p));
        }
    }
}
```

```

    }
}
return 0;
}

```

6.3.7 Prime Generation

Generate prime numbers using the Sieve of Eratosthenes, which repeatedly marks the multiples of each prime as composite.

The range overload `sieve(lo, hi)` uses a segmented sieve, letting primes in a high but narrow window be found without sieving everything below it. Since every composite at most `hi` has a prime factor at most $\sqrt{hi}$, it first sieves the primes up to $\sqrt{hi}$, then uses only those primes to mark composites within `[lo, hi]` (each prime p starting at the first of its multiples that is at least both p^2 and `lo`). This needs only $O(hi - lo + \sqrt{hi})$ space instead of the $O(hi)$ that sieving all of `[2, hi]` would require.

- `sieve(n)` returns a vector of all the primes less than or equal to `n`.
- `sieve(lo, hi)` returns a vector of all the primes in the range `[lo, hi]`.
- `linear_sieve(n, least_out)` returns a vector of all primes less than or equal to `n` in linear time. If `least_out` is not null, it is filled so that `(*least_out)[x]` is the least prime factor of `x` for every `x` ≥ 2 .

Time Complexity:

- $O(n \log(\log(n)))$ per call to `sieve(n)`.
- $O(\sqrt{hi} \cdot \log(\log(hi - lo)))$ per call to `sieve(lo, hi)`.
- $O(n)$ per call to `linear_sieve(n, least_out)`.

Space Complexity:

- $O(n)$ auxiliary heap space per call to `sieve(n)`.
- $O(hi - lo + \sqrt{hi})$ auxiliary heap space per call to `sieve(lo, hi)`.
- $O(n)$ auxiliary heap space per call to `linear_sieve(n, least_out)`.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <vector>

std::vector<int> sieve(int n) {
    if (n < 2) {
        return {};
    }
    std::vector<bool> prime(n + 1, true);
    for (int i = 2; i <= n / i; i++) {
        if (prime[i]) {
            for (int64_t j = 1LL * i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }
    std::vector<int> res;
    for (int i = 2; i <= n; i++) {
        if (prime[i]) {
            res.push_back(i);
        }
    }
    return res;
}

std::vector<int> sieve(int lo, int hi) {
    if (hi < 2 || lo > hi) {
        return {};
    }

```

```

    }
    int sqrt_hi = sqrt(hi);
    while (1LL * (sqrt_hi + 1) * (sqrt_hi + 1) <= hi) {
        sqrt_hi++;
    }
    int fourth_root_hi = sqrt(sqrt_hi);
    while (1LL * (fourth_root_hi + 1) * (fourth_root_hi + 1) <= sqrt_hi) {
        fourth_root_hi++;
    }
    std::vector<bool> prime1(sqrt_hi + 1, true), prime2(hi - lo + 1, true);
    for (int i = 2; i <= fourth_root_hi; i++) {
        if (prime1[i]) {
            for (int64_t j = 1LL * i * i; j <= sqrt_hi; j += i) {
                prime1[j] = false;
            }
        }
    }
    for (int i = 2; i <= sqrt_hi; i++) {
        if (prime1[i]) {
            int64_t start = std::max(1LL * i * i, ((static_cast<int64_t>(lo) + i - 1) / i) * i);
            for (int64_t j = start; j <= hi; j += i) {
                prime2[j - lo] = false;
            }
        }
    }
    std::vector<int> res;
    for (int i = (lo > 1) ? lo : 2; i <= hi; i++) {
        if (prime2[i - lo]) {
            res.push_back(i);
        }
    }
    return res;
}

std::vector<int> linear_sieve(int n, std::vector<int> *least_out = nullptr) {
    if (n < 2) {
        if (least_out != nullptr) {
            least_out->assign(std::max(n + 1, 0), 0);
        }
        return {};
    }
    std::vector<int> least(n + 1, 0), primes;
    for (int i = 2; i <= n; i++) {
        if (least[i] == 0) {
            least[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if (p > least[i] || i > n / p) {
                break;
            }
            least[i * p] = p;
        }
    }
    if (least_out != nullptr) {
        *least_out = least;
    }
    return primes;
}

```

Example Usage

```

#include <cassert>
#include <ctime>
#include <iostream>
using namespace std;

```

```

int main() {
    int pmax = 10000000;
    time_t start;
    double delta;

    start = clock();
    auto p = sieve(pmax);
    delta = static_cast<double>((clock() - start)) / CLOCKS_PER_SEC;
    cout << "sieve(n=" << pmax << "): " << delta << "s" << endl;

    vector<int> least;
    assert(linear_sieve(pmax, &least) == p);
    assert(least[999983] == 999983);
    assert(least[999984] == 2);

    int l = 1000000000, h = 1005000000;
    start = clock();
    p = sieve(l, h);
    delta = static_cast<double>((clock() - start)) / CLOCKS_PER_SEC;
    cout << "sieve([" << l << ", " << h << "]): " << delta << "s" << endl;
    return 0;
}

```

Example Output

```

sieve(n=10000000): 0.042641s
sieve([1000000000, 1005000000]): 0.01969s

```

6.3.8 Prime Tests and Factorization

Test whether an integer is prime and compute the prime factorization of an integer n . The Miller-Rabin test writes $n - 1 = d \cdot 2^r$ and squares a^d repeatedly, checking that the sequence behaves the only way it can for a prime; a base a that breaks the pattern is a witness that n is composite. For large composite inputs, Pollard's rho iterates a pseudorandom map modulo n ; two iterates that collide modulo a hidden prime factor reveal that factor through a GCD with their difference. Prime factorizations are represented as sorted vectors of (prime, exponent) pairs. For 0 and 1, the prime factorization is empty.

- `is_prime_slow(n)` returns whether the integer `n` is prime using trial division.
- `factorize_slow(n)` returns the prime factorization of `n` using trial division.
- `is_probable_prime(n, k)` returns whether `n` is probably prime using `k` random Miller-Rabin bases. The result is guaranteed correct when `n` is prime. When `n` is composite, it returns false with probability at least $1 - (1/4)^k$, so a true result has one-sided error probability at most $(1/4)^k$. This is mostly useful when adapting Miller-Rabin outside the known 64-bit deterministic setting.
- `rho_factor(n)` returns a factor of `n` that is not necessarily prime using Pollard's rho with Brent's optimization. If `n` is prime, then `n` itself is returned. While this algorithm is non-deterministic and may fail to detect factors on certain runs of the same input, it can be retried until a nontrivial factor is found, as done in `factorize()`.
- `is_prime(n)` returns whether the signed 64-bit integer `n` is prime using deterministic Miller-Rabin bases sufficient up to and including $2^{63} - 1$.
- `cached_sieve(n)` returns a cached linear sieve up to and including `n`, storing both primes and least prime factors.
- `merge_factors(a, b)` merges two sorted compressed factorizations.
- `factorize_rho(n)` returns the prime factorization of a 64-bit integer using Miller-Rabin and Pollard's rho without initial trial division.
- `factorize(n, small_prime_limit)` returns the prime factorization of a 64-bit integer `n` using a combination of trial division, the Miller-Rabin primality test, and Pollard's rho algorithm. `small_prime_limit` specifies the largest prime to test with trial division before falling back to the rho algorithm. This supports 64-bit integers up to and including $2^{63} - 1$.

- `divisors_from_factors(factors)` returns all divisors from a compressed factorization.
- `get_divisors(n)` returns all divisors of a 64-bit integer using `factorize()` followed by `divisors_from_factors()`.

Modular multiplication inside Miller-Rabin and Pollard's rho uses `__uint128_t` when available for speed, with a slower portable double-and-add fallback to avoid overflow on compilers without 128-bit integers.

Time Complexity:

- $O(\sqrt{n})$ per call to `is_prime_slow(n)`.
- $O(k \log^3(n))$ per call to `is_probable_prime(n, k)`.
- $O(\log^3(n))$ per call to `is_prime(n)`.
- $O(\sqrt{n})$ per call to `factorize_slow(n)`.
- $O(n)$ to rebuild the prime cache in `cached_sieve(n)`.
- Unknown, but approximately $O(n^{1/4})$ per call to `rho_factor(n)` and `factorize(n)`.

Space Complexity:

- $O(f)$ auxiliary heap space for all operations, where f is the number of factors returned.
- $O(c)$ cached heap space for the largest `cached_sieve(c)` call.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <numeric>
#include <random>
#include <utility>
#include <vector>

template<typename Int>
bool is_prime_slow(Int n) {
    if (n == 2 || n == 3) {
        return true;
    }
    if (n < 2 || n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    for (Int i = 5, w = 4; i <= n / i; i += w) {
        if (n % i == 0) {
            return false;
        }
        w = 6 - w;
    }
    return true;
}

template<typename Int>
std::vector<std::pair<Int, int>> factorize_slow(Int n) {
    if (n <= 1) {
        return {};
    }
    std::vector<std::pair<Int, int>> res;
    for (Int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            int p = 0;
            do {
                n /= i;
                p++;
            } while (n % i == 0);
            res.emplace_back(i, p);
        }
    }
    if (n > 1) {
        res.emplace_back(n, 1);
    }
    return res;
}
```

```

uint64_t mulmod(uint64_t x, uint64_t n, uint64_t m) {
#ifdef __SIZEOF_INT128__
    return static_cast<uint64_t>(static_cast<__uint128_t>(x) * n % m);
#else
    uint64_t a = 0, b = x % m;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = a >= m - b ? a - (m - b) : a + b;
        }
        b = b >= m - b ? b - (m - b) : b + b;
    }
    return a;
#endif
}

uint64_t powmod(uint64_t x, uint64_t n, uint64_t m) {
    uint64_t a = 1, b = x;
    for (; n > 0; n >>= 1) {
        if (n & 1) {
            a = mulmod(a, b, m);
        }
        b = mulmod(b, b, m);
    }
    return a % m;
}

uint64_t rand64u() {
    static std::mt19937_64 rng(std::random_device{}());
    return rng();
}

bool is_probable_prime(int64_t n, int k = 20) {
    if (n == 2 || n == 3) {
        return true;
    }
    if (n < 2 || n % 2 == 0 || n % 3 == 0) {
        return false;
    }
    uint64_t s = n - 1, p = n - 1;
    while (!(s & 1)) {
        s >>= 1;
    }
    for (int i = 0; i < k; i++) {
        uint64_t x, r = powmod(rand64u() % (n - 3) + 2, s, n); // random witness base in [2, n - 2]
        for (x = s; x != p && r != 1 && r != p; x <<= 1) {
            r = mulmod(r, r, n);
        }
        if (r != p && !(x & 1)) {
            return false;
        }
    }
    return true;
}

int64_t rho_factor(int64_t n) {
    if (n % 2 == 0) {
        return 2;
    }
    uint64_t y = rand64u() % (n - 1) + 1;
    uint64_t c = rand64u() % (n - 1) + 1;
    uint64_t m = rand64u() % (n - 1) + 1;
    uint64_t g = 1, r = 1, q = 1, ys = 0, x = 0;
    for (r = 1; g == 1; r <<= 1) {
        x = y;
        for (uint64_t i = 0; i < r; i++) {
            y = (mulmod(y, y, n) + c) % n;
        }
        for (uint64_t k = 0; k < r && g == 1; k += m) {

```



```

    ys = y;
    int64_t lim = std::min(m, r - k);
    for (int j = 0; j < lim; j++) {
        y = (mulmod(y, y, n) + c) % n;
        q = mulmod(q, (x > y) ? (x - y) : (y - x), n);
    }
    g = std::gcd(q, n);
}
}
if (g == static_cast<uint64_t>(n)) {
    do {
        ys = (mulmod(ys, ys, n) + c) % n;
        g = std::gcd((x > ys) ? (x - ys) : (ys - x), n);
    } while (g <= 1);
}
return g;
}

bool is_prime(int64_t n) {
    if (n < 2) {
        return false;
    }
    static const int small_primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
    for (int p : small_primes) {
        if (n % p == 0) {
            return n == p;
        }
    }
    if (n < 31 * 31) {
        return true;
    }
    uint64_t t;
    int s = 0;
    for (t = n - 1; !(t & 1); t >>= 1) {
        s++;
    }
    static const uint64_t bases[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    for (uint64_t a : bases) {
        if (a % n == 0) {
            continue;
        }
        uint64_t r = powmod(a, t, n);
        if (r == 1) {
            continue;
        }
        bool ok = false;
        for (int j = 0; j < s && !ok; j++) {
            ok |= (r == static_cast<uint64_t>(n) - 1);
            r = mulmod(r, r, n);
        }
        if (!ok) {
            return false;
        }
    }
    return true;
}

struct SieveCache {
    int limit = 1;
    std::vector<int> least{0, 1}, primes;
};

SieveCache &cached_sieve(int n) {
    static SieveCache cache;
    if (n <= cache.limit) {
        return cache;
    }
    cache.limit = std::max(1, n);

```

```

cache.least.assign(cache.limit + 1, 0);
cache.primes.clear();
for (int i = 2; i <= cache.limit; i++) {
    if (cache.least[i] == 0) {
        cache.least[i] = i;
        cache.primes.push_back(i);
    }
    for (int p : cache.primes) {
        if (p > cache.least[i] || i > cache.limit / p) {
            break;
        }
        cache.least[i * p] = p;
    }
}
return cache;
}

using vfactors = std::vector<std::pair<int64_t, int>>;

vfactors merge_factors(const vfactors &a, const vfactors &b) {
    vfactors res;
    int i = 0, j = 0;
    while (i < static_cast<int>(a.size()) || j < static_cast<int>(b.size())) {
        if (i < static_cast<int>(a.size()) && j < static_cast<int>(b.size()) &&
            a[i].first == b[j].first) {
            res.emplace_back(a[i].first, a[i].second + b[j].second);
            i++;
            j++;
        } else if (
            j == static_cast<int>(b.size()) ||
            (i < static_cast<int>(a.size()) && a[i].first < b[j].first)
        ) {
            res.push_back(a[i++]);
        } else {
            res.push_back(b[j++]);
        }
    }
    return res;
}

vfactors factorize_rho(int64_t n) {
    if (n <= 1) {
        return {};
    }
    if (n % 2 == 0) {
        return merge_factors(vfactors{{2, 1}}, factorize_rho(n / 2));
    }
    if (is_prime(n)) {
        return {{n, 1}};
    }
    int64_t p;
    do {
        p = rho_factor(n);
    } while (p == n);
    return merge_factors(factorize_rho(p), factorize_rho(n / p));
}

vfactors factorize(int64_t n, int small_prime_limit = 1000000) {
    if (n <= 1) {
        return {};
    }
    const SieveCache &sieve = cached_sieve(small_prime_limit);
    if (n <= sieve.limit) {
        vfactors res;
        while (n > 1) {
            int p = sieve.least[n], cnt = 0;
            do {
                n /= p;
            } while (n % p == 0);
            res.push_back({p, cnt});
        }
        return res;
    }
    return merge_factors(factorize_rho(n), factorize_rho(n / p));
}

```

```

        cnt++;
    } while (n > 1 && sieve.least[n] == p);
    res.emplace_back(p, cnt);
}
return res;
}
vfactors res;
const std::vector<int> &primes = sieve.primes;
for (int p : primes) {
    if (p > small_prime_limit || static_cast<int64_t>(p) > n / p) {
        break;
    }
    if (n % p == 0) {
        int cnt = 0;
        do {
            n /= p;
            cnt++;
        } while (n % p == 0);
        res.emplace_back(p, cnt);
    }
}
return merge_factors(res, factorize_rho(n));
}

std::vector<int64_t> divisors_from_factors(const vfactors &factors) {
    std::vector<int64_t> res{1};
    for (const auto &factor : factors) {
        int old_size = static_cast<int>(res.size());
        int64_t power = 1;
        for (int e = 1; e <= factor.second; e++) {
            power *= factor.first;
            for (int i = 0; i < old_size; i++) {
                res.push_back(res[i] * power);
            }
        }
    }
    std::sort(res.begin(), res.end());
    return res;
}

std::vector<int64_t> get_divisors(int64_t n) {
    if (n <= 1) {
        return (n < 1) ? std::vector<int64_t>() : std::vector<int64_t>(1, 1);
    }
    return divisors_from_factors(factorize(n));
}

```

Example Usage

```

#include <cassert>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <set>
using namespace std;

void validate(int64_t n, const vfactors &factors) {
    if (n <= 1) {
        assert(factors.empty());
        return;
    }
    if (is_prime(n)) {
        assert((factors == vfactors{{n, 1}}));
        return;
    }
    int64_t prod = 1;

```

```

for (const auto &factor : factors) {
    assert(is_prime(factor.first));
    for (int i = 0; i < factor.second; i++) {
        prod *= factor.first;
    }
}
assert(prod == n);
}

int main() {
    { // Small primality tests.
        vector<pair<int, bool>> cases{
            {-10, false}, {-1, false}, {0, false}, {1, false}, // non-primes
            {2, true}, {3, true}, {4, false}, {5, true},
            {9, false}, // divisible by 3
            {25, false}, // square of small prime
            {29, true}, // last small-prime table entry
            {31, true}, // just past small-prime table
            {49, false}, // square, not divisible by 2 or 3
            {121, false}, // 11^2
            {169, false}, // 13^2
            {961, false}, // 31^2, boundary for n < 31*31 shortcut
            {997, true}, // prime near 1000
            {1001, false}, // 7*11*13
            {2047, false}, // 23*89, pseudoprime to base 2
            {341, false}, // 11*31, Fermat pseudoprime to base 2
            {561, false}, // Carmichael number
        };
        for (const auto [n, expected] : cases) {
            assert(is_prime_slow(n) == expected);
            assert(is_probable_prime(n) == expected);
            assert(is_prime(n) == expected);
        }
    }
    { // Primality test benchmark.
        vector<int64_t> nums{
            772023803LL, 2147483647LL, 5705234089LL, 6339503641LL, 999966000289LL,
        };
        cout << "Primality test:" << endl;
        cout << setw(15) << left << "num:";
        cout << setw(18) << left << "is_prime_slow():";
        cout << setw(20) << left << "is_probable_prime():";
        cout << setw(13) << right << "is_prime():" << endl;
        cout << fixed << setprecision(3);
        for (int64_t n : nums) {
            clock_t start = clock();
            bool p1 = is_prime_slow(n);
            double t1 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
            start = clock();
            bool p2 = is_probable_prime(n);
            double t2 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
            start = clock();
            bool p3 = is_prime(n);
            double t3 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
            assert(p1 == p2 && p1 == p3);
            cout << setw(12) << left << n << right;
            cout << setw(14) << 1000 * t1 << "ms";
            cout << setw(16) << 1000 * t2 << "ms";
            cout << setw(16) << 1000 * t3 << "ms" << endl;
        }
        cout << endl;
    }
    { // Small factorization tests.
        for (int64_t i = 1; i <= 10000; i++) {
            auto v1 = factorize_slow(i);
            auto v2 = factorize(i);
            validate(i, v1);
            assert(v1 == v2);
        }
    }
}

```

```

    auto d = get_divisors(i);
    set<int> s(d.begin(), d.end());
    assert(d.size() == s.size());
    for (int j = 1; j <= i; j++) {
        if (i % j == 0) {
            assert(s.count(j));
        }
    }
}
}
{ // Compressed factors are convenient for building divisors.
    vfactors factors{{2, 3}, {3, 2}, {5, 1}};
    vector<int64_t> divisors = divisors_from_factors(factors);
    assert(divisors.size() == 24);
    assert(divisors.front() == 1 && divisors.back() == 360);
    assert(factorize(360) == factors);
    assert(get_divisors(360) == divisors);
}
{ // Large factorization tests.
    const vector<int64_t> nums{
        (1LL << 62), // high power of two
        9223372036854775807LL, // 2^63 - 1, many small factors
        999983LL * 999983, // square of a large prime
        1900009LL * 1910009 * 2541547, // three medium-size prime factors
        50000017LL * 50001037, // 16-digit semiprime
    };
    cout << "Factorization test:" << endl;
    cout << setw(24) << left << "num:";
    cout << setw(20) << left << "factorize_slow():";
    cout << "factorize():" << endl;
    cout << fixed << setprecision(3);
    for (int64_t n : nums) {
        clock_t start = clock();
        vfactors factors1 = factorize_slow(n);
        double t1 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
        start = clock();
        vfactors factors2 = factorize(n);
        double t2 = static_cast<double>(clock() - start) / CLOCKS_PER_SEC;
        validate(n, factors1);
        validate(n, factors2);
        assert(factors1 == factors2);
        cout << setw(26) << left << n << right << setw(8) << 1000 * t1 << "ms";
        cout << setw(16) << 1000 * t2 << "ms" << endl;
        cout << left;
    }
}
return 0;
}

```

Example Output

```

Primality test:
num:          is_prime_slow():  is_probable_prime():  is_prime():
772023803      0.006ms           0.005ms           0.001ms
2147483647      0.009ms           0.006ms           0.002ms
5705234089      0.014ms           0.000ms           0.001ms
6339503641      0.013ms           0.001ms           0.001ms
999966000289    0.171ms            0.001ms           0.001ms

Factorization test:
num:          factorize_slow():  factorize():
4611686018427387904    0.002ms           0.002ms
9223372036854775807    0.049ms           0.007ms
999966000289           0.503ms           0.043ms
9223361212852495307    0.952ms           0.096ms
2500052700017629       24.587ms          0.194ms

```

6.3.9 Euler's Totient Function

Euler's totient function $\phi(n)$ returns the number of positive integers less than or equal to n that are relatively prime to n ; that is, the number of integers k in the range $[1, n]$ for which $\gcd(n, k) = 1$.

The function is multiplicative, with $\phi(p^k) = p^k - p^{k-1}$ for a prime power, which yields the product formula $\phi(n) = n \prod_{p|n} (1 - 1/p)$ taken over the distinct primes p dividing n . `phi(n)` applies this formula directly, factoring `n` by trial division. `phi_table(n)` computes the whole range at once with a sieve of Eratosthenes: starting from `v[i] = i`, it sweeps each prime p across its multiples `j` and folds in the factor $(1 - 1/p)$ by replacing `v[j]` with `v[j] - v[j] / p`. This is faster than the alternative that uses the divisor-sum identity $\sum_{d|n} \phi(d) = n$ to subtract each `v[i]` from its multiples, which costs $O(n \log n)$.

The overload `phi_table(lo, hi)` computes the totients of a high but narrow range with a segmented sieve, analogous to the segmented prime sieve. It sieves the primes up to $\sqrt{hi}$, applies each one's $(1 - 1/p)$ factor to its multiples within $[lo, hi]$ while dividing that prime out of a parallel copy of every value, and finally folds in any single leftover prime factor greater than $\sqrt{hi}$ (a number at most `hi` can have at most one such factor).

- `phi(n)` returns Euler's totient of `n`.
- `phi_table(n)` returns a vector `v` of length `n + 1` such that `v[i]` stores `phi(i)` for every `i` in the range $[0, n]$.
- `phi_table(lo, hi)` returns a vector `v` of length `hi - lo + 1` such that `v[i - lo]` stores `phi(i)` for every `i` in the range $[lo, hi]$. This overload assumes `lo` ≥ 0 ; if `hi` $< lo$, it returns an empty vector.

Time Complexity:

- $O(\sqrt{n})$ per call to `phi(n)`.
- $O(n \log \log n)$ per call to `phi_table(n)`.
- $O((hi - lo) \cdot \log(\log(hi)) + \sqrt{hi})$ per call to `phi_table(lo, hi)`.

Space Complexity:

- $O(1)$ auxiliary space for `phi(n)`.
- $O(n)$ auxiliary heap space for `phi_table(n)`.
- $O(hi - lo + \sqrt{hi})$ auxiliary heap space for `phi_table(lo, hi)`.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <numeric>
#include <vector>

int phi(int n) {
    int res = n;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            while (n % i == 0) {
                n /= i;
            }
            res -= res / i;
        }
    }
    if (n > 1) {
        res -= res / n;
    }
    return res;
}

std::vector<int> phi_table(int n) {
    std::vector<int> res(n + 1);
    std::iota(res.begin(), res.end(), 0);
    for (int i = 2; i <= n; i++) {
        if (res[i] == i) { // i is prime, so apply its (1 - 1/i) factor to every multiple.
            for (int j = i; j <= n; j += i) {
                res[j] -= res[j] / i;
            }
        }
    }
}
```

```

    }
}
return res;
}

std::vector<int> phi_table(int lo, int hi) {
    if (hi < lo) {
        return {};
    }
    int root = static_cast<int>(sqrt(hi));
    while (1LL * (root + 1) * (root + 1) <= hi) {
        root++;
    }
    // Sieve the primes up to sqrt(hi); these are the only ones that can divide a value in [lo, hi].
    std::vector<bool> composite(root + 1, false);
    std::vector<int> primes;
    for (int i = 2; i <= root; i++) {
        if (!composite[i]) {
            primes.push_back(i);
            for (int64_t j = 1LL * i * i; j <= root; j += i) {
                composite[j] = true;
            }
        }
    }
    std::vector<int> res(hi - lo + 1);
    std::vector<int64_t> remaining(hi - lo + 1);
    for (int i = lo; i <= hi; i++) {
        res[i - lo] = i;
        remaining[i - lo] = i;
    }
    for (int p : primes) {
        // Start at the first multiple of p that is >= lo, but never below p itself (this skips 0).
        int64_t start = std::max(1LL * p, ((1LL * lo + p - 1) / p) * p);
        for (int64_t j = start; j <= hi; j += p) {
            int idx = static_cast<int>(j - lo);
            res[idx] -= res[idx] / p;
            while (remaining[idx] % p == 0) {
                remaining[idx] /= p;
            }
        }
    }
    for (int i = lo; i <= hi; i++) {
        int idx = i - lo;
        if (remaining[idx] > 1) { // a single prime factor greater than sqrt(hi) is left over
            res[idx] -= static_cast<int>(res[idx] / remaining[idx]);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    assert(phi(1) == 1);
    assert(phi(9) == 6);
    assert(phi(1234567) == 1224720);
    const int n = 1000;
    vector<int> pt = phi_table(n);
    for (int i = 0; i <= n; i++) {
        assert(pt[i] == phi(i));
    }
    // The segmented overload agrees with phi(), including over a low range (with 0 and 1) and a
    // high, narrow window where many values have a prime factor exceeding sqrt(hi).
}

```

```

vector<int> lphi = phi_table(0, 50);
for (int i = 0; i <= 50; i++) {
    assert(lphi[i] == phi(i));
}
int lo = 1000000000, hi = 1000000100;
vector<int> hphi = phi_table(lo, hi);
for (int i = lo; i <= hi; i++) {
    assert(hphi[i - lo] == phi(i));
}
return 0;
}

```

6.3.10 Mobius Function and Inclusion-Exclusion

The Mobius function $\mu(n)$ is the multiplicative function defined as 1 if $n = 1$, 0 if n is divisible by the square of a prime, and $(-1)^k$ if n is a product of k distinct primes. It is the coefficient that appears in Mobius inversion: if $g(n) = \sum_{d|n} f(d)$ for all n , then $f(n) = \sum_{d|n} \mu(d) g(n/d)$. This formalizes the principle of inclusion-exclusion in the divisor lattice, letting one recover a summand function from its divisor sums.

Inclusion-exclusion itself counts the size of a union by alternately adding and subtracting the sizes of intersections. A canonical number-theoretic instance is counting integers in $[1, n]$ that are coprime to a modulus m : subtract the multiples of each prime factor of m , add back those counted twice, and so on. The signed terms are precisely the values of `mu` over the squarefree divisors of m , so `count_coprime(n, m)` equals $\sum_{d|m} \mu(d) \lfloor n/d \rfloor$.

- `mobius(n)` returns $\mu(n)$ for a single `n` by trial division.
- `mobius_sieve(n)` returns a vector of length $n + 1$ where index i holds $\mu(i)$, computed together with a linear sieve.
- `count_coprime(n, m)` returns the number of integers in $[1, n]$ that are coprime to m , via inclusion-exclusion over the distinct prime factors of `m`.

Time Complexity:

- $O(\sqrt{n})$ per call to `mobius(n)`.
- $O(n)$ per call to `mobius_sieve(n)`.
- $O(\sqrt{m} + 2^k \cdot k)$ per call to `count_coprime(n, m)`, where k is the number of distinct prime factors of m (at most 15 for m within 64-bit range).

Space Complexity:

- $O(n)$ auxiliary heap space for `mobius_sieve(n)`.
- $O(k)$ auxiliary heap space for `count_coprime(n, m)`.
- $O(1)$ auxiliary for `mobius(n)`.

```

#include <stdint>
#include <vector>

int mobius(int n) {
    if (n == 1) {
        return 1;
    }
    int res = 1;
    for (int p = 2; p <= n / p; p++) {
        if (n % p == 0) {
            n /= p;
            if (n % p == 0) {
                return 0; // p^2 divides the original n.
            }
            res = -res;
        }
    }
    if (n > 1) {

```



```

    res = -res; // One remaining prime factor larger than its square root.
}
return res;
}

std::vector<int> mobius_sieve(int n) {
    std::vector<int> mu(n + 1, 0), primes;
    std::vector<bool> composite(n + 1, false);
    if (n >= 1) {
        mu[1] = 1;
    }
    for (int i = 2; i <= n; i++) {
        if (!composite[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if (p > n / i) {
                break;
            }
            composite[i * p] = true;
            if (i % p == 0) {
                mu[i * p] = 0; // p^2 divides i * p.
                break;
            }
            mu[i * p] = -mu[i];
        }
    }
    return mu;
}

int64_t count_coprime(int64_t n, int64_t m) {
    std::vector<int64_t> primes;
    for (int64_t p = 2; p <= m / p; p++) {
        if (m % p == 0) {
            primes.push_back(p);
            while (m % p == 0) {
                m /= p;
            }
        }
    }
    if (m > 1) {
        primes.push_back(m);
    }
    int k = static_cast<int>(primes.size());
    int64_t total = 0;
    for (int mask = 0; mask < (1 << k); mask++) {
        int64_t product = 1;
        int bits = 0;
        for (int i = 0; i < k; i++) {
            if ((mask >> i) & 1) {
                product *= primes[i];
                bits++;
            }
        }
        total += (bits & 1 ? -1 : 1) * (n / product);
    }
    return total;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {

```

```

assert(mobius(1) == 1);
assert(mobius(2) == -1);
assert(mobius(6) == 1);    // 2 * 3, two distinct primes.
assert(mobius(12) == 0);   // Divisible by 2^2.
assert(mobius(30) == -1);  // 2 * 3 * 5, three distinct primes.

vector<int> mu = mobius_sieve(12);
assert((mu == vector<int>{0, 1, -1, -1, 0, -1, 1, -1, 0, 0, 1, -1, 0}));

assert(count_coprime(10, 6) == 3); // 1, 5, 7
assert(count_coprime(100, 1) == 100);
return 0;
}

```

6.4 Arbitrary Precision Arithmetic

6.4.1 Big Integers (Simple)

Perform simple arithmetic operations on arbitrary precision big integers whose digits are internally represented as an `std::string` in little-endian order.

This version is intentionally small and decimal-oriented. Addition and subtraction are the usual schoolbook digit scans with carry or borrow. Multiplication keeps a shifted copy of the left operand for each digit of the right operand, adds that row once per digit value, then shifts by one decimal place. Division is long division: it scans the dividend from most significant digit to least, maintains a running remainder, and repeatedly subtracts the divisor to discover each quotient digit.

- `BigInt(n)` constructs a big integer from an integer `n` (default: 0).
- `BigInt(s)` constructs a big integer from a string `s`, which must strictly consist of a sequence of numeric digits, optionally preceded by a minus sign.
- `to_string()` returns the string representation of the big integer.
- `comp(a, b)` returns `-1`, `0`, or `1` depending on whether the big integers `a` and `b` compare less, equal, or greater, respectively.
- `add(a, b)` returns the sum of big integers `a` and `b`.
- `sub(a, b)` returns the difference of big integers `a` and `b`.
- `mul(a, b)` returns the product of big integers `a` and `b`.
- `div(a, b)` returns the quotient of big integers `a` and `b`.

Time Complexity:

- $O(n)$ per call to the constructor, `to_string()`, `comp()`, `add()`, and `sub()`, where n is the total number of digits in the arguments and result for each operation.
- $O(n \cdot m)$ per call to `mul(a, b)` and `div(a, b)` where n is the number of digits in `a` and m is the number of digits in `b`.

Space Complexity:

- $O(n)$ for storage of the big integer, where n is the number of digits.
- $O(n)$ auxiliary heap space for `to_string()`, `add()`, `sub()`, `mul()`, and `div()`, where n is the total number of digits in the arguments and result for each operation.

```

#include <algorithm>
#include <cctype>
#include <cstdint>
#include <stdexcept>
#include <string>

class BigInt {

```

```

std::string digits;
int sign;

void normalize() {
    std::size_t pos = digits.find_last_not_of('0');
    if (pos != std::string::npos) {
        digits.erase(pos + 1);
    } else {
        digits = "0"; // An all-zero (or empty) magnitude collapses to a single "0".
    }
    if (digits.size() == 1 && digits[0] == '0') {
        sign = 1;
    }
}

static int comp(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign != bsign) {
        return asign < bsign ? -1 : 1;
    }
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -asign : asign;
    }
    for (int i = static_cast<int>(a.size()) - 1; i >= 0; i--) {
        if (a[i] != b[i]) {
            return a[i] < b[i] ? -asign : asign;
        }
    }
    return 0;
}

static BigInt add(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign != bsign) {
        return (asign == 1) ? sub(a, b, asign, 1) : sub(b, a, bsign, 1);
    }
    BigInt res;
    res.sign = asign;
    res.digits.resize(std::max(a.size(), b.size()) + 1, '0');
    for (int i = 0, carry = 0; i < static_cast<int>(res.digits.size()); i++) {
        int d = carry;
        if (i < static_cast<int>(a.size())) {
            d += a[i] - '0';
        }
        if (i < static_cast<int>(b.size())) {
            d += b[i] - '0';
        }
        res.digits[i] = '0' + (d % 10);
        carry = d / 10;
    }
    res.normalize();
    return res;
}

static BigInt sub(const std::string &a, const std::string &b, int asign, int bsign) {
    if (asign == -1 || bsign == -1) {
        return add(a, b, asign, -bsign);
    }
    BigInt res;
    if (comp(a, b, asign, bsign) < 0) {
        res = sub(b, a, bsign, asign);
        res.sign = -1;
        return res;
    }
    res.digits.assign(a.size(), '0');
    for (int i = 0, borrow = 0; i < static_cast<int>(res.digits.size()); i++) {
        int d = (i < static_cast<int>(b.size())) ? a[i] - b[i] : a[i] - '0' - borrow;
        if (a[i] > '0') {
            borrow = 0;
        }
    }
}

```

```

    if (d < 0) {
        d += 10;
        borrow = 1;
    }
    res.digits[i] = '0' + (d % 10);
}
res.normalize();
return res;
}

public:
BigInt(int64_t n = 0) {
    sign = (n < 0) ? -1 : 1;
    if (n == 0) {
        digits = "0";
        return;
    }
    for (n = (n > 0) ? n : -n; n > 0; n /= 10) {
        digits += '0' + (n % 10);
    }
    normalize();
}

BigInt(const std::string &s) {
    if (s.empty() || (s[0] == '-' && s.size() == 1)) {
        throw std::runtime_error("Invalid string format to construct BigInt.");
    }
    digits.assign(s.rbegin(), s.rend());
    if (s[0] == '-') {
        sign = -1;
        digits.erase(digits.size() - 1);
    } else {
        sign = 1;
    }
    if (digits.find_first_not_of("0123456789") != std::string::npos) {
        throw std::runtime_error("Invalid string format to construct BigInt.");
    }
    normalize();
}

std::string to_string() const {
    return (sign < 0 ? "-" : "") + std::string(digits.rbegin(), digits.rend());
}

friend int comp(const BigInt &a, const BigInt &b) {
    return comp(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt add(const BigInt &a, const BigInt &b) {
    return add(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt sub(const BigInt &a, const BigInt &b) {
    return sub(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt mul(const BigInt &a, const BigInt &b) {
    BigInt res, row(a);
    for (char dc : b.digits) {
        for (int j = 0; j < (dc - '0'); j++) {
            res = add(res.digits, row.digits, res.sign, row.sign);
        }
        if (row.digits.size() > 1 || row.digits[0] != '0') {
            row.digits.insert(0, "0");
        }
    }
    res.sign = a.sign * b.sign;
    res.normalize();
}

```

```

    return res;
}

friend BigInt div(const BigInt &a, const BigInt &b) {
    if (b.digits.size() == 1 && b.digits[0] == '0') {
        throw std::runtime_error("Division by zero in BigInt.");
    }
    BigInt res, row;
    res.digits.assign(a.digits.size(), '0');
    row.digits.clear(); // Running remainder; kept normalized and nonnegative below.
    for (int i = static_cast<int>(a.digits.size()) - 1; i >= 0; i--) {
        row.digits.insert(row.digits.begin(), a.digits[i]);
        row.normalize(); // Strip the spurious high-order zero left by the previous remainder.
        // Subtract while the remainder is >= b. A strict ">" undercounts the quotient digit when the
        // remainder equals b, and leaves the remainder too large, overflowing the next digit past 9.
        while (comp(row.digits, b.digits, 1, 1) >= 0) {
            res.digits[i]++;
            row = sub(row.digits, b.digits, 1, 1);
        }
    }
    res.sign = a.sign * b.sign;
    res.normalize();
    return res;
}
};

```

Example Usage

```

#include <cassert>

int main() {
    BigInt a("-9899819294989142124"), b("12398124981294214");
    assert(add(a, b).to_string() == "-9887421170007847910");
    assert(sub(a, b).to_string() == "-9912217419970436338");
    assert(mul(a, b).to_string() == "-122739196911503356525379735104870536");
    assert(div(a, b).to_string() == "-798");
    assert(comp(a, b) == -1 && comp(a, a) == 0 && comp(b, a) == 1);
    return 0;
}

```

6.4.2 Big Integers

Perform operations on arbitrary precision big integers internally represented as a vector of base-10⁹ digits in little-endian order. Typical arithmetic operations involving mixed numeric primitives and strings are supported through implicit construction and hidden friend operators, as long as at least one operand is a `BigInt` at any given level of evaluation.

- `BigInt(n)` constructs a big integer from an integer `n` (default: 0).
- `BigInt(s)` constructs a big integer from a C string or an `std::string s`.
- `operator=` is defined to copy from another big integer or to assign from a 64-bit integer primitive.
- `size()` returns the number of digits in the base-10 representation.
- Operators `>>` and `<<` are defined to support stream-based input and output.
- `v.to_string()`, `v.to_llong()`, `v.to_double()`, and `v.to_ldouble()` return the big integer `v` converted to an `std::string`, `int64_t`, `double`, and `long double` respectively. For the latter three data types, overflow behavior is based on that of inputting from `std::istream`.
- `v.abs()` returns the absolute value of big integer `v`.
- `a.comp(b)` returns `-1`, `0`, or `1` depending on whether the big integers `a` and `b` compare less, equal, or greater, respectively.

- Operators `<`, `>`, `<=`, `>=`, `==`, `!=`, `+`, `-`, `*`, `/`, `%`, `++`, `--`, `+=`, `-=`, `*=`, `/=`, and `%=` are defined analogous to those on integer primitives. Addition, subtraction, and comparisons are performed using the standard linear algorithms. Multiplication first converts limbs from base 10^9 to base 10^4 so coefficient products fit safely, then uses either Karatsuba multiplication (if `USE_FFT_MULT` is false) or complex FFT convolution (if `USE_FFT_MULT` is true), converting the result back to base 10^9 . Division and modulo are computed together by normalized long division: scale the operands so the divisor's leading limb is large, estimate each quotient limb from the top one or two limbs of the running remainder, correct the estimate by adding the divisor back if necessary, then unscale the remainder.
- `a.div(b)` returns a pair consisting of the quotient and remainder.
- `v.pow(n)` returns `v` raised to the power of `n`.
- `v.sqrt()` returns the integral part of the square root of big integer `v`.
- `v.nth_root(n)` returns the integral part of the n -th root of big integer `v`.
- `rand(n)` returns a random, positive big integer with n digits.

`pow(n)` uses binary exponentiation. `sqrt()` uses a digit-by-digit square-root algorithm in the internal base, while `nth_root(n)` uses binary search over the answer range.

Time Complexity:

- $O(n)$ per call to the constructors, `size()`, `to_string()`, `to_llong()`, `to_double()`, `to_ldouble()`, `abs()`, `comp()`, `rand()`, and all comparison and arithmetic operators except multiplication, division, and modulo, where n is total number of digits in the arguments and result for each operation.
- $O(n \cdot \log(n) \cdot \log(\log(n)))$ or $O(n^{1.585})$ per call to multiplication operations, depending on whether `USE_FFT_MULT` is set to true or false.
- $O(n \cdot m)$ per call to division and modulo operations, where n and m are the number of digits in the dividend and divisor, respectively.
- $O(M(d) \log n)$ per call to `pow(n)`, where d is the maximum digit length reached during exponentiation and $M(d)$ denotes the cost of one multiplication of two d -digit big integers using the selected multiplication method above.

Space Complexity:

- $O(n)$ for storage of the big integer.
- $O(n)$ auxiliary heap space for negation, addition, subtraction, multiplication, division, `abs()`, `sqrt()`, `pow()`, and `nth_root()`.
- $O(1)$ auxiliary space for all other operations.

```
#include <algorithm>
#include <cmath>
#include <complex>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <iomanip>
#include <istream>
#include <ostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

class BigInt {
    static const int BASE = 1000000000, BASE_DIGITS = 9;
    static const bool USE_FFT_MULT = true;

    using vint = std::vector<int>;
    using vint64 = std::vector<int64_t>;
    using vcd = std::vector<std::complex<double>>;

    vint digits;
    int sign;
```

```

void normalize() {
    while (!digits.empty() && digits.back() == 0) {
        digits.pop_back();
    }
    if (digits.empty()) {
        sign = 1;
    }
}

void read(int n, const char *s) {
    sign = 1;
    digits.clear();
    int pos = 0;
    while (pos < n && (s[pos] == '-' || s[pos] == '+')) {
        if (s[pos] == '-') {
            sign = -sign;
        }
        pos++;
    }
    for (int i = n - 1; i >= pos; i -= BASE_DIGITS) {
        int x = 0;
        for (int j = std::max(pos, i - BASE_DIGITS + 1); j <= i; j++) {
            x = x * 10 + s[j] - '0';
        }
        digits.push_back(x);
    }
    normalize();
}

static int comp(const vint &a, const vint &b, int asign, int bsign) {
    if (asign != bsign) {
        return asign < bsign ? -1 : 1;
    }
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -asign : asign;
    }
    for (int i = static_cast<int>(a.size()) - 1; i >= 0; i--) {
        if (a[i] != b[i]) {
            return a[i] < b[i] ? -asign : asign;
        }
    }
    return 0;
}

static BigInt add(const vint &a, const vint &b, int asign, int bsign) {
    if (asign != bsign) {
        return (asign == 1) ? sub(a, b, asign, 1) : sub(b, a, bsign, 1);
    }
    BigInt res;
    res.digits = a;
    res.sign = asign;
    int carry = 0, size = static_cast<int>(std::max(a.size(), b.size()));
    for (int i = 0; i < size || carry; i++) {
        if (i == static_cast<int>(res.digits.size())) {
            res.digits.push_back(0);
        }
        res.digits[i] += carry + (i < static_cast<int>(b.size()) ? b[i] : 0);
        carry = (res.digits[i] >= BASE) ? 1 : 0;
        if (carry) {
            res.digits[i] -= BASE;
        }
    }
    return res;
}

static BigInt sub(const vint &a, const vint &b, int asign, int bsign) {
    if (asign == -1 || bsign == -1) {

```

```

    return add(a, b, asign, -bsign);
}
BigInt res;
if (comp(a, b, asign, bsign) < 0) {
    res = sub(b, a, bsign, asign);
    res.sign = -1;
    return res;
}
res.digits = a;
res.sign = asign;
for (int i = 0, borrow = 0; i < static_cast<int>(a.size()) || borrow; i++) {
    res.digits[i] -= borrow + (i < static_cast<int>(b.size()) ? b[i] : 0);
    borrow = res.digits[i] < 0;
    if (borrow) {
        res.digits[i] += BASE;
    }
}
res.normalize();
return res;
}

static vint convert_base(const vint &digits, int l1, int l2) {
    vint64 p(std::max(l1, l2) + 1);
    p[0] = 1;
    for (int i = 1; i < static_cast<int>(p.size()); i++) {
        p[i] = p[i - 1] * 10;
    }
    vint res;
    int64_t curr = 0;
    for (int i = 0, curr_digits = 0; i < static_cast<int>(digits.size()); i++) {
        curr += digits[i] * p[curr_digits];
        curr_digits += l1;
        while (curr_digits >= l2) {
            res.push_back(static_cast<int>((curr % p[l2])));
            curr /= p[l2];
            curr_digits -= l2;
        }
    }
    res.push_back(static_cast<int>(curr));
    while (!res.empty() && res.back() == 0) {
        res.pop_back();
    }
    return res;
}

template<typename It>
static vint64 karatsuba(It alo, It ahi, It blo, It bhi) {
    int n = std::distance(alo, ahi), k = n / 2;
    vint64 res(n * 2);
    if (n <= 32) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                res[i + j] += alo[i] * blo[j];
            }
        }
        return res;
    }
    auto a1b1 = karatsuba(alo, alo + k, blo, blo + k);
    auto a2b2 = karatsuba(alo + k, ahi, blo + k, bhi);
    vint64 a2(alo + k, ahi), b2(blo + k, bhi);
    for (int i = 0; i < k; i++) {
        a2[i] += alo[i];
        b2[i] += blo[i];
    }
    auto r = karatsuba(a2.begin(), a2.end(), b2.begin(), b2.end());
    for (int i = 0; i < static_cast<int>(a1b1.size()); i++) {
        r[i] -= a1b1[i];
        res[i] += a1b1[i];
    }
}

```



```

    }
    for (int i = 0; i < static_cast<int>(a2b2.size()); i++) {
        r[i] -= a2b2[i];
        res[i + n] += a2b2[i];
    }
    for (int i = 0; i < static_cast<int>(r.size()); i++) {
        res[i + k] += r[i];
    }
    return res;
}

template<typename It>
static vcd fft(It lo, It hi, bool invert = false) {
    int n = std::distance(lo, hi), k = 0, high1 = -1;
    while ((1 << k) < n) {
        k++;
    }
    std::vector<int> rev(n, 0);
    for (int i = 1; i < n; i++) {
        if (!(i & (i - 1))) {
            high1++;
        }
        rev[i] = rev[i ^ (1 << high1)];
        rev[i] |= (1 << (k - high1 - 1));
    }
    vcd roots(n), res(n);
    for (int i = 0; i < n; i++) {
        double alpha = 2 * 3.14159265358979323846 * i / n;
        roots[i] = std::complex<double>(cos(alpha), sin(alpha));
        res[i] = *(lo + rev[i]);
    }
    for (int len = 1; len < n; len <= 1) {
        vcd tmp(n);
        int rstep = roots.size() / (len <= 1);
        for (int pdest = 0; pdest < n; pdest += len) {
            int p = pdest;
            for (int i = 0; i < len; i++) {
                std::complex<double> c = roots[i * rstep] * res[p + len];
                tmp[pdest] = res[p] + c;
                tmp[pdest + len] = res[p] - c;
                pdest++;
                p++;
            }
        }
        res.swap(tmp);
    }
    if (invert) {
        for (int i = 0; i < static_cast<int>(res.size()); i++) {
            res[i] /= n;
        }
        std::reverse(res.begin() + 1, res.end());
    }
    return res;
}

public:
    BigInt() : sign(1) {}
    BigInt(int v) { *this = static_cast<int64_t>(v); }
    BigInt(int64_t v) { *this = v; }
    BigInt(const char *s) { read(strlen(s), s); }
    BigInt(const std::string &s) { read(s.size(), s.c_str()); }

    BigInt &operator=(const BigInt &v) {
        sign = v.sign;
        digits = v.digits;
        return *this;
    }
}

```

```

BigInt &operator=(int64_t v) {
    sign = 1;
    if (v < 0) {
        sign = -1;
        v = -v;
    }
    digits.clear();
    for (; v > 0; v /= BASE) {
        digits.push_back(v % BASE);
    }
    return *this;
}

int size() const {
    if (digits.empty()) {
        return 1;
    }
    std::ostringstream oss;
    oss << digits.back();
    return oss.str().length() + BASE_DIGITS * (digits.size() - 1);
}

friend std::istream &operator>>(std::istream &in, BigInt &v) {
    std::string s;
    in >> s;
    v.read(s.size(), s.c_str());
    return in;
}

friend std::ostream &operator<<(std::ostream &out, const BigInt &v) {
    if (v.sign == -1) {
        out << '-';
    }
    out << (v.digits.empty() ? 0 : v.digits.back());
    for (int i = static_cast<int>(v.digits.size()) - 2; i >= 0; i--) {
        out << std::setw(BASE_DIGITS) << std::setfill('0') << v.digits[i];
    }
    return out;
}

std::string to_string() const {
    std::ostringstream oss;
    if (sign == -1) {
        oss << '-';
    }
    oss << (digits.empty() ? 0 : digits.back());
    for (int i = static_cast<int>(digits.size()) - 2; i >= 0; i--) {
        oss << std::setw(BASE_DIGITS) << std::setfill('0') << digits[i];
    }
    return oss.str();
}

int64_t to_llong() const {
    int64_t res = 0;
    for (int i = static_cast<int>(digits.size()) - 1; i >= 0; i--) {
        res = res * BASE + digits[i];
    }
    return res * sign;
}

double to_double() const {
    std::stringstream ss(to_string());
    double res;
    ss >> res;
    return res;
}

long double to_ldouble() const {

```

```

    std::stringstream ss(to_string());
    long double res;
    ss >> res;
    return res;
}

int comp(const BigInt &v) const { return comp(digits, v.digits, sign, v.sign); }

// The comparison and binary arithmetic operators are hidden friends, so a raw integer operand on
// either side converts through the implicit constructor.
friend bool operator<(const BigInt &a, const BigInt &b) { return a.comp(b) < 0; }
friend bool operator>(const BigInt &a, const BigInt &b) { return a.comp(b) > 0; }
friend bool operator<=(const BigInt &a, const BigInt &b) { return a.comp(b) <= 0; }
friend bool operator>=(const BigInt &a, const BigInt &b) { return a.comp(b) >= 0; }
friend bool operator==(const BigInt &a, const BigInt &b) { return a.comp(b) == 0; }
friend bool operator!=(const BigInt &a, const BigInt &b) { return a.comp(b) != 0; }

BigInt abs() const { BigInt res(*this); res.sign = 1; return res; }
BigInt operator-() const { BigInt res(*this); res.sign = -sign; return res; }

friend BigInt operator+(const BigInt &a, const BigInt &b) {
    return add(a.digits, b.digits, a.sign, b.sign);
}

friend BigInt operator-(const BigInt &a, const BigInt &b) {
    return sub(a.digits, b.digits, a.sign, b.sign);
}

void operator*=(int v) {
    if (v < 0) {
        sign = -sign;
        v = -v;
    }
    for (int i = 0, carry = 0; i < static_cast<int>(digits.size()) || carry; i++) {
        if (i == static_cast<int>(digits.size())) {
            digits.push_back(0);
        }
        int64_t curr = digits[i] * static_cast<int64_t>(v) + carry;
        carry = static_cast<int>((curr / BASE));
        digits[i] = static_cast<int>((curr % BASE));
    }
    normalize();
}

BigInt operator*(int v) const {
    BigInt res(*this);
    res *= v;
    return res;
}

friend BigInt operator*(const BigInt &u, const BigInt &v) {
    static const int TEMP_BASE = 10000, TEMP_BASE_DIGITS = 4;
    vint a = convert_base(u.digits, BASE_DIGITS, TEMP_BASE_DIGITS);
    vint b = convert_base(v.digits, BASE_DIGITS, TEMP_BASE_DIGITS);
    int n = 1;
    while (n < 2 * static_cast<int>(std::max(a.size(), b.size()))) {
        n <<= 1;
    }
    a.resize(n, 0);
    b.resize(n, 0);
    vint64 c;
    if (USE_FFT_MULT) {
        auto at = fft(a.begin(), a.end()), bt = fft(b.begin(), b.end());
        for (int i = 0; i < n; i++) {
            at[i] *= bt[i];
        }
        at = fft(at.begin(), at.end(), true);
        c.resize(n);
    }

```

```

        for (int i = 0; i < n; i++) {
            c[i] = at[i].real() + 0.5;
        }
    } else {
        c = karatsuba(a.begin(), a.end(), b.begin(), b.end());
    }
    BigInt res;
    res.sign = u.sign * v.sign;
    for (int i = 0, carry = 0; i < static_cast<int>(c.size()); i++) {
        int64_t d = c[i] + carry;
        res.digits.push_back(d % TEMP_BASE);
        carry = d / TEMP_BASE;
    }
    res.digits = convert_base(res.digits, TEMP_BASE_DIGITS, BASE_DIGITS);
    res.normalize();
    return res;
}

BigInt &operator/=(int v) {
    if (v == 0) {
        throw std::runtime_error("Division by zero in BigInt.");
    }
    if (v < 0) {
        sign = -sign;
        v = -v;
    }
    for (int i = static_cast<int>(digits.size()) - 1, rem = 0; i >= 0; i--) {
        int64_t curr = digits[i] + rem * static_cast<int64_t>(BASE);
        digits[i] = static_cast<int>((curr / v));
        rem = static_cast<int>((curr % v));
    }
    normalize();
    return *this;
}

BigInt operator/(int v) const {
    BigInt res(*this);
    res /= v;
    return res;
}

int operator%(int v) const {
    if (v == 0) {
        throw std::runtime_error("Division by zero in BigInt.");
    }
    if (v < 0) {
        v = -v;
    }
    int m = 0;
    for (int i = static_cast<int>(digits.size()) - 1; i >= 0; i--) {
        m = (digits[i] + m * static_cast<int64_t>(BASE)) % v;
    }
    return m * sign;
}

std::pair<BigInt, BigInt> div(const BigInt &v) const {
    if (v == 0) {
        throw std::runtime_error("Division by zero in BigInt.");
    }
    if (comp(digits, v.digits, 1, 1) < 0) {
        return {BigInt(0), *this};
    }
    int norm = BASE / (v.digits.back() + 1);
    BigInt an = abs() * norm, bn = v.abs() * norm, q, r;
    q.digits.resize(an.digits.size());
    for (int i = static_cast<int>(an.digits.size()) - 1; i >= 0; i--) {
        r *= BASE;
        r += an.digits[i];
    }

```

```

    int s1 = (r.digits.size() <= bn.digits.size()) ? 0 : r.digits[bn.digits.size()];
    int s2 = (r.digits.size() <= bn.digits.size() - 1) ? 0 : r.digits[bn.digits.size() - 1];
    int d = (static_cast<int64_t>(s1) * BASE + s2) / bn.digits.back();
    for (r -= bn * d; r < 0; r += bn) {
        d--;
    }
    q.digits[i] = d;
}
q.sign = sign * v.sign;
r.sign = sign;
q.normalize();
r.normalize();
return {q, r / norm};
}

friend BigInt operator/(const BigInt &a, const BigInt &b) { return a.div(b).first; }
friend BigInt operator%(const BigInt &a, const BigInt &b) { return a.div(b).second; }
BigInt operator++(int){ BigInt t(*this); operator++(); return t; }
BigInt operator--(int){ BigInt t(*this); operator--(); return t; }
BigInt &operator++() { *this = *this + BigInt(1); return *this; }
BigInt &operator--() { *this = *this - BigInt(1); return *this; }
BigInt &operator+=(const BigInt &v) { *this = *this + v; return *this; }
BigInt &operator-=(const BigInt &v) { *this = *this - v; return *this; }
BigInt &operator*=(const BigInt &v) { *this = *this * v; return *this; }
BigInt &operator/=(const BigInt &v) { *this = *this / v; return *this; }
BigInt &operator%=(const BigInt &v) { *this = *this % v; return *this; }

BigInt pow(int n) const {
    if (n == 0) {
        return BigInt(1);
    }
    if (*this == 0 || n < 0) {
        return BigInt(0);
    }
    BigInt x(*this), res(1);
    for (; n != 0; n >>= 1) {
        if (n & 1) {
            res *= x;
        }
        x *= x;
    }
    return res;
}

BigInt sqrt() const {
    if (sign == -1) {
        throw std::runtime_error("Cannot take square root of a negative number.");
    }
    BigInt v(*this);
    while (v.digits.empty() || v.digits.size() % 2 == 1) {
        v.digits.push_back(0);
    }
    int n = static_cast<int>(v.digits.size());
    int ldig =
        static_cast<int> (::sqrt(static_cast<double>(v.digits[n - 1]) * BASE + v.digits[n - 2]));
    int norm = BASE / (ldig + 1);
    v *= norm;
    v *= norm;
    while (v.digits.empty() || v.digits.size() % 2 == 1) {
        v.digits.push_back(0);
    }
    BigInt r(static_cast<int64_t>(v.digits[n - 1]) * BASE + v.digits[n - 2]);
    int q = ldig =
        static_cast<int> (::sqrt(static_cast<double>(v.digits[n - 1]) * BASE + v.digits[n - 2]));
    BigInt res;
    for (int j = n / 2 - 1; j >= 0; j--) {
        for (; q-- > 0) {
            BigInt r1 =

```

```

        (r - (res * 2 * BASE + q) * q) * BASE * BASE +
        (j > 0 ? static_cast<int64_t>(v.digits[2 * j - 1]) * BASE + v.digits[2 * j - 2] : 0);
    if (r1 >= 0) {
        r = r1;
        break;
    }
}
res = res * BASE + q;
if (j > 0) {
    int sz1 = static_cast<int>(res.digits.size());
    int sz2 = static_cast<int>(r.digits.size());
    int d1 = (sz1 + 2 < sz2) ? r.digits[sz1 + 2] : 0;
    int d2 = (sz1 + 1 < sz2) ? r.digits[sz1 + 1] : 0;
    int d3 = (sz1 < sz2) ? r.digits[sz1] : 0;
    q = (static_cast<int64_t>(d1) * BASE * BASE + static_cast<int64_t>(d2) * BASE + d3) /
        (ldig * 2);
}
res.normalize();
return res / norm;
}

BigInt nth_root(int n) const {
    if (sign == -1 && n % 2 == 0) {
        throw std::runtime_error("Cannot take even root of a negative number.");
    }
    if (*this == 0 || n < 0) {
        return BigInt(0);
    }
    if (n >= size()) {
        int p = 1;
        while (comp(BigInt(p).pow(n)) > 0) {
            p++;
        }
        return comp(BigInt(p).pow(n)) < 0 ? p - 1 : p;
    }
    BigInt lo(BigInt(10).pow(static_cast<int>(ceil(static_cast<double>(size()) / n)) - 1)),
        hi(lo * 10), mid;
    while (lo < hi) {
        mid = (lo + hi) / 2;
        int cmp = comp(digits, mid.pow(n).digits, 1, 1);
        if (lo < mid && cmp > 0) {
            lo = mid;
        } else if (mid < hi && cmp < 0) {
            hi = mid;
        } else {
            return (sign == -1) ? -mid : mid;
        }
    }
    return (sign == -1) ? -(mid + 1) : (mid + 1);
}

static BigInt rand(int n) {
    if (n == 0) {
        return BigInt(0);
    }
    std::string s(1, '1' + (::rand() % 9));
    for (int i = 1; i < n; i++) {
        s += '0' + (::rand() % 10);
    }
    return BigInt(s);
}

friend int comp(const BigInt &a, const BigInt &b) { return a.comp(b); }
friend BigInt abs(const BigInt &v) { return v.abs(); }
friend BigInt pow(const BigInt &v, int n) { return v.pow(n); }
friend BigInt sqrt(const BigInt &v) { return v.sqrt(); }
friend BigInt nth_root(const BigInt &v, int n) { return v.nth_root(n); }

```

```
};
```

Example Usage

```
#include <cassert>

int main() {
    BigInt a("-9899819294989142124"), b("12398124981294214");
    assert(a + b == "-9887421170007847910");
    assert(a - b == "-9912217419970436338");
    assert(a * b == "-122739196911503356525379735104870536");
    assert(a / b == "-798");

    // Raw integers and strings work on either side of comparisons and arithmetic.
    assert("12398124981294214" == b);
    assert(5 + BigInt(7) == 12);
    assert(-100 < b && 5 % BigInt(3) == 2);

    assert(BigInt(20).pow(12345).size() == 16062);
    assert(BigInt("9812985918924981892491829").nth_root(4) == 1769906);
    for (int i = -100; i <= 100; i++) {
        if (i >= 0) {
            assert(BigInt(i).sqrt() == static_cast<int>(sqrt(i)));
        }
        for (int j = -100; j <= 100; j++) {
            assert(BigInt(i) + BigInt(j) == i + j);
            assert(BigInt(i) - BigInt(j) == i - j);
            assert(BigInt(i) * BigInt(j) == i * j);
            if (j != 0) {
                assert(BigInt(i) / BigInt(j) == i / j);
            }
            if (0 < i && i <= 10 && 0 < j && j <= 10) {
                assert(BigInt(i).nth_root(j) == static_cast<int64_t>((pow(i, 1.0 / j) + 1E-5)));
                int64_t p = 1;
                for (int k = 0; k < j; k++) {
                    p *= i;
                }
                assert(BigInt(i).pow(j) == p);
            }
        }
    }
    for (int i = 0; i < 20; i++) {
        int n = rand() % 100 + 1;
        BigInt a(BigInt::rand(n)), s(a.sqrt()), xx(s * s), yy(s + 1);
        yy *= yy;
        assert(xx <= a && a < yy);
        BigInt b(BigInt::rand(rand() % n + 1) + 1), q(a / b);
        xx = q * b;
        yy = b * (q + 1);
        assert(a >= xx && a < yy);
    }
    BigInt x(-6);
    assert(x.to_string() == "-6");
    assert(x.to_llong() == -6LL);
    assert(x.to_double() == -6.0);
    assert(x.to_ldouble() == -6.0);
    return 0;
}
```

6.4.3 Rational Numbers

Perform operations on rational numbers internally represented as two integers: a numerator and a denominator. The template integer type must support streamed input/output, comparisons, and arithmetic operations.

Overflow is not checked for in internal operations: comparisons and arithmetic cross-multiply numerators and denominators, so instantiate with a wider integer type (such as `__int128`, or even `BigInt`) if the values may grow large.

- `Rational<Int>(n)` constructs a rational number with numerator `n` and denominator 1.
- `Rational<Int>(n, d)` constructs a rational number with numerator `n` and denominator `d`.
- `operator>>` inputs a rational number using the next integer from the stream as the numerator and 1 as the denominator.
- `operator<<` outputs the rational number as a string consisting of possibly a minus sign followed by the numerator, followed by a slash, followed by the denominator.
- `v.to_string()`, `v.to_llong()`, `v.to_double()`, and `v.to_ldouble()` return the rational `v` converted to an `std::string`, `int64_t`, `double`, and `long double` respectively.
- `v.abs()`, `abs(v)`, `v.floor()`, and `v.ceil()` return the absolute value, floor, and ceiling of rational `v`.
- Operators `<`, `>`, `<=`, `>=`, `==`, `!=`, `+`, `-`, `*`, `/`, `%`, `++`, `-`, `+=`, `-=`, `*=`, `/=`, and `%=` are defined analogous to those on numerical primitives. The comparisons and binary arithmetic are hidden friends, so a raw integer operand works on either side.

Time Complexity:

- $O(\log(n + d))$ per call to constructor `Rational(n, d)`.
- $O(1)$ per call to all other operations, assuming that corresponding operations on the template integer type are $O(1)$ as well.

Space Complexity:

- $O(1)$ for storage of the rational number.
- $O(1)$ auxiliary space for all operations.

```
#include <stdint>
#include <istream>
#include <ostream>
#include <sstream>
#include <string>

template<typename Int>
class Rational {
    Int num, den;

public:
    Rational() : num(0), den(1) {}
    Rational(const Int &n) : num(n), den(1) {}

    template<typename T1, typename T2>
    Rational(const T1 &n, const T2 &d) : num(n), den(d) {
        if (den == 0) {
            throw std::runtime_error("Division by zero in Rational.");
        }
        if (den < 0) {
            num = -num;
            den = -den;
        }
        Int a(num < 0 ? -num : num), b(den), tmp;
        while (a != 0 && b != 0) {
            tmp = a % b;
            a = b;
            b = tmp;
        }
        Int gcd = (b == 0) ? a : b;
        num /= gcd;
        den /= gcd;
    }

    friend std::istream &operator>>(std::istream &in, Rational &r) {
        in >> r.num;
```



```

    r.den = 1;
    return in;
}

friend std::ostream &operator<<(std::ostream &out, const Rational &r) {
    out << r.num << "/" << r.den;
    return out;
}

std::string to_string() const {
    std::stringstream ss;
    ss << num << "/" << den;
    return ss.str();
}

// to_llong/to_double/to_ldouble round-trip through a stringstream so that any Int supporting
// streamed I/O (e.g. a big-integer type) converts, even without a direct cast to the target type.
int64_t to_llong() const {
    std::stringstream ss;
    ss << num << " " << den;
    int64_t n, d;
    ss >> n >> d;
    return n / d;
}

double to_double() const {
    std::stringstream ss;
    ss << num << " " << den;
    double n, d;
    ss >> n >> d;
    return n / d;
}

long double to_ldouble() const {
    long double n, d;
    std::stringstream ss;
    ss << num << " " << den;
    ss >> n >> d;
    return n / d;
}

// The comparison and binary arithmetic operators are hidden friends, so a raw integer operand on
// either side converts through the implicit constructor.
friend bool operator<(const Rational &a, const Rational &b) {
    return a.num * b.den < b.num * a.den;
}

friend bool operator==(const Rational &a, const Rational &b) {
    return a.num == b.num && a.den == b.den;
}

friend bool operator>(const Rational &a, const Rational &b) { return b < a; }
friend bool operator<=(const Rational &a, const Rational &b) { return !(b < a); }
friend bool operator>=(const Rational &a, const Rational &b) { return !(a < b); }
friend bool operator!=(const Rational &a, const Rational &b) { return !(a == b); }

Rational abs() const { return Rational(num < 0 ? -num : num, den); }
friend Rational abs(const Rational &r) { return r.abs(); }
Int floor() const { return num < 0 ? -((-num + den - 1) / den) : num / den; }
Int ceil() const { return num < 0 ? -(-num / den) : (num + den - 1) / den; }

friend Rational operator+(const Rational &a, const Rational &b) {
    return Rational(a.num * b.den + b.num * a.den, a.den * b.den);
}

friend Rational operator-(const Rational &a, const Rational &b) {
    return Rational(a.num * b.den - b.num * a.den, a.den * b.den);
}

```

```

friend Rational operator*(const Rational &a, const Rational &b) {
    return Rational(a.num * b.num, a.den * b.den);
}

friend Rational operator/(const Rational &a, const Rational &b) {
    return Rational(a.num * b.den, a.den * b.num);
}

friend Rational operator%(const Rational &a, const Rational &b) {
    return a - b * Rational(a.num * b.den / (b.num * a.den), 1);
}

Rational operator-() const { return Rational(-num, den); }
Rational operator++(int) { Rational t(*this); operator++(); return t; }
Rational operator--(int) { Rational t(*this); operator--(); return t; }
Rational &operator++() { *this = *this + 1; return *this; }
Rational &operator--() { *this = *this - 1; return *this; }
Rational &operator+=(const Rational &r) { *this = *this + r; return *this; }
Rational &operator-=(const Rational &r) { *this = *this - r; return *this; }
Rational &operator*=(const Rational &r) { *this = *this * r; return *this; }
Rational &operator/=(const Rational &r) { *this = *this / r; return *this; }
Rational &operator%=(const Rational &r) { *this = *this % r; return *this; }
};

```

Example Usage

```

#include <cassert>
#include <cmath>

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    using Rational = Rational<int64_t>;
    assert(Rational(-21, 1) % 2 == -1);
    Rational r(Rational(-53, 10) % Rational(-17, 10));
    assert(EQ(r.to_ldouble(), fmod(-5.3, -1.7)));
    assert(r.to_string() == "-1/5");

    // Raw integers work on either side of comparisons and arithmetic.
    assert(2 + Rational(1, 2) == Rational(5, 2));
    assert(1 < Rational(3, 2) && Rational(3, 2) < 2);
    assert(2 % Rational(3, 4) == Rational(1, 2));
    return 0;
}

```

6.5 Linear Algebra

6.5.1 Matrix Utilities

Basic matrix operations defined on a two-dimensional vector of numeric values.

- `make_matrix<T>(m, n, v)` constructs and returns an m by n matrix with 0-based indices (row indices $[0, m)$ and column indices $[0, n)$), where every value is initialized to `v`.
- `identity_matrix<T>(n)` returns the n by n identity matrix, that is, a matrix where each `a[i][j]` equals 1 if $i = j$, or 0 otherwise.
- `rows(a)` returns the number of rows in matrix `a`.
- `columns(a)` returns the number of columns in matrix `a`.

- `a[i][j]` may be used to access or modify the specified entry of `a`.
- Operators `<`, `>`, `<=`, `>=`, `==`, and `!=` define lexicographical comparison based on that of `std::vector`.
- Operators `+`, `-`, `*`, `/`, `+=`, `-=`, `*=`, and `/=` define scalar addition, subtraction, multiplication, and division involving a matrix and a numeric scalar value.
- Operators `*` and `*=` define vector and matrix multiplication.
- Operators `~` and `~=` define iterative binary matrix exponentiation of a square matrix `a` by an integer power `p`.
- `power_sum(a, p)` returns the power sum of a square matrix `a` up to an integer power `p`, that is, $a + a^2 + \dots + a^p$.
- `transpose(a)` returns the transpose of an m by n matrix `a`, that is, a new n by m matrix `b` such that `a[i][j] = b[j][i]` for every $i \in [0, m)$ and $j \in [0, n)$.
- `transpose_in_place(a)` assigns the square matrix `a` to its transpose, returning a reference to the modified argument itself.
- `rotate(a, d)` returns the matrix `a` rotated `d` degrees clockwise. A negative `d` specifies a counter-clockwise rotation, and `d` must be a multiple of 90.
- `rotate_in_place(a, d)` assigns the square matrix `a` to its rotation by `d` degrees clockwise, returning a reference to the modified argument itself. A negative `d` specifies a counter-clockwise rotation, and `d` must be a multiple of 90.

Exponentiation uses the usual iterative binary exponentiation pattern: keep an accumulated result, square the base each round, and multiply the result when the current exponent bit is set. The power sum uses a block matrix identity that $\begin{bmatrix} A & A \\ 0 & I \end{bmatrix}^p$ has $A + A^2 + \dots + A^p$ in its upper-right block.

Time Complexity:

- $O(m \cdot n)$ for construction, output, comparison, and scalar arithmetic of m by n matrices.
- $O(1)$ for `rows(a)` and `columns(a)`.
- $O(m \cdot n)$ for matrix-matrix addition and subtraction of m by n matrices.
- $O(n^3 \log p)$ for exponentiation of an n by n matrix to power p .
- $O(n^3 \log p)$ for power sum of an n by n matrix to power p .
- $O(m \cdot n \cdot k)$ for multiplication of an m by n matrix by an n by k matrix.
- $O(m \cdot n)$ for `transpose()`, `transpose_in_place()`, `rotate()`, and `rotate_in_place()` of m by n matrices.

Space Complexity:

- $O(1)$ auxiliary space for `rows()`, `columns()`, `a[i][j]` access, comparison operators, and in-place operations.
- $O(n^2)$ auxiliary heap space for exponentiation of an n by n matrix to power p .
- $O(n^2)$ auxiliary heap space for `power_sum()` of an n by n matrix up to power p .
- $O(m \cdot n)$ auxiliary heap space for all non-in-place operations returning an m by n matrix, `transpose()`, and `rotate()`.

```
#include <algorithm>
#include <cstdint>
#include <iomanip>
#include <ostream>
#include <stdexcept>
#include <vector>

template<typename T>
using Matrix = std::vector<std::vector<T>>>;

template<typename T = int>
Matrix<T> make_matrix(int m, int n, const T &v = T{}) {
    return Matrix<T>(m, std::vector<T>(n, v));
}

template<typename T = int>
Matrix<T> identity_matrix(int n) {
    Matrix<T> res = make_matrix<T>(n, n);
    for (int i = 0; i < n; i++) {
```

```

    res[i][i] = 1;
}
return res;
}

template<typename T>
int rows(const Matrix<T> &a) {
    return static_cast<int>(a.size());
}

template<typename T>
int columns(const Matrix<T> &a) {
    return a.empty() ? 0 : static_cast<int>(a[0].size());
}

template<typename T>
std::ostream &operator<<(std::ostream &out, const Matrix<T> &a) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            out << std::setw(10) << std::fixed << std::setprecision(5) << a[i][j];
        }
        out << std::endl;
    }
    return out;
}

template<typename T, typename U>
Matrix<T> &operator+=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] += v;
        }
    }
    return a;
}

template<typename T, typename U>
Matrix<T> &operator-=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] -= v;
        }
    }
    return a;
}

template<typename T, typename U>
Matrix<T> &operator*=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] *= v;
        }
    }
    return a;
}

template<typename T, typename U>
Matrix<T> &operator/=(Matrix<T> &a, const U &v) {
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] /= v;
        }
    }
    return a;
}

template<typename T>
Matrix<T> &operator+=(Matrix<T> &a, const Matrix<T> &b) {

```

```

    if (rows(a) != rows(b) || columns(a) != columns(b)) {
        throw std::runtime_error("Invalid dimensions for matrix addition.");
    }
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] += b[i][j];
        }
    }
    return a;
}

template<typename T>
Matrix<T> &operator--(Matrix<T> &a, const Matrix<T> &b) {
    if (rows(a) != rows(b) || columns(a) != columns(b)) {
        throw std::runtime_error("Invalid dimensions for matrix subtraction.");
    }
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(a); j++) {
            a[i][j] -= b[i][j];
        }
    }
    return a;
}

template<typename T>
Matrix<T> operator+(const Matrix<T> &a, const Matrix<T> &b) {
    Matrix<T> c(a);
    return c += b;
}

template<typename T>
Matrix<T> operator-(const Matrix<T> &a, const Matrix<T> &b) {
    Matrix<T> c(a);
    return c -= b;
}

template<typename T>
Matrix<T> &operator*=(Matrix<T> &a, const std::vector<T> &v) {
    if (columns(a) != static_cast<int>(v.size()) || v.empty()) {
        throw std::runtime_error("Invalid dimensions for matrix-vector multiplication.");
    }
    int cols = columns(a);
    for (int i = 0; i < rows(a); i++) {
        T sum = 0;
        for (int j = 0; j < cols; j++) {
            sum += a[i][j] * v[j];
        }
        a[i].assign(1, sum);
    }
    return a;
}

template<typename T>
Matrix<T> operator*(const Matrix<T> &a, const Matrix<T> &b) {
    if (columns(a) != rows(b)) {
        throw std::runtime_error("Invalid dimensions for matrix multiplication.");
    }
    Matrix<T> res = make_matrix<T>(rows(a), columns(b));
    for (int i = 0; i < rows(a); i++) {
        for (int j = 0; j < columns(b); j++) {
            for (int k = 0; k < rows(b); k++) {
                res[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    return res;
}

```

```

template<typename T>
Matrix<T> &operator*=(Matrix<T> &a, const Matrix<T> &b) { return a = a * b; }

template<typename T, typename U>
Matrix<T> operator+(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m += v; }

template<typename T, typename U>
Matrix<T> operator-(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m -= v; }

template<typename T, typename U>
Matrix<T> operator*(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m *= v; }

template<typename T, typename U>
Matrix<T> operator/(const Matrix<T> &a, const U &v) { Matrix<T> m(a); return m /= v; }

template<typename T, typename U>
Matrix<T> operator+(const U &v, const Matrix<T> &a) { return a + v; }

template<typename T, typename U>
Matrix<T> operator*(const U &v, const Matrix<T> &a) { return a * v; }

template<typename T, typename U>
Matrix<T> operator-(const U &v, const Matrix<T> &a) {
    Matrix<T> m = make_matrix<T>(rows(a), columns(a), v);
    return m -= a;
}

template<typename T>
Matrix<T> operator^(Matrix<T> a, unsigned int p) {
    if (rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for exponentiation.");
    }
    Matrix<T> res = identity_matrix<T>(rows(a));
    while (p > 0) {
        if (p & 1) {
            res *= a;
        }
        a *= a;
        p >>= 1;
    }
    return res;
}

template<typename T>
Matrix<T> operator^(Matrix<T> &a, unsigned int p) {
    return a = a ^ p;
}

template<typename T>
Matrix<T> power_sum(const Matrix<T> &a, unsigned int p) {
    if (rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for power_sum.");
    }
    int n = rows(a);
    if (p == 0) {
        return make_matrix<T>(n, n);
    }
    // [A A]^p = [A^p A + A^2 + ... + A^p]
    // [0 I]    [0          I          ]
    Matrix<T> block = make_matrix<T>(2 * n, 2 * n);
    for (int i = 0; i < n; i++) {
        block[i][i] = a[i][i];
        block[i + n][i + n] = 1;
        for (int j = 0; j < n; j++) {
            block[i][j] = a[i][j];
            block[i][j + n] = a[i][j];
        }
    }
}

```

```

    block = block ^ p;
    Matrix<T> res = make_matrix<T>(n, n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            res[i][j] = block[i][j + n];
        }
    }
    return res;
}

template<typename T>
Matrix<T> transpose(const Matrix<T> &a) {
    Matrix<T> res = make_matrix<T>(columns(a), rows(a));
    for (int i = 0; i < rows(res); i++) {
        for (int j = 0; j < columns(res); j++) {
            res[i][j] = a[j][i];
        }
    }
    return res;
}

template<typename T>
Matrix<T> &transpose_in_place(Matrix<T> &a) {
    if (rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for transpose_in_place.");
    }
    for (int i = 0; i < rows(a); i++) {
        for (int j = i + 1; j < columns(a); j++) {
            std::swap(a[i][j], a[j][i]);
        }
    }
    return a;
}

template<typename T>
Matrix<T> rotate(const Matrix<T> &a, int degrees = 90) {
    if (degrees % 90 != 0) {
        throw std::runtime_error("Rotation must be by a multiple of 90 degrees.");
    }
    if (degrees < 0) {
        degrees = 360 - ((-degrees) % 360);
    }
    Matrix<T> res;
    switch (degrees % 360) {
        case 90: {
            res = make_matrix<T>(columns(a), rows(a));
            for (int i = 0; i < columns(a); i++) {
                for (int j = 0; j < rows(a); j++) {
                    res[i][j] = a[rows(a) - j - 1][i];
                }
            }
            break;
        }
        case 180: {
            res = make_matrix<T>(rows(a), columns(a));
            for (int i = 0; i < rows(a); i++) {
                for (int j = 0; j < columns(a); j++) {
                    res[i][j] = a[rows(a) - i - 1][columns(a) - j - 1];
                }
            }
            break;
        }
        case 270: {
            res = make_matrix<T>(columns(a), rows(a));
            for (int i = 0; i < columns(a); i++) {
                for (int j = 0; j < rows(a); j++) {
                    res[i][j] = a[j][columns(a) - i - 1];
                }
            }
        }
    }
}

```

```

    }
    break;
}
default: {
    res = a;
}
}
return res;
}

template<typename T>
Matrix<T> &rotate_in_place(Matrix<T> &a, int degrees = 90) {
    if (degrees % 90 != 0) {
        throw std::runtime_error("Rotation must be by a multiple of 90 degrees.");
    }
    if (degrees % 180 != 0 && rows(a) != columns(a)) {
        throw std::runtime_error("Matrix must be square for rotate_in_place.");
    }
    if (degrees < 0) {
        degrees = 360 - ((-degrees) % 360);
    }
    int n = rows(a);
    switch (degrees % 360) {
        case 90: {
            transpose_in_place(a);
            for (auto &row : a) {
                std::reverse(row.begin(), row.end());
            }
            break;
        }
        case 180: {
            // A 180-degree rotation is a horizontal flip (reverse each row) followed by a vertical flip
            // (reverse the row order). This works for non-square matrices too, unlike a row/column index
            // scheme keyed on a single dimension.
            for (auto &row : a) {
                std::reverse(row.begin(), row.end());
            }
            std::reverse(a.begin(), a.end());
            break;
        }
        case 270: {
            transpose_in_place(a);
            for (int j = 0; j < n; j++) {
                for (int i = 0, k = columns(a) - 1; i < k; i++, k--) {
                    std::swap(a[i][j], a[k][j]);
                }
            }
            break;
        }
    }
    return a;
}

```

Example Usage

```

#include <cassert>
#include <iostream>
using namespace std;

int main() {
    using matrix = Matrix<int>;
    matrix a{{1, 2, 3}, {4, 5, 6}};
    matrix a90{{4, 1}, {5, 2}, {6, 3}};
    matrix a180{{6, 5, 4}, {3, 2, 1}};
    matrix a270{{3, 6}, {2, 5}, {1, 4}};
    cout << a << endl;
}

```



```

assert(rotate(a, -270) == a90);
assert(rotate(a, -180) == a180);
assert(rotate(a, -90) == a270);
assert(rotate(a, 0) == a);
assert(rotate(a, 90) == a90);
assert(rotate(a, 180) == a180);
assert(rotate(a, 270) == a270);
assert(rotate(a, 360) == a);

matrix b{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
for (int d = -360; d <= 360; d += 90) {
    matrix m = b;
    assert(rotate_in_place(m, d) == rotate(b, d));
}

matrix m = make_matrix(5, 5, 10) + 10;
vector<int> v{1, 2, 3, 4, 5};
matrix mv{{300}, {300}, {300}, {300}, {300}};
assert(m * v == mv);

m[0][0] += 5;
assert(m[0][0] == 25 && m[1][1] == 20);
assert((m ^ 0) == identity_matrix(5));
assert(power_sum(m, 3) == m + m * m + (m ^ 3));

Matrix<double> d = make_matrix<double>(2, 2, 0.5);
assert(rows(d) == 2 && columns(d) == 2);
assert((d + 0.25)[0][0] == 0.75);
return 0;
}

```

6.5.2 Row Reduction

Converts a matrix to reduced row echelon form using Gaussian elimination to solve a system of linear equations and compute rank. Each round finds a row with a nonzero entry in the current leading column, swaps the largest-magnitude candidate into place for partial pivoting, normalizes that row, and subtracts multiples of it from every other row to clear the column. In practice, this method can still be prone to rounding error on certain matrices. For a more accurate algorithm for solving systems of linear equations, LU decomposition with row partial pivoting should be used.

- `row_reduce(a)` assigns the matrix `a` to its reduced row echelon form, returning a reference to the modified argument itself.
- `matrix_rank(a)` returns the rank of matrix `a`, i.e. the number of nonzero rows after row reduction.
- `solve_system(a, b, &x)` solves the system of linear equations $ax = b$ given an m by n matrix `a` of real values, and a length m vector `b`, returning 0 if there is one solution, -1 if there are zero solutions, or -2 if there are infinite solutions. If there is exactly one solution, then the output pointer `x` is populated with the solution vector of length n .

Time Complexity: $O(m^2 \cdot n)$ per call to `row_reduce(a)`, `matrix_rank(a)`, and `solve_system(a)`, where m and n are the number of rows and columns of `a`, respectively.

Space Complexity:

- $O(1)$ auxiliary for `row_reduce(a)`.
- $O(m \cdot n)$ auxiliary heap space for `matrix_rank(a)` and `solve_system(a)`.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <stdexcept>
#include <vector>

```

```

const double EPS = 1e-9;

template<typename Matrix>
Matrix &row_reduce(Matrix &a) {
    if (a.empty()) {
        return a;
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    for (int r = 0, lead = 0; r < rows && lead < cols; lead++) {
        int pivot = r;
        for (int i = r + 1; i < rows; i++) {
            if (fabs(a[i][lead]) > fabs(a[pivot][lead])) {
                pivot = i;
            }
        }
        if (fabs(a[pivot][lead]) < EPS) {
            continue;
        }
        std::swap(a[pivot], a[r]);
        auto lv = a[r][lead];
        for (int j = 0; j < cols; j++) {
            a[r][j] /= lv;
        }
        for (int i = 0; i < rows; i++) {
            if (i != r) {
                lv = a[i][lead];
                for (int j = 0; j < cols; j++) {
                    a[i][j] -= lv * a[r][j];
                }
            }
        }
        for (int j = 0; j < lead; j++) {
            a[r][j] = 0;
        }
        a[r][lead] = 1;
        r++;
    }
    return a;
}

template<typename Matrix>
int matrix_rank(Matrix a) {
    row_reduce(a);
    int rank = 0;
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        for (int j = 0; j < static_cast<int>(a[i].size()); j++) {
            if (fabs(a[i][j]) > EPS) {
                rank++;
                break;
            }
        }
    }
    return rank;
}

template<typename Matrix, typename T>
int solve_system(const Matrix &a, const std::vector<T> &b, std::vector<T> *x) {
    if (x == nullptr || a.empty() || a.size() != b.size()) {
        return -1;
    }
    int rows = static_cast<int>(a.size()), cols = static_cast<int>(a[0].size());
    if (rows < cols) {
        return -2;
    }
    Matrix m(a);
    for (int i = 0; i < rows; i++) {
        m[i].push_back(b[i]);
    }
}

```

```

row_reduce(m);
int rank = 0;
for (int i = 0; i < rows; i++) {
    int lead = -1;
    for (int j = 0; j < cols && lead < 0; j++) {
        if (fabs(m[i][j]) > EPS) {
            lead = j;
        }
    }
    if (lead < 0) {
        // A zero coefficient row with a nonzero constant means 0 = nonzero (no solution).
        if (fabs(m[i][cols]) > EPS) {
            return -1;
        }
    } else {
        rank++;
    }
}
// A unique solution requires a pivot in every column; fewer means free variables remain. This
// also catches trailing free columns, which a per-row `lead > i` test alone would miss.
if (rank < cols) {
    return -2;
}
x->resize(cols);
for (int i = 0; i < cols; i++) {
    (*x)[i] = m[i][cols];
}
return 0;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<double>> a{{-1, 2, 5}, {1, 0, -6}, {-4, 2, 2}};
    vector<double> b{3, 1, -2};
    vector<double> x;
    assert(solve_system(a, b, &x) == 0);
    assert(matrix_rank(a) == 3);
    assert(matrix_rank(vector<vector<double>>{{1, 2, 3}, {2, 4, 6}, {0, 1, 1}}) == 2);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        double sum = 0;
        for (int j = 0; j < static_cast<int>(a[i].size()); j++) {
            sum += a[i][j] * x[j];
        }
        assert(fabs(sum - b[i]) < EPS);
    }
    return 0;
}

```

6.5.3 Determinant and Inverse

Computes the determinant and inverse of a square matrix using Gaussian elimination. The inverse of a matrix a is another matrix b such that ab equals the identity matrix. The inverse of a exists if and only if the determinant of a is nonzero. In this case, a is called invertible or non-singular. The determinant falls out of elimination as the product of the pivots, negated once per row swap. The inverse is found by appending the identity matrix and row-reducing the combined matrix: the operations that turn a into the identity turn the identity into the inverse. In practice, simple Gaussian elimination is prone to rounding error on certain matrices. For a more accurate algorithm for solving systems of linear equations, use LU decomposition with row partial pivoting. For an integer

matrix whose determinant is wanted exactly, the Bareiss algorithm runs a fraction-free elimination: every intermediate value is itself a determinant of a submatrix, so the divisions are always exact and the arithmetic stays in integers.

- `det_naive(a)` returns the determinant of an n by n matrix `a`, using the classic divide-and-conquer algorithm by Laplace expansions. It is division-free and computes in the matrix's element type, so an integer matrix yields an exact integer determinant; but at $O(n!)$ it is only practical for tiny `n`, so prefer `det_bareiss` for exact integer determinants.
- `det(a)` returns the determinant of an n by n matrix `a` using Gaussian elimination.
- `det_bareiss(a)` returns the exact determinant of an integer matrix `a` using fraction-free elimination, with no rounding error. Intermediate values are bounded by the magnitudes of the matrix's minors, so the entries must stay within `int64_t`; substitute a big-integer type for large matrices.
- `invert(a)` assigns the n by n matrix `a` to its inverse (if it exists), returning a reference to the modified argument itself. If `a` is not invertible, then its assigned values after the function call will be undefined (`+/-Inf` or `+/-NaN`).

Time Complexity:

- $O(n!)$ per call to `det_naive()`, where n is the dimension of the matrix.
- $O(n^3)$ per call to `det()`, `det_bareiss()`, and `invert()` where n is the dimension of the matrix.

Space Complexity:

- $O(n)$ auxiliary stack space and $O(n! \cdot n)$ auxiliary heap space for `det_naive()`, where n is the dimension of the matrix.
- $O(n^2)$ auxiliary heap space for `det()`, `det_bareiss()`, and `invert()`.

```
#include <cmath>
#include <cstdlib>
#include <type_traits>
#include <vector>

template<typename SquareMatrix>
auto det_naive(const SquareMatrix &a) {
    int n = static_cast<int>(a.size());
    if (n == 1) {
        return a[0][0];
    }
    if (n == 2) {
        return a[0][0] * a[1][1] - a[0][1] * a[1][0];
    }
    std::decay_t<decltype(a[0][0])> res = 0;
    SquareMatrix temp(n - 1, typename SquareMatrix::value_type(n - 1));
    for (int p = 0; p < n; p++) {
        int h = 0, k = 0;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (j == p) {
                    continue;
                }
                temp[h][k++] = a[i][j];
                if (k == n - 1) {
                    h++;
                    k = 0;
                }
            }
        }
        res += (p % 2 == 0 ? 1 : -1) * a[0][p] * det_naive(temp);
    }
    return res;
}

template<typename SquareMatrix>
double det(const SquareMatrix &a, double EPS = 1e-10) {
    int n = static_cast<int>(a.size());
```

```

SquareMatrix b(a);
double res = 1.0;
for (int i = 0; i < n; i++) {
    // Partial pivoting: bring the largest-magnitude entry in column i to the diagonal. Each row
    // swap negates the determinant, so the swap sign must be tracked. Selecting pivot rows out of
    // order without this sign correction can return |det| with the wrong sign.
    int p = i;
    for (int r = i + 1; r < n; r++) {
        if (fabs(b[r][i]) > fabs(b[p][i])) {
            p = r;
        }
    }
    if (fabs(b[p][i]) < EPS) {
        return 0;
    }
    if (p != i) {
        std::swap(b[p], b[i]);
        res = -res;
    }
    res *= b[i][i];
    for (int j = i + 1; j < n; j++) {
        double z = b[j][i] / b[i][i];
        for (int k = i; k < n; k++) {
            b[j][k] -= z * b[i][k];
        }
    }
}
return res;
}

```

```

int64_t det_bareiss(std::vector<std::vector<int64_t>> a) {
    int n = static_cast<int>(a.size());
    int64_t prev = 1, sign = 1;
    for (int k = 0; k < n; k++) {
        if (a[k][k] == 0) { // Swap in a nonzero pivot; each swap flips the sign.
            int p = -1;
            for (int r = k + 1; r < n; r++) {
                if (a[r][k] != 0) {
                    p = r;
                    break;
                }
            }
            if (p == -1) {
                return 0;
            }
            std::swap(a[k], a[p]);
            sign = -sign;
        }
        for (int i = k + 1; i < n; i++) {
            for (int j = k + 1; j < n; j++) {
                // The division by the previous pivot is always exact (Bareiss's theorem).
                a[i][j] = (a[i][j] * a[k][k] - a[i][k] * a[k][j]) / prev;
            }
        }
        prev = a[k][k];
    }
    return n == 0 ? 1 : sign * a[n - 1][n - 1];
}

```

```

template<typename SquareMatrix>
SquareMatrix &invert(SquareMatrix &a) {
    int n = static_cast<int>(a.size());
    for (int i = 0; i < n; i++) {
        a[i].resize(2 * n);
        for (int j = n; j < n * 2; j++) {
            a[i][j] = (i == j - n ? 1 : 0);
        }
    }
}

```

```

for (int i = 0; i < n; i++) {
    // Partial pivoting: without it a zero on the diagonal divides by zero and fails even for an
    // invertible matrix (e.g. a permutation matrix). Picking the largest-magnitude pivot also
    // improves numerical stability.
    int p = i;
    for (int r = i + 1; r < n; r++) {
        if (fabs(a[r][i]) > fabs(a[p][i])) {
            p = r;
        }
    }
    std::swap(a[p], a[i]);
    double z = a[i][i];
    for (int j = i; j < n * 2; j++) {
        a[i][j] /= z;
    }
    for (int j = 0; j < n; j++) {
        if (i != j) {
            double z = a[j][i];
            for (int k = 0; k < n * 2; k++) {
                a[j][k] -= z * a[i][k];
            }
        }
    }
}
for (int i = 0; i < n; i++) {
    a[i].erase(a[i].begin(), a[i].begin() + n);
}
return a;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<double>> a{{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
    int n = static_cast<int>(a.size());
    vector<vector<double>> res(n, vector<double>(n, 0));
    double d = det(a);
    assert(fabs(d - det_naive(a)) < 1e-10);

    // Bareiss gives the determinant of an integer matrix exactly, with no rounding.
    vector<vector<int64_t>> ai{{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
    assert(det_naive(ai) == -306); // Division-free, so also exact in the int64_t element type.
    assert(det_bareiss(ai) == -306);
    assert(det_bareiss({{2, 0, 0}, {0, 3, 0}, {0, 0, 5}}) == 30);
    assert(det_bareiss({{1, 2}, {2, 4}}) == 0); // Singular.
    vector<vector<double>> inv = a;
    invert(inv);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                res[i][j] += a[i][k] * inv[k][j];
            }
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            assert(fabs(res[i][j] - (i == j ? 1 : 0)) < 1e-10);
        }
    }
    return 0;
}

```

6.5.4 LU Decomposition

The LU decomposition of a matrix a with row-partial pivoting is a factorization of a (after some rows are possibly permuted by a permutation matrix p) as a product of a lower triangular matrix l and an upper triangular matrix u . This factorization can be used to tackle many common problems in linear algebra such as solving systems of linear equations and computing determinants. An improvement on basic row reduction, LU decomposition by row-partial pivoting keeps the relative magnitude of matrix values small, thus reducing the relative error due to rounding in computed solutions.

- `lu_decompose(a, &picol)` assigns the m by n matrix `a` to merged LU decomposition matrix `lu`, returning either 0 or 1 denoting the “sign” of the permutation parity (0 if the number of overall row swaps performed is even, or 1 if it is odd), or -1 denoting a degenerate matrix (i.e. singular for square matrices). The merged matrix `lu` has `lu[i][j] = l[i][j]` for $i > j$ and `lu[i][j] = u[i][j]` for $i \leq j$. Note that the algorithm always yields an atomic lower triangular matrix for which the diagonal entries `l[i][i]` are always equal to 1, so this is not explicitly stored in the resulting merged matrix. For general i and j , the values of the lower and upper triangular matrices should be accessed via the `getl(lu, i, j)` and `getu(lu, i, j)` functions. Optionally, a `vector<int>` pointer `picol` may be passed to return the permutation vector `picol` where `picol[i]` stores the only column that is equal to 1 in row i of the permutation matrix p (all other columns in row i of p are implicitly 0). The permutation matrix p corresponding to `picol` satisfies $pa = lu$.
- `solve_system(a, b, &x)` solves the system of linear equations $ax = b$ given an m by n matrix `a` of real values, and a length m vector `b`, returning 0 if there is one solution or -1 if there are zero or infinite solutions. If there is exactly one solution, then the output pointer `x` is populated with the solution vector of length n .
- `det(a)` returns the determinant of an n by n matrix `a` using LU decomposition.
- `invert(a)` assigns the n by n matrix `a` to its inverse (if it exists), returning 0 if the inversion was successful or -1 if `a` has no inverse.

Time Complexity:

- $O(m^2 \cdot n)$ per call to `lu_decompose(a)` and `solve_system(a, b)`, where m and n are the number of rows and columns of `a`, respectively.
- $O(n^3)$ per call to `det(a)` and `invert(a)`, where n is the length of square matrix `a`.

Space Complexity:

- $O(1)$ auxiliary for `lu_decompose()`.
- $O(n^2)$ for `det(a)` and `invert(a)`.
- $O(m \cdot n)$ auxiliary heap space for `solve_system(a, b)`.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>
#include <numeric>
#include <vector>

template<typename Matrix>
int lu_decompose(Matrix &a, std::vector<int> *picol = nullptr, const double EPS = 1e-10) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    int parity = 0;
    if (picol != nullptr) {
        picol->resize(rows);
        std::iota(picol->begin(), picol->end(), 0);
    }
    for (int i = 0; i < rows && i < cols; i++) {
        int pi = i;
        for (int k = i + 1; k < rows; k++) {
            if (fabs(a[k][i]) > fabs(a[pi][i])) {
                pi = k;
            }
        }
        if (fabs(a[pi][i]) < EPS) {

```

```

    return -1;
}
if (pi != i) {
    if (p1col != nullptr) {
        std::iter_swap(p1col->begin() + i, p1col->begin() + pi);
    }
    std::iter_swap(a.begin() + i, a.begin() + pi);
    parity = 1 - parity;
}
for (int j = i + 1; j < rows; j++) {
    a[j][i] /= a[i][i];
    for (int k = i + 1; k < cols; k++) {
        a[j][k] -= a[j][i] * a[i][k];
    }
}
}
return parity;
}

template<typename Matrix>
double getl(const Matrix &lu, int i, int j) {
    return i > j ? lu[i][j] : (i < j ? 0 : 1);
}

template<typename Matrix>
double getu(const Matrix &lu, int i, int j) {
    return i <= j ? lu[i][j] : 0;
}

template<typename Matrix, typename T>
int solve_system(
    const Matrix &a, const std::vector<T> &b, std::vector<T> *x, const double EPS = 1e-10
) {
    int rows = static_cast<int>(a.size());
    int cols = a.empty() ? 0 : static_cast<int>(a[0].size());
    if (x == nullptr || a.empty() || a.size() != b.size() || rows < cols) {
        return -1;
    }
    x->resize(cols);
    std::vector<int> p1col;
    Matrix lu;
    int status = lu_decompose(lu = a, &p1col, EPS);
    if (status < 0) {
        return status;
    }
    for (int i = 0; i < cols; i++) {
        (*x)[i] = b[p1col[i]];
        for (int k = 0; k < i; k++) {
            (*x)[i] -= getl(lu, i, k) * (*x)[k];
        }
    }
    for (int i = cols - 1; i >= 0; i--) {
        for (int k = i + 1; k < cols; k++) {
            (*x)[i] -= getu(lu, i, k) * (*x)[k];
        }
        (*x)[i] /= getu(lu, i, i);
    }
    for (int i = 0; i < rows; i++) {
        double val = 0;
        for (int j = 0; j < cols; j++) {
            val += a[i][j] * (*x)[j];
        }
        // Mixed absolute/relative tolerance: dividing by b[i] alone would skip the check for negative
        // b[i] (ratio goes negative) and divide by zero when b[i] == 0.
        if (fabs(val - b[i]) > EPS * (1.0 + fabs(b[i]))) {
            return -1;
        }
    }
}

```



```

    return 0;
}

template<typename T>
double det(const T &a) {
    int n = static_cast<int>(a.size());
    T lu;
    int status = lu_decompose(lu = a);
    if (status < 0) {
        return 0;
    }
    double res = 1;
    for (int i = 0; i < n; i++) {
        res *= lu[i][i];
    }
    return status == 0 ? res : -res;
}

template<typename T>
int invert(T &a) {
    int n = static_cast<int>(a.size());
    std::vector<int> p1col;
    int status = lu_decompose(a, &p1col);
    if (status < 0) {
        return status;
    }
    T ia(n, typename T::value_type(n, 0));
    for (int j = 0; j < n; j++) {
        for (int i = 0; i < n; i++) {
            if (p1col[i] == j) {
                ia[i][j] = 1.0;
            } else {
                ia[i][j] = 0.0;
            }
            for (int k = 0; k < i; k++) {
                ia[i][j] -= getl(a, i, k) * ia[k][j];
            }
        }
        for (int i = n - 1; i >= 0; i--) {
            for (int k = i + 1; k < n; k++) {
                ia[i][j] -= getu(a, i, k) * ia[k][j];
            }
            ia[i][j] /= getu(a, i, i);
        }
    }
    a.swap(ia);
    return 0;
}

```

Example Usage

```

#include <cassert>
using namespace std;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    { // Solve a system.
        vector<vector<double>> a{{-1, 2, 5}, {1, 0, -6}, {-4, 2, 2}};
        vector<double> b{3, 1, -2};
        vector<double> x;
        assert(solve_system(a, b, &x) == 0);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            double sum = 0;

```

```

    for (int j = 0; j < static_cast<int>(a[0].size()); j++) {
        sum += a[i][j] * x[j];
    }
    assert(EQ(sum, b[i]));
}
}
{ // Find the determinant.
    vector<vector<double>> a{{1, 3, 5}, {2, 4, 7}, {1, 1, 0}};
    assert(EQ(det(a), 4));
}
{ // Find the inverse.
    vector<vector<double>> a{{6, 1, 1}, {4, -2, 5}, {2, 8, 7}};
    vector<vector<double>> inv = a;
    int n = static_cast<int>(a.size());
    vector<vector<double>> res(n, vector<double>(n, 0));
    assert(invert(inv) == 0);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                res[i][j] += a[i][k] * inv[k][j];
            }
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            assert(EQ(res[i][j], i == j ? 1 : 0));
        }
    }
    return 0;
}
}

```

6.5.5 Characteristic Polynomial

Computes the characteristic polynomial of a square matrix over a prime field: the coefficients of $\det(xI - A)$, a monic degree- n polynomial whose roots are the eigenvalues of A . Its constant term recovers the determinant (up to sign), its second-highest coefficient is the negated trace, and evaluating it at a scalar gives $\det(xI - A)$ for that x . It is the standard tool behind eigenvalue counting, matrix-power recurrences via Cayley-Hamilton, and problems that reduce to a determinant in an unknown.

A direct expansion would cost a determinant per coefficient. Instead the matrix is first reduced to upper Hessenberg form, where every entry more than one place below the diagonal is zero. This reduction uses similarity transforms (a row operation paired with the inverse column operation), which leave the characteristic polynomial unchanged. The characteristic polynomial of a Hessenberg matrix then follows from a short recurrence over its leading principal submatrices, adding one row and column at a time, for an overall cost of $O(n^3)$. The arithmetic is exact modulo a prime, since the Hessenberg reduction divides by pivots and is not numerically stable over the reals.

- `characteristic_polynomial(a)` returns a vector c of length $n + 1$ with $\det(xI - \mathbf{a}) = c_0 + c_1x + \dots + c_nx^n$ modulo `MOD`, where $c_n = 1$. The matrix `a` must be square; its entries are reduced modulo `MOD` internally.

Time Complexity: $O(n^3)$ per call to `characteristic_polynomial()`, where n is the dimension of the matrix.

Space Complexity: $O(n^2)$ auxiliary heap space.

```

#include <cstdint>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t b, int64_t e, int64_t m) {
    int64_t res = 1;

```

```

for (b %= m; e > 0; e >>= 1) {
    if (e & 1) {
        res = res * b % m;
    }
    b = b * b % m;
}
return res;
}

int64_t inv(int64_t a) {
    return powmod((a % MOD + MOD) % MOD, MOD - 2, MOD);
}

void to_hessenberg(std::vector<std::vector<int64_t>> &m) {
    int n = static_cast<int>(m.size());
    for (int j = 0; j + 2 < n; j++) {
        int pivot = -1;
        for (int i = j + 1; i < n; i++) {
            if (m[i][j] != 0) {
                pivot = i;
                break;
            }
        }
        if (pivot == -1) {
            continue;
        }
        if (pivot != j + 1) { // Move the pivot to the subdiagonal via a similarity swap.
            std::swap(m[pivot], m[j + 1]);
            for (int r = 0; r < n; r++) {
                std::swap(m[r][pivot], m[r][j + 1]);
            }
        }
        int64_t pinv = inv(m[j + 1][j]);
        for (int k = j + 2; k < n; k++) {
            int64_t u = m[k][j] * pinv % MOD;
            if (u == 0) {
                continue;
            }
            for (int c = 0; c < n; c++) { // Row k -= u * row (j + 1).
                m[k][c] = (m[k][c] - u * m[j + 1][c] % MOD + MOD) % MOD;
            }
            for (int r = 0; r < n; r++) { // Inverse column operation keeps it a similarity.
                m[r][j + 1] = (m[r][j + 1] + u * m[r][k]) % MOD;
            }
        }
    }
}

std::vector<int64_t> characteristic_polynomial(std::vector<std::vector<int64_t>> a) {
    int n = static_cast<int>(a.size());
    for (auto &row : a) {
        for (int64_t &x : row) {
            x = (x % MOD + MOD) % MOD;
        }
    }
    to_hessenberg(a);
    std::vector<std::vector<int64_t>> p(n + 1);
    p[0] = {1};
    for (int k = 0; k < n; k++) {
        p[k + 1].assign(k + 2, 0);
        for (int i = 0; i < static_cast<int>(p[k].size()); i++) {
            p[k + 1][i + 1] = (p[k + 1][i + 1] + p[k][i]) % MOD; // x * p[k].
            p[k + 1][i] = (p[k + 1][i] - a[k][k] * p[k][i] % MOD + MOD) % MOD; // -a[k][k] * p[k].
        }
        int64_t t = 1;
        for (int i = 0; i < k; i++) {
            t = t * a[k - i][k - i - 1] % MOD; // Running product of subdiagonal entries.
            int64_t coef = t * a[k - i - 1][k] % MOD;
        }
    }
}

```

```

    for (int j = 0; j < static_cast<int>(p[k - i - 1].size()); j++) {
        p[k + 1][j] = (p[k + 1][j] - coef * p[k - i - 1][j] % MOD + MOD) % MOD;
    }
}
return p[n];
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // [[1, 2], [3, 4]]: det(xI - A) = x^2 - 5x - 2.
    vector<vector<int64_t>> a{{1, 2}, {3, 4}};
    vector<int64_t> c = characteristic_polynomial(a);
    assert((c == vector<int64_t>{MOD - 2, MOD - 5, 1})); // -2, -5, 1.

    // For any matrix, c_{n-1} = -trace and c_0 = (-1)^n det.
    vector<vector<int64_t>> b{{2, 0, 1}, {0, 3, 0}, {4, 1, 5}};
    vector<int64_t> cb = characteristic_polynomial(b);
    assert(cb.size() == 4 && cb[3] == 1);
    assert(cb[2] == (MOD - (2 + 3 + 5)) % MOD); // Negated trace.
    // det(B) = 2(15) - 0 + 1(0 - 12) = 18, so c_0 = (-1)^3 * 18 = -18.
    assert(cb[0] == (MOD - 18) % MOD);
    return 0;
}

```

6.5.6 Matrix-Tree Theorem

Kirchhoff's matrix-tree theorem counts the spanning trees of an undirected graph. Form the Laplacian matrix $L = D - A$, where D is the diagonal matrix of degrees and A is the adjacency matrix (with multiplicities for parallel edges). Every cofactor of L is equal, and that common value is the number of spanning trees. Equivalently, delete any one row and the matching column from L and take the determinant of what remains.

The determinant is evaluated modulo a prime by Gaussian elimination, so the count is returned modulo `MOD`. Spanning-tree counts grow astronomically, so a modular count is the usual contest form; run the routine under a few different primes and combine with the Chinese remainder theorem to recover a large exact value. The same construction handles weighted graphs (sum edge weights into the Laplacian to get the weighted tree count) and, with the directed Laplacian and a fixed deleted row, counts rooted spanning arborescences.

- `count_spanning_trees(n, edges)` returns the number of spanning trees of the undirected graph on nodes $[0, n)$ with the given edge list, modulo `MOD`. Parallel edges are honored; self-loops do not affect the count. The result is 0 when the graph is disconnected, and 1 when n is 1.

Time Complexity: $O(n^3)$ per call to `count_spanning_trees()`, where n is the number of nodes.

Space Complexity: $O(n^2)$ auxiliary heap space.

```

#include <cstdint>
#include <utility>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t b, int64_t e, int64_t m) {
    int64_t res = 1;
    for (b %= m; e > 0; e >>= 1) {
        if (e & 1) {
            res = res * b % m;
        }
    }
}

```

```

    }
    b = b * b % m;
}
return res;
}

int64_t count_spanning_trees(int n, const std::vector<std::pair<int, int>> &edges) {
    if (n <= 1) {
        return n == 1 ? 1 : 0;
    }
    // Build the Laplacian, then drop the last row and column to form an (n-1) by (n-1) cofactor.
    int m = n - 1;
    std::vector<std::vector<int64_t>> lap(m, std::vector<int64_t>(m, 0));
    for (const auto &e : edges) {
        int u = e.first, v = e.second;
        if (u == v) {
            continue; // Self-loops contribute nothing to the Laplacian.
        }
        if (u < m) {
            lap[u][u] = (lap[u][u] + 1) % MOD;
        }
        if (v < m) {
            lap[v][v] = (lap[v][v] + 1) % MOD;
        }
        if (u < m && v < m) {
            lap[u][v] = (lap[u][v] - 1 + MOD) % MOD;
            lap[v][u] = (lap[v][u] - 1 + MOD) % MOD;
        }
    }
    // Determinant modulo MOD by Gaussian elimination.
    int64_t det = 1;
    for (int col = 0; col < m; col++) {
        int pivot = -1;
        for (int r = col; r < m; r++) {
            if (lap[r][col] != 0) {
                pivot = r;
                break;
            }
        }
        if (pivot == -1) {
            return 0; // Singular cofactor: the graph is disconnected.
        }
        if (pivot != col) {
            std::swap(lap[pivot], lap[col]);
            det = (MOD - det) % MOD;
        }
        det = det * lap[col][col] % MOD;
        int64_t pinv = powmod(lap[col][col], MOD - 2, MOD);
        for (int r = col + 1; r < m; r++) {
            int64_t factor = lap[r][col] * pinv % MOD;
            if (factor == 0) {
                continue;
            }
            for (int k = col; k < m; k++) {
                lap[r][k] = (lap[r][k] - factor * lap[col][k]) % MOD;
                if (lap[r][k] < 0) {
                    lap[r][k] += MOD;
                }
            }
        }
    }
    return det;
}

```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // A path 0-1-2 and a single triangle each have a known number of spanning trees.
    assert(count_spanning_trees(3, {{0, 1}, {1, 2}}) == 1); // A tree has exactly one.
    assert(count_spanning_trees(3, {{0, 1}, {1, 2}, {2, 0}}) == 3); // Triangle: drop any one edge.

    // Complete graphs: Cayley's formula gives  $n^{n-2}$  spanning trees.
    auto complete = [](int n) {
        vector<pair<int, int>> e;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                e.emplace_back(i, j);
            }
        }
        return count_spanning_trees(n, e);
    };
    assert(complete(4) == 16); //  $4^2$ .
    assert(complete(5) == 125); //  $5^3$ .

    // Disconnected graphs have no spanning tree.
    assert(count_spanning_trees(4, {{0, 1}, {2, 3}}) == 0);
    return 0;
}
```

6.5.7 Sparse Matrix

A sparse matrix stores only nonzero entries, which is useful when the matrix is large but most values are zero. This implementation keeps both row maps and column maps in sync: `row(i)` can iterate all nonzeros in row `i`, while `column(j)` can iterate all nonzeros in column `j`. That bidirectional storage is especially useful for sparse elimination or graph-like matrix operations, where both row updates and column pivot lookups are common.

- `SparseMatrix<T>(h, w)` constructs an `h` by `w` sparse matrix.
- `height()` returns the number of rows.
- `width()` returns the number of columns.
- `nonzeros()` returns the number of stored nonzero entries.
- `get(i, j)` returns the value at row `i`, column `j`, or 0 if the entry is not stored.
- `set(i, j, value)` assigns an entry. Assigning zero erases it from both maps.
- `add(i, j, delta)` adds `delta` to an entry. If the result becomes zero, the entry is erased.
- `row(i)` returns a map from column index to value for the nonzero entries in row `i`.
- `column(j)` returns a map from row index to value for the nonzero entries in column `j`.
- `swap_rows(i, j)` swaps two rows while keeping the column maps synchronized.
- `transpose()` transposes the matrix in place.
- `multiply_vector(x)` returns the matrix-vector product with vector `x`.
- `row_reduce(a, limit)` converts columns `[0, limit)` of `a` to sparse row echelon form, returning the rank found in those columns.
- `det(a)` returns the determinant of a square sparse matrix.
- `sparse_rank(a)` returns the rank of a sparse matrix.
- `solve_system(a, b, &x)` solves the system $ax = b$, returning 0 for one solution, -1 for no solution, or -2 for infinitely many solutions. When one solution exists, `x` is populated.

The class assumes that `T{} is the additive zero and that value == T{} is a valid zero test. The elimination routines also require field-like arithmetic because they divide by pivots, so use types such as double, rational numbers, or modular integers. For floating-point matrices, replace is_zero() with an epsilon comparison if small roundoff values should be erased.`

Sparse elimination chooses, in each pivot column, the available pivot row with the fewest stored entries. This heuristic can reduce fill-in, although sparse elimination can still become dense on unfriendly inputs.

Time Complexity:

- $O(\log d)$ per call to `get()`, `set()`, and `add()`, where d is the number of nonzeros in the touched row or column.
- $O(n)$ per call to `transpose()` and `multiply_vector()`, where n is the number of stored nonzero entries.
- Sparse elimination costs $O(f \cdot \log d)$, where f is the number of entry updates performed after fill-in and d is a touched row or column size. In the worst case this is still cubic.
- $O(f \cdot \log d)$ for `det()`, `sparse_rank()`, and `solve_system()`.
- $O(1)$ per call to `height()`, `width()`, `nonzeros()`, `row()`, and `column()`.

Space Complexity:

- $O(n)$ storage.
- $O(1)$ auxiliary for `get()`, `set()`, `add()`, `row()`, `column()`, and `transpose()`.
- $O(h)$ auxiliary heap space for `multiply_vector()`.

```
#include <cassert>
#include <cstdint>
#include <map>
#include <set>
#include <utility>
#include <vector>

template<typename T>
class SparseMatrix {
    int h = 0, w = 0, stored = 0;
    std::vector<std::map<int, T>> rows, cols;

    bool is_zero(const T &value) const { return value == T{}; }

public:
    SparseMatrix(int h_, int w_) : h(h_), w(w_), rows(h_), cols(w_) {}

    int height() const { return h; }
    int width() const { return w; }
    int nonzeros() const { return stored; }

    T get(int i, int j) const {
        auto it = rows[i].find(j);
        return it == rows[i].end() ? T{} : it->second;
    }

    void set(int i, int j, const T &value) {
        auto it = rows[i].find(j);
        if (is_zero(value)) {
            if (it != rows[i].end()) {
                rows[i].erase(it);
                cols[j].erase(i);
                stored--;
            }
            return;
        }
        if (it == rows[i].end()) {
            stored++;
        }
        rows[i][j] = value;
        cols[j][i] = value;
    }

    void add(int i, int j, const T &delta) {
        if (is_zero(delta)) {
            return;
        }
    }
};
```

```

    set(i, j, get(i, j) + delta);
}

const std::map<int, T> &row(int i) const { return rows[i]; }
const std::map<int, T> &column(int j) const { return cols[j]; }

void swap_rows(int i, int j) {
    if (i == j) {
        return;
    }
    std::set<int> touched;
    for (const auto &[col, value] : rows[i]) {
        touched.insert(col);
    }
    for (const auto &[col, value] : rows[j]) {
        touched.insert(col);
    }
    for (int col : touched) {
        T x = get(i, col), y = get(j, col);
        set(i, col, y);
        set(j, col, x);
    }
}

void transpose() {
    std::swap(h, w);
    std::swap(rows, cols);
}

std::vector<T> multiply_vector(const std::vector<T> &x) const {
    assert(static_cast<int>(x.size()) == w);
    std::vector<T> res(h);
    for (int i = 0; i < h; i++) {
        for (const auto &[j, value] : rows[i]) {
            res[i] += value * x[j];
        }
    }
    return res;
}

};

template<typename T>
int row_reduce(SparseMatrix<T> &a, int limit) {
    assert(0 <= limit && limit <= a.width());
    int r = 0;
    for (int c = 0; c < limit && r < a.height(); c++) {
        int pivot = -1;
        for (const auto &[i, value] : a.column(c)) {
            if (i >= r && (pivot == -1 || a.row(i).size() < a.row(pivot).size())) {
                pivot = i;
            }
        }
        if (pivot == -1) {
            continue;
        }
        a.swap_rows(r, pivot);
        T inv_pivot = T{1} / a.get(r, c);
        std::vector<std::pair<int, T>> pivot_row(a.row(r).begin(), a.row(r).end());
        std::vector<int> touched_rows;
        for (const auto &[i, value] : a.column(c)) {
            if (i > r) {
                touched_rows.push_back(i);
            }
        }
        for (int i : touched_rows) {
            T coeff = -a.get(i, c) * inv_pivot;
            for (const auto &[j, value] : pivot_row) {
                if (j != c) {

```



```

        a.add(i, j, coeff * value);
    }
}
a.set(i, c, T{});
}
r++;
}
return r;
}

template<typename T>
T det(SparseMatrix<T> a) {
    assert(a.height() == a.width());
    int swaps = 0, r = 0, n = a.height();
    T det = 1;
    for (int c = 0; c < n && r < n; c++) {
        int pivot = -1;
        for (const auto &[i, value] : a.column(c)) {
            if (i >= r && (pivot == -1 || a.row(i).size() < a.row(pivot).size())) {
                pivot = i;
            }
        }
        if (pivot == -1) {
            return T{};
        }
        if (pivot != r) {
            a.swap_rows(r, pivot);
            swaps++;
        }
        T pivot_value = a.get(r, c);
        det *= pivot_value;
        T inv_pivot = T{1} / pivot_value;
        std::vector<std::pair<int, T>> pivot_row(a.row(r).begin(), a.row(r).end());
        std::vector<int> touched_rows;
        for (const auto &[i, value] : a.column(c)) {
            if (i > r) {
                touched_rows.push_back(i);
            }
        }
        for (int i : touched_rows) {
            T coeff = -a.get(i, c) * inv_pivot;
            for (const auto &[j, value] : pivot_row) {
                if (j != c) {
                    a.add(i, j, coeff * value);
                }
            }
            a.set(i, c, T{});
        }
        r++;
    }
    return (r == n ? (swaps % 2 == 0 ? det : -det) : T{});
}

template<typename T>
int sparse_rank(SparseMatrix<T> a) {
    return row_reduce(a, a.width());
}

template<typename T>
int solve_system(SparseMatrix<T> a, const std::vector<T> &b, std::vector<T> *x) {
    if (x == nullptr || a.height() != static_cast<int>(b.size())) {
        return -1;
    }
    int h = a.height(), w = a.width();
    SparseMatrix<T> aug(h, w + 1);
    for (int i = 0; i < h; i++) {
        for (const auto &[j, value] : a.row(i)) {
            aug.set(i, j, value);
        }
    }
}

```

```

    }
    aug.set(i, w, b[i]);
}
row_reduce(aug, w);
std::vector<int> pivot_col;
for (int i = 0; i < h; i++) {
    int lead = -1;
    for (const auto &[j, value] : aug.row(i)) {
        if (j < w) {
            lead = j;
            break;
        }
    }
    if (lead == -1) {
        if (aug.get(i, w) != T{}) {
            return -1;
        }
    } else {
        pivot_col.push_back(lead);
    }
}
if (static_cast<int>(pivot_col.size()) < w) {
    return -2;
}
x->assign(w, T{});
for (int row = static_cast<int>(pivot_col.size()) - 1; row >= 0; row--) {
    int c = pivot_col[row];
    T rhs = aug.get(row, w);
    for (const auto &[j, value] : aug.row(row)) {
        if (j > c && j < w) {
            rhs -= value * (*x)[j];
        }
    }
    (*x)[c] = rhs / aug.get(row, c);
}
return 0;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    SparseMatrix<int64_t> a(3, 4);
    a.set(0, 1, 5);
    a.set(2, 3, 7);
    a.add(0, 1, -2);
    a.add(1, 2, 4);

    assert(a.height() == 3 && a.width() == 4);
    assert(a.nonzeros() == 3);
    assert(a.get(0, 1) == 3);
    assert(a.get(0, 0) == 0);
    assert(a.row(0).begin()->first == 1);
    assert(a.column(2).begin()->first == 1);

    vector<int64_t> x{10, 20, 30, 40};
    assert((a.multiply_vector(x) == vector<int64_t>{60, 120, 280}));

    a.add(0, 1, -3); // Entries that become zero are removed from both row and column maps.
    assert(a.get(0, 1) == 0);
    assert(a.nonzeros() == 2);
    assert(a.row(0).empty());
    assert(a.column(1).empty());
}

```

```

a.transpose();
assert(a.height() == 4 && a.width() == 3);
assert(a.get(3, 2) == 7);
assert(a.get(2, 1) == 4);

SparseMatrix<double> d(3, 3);
d.set(0, 0, 2);
d.set(0, 1, 1);
d.set(1, 1, 3);
d.set(2, 0, 1);
d.set(2, 2, 4);
double determinant = det(d);
assert(determinant > 24 - 1e-9 && determinant < 24 + 1e-9);
assert(sparse_rank(d) == 3);

SparseMatrix<double> sys(3, 3);
sys.set(0, 0, -1);
sys.set(0, 1, 2);
sys.set(0, 2, 5);
sys.set(1, 0, 1);
sys.set(1, 2, -6);
sys.set(2, 0, -4);
sys.set(2, 1, 2);
sys.set(2, 2, 2);
vector<double> rhs{3, 1, -2}, sol;
assert(solve_system(sys, rhs, &sol) == 0);
for (int i = 0; i < sys.height(); i++) {
    double sum = 0;
    for (const auto &[j, value] : sys.row(i)) {
        sum += value * sol[j];
    }
    assert(sum > rhs[i] - 1e-9 && sum < rhs[i] + 1e-9);
}
return 0;
}

```

6.6 Polynomials

6.6.1 Number Theoretic Transform

The number theoretic transform (NTT) is the discrete Fourier transform carried out in modular arithmetic instead of over the complex numbers. It multiplies two polynomials (equivalently, computes the convolution of two sequences) modulo a prime in $O(n \log n)$ time, with no floating point error. The modulus must be of the form $c \cdot 2^k + 1$ so that the ring of integers modulo it contains a 2^k -th root of unity, which limits transform sizes to powers of two up to 2^k . The code below uses the common prime $998244353 = 119 \cdot 2^{23} + 1$ with primitive root 3, supporting transforms of any power-of-two length up to 2^{23} .

The transform replaces a coefficient vector with its evaluations at the powers of a root of unity; pointwise multiplying two such evaluation vectors and transforming back gives their convolution. This is the same divide-and-conquer butterfly as the fast Fourier transform, with the root of unity and all arithmetic taken modulo the prime.

- `ntt(a, invert)` transforms the vector `a` in place, whose length must be a power of two. The forward transform uses `invert = false`; the inverse uses `invert = true`.
- `convolve(a, b)` returns the convolution of `a` and `b` modulo the prime, that is, the coefficients of the product of the two polynomials whose coefficients are the inputs. The result has length `a.size() + b.size() - 1`, or is empty if either input is empty. Input values are assumed to lie in $[0, \text{MOD})$. Small inputs use direct multiplication to avoid NTT overhead.

Time Complexity:

- $O(n \log n)$ per call to `ntt()`, where n is the length of the vector.
- $O(|a||b|)$ for the small-input branch of `convolve()`, otherwise $O(n \log n)$, where n is the padded transform length.

Space Complexity:

- $O(1)$ auxiliary for `ntt()`, transforming in place.
- $O(n)$ auxiliary heap space for `convolve()`.

```

#include <algorithm>
#include <cassert>
#include <stdint>
#include <vector>

const int64_t MOD = 998244353;
const int64_t ROOT = 3;
const int MAX_POWER_OF_TWO = 23;
const int NAIVE_CUTOFF = 150;

int64_t powmod(int64_t b, int64_t e, int64_t m) {
    int64_t res = 1;
    for (b %= m; e > 0; e >>= 1) {
        if (e & 1) {
            res = res * b % m;
        }
        b = b * b % m;
    }
    return res;
}

void ntt(std::vector<int64_t> &a, bool invert) {
    int n = static_cast<int>(a.size());
    assert((n & (n - 1)) == 0 && n <= (1 << MAX_POWER_OF_TWO));
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) {
            std::swap(a[i], a[j]);
        }
    }
    for (int len = 2; len <= n; len <<= 1) {
        int64_t root = powmod(ROOT, (MOD - 1) / len, MOD);
        if (invert) {
            root = powmod(root, MOD - 2, MOD);
        }
        for (int i = 0; i < n; i += len) {
            int64_t w = 1;
            for (int k = 0; k < len / 2; k++) {
                int64_t u = a[i + k], v = a[i + k + len / 2] * w % MOD;
                a[i + k] = (u + v) % MOD;
                a[i + k + len / 2] = (u - v + MOD) % MOD;
                w = w * root % MOD;
            }
        }
    }
    if (invert) {
        int64_t n_inv = powmod(n, MOD - 2, MOD);
        for (int64_t &x : a) {
            x = x * n_inv % MOD;
        }
    }
}

```

```

std::vector<int64_t> convolve(std::vector<int64_t> a, std::vector<int64_t> b) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int result_size = a.size() + b.size() - 1;
    if (std::min(a.size(), b.size()) < NAIVE_CUTOFF) {
        std::vector<int64_t> res(result_size);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            for (int j = 0; j < static_cast<int>(b.size()); j++) {
                res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
            }
        }
        return res;
    }
    int n = 1;
    while (n < result_size) {
        n <<= 1;
    }
    assert(n <= (1 << MAX_POWER_OF_TWO));
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * b[i] % MOD;
    }
    ntt(a, true);
    a.resize(result_size);
    return a;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // (1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4.
    vector<int64_t> a{1, 2, 3}, b{4, 5, 6};
    vector<int64_t> c = convolve(a, b);
    assert((c == vector<int64_t>{4, 13, 28, 27, 18}));
    vector<int64_t> ones(160, 1);
    vector<int64_t> d = convolve(ones, ones);
    assert(d[0] == 1 && d[159] == 160 && d[318] == 1);
    return 0;
}

```

6.6.2 Fast Fourier Transform

The fast Fourier transform (FFT) multiplies two polynomials, or equivalently convolves two sequences, in $O(n \log n)$ time by evaluating them at the complex roots of unity. Like the number theoretic transform of the previous section, it replaces a coefficient vector with its evaluations at the powers of a root of unity, multiplies two such evaluation vectors pointwise, and transforms back. The difference is the arithmetic: the FFT works over the complex numbers with $e^{2\pi i/n}$ as the root of unity, so it convolves arbitrary real coefficients rather than residues modulo a prime.

The trade-off is precision. Complex arithmetic in double precision accumulates rounding error, so the result is recovered by rounding each output to the nearest integer; this is reliable as long as the true coefficients stay well within the roughly 15 significant decimal digits a `double` can hold. When exact results modulo a prime are

required, or coefficients are large, prefer the number theoretic transform; use the FFT for real-valued convolution or big-integer multiplication.

- `fft(a, invert)` transforms the complex vector `a` in place, whose length must be a power of two. The forward transform uses `invert = false`; the inverse uses `invert = true`.
- `convolve(a, b)` returns the convolution of two integer sequences `a` and `b`, that is, the coefficients of the product of the two polynomials whose coefficients are the inputs. The result has length `a.size() + b.size() - 1`, or is empty if either input is empty, with each entry rounded to the nearest integer. Small inputs use direct multiplication to avoid FFT overhead.

Time Complexity:

- $O(n \log n)$ per call to `fft()`, where n is the length of the vector.
- $O(|a||b|)$ for the small-input branch of `convolve()`, otherwise $O(n \log n)$, where n is the padded transform length.

Space Complexity:

- $O(1)$ auxiliary for `fft()`, transforming in place.
- $O(n)$ auxiliary heap space for `convolve()`.

```
#include <algorithm>
#include <cmath>
#include <complex>
#include <cstdint>
#include <vector>

typedef std::complex<double> cd;
const double PI = acos(-1.0);
const int NAIVE_CUTOFF = 150;

void fft(std::vector<cd> &a, bool invert) {
    int n = static_cast<int>(a.size());
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) {
            std::swap(a[i], a[j]);
        }
    }
    for (int len = 2; len <= n; len <= 1) {
        double angle = 2 * PI / len * (invert ? -1 : 1);
        cd root(cos(angle), sin(angle));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int k = 0; k < len / 2; k++) {
                cd u = a[i + k], v = a[i + k + len / 2] * w;
                a[i + k] = u + v;
                a[i + k + len / 2] = u - v;
                w *= root;
            }
        }
    }
    if (invert) {
        for (cd &x : a) {
            x /= n;
        }
    }
}

std::vector<int64_t> convolve(const std::vector<int64_t> &a, const std::vector<int64_t> &b) {
    if (a.empty() || b.empty()) {
        return {};
    }
}
```

```

}
int result_size = a.size() + b.size() - 1;
if (std::min(a.size(), b.size()) < NAIVE_CUTOFF) {
    std::vector<int64_t> res(result_size);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        for (int j = 0; j < static_cast<int>(b.size()); j++) {
            res[i + j] += a[i] * b[j];
        }
    }
    return res;
}
int n = 1;
while (n < result_size) {
    n <<= 1;
}
std::vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
fa.resize(n);
fb.resize(n);
fft(fa, false);
fft(fb, false);
for (int i = 0; i < n; i++) {
    fa[i] *= fb[i];
}
fft(fa, true);
std::vector<int64_t> res(result_size);
for (int i = 0; i < result_size; i++) {
    res[i] = llround(fa[i].real());
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // (1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4.
    vector<int64_t> a{1, 2, 3}, b{4, 5, 6};
    vector<int64_t> c = convolve(a, b);
    assert((c == vector<int64_t>{4, 13, 28, 27, 18}));
    vector<int64_t> ones(160, 1);
    vector<int64_t> d = convolve(ones, ones);
    assert(d[0] == 1 && d[159] == 160 && d[318] == 1);
    return 0;
}

```

6.6.3 Polynomial Operations

Polynomial operations treat a vector `a` as coefficients of $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ over a field. This section implements the core algebra needed for formal power series and polynomial division modulo the prime 998244353. Multiplication uses the number theoretic transform, so inverse and division are fast enough for large polynomials while still keeping the API close to the underlying coefficient vectors.

The code aliases one modular field element as `Coeff` and one coefficient vector as `Poly`.

- `trim(a)` removes trailing zero coefficients from `a`.
- `add(a, b)` returns `a + b`.
- `subtract(a, b)` returns `a - b`.
- `multiply(a, b)` returns `a * b`.
- `derivative(a)` returns the formal derivative of `a`.

- `integral(a)` returns the formal antiderivative of `a` with constant term zero.
- `inverse(a, n)` returns the first `n` coefficients of `1 / a`, requiring `a[0]` to be nonzero.
- `divide(a, b)` returns the polynomial quotient of `a / b`, requiring `b` to be nonzero.
- `modulo(a, b)` returns the polynomial remainder of `a / b`, requiring `b` to be nonzero.

The inverse is a formal power series inverse modulo x^n : it finds b such that $a \cdot b \equiv 1 \pmod{x^n}$. Division uses the standard reversal trick: reverse both polynomials, compute a truncated series inverse of the reversed divisor, multiply, truncate, and reverse back.

Time Complexity:

- $O(n)$ for `trim()`, `add()`, `subtract()`, `derivative()`, and `integral()`.
- $O(|a||b|)$ for the small-input branch of `multiply()`, otherwise $O(n \log n)$, where n is the padded transform length.
- $O(n \log^2 n)$ for `inverse(a, n)`, where n is the requested length.
- $O(n \log n + m \log^2 m)$ for `divide()` and `modulo()`, where n is the output quotient length and m is the divisor length.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <vector>

using Coeff = int64_t;
using Poly = std::vector<Coeff>;

const Coeff MOD = 998244353;
const Coeff ROOT = 3;
const int MAX_POWER_OF_TWO = 23;
const int NAIVE_CUTOFF = 150;

Coeff powmod(Coeff b, Coeff e, Coeff m = MOD) {
    Coeff res = 1;
    for (b %= m; e > 0; e >>= 1) {
        if (e & 1) {
            res = res * b % m;
        }
        b = b * b % m;
    }
    return res;
}

void ntt(Poly &a, bool invert) {
    int n = static_cast<int>(a.size());
    assert((n & (n - 1)) == 0 && n <= (1 << MAX_POWER_OF_TWO));
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if (i < j) {
            std::swap(a[i], a[j]);
        }
    }
    for (int len = 2; len <= n; len <<= 1) {
        Coeff root = powmod(ROOT, (MOD - 1) / len);
        if (invert) {
            root = powmod(root, MOD - 2);
        }
        for (int i = 0; i < n; i += len) {
            Coeff w = 1;
            for (int k = 0; k < len / 2; k++) {
                Coeff u = a[i + k], v = a[i + k + len / 2] * w % MOD;
```



```

        a[i + k] = (u + v) % MOD;
        a[i + k + len / 2] = (u - v + MOD) % MOD;
        w = w * root % MOD;
    }
}
}
if (invert) {
    Coeff n_inv = powmod(n, MOD - 2);
    for (Coeff &x : a) {
        x = x * n_inv % MOD;
    }
}
}

void trim(Poly &a) {
    while (!a.empty() && a.back() == 0) {
        a.pop_back();
    }
}

Poly add(const Poly &a, const Poly &b) {
    Poly res(std::max(a.size(), b.size()));
    for (int i = 0; i < static_cast<int>(res.size()); i++) {
        Coeff x = i < static_cast<int>(a.size()) ? a[i] : 0;
        Coeff y = i < static_cast<int>(b.size()) ? b[i] : 0;
        res[i] = (x + y) % MOD;
    }
    trim(res);
    return res;
}

Poly subtract(const Poly &a, const Poly &b) {
    Poly res(std::max(a.size(), b.size()));
    for (int i = 0; i < static_cast<int>(res.size()); i++) {
        Coeff x = i < static_cast<int>(a.size()) ? a[i] : 0;
        Coeff y = i < static_cast<int>(b.size()) ? b[i] : 0;
        res[i] = (x - y + MOD) % MOD;
    }
    trim(res);
    return res;
}

Poly multiply(Poly a, Poly b) {
    if (a.empty() || b.empty()) {
        return {};
    }
    int result_size = a.size() + b.size() - 1;
    if (std::min(a.size(), b.size()) < NAIVE_CUTOFF) {
        Poly res(result_size);
        for (int i = 0; i < static_cast<int>(a.size()); i++) {
            for (int j = 0; j < static_cast<int>(b.size()); j++) {
                res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
            }
        }
        trim(res);
        return res;
    }
    int n = 1;
    while (n < result_size) {
        n <<= 1;
    }
    assert(n <= (1 << MAX_POWER_OF_TWO));
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * b[i] % MOD;
    }
}

```

```

    }
    ntt(a, true);
    a.resize(result_size);
    trim(a);
    return a;
}

Poly derivative(const Poly &a) {
    if (a.size() <= 1) {
        return {};
    }
    Poly res(a.size() - 1);
    for (int i = 1; i < static_cast<int>(a.size()); i++) {
        res[i - 1] = a[i] * i % MOD;
    }
    trim(res);
    return res;
}

Poly integral(const Poly &a) {
    Poly res(a.size() + 1);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        res[i + 1] = a[i] * powmod(i + 1, MOD - 2) % MOD;
    }
    return res;
}

Poly inverse(const Poly &a, int n) {
    assert(n >= 0 && !a.empty() && a[0] != 0);
    if (n == 0) {
        return {};
    }
    Poly res{powmod(a[0], MOD - 2)};
    while (static_cast<int>(res.size()) < n) {
        int len = static_cast<int>(res.size()) << 1;
        Poly cut(a.begin(), a.begin() + std::min(static_cast<int>(a.size()), len));
        Poly prod = multiply(multiply(res, res), cut);
        res.resize(len);
        for (int i = len / 2; i < std::min(len, static_cast<int>(prod.size())); i++) {
            res[i] = (MOD - prod[i]) % MOD;
        }
    }
    res.resize(n);
    return res;
}

Poly divide(Poly a, Poly b) {
    trim(a);
    trim(b);
    assert(!b.empty());
    if (a.size() < b.size()) {
        return {};
    }
    int quotient_size = static_cast<int>(a.size() - b.size() + 1);
    std::reverse(a.begin(), a.end());
    std::reverse(b.begin(), b.end());
    b.resize(quotient_size);
    Poly q = multiply(a, inverse(b, quotient_size));
    q.resize(quotient_size);
    std::reverse(q.begin(), q.end());
    trim(q);
    return q;
}

Poly modulo(const Poly &a, const Poly &b) {
    Poly q = divide(a, b);
    Poly res = subtract(a, multiply(q, b));
    if (res.size() >= b.size()) {

```

```

    res.resize(b.size() - 1);
}
trim(res);
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // (1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4.
    Poly a{1, 2, 3}, b{4, 5, 6};
    assert((add(a, b) == Poly{5, 7, 9}));
    assert((subtract(b, a) == Poly{3, 3, 3}));
    assert((multiply(a, b) == Poly{4, 13, 28, 27, 18}));

    // d/dx (5 + 7x + 11x^2) = 7 + 22x; integrating back chooses constant term 0.
    Poly p{5, 7, 11};
    assert((derivative(p) == Poly{7, 22}));
    assert((integral(derivative(p)) == Poly{0, 7, 11}));

    // (1 - x)^-1 = 1 + x + x^2 + x^3 + ... modulo x^6.
    Poly inv = inverse({1, MOD - 1}, 6);
    assert((inv == Poly{1, 1, 1, 1, 1, 1}));

    // (x^3 - 1) / (x - 1) = x^2 + x + 1, remainder 0.
    Poly dividend{MOD - 1, 0, 0, 1};
    Poly divisor{MOD - 1, 1};
    assert((divide(dividend, divisor) == Poly{1, 1, 1}));
    assert(modulo(dividend, divisor).empty());

    Poly ones(160, 1);
    Poly square = multiply(ones, ones);
    assert(square[0] == 1 && square[159] == 160 && square[318] == 1);
    return 0;
}

```

6.6.4 Berlekamp-Massey

The Berlekamp-Massey algorithm finds the shortest linear recurrence that generates a given sequence over a field. Given the first terms of a sequence taken modulo a prime, it returns coefficients $\Lambda_0, \dots, \Lambda_{L-1}$ of the minimal recurrence $s_i = \Lambda_0 s_{i-1} + \Lambda_1 s_{i-2} + \dots + \Lambda_{L-1} s_{i-L} \pmod{p}$, holding for every $i \geq L$. It scans the sequence once, maintaining a current candidate recurrence and correcting it whenever a term fails to match the prediction, using the last position where a correction was needed. The order L never decreases, and $2L + 1$ terms suffice to certify a recurrence of order L . This is the standard tool for guessing a linear recurrence from sampled values, after which the sequence can be extended or its n -th term found by fast methods.

A recurrence of order L is only certified by at least $2L + 1$ terms, so the result is trustworthy only when $\mathbf{s}$ is comfortably longer than twice the returned length. If the returned L is close to $n/2$, the sequence is likely under-determined and the recurrence may be a spurious overfit.

The code operates modulo the prime 998244353; change `MOD` to use a different prime field.

- `berlekamp_massey(s)` returns the coefficients of the shortest recurrence that the sequence $\mathbf{s}$ satisfies, as described above. The returned length is the order L of the recurrence. An empty vector is returned when $\mathbf{s}$ is entirely zero (no recurrence is needed).

Time Complexity: $O(n^2 + L \log p)$ per call, where n is the length of $\mathbf{s}$, L is the order of the recurrence found, and p is the modulus.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <cstdint>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t b, int64_t e, int64_t m) {
    int64_t res = 1;
    for (b %= m; e > 0; e >>= 1) {
        if (e & 1) {
            res = res * b % m;
        }
        b = b * b % m;
    }
    return res;
}

std::vector<int64_t> berlekamp_massey(const std::vector<int64_t> &s) {
    int n = static_cast<int>(s.size()), len = 0, m = 0;
    std::vector<int64_t> cur(n), last(n), prev;
    cur[0] = last[0] = 1;
    int64_t inv_last_delta = 1;
    for (int i = 0; i < n; i++) {
        m++;
        int64_t delta = s[i] % MOD;
        for (int j = 1; j <= len; j++) {
            delta = (delta + cur[j] * (s[i - j] % MOD)) % MOD;
        }
        if (delta == 0) {
            continue;
        }
        prev = cur;
        int64_t coef = delta * inv_last_delta % MOD;
        for (int j = m; j < n; j++) {
            cur[j] = (cur[j] - coef * last[j - m] % MOD + MOD) % MOD;
        }
        if (2 * len > i) {
            continue;
        }
        len = i + 1 - len;
        last = prev;
        inv_last_delta = powmod(delta, MOD - 2, MOD); // Refresh only when the length increases.
        m = 0;
    }
    cur.resize(len + 1);
    cur.erase(cur.begin());
    for (int64_t &x : cur) {
        x = (MOD - x) % MOD;
    }
    return cur;
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Fibonacci: s[i] = s[i-1] + s[i-2].
    vector<int64_t> fib{1, 1, 2, 3, 5, 8, 13, 21};
    assert((berlekamp_massey(fib) == vector<int64_t>{1, 1}));

    // Geometric: s[i] = 2 * s[i-1].
    vector<int64_t> geo{1, 2, 4, 8, 16, 32};
    assert((berlekamp_massey(geo) == vector<int64_t>{2}));
}
```

```
// Tribonacci: s[i] = s[i-1] + s[i-2] + s[i-3].
vector<int64_t> trib{0, 1, 1, 2, 4, 7, 13, 24, 44};
assert((berlekamp_massey(trib) == vector<int64_t>{1, 1, 1}));
return 0;
}
```

6.6.5 Linear Recurrence (Kitamasa)

Computes the n -th term of a linear recurrence modulo a prime in logarithmic time. Given a recurrence of order L , $s_i = c_0 s_{i-1} + c_1 s_{i-2} + \dots + c_{L-1} s_{i-L}$, together with the first L terms, the naive approach unrolls n steps. Kitamasa's method instead computes n terms ahead in $O(L^2 \log n)$ by working with the characteristic polynomial. The coefficient layout matches the output of Berlekamp-Massey, so the two compose directly: guess the recurrence from sampled values, then jump to any index.

The key identity is that the n -th term is a fixed linear combination of the first L terms, with weights given by x^n reduced modulo the characteristic polynomial $f(x) = x^L - c_0 x^{L-1} - \dots - c_{L-1}$. The reduction $x^L = c_0 x^{L-1} + \dots + c_{L-1}$ lets any product of two degree- $< L$ polynomials be folded back to degree $< L$, so $x^n \bmod f$ is obtained by binary exponentiation, and the n -th term is the dot product of its coefficients with the initial terms.

- `kth_term(rec, init, n)` returns s_n modulo `MOD`, where `rec` holds the coefficients $c_0, \dots, c_{L-1}$ and `init` holds at least L initial terms $s_0, \dots, s_{L-1}$. Indexing is 0-based and `n` ≥ 0 . Values are reduced modulo `MOD` internally.

Time Complexity: $O(L^2 \log n)$ per call to `kth_term()`.

Space Complexity: $O(L)$ auxiliary heap space.

```
#include <cstdlib>
#include <vector>

const int64_t MOD = 998244353;

void reduce(std::vector<int64_t> &p, const std::vector<int64_t> &c) {
    int L = static_cast<int>(c.size());
    for (int d = static_cast<int>(p.size()) - 1; d >= L; d--) {
        if (p[d] != 0) {
            for (int j = 0; j < L; j++) {
                p[d - 1 - j] = (p[d - 1 - j] + p[d] * c[j]) % MOD;
            }
            p[d] = 0;
        }
    }
    p.resize(L);
}

std::vector<int64_t> combine(
    const std::vector<int64_t> &a, const std::vector<int64_t> &b, const std::vector<int64_t> &c
) {
    std::vector<int64_t> res(a.size() + b.size() - 1, 0);
    for (int i = 0; i < static_cast<int>(a.size()); i++) {
        for (int j = 0; j < static_cast<int>(b.size()); j++) {
            res[i + j] = (res[i + j] + a[i] * b[j]) % MOD;
        }
    }
    reduce(res, c);
    return res;
}

int64_t kth_term(const std::vector<int64_t> &rec, const std::vector<int64_t> &init, int64_t n) {
    int L = static_cast<int>(rec.size());
    if (n < static_cast<int64_t>(init.size())) {
```

```

    return ((init[n] % MOD) + MOD) % MOD;
}
if (L == 0) {
    return 0;
}
std::vector<int64_t> result(L, 0), base(L, 0);
result[0] = 1; // The polynomial 1 = x^0.
if (L == 1) {
    base[0] = rec[0] % MOD; // x is congruent to c_0 modulo f.
} else {
    base[1] = 1; // The polynomial x.
}
for (int64_t e = n; e > 0; e >= 1) {
    if (e & 1) {
        result = combine(result, base, rec);
    }
    base = combine(base, base, rec);
}
int64_t ans = 0;
for (int j = 0; j < L; j++) {
    ans = (ans + result[j] * (((init[j] % MOD) + MOD) % MOD)) % MOD;
}
return ans;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // Fibonacci s_i = s_{i-1} + s_{i-2} with s_0 = s_1 = 1: 1, 1, 2, 3, 5, 8, ...
    vector<int64_t> fib_rec{1, 1}, fib_init{1, 1};
    assert(kth_term(fib_rec, fib_init, 0) == 1);
    assert(kth_term(fib_rec, fib_init, 9) == 55);
    assert(kth_term(fib_rec, fib_init, 10) == 89);

    // Tribonacci s_i = s_{i-1} + s_{i-2} + s_{i-3} with s_0,s_1,s_2 = 0,1,1.
    vector<int64_t> trib_rec{1, 1, 1}, trib_init{0, 1, 1};
    assert(kth_term(trib_rec, trib_init, 8) == 44);

    // Geometric s_i = 3 s_{i-1}, s_0 = 1: matches 3^n modulo MOD.
    vector<int64_t> geo_rec{3}, geo_init{1};
    int64_t pw = 1;
    for (int i = 0; i < 50; i++) {
        assert(kth_term(geo_rec, geo_init, i) == pw);
        pw = pw * 3 % MOD;
    }
    return 0;
}

```

6.6.6 Polynomial Interpolation

Recovers the unique polynomial of degree less than n that passes through n given points, working modulo a prime. The x coordinates must be distinct; the y coordinates may be anything. Two operations are provided: reconstructing the full coefficient vector, and evaluating the interpolating polynomial at a single point without ever forming the coefficients.

Coefficient recovery uses Newton's divided differences. It first replaces the sample values by their successive divided differences in place, then expands the Newton basis $1, (x - x_0), (x - x_0)(x - x_1), \dots$ into ordinary coefficients by multiplying through one factor at a time. Point evaluation instead applies the Lagrange formula

directly, summing each sample weighted by the product of $(t - x_j)/(x_k - x_j)$ over the other nodes. Both run in $O(n^2)$ modular operations.

- `interpolate(x, y)` returns the coefficients $a_0, \dots, a_{n-1}$ (constant term first) of the polynomial P with $P(x[i]) = y[i]$, modulo `MOD`. The entries of `x` must be distinct modulo `MOD`.
- `interpolate_at(x, y, t)` returns $P(t)$ modulo `MOD` for that same polynomial, without building its coefficients.

Time Complexity: $O(n^2)$ per call to `interpolate()` and `interpolate_at()`, where n is the number of points.

Space Complexity: $O(n)$ auxiliary heap space.

```
#include <stdint>
#include <vector>

const int64_t MOD = 998244353;

int64_t powmod(int64_t b, int64_t e, int64_t m) {
    int64_t res = 1;
    for (b %= m; e > 0; e >>= 1) {
        if (e & 1) {
            res = res * b % m;
        }
        b = b * b % m;
    }
    return res;
}

int64_t inv(int64_t a) {
    return powmod(((a % MOD) + MOD) % MOD, MOD - 2, MOD);
}

std::vector<int64_t> interpolate(std::vector<int64_t> x, std::vector<int64_t> y) {
    int n = static_cast<int>(x.size());
    for (int i = 0; i < n; i++) {
        x[i] = ((x[i] % MOD) + MOD) % MOD;
        y[i] = ((y[i] % MOD) + MOD) % MOD;
    }
    // Replace y by its divided differences in place.
    for (int k = 0; k < n; k++) {
        for (int i = k + 1; i < n; i++) {
            y[i] = (y[i] - y[k] + MOD) % MOD * inv((x[i] - x[k] + MOD) % MOD) % MOD;
        }
    }
    // Expand the Newton basis into ordinary coefficients.
    std::vector<int64_t> res(n, 0), temp(n, 0);
    temp[0] = 1;
    for (int k = 0; k < n; k++) {
        int64_t last = 0;
        for (int i = 0; i < n; i++) {
            res[i] = (res[i] + y[k] * temp[i]) % MOD;
            int64_t old = temp[i];
            temp[i] = last;
            last = old;
            temp[i] = (temp[i] - last * x[k] % MOD + MOD) % MOD;
        }
    }
    return res;
}

int64_t interpolate_at(const std::vector<int64_t> &x, const std::vector<int64_t> &y, int64_t t) {
    int n = static_cast<int>(x.size());
    t = ((t % MOD) + MOD) % MOD;
    int64_t res = 0;
    for (int k = 0; k < n; k++) {
        int64_t num = 1, den = 1;
        for (int j = 0; j < n; j++) {
```

```

    if (j != k) {
        num = num * ((t - x[j]) % MOD + MOD) % MOD;
        den = den * ((x[k] - x[j]) % MOD + MOD) % MOD;
    }
}
res = (res + ((y[k] % MOD) + MOD) % MOD * num % MOD * inv(den)) % MOD;
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    // P(x) = x^2 + 2x + 3 sampled at x = 0, 1, 2 gives y = 3, 6, 11.
    vector<int64_t> x{0, 1, 2}, y{3, 6, 11};
    vector<int64_t> coef = interpolate(x, y);
    assert((coef == vector<int64_t>{3, 2, 1}));
    assert(interpolate_at(x, y, 5) == 38); // 25 + 10 + 3.

    // Non-consecutive nodes for P(x) = 2x^3 - x + 4.
    auto poly = [](int64_t t) { return ((2 * t * t * t - t + 4) % MOD + MOD) % MOD; };
    vector<int64_t> xs{2, 5, 7, 10}, ys;
    for (int64_t v : xs) {
        ys.push_back(poly(v));
    }
    vector<int64_t> c = interpolate(xs, ys);
    assert((c == vector<int64_t>{4, MOD - 1, 0, 2})); // 4 - x + 2x^3.
    assert(interpolate_at(xs, ys, 13) == poly(13));
    return 0;
}

```

6.7 Game Theory

6.7.1 Nim

Solves normal-play nim and related impartial games where each position is the disjoint sum of independent piles. In normal play, the player who makes the last move wins. A nim position is losing exactly when the XOR of all pile sizes is 0.

This is the base case of the Sprague-Grundy theorem: every finite impartial game position is equivalent to a Nim pile of size equal to its Grundy number, and sums of games combine by XOR.

- `nim_sum(piles)` returns the XOR of all pile sizes.
- `first_player_wins(piles)` returns whether the player to move has a winning strategy.
- `winning_move(piles)` returns a move (pile, new_size) that moves to XOR 0, or $(-1, -1)$ if the position is losing.

Time Complexity: $O(n)$ per call, where n is the number of piles.

Space Complexity: $O(1)$ auxiliary.

```

#include <utility>
#include <vector>

int nim_sum(const std::vector<int> &piles) {
    int x = 0;
    for (int p : piles) {

```



```

    x ^= p;
}
return x;
}

bool first_player_wins(const std::vector<int> &piles) {
    return nim_sum(piles) != 0;
}

std::pair<int, int> winning_move(const std::vector<int> &piles) {
    int x = nim_sum(piles);
    if (x == 0) {
        return {-1, -1};
    }
    for (int i = 0; i < static_cast<int>(piles.size()); i++) {
        if (int target = piles[i] ^ x; target < piles[i]) {
            return {i, target};
        }
    }
    return {-1, -1};
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> piles{3, 4, 5};
    assert(nim_sum(piles) == 2);
    assert(first_player_wins(piles));
    auto [pile_idx, new_size] = winning_move(piles);
    piles[pile_idx] = new_size;
    assert(nim_sum(piles) == 0);

    vector<int> losing{1, 2, 3};
    assert(!first_player_wins(losing));
    assert(winning_move(losing).first == -1);
    return 0;
}

```

6.7.2 Nim Product

Nimbers are the values of impartial games, added by XOR. Under the additional operation of nim multiplication they form a field: the nonnegative integers below 2^{2^k} are closed under both operations, with XOR as addition and nim product as multiplication. Writing $\otimes$ for the nim product and $\oplus$ for XOR, nim multiplication is commutative, associative, and distributive over $\oplus$, and is defined recursively by $a \otimes b = \text{mex}\{(a' \otimes b) \oplus (a \otimes b') \oplus (a' \otimes b') : 0 \leq a' < a, 0 \leq b' < b\}$. Its main use is multiplying the Grundy values of independent “coin-turning” games whose move sets combine as a product, such as two-dimensional turning games built from one-dimensional ones.

Rather than evaluate the recursive definition directly, this implementation precomputes the nim product of every pair of single bits $2^i \otimes 2^j$ for $i, j < 64$. Each such product follows from the Fermat-like rule $2^{2^k} \otimes 2^{2^k} = 2^{2^k} \oplus 2^{2^k-1}$ and the fact that distinct Fermat powers multiply like ordinary powers of two. A full product then XORs together the bit-pair products selected by the set bits of the operands. Because every 64-bit value lies in the field of size 2^{64} , the operation is total on `uint64_t` and the nonzero values have multiplicative inverses.

- `nim_product(x, y)` returns the nim product of `x` and `y`.
- `nim_pow(b, e)` returns `b` raised to the `e`-th power under nim multiplication.
- `nim_inverse(b)` returns the multiplicative inverse of a nonzero `b`, so that `nim_product(b, nim_inverse(b)) == 1`.

Time Complexity:

- $O(1)$ table construction on first use (a fixed 64 by 64 table).
- $O(64^2)$ per call to `nim_product()`; $O(64^2 \log e)$ per call to `nim_pow()` and `nim_inverse()`.

Space Complexity: $O(1)$: a fixed 64 by 64 table of 64-bit products.

```
#include <stdint>

uint64_t bit_prod[64][64];

// Build the single-bit products 2^i (x) 2^j. Each entry depends only on entries with a smaller
// first index, so a single ascending pass fills the table.
const bool nim_product_init = [] {
    for (int i = 0; i < 64; i++) {
        for (int j = 0; j < 64; j++) {
            if ((i & j) == 0) {
                bit_prod[i][j] = uint64_t(1) << (i | j); // Distinct Fermat powers multiply like 2^(i|j).
            } else {
                int a = (i & j) & ~(i & j); // Lowest Fermat power shared by both exponents.
                bit_prod[i][j] = bit_prod[i ^ a][j ^ a] ^ bit_prod[(i ^ a) | (a - 1)][(j ^ a) | (i & (a - 1))];
            }
        }
    }
    return true;
}();

uint64_t nim_product(uint64_t x, uint64_t y) {
    uint64_t res = 0;
    for (int i = 0; i < 64 && (x >> i) != 0; i++) {
        if ((x >> i) & 1) {
            for (int j = 0; j < 64 && (y >> j) != 0; j++) {
                if ((y >> j) & 1) {
                    res ^= bit_prod[i][j];
                }
            }
        }
    }
    return res;
}

uint64_t nim_pow(uint64_t b, uint64_t e) {
    uint64_t res = 1;
    for (; e > 0; e >>= 1) {
        if (e & 1) {
            res = nim_product(res, b);
        }
        b = nim_product(b, b);
    }
    return res;
}

uint64_t nim_inverse(uint64_t b) {
    // The multiplicative group of the size-2^64 field has order 2^64 - 1, so b^(2^64 - 2) = b^{-1}.
    return nim_pow(b, static_cast<uint64_t>(-2));
}
```

Example Usage

```
#include <cassert>
using namespace std;

int main() {
    // Smallest nontrivial products: {0,1,2,3} form the field GF(4) under XOR and nim product.
    assert(nim_product(2, 2) == 3);
    assert(nim_product(2, 3) == 1);
}
```

```

assert(nim_product(3, 3) == 2);
assert(nim_product(1, 12345) == 12345); // 1 is the multiplicative identity.
assert(nim_product(0, 12345) == 0);

// Field axioms on a range of values: commutativity, distributivity over XOR, and inverses.
for (uint64_t a = 0; a < 64; a++) {
    for (uint64_t b = 0; b < 64; b++) {
        assert(nim_product(a, b) == nim_product(b, a));
        for (uint64_t c = 0; c < 8; c++) {
            assert(nim_product(a, b ^ c) == (nim_product(a, b) ^ nim_product(a, c)));
        }
    }
    if (a != 0) {
        assert(nim_product(a, nim_inverse(a)) == 1);
    }
}
assert(nim_product(0x1234567890abcdefULL, nim_inverse(0x1234567890abcdefULL)) == 1);
return 0;
}

```

6.7.3 Sprague-Grundy

Computes Grundy numbers for finite impartial games. The Grundy number of a game position is the minimum excluded nonnegative integer (MEX) of the Grundy numbers reachable in one move. A position is losing exactly when its Grundy number is 0.

For a sum of independent impartial games, the combined Grundy number is the XOR of the component Grundy numbers. This is the Sprague-Grundy theorem.

- `mex(values)` returns the minimum excluded nonnegative integer.
- `subtraction_game_grundy(max_stones, moves)` computes Grundy numbers for a game where a move subtracts one value in `moves` from the pile.
- `sum_grundy(grundies)` returns the XOR of component Grundy numbers.

Time Complexity: $O(n \cdot m)$ for `subtraction_game_grundy(max_stones, moves)`, where n is `max_stones` and m is the number of allowed moves.

Space Complexity: $O(n + m)$ auxiliary heap space.

```

#include <vector>

int mex(const std::vector<int> &values) {
    std::vector<bool> seen(values.size() + 1, false);
    for (int v : values) {
        if (0 <= v && v < static_cast<int>(seen.size())) {
            seen[v] = true;
        }
    }
    for (int i = 0; i < static_cast<int>(seen.size()); i++) {
        if (!seen[i]) {
            return i;
        }
    }
    return static_cast<int>(seen.size());
}

std::vector<int> subtraction_game_grundy(int max_stones, const std::vector<int> &moves) {
    std::vector<int> grundy(max_stones + 1, 0);
    for (int stones = 1; stones <= max_stones; stones++) {
        std::vector<int> reachable;
        for (int m : moves) {
            if (m <= stones) {
                reachable.push_back(grundy[stones - m]);
            }
        }
    }
}

```

```

    }
}
grundy[stones] = mex(reachable);
}
return grundy;
}

int sum_grundy(const std::vector<int> &grundies) {
    int x = 0;
    for (int g : grundies) {
        x ^= g;
    }
    return x;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<int> moves{1, 3, 4};
    auto g = subtraction_game_grundy(10, moves);
    assert(g[0] == 0);
    assert(g[1] == 1);
    assert(g[2] == 0);
    assert(g[4] == 2);
    assert(g[7] == 0);

    vector<int> heaps{g[5], g[7], g[10]};
    assert(sum_grundy(heaps) == (g[5] ^ g[7] ^ g[10]));
    return 0;
}

```

6.7.4 Grundy Numbers on DAG

Computes Grundy numbers for impartial games whose positions form a directed acyclic graph. Each vertex is a game position, and each outgoing edge is a legal move. Terminal vertices have Grundy number 0; all other vertices take the MEX of their successors' Grundy numbers.

The implementation uses memoized DFS. It assumes the graph is acyclic; if cycles are present, the usual finite impartial-game Grundy recurrence is not directly valid without additional analysis.

- `grundy_on_dag(g)` returns the Grundy number of every vertex in graph `g`, where `g[u]` contains all positions reachable from position `u` in one move.

Time Complexity: $O(n + m + s)$, where n is the number of vertices, m is the number of edges, and s is the total cost of MEX sets over outgoing edges.

Space Complexity: $O(n + d)$ auxiliary heap and stack space, where d is DFS depth.

```

#include <vector>

int grundy_dfs(int u, const std::vector<std::vector<int>> &g, std::vector<int> *memo) {
    if ((*memo)[u] != -1) {
        return (*memo)[u];
    }
    std::vector<bool> seen(g[u].size() + 1, false);
    for (int v : g[u]) {
        int x = grundy_dfs(v, g, memo);
        if (x < static_cast<int>(seen.size())) {
            seen[x] = true;
        }
    }
    int mex = 0;
    while (seen[mex]) mex++;
    (*memo)[u] = mex;
    return mex;
}

```

```

    }
}
int res = 0;
while (res < static_cast<int>(seen.size()) && seen[res]) {
    res++;
}
(*memo)[u] = res;
return res;
}

std::vector<int> GrundyOnDag(const std::vector<std::vector<int>> &g) {
    std::vector<int> memo(g.size(), -1);
    for (int u = 0; u < static_cast<int>(g.size()); u++) {
        GrundyDfs(u, g, &memo);
    }
    return memo;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<vector<int>> g{{1, 2}, {3}, {3, 4}, {}, {}};
    auto Grundy = GrundyOnDag(g);
    assert(Grundy[3] == 0);
    assert(Grundy[4] == 0);
    assert(Grundy[1] == 1);
    assert(Grundy[2] == 1);
    assert(Grundy[0] == 0);
    return 0;
}

```

6.7.5 Minimax and Alpha-Beta Pruning

Searches a finite two-player, zero-sum, perfect-information game tree using minimax and alpha-beta pruning. Minimax assumes both players play optimally: the maximizing player chooses the child with largest value, and the minimizing player chooses the child with smallest value.

Alpha-beta pruning computes the same value as minimax, but avoids exploring branches that cannot affect the final answer. It is most effective when good moves are searched first.

The example game is “take 1 or 2 stones”: starting with `stones` stones, players alternate taking 1 or 2 stones, and the player who takes the last stone wins. The evaluation returns 1 for a win for the player to move at the root and -1 for a loss.

- `minimax(stones, maximizing)` returns the exact game-tree value.
- `alpha_beta(stones, maximizing, alpha, beta)` returns the same value, with pruning.
- `best_take(stones)` returns an optimal first move, either 1 or 2.

Time Complexity: $O(b^d)$ in the worst case, where b is branching factor and d is search depth. Alpha-beta can reduce this substantially with good move ordering.

Space Complexity: $O(d)$ auxiliary recursion stack space.

```

#include <algorithm>

int minimax(int stones, bool maximizing) {
    if (stones == 0) {
        return maximizing ? -1 : 1;
    }
}

```

```

}
if (maximizing) {
    int best = -2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        best = std::max(best, minimax(stones - take, false));
    }
    return best;
} else {
    int best = 2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        best = std::min(best, minimax(stones - take, true));
    }
    return best;
}
}

int alpha_beta(int stones, bool maximizing, int alpha = -2, int beta = 2) {
    if (stones == 0) {
        return maximizing ? -1 : 1;
    }
    if (maximizing) {
        int value = -2;
        for (int take = 1; take <= 2 && take <= stones; take++) {
            value = std::max(value, alpha_beta(stones - take, false, alpha, beta));
            alpha = std::max(alpha, value);
            if (alpha >= beta) {
                break;
            }
        }
        return value;
    } else {
        int value = 2;
        for (int take = 1; take <= 2 && take <= stones; take++) {
            value = std::min(value, alpha_beta(stones - take, true, alpha, beta));
            beta = std::min(beta, value);
            if (alpha >= beta) {
                break;
            }
        }
        return value;
    }
}

int best_take(int stones) {
    int best_move = 1, best_value = -2;
    for (int take = 1; take <= 2 && take <= stones; take++) {
        int value = alpha_beta(stones - take, false);
        if (value > best_value) {
            best_value = value;
            best_move = take;
        }
    }
    return best_move;
}

```

Example Usage

```

#include <cassert>

int main() {
    assert(minimax(1, true) == 1);
    assert(minimax(3, true) == -1);
    for (int stones = 1; stones <= 10; stones++) {
        assert(minimax(stones, true) == alpha_beta(stones, true));
    }
    assert(best_take(4) == 1); // Move to losing state with 3 stones.
}

```

```
    assert(best_take(5) == 2); // Move to losing state with 3 stones.  
    return 0;  
}
```

Chapter 7

Geometry

7.1 Geometry Primitives

7.1.1 Point

A 2D point class template supporting epsilon comparisons. The coordinate type `T` defaults to `double`. Integer types (e.g. `int`) fully support all exact operations. Floating-point-only operations (`norm`, `arg`, `normalize`, `rotateCW/CCW`, `reflect` across a line) return coordinates of type `fp_t`, which is `double` when `T` is integral and `T` itself otherwise.

Exact operations (return `TPoint<T>` or `T`, no precision lost for integers):

- element-wise arithmetic, `dot()`, `cross()`, `sqnorm()`, cardinal rotations, `reflect(point)`, comparisons.

Overflow warning: the exact products `dot()`, `cross()`, and `sqnorm()` grow like the squared coordinate magnitude. With `TPoint<int>` these overflow a 32-bit `int` once coordinates exceed ~46000, so use `PointL` (`TPoint<long long>`) for larger integer coordinates.

Floating-point-only operations (return `TPoint<fp_t>` or `fp_t`):

- `norm()`, `arg()`, `proj()`, `normalize()`, `rotateCW()`, `rotateCCW()`, `reflect(line)`.
- `operator/` also promotes to `fp_t`.

Implicit conversion from `TPoint<U>` to `TPoint<T>` is provided when `U` is integral and `T` is floating-point, so `PointI` can be passed wherever `PointD` is expected.

Non-floating-point coordinate types such as `Modular` or `Rational` compose for the exact operations (arithmetic, `dot`, `cross`, `sqnorm`, comparisons): these route to exact `==/<` rather than epsilon comparisons. The floating-point-only operations are simply not instantiated unless called.

Type aliases:

- `PointI = TPoint<int>`: exact integer geometry (small coordinates only; see overflow warning)
- `PointL = TPoint<long long>`: exact integer geometry for large coordinates
- `PointD = TPoint<double>`: standard floating-point
- `PointLD = TPoint<long double>`: extra precision
- `Point = PointD`: default point type is double

Time Complexity: $O(1)$ per call to the constructor and all other operations.

Space Complexity:

- $O(1)$ for storage of the point.
- $O(1)$ auxiliary for all operations.


```

#include <cmath>
#include <stdint>
#include <ostream>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T>
struct TPoint {
    // fp_t is T for floating-point types, double for integral types.
    using fp_t = std::conditional_t<std::is_integral<T>::value, double, T>;

    T x, y;

    TPoint() : x(0), y(0) {}
    TPoint(T x, T y) : x(x), y(y) {}
    explicit TPoint(const std::pair<T, T> &p) : x(p.first), y(p.second) {}

    // Implicit conversion from integral to floating-point point (optional, can just use to_double()).
    template<
        typename U,
        typename = std::enable_if_t<std::is_integral<U>::value && std::is_floating_point<T>::value>>
    TPoint(const TPoint<U> &p) : x(static_cast<T>(p.x)), y(static_cast<T>(p.y)) {}

    bool operator==(const TPoint &p) const {
        if constexpr (std::is_floating_point<T>::value) {
            return EQ(x, p.x) && EQ(y, p.y);
        }
        return x == p.x && y == p.y; // exact for ints and types like Modular/Rational
    }

    bool operator!=(const TPoint &p) const { return !(*this == p); }

    bool operator<(const TPoint &p) const {
        if constexpr (std::is_floating_point<T>::value) {
            return EQ(x, p.x) ? LT(y, p.y) : LT(x, p.x);
        }
        return x == p.x ? y < p.y : x < p.x; // exact for ints and types like Modular/Rational
    }

    bool operator>(const TPoint &p) const { return p < *this; }
    bool operator<=(const TPoint &p) const { return !(*this > p); }
    bool operator>=(const TPoint &p) const { return !(*this < p); }
    TPoint operator+(const TPoint &p) const { return {x + p.x, y + p.y}; }
    TPoint operator-(const TPoint &p) const { return {x - p.x, y - p.y}; }
    TPoint operator+(T v) const { return {x + v, y + v}; }
    TPoint operator-(T v) const { return {x - v, y - v}; }
    TPoint operator*(T v) const { return {x * v, y * v}; }

    // Division always promotes to fp_t to avoid integer truncation.
    TPoint<fp_t> operator/(fp_t v) const { return {(fp_t)x / v, (fp_t)y / v}; }

    TPoint &operator+=(const TPoint &p) { x += p.x; y += p.y; return *this; }
    TPoint &operator-=(const TPoint &p) { x -= p.x; y -= p.y; return *this; }
    TPoint &operator+=(T v) { x += v; y += v; return *this; }
    TPoint &operator-=(T v) { x -= v; y -= v; return *this; }
    TPoint &operator*=(T v) { x *= v; y *= v; return *this; }

```

```

friend TPoint operator+(T v, const TPoint &p) { return p + v; }
friend TPoint operator*(T v, const TPoint &p) { return p * v; }

// --- Exact operations: return T or TPoint<T>, work for any coordinate type ---

// Overflow warning: these products grow like the squared coordinate magnitude, so for integer T
// with large coordinates (beyond a few thousand for 32-bit int) use a wider type such as PointL.
T sqnorm() const { return x * x + y * y; }
T dot(const TPoint &p) const { return x * p.x + y * p.y; }
T cross(const TPoint &p) const { return x * p.y - y * p.x; }

// --- Floating-point operations: return fp_t or TPoint<fp_t> ---

fp_t norm() const { return sqrt(static_cast<fp_t>(sqnorm())); }
fp_t arg() const { return atan2(static_cast<fp_t>(y), static_cast<fp_t>(x)); }
fp_t proj(const TPoint &p) const { return static_cast<fp_t>(dot(p)) / p.norm(); }

TPoint<fp_t> normalize() const {
    fp_t n = norm();
    if (n < static_cast<fp_t>(1e-30)) { // guard against dividing by a near-zero norm, not a
        return {fp_t(0), fp_t(0)}; // geometric tolerance (EPS is too coarse here)
    }
    return {static_cast<fp_t>(x) / n, static_cast<fp_t>(y) / n};
}

// --- Cardinal rotations: exact for all coordinate types including int ---

// Returns (x, y) rotated 90/180/270 degrees counter-clockwise about the origin.
TPoint rotate90() const { return {-y, x}; }
TPoint rotate180() const { return {-x, -y}; }
TPoint rotate270() const { return {y, -x}; }

// Returns (x, y) rotated 90/180/270 degrees counter-clockwise about point p.
TPoint rotate90(const TPoint &p) const { return (*this - p).rotate90() + p; }
TPoint rotate180(const TPoint &p) const { return (*this - p).rotate180() + p; }
TPoint rotate270(const TPoint &p) const { return (*this - p).rotate270() + p; }

// --- Arbitrary-angle rotations: always return floating-point ---

// Returns (x, y) rotated t radians clockwise about the origin.
TPoint<fp_t> rotateCW(fp_t t) const {
    fp_t fx = (fp_t)x, fy = (fp_t)y;
    return {fx * cos(t) + fy * sin(t), fy * cos(t) - fx * sin(t)};
}

// Returns (x, y) rotated t radians counter-clockwise about the origin.
TPoint<fp_t> rotateCCW(fp_t t) const {
    fp_t fx = (fp_t)x, fy = (fp_t)y;
    return {fx * cos(t) - fy * sin(t), fx * sin(t) + fy * cos(t)};
}

// Returns (x, y) rotated t radians clockwise about point p.
TPoint<fp_t> rotateCW(const TPoint &p, fp_t t) const {
    return TPoint<fp_t>{(fp_t)(x - p.x), (fp_t)(y - p.y)}.rotateCW(t) +
        TPoint<fp_t>{(fp_t)p.x, (fp_t)p.y};
}

// Returns (x, y) rotated t radians counter-clockwise about point p.
TPoint<fp_t> rotateCCW(const TPoint &p, fp_t t) const {
    return TPoint<fp_t>{(fp_t)(x - p.x), (fp_t)(y - p.y)}.rotateCCW(t) +
        TPoint<fp_t>{(fp_t)p.x, (fp_t)p.y};
}

// --- Reflections ---

// Returns (x, y) reflected across point p. Exact for any coordinate type.
TPoint reflect(const TPoint &p) const { return {2 * p.x - x, 2 * p.y - y}; }

```

```

// Returns (x, y) reflected across the line containing points p and q.
// Always returns floating-point coordinates.
TPoint<fp_t> reflect(const TPoint &p, const TPoint &q) const {
    TPoint<fp_t> fp{(fp_t)p.x, (fp_t)p.y};
    if (p == q) {
        return TPoint<fp_t>{(fp_t)x, (fp_t)y}.reflect(fp);
    }
    TPoint<fp_t> r{(fp_t)(x - p.x), (fp_t)(y - p.y)}, s{(fp_t)(q.x - p.x), (fp_t)(q.y - p.y)};
    fp_t ssq = s.sqnorm();
    r = TPoint<fp_t>{(r.x * s.x + r.y * s.y) / ssq, (r.x * s.y - r.y * s.x) / ssq};
    return TPoint<fp_t>{r.x * s.x - r.y * s.y + fp.x, r.x * s.y + r.y * s.x + fp.y};
}

// --- Explicit type conversions ---

TPoint<double> to_double() const { return {static_cast<double>(x), static_cast<double>(y)}; }

TPoint<long double> to_ldouble() const {
    return {static_cast<long double>(x), static_cast<long double>(y)};
}

// --- Friend free-function versions ---

friend T sqnorm(const TPoint &p) { return p.sqnorm(); }
friend fp_t norm(const TPoint &p) { return p.norm(); }
friend fp_t arg(const TPoint &p) { return p.arg(); }
friend T dot(const TPoint &p, const TPoint &q) { return p.dot(q); }
friend T cross(const TPoint &p, const TPoint &q) { return p.cross(q); }
friend fp_t proj(const TPoint &p, const TPoint &q) { return p.proj(q); }
friend TPoint<fp_t> normalize(const TPoint &p) { return p.normalize(); }
friend TPoint rotate90(const TPoint &p) { return p.rotate90(); }
friend TPoint rotate180(const TPoint &p) { return p.rotate180(); }
friend TPoint rotate270(const TPoint &p) { return p.rotate270(); }
friend TPoint rotate90(const TPoint &p, const TPoint &q) { return p.rotate90(q); }
friend TPoint rotate180(const TPoint &p, const TPoint &q) { return p.rotate180(q); }
friend TPoint rotate270(const TPoint &p, const TPoint &q) { return p.rotate270(q); }
friend TPoint<fp_t> rotateCW(const TPoint &p, fp_t t) { return p.rotateCW(t); }
friend TPoint<fp_t> rotateCCW(const TPoint &p, fp_t t) { return p.rotateCCW(t); }
friend TPoint<fp_t> rotateCW(const TPoint &p, const TPoint &q, fp_t t) { return p.rotateCW(q, t); }
friend TPoint<fp_t> rotateCCW(const TPoint &p, const TPoint &q, fp_t t) { return p.rotateCCW(q, t); }
friend TPoint reflect(const TPoint &p, const TPoint &q) { return p.reflect(q); }
friend TPoint<fp_t> reflect(const TPoint &p, const TPoint &a, const TPoint &b) { return p.reflect(a, b); }

friend std::ostream &operator<<(std::ostream &out, const TPoint &p) {
    if constexpr (std::is_floating_point<T>::value) {
        return out << "(" << (fabs(p.x) < EPS ? 0 : p.x) << "," << (fabs(p.y) < EPS ? 0 : p.y) << ")";
    }
    return out << "(" << p.x << "," << p.y << ")";
}
};

using PointI = TPoint<int>;
using PointL = TPoint<long long>;
using PointD = TPoint<double>;
using PointLD = TPoint<long double>;
using Point = PointD; // Default point type is double.

```

Example Usage

```

#include <cassert>

const double PI = acos(-1.0);

int main() {
    Point p(-10, 3);
    assert(Point(-18, 29) == p + Point(-3, 9) * 6.0 / 2.0 - Point(-1, 1));
}

```

```

assert(EQ(109, p.sqnorm()));
assert(EQ(10.44030650891, p.norm()));
assert(EQ(2.850135859112, p.arg()));
assert(EQ(0, p.dot(Point(3, 10))));
assert(EQ(0, p.cross(Point(10, -3))));
assert(EQ(10, p.proj(Point(-10, 0))));
assert(EQ(1, p.normalize().norm()));
assert(Point(-3, -10) == p.rotate90());
assert(Point(10, -3) == p.rotate180());
assert(Point(3, 10) == p.rotate270());
assert(Point(3, 12) == p.rotateCW(Point(1, 1), PI / 2));
assert(Point(1, -10) == p.rotateCCW(Point(2, 2), PI / 2));
assert(Point(10, -3) == p.reflect(Point(0, 0)));
assert(Point(-10, -3) == p.reflect(Point(-2, 0), Point(5, 0)));

// Integer point - exact arithmetic, float-only ops return PointD.
PointI a(3, 4), b(1, 0);
assert(a + b == PointI(4, 4));
assert(a - b == PointI(2, 4));
assert(a * 2 == PointI(6, 8));
assert(a.sqnorm() == 25);
assert(a.dot(b) == 3);
assert(a.cross(b) == -4);
assert(EQ(a.norm(), 5.0));
assert(a.rotate90() == PointI(-4, 3));
assert(a.rotate180() == PointI(-3, -4));
assert(a.rotate270() == PointI(4, -3));
PointD anorm = a.normalize(); // returns PointD
assert(EQ(anorm.norm(), 1.0));
PointD adiv = a / 2.0; // returns PointD (division always promotes)
assert(EQ(adiv.x, 1.5) && EQ(adiv.y, 2.0));
auto rotated = PointI(1, 0).rotateCCW(PI / 4); // returns PointD (arbitrary rotation promotes)
double root2over2 = sqrt(2.0) / 2.0;
assert(EQ(rotated.x, root2over2) && EQ(rotated.y, root2over2));

// Implicit conversion PointI -> PointD.
PointD promoted = PointI(1, 2);
assert(promoted == PointD(1, 2));
return 0;
}

```

7.1.2 Line

A straight line in two dimensions, templated on the coordinate type `T` (default `double`). The line $ax + by + c = 0$ is stored unnormalized, so a line through integer points (p_x, p_y) and (q_x, q_y) keeps the integer coefficients: $a = q_y - p_y$, $b = p_x - q_x$, $c = -(a \cdot p_x + b \cdot p_y)$. Storage and the exact predicates `contains()`, `is_parallel()`, and `is_perpendicular()` therefore stay exact for integral type `T`. The inherently-fractional operations `slope()`, `x()`, and `y()` return `fp_t`, which is `double` when `T` is integral and `T` itself otherwise.

Because coefficients are not normalized to a canonical form, `operator==` compares lines by proportionality (whether they are the same geometric line) rather than by coefficient equality.

Operations include `horizontal()`, `vertical()`, `slope()`, evaluating y at some x with `y()` (and vice versa with `x()`), checking if a point falls on the line with `contains()`, checking if another line is parallel or perpendicular with `is_parallel()` or `is_perpendicular()`, and finding the `parallel()` or `perpendicular()` line through a point.

Overflow warning: the exact predicates form products of coefficients (e.g. cross terms in `is_parallel()`/`is_perpendicular()` and `a*p.x + b*p.y` in `contains()`), which grow like the squared coordinate magnitude. For integral type `T`, use `LineL` (`TLine<int64_t>`) once coordinates exceed ~ 46000 , or the 32-bit products overflow.

Type aliases:

- `LineI = TLine<int>`: exact integer-coefficient lines (small values only; see overflow warning)
- `LineL = TLine<int64_t>`: exact integer-coefficient lines for large coordinates
- `LineD = TLine<double>`: standard floating-point
- `LineLD = TLine<long double>`: extra precision
- `Line = LineD`: default line type is double

Time Complexity: $O(1)$ per call to the constructor and all other operations.

Space Complexity:

- $O(1)$ for storage of the line.
- $O(1)$ auxiliary for all operations.

```
#include <cmath>
#include <cstdint>
#include <limits>
#include <ostream>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;
const double M_NAN = std::numeric_limits<double>::quiet_NaN();

// SFINAE guard: valid only when Pt exposes numeric .x/.y members, so templated point constructors
// don't hijack scalar-argument calls.
template<typename Pt>
using if_point = decltype(std::declval<const Pt &>().x, std::declval<const Pt &>().y, void());

template<typename T>
struct TLine {
    // fp_t is T for floating-point types, double for integral types.
    using fp_t = std::conditional_t<std::is_integral<T>::value, double, T>;

    T a, b, c;

    TLine() : a(0), b(0), c(0) {} // Invalid or uninitialized line.
    TLine(T a, T b, T c) : a(a), b(b), c(c) {}

    // Line through two points. Coefficients are exact (type T) for integer points.
    template<typename Pt, typename = if_point<Pt>>
    TLine(const Pt &p, const Pt &q) : a(q.y - p.y), b(p.x - q.x), c(-(a * p.x + b * p.y)) {}

    // Line of a given slope through a point. Intended for floating-point T (a generic slope
    // produces non-integer coefficients), so this constructor is disabled for integer T.
    template<
        class Pt, typename = if_point<Pt>, typename U = T,
        class = std::enable_if_t<std::is_floating_point<U>::value>>
    TLine(fp_t slope, const Pt &p) : a(-slope), b(1), c(slope * p.x - p.y) {}

    // Epsilon-aware for floating-point T; exact for int and for coordinate types like Modular or
    // Rational, which therefore compose for all of the predicates below.
    template<typename A, typename B>
    static bool eq(const A &p, const B &q) {
        if constexpr (std::is_floating_point<T>::value) {
            return fabs(p - q) <= EPS;
        }
        return p == q;
    }

    // Two lines are equal iff their coefficient vectors are proportional (the same line).
    bool operator==(const TLine &l) const {
        return eq(a * l.b, l.a * b) && eq(b * l.c, l.b * c) && eq(a * l.c, l.a * c);
    }

    bool operator!=(const TLine &l) const { return !(*this == l); }

    // Returns whether the line is a valid line (not both a and b zero).
```

```

bool valid() const { return !(eq(a, 0) && eq(b, 0)); }
bool horizontal() const { return valid() && eq(a, 0); }
bool vertical() const { return valid() && eq(b, 0); }

// Slope -a/b, or NaN if the line is vertical or invalid.
fp_t slope() const { return (!valid() || eq(b, 0)) ? M_NAN : -(fp_t)a / b; }

// Solve for x at a given y (NaN if horizontal or invalid).
fp_t x(fp_t y) const { return (!valid() || eq(a, 0)) ? M_NAN : -((fp_t)b * y + c) / a; }

// Solve for y at a given x (NaN if vertical or invalid).
fp_t y(fp_t x) const { return (!valid() || eq(b, 0)) ? M_NAN : -((fp_t)a * x + c) / b; }

// Whether point p lies on the line. Exact for integer T and integer p.
template<typename Pt>
bool contains(const Pt &p) const {
    return eq(a * p.x + b * p.y + c, 0);
}

// Parallel iff the normals are parallel (cross == 0); perpendicular iff normals are
// perpendicular (dot == 0). Both exact for integer T.
bool is_parallel(const TLine &l) const { return eq(a * l.b, l.a * b); }
bool is_perpendicular(const TLine &l) const { return eq(a * l.a, -(b * l.b)); }

// Parallel/perpendicular line through point p. Exact for integer T and integer p.
template<typename Pt, typename = if_point<Pt>>
TLine parallel(const Pt &p) const {
    return TLine(a, b, -(a * p.x + b * p.y));
}

template<typename Pt, typename = if_point<Pt>>
TLine perpendicular(const Pt &p) const {
    return TLine(-b, a, b * p.x - a * p.y);
}

friend std::ostream &operator<<(std::ostream &out, const TLine &l) {
    auto clean = [](T v) -> double {
        return fabs(static_cast<double>(v)) < EPS ? 0 : static_cast<double>(v);
    };
    return out << clean(l.a) << "x" << std::showpos << clean(l.b) << "y" << clean(l.c) << "=0"
        << std::noshowpos;
}

};

using LineI = TLine<int>;
using LineL = TLine<int64_t>;
using LineD = TLine<double>;
using LineLD = TLine<long double>;
using Line = LineD; // Default line type is double.

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    // Floating-point line.

```

```

Line l(2, -5, -8);
Line para = Line(2, -5, -8).parallel(Point(-6, -2));
Line perp = Line(2, -5, -8).perpendicular(Point(-6, -2));
assert(l.is_parallel(para) && l.is_perpendicular(perp));
assert(l.slope() == 0.4);
// operator== is proportionality-based: para is the same line as -0.4x + y - 0.4 = 0.
assert(para == Line(-0.4, 1, -0.4));
assert(perp == Line(2.5, 1, 17));
assert(l.contains(Point(1.5, -1))); // 2(1.5) - 5(-1) - 8 = 0.

// Integer line through integer points - exact coefficients and predicates.
LineI il(PointI(0, 0), PointI(2, 1)); // a=1, b=-2, c=0 -> x - 2y = 0
assert(il.a == 1 && il.b == -2 && il.c == 0);
assert(il.contains(PointI(4, 2))); // exact integer check
assert(!il.contains(PointI(4, 3)));
assert(il.is_parallel(LineI(PointI(1, 1), PointI(3, 2)))); // both slope 1/2
assert(il.is_perpendicular(LineI(PointI(0, 0), PointI(1, -2)))); // dot of normals = 0
assert(il.slope() == 0.5); // fp_t result
return 0;
}

```

7.1.3 Circle

A circle in two dimensions supporting epsilon comparisons. The circle centered at (h, k) is represented by the relation $(x - h)^2 + (y - k)^2 = r^2$, where the radius r is normalized to a nonnegative number.

This implementation stores and computes circle parameters in `double`. Point-accepting constructors and predicates are templated on the point type `Pt` and only read `.x/.y`, so they accept `Point/PointD/PointI` from 7.1.1 or any struct with numeric `.x` and `.y` fields.

- `Circle()` constructs the zero-radius circle centered at the origin.
- `Circle(r)` constructs a circle of radius `abs(r)` centered at the origin.
- `Circle(h, k, r)` constructs a circle of radius `abs(r)` centered at (h, k) .
- `Circle(o, r)` constructs a circle of radius `abs(r)` centered at point `o`.
- `Circle(a, b)` constructs the circle whose diameter is segment `a-b`.
- `Circle(a, b, c)` constructs the circumcircle through non-collinear points `a`, `b`, and `c`, throwing if the points are collinear.
- `Circle(a, b, r)` constructs one circle of radius `abs(r)` passing through points `a` and `b`, throwing if no finite circle is determined by the inputs.
- `operator==` and `operator!=` compare centers and radii using `EPS`.
- `contains(p)` returns whether point `p` lies inside or on the circle.
- `on_edge(p)` returns whether point `p` lies on the circle boundary.
- `incircle(a, b, c)` returns the circle inscribed in triangle `abc`.
- `in_circumcircle(a, b, c, d)` returns 1 if `d` lies strictly inside the circle through `a`, `b`, and `c`, 0 if it lies on the circle, or -1 if it lies outside. This has exact reasoning for integral points by evaluating the determinant directly without constructing a `Circle` or taking square roots. For integer inputs, the sign test is exact provided the chosen intermediate type does not overflow. The determinant has degree four in the coordinate magnitude, so use a wider type for large integer coordinates.

Time Complexity: $O(1)$ per call to the constructors and all other operations.

Space Complexity:

- $O(1)$ for storage of the circle.
- $O(1)$ auxiliary for all operations.

```

#include <cmath>
#include <cstdint>
#include <ostream>
#include <stdexcept>

```

```

#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }

// SFINAE guard: valid only when Pt exposes numeric .x/.y members. This keeps the templated point
// constructors from hijacking calls like Circle(double, double, double).
template<typename Pt>
using if_point = decltype(std::declval<const Pt &>().x, std::declval<const Pt &>().y, void());

struct Circle {
    double h, k, r;

    Circle() : h(0), k(0), r(0) {}
    explicit Circle(double r) : h(0), k(0), r(fabs(r)) {}
    Circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &o, double r) : h(o.x), k(o.y), r(fabs(r)) {}

    // Circle with the line segment ab as a diameter.
    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = hypot(a.x - h, a.y - k);
    }

    // Circumcircle of three points.
    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b, const Pt &c) {
        double an =
            static_cast<double>(b.x - c.x) * (b.x - c.x) + static_cast<double>(b.y - c.y) * (b.y - c.y);
        double bn =
            static_cast<double>(a.x - c.x) * (a.x - c.x) + static_cast<double>(a.y - c.y) * (a.y - c.y);
        double cn =
            static_cast<double>(a.x - b.x) * (a.x - b.x) + static_cast<double>(a.y - b.y) * (a.y - b.y);
        double wa = an * (bn + cn - an);
        double wb = bn * (an + cn - bn);
        double wc = cn * (an + bn - cn);
        double w = wa + wb + wc;
        if (EQ(w, 0)) {
            throw std::runtime_error("No circumcircle from collinear points.");
        }
        h = (wa * a.x + wb * b.x + wc * c.x) / w;
        k = (wa * a.y + wb * b.y + wc * c.y) / w;
        r = hypot(a.x - h, a.y - k);
    }

    // Circle of radius r that contains points a and b. In the general case, there will be two
    // possible circles and only one is chosen arbitrarily. However if the diameter is equal to
    // dist(a, b) = 2*r, then there is only one possible center. If points a and b are identical, then
    // there are infinite circles. If the points are too far away relative to the radius, then there
    // is no possible Circle. In the latter two cases, an exception is thrown.
    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b, double r) : r(fabs(r)) {
        if (LE(r, 0) && EQ(a.x, b.x) && EQ(a.y, b.y)) { // Zero-radius circle.
            h = a.x;

```



```

    k = a.y;
    return;
}
double d = hypot(b.x - a.x, b.y - a.y);
if (EQ(d, 0)) {
    throw std::runtime_error("Identical points, infinite circles.");
}
if (LT(r * 2.0, d)) {
    throw std::runtime_error("Points too far away to make Circle.");
}
double v = sqrt(r * r - d * d / 4.0) / d;
double mx = (a.x + b.x) / 2.0, my = (a.y + b.y) / 2.0;
h = mx + v * (a.y - b.y);
k = my + v * (b.x - a.x);
// The other answer is (h, k) = (mx - v*(a.y - b.y), my - v*(b.x - a.x)).
}

bool operator==(const Circle &c) const { return EQ(h, c.h) && EQ(k, c.k) && EQ(r, c.r); }
bool operator!=(const Circle &c) const { return !(*this == c); }

template<typename Pt>
bool contains(const Pt &p) const {
    return LE(
        static_cast<double>(p.x - h) * (p.x - h) + static_cast<double>(p.y - k) * (p.y - k), r * r
    );
}

template<typename Pt>
bool on_edge(const Pt &p) const {
    return EQ(
        static_cast<double>(p.x - h) * (p.x - h) + static_cast<double>(p.y - k) * (p.y - k), r * r
    );
}

friend std::ostream &operator<<(std::ostream &out, const Circle &c) {
    return out << std::showpos << "(x" << -(fabs(c.h) < EPS ? 0 : c.h) << ")^2+"
        << "(y" << -(fabs(c.k) < EPS ? 0 : c.k) << ")^2" << std::noshowpos << "="
        << (fabs(c.r) < EPS ? 0 : c.r * c.r);
}

};

// Returns the Circle inscribed inside the triangle abc.
template<typename Pt>
Circle incircle(const Pt &a, const Pt &b, const Pt &c) {
    double al = hypot(b.x - c.x, b.y - c.y);
    double bl = hypot(a.x - c.x, a.y - c.y);
    double cl = hypot(a.x - b.x, a.y - b.y);
    double l = al + bl + cl;
    double px = a.x - c.x, py = a.y - c.y, qx = b.x - c.x, qy = b.y - c.y;
    return EQ(l, 0) ? Circle(a.x, a.y, 0)
        : Circle(
            (al * a.x + bl * b.x + cl * c.x) / l, (al * a.y + bl * b.y + cl * c.y) / l,
            fabs(px * qy - py * qx) / l
        );
}

// Returns +1 if point d lies strictly inside the circle through a, b, c, 0 if d lies exactly on
// it, or -1 if d is outside. This uses a determinant (no square root), so it is exact for integer
// coordinates: the result is computed in `int64_t` for integral inputs (watch overflow for large
// coordinates, where the determinant is degree four) and `long double` otherwise. The points a, b,
// c must not be collinear; the result does not depend on their orientation.
template<typename Pt>
int in_circumcircle(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    using CoordT = decltype(a.x + a.x);
    using W = std::conditional_t<std::is_integral<CoordT>::value, int64_t, long double>;
    W adx = (W)a.x - d.x, ady = (W)a.y - d.y;
    W bdx = (W)b.x - d.x, bdy = (W)b.y - d.y;
    W cdx = (W)c.x - d.x, cdy = (W)c.y - d.y;

```

```

W det = (adx * adx + ady * ady) * (bdx * cdy - cdx * bdy) +
        (bdx * bdx + bdy * bdy) * (cdx * ady - adx * cdy) +
        (cdx * cdx + cdy * cdy) * (adx * bdy - bdx * ady);
W orient = ((W)b.x - a.x) * ((W)c.y - a.y) - ((W)b.y - a.y) * ((W)c.x - a.x);
W val = orient > 0 ? det : -det;
return val > 0 ? 1 : (val < 0 ? -1 : 0);
}

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    Circle c(-2, 5, sqrt(10));
    assert(c == Circle(Point(-2, 5), sqrt(10)));
    assert(c == Circle(Point(1, 6), Point(-5, 4)));
    assert(c == Circle(Point(-3, 2), Point(-3, 8), Point(-1, 8)));
    assert(c == incircle(Point(-12, 5), Point(3, 0), Point(0, 9)));
    assert(c.contains(Point(-2, 8)) && !c.contains(Point(-2, 9)));
    assert(c.on_edge(Point(-1, 2)) && !c.on_edge(Point(-1.01, 2)));

    // Integer-coordinate points are accepted; the Circle is computed in double.
    assert(c == Circle(PointI(1, 6), PointI(-5, 4)));
    assert(c == Circle(PointI(-3, 2), PointI(-3, 8), PointI(-1, 8)));
    assert(c == incircle(PointI(-12, 5), PointI(3, 0), PointI(0, 9)));
    assert(c.contains(PointI(-2, 8)) && !c.contains(PointI(-2, 9)));
    assert(c.on_edge(PointI(-1, 2)));

    // Exact integer in-circle predicate: unit circle through (1,0), (0,1), (-1,0).
    assert(in_circumcircle(PointI(1, 0), PointI(0, 1), PointI(-1, 0), PointI(0, 0)) == 1); // inside
    assert(
        in_circumcircle(PointI(1, 0), PointI(0, 1), PointI(-1, 0), PointI(2, 0)) == -1
    ); // outside
    assert(
        in_circumcircle(PointI(1, 0), PointI(0, 1), PointI(-1, 0), PointI(0, -1)) == 0
    ); // on edge
    // Orientation-independent: reversing a, b, c gives the same answer.
    assert(in_circumcircle(PointI(-1, 0), PointI(0, 1), PointI(1, 0), PointI(0, 0)) == 1);
    return 0;
}

```

7.1.4 Triangle

Common triangle calculations in two dimensions. The functions are templated on the point type `Pt`, which should work with `Point`/`PointD`/`PointI` from 7.1.1 or any struct with numeric `.x` and `.y` fields. `triangle_area()` returns `double` regardless of input type (the area is fractional via the `/2`). `same_side()` and `point_in_triangle()` do all arithmetic in the point's own coordinate type and are exact for integer points: they reduce each cross product to a sign before combining, so no precision is lost and cross products are never multiplied together.

- `triangle_area(a, b, c)` returns the area of the triangle with vertices `a`, `b`, and `c`.
- `triangle_area_sides(s1, s2, s3)` returns the area of a triangle with side lengths `s1`, `s2`, and `s3`. The given lengths must be nonnegative and form a valid triangle.

- `triangle_area_medians(m1, m2, m3)` returns the area of a triangle with medians of lengths `m1`, `m2`, and `m3`. The median of a triangle is a line segment joining a vertex to the midpoint of the opposing edge.
- `triangle_area_altitudes(h1, h2, h3)` returns the area of a triangle with altitudes `h1`, `h2`, and `h3`. An altitude of a triangle is the shortest line between a vertex and the infinite line that is extended from its opposite edge.
- `same_side(p1, p2, a, b)` returns whether points `p1` and `p2` lie on the same side of the line containing points `a` and `b`. If one or both points lie exactly on the line, then the result will depend on the setting of `EDGE_IS_SAME_SIDE`.
- `point_in_triangle(p, a, b, c)` returns whether point `p` lies within the triangle with vertices `a`, `b`, and `c`. If the point lies on or close to an edge (by roughly `EPS`), then the result will depend on the setting of `EDGE_IS_SAME_SIDE` in the function above.

Use an integral point type for integer coordinates, and a floating-point type for fractional coordinates.

Overflow warning: each individual cross product is still on the order of the squared coordinate magnitude, so for integer point types use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) once coordinates exceed ~ 46000 .

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename Pt>
double triangle_area(const Pt &a, const Pt &b, const Pt &c) {
    double acx = a.x - c.x, acy = a.y - c.y, bcx = b.x - c.x, bcy = b.y - c.y;
    return fabs(acx * bcy - acy * bcx) / 2.0;
}

double triangle_area_sides(double s1, double s2, double s3) {
    double s = (s1 + s2 + s3) / 2.0;
    return sqrt(s * (s - s1) * (s - s2) * (s - s3));
}

double triangle_area_medians(double m1, double m2, double m3) {
    return 4.0 * triangle_area_sides(m1, m2, m3) / 3.0;
}

double triangle_area_altitudes(double h1, double h2, double h3) {
    if (EQ(h1, 0) || EQ(h2, 0) || EQ(h3, 0)) {
        return 0;
    }
    double x = h1 * h1, y = h2 * h2, z = h3 * h3;
    double v = 2.0 / (x * y) + 2.0 / (x * z) + 2.0 / (y * z);
    return 1.0 / sqrt(v - 1.0 / (x * x) - 1.0 / (y * y) - 1.0 / (z * z));
}

template<typename Pt>
bool same_side(const Pt &p1, const Pt &p2, const Pt &a, const Pt &b) {
    static const bool EDGE_IS_SAME_SIDE = true;
    // Cross products in the point's native coordinate type (exact for integer points).
    auto c1 = (b.x - a.x) * (p1.y - a.y) - (b.y - a.y) * (p1.x - a.x);
```

```

auto c2 = (b.x - a.x) * (p2.y - a.y) - (b.y - a.y) * (p2.x - a.x);
// Compare the signs, not the product c1 * c2, to avoid integer overflow.
int s1 = LT(0, c1) ? 1 : (LT(c1, 0) ? -1 : 0);
int s2 = LT(0, c2) ? 1 : (LT(c2, 0) ? -1 : 0);
return EDGE_IS_SAME_SIDE ? (s1 * s2 >= 0) : (s1 * s2 > 0);
}

template<typename Pt>
bool point_in_triangle(const Pt &p, const Pt &a, const Pt &b, const Pt &c) {
    return same_side(p, a, b, c) && same_side(p, b, a, c) && same_side(p, c, a, b);
}

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    assert(EQ(6, triangle_area(Point(0, -1), Point(4, -1), Point(0, -4))));
    assert(EQ(6, triangle_area_sides(3, 4, 5)));
    assert(EQ(6, triangle_area_medians(3.605551275, 2.5, 4.272001873)));
    assert(EQ(6, triangle_area_altitudes(3, 4, 2.4)));

    // Fractional coordinates require a floating-point point type.
    assert(point_in_triangle(Point(0, 0), Point(-1, 0), Point(0, -2), Point(4, 0)));
    assert(!point_in_triangle(Point(0, 1), Point(-1, 0), Point(0, -2), Point(4, 0)));
    assert(point_in_triangle(Point(-2.44, 0.82), Point(-1, 0), Point(-3, 1), Point(4, 0)));
    assert(!point_in_triangle(Point(-2.44, 0.7), Point(-1, 0), Point(-3, 1), Point(4, 0)));

    // Integer coordinates: same_side / point_in_triangle are computed exactly in int.
    assert(EQ(8, triangle_area(PointI(0, 0), PointI(4, 0), PointI(0, 4))));
    assert(point_in_triangle(PointI(1, 1), PointI(0, 0), PointI(4, 0), PointI(0, 4)));
    assert(!point_in_triangle(PointI(5, 5), PointI(0, 0), PointI(4, 0), PointI(0, 4)));
    assert(point_in_triangle(PointI(2, 2), PointI(0, 0), PointI(4, 0), PointI(0, 4))); // on edge
    return 0;
}

```

7.1.5 Rectangle

Common axis-aligned rectangle calculations in two dimensions. The functions are templated on the point type `Pt`, which should work with `Point`/`PointD`/`PointI` from 7.1.1, or any struct with numeric `x` and `y` fields.

- `point_in_rectangle(p, v, w, h, EDGE_IS_INSIDE)` returns whether point `p` lies inside the axis-aligned rectangle with corner `v`, width `w`, and height `h`. Negative widths and heights are supported. The `EDGE_IS_INSIDE` flag (default `true`) controls whether boundary points count.
- `point_in_rectangle(p, a, b, EDGE_IS_INSIDE)` returns whether point `p` lies inside the axis-aligned rectangle with opposite corners `a` and `b`.
- `rectangle_intersection(a1, b1, a2, b2, &p, &q, EDGE_IS_INSIDE)` computes the intersection of two axis-aligned rectangles, where `a1/b1` and `a2/b2` are opposite-corner pairs. Returns -1 if the rectangles are disjoint, 0 if they partially intersect, 1 if the first rectangle is completely inside the second, and 2 if the second rectangle is completely inside the first. If an intersection exists, its lower-left and upper-right corners are stored in `p`

and `q` when those pointers are non-null. The `EDGE_IS_INSIDE` flag (default `true`) controls whether boundary-only contact counts as intersecting.

For integer-coordinate inputs, comparisons are exact as long as intermediate coordinate additions, subtractions, and min/max values do not overflow. Floating-point inputs use EPS-based comparisons.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool GT(T a, U b) { return LT(b, a); }
template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }
template<typename T, typename U> bool GE(T a, U b) { return !LT(a, b); }

template<typename Pt, typename T>
bool point_in_rectangle(
    const Pt &p, const Pt &v, const T &w, const T &h, const bool EDGE_IS_INSIDE = true
) {
    if (w < 0) {
        return point_in_rectangle(p, Pt(v.x + w, v.y), -w, h, EDGE_IS_INSIDE);
    }
    if (h < 0) {
        return point_in_rectangle(p, Pt(v.x, v.y + h), w, -h, EDGE_IS_INSIDE);
    }
    return EDGE_IS_INSIDE ? (GE(p.x, v.x) && LE(p.x, v.x + w) && GE(p.y, v.y) && LE(p.y, v.y + h))
        : (GT(p.x, v.x) && LT(p.x, v.x + w) && GT(p.y, v.y) && LT(p.y, v.y + h));
}

template<typename Pt>
bool point_in_rectangle(const Pt &p, const Pt &a, const Pt &b, const bool EDGE_IS_INSIDE = true) {
    auto x1 = std::min(a.x, b.x), y1 = std::min(a.y, b.y);
    auto xh = std::max(a.x, b.x), yh = std::max(a.y, b.y);
    return point_in_rectangle(p, Pt(x1, y1), xh - x1, yh - y1, EDGE_IS_INSIDE);
}

template<typename Pt>
int rectangle_intersection(
    const Pt &a1, const Pt &b1, const Pt &a2, const Pt &b2, Pt *p = nullptr, Pt *q = nullptr,
    const bool EDGE_IS_INSIDE = true
) {
    // Normalize each rectangle to [x1, xh] x [y1, yh] with coordinate-wise min/max only.
    // (Using auto keeps the native coordinate type, so integer rectangles stay exact.)
    auto x11 = std::min(a1.x, b1.x), xh1 = std::max(a1.x, b1.x);
    auto y11 = std::min(a1.y, b1.y), yh1 = std::max(a1.y, b1.y);
    auto x12 = std::min(a2.x, b2.x), xh2 = std::max(a2.x, b2.x);
    auto y12 = std::min(a2.y, b2.y), yh2 = std::max(a2.y, b2.y);
    // The intersection is the overlap of the two coordinate intervals.
    auto ix1 = std::max(x11, x12), ixh = std::min(xh1, xh2);
    auto iy1 = std::max(y11, y12), iyh = std::min(yh1, yh2);
    bool disjoint = EDGE_IS_INSIDE ? (GT(ix1, ixh) || GT(iy1, iyh)) : (GE(ix1, ixh) || GE(iy1, iyh));
    if (disjoint) {
        return -1; // Completely disjoint.
    }
}
```

```

}
// Opposing corners of the overlap: bottom-left in p, top-right in q.
if (p != nullptr) {
    *p = Pt(ixl, iyl);
}
if (q != nullptr) {
    *q = Pt(ixh, iyh);
}
// The overlap equals a rectangle exactly when that rectangle is contained in the other.
if (EQ(ixl, x1l) && EQ(ixh, xh1) && EQ(iyl, y1l) && EQ(iyh, yh1)) {
    return 1; // Rectangle 1 completely inside 2.
}
if (EQ(ixl, x12) && EQ(ixh, xh2) && EQ(iyl, y12) && EQ(iyh, yh2)) {
    return 2; // Rectangle 2 completely inside 1.
}
return 0; // Partial intersection.
}

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
};

int main() {
    assert(point_in_rectangle(Point(0, -1), Point(0, -3), 3, 2));
    assert(!point_in_rectangle(Point(0, -1), Point(0, -3), 3, 2, false));
    assert(point_in_rectangle(Point(2, -2), Point(3, -3), -3, 2));
    assert(!point_in_rectangle(Point(0, 0), Point(3, -1), -3, -2));
    assert(point_in_rectangle(Point(2, -2), Point(3, -3), Point(0, -1)));
    assert(!point_in_rectangle(Point(0, -1), Point(3, -3), Point(0, -1), false));
    assert(!point_in_rectangle(Point(-1, -2), Point(3, -3), Point(0, -1)));

    Point p, q;
    assert(-1 == rectangle_intersection(Point(0, 0), Point(1, 1), Point(2, 2), Point(3, 3)));
    assert(0 == rectangle_intersection(Point(1, 1), Point(7, 7), Point(5, 5), Point(0, 0), &p, &q));
    assert(p == Point(1, 1) && q == Point(5, 5));
    assert(1 == rectangle_intersection(Point(1, 1), Point(0, 0), Point(0, 0), Point(1, 10), &p, &q));
    assert(p == Point(0, 0) && q == Point(1, 1));
    assert(2 == rectangle_intersection(Point(0, 5), Point(5, 7), Point(1, 6), Point(2, 5), &p, &q));
    assert(p == Point(1, 5) && q == Point(2, 6));
    assert(
        -1 ==
        rectangle_intersection(Point(0, 0), Point(1, 1), Point(1, 1), Point(2, 2), &p, &q, false)
    );
    return 0;
}

```

7.2 Angles, Distances, and Intersections

7.2.1 Angles

Angle calculations in two dimensions. All operations are inherently floating-point (`atan2`, `acos`, trigonometry), so these functions use the local double-coordinate `PointD` type. Convert integer points to `PointD` when asking for angles. The constants `DEG` and `RAD` may be used as multipliers to convert between degrees and radians. For

example, if `t` is a value in degrees, then `t * DEG` is the equivalent angle in radians; if `t` is in radians, then `t * RAD` is the equivalent angle in degrees.

- `reduce_deg(t)` takes an angle `t` degrees and returns an equivalent angle in the range $[0, 360)$ degrees (e.g. -630 becomes 90).
- `reduce_rad(t)` takes an angle `t` radians and returns an equivalent angle in the range $[0, 2\pi)$ radians (e.g. 720.5 becomes 0.5).
- `polar_point(r, t)` returns a two-dimensional Cartesian point given radius `r` and angle `t` radians in polar coordinates (see `std::polar()`).
- `polar_angle(p)` returns the angle in radians of the line segment from $(0, 0)$ to point `p`, relative counterclockwise to the positive x -axis.
- `angle(a, o, b)` returns the smallest angle in radians formed by the points `a`, `o`, `b` with vertex at point `o`.
- `angle_between(a, b)` returns the angle in radians of segment from point `a` to point `b`, relative counterclockwise to the positive x -axis.
- `angle_between(a1, b1, a2, b2)` returns the smaller angle in radians between two lines $a_1x + b_1y + c_1 = 0$ and $a_2x + b_2y + c_2 = 0$, limited to $[0, \pi/2]$.
- `cross(a, b, o)` returns the magnitude (Euclidean norm) of the three-dimensional cross product between points `a` and `b` where the z -component is implicitly zero and the origin is implicitly shifted to point `o`. This operation is also equal to double the signed area of the triangle from these three points.
- `turn(a, o, b)` returns -1 if the path `a` $\rightarrow$ `o` $\rightarrow$ `b` forms a left turn on the plane, 0 if the path forms a straight line segment, or 1 if it forms a right turn.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <type_traits>

const double EPS = 1e-9;
const double PI = acos(-1.0);
const double DEG = PI / 180;
const double RAD = 180 / PI;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

double reduce_deg(double t) {
    if (t < -360) {
        return reduce_deg(fmod(t, 360));
    }
    if (t < 0) {
        return t + 360;
    }
    return (t >= 360) ? fmod(t, 360) : t;
}

double reduce_rad(double t) {
    if (t < -2 * PI) {
        return reduce_rad(fmod(t, 2 * PI));
    }
    if (t < 0) {
        return t + 2 * PI;
    }
    return (t >= 2 * PI) ? fmod(t, 2 * PI) : t;
}
```

```

template<typename OutPt>
OutPt polar_point(double r, double t) {
    return OutPt(r * cos(t), r * sin(t));
}

template<typename Pt>
double polar_angle(const Pt &p) {
    double t = atan2(static_cast<double>(p.y), static_cast<double>(p.x));
    return (t < 0) ? (t + 2 * PI) : t;
}

template<typename Pt>
double angle(const Pt &a, const Pt &o, const Pt &b) {
    double ux = o.x - a.x, uy = o.y - a.y;
    double vx = o.x - b.x, vy = o.y - b.y;
    double cosine = (ux * vx + uy * vy) / (std::hypot(ux, uy) * std::hypot(vx, vy));
    return acos(std::max(-1.0, std::min(1.0, cosine)));
}

template<typename Pt>
double angle_between(const Pt &a, const Pt &b) {
    double t = atan2(a.x * b.y - a.y * b.x, a.x * b.x + a.y * b.y);
    return (t < 0) ? (t + 2 * PI) : t;
}

double angle_between(const double &a1, const double &b1, const double &a2, const double &b2) {
    double t = atan2(a1 * b2 - a2 * b1, a1 * a2 + b1 * b2);
    if (t < 0) {
        t += PI;
    }
    return LT(PI / 2, t) ? (PI - t) : t;
}

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o = Pt(0, 0)) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// Built on cross(a, b, o), whose sign is the opposite of the path-direction cross product, so a
// positive cross here is a right turn: this returns +1 for right/clockwise (NOT the more common
// +1 = counter-clockwise convention), -1 for left/counter-clockwise, and 0 if collinear.
template<typename Pt>
int turn(const Pt &a, const Pt &o, const Pt &b) {
    auto c = cross(a, b, o);
    return LT(c, 0) ? -1 : (LT(0, c) ? 1 : 0);
}

```

Example Usage

```

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
};

#include <cassert>

int main() {
    assert(EQ(123, reduce_deg(-8 * 360 + 123)));
    assert(EQ(1.2345, reduce_rad(2 * PI * 8 + 1.2345)));
    assert(polar_point<Point>(4, PI) == Point(-4, 0));
    assert(polar_point<Point>(4, -PI / 2) == Point(0, -4));
    assert(EQ(45, polar_angle(Point(5, 5)) * RAD));
    assert(EQ(135 * DEG, polar_angle(Point(-4, 4))));
    assert(EQ(90 * DEG, angle(Point(5, 0), Point(0, 5), Point(-5, 0))));
}

```



```

assert(EQ(225 * DEG, angle_between(Point(0, 5), Point(5, -5))));
assert(-1 == cross(Point(0, 1), Point(1, 0), Point(0, 0)));
assert(1 == turn(Point(0, 1), Point(0, 0), Point(-5, -5)));
return 0;
}

```

7.2.2 Distances

Distance calculations in two dimensions for points, lines, and line segments. The functions are templated on the point type `Pt`: pass any struct with numeric `.x` and `.y` fields (for example `PointD`/`PointI` from 7.1.1, or the local example struct). `sqdist` reverses the coordinate type and is exact for integer points. Functions that return an actual distance use `double` for the final division/square root. `closest_point` returns a point of the same type as its inputs, so use a floating-point `Pt` there for a meaningful (non-truncated) result.

- `dist(a, b)` and `sqdist(a, b)` respectively return the distance and squared distance between points `a` and `b`.
- `line_dist(p, a, b, c)` returns the distance from point `p` to the line $ax + by + c = 0$.
- `line_dist(p, a, b)` returns the distance from point `p` to the infinite line containing points `a` and `b`. If `a = b`, the distance from `p` to `a` is returned.
- `line_dist(a1, b1, c1, a2, b2, c2)` returns the distance between two lines.
- `seg_dist(p, a, b)` returns the distance from point `p` to the line segment `a-b`.
- `seg_dist(a, b, c, d)` returns the minimum distance between line segments `a-b` and `c-d`, or 0 if the segments touch or intersect.
- `closest_point(a, b, p)` returns the point on segment `a-b` closest to point `p`.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }
template<typename T, typename U> bool GE(T a, U b) { return !LT(a, b); }

// sqdist returns the coordinate type (exact for integer points).
template<typename Pt>
auto sqdist(const Pt &a, const Pt &b) {
    auto dx = b.x - a.x, dy = b.y - a.y;
    return dx * dx + dy * dy;
}

template<typename Pt>
double dist(const Pt &a, const Pt &b) {
    return sqrt(static_cast<double>(sqdist(a, b)));
}

template<typename Pt, typename T>
double line_dist(const Pt &p, const T &a, const T &b, const T &c) {
    return fabs(static_cast<double>(a) * p.x + static_cast<double>(b) * p.y + c) /
        sqrt(static_cast<double>(a) * a + static_cast<double>(b) * b);
}

```

```

}

template<typename Pt>
double line_dist(const Pt &p, const Pt &a, const Pt &b) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return dist(p, a);
    }
    auto n = sqdist(a, b);
    auto d = (p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y);
    double u = static_cast<double>(d) / n;
    double dx = a.x + u * (b.x - a.x) - p.x, dy = a.y + u * (b.y - a.y) - p.y;
    return sqrt(dx * dx + dy * dy);
}

template<typename T>
double line_dist(const T &a1, const T &b1, const T &c1, const T &a2, const T &b2, const T &c2) {
    if (EQ(a1 * b2, a2 * b1)) {
        double factor = EQ(b1, 0) ? (static_cast<double>(a1) / a2) : (static_cast<double>(b1) / b2);
        return EQ(c1, c2 * factor)
            ? 0
            : fabs(c2 * factor - c1) /
                sqrt(static_cast<double>(a1) * a1 + static_cast<double>(b1) * b1);
    }
    return 0;
}

template<typename Pt>
double seg_dist(const Pt &p, const Pt &a, const Pt &b) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return dist(p, a);
    }
    auto n = sqdist(a, b);
    auto d = (p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y);
    if (LE(d, 0) || EQ(n, 0)) {
        return dist(p, a);
    }
    if (GE(d, n)) {
        return dist(p, b);
    }
    double t = static_cast<double>(d) / n;
    double dx = a.x + t * (b.x - a.x) - p.x, dy = a.y + t * (b.y - a.y) - p.y;
    return sqrt(dx * dx + dy * dy);
}

template<typename Pt>
double seg_dist(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        return seg_dist(a, c, d);
    }
    if (EQ(c.x, d.x) && EQ(c.y, d.y)) {
        return seg_dist(c, a, b);
    }
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto c1 = ab_x * cd_y - ab_y * cd_x;
    auto c2 = ac_x * ab_y - ac_y * ab_x;
    if (EQ(c1, 0) && EQ(c2, 0)) {
        auto less = [](const Pt &u, const Pt &v) { return u.x != v.x ? u.x < v.x : u.y < v.y; };
        auto minp = [&](const Pt &u, const Pt &v) -> const Pt & { return less(v, u) ? v : u; };
        auto maxp = [&](const Pt &u, const Pt &v) -> const Pt & { return less(u, v) ? v : u; };
        const Pt &res1 = maxp(minp(a, b), minp(c, d));
        const Pt &res2 = minp(maxp(a, b), maxp(c, d));
        if (!less(res2, res1)) {
            return 0;
        }
    }
    else if (!EQ(c1, 0)) {
        auto t_num = ac_x * cd_y - ac_y * cd_x;

```

```

    bool c1_pos = c1 > 0;
    bool t_between_01 = c1_pos ? (LE(0, t_num) && LE(t_num, c1)) : (LE(t_num, 0) && LE(c1, t_num));
    bool u_between_01 = c1_pos ? (LE(0, c2) && LE(c2, c1)) : (LE(c2, 0) && LE(c1, c2));
    if (t_between_01 && u_between_01) {
        return 0;
    }
}
return std::min(
    std::min(seg_dist(a, c, d), seg_dist(b, c, d)), std::min(seg_dist(c, a, b), seg_dist(d, a, b))
);
}

template<typename Pt>
Pt closest_point(const Pt &a, const Pt &b, const Pt &p) {
    Pt res{};
    if (EQ(a.x, b.x) && EQ(a.y, b.y)) {
        res.x = a.x;
        res.y = a.y;
        return res;
    }
    auto n = sqdist(a, b);
    auto d = (p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y);
    double t = static_cast<double>(d) / n;
    if (t <= 0) {
        res.x = a.x;
        res.y = a.y;
    } else if (t >= 1) {
        res.x = b.x;
        res.y = b.y;
    } else {
        res.x = a.x + t * (b.x - a.x);
        res.y = a.y + t * (b.y - a.y);
    }
    return res;
}

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    assert(EQ(5, dist(Point(-1, -1), Point(2, 3))));
    assert(EQ(25, sqdist(Point(-1, -1), Point(2, 3))));
    assert(EQ(1.2, line_dist(Point(2, 1), -4, 3, -1)));
    assert(EQ(0.8, line_dist(Point(3, 3), Point(-1, -1), Point(2, 3))));
    assert(EQ(1.2, line_dist(Point(2, 1), Point(-1, -1), Point(2, 3))));
    assert(EQ(0, line_dist(-4, 3, -1, 8, 6, 2)));
    assert(EQ(0.8, line_dist(-4, 3, -1, -8, 6, -10)));
    assert(EQ(1.0, seg_dist(Point(3, 3), Point(-1, -1), Point(2, 3))));
    assert(EQ(1.2, seg_dist(Point(2, 1), Point(-1, -1), Point(2, 3))));
    assert(EQ(0, seg_dist(Point(0, 2), Point(3, 3), Point(-1, -1), Point(2, 3))));
    assert(EQ(0.6, seg_dist(Point(-1, 0), Point(-2, 2), Point(-1, -1), Point(2, 3))));

    // Integer points: sqdist is exact, dist returns double.
    PointI ia{-1, -1}, ib{2, 3};
    assert(sqdist(ia, ib) == 25);    // int result
}

```

```

assert(EQ(dist(ia, ib), 5.0)); // double result
return 0;
}

```

7.2.3 Line Intersection

Intersection calculations in two dimensions for straight lines and line segments. The functions are templated on the point type `Pt`: pass any struct with numeric `.x` and `.y` fields (for example `PointD`/`PointI` from 7.1.1, or the local example struct) for which `operator<` orders points lexicographically. Point outputs are written through a caller-supplied pointer whose type is deduced, so the output point type may differ from the input – e.g. integer endpoints with a floating-point output point.

`seg_intersection()` has a detection-only overload (called without the output pointers) whose calculations are exact when `Pt` has integer coordinates.

- `line_intersection(a1, b1, c1, a2, b2, c2, &p)` intersects lines $a_1x + b_1y + c_1 = 0$ and $a_2x + b_2y + c_2 = 0$, returning -1 (parallel), 0 (one point, stored into `p`), or 1 (identical).
- `line_intersection(p1, p2, p3, p4, &p)` intersects the infinite lines through `p1`-`p2` and `p3`-`p4`, returning -1 (parallel), 0 (one point, stored into `p`), or 1 (identical).
- `seg_intersection(a, b, c, d, &p, &q, TOUCH_IS_INTERSECT)` intersects segments `a`-`b` and `c`-`d`, returning -1 (none), 0 (one point), or 1 (overlapping segment). The `TOUCH_IS_INTERSECT` flag (default `true`) controls whether segments that meet only at an endpoint count as intersecting. If the intersection is a point (and `p` is not `nullptr`), it will be stored into `p`. If the intersection is an overlapping segment (and `p` and `q` are not `nullptr`) then their endpoints will be stored into pointers `p` and `q`, which must reference a floating-point point type.
- `seg_intersection(a, b, c, d, TOUCH_IS_INTERSECT)` is a simplified, detection-only version of the above. Both versions are exact for detection if the input point type is integral.
- `closest_point(a, b, c, p)` returns the closest point on line $ax + by + c = 0$ to point `p`.

Overflow warning: the exact integer paths multiply coordinate differences (cross products and squared lengths), which grow like the squared coordinate magnitude. With 32-bit `int` coordinates these overflow once coordinates exceed ~ 46000 , so for larger integer inputs use a point type with 64-bit (`int64_t`) coordinates.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <cstdint>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }

template<typename OutPt>
int line_intersection(double a1, double b1, double c1, double a2, double b2, double c2, OutPt *p) {
    // Cramer's rule for a1*x + b1*y = -c1, a2*x + b2*y = -c2.
    double det = a1 * b2 - a2 * b1;

```

```

    if (EQ(det, 0)) { // Parallel (lines need not be given in normalized form).
        // Identical iff the other two 2x2 determinants also vanish.
        return (EQ(a1 * c2 - a2 * c1, 0) && EQ(b1 * c2 - b2 * c1, 0)) ? 1 : -1;
    }
    p->x = (b1 * c2 - b2 * c1) / det;
    if (!EQ(b1, 0)) {
        p->y = -(a1 * p->x + c1) / b1;
    } else {
        p->y = -(a2 * p->x + c2) / b2;
    }
    return 0;
}

template<typename Pt, typename OutPt>
int line_intersection(const Pt &p1, const Pt &p2, const Pt &p3, const Pt &p4, OutPt *p) {
    auto a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    auto c1 = -(p1.x * p2.y - p2.x * p1.y);
    auto a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    auto c2 = -(p3.x * p4.y - p4.x * p3.y);
    auto x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    auto det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    p->x = static_cast<double>(x) / det;
    p->y = static_cast<double>(y) / det;
    return 0;
}

template<typename Pt>
bool point_on_segment(const Pt &p, const Pt &a, const Pt &b) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return EQ((p.x - a.x) * (b.y - a.y) - (p.y - a.y) * (b.x - a.x), 0) &&
        LE(std::min(a.x, b.x), p.x) && LE(p.x, std::max(a.x, b.x)) &&
        LE(std::min(a.y, b.y), p.y) && LE(p.y, std::max(a.y, b.y));
}

// Detection is exact for integer Pt. Intersection point output may use a separate float OutPt.
template<typename Pt, typename OutPt>
int seg_intersection(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, OutPt *p = nullptr, OutPt *q = nullptr,
    const bool TOUCH_IS_INTERSECT = true
) {
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto ab2 = ab_x * ab_x + ab_y * ab_y;
    auto cd2 = cd_x * cd_x + cd_y * cd_y;
    if (EQ(ab2, 0)) {
        if (TOUCH_IS_INTERSECT && point_on_segment(a, c, d)) {
            if (p != nullptr) {
                p->x = static_cast<double>(a.x);
                p->y = static_cast<double>(a.y);
            }
            return 0; // Degenerate: first segment is a point intersecting the second segment.
        }
        return -1; // Degenerate: first segment is a point not intersecting the second segment.
    }
    if (EQ(cd2, 0)) {
        if (TOUCH_IS_INTERSECT && point_on_segment(c, a, b)) {
            if (p != nullptr) {
                p->x = static_cast<double>(c.x);
                p->y = static_cast<double>(c.y);
            }
            return 0; // Degenerate: second segment is a point intersecting the first segment.
        }
        return -1; // Degenerate: second segment is a point not intersecting the first segment.
    }
}

```

```

auto c1 = ab_x * cd_y - ab_y * cd_x;
auto c2 = ac_x * ab_y - ac_y * ab_x;
if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
    Pt res1 = std::max(std::min(a, b), std::min(c, d));
    Pt res2 = std::min(std::max(a, b), std::max(c, d));
    bool overlap = TOUCH_IS_INTERSECT ? !(res2 < res1) : (res1 < res2);
    if (overlap) {
        if (res1 == res2) {
            if (p != nullptr) {
                p->x = static_cast<double>(res1.x);
                p->y = static_cast<double>(res1.y);
            }
            return 0; // Collinear and overlapping: touch at one endpoint.
        }
        if (p != nullptr && q != nullptr) {
            p->x = static_cast<double>(res1.x);
            p->y = static_cast<double>(res1.y);
            q->x = static_cast<double>(res2.x);
            q->y = static_cast<double>(res2.y);
        }
        return 1; // Collinear and overlapping: overlap is a segment.
    }
    return -1; // Collinear and disjoint.
}
if (EQ(c1, 0)) {
    return -1; // Parallel and disjoint.
}
auto t_num = ac_x * cd_y - ac_y * cd_x;
bool c1_pos = c1 > 0;
bool t_ok = c1_pos ? (TOUCH_IS_INTERSECT ? (LE(0, t_num) && LE(t_num, c1))
                                           : (LT(0, t_num) && LT(t_num, c1)))
                  : (TOUCH_IS_INTERSECT ? (LE(c1, t_num) && LE(t_num, 0))
                                           : (LT(c1, t_num) && LT(t_num, 0)));
bool u_ok = c1_pos ? (TOUCH_IS_INTERSECT ? (LE(0, c2) && LE(c2, c1)) : (LT(0, c2) && LT(c2, c1)))
                  : (TOUCH_IS_INTERSECT ? (LE(c1, c2) && LE(c2, 0)) : (LT(c1, c2) && LT(c2, 0)));
if (t_ok && u_ok) {
    if (p != nullptr) {
        double t = static_cast<double>(t_num) / c1;
        p->x = static_cast<double>(a.x + t * ab_x);
        p->y = static_cast<double>(a.y + t * ab_y);
    }
    return 0; // Non-parallel with one intersection.
}
return -1; // Non-parallel with no intersection.
}

// Simplified detection-only version (no output points needed); exact for integer Pt.
// Alternatively, just call the version above and pass static_cast<Pt *>(nullptr) for p and q.
template<typename Pt>
int seg_intersection(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, const bool TOUCH_IS_INTERSECT = true
) {
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto ab2 = ab_x * ab_x + ab_y * ab_y;
    auto cd2 = cd_x * cd_x + cd_y * cd_y;
    if (EQ(ab2, 0)) {
        return (TOUCH_IS_INTERSECT && point_on_segment(a, c, d)) ? 0 : -1;
    }
    if (EQ(cd2, 0)) {
        return (TOUCH_IS_INTERSECT && point_on_segment(c, a, b)) ? 0 : -1;
    }
    auto c1 = ab_x * cd_y - ab_y * cd_x;
    auto c2 = ac_x * ab_y - ac_y * ab_x;
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        Pt res1 = std::max(std::min(a, b), std::min(c, d));
        Pt res2 = std::min(std::max(a, b), std::max(c, d));
    }
}

```

```

    bool overlap = TOUCH_IS_INTERSECT ? !(res2 < res1) : (res1 < res2);
    if (overlap) {
        return (res1 == res2) ? 0 : 1;
    }
    return -1;
}
if (EQ(c1, 0)) {
    return -1;
}
auto t_num = ac_x * cd_y - ac_y * cd_x;
bool c1_pos = c1 > 0;
bool t_ok = c1_pos ? (TOUCH_IS_INTERSECT ? (LE(0, t_num) && LE(t_num, c1))
                                           : (LT(0, t_num) && LT(t_num, c1)))
                  : (TOUCH_IS_INTERSECT ? (LE(c1, t_num) && LE(t_num, 0))
                                           : (LT(c1, t_num) && LT(t_num, 0)));

bool u_ok = c1_pos ? (TOUCH_IS_INTERSECT ? (LE(0, c2) && LE(c2, c1)) : (LT(0, c2) && LT(c2, c1)))
                  : (TOUCH_IS_INTERSECT ? (LE(c1, c2) && LE(c2, 0)) : (LT(c1, c2) && LT(c2, 0)));
return (t_ok && u_ok) ? 0 : -1;
}

template<typename Pt>
Pt closest_point(double a, double b, double c, const Pt &p) {
    Pt res{};
    if (EQ(a, 0)) {
        res.x = p.x;
        res.y = -c / b;
    } else if (EQ(b, 0)) {
        res.x = -c / a;
        res.y = p.y;
    } else {
        line_intersection(a, b, c, -b, a, b * p.x - a * p.y, &res);
    }
    return res;
}

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return LT(x, p.x) || (EQ(x, p.x) && LT(y, p.y)); }
    bool operator>(const Point &p) const { return p < *this; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const PointI &p) const { return p < *this; }
};

int main() {
    Point p, q;

    assert(line_intersection(-1.0, 1.0, 0.0, 1.0, 1.0, -3.0, &p) == 0);
    assert(p == Point(1.5, 1.5));
    assert(line_intersection(Point(0, 0), Point(1, 1), Point(0, 4), Point(4, 0), &p) == 0);
    assert(p == Point(2, 2));

    auto test = [&](double a, double b, double c, double d, double e, double f, double g, double h) {

```

```

    return seg_intersection(Point(a, b), Point(c, d), Point(e, f), Point(g, h), &p, &q);
};
assert(0 == test(-4, 0, 4, 0, 0, -4, 0, 4) && p == Point(0, 0));
assert(0 == test(0, 0, 10, 10, 2, 2, 16, 4) && p == Point(2, 2));
assert(0 == test(-2, 2, -2, -2, -2, 0, 0, 0) && p == Point(-2, 0));
assert(0 == test(0, 4, 4, 4, 4, 0, 4, 8) && p == Point(4, 4));
assert(1 == test(10, 10, 0, 0, 2, 2, 6, 6));
assert(p == Point(2, 2) && q == Point(6, 6));
assert(-1 == test(6, 8, 8, 10, 12, 12, 4, 4));
assert(-1 == test(-4, 2, -8, 8, 0, 0, -4, 6));

assert(Point(2.5, 2.5) == closest_point(-1.0, -1.0, 5.0, Point(0, 0)));
assert(Point(3, 0) == closest_point(1.0, 0.0, -3.0, Point(0, 0)));

// Detection-only with integer coordinates (exact, no float needed).
assert(seg_intersection(PointI(0, 0), PointI(4, 4), PointI(0, 4), PointI(4, 0)) == 0);
assert(seg_intersection(PointI(0, 0), PointI(1, 0), PointI(2, 0), PointI(3, 0)) == -1);
assert(seg_intersection(PointI(0, 0), PointI(4, 4), PointI(0, 4), PointI(4, 0), &p) == 0);
assert(p == Point(2, 2));

// TOUCH_IS_INTERSECT = false treats endpoint-only contact as non-intersecting. The T-junction
// and shared-endpoint cases below intersect by default but are rejected when the flag is off.
assert(0 == seg_intersection(Point(0, 4), Point(4, 4), Point(4, 0), Point(4, 8), &p, &q));
assert(-1 == seg_intersection(Point(0, 4), Point(4, 4), Point(4, 0), Point(4, 8), &p, &q, false));
assert(-1 == seg_intersection(PointI(0, 0), PointI(2, 2), PointI(2, 2), PointI(6, 6), false));
return 0;
}

```

7.2.4 Circle Intersection

Circle tangent and intersection calculations in two dimensions. Such calculations are floating-point by nature, so this section mostly computes in `double`. The input point `p` to `tangent()` can be any type with numeric `.x` and `.y` fields, and intersection point outputs are written through a caller-supplied point type.

- `tangent(c, p, &l1, &l2)` determines the line(s) tangent to circle `c` that pass through point `p`, returning `-1` if there is no tangent line because `p` is strictly inside `c`, `0` if there is exactly one tangent line because `p` is on the boundary of `c` (in which case the line will be stored into pointer `l1` if it's not `nullptr`), or `1` if there are two tangent lines because `p` is strictly outside of `c` (in which case the lines will be stored into pointers `l1` and `l2` if they are not `nullptr`).
- `intersection(c, l, &p, &q)` determines the intersection between the circle `c` and line `l`, returning `-1` if there is no intersection, `0` if the line has one intersection point because the line is tangent (in which case it will be stored into pointer `p` if it's not `nullptr`), or `1` if there are two intersection points because the line crosses through the circle (in which case they will be stored into pointers `p` and `q` if they are not `nullptr`).
- `intersection(c1, c2, &p, &q)` determines the intersection points between two circles `c1` and `c2`, returning `-2` if circle `c2` completely encloses circle `c1`, `-1` if circle `c1` completely encloses circle `c2`, `0` if the circles are completely disjoint, `1` if the circles are tangent with one intersection (stored in `p` if it's not `nullptr`), `2` if the circles intersect at two points (stored in `p` and `q` if they are not `nullptr`), `3` if the circles are equal and intersect at infinite points.
- `intersection_area(c1, c2)` returns the intersection area of circles `c1` and `c2`.

Time Complexity: $O(1)$ for all operations.

Space Complexity: $O(1)$ auxiliary for all operations.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <type_traits>
#include <utility>

const double EPS = 1e-9, PI = acos(-1.0);

```



```

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }

struct Circle {
    double h, k, r;

    Circle(double h, double k, double r) {
        this->h = h;
        this->k = k;
        this->r = r;
    }
};

struct Line {
    double a, b, c;

    Line() : a(0), b(0), c(0) {}

    Line(double a, double b, double c) {
        if (!EQ(b, 0)) {
            this->a = a / b;
            this->c = c / b;
            this->b = 1;
        } else {
            this->c = c / a;
            this->a = 1;
            this->b = 0;
        }
    }

    Line(double px, double py, double qx, double qy) : a(0), b(0), c(0) {
        if (EQ(px, qx)) {
            if (!EQ(py, qy)) { // Vertical line.
                a = 1;
                b = 0;
                c = -px;
            } // Else, invalid line.
        } else {
            a = -static_cast<double>(py - qy) / (px - qx);
            b = 1;
            c = -(a * px) - (b * py);
        }
    }

    bool operator==(const Line &l) const { return EQ(a, l.a) && EQ(b, l.b) && EQ(c, l.c); }
};

template<typename Pt>
int tangent(const Circle &c, const Pt &p, Line *l1 = nullptr, Line *l2 = nullptr) {
    auto sqnorm = [](double x, double y) { return x * x + y * y; };
    double vopx = p.x - c.h, vopy = p.y - c.k;
    if (EQ(sqnorm(vopx, vopy), c.r * c.r)) { // Point on an edge.
        if (l1 != nullptr) { // Tangent through p is perpendicular to the radius cp.
            *l1 = Line(vopx, vopy, -(vopx * p.x + vopy * p.y));
        }
        return 0;
    }
    if (LE(sqnorm(vopx, vopy), c.r * c.r)) {
        return -1; // Point inside Circle, no intersection.
    }
}

```

```

double qx = vopx / c.r, qy = vopy / c.r;
double n = sqnorm(qx, qy), d = qy * sqrt(sqnorm(qx, qy) - 1.0);
double t1x = (qx - d) / n, t1y = c.k;
double t2x = (qx + d) / n, t2y = c.k;
if (!EQ(qy, 0)) { // Common case.
    t1y += c.r * (1.0 - t1x * qx) / qy;
    t2y += c.r * (1.0 - t2x * qx) / qy;
} else { // Point at center horizontal, y = 0.
    d = c.r * sqrt(1.0 - t1x * t1x);
    t1y += d;
    t2y -= d;
}
t1x = t1x * c.r + c.h;
t2x = t2x * c.r + c.h;
// Note: here, t1 and t2 are the two points of tangencies
if (l1 != nullptr) {
    *l1 = Line(p.x, p.y, t1x, t1y);
}
if (l2 != nullptr) {
    *l2 = Line(p.x, p.y, t2x, t2y);
}
return 1;
}

template<typename OutPt>
int intersection(const Circle &c, const Line &l, OutPt *p = nullptr, OutPt *q = nullptr) {
    double v = c.h * l.a + c.k * l.b + l.c;
    double aabb = l.a * l.a + l.b * l.b;
    double disc = v * v / aabb - c.r * c.r;
    if (disc > EPS) {
        return -1;
    }
    double x0 = -l.a * v / aabb, y0 = -l.b * v / aabb;
    if (disc > -EPS) {
        if (p != nullptr) {
            p->x = x0 + c.h;
            p->y = y0 + c.k;
        }
        return 0;
    }
    double k = sqrt(std::max(0.0, disc / -aabb));
    if (p != nullptr && q != nullptr) {
        p->x = x0 + k * l.b + c.h;
        p->y = y0 - k * l.a + c.k;
        q->x = x0 - k * l.b + c.h;
        q->y = y0 + k * l.a + c.k;
    }
    return 1;
}

template<typename OutPt>
int intersection(const Circle &c1, const Circle &c2, OutPt *p = nullptr, OutPt *q = nullptr) {
    if (EQ(c1.h, c2.h) && EQ(c1.k, c2.k)) {
        return EQ(c1.r, c2.r) ? 3 : (c1.r > c2.r ? -1 : -2);
    }
    double d12x = c2.h - c1.h, d12y = c2.k - c1.k;
    double d = hypot(d12x, d12y);
    if (LT(c1.r + c2.r, d)) {
        return 0;
    }
    if (LT(d, fabs(c1.r - c2.r))) {
        return c1.r > c2.r ? -1 : -2;
    }
    double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d);
    double x0 = c1.h + (d12x * a / d), y0 = c1.k + (d12y * a / d);
    double s = sqrt(c1.r * c1.r - a * a), rx = -d12y * s / d, ry = d12x * s / d;
    if (EQ(rx, 0) && EQ(ry, 0)) {
        if (p != nullptr) {

```

```

    p->x = x0;
    p->y = y0;
}
return 1;
}
if (p != nullptr && q != nullptr) {
    p->x = x0 - rx;
    p->y = y0 - ry;
    q->x = x0 + rx;
    q->y = y0 + ry;
}
return 2;
}

double intersection_area(const Circle &c1, const Circle &c2) {
    double r = std::min(c1.r, c2.r), R = std::max(c1.r, c2.r);
    double d = std::hypot(c2.h - c1.h, c2.k - c1.k);
    if (LE(d, R - r)) {
        return PI * r * r;
    }
    if (LE(R + r, d)) {
        return 0;
    }
    double alpha = std::max(-1.0, std::min(1.0, (d * d + r * r - R * R) / 2 / d / r));
    double beta = std::max(-1.0, std::min(1.0, (d * d + R * R - r * r) / 2 / d / R));
    return r * r * acos(alpha) + R * R * acos(beta) -
        0.5 * sqrt((-d + r + R) * (d + r - R) * (d - r + R) * (d + r + R));
}

```

Example Usage

```

#include <cassert>

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
};

int main() {
    Line l1, l2;
    assert(-1 == tangent(Circle(0, 0, 4), Point(1, 1), &l1, &l2));
    assert(0 == tangent(Circle(0, 0, sqrt(2)), Point(1, 1), &l1, &l2));
    assert(l1 == Line(-1, -1, 2));
    assert(1 == tangent(Circle(0, 0, 2), Point(2, 2), &l1, &l2));
    assert(l1 == Line(0, -2, 4));
    assert(l2 == Line(2, 0, -4));

    Point p, q;
    assert(-1 == intersection(Circle(1, 1, 3), Line(5, 3, -30), &p, &q));
    assert(0 == intersection(Circle(1, 1, 3), Line(0, 1, -4), &p, &q));
    assert(p == Point(1, 4));
    assert(1 == intersection(Circle(1, 1, 3), Line(0, 1, -1), &p, &q));
    assert(p == Point(4, 1));
    assert(q == Point(-2, 1));

    assert(-2 == intersection(Circle(1, 1, 1), Circle(0, 0, 3), &p, &q));
    assert(-1 == intersection(Circle(0, 0, 3), Circle(1, 1, 1), &p, &q));
    assert(0 == intersection(Circle(5, 0, 4), Circle(-5, 0, 4), &p, &q));
    assert(1 == intersection(Circle(-5, 0, 5), Circle(5, 0, 5), &p, &q));
    assert(p == Point(0, 0));
    assert(2 == intersection(Circle(-0.5, 0, 1), Circle(0.5, 0, 1), &p, &q));
    assert(p == Point(0, -sqrt(3) / 2));
    assert(q == Point(0, sqrt(3) / 2));

    // Each circle passes through the other's center.

```

```
double r = 3, a = intersection_area(Circle(-r / 2, 0, r), Circle(r / 2, 0, r));
assert(EQ(a, r * r * (2 * PI / 3 - sqrt(3) / 2)));
return 0;
}
```

7.2.5 Circle Tangents

Given two circles, compute their common tangent lines by returning the two points of tangency on each circle. External tangents touch both circles on the same side of the line joining their centers; internal tangents cross between the circles. A point can be treated as a zero-radius circle, so this also covers tangents from a point to a circle.

- `circle_tangents(c1, r1, c2, r2, INTERNAL)` returns 0, 1, or 2 tangent point pairs. Each pair `{p, q}` means the tangent line touches the first circle at `p` and the second circle at `q`. Set `INTERNAL = true` to get internal tangents; otherwise external tangents are returned.
- `all_circle_tangents(c1, r1, c2, r2)` returns all external and internal tangents.

If the two circles are identical, there are infinitely many tangents and the functions return an empty vector. If one circle strictly contains the other, there are no external tangents; if the circles overlap, there are no internal tangents.

Time Complexity: $O(1)$ per call.

Space Complexity: $O(1)$ auxiliary, excluding the returned vector.

```
#include <algorithm>
#include <cmath>
#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    Point operator+(const Point &p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point &p) const { return {x - p.x, y - p.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
    Point operator/(double k) const { return {x / k, y / k}; }
    double sqnorm() const { return x * x + y * y; }
    Point rotate90() const { return {-y, x}; }
};

template<typename PtA, typename PtB>
std::vector<std::pair<Point, Point>> circle_tangents(
    const PtA &c1, double r1, const PtB &c2, double r2, const bool INTERNAL = false
) {
    r1 = fabs(r1);
    r2 = fabs(r2);
    if (INTERNAL) {
        r2 = -r2;
    }
}
```

```

    }
    Point d(c2.x - c1.x, c2.y - c1.y);
    double dr = r1 - r2, d2 = d.sqnorm(), h2 = d2 - dr * dr;
    if (EQ(d2, 0) || LT(h2, 0)) {
        return {};
    }
    std::vector<std::pair<Point, Point>> res;
    for (double s : {-1.0, 1.0}) {
        Point v = (d * dr + d.rotate90() * (sqrt(std::max(0.0, h2)) * s)) / d2;
        res.push_back({Point(c1.x, c1.y) + v * r1, Point(c2.x, c2.y) + v * r2});
    }
    if (EQ(h2, 0)) {
        res.pop_back();
    }
    return res;
}

template<typename PtA, typename PtB>
std::vector<std::pair<Point, Point>> all_circle_tangents(
    const PtA &c1, double r1, const PtB &c2, double r2
) {
    auto res = circle_tangents(c1, r1, c2, r2);
    auto internal = circle_tangents(c1, r1, c2, r2, true);
    res.insert(res.end(), internal.begin(), internal.end());
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    Point a(0, 0), b(4, 0);
    auto ext = circle_tangents(a, 1, b, 1);
    assert(ext.size() == 2);
    assert(ext[0].first == Point(0, -1) && ext[0].second == Point(4, -1));
    assert(ext[1].first == Point(0, 1) && ext[1].second == Point(4, 1));

    auto in = circle_tangents(a, 1, b, 1, true);
    assert(in.size() == 2);
    assert(in[0].first == Point(0.5, -sqrt(3.0) / 2) && in[0].second == Point(3.5, sqrt(3.0) / 2));
    assert(in[1].first == Point(0.5, sqrt(3.0) / 2) && in[1].second == Point(3.5, -sqrt(3.0) / 2));

    auto point_to_circle = circle_tangents(Point(0, 0), 0, Point(2, 0), 1);
    assert(point_to_circle.size() == 2);
    assert(point_to_circle[0].first == Point(0, 0));

    assert(circle_tangents(Point(0, 0), 5, Point(1, 0), 1).empty());           // contained
    assert(circle_tangents(Point(0, 0), 2, Point(3, 0), 2, true).empty());   // overlapping
    assert(all_circle_tangents(Point(0, 0), 1, Point(4, 0), 1).size() == 4);
    return 0;
}

```

7.3 Polygons and Point Sets

7.3.1 Polygon Sorting and Area

Given a list of distinct points in two-dimensions, order them into a valid polygon and determine the area. The sorting comparators order two points by the sign of their cross product around a chosen center, that is, by angle.

The area functions use the shoelace formula, summing the cross products of consecutive vertex pairs. The functions are templated on the point type. The local `Point` struct (`double` coordinates) is the default; replace it with `Point`/`PointD`/`PointI` from 7.1.1 or any struct with numeric `.x` and `.y` fields and `<` / `==` operators.

- `cw_comp(a, b, c)` returns whether point `a` compares clockwise before `b` about `c`.
- `CWComparator(c)` / `CCWComparator(c)` return comparators for `std::sort`.
- `polygon_area_2x(lo, hi)` returns exactly double the area of the polygon with vertices specified by the range `[lo, hi)` of points in either clockwise or counter-clockwise order. The return value is integral or floating-point, depending on the input point type. For integer vertices, divide by 2 in the caller if the exact area is needed.
- `polygon_area(lo, hi)` returns the area as `double`.
- `polygon_centroid(lo, hi)` returns the centroid (center of mass) of a non-degenerate simple polygon with vertices in boundary order as an `std::pair<double, double>`. It uses signed shoelace weights, so either clockwise or counter-clockwise inputs will work.

Overflow warning: `cw_comp` and `polygon_area_2x` form cross products that grow like the squared coordinate magnitude (and the shoelace sum accumulates over all vertices). For integer point types use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) for large or numerous coordinates.

Time Complexity:

- $O(n)$ per call to `polygon_area`, where n is the number of points.
- $O(n)$ per call to `polygon_centroid`.
- $O(1)$ per call to `cw_comp` and the comparators.

Space Complexity: $O(1)$ auxiliary for all operations.

```
#include <algorithm>
#include <cmath>
#include <random>
#include <stdexcept>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

// Comparator (clockwise angular order about c). Exact: comparisons are pure sign tests on
// coordinate differences and the cross product, so it is a valid strict weak ordering and is
// exact for integer-coordinate points.
template<typename Pt>
bool cw_comp(const Pt &a, const Pt &b, const Pt &c) {
    if (a.x - c.x >= 0 && b.x - c.x < 0) {
        return true;
    }
    if (a.x - c.x < 0 && b.x - c.x >= 0) {
        return false;
    }
    if (a.x - c.x == 0 && b.x - c.x == 0) {
        if (a.y - c.y >= 0 || b.y - c.y >= 0) {
            return a.y > b.y;
        }
        return b.y > a.y;
    }
    auto acx = a.x - c.x, acy = a.y - c.y;
    auto bcx = b.x - c.x, bcy = b.y - c.y;
    auto det = acx * bcy - acy * bcx;
    if (det == 0) {
        auto acnorm = acx * acx + acy * acy;
        auto bcnorm = bcx * bcx + bcy * bcy;
        return acnorm > bcnorm;
    }
}
```

```

    }
    return det < 0;
}

// Returns 2 * area. Result is exact (integer) for integer-coordinate points.
template<typename It>
auto polygon_area_2x(It lo, It hi) {
    using T = decltype(lo->x * lo->y);
    if (lo == hi) {
        return T(0);
    }
    T area = 0;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        area += (T)(j->x - i->x) * (T)(j->y + i->y);
    }
    return area < T(0) ? -area : area;
}

template<typename It>
double polygon_area(It lo, It hi) {
    return static_cast<double>(polygon_area_2x(lo, hi)) / 2.0;
}

template<typename It>
std::pair<double, double> polygon_centroid(It lo, It hi) {
    if (hi - lo < 3) {
        throw std::runtime_error("Cannot compute centroid of a degenerate polygon.");
    }
    double cx = 0, cy = 0, area2 = 0;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        double cross = static_cast<double>(j->x) * i->y - static_cast<double>(i->x) * j->y;
        cx += (j->x + i->x) * cross;
        cy += (j->y + i->y) * cross;
        area2 += cross;
    }
    if (EQ(area2, 0)) {
        throw std::runtime_error("Cannot compute centroid of zero-area polygon.");
    }
    return {cx / (3 * area2), cy / (3 * area2)};
}

```

Example Usage

```

#include <cassert>
#include <iostream>
#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const Point &p) const { return p < *this; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const PointI &p) const { return p < *this; }
};

```

```

template<typename It>
Point mean_center(It lo, It hi) {
    if (lo == hi) {
        throw std::runtime_error("Cannot get center of an empty range.");
    }
    double x_sum = 0, y_sum = 0, n = hi - lo;
    for (It it = lo; it != hi; ++it) {
        x_sum += it->x;
        y_sum += it->y;
    }
    return Point(x_sum / n, y_sum / n);
}

int main() {
    vector<Point> points{Point(1, 3), Point(1, 2), Point(2, 1), Point(0, 0), Point(-1, 3)};
    vector<Point> v(points);
    std::mt19937 rng(1234567);
    std::shuffle(v.begin(), v.end(), rng);
    Point c = mean_center(v.begin(), v.end());
    assert(EQ(c.x, 0.6) && EQ(c.y, 1.8));
    sort(v.begin(), v.end(), [c](const Point &a, const Point &b) { return cw_comp(a, b, c); });
    cout << "Clockwise vertices: ";
    for (const auto &p : v) {
        cout << "(" << p.x << ", " << p.y << ") ";
    }
    cout << endl;
    assert(EQ(polygon_area(v.begin(), v.end()), 5));

    sort(v.begin(), v.end(), [c](const Point &a, const Point &b) { return cw_comp(b, a, c); });
    cout << "Counterclockwise vertices: ";
    for (const auto &p : v) {
        cout << "(" << p.x << ", " << p.y << ") ";
    }
    cout << endl;
    assert(EQ(polygon_area(v.begin(), v.end()), 5));

    // Integer points: polygon_area_2x is exact (no float arithmetic).
    vector<PointI> iv{{0, 0}, {4, 0}, {0, 3}}; // right triangle, area = 6
    assert(polygon_area_2x(iv.begin(), iv.end()) == 12); // exact int
    assert(EQ(polygon_area(iv.begin(), iv.end()), 6.0));
    auto centroid = polygon_centroid(iv.begin(), iv.end());
    assert(EQ(centroid.first, 4.0 / 3.0) && EQ(centroid.second, 1.0));
    return 0;
}

```

Example Output

```

Clockwise vertices: (1, 3) (1, 2) (2, 1) (0, 0) (-1, 3)
Counterclockwise vertices: (-1, 3) (0, 0) (2, 1) (1, 2) (1, 3)

```

7.3.2 Point-in-Polygon

Given a point `p` and a polygon, determines whether `p` lies inside using ray casting. A ray cast from `p` must cross the polygon's boundary an odd number of times exactly when `p` is inside, so the function counts crossings between a horizontal ray and each polygon edge. The function is templated on the point type `Pt`. The local `Point` struct (double coordinates) is the default; replace it with `Point`/`PointD`/`PointI` from 7.1.1 or any struct with numeric `.x` and `.y` fields. All comparisons are exact (no epsilon): the ray-crossing parity needs a consistent comparison to be correct, and the result is exact for integer-coordinate points.

- `point_in_polygon(p, lo, hi, EDGE_IS_INSIDE)` returns whether `p` lies within the polygon with vertices specified by the range `[lo, hi)` of points in either clockwise or counter-clockwise order. The `EDGE_IS_INSIDE` flag (default `true`) controls whether points exactly on an edge or vertex count as inside.

Overflow warning: the edge orientation test forms a cross product that grows like the squared coordinate magnitude. For integer point types use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) once coordinates exceed ~ 46000 .

Time Complexity: $O(n)$ per call, where n is the distance between `lo` and `hi`.

Space Complexity: $O(1)$ auxiliary.

```
#include <stdint>

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o) {
    // Overflow risk for integer Pt: these products are  $\sim O(\text{max\_coord}^2)$ ; use int64_t if necessary.
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// Detection is exact for integer-coordinate Pt.
template<typename Pt, typename It>
bool point_in_polygon(const Pt &p, It lo, It hi, bool EDGE_IS_INSIDE = true) {
    if (lo == hi) {
        return false;
    }
    bool res = false;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        if (i->y == p.y && (i->x == p.x || (j->y == p.y && (i->x <= p.x || j->x <= p.x)))) {
            return EDGE_IS_INSIDE;
        }
        if ((p.y < i->y) != (p.y < j->y)) {
            auto det = cross(*i, *j, p);
            if (det == 0) {
                return EDGE_IS_INSIDE;
            }
            if ((0 < det) != (i->y < j->y)) {
                res = !res;
            }
        }
    }
    return res;
}
```

Example Usage

```
#include <cassert>
#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    vector<Point> poly{Point(-1, 3), Point(1, 3), Point(2, 1), Point(0, 0)};
    assert(point_in_polygon(Point(1, 2), poly.begin(), poly.end()));
    assert(point_in_polygon(Point(0, 3), poly.begin(), poly.end()));
    assert(!point_in_polygon(Point(0, 3), poly.begin(), poly.end(), false));
    assert(!point_in_polygon(Point(0, 3.01), poly.begin(), poly.end()));
    assert(!point_in_polygon(Point(2, 2), poly.begin(), poly.end()));

    // Integer-coordinate points: exact detection.
    vector<PointI> ipoly{{0, 0}, {4, 0}, {4, 4}, {0, 4}};
```

```

assert(point_in_polygon(PointI(2, 2), ipoly.begin(), ipoly.end()));
assert(!point_in_polygon(PointI(4, 2), ipoly.begin(), ipoly.end(), false));
assert(!point_in_polygon(PointI(5, 2), ipoly.begin(), ipoly.end()));
return 0;
}

```

7.3.3 Convex Hull and Diametral Pair

Given a list of points in two dimensions, computes the convex hull using the monotone chain algorithm and the diametral pair using rotating calipers. Monotone chain sorts the points lexicographically and builds the lower and upper hulls in one pass each, popping any point that would create a clockwise turn. Rotating calipers then walks two antipodal pointers around the hull, advancing whichever increases the separation, visiting every candidate diametral pair in linear time. The functions are templated on the iterator type; the value type of the iterator is used as the point type. Both functions accept either floating-point or integral coordinates, but since they use only cross product comparisons, they happen to be exact for integral points.

- `convex_hull(lo, hi)` returns the convex hull in clockwise order for a range `[lo, hi)` of points. The input range is sorted lexicographically after the call. To instead return the hull points counter-clockwise order, replace every `>= 0` with `<= 0`.
- `diametral_pair(lo, hi)` returns the maximum-distance pair of points.

Overflow warning: `cross()` and the squared distance in `diametral_pair()` grow like the squared coordinate magnitude. With 32-bit `int` coordinates they overflow once coordinates exceed a few tens of thousands; use a 64-bit (`int64_t`) coordinate type for larger integer inputs.

Time Complexity: $O(n \log n)$ per call, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary for storage of the convex hull.

```

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <random>
#include <utility>
#include <vector>

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// Convex hull: exact for integer-coordinate points.
template<typename It>
auto convex_hull(It lo, It hi) {
    using Pt = typename std::iterator_traits<It>::value_type;
    int k = 0;
    if (hi - lo <= 1) {
        return std::vector<Pt>(lo, hi);
    }
    std::vector<Pt> res(2 * static_cast<int>(hi - lo));
    std::sort(lo, hi);
    for (It it = lo; it != hi; ++it) {
        while (k >= 2 && cross(res[k - 1], *it, res[k - 2]) >= 0) {
            k--;
        }
        res[k++] = *it;
    }
    int t = k + 1;
    for (It it = hi - 1; it != lo; --it) {
        while (k >= t && cross(res[k - 1], *it, res[k - 2]) >= 0) {

```

```

        k--;
    }
    res[k++] = *it;
}
res.resize(k - 1);
return res;
}

// Diametral pair: squared-distance comparisons are exact for integer-coordinate points.
template<typename It>
auto diametral_pair(It lo, It hi) {
    using Pt = typename std::iterator_traits<It>::value_type;
    auto h = convex_hull(lo, hi);
    int m = static_cast<int>(h.size());
    if (m == 0) {
        return std::pair<Pt, Pt>{};
    }
    if (m == 1) {
        return std::pair<Pt, Pt>{h[0], h[0]};
    }
    if (m == 2) {
        return std::pair<Pt, Pt>{h[0], h[1]};
    }
    int k = 1;
    while (std::abs(cross(h[0], h[(k + 1) % m], h[m - 1])) > std::abs(cross(h[0], h[k], h[m - 1]))) {
        k++;
    }
    auto sqdist = [](const Pt &a, const Pt &b) {
        // Overflow risk for integer Pt: ~0(max_coord^2); use int64_t if necessary.
        auto dx = a.x - b.x, dy = a.y - b.y;
        return dx * dx + dy * dy;
    };
    auto maxsq = sqdist(h[0], h[0]);
    std::pair<Pt, Pt> res{h[0], h[0]};
    for (int i = 0, j = k; i <= k && j < m; i++) {
        auto d = sqdist(h[i], h[j]);
        if (d > maxsq) {
            maxsq = d;
            res = {h[i], h[j]};
        }
        while (j < m && std::abs(cross(h[(i + 1) % m], h[(j + 1) % m], h[i])) >
            std::abs(cross(h[(i + 1) % m], h[j], h[i]))) {
            d = sqdist(h[i], h[(j + 1) % m]);
            if (d > maxsq) {
                maxsq = d;
                res = {h[i], h[(j + 1) % m]};
            }
            j++;
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &o) const { return x == o.x && y == o.y; }
    bool operator!=(const PointI &o) const { return !(*this == o); }
    bool operator<(const PointI &o) const { return x != o.x ? x < o.x : y < o.y; }
    bool operator>(const PointI &o) const { return o < *this; }
}

```

```

};

int main() {
    {
        vector<PointI> v{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
        std::mt19937 rng(12345);
        std::shuffle(v.begin(), v.end(), rng);
        vector<PointI> h{{-1, 3}, {1, 3}, {2, 1}, {0, 0}};
        assert(convex_hull(v.begin(), v.end()) == h);
    }
    {
        vector<PointI> v{{0, 0}, {3, 0}, {0, 3}, {1, 1}, {4, 4}};
        auto [p1, p2] = diametral_pair(v.begin(), v.end());
        assert(p1 == PointI(0, 0));
        assert(p2 == PointI(4, 4));
    }
    {
        vector<PointI> v{{0, 0}, {4, 0}, {4, 4}, {0, 4}, {2, 2}};
        auto h = convex_hull(v.begin(), v.end());
        assert(h.size() == 4); // interior point (2,2) excluded
        auto [p1, p2] = diametral_pair(v.begin(), v.end());
        // diametral pair is exact
        assert(
            (p1 == PointI(0, 0) && p2 == PointI(4, 4)) || (p1 == PointI(4, 4) && p2 == PointI(0, 0)) ||
            (p1 == PointI(4, 0) && p2 == PointI(0, 4)) || (p1 == PointI(0, 4) && p2 == PointI(4, 0))
        );
    }
    return 0;
}

```

7.3.4 Convex Polygon Queries

Given a convex polygon in counter-clockwise order, answer containment and tangent/intersection queries. Point containment is logarithmic by splitting the polygon into triangles sharing `poly[0]`. The line and tangent helpers below are written as simple linear scans; this is often shorter and less assumption-heavy than the classic logarithmic versions, while still documenting the exact query semantics in one place.

- `point_in_convex_polygon(poly, p, EDGE_IS_INSIDE)` returns whether point `p` lies inside the convex polygon `poly`. The polygon must be in counter-clockwise order, with no repeated final vertex. Points on the boundary count as inside unless `EDGE_IS_INSIDE` is set to `false`.
- `line_convex_polygon_intersection(poly, a, b)` describes where the infinite directed line through `a` $\rightarrow$ `b` hits the polygon: $-1, -1$ for no intersection, $i, -1$ for touching vertex i , i, i for containing side $i \rightarrow i + 1$, or i, j for crossing sides $i \rightarrow i + 1$ and $j \rightarrow j + 1$.
- `convex_polygon_tangents(poly, p)` returns the two tangent vertex indices from an outside point `p` as (left, right). The left tangent has all polygon vertices on or to the left of the directed line `p` $\rightarrow$ `poly[left]`; the right tangent is analogous for the right side.

For integer-coordinate inputs, all tests are exact provided the cross products do not overflow.

Time Complexity:

- $O(\log n)$ per query, where n is the number of polygon vertices.
- $O(n)$ per call to `line_convex_polygon_intersection` and `convex_polygon_tangents`.

Space Complexity: $O(1)$ auxiliary.

```

#include <algorithm>
#include <array>
#include <cassert>
#include <cstdint>
#include <vector>

```

```

template<typename Pt>
auto cross(const Pt &a, const Pt &b, const Pt &o) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

template<typename T>
int signum(const T &x) {
    return (T(0) < x) - (x < T(0));
}

template<typename Pt>
bool on_segment(const Pt &p, const Pt &a, const Pt &b) {
    return cross(a, b, p) == 0 && std::min(a.x, b.x) <= p.x && p.x <= std::max(a.x, b.x) &&
        std::min(a.y, b.y) <= p.y && p.y <= std::max(a.y, b.y);
}

template<typename Pt>
bool point_in_convex_polygon(
    const std::vector<Pt> &poly, const Pt &p, const bool EDGE_IS_INSIDE = true
) {
    int n = static_cast<int>(poly.size());
    if (n == 0) {
        return false;
    }
    if (n == 1) {
        return EDGE_IS_INSIDE && p.x == poly[0].x && p.y == poly[0].y;
    }
    if (n == 2) {
        return EDGE_IS_INSIDE && on_segment(p, poly[0], poly[1]);
    }
    int left = signum(cross(poly[1], p, poly[0]));
    int right = signum(cross(poly[n - 1], p, poly[0]));
    if (left < 0 || right > 0) {
        return false;
    }
    if (left == 0) {
        return EDGE_IS_INSIDE && on_segment(p, poly[0], poly[1]);
    }
    if (right == 0) {
        return EDGE_IS_INSIDE && on_segment(p, poly[0], poly[n - 1]);
    }
    int lo = 1, hi = n - 1;
    while (hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if (cross(poly[mid], p, poly[0]) >= 0) {
            lo = mid;
        } else {
            hi = mid;
        }
    }
    int s = signum(cross(poly[lo + 1], p, poly[lo]));
    return EDGE_IS_INSIDE ? s >= 0 : s > 0;
}

template<typename Pt>
std::array<int, 2> line_convex_polygon_intersection(
    const std::vector<Pt> &poly, const Pt &a, const Pt &b
) {
    int n = static_cast<int>(poly.size());
    std::vector<int> on, cross_edges;
    bool has_pos = false, has_neg = false;
    for (int i = 0; i < n; i++) {
        int s = signum(cross(b, poly[i], a));
        has_pos |= s > 0;
        has_neg |= s < 0;
        if (s == 0) {
            on.push_back(i);
        }
    }
}

```

```

    }
}
if (!has_pos && !has_neg) {
    return n >= 2 ? std::array<int, 2>{0, 0} : std::array<int, 2>{-1, -1};
}
if (!has_pos && has_neg) {
    for (int i = 0; i < static_cast<int>(on.size()); i++) {
        int j = (i + 1) % static_cast<int>(on.size());
        if ((on[i] + 1) % n == on[j]) {
            return {on[i], on[i]};
        }
    }
    return on.empty() ? std::array<int, 2>{-1, -1} : std::array<int, 2>{on[0], -1};
}
for (int i = 0; i < n; i++) {
    int j = (i + 1) % n;
    int si = signum(cross(b, poly[i], a)), sj = signum(cross(b, poly[j], a));
    if (si == 0 || si * sj < 0) {
        cross_edges.push_back(i);
    }
}
if (cross_edges.size() == 1) {
    return {cross_edges[0], -1};
}
auto line_parameter = [&](int i) {
    int j = (i + 1) % n;
    auto ex = poly[j].x - poly[i].x, ey = poly[j].y - poly[i].y;
    auto ax = a.x, ay = a.y, dx = b.x - a.x, dy = b.y - a.y;
    auto det = dx * ey - dy * ex;
    return static_cast<double>((poly[i].x - ax) * ey - (poly[i].y - ay) * ex) / det;
};
if (line_parameter(cross_edges[1]) < line_parameter(cross_edges[0])) {
    std::swap(cross_edges[0], cross_edges[1]);
}
return {cross_edges[0], cross_edges[1]};
}

template<typename Pt>
std::array<int, 2> convex_polygon_tangents(const std::vector<Pt> &poly, const Pt &p) {
    int n = static_cast<int>(poly.size());
    std::array<int, 2> res{-1, -1};
    for (int i = 0; i < n; i++) {
        bool left = true, right = true;
        for (int j = 0; j < n; j++) {
            int s = signum(cross(poly[i], poly[j], p));
            left &= s >= 0;
            right &= s <= 0;
        }
        if (left) {
            res[0] = i;
        }
        if (right) {
            res[1] = i;
        }
    }
    return res;
}

```

Example Usage

```

#include <algorithm>
using namespace std;

struct PointI {
    int64_t x, y;
    PointI(int64_t x = 0, int64_t y = 0) : x(x), y(y) {}
}

```

```

};

int main() {
    vector<PointI> square{{0, 0}, {4, 0}, {4, 4}, {0, 4}};
    assert(point_in_convex_polygon(square, PointI(2, 2)));
    assert(point_in_convex_polygon(square, PointI(4, 2)));
    assert(!point_in_convex_polygon(square, PointI(4, 2), false));
    assert(!point_in_convex_polygon(square, PointI(5, 2)));

    vector<PointI> pentagon{{0, 0}, {3, 0}, {5, 2}, {2, 5}, {-1, 3}};
    assert(point_in_convex_polygon(pentagon, PointI(2, 3)));
    assert(point_in_convex_polygon(pentagon, PointI(0, 0)));
    assert(!point_in_convex_polygon(pentagon, PointI(0, 0), false));
    assert(!point_in_convex_polygon(pentagon, PointI(5, 4)));

    assert(
        (line_convex_polygon_intersection(square, PointI(-1, 2), PointI(5, 2)) == array<int, 2>{3, 1})
    );
    assert(
        (line_convex_polygon_intersection(square, PointI(0, 0), PointI(4, 0)) == array<int, 2>{0, 0})
    );
    assert((
        line_convex_polygon_intersection(square, PointI(5, 0), PointI(5, 4)) == array<int, 2>{-1, -1}
    ));

    auto tangents = convex_polygon_tangents(square, PointI(6, 2));
    assert((tangents == array<int, 2>{2, 1}));
    return 0;
}

```

7.3.5 Minimum Enclosing Circle

Given a set of points in two dimensions, finds the unique circle of minimum radius (equivalently, minimum area) containing all points.

This implementation uses Welzl's randomized incremental algorithm. The points are shuffled and processed one at a time while maintaining the current minimum enclosing circle. Whenever a point lies outside the current circle, the circle is rebuilt with that point constrained to lie on the boundary. The final circle is determined by at most three boundary points.

- `minimum_enclosing_circle(lo, hi)` returns the minimum enclosing circle for the range `[lo, hi)` of points, where `lo` and `hi` must be random-access iterators. The input range is shuffled in place. The point type may be any type exposing numeric `.x` and `.y` members. The returned circle always uses `double` coordinates, so integer-coordinate inputs are accepted.

Time Complexity: $O(n)$ expected time per call to `minimum_enclosing_circle(lo, hi)`, where n is the distance between `lo` and `hi`, or $O(n^3)$ in the worst-case for a particular shuffle order.

Space Complexity: $O(1)$ auxiliary.

```

#include <algorithm>
#include <cmath>
#include <random>
#include <stdexcept>
#include <type_traits>
#include <utility>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

```

```

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}
template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }

double sqnorm(double x, double y) {
    return x * x + y * y;
}

// SFINAE guard: valid only when Pt exposes numeric .x/.y members. This keeps the templated point
// constructors from hijacking calls like Circle(double, double, double).
template<typename Pt>
using if_point = decltype(std::declval<const Pt &>().x, std::declval<const Pt &>().y, void());

struct Circle {
    double h, k, r;

    Circle() : h(0), k(0), r(0) {}
    Circle(double h, double k, double r) : h(h), k(k), r(fabs(r)) {}

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b) {
        h = (a.x + b.x) / 2.0;
        k = (a.y + b.y) / 2.0;
        r = std::hypot(a.x - h, a.y - k);
    }

    template<typename Pt, typename = if_point<Pt>>
    Circle(const Pt &a, const Pt &b, const Pt &c) {
        double an = sqnorm(b.x - c.x, b.y - c.y);
        double bn = sqnorm(a.x - c.x, a.y - c.y);
        double cn = sqnorm(a.x - b.x, a.y - b.y);
        double wa = an * (bn + cn - an), wb = bn * (an + cn - bn), wc = cn * (an + bn - cn);
        double w = wa + wb + wc;
        if (EQ(w, 0)) {
            throw std::runtime_error("No circumcircle from collinear points.");
        }
        h = (wa * a.x + wb * b.x + wc * c.x) / w;
        k = (wa * a.y + wb * b.y + wc * c.y) / w;
        r = std::hypot(a.x - h, a.y - k);
    }

    template<typename Pt, typename = if_point<Pt>>
    bool contains(const Pt &p) const {
        return LE(sqnorm(p.x - h, p.y - k), r * r);
    }
};

// Input points can be any type with .x and .y fields; Circle output is always double.
template<typename It>
Circle minimum_enclosing_circle(It lo, It hi) {
    if (lo == hi) {
        return Circle(0, 0, 0);
    }
    if (lo + 1 == hi) {
        return Circle(lo->x, lo->y, 0);
    }
    static std::mt19937 rng(std::random_device{}());
    std::shuffle(lo, hi, rng);
    Circle res(*lo, *(lo + 1));
    for (It i = lo + 2; i != hi; ++i) {
        if (res.contains(*i)) {
            continue;
        }
        res = Circle(*lo, *i);
        for (It j = lo + 1; j != i; ++j) {
            if (res.contains(*j)) {

```



```

        continue;
    }
    res = Circle(*i, *j);
    for (It k = lo; k != j; ++k) {
        if (!res.contains(*k)) {
            res = Circle(*i, *j, *k);
        }
    }
}
}
return res;
}

```

Example Usage

```

#include <cassert>
#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    vector<Point> v{{0, 0}, {0, 1}, {1, 0}, {1, 1}};
    Circle res = minimum_enclosing_circle(v.begin(), v.end());
    assert(EQ(res.h, 0.5) && EQ(res.k, 0.5) && EQ(res.r, 1 / sqrt(2)));

    // Integer-coordinate input: Circle output is always double.
    vector<PointI> iv{{0, 0}, {0, 2}, {2, 0}, {2, 2}};
    Circle ic = minimum_enclosing_circle(iv.begin(), iv.end());
    assert(EQ(ic.h, 1.0) && EQ(ic.k, 1.0));
    return 0;
}

```

7.3.6 Closest Pair

Given a list of points in two dimensions, finds the closest pair using a divide-and-conquer algorithm. The points are split in half by x -coordinate and each half is solved recursively; the combining step then only needs to examine points within the best distance so far of the dividing line, where each point in this strip is compared to a constant number of y -ordered neighbors.

- `closest_pair(lo, hi, &res)` returns the minimum squared distance between any two points in the range $[lo, hi)$, where `lo` and `hi` must be random-access iterators. The input range is reordered during the computation and is sorted by y -coordinate on return. If `res` is non-null, one closest pair is stored there in lexicographic order. The function is templated on the point type and works with any type exposing numeric `.x` and `.y` members.

The returned distance preserves the coordinate arithmetic type. For integer-coordinate inputs, the result is therefore an exact squared distance provided intermediate products do not overflow. The returned pair contains the original point type.

Overflow warning: squared distances grow like the square of the coordinate magnitude. For integer point types, use a 64-bit coordinate type (e.g. `PointL` from 7.1.1) when coordinates may exceed a few tens of thousands.

Time Complexity: $O(n \log^2 n)$ per call to `closest_pair(lo, hi, &res)`, where n is the distance between `lo` and `hi`.

Space Complexity:

- $O(n)$ auxiliary heap space.
- $O(\log n)$ recursion stack space.

```
#include <algorithm>
#include <cmath>
#include <iterator>
#include <limits>
#include <utility>
#include <vector>

template<typename Pt>
auto sqdist(const Pt &a, const Pt &b) {
    auto dx = a.x - b.x, dy = a.y - b.y;
    return dx * dx + dy * dy; // Overflow warning!
}

template<typename It, typename T>
T closest_pair_rec(
    It lo, It hi,
    std::pair<
        typename std::iterator_traits<It>::value_type,
        typename std::iterator_traits<It>::value_type> *res
) {
    using Pt = typename std::iterator_traits<It>::value_type;
    int n = hi - lo;
    if (n < 2) {
        return std::numeric_limits<T>::max();
    }
    std::sort(lo, hi, [](const Pt &a, const Pt &b) { return a.x != b.x ? a.x < b.x : a.y < b.y; });
    if (n <= 3) {
        T best = std::numeric_limits<T>::max();
        for (It i = lo; i != hi; ++i) {
            for (It j = i + 1; j != hi; ++j) {
                T d = sqdist(*i, *j);
                if (d < best) {
                    best = d;
                    if (res != nullptr) {
                        *res = std::minmax(*i, *j);
                    }
                }
            }
        }
        return best;
    }
    It mid = lo + n / 2;
    auto midx = mid->x;
    std::pair<Pt, Pt> lres, rres;
    T dl = closest_pair_rec<It, T>(lo, mid, &lres);
    T dr = closest_pair_rec<It, T>(mid, hi, &rres);
    T best;
    if (dl <= dr) {
        best = dl;
        if (res && dl != std::numeric_limits<T>::max()) {
            *res = lres;
        }
    } else {
        best = dr;
        if (res && dr != std::numeric_limits<T>::max()) {
            *res = rres;
        }
    }
    std::sort(lo, hi, [](const Pt &a, const Pt &b) { return a.y != b.y ? a.y < b.y : a.x < b.x; });
    std::vector<It> strip;
```

```

    for (It it = lo; it != hi; ++it) {
        auto dx = it->x - midx;
        if (dx * dx < best) {
            strip.push_back(it);
        }
    }
    for (int i = 0; i < static_cast<int>(strip.size()); i++) {
        for (int j = i + 1; j < static_cast<int>(strip.size()); j++) {
            auto dy = strip[j]->y - strip[i]->y;
            if (dy * dy >= best) {
                break;
            }
            T d = sqdist(*strip[i], *strip[j]);
            if (d < best) {
                best = d;
                if (res != nullptr) {
                    *res = std::minmax(*strip[i], *strip[j]);
                }
            }
        }
    }
    return best;
}

template<typename It>
auto closest_pair(
    It lo, It hi,
    std::pair<
        typename std::iterator_traits<It>::value_type,
        typename std::iterator_traits<It>::value_type> *res = nullptr
) {
    using T = decltype(sqdist(*lo, *lo));
    return closest_pair_rec<It, T>(lo, hi, res);
}

```

Example Usage

```

#include <cassert>
#include <type_traits>
#include <vector>
using namespace std;

const double EPS = 1e-9;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator<(const PointI &o) const { return x != o.x ? x < o.x : y < o.y; }
};

int main() {
    vector<Point> v{{2, 3}, {12, 30}, {40, 50}, {5, 1}, {12, 10}, {3, 4}};
    pair<Point, Point> res;
    assert(EQ(closest_pair(v.begin(), v.end(), &res), 2));
}

```

```

auto [p1, p2] = res;
assert(p1 == Point(2, 3) && p2 == Point(3, 4));

// Integer-coordinate input: exact pair selection, int squared distance returned.
vector<PointI> iv{{0, 0}, {10, 10}, {3, 4}};
pair<PointI, PointI> ires;
assert(closest_pair(iv.begin(), iv.end(), &ires) == 25); // (0,0)-(3,4) or (3,4)-(0,0)
auto [i1, i2] = ires;
assert((i1 == PointI(0, 0) && i2 == PointI(3, 4)) || (i1 == PointI(3, 4) && i2 == PointI(0, 0)));
return 0;
}

```

7.3.7 Multiple Segment Intersection

Given a list of line segments in two dimensions, determine whether any pair of segments intersect using a sweep line algorithm. Endpoint events are processed from left to right while an ordered set maintains the segments currently crossing the sweep line, sorted by y ; only segments that become adjacent in this set need to be tested, since any leftmost intersection must involve a pair that is adjacent just before it occurs. The input type `Segment<Pt>` is templated on the point type, so endpoints may be integer (`PointI`) or floating-point (`Point`/`PointD`), but must support `operator<` which orders points lexicographically. The cross-product sign tests in `seg_intersection` are exact for integer endpoints, so intersection detection is exact.

- `find_intersection(lo, hi, &res1, &res2, TOUCH_IS_INTERSECT)` returns whether any pair of segments intersect given a range `[lo, hi)` of segments, where `lo` and `hi` are random-access iterators. If an intersection is found, then one such pair of segments will be stored into pointers `res1` and `res2`. The `TOUCH_IS_INTERSECT` flag (default `true`) controls whether segments that meet only at an endpoint count as intersecting; set it to `false` to report proper crossings only, as when testing a polygon for self-intersection among edges that legitimately share endpoints.

Overflow warning: `seg_intersection` forms the usual quadratic cross products, but the y -ordering cross-multiplication (`ay * bdx`) is cubic in the coordinate magnitude. With 32-bit `int` endpoints this overflows once coordinates reach the low thousands, so use a 64-bit (`int64_t`) coordinate type for any non-trivial integer inputs.

Time Complexity: $O(n \log n)$ per call to `find_intersection(lo, hi, &res1, &res2)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary heap space for `find_intersection(lo, hi, &res1, &res2)`, where n is the distance between `lo` and `hi`.

```

#include <algorithm>
#include <cassert>
#include <cstdint>
#include <set>
#include <type_traits>
#include <utility>
#include <vector>

template<typename Pt>
bool point_on_segment(const Pt &p, const Pt &a, const Pt &b) {
    return (p.x - a.x) * (b.y - a.y) == (p.y - a.y) * (b.x - a.x) && std::min(a.x, b.x) <= p.x &&
        p.x <= std::max(a.x, b.x) && std::min(a.y, b.y) <= p.y && p.y <= std::max(a.y, b.y);
}

// Simplified detection-only version of `seg_intersection()` from 7.2.3 (exact for integral Pt).
template<typename Pt>
int seg_intersection(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, const bool TOUCH_IS_INTERSECT = true
) {
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;

```

```

auto cd_x = d.x - c.x, cd_y = d.y - c.y;
auto ab2 = ab_x * ab_x + ab_y * ab_y;
auto cd2 = cd_x * cd_x + cd_y * cd_y;
if (ab2 == 0) {
    return (TOUCH_IS_INTERSECT && point_on_segment(a, c, d)) ? 0 : -1;
}
if (cd2 == 0) {
    return (TOUCH_IS_INTERSECT && point_on_segment(c, a, b)) ? 0 : -1;
}
auto c1 = ab_x * cd_y - ab_y * cd_x;
auto c2 = ac_x * ab_y - ac_y * ab_x;
if (c1 == 0 && c2 == 0) {
    Pt res1 = std::max(std::min(a, b), std::min(c, d));
    Pt res2 = std::min(std::max(a, b), std::max(c, d));
    bool overlap = TOUCH_IS_INTERSECT ? !(res2 < res1) : (res1 < res2);
    if (overlap) {
        return (res1 == res2) ? 0 : 1;
    }
    return -1;
}
if (c1 == 0) {
    return -1;
}
auto t_num = ac_x * cd_y - ac_y * cd_x;
bool c1_pos = c1 > 0;
bool t_between_01 =
    c1_pos ? (TOUCH_IS_INTERSECT ? (0 <= t_num && t_num <= c1) : (0 < t_num && t_num < c1))
    : (TOUCH_IS_INTERSECT ? (c1 <= t_num && t_num <= 0) : (c1 < t_num && t_num < 0));
bool u_between_01 = c1_pos ? (TOUCH_IS_INTERSECT ? (0 <= c2 && c2 <= c1) : (0 < c2 && c2 < c1))
    : (TOUCH_IS_INTERSECT ? (c1 <= c2 && c2 <= 0) : (c1 < c2 && c2 < 0));
return (t_between_01 && u_between_01) ? 0 : -1;
}

// The point type Pt must support operator< (used to canonicalize endpoint order).
template<typename Pt>
struct Segment {
    Pt p, q;
    Segment() {}
    Segment(const Pt &p, const Pt &q) : p(std::min(p, q)), q(std::max(p, q)) {}
};

template<typename Pt>
bool intersects(
    const Segment<Pt> &s1, const Segment<Pt> &s2, const bool TOUCH_IS_INTERSECT = true
) {
    return seg_intersection(s1.p, s1.q, s2.p, s2.q, TOUCH_IS_INTERSECT) >= 0;
}

// Seg is deduced from the output pointers (e.g. Segment<PointI>* or Segment<PointD>*).
template<typename It, typename Seg>
bool find_intersection(It lo, It hi, Seg *res1, Seg *res2, const bool TOUCH_IS_INTERSECT = true) {
    using Pt = std::decay_t<decltype(lo->p)>;
    struct Event {
        Pt p;
        int type;
        It seg;
    };
    std::vector<Event> e;
    for (It it = lo; it != hi; ++it) {
        if (it->p > it->q) {
            std::swap(it->p, it->q);
        }
        e.push_back(Event{it->p, 1, it});
        e.push_back(Event{it->q, -1, it});
    }
    std::sort(e.begin(), e.end(), [](const Event &a, const Event &b) {
        if (a.p.x != b.p.x) {
            return a.p.x < b.p.x;
        }
    });
}

```

```

    }
    if (a.type != b.type) {
        return b.type < a.type;
    }
    return a.p.y < b.p.y;
});
// Compare y-values at sweep coordinate x without division: y = (p.y*dx + dy*(x - p.x)) / dx.
// Vertical segments use their lower endpoint
auto ycmp = [](const auto &a, const auto &b, auto x) {
    // Overflow risk for integer Pt: the final cross-multiply is ~0(max_coord^3); use 64-bit
    // coordinates for non-trivial integer inputs.
    auto adx = a.q.x - a.p.x, bdx = b.q.x - b.p.x;
    auto ay = a.p.y * adx + (a.q.y - a.p.y) * (x - a.p.x);
    auto by = b.p.y * bdx + (b.q.y - b.p.y) * (x - b.p.x);
    if (adx == 0) {
        ay = a.p.y;
        adx = 1;
    }
    if (bdx == 0) {
        by = b.p.y;
        bdx = 1;
    }
    auto lhs = ay * bdx, rhs = by * adx;
    return (lhs < rhs) ? -1 : ((rhs < lhs) ? 1 : 0);
};
auto cmp = [lo, ycmp](It a, It b) {
    if (a == b) {
        return false;
    }
    auto x = std::max(a->p.x, b->p.x);
    int order = ycmp(*a, *b, x);
    return (order == 0) ? (a - lo < b - lo) : (order < 0);
};
using active_set = std::set<It, decltype(cmp)>;
active_set s(cmp);
std::vector<typename active_set::iterator> position(hi - lo);
for (const auto &ev : e) {
    It seg = ev.seg;
    if (ev.type == 1) {
        auto it = s.insert(seg).first;
        position[seg - lo] = it;
        auto next = it;
        if (++next != s.end() && intersects(**next, *seg, TOUCH_IS_INTERSECT)) {
            *res1 = **next;
            *res2 = *seg;
            return true;
        }
        if (it != s.begin() && intersects(**--it, *seg, TOUCH_IS_INTERSECT)) {
            *res1 = **it;
            *res2 = *seg;
            return true;
        }
    }
    else {
        auto it = position[seg - lo];
        auto next = it;
        if (++next != s.end() && it != s.begin()) {
            auto prev = it;
            if (intersects(**next, **--prev, TOUCH_IS_INTERSECT)) {
                *res1 = **next;
                *res2 = **prev;
                return true;
            }
        }
        s.erase(it);
    }
}
return false;
}

```

Example Usage

```
#include <vector>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const Point &p) const { return p < *this; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
    bool operator>(const PointI &p) const { return p < *this; }
};

int main() {
    { // Double-coordinate segments.
        vector<Segment<Point>> v{
            Segment<Point>(Point(0, 0), Point(2, 2)), Segment<Point>(Point(3, 0), Point(0, -1)),
            Segment<Point>(Point(0, 2), Point(2, -2)), Segment<Point>(Point(0, 3), Point(9, 0))
        };
        Segment<Point> res1, res2;
        assert(find_intersection(v.begin(), v.end(), &res1, &res2));
        assert(res1.p == Point(0, 0) && res1.q == Point(2, 2));
        assert(res2.p == Point(0, 2) && res2.q == Point(2, -2));
    }
    { // Integer-coordinate segments: detection is exact.
        vector<Segment<PointI>> v{
            Segment<PointI>({0, 0}, {2, 2}), Segment<PointI>({3, 0}, {0, -1}),
            Segment<PointI>({0, 2}, {2, -2}), Segment<PointI>({0, 3}, {9, 0})
        };
        Segment<PointI> res1, res2;
        assert(find_intersection(v.begin(), v.end(), &res1, &res2));

        vector<Segment<PointI>> disjoint{
            Segment<PointI>({0, 0}, {1, 0}), Segment<PointI>({0, 5}, {1, 5})
        };
        assert(!find_intersection(disjoint.begin(), disjoint.end(), &res1, &res2));
    }
    { // TOUCH_IS_INTERSECT = false reports proper crossings only, ignoring shared endpoints.
        vector<Segment<PointI>> shared{
            Segment<PointI>({0, 0}, {2, 2}), Segment<PointI>({2, 2}, {4, 0})
        };
        Segment<PointI> res1, res2;
        assert(find_intersection(shared.begin(), shared.end(), &res1, &res2));
        assert(!find_intersection(shared.begin(), shared.end(), &res1, &res2, false));
    }
    return 0;
}
```

7.3.8 Maximum Collinear Points

Given a set of points with integer coordinates, compute the maximum number of points that lie on one line. For every anchor point, normalize the direction vector to every other point by dividing by `gcd(abs(dx), abs(dy))` and choosing a canonical sign; equal normalized directions from the same anchor are collinear with it.

- `max_collinear_points(points)` returns the largest number of collinear points. Duplicate points are counted as separate input points.

Time Complexity: $O(n^2 \log n)$, where n is the number of points.

Space Complexity: $O(n)$ auxiliary.

```
#include <algorithm>
#include <cassert>
#include <stdint>
#include <map>
#include <numeric>
#include <utility>
#include <vector>

struct Point {
    int64_t x, y;
    Point(int64_t x = 0, int64_t y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return x == p.x && y == p.y; }
};

int max_collinear_points(const std::vector<Point> &p) {
    int n = static_cast<int>(p.size()), ans = std::min(n, 1);
    for (int i = 0; i < n; i++) {
        std::map<std::pair<int64_t, int64_t>, int> cnt;
        int duplicates = 0, best = 0;
        for (int j = 0; j < n; j++) {
            if (i == j) {
                continue;
            }
            int64_t dx = p[j].x - p[i].x, dy = p[j].y - p[i].y;
            if (dx == 0 && dy == 0) {
                duplicates++;
                continue;
            }
            int64_t g = std::gcd(std::abs(dx), std::abs(dy));
            dx /= g;
            dy /= g;
            if (dx < 0 || (dx == 0 && dy < 0)) {
                dx = -dx;
                dy = -dy;
            }
            best = std::max(best, ++cnt[{dx, dy}]);
        }
        ans = std::max(ans, best + duplicates + 1);
    }
    return ans;
}
```

Example Usage

```
using namespace std;

int main() {
    vector<Point> p{{0, 0}, {1, 1}, {2, 2}, {3, 5}, {4, 6}, {3, 3}};
    assert(max_collinear_points(p) == 4);
    p.push_back(Point(1, 1));
    assert(max_collinear_points(p) == 5);
    assert(max_collinear_points({}) == 0);
    assert(max_collinear_points({Point(2, 3)}) == 1);
    return 0;
}
```


7.4 Advanced Planar Geometry

7.4.1 Convex Polygon Cut

Given a convex polygon and a directed line from `p` to `q`, clips the polygon to the closed left half-plane of that line. The polygon is walked edge by edge: vertices on the left side of the line are kept, and whenever an edge crosses the line, the intersection point is appended to the output.

- `convex_cut(lo, hi, p, q)` returns the portion of the polygon lying on or to the left of the directed line `p` → `q`. The input range `[lo, hi)` must contain the vertices of a convex polygon in boundary order, either clockwise or counterclockwise. The returned polygon preserves that boundary order. If `p` equals `q`, the cutting line is invalid and an error is thrown.

The function is templated on the input point type. Side classification is done with cross products. For integer-coordinate inputs, classification is exact only if the intermediate products do not overflow. Edge-line intersection points are computed in floating point, and the returned polygon uses `Point` with `double` coordinates.

Time Complexity: $O(n)$ per call to `convex_cut(lo, hi, p, q)`, where n is the distance between `lo` and `hi`.

Space Complexity: $O(n)$ auxiliary space for the returned polygon.

```
#include <cmath>
#include <cstdint>
#include <stdexcept>
#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
};

template<typename PtA, typename PtB, typename PtO>
double cross(const PtA &a, const PtB &b, const PtO &o) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

template<typename PtA, typename PtO, typename PtB>
int turn(const PtA &a, const PtO &o, const PtB &b) {
    double c = cross(a, b, o);
    return LT(c, 0) ? -1 : (LT(0, c) ? 1 : 0);
}

template<typename PtA, typename PtB>
int line_intersection(
    const PtA &p1, const PtA &p2, const PtB &p3, const PtB &p4, Point *p = nullptr
) {
    double a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    double c1 = -(p1.x * p2.y - p2.x * p1.y);
    double a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    double c2 = -(p3.x * p4.y - p4.x * p3.y);
```

```

double x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
double det = a1 * b2 - a2 * b1;
if (EQ(det, 0)) {
    return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
}
if (p != nullptr) {
    *p = Point(x / det, y / det);
}
return 0;
}

template<typename It, typename Pt>
std::vector<Point> convex_cut(It lo, It hi, const Pt &p, const Pt &q) {
    if (EQ(p.x, q.x) && EQ(p.y, q.y)) {
        throw std::runtime_error("Cannot cut using line from identical points.");
    }
    if (lo == hi) {
        return {};
    }
    std::vector<Point> res;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        Point pj(static_cast<double>(j->x), static_cast<double>(j->y));
        Point pi(static_cast<double>(i->x), static_cast<double>(i->y));
        int d1 = turn(q, p, pj), d2 = turn(q, p, pi);
        if (d1 >= 0) {
            res.push_back(pj);
        }
        if (d1 * d2 < 0) {
            Point r;
            line_intersection(p, q, pj, pi, &r);
            res.push_back(r);
        }
    }
    return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
};

int main() {
    {
        vector<Point> v{{1, 3}, {2, 2}, {2, 1}, {0, 0}, {-1, 3}};
        // Cut using the vertical line through (0, 0).
        vector<Point> c{{-1, 3}, {0, 3}, {0, 0}};
        assert(convex_cut(v.begin(), v.end(), Point(0, 0), Point(0, 1)) == c);
    }
    { // On a non-convex input, the result may be multiple disjoint polygons!
        vector<Point> v{{0, 0}, {2, 2}, {0, 4}, {3, 4}, {3, 0}};
        vector<Point> c{{1, 0}, {0, 0}, {1, 1}, {1, 3}, {0, 4}, {1, 4}};
        assert(convex_cut(v.begin(), v.end(), Point(1, 0), Point(1, 4)) == c);
    }
    {
        vector<PointI> v{{0, 0}, {4, 0}, {4, 4}, {0, 4}};
        vector<Point> c{{0, 4}, {0, 0}, {2, 0}, {2, 4}};
        assert(convex_cut(v.begin(), v.end(), PointI(2, 0), PointI(2, 4)) == c);
    }
    return 0;
}

```

7.4.2 Half-Plane Intersection

Given a list of directed lines, compute the convex polygon contained in every left half-plane. Half-plane intersection sorts the lines by angle, keeps only the tightest representative among parallel lines, and maintains a deque of lines whose pairwise intersections form the current boundary.

- `half_plane_intersection(planes)` returns the polygon cut out by the half-planes in counter-clockwise order. Each half-plane is represented by `HalfPlane(p, q)`, meaning the closed region to the left of the directed line $p \rightarrow q$.
- If the intersection is empty, the function returns an empty vector.
- If the true intersection is unbounded, the returned polygon is not meaningful. Add explicit bounding-box half-planes when a bounded polygon is required.

Time Complexity: $O(n \log n)$, where n is the number of half-planes.

Space Complexity: $O(n)$ auxiliary.

```
#include <algorithm>
#include <cmath>
#include <deque>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    Point operator+(const Point &p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point &p) const { return {x - p.x, y - p.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
    double cross(const Point &p) const { return x * p.y - y * p.x; }
};

struct HalfPlane {
    Point p, dir;
    double ang;

    HalfPlane() : p(), dir(), ang(0) {}
    HalfPlane(Point p, Point q) : p(p), dir(q - p), ang(atan2(dir.y, dir.x)) {}

    bool out(const Point &q) const { return LT(dir.cross(q - p), 0); }
    Point intersection(const HalfPlane &h) const {
        double t = h.dir.cross(p - h.p) / dir.cross(h.dir);
        return p + dir * t;
    }
    bool operator<(const HalfPlane &h) const {
        if (!EQ(ang, h.ang)) {
            return ang < h.ang;
        }
        return dir.cross(h.p - p) < 0; // tighter half-plane first among parallel lines
    }
};

std::vector<Point> half_plane_intersection(std::vector<HalfPlane> planes) {
```

```

std::sort(planes.begin(), planes.end());
std::vector<HalfPlane> unique;
for (const HalfPlane &h : planes) {
    if (!unique.empty() && EQ(unique.back().dir.cross(h.dir), 0)) {
        continue;
    }
    unique.push_back(h);
}
std::deque<HalfPlane> dq;
auto bad_back = [&](const HalfPlane &h) {
    return dq.size() >= 2 && h.out(dq[dq.size() - 2].intersection(dq.back()));
};
auto bad_front = [&](const HalfPlane &h) {
    return dq.size() >= 2 && h.out(dq[0].intersection(dq[1]));
};
for (const HalfPlane &h : unique) {
    while (bad_back(h)) {
        dq.pop_back();
    }
    while (bad_front(h)) {
        dq.pop_front();
    }
    dq.push_back(h);
}
while (dq.size() >= 3 && dq.front().out(dq[dq.size() - 2].intersection(dq.back()))) {
    dq.pop_back();
}
while (dq.size() >= 3 && dq.back().out(dq[0].intersection(dq[1]))) {
    dq.pop_front();
}
if (dq.size() < 3) {
    return {};
}
std::vector<Point> res;
for (int i = 0; i < static_cast<int>(dq.size()); i++) {
    if (EQ(dq[i].dir.cross(dq[(i + 1) % dq.size()].dir), 0)) {
        return {};
    }
    res.push_back(dq[i].intersection(dq[(i + 1) % dq.size()]));
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

double polygon_area(const vector<Point> &p) {
    double area = 0;
    for (int i = 0, j = static_cast<int>(p.size()) - 1; i < static_cast<int>(p.size()); j = i++) {
        area += p[j].cross(p[i]);
    }
    return fabs(area) / 2.0;
}

int main() {
    vector<HalfPlane> box{
        HalfPlane(Point(0, 0), Point(4, 0)),
        HalfPlane(Point(4, 0), Point(4, 4)),
        HalfPlane(Point(4, 4), Point(0, 4)),
        HalfPlane(Point(0, 4), Point(0, 0)),
    };
    auto square = half_plane_intersection(box);
    assert(square.size() == 4);
    assert(EQ(polygon_area(square), 16));
}

```

```

box.push_back(HalfPlane(Point(2, 0), Point(2, 4))); // x <= 2
auto rect = half_plane_intersection(box);
assert(rect.size() == 4);
assert(EQ(polygon_area(rect), 8));

vector<HalfPlane> empty{
    HalfPlane(Point(0, 0), Point(1, 0)), // y >= 0
    HalfPlane(Point(1, 1), Point(0, 1)), // y <= 1
    HalfPlane(Point(0, 2), Point(1, 2)), // y >= 2
};
assert(half_plane_intersection(empty).empty());
return 0;
}

```

7.4.3 Minkowski Sum

Given two convex polygons, compute their Minkowski sum: the set of all points $a + b$ where a comes from the first polygon and b comes from the second. After rotating both polygons to start at their lowest-leftmost vertex, the boundary edges of the sum are obtained by merging the two cyclic edge-angle lists. Minkowski sums are useful for collision/translation problems, convex polygon distance checks, and diameter computations via $P + (-P)$.

- `minkowski_sum(a, b)` returns the convex polygon representing the sum. Inputs must be convex polygons in boundary order, clockwise or counter-clockwise, with no repeated final vertex. Collinear consecutive output vertices are removed.

Time Complexity: $O(n + m)$, where n and m are the polygon sizes.

Space Complexity: $O(n + m)$ auxiliary.

```

#include <algorithm>
#include <cmath>
#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    Point operator+(const Point &p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point &p) const { return {x - p.x, y - p.y}; }
    double cross(const Point &p) const { return x * p.y - y * p.x; }
};

double polygon_area_2x(const std::vector<Point> &p) {
    double area = 0;
    for (int i = 0, j = static_cast<int>(p.size()) - 1; i < static_cast<int>(p.size()); j = i++) {
        area += p[j].cross(p[i]);
    }
    return area;
}

std::vector<Point> normalize(std::vector<Point> p) {
    while (p.size() > 1 && p.front() == p.back()) {
        p.pop_back();
    }
}

```

```

    if (p.size() <= 1) {
        return p;
    }
    if (polygon_area_2x(p) < 0) {
        std::reverse(p.begin(), p.end());
    }
    int start = 0;
    for (int i = 1; i < static_cast<int>(p.size()); i++) {
        if (p[i].y < p[start].y || (EQ(p[i].y, p[start].y) && p[i].x < p[start].x)) {
            start = i;
        }
    }
    std::rotate(p.begin(), p.begin() + start, p.end());
    return p;
}

std::vector<Point> remove_collinear(std::vector<Point> p) {
    std::vector<Point> res;
    for (const Point &q : p) {
        while (res.size() >= 2 && EQ((res.back() - res[res.size() - 2]).cross(q - res.back()), 0)) {
            res.pop_back();
        }
        if (res.empty() || !(q == res.back())) {
            res.push_back(q);
        }
    }
    while (res.size() >= 3 && EQ((res.back() - res[res.size() - 2]).cross(res[0] - res.back()), 0)) {
        res.pop_back();
    }
    while (res.size() >= 3 && EQ((res[0] - res.back()).cross(res[1] - res[0]), 0)) {
        res.erase(res.begin());
    }
    return res;
}

std::vector<Point> minkowski_sum(std::vector<Point> a, std::vector<Point> b) {
    a = normalize(std::move(a));
    b = normalize(std::move(b));
    if (a.empty() || b.empty()) {
        return {};
    }
    if (a.size() == 1) {
        for (Point &p : b) {
            p = p + a[0];
        }
        return b;
    }
    if (b.size() == 1) {
        for (Point &p : a) {
            p = p + b[0];
        }
        return a;
    }
    int n = static_cast<int>(a.size()), m = static_cast<int>(b.size());
    a.push_back(a[0]);
    a.push_back(a[1]);
    b.push_back(b[0]);
    b.push_back(b[1]);
    std::vector<Point> res;
    int i = 0, j = 0;
    while (i < n || j < m) {
        res.push_back(a[i] + b[j]);
        if (i == n) {
            j++;
            continue;
        }
        if (j == m) {
            i++;

```

```

        continue;
    }
    double cr = (a[i + 1] - a[i]).cross(b[j + 1] - b[j]);
    if (cr > -EPS) {
        i++;
    }
    if (cr < EPS) {
        j++;
    }
}
return remove_collinear(res);
}

```

Example Usage

```

#include <cassert>
using namespace std;

int main() {
    vector<Point> square{{0, 0}, {1, 0}, {1, 1}, {0, 1}};
    vector<Point> tri{{0, 0}, {2, 0}, {0, 2}};
    vector<Point> sum = minkowski_sum(square, tri);
    vector<Point> expected{{0, 0}, {3, 0}, {3, 1}, {1, 3}, {0, 3}};
    assert(sum == expected);

    vector<Point> shifted = minkowski_sum(square, {Point(2, 3)});
    vector<Point> expected_shifted{{2, 3}, {3, 3}, {3, 4}, {2, 4}};
    assert(shifted == expected_shifted);
    return 0;
}

```

7.4.4 Polygon Intersection and Union

Given two simple (non-self-intersecting) polygons, determine the areas of their intersection and union using a sweep line algorithm and the inclusion-exclusion principle. Every vertex and every pairwise edge intersection contributes its x -coordinate as a sweep boundary, so within each vertical slab between consecutive boundaries the two borders cannot cross, and the slab's overlap is accumulated exactly from trapezoid areas. The union then follows by inclusion-exclusion as $\text{area}(A) + \text{area}(B)$ minus the intersection.

- `intersection_area(lo1, hi1, lo2, hi2)` returns the intersection area of two polygons respectively specified by two ranges `[lo1, hi1)` and `[lo2, hi2)` of vertices in clockwise order, where `lo1`, `hi1`, `lo2`, and `hi2` must be random-access iterators.
- `union_area(lo1, hi1, lo2, hi2)` returns the union area of two polygons respectively specified by two ranges `[lo1, hi1)` and `[lo2, hi2)` of vertices in clockwise order, where `lo1`, `hi1`, `lo2`, and `hi2` must be random-access iterators.

Overflow warning: the segment-intersection tests form cross products in the input point's coordinate type, which grow like the squared coordinate magnitude. For integral point types, use 64-bit coordinates (e.g. `PointL` from 7.1.1) if necessary.

Time Complexity: $O(n \cdot m \cdot (n + m) \cdot \log(n + m))$ per call to `intersection_area(lo1, hi1, lo2, hi2)` and `union_area(lo1, hi1, lo2, hi2)`, where n the number of vertices in the first polygon, and m is the number of points in the second polygon (or equivalently $O(N^3 \log N)$ for $N = n + m$).

Space Complexity: $O(nm)$ auxiliary heap space for `intersection_area(lo1, hi1, lo2, hi2)` and `union_area(lo1, hi1, lo2, hi2)`, where n is the sum of distances between `lo1` and `hi1` and `lo2` and `hi2` respectively.

```

#include <algorithm>

```

```

#include <cmath>
#include <cstdint>
#include <set>
#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }

template<typename Pt>
bool point_on_segment(const Pt &p, const Pt &a, const Pt &b) {
    // Overflow risk for integer Pt: these products are ~0(max_coord^2); use int64_t if necessary.
    return EQ((p.x - a.x) * (b.y - a.y) - (p.y - a.y) * (b.x - a.x), 0) &&
        LE(std::min(a.x, b.x), p.x) && LE(p.x, std::max(a.x, b.x)) &&
        LE(std::min(a.y, b.y), p.y) && LE(p.y, std::max(a.y, b.y));
}

// Specialized version of seg_intersection() from 7.2.3, simplified for TOUCH_IS_INTERSECT = true,
// returning -1 for no intersection, 0 for one intersection point, 1 for positive length overlap.
template<typename Pt>
int seg_intersection1(
    const Pt &a, const Pt &b, const Pt &c, const Pt &d, double *outx, double *outy
) {
    auto ab_x = b.x - a.x, ab_y = b.y - a.y;
    auto ac_x = c.x - a.x, ac_y = c.y - a.y;
    auto cd_x = d.x - c.x, cd_y = d.y - c.y;
    auto ab2 = ab_x * ab_x + ab_y * ab_y;
    auto cd2 = cd_x * cd_x + cd_y * cd_y;
    if (EQ(ab2, 0)) {
        if (point_on_segment(a, c, d)) {
            *outx = a.x;
            *outy = a.y;
            return 0;
        }
        return -1;
    }
    if (EQ(cd2, 0)) {
        if (point_on_segment(c, a, b)) {
            *outx = c.x;
            *outy = c.y;
            return 0;
        }
        return -1;
    }
    auto c1 = ab_x * cd_y - ab_y * cd_x;
    auto c2 = ac_x * ab_y - ac_y * ab_x;
    if (EQ(c1, 0) && EQ(c2, 0)) { // Collinear.
        Pt res1 = std::max(std::min(a, b), std::min(c, d));
        Pt res2 = std::min(std::max(a, b), std::max(c, d));
        if (!(res2 < res1)) {
            if (res1 == res2) {
                *outx = static_cast<double>(res1.x);
                *outy = static_cast<double>(res1.y);
                return 0;
            }
            return 1;
        }
    }
    return -1;
}

```



```

}
if (EQ(c1, 0)) {
    return -1;
}
auto t_num = ac_x * cd_y - ac_y * cd_x;
bool c1_pos = c1 > 0;
bool t_ok = c1_pos ? (LE(0, t_num) && LE(t_num, c1)) : (LE(c1, t_num) && LE(t_num, 0));
bool u_ok = c1_pos ? (LE(0, c2) && LE(c2, c1)) : (LE(c1, c2) && LE(c2, 0));
if (t_ok && u_ok) {
    double t = static_cast<double>(t_num) / c1;
    *outx = static_cast<double>(a.x + t * ab_x);
    *outy = static_cast<double>(a.y + t * ab_y);
    return 0;
}
return -1;
}

// The two segments may use different point types (e.g. polygon edge vs. sweep line).
template<typename PtA, typename PtB>
int line_intersection1(
    const PtA &p1, const PtA &p2, const PtB &p3, const PtB &p4, double *outx, double *outy
) {
    auto a1 = p2.y - p1.y, b1 = p1.x - p2.x;
    auto c1 = -(p1.x * p2.y - p2.x * p1.y);
    auto a2 = p4.y - p3.y, b2 = p3.x - p4.x;
    auto c2 = -(p3.x * p4.y - p4.x * p3.y);
    auto x = -(c1 * b2 - c2 * b1), y = -(a1 * c2 - a2 * c1);
    auto det = a1 * b2 - a2 * b1;
    if (EQ(det, 0)) {
        return (EQ(x, 0) && EQ(y, 0)) ? 1 : -1;
    }
    if (outx != nullptr && outy != nullptr) {
        *outx = static_cast<double>(x) / det;
        *outy = static_cast<double>(y) / det;
    }
    return 0;
}

struct Event {
    double y;
    int mask_delta;
    Event(double y = 0, int mask_delta = 0) : y(y), mask_delta(mask_delta) {}
};

template<typename It>
double intersection_area(It lo1, It hi1, It lo2, It hi2) {
    if (lo1 == hi1 || lo2 == hi2) {
        return 0;
    }
    struct _PointD {
        double x, y;
    }; // For line intersection with the sweep line.
    std::vector<It> plo{lo1, lo2}, phi{hi1, hi2};
    std::set<double> xs;
    for (It it = lo1; it != hi1; ++it) {
        xs.insert(it->x);
    }
    for (It it = lo2; it != hi2; ++it) {
        xs.insert(it->x);
    }
    for (It i1 = lo1, j1 = hi1 - 1; i1 != hi1; j1 = i1++) {
        for (It i2 = lo2, j2 = hi2 - 1; i2 != hi2; j2 = i2++) {
            double outx, outy;
            if (seg_intersection1(*i1, *j1, *i2, *j2, &outx, &outy) == 0) {
                xs.insert(outx);
            }
        }
    }
}

```

```

std::vector<double> xsa(xs.begin(), xs.end());
double res = 0;
for (int k = 0; k + 1 < static_cast<int>(xsa.size()); k++) {
    double x = (xsa[k] + xsa[k + 1]) / 2;
    _PointD sweep0{x, 0}, sweep1{x, 1};
    std::vector<Event> events;
    for (int poly = 0; poly < 2; poly++) {
        It lo = plo[poly], hi = phi[poly];
        double area = 0;
        for (It i = lo, j = hi - 1; i != hi; j = i++) {
            area += (j->x - i->x) * (j->y + i->y);
        }
        for (It j = lo, i = hi - 1; j != hi; i = j++) {
            double px, py;
            if (line_intersection1(*j, *i, sweep0, sweep1, &px, &py) == 0) {
                double y = py, x0 = i->x, x1 = j->x;
                int sgn_area = (area < 0 ? 1 : (area > 0 ? -1 : 0));
                if (x0 < x && x1 > x) {
                    events.emplace_back(y, sgn_area * (1 << poly));
                } else if (x0 > x && x1 < x) {
                    events.emplace_back(y, -sgn_area * (1 << poly));
                }
            }
        }
    }
    std::sort(events.begin(), events.end(), [](const Event &e1, const Event &e2) {
        if (!EQ(e1.y, e2.y)) {
            return e1.y < e2.y;
        }
        return e1.mask_delta < e2.mask_delta;
    });
    double a = 0;
    int cnt[2] = {0, 0};
    for (int j = 0; j < static_cast<int>(events.size()); j++) {
        if (cnt[0] != 0 && cnt[1] != 0) {
            a += events[j].y - events[j - 1].y;
        }
        int bit = std::abs(events[j].mask_delta);
        int poly = (bit == 1 ? 0 : 1);
        cnt[poly] += events[j].mask_delta / bit;
    }
    res += a * (xsa[k + 1] - xsa[k]);
}
return res;
}

template<typename It>
double polygon_area(It lo, It hi) {
    if (lo == hi) {
        return 0;
    }
    double area = 0;
    for (It i = lo, j = hi - 1; i != hi; j = i++) {
        area += (j->x - i->x) * (j->y + i->y);
    }
    return fabs(area / 2.0);
}

template<typename It>
double union_area(It lo1, It hi1, It lo2, It hi2) {
    return polygon_area(lo1, hi1) + polygon_area(lo2, hi2) - intersection_area(lo1, hi1, lo2, hi2);
}

```

Example Usage

```
#include <cassert>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator!=(const Point &p) const { return !(*this == p); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &o) const { return x == o.x && y == o.y; }
    bool operator!=(const PointI &p) const { return !(*this == p); }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
};

int main() {
    vector<Point> p, s;
    // Irregular pentagon a triangle of area 1.5 overlapping quadrant 2.
    p.emplace_back(1, 3);
    p.emplace_back(1, 2);
    p.emplace_back(2, 1);
    p.emplace_back(0, 0);
    p.emplace_back(-1, 3);
    // Square of area 12.5 in quadrant 2.
    s.emplace_back(0, 0);
    s.emplace_back(0, 3);
    s.emplace_back(-3, 3);
    s.emplace_back(-3, 0);
    assert(EQ(1.5, intersection_area(p.begin(), p.end(), s.begin(), s.end())));
    assert(EQ(12.5, union_area(p.begin(), p.end(), s.begin(), s.end())));

    // Integer-coordinate polygons are accepted (computation proceeds in double).
    vector<PointI> ip{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<PointI> is{{0, 0}, {0, 3}, {-3, 3}, {-3, 0}};
    assert(EQ(1.5, intersection_area(ip.begin(), ip.end(), is.begin(), is.end())));
    assert(EQ(12.5, union_area(ip.begin(), ip.end(), is.begin(), is.end())));
    return 0;
}
```

7.4.5 Delaunay Triangulation (Simple)

Given a set P of distinct two-dimensional points, a Delaunay triangulation of P is a triangulation of the convex hull of P such that no point of P lies strictly inside the circumcircle of any triangle in the triangulation. The Delaunay triangulation is not necessarily unique when four or more points are cocircular.

This implementation produces one valid triangulation using a simple brute-force algorithm. Each candidate triangle is tested against the empty-circumcircle condition, and candidates whose edges would properly cross already accepted triangles are rejected.

- `delaunay_triangulation(p)` returns a Delaunay triangulation of the distinct points in `p` as a vector of clockwise-oriented vertex triples, or an empty vector if no triangulation exists (e.g. fewer than three points, or all points collinear).

All arithmetic uses the point's own coordinate type, so integer inputs yield an exact triangulation. The lifted-paraboloid test scales as $\sim 96 \cdot C^4$ where `C` is the coordinate magnitude, so a 64-bit integer coordinate type is safe up to $|C| \leq \sim 17600$. For floating-point coordinates the test is subject to the usual rounding error.

Time Complexity: $O(n^6)$ per call to `delaunay_triangulation(p)`, where n is the number of points. The empty-circumcircle test contributes $O(n^4)$; the remaining factor comes from checking candidate triangles against previously accepted triangles.

Space Complexity: $O(n)$ auxiliary heap space, excluding the returned triangulation.

```
#include <cmath>
#include <tuple>
#include <type_traits>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T, typename U> bool LE(T a, U b) { return !LT(b, a); }

// Specialized version of seg_intersection() from 7.2.3, simplified for TOUCH_IS_INTERSECT = false,
// i.e. we're detecting proper crossings of two non-parallel segments whose interiors intersect.
template<typename Pt>
bool proper_seg_intersection(const Pt &a, const Pt &b, const Pt &c, const Pt &d) {
    auto cross = [](const Pt &o, const Pt &p, const Pt &q) {
        return (p.x - o.x) * (q.y - o.y) - (p.y - o.y) * (q.x - o.x); // Overflow warning!
    };
    auto c1 = cross(a, b, c), c2 = cross(a, b, d), c3 = cross(c, d, a), c4 = cross(c, d, b);
    return ((LT(c1, 0) && LT(0, c2)) || (LT(c2, 0) && LT(0, c1))) &&
        ((LT(c3, 0) && LT(0, c4)) || (LT(c4, 0) && LT(0, c3)));
}

template<typename Pt>
std::vector<std::tuple<Pt, Pt, Pt>> delaunay_triangulation(const std::vector<Pt> &p) {
    int n = static_cast<int>(p.size());
    std::vector<decltype(Pt::x)> z;
    for (const auto &[px, py] : p) {
        z.emplace_back(px * px + py * py); // Overflow warning!
    }
    std::vector<std::tuple<Pt, Pt, Pt>> res;
    std::vector<std::tuple<int, int, int>> res_idx;
    for (int i = 0; i < n - 2; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = i + 1; k < n; k++) {
                if (j == k) {
                    continue;
                }
                // Overflow warning!
                auto nx = (p[j].y - p[i].y) * (z[k] - z[i]) - (p[k].y - p[i].y) * (z[j] - z[i]);
                auto ny = (p[k].x - p[i].x) * (z[j] - z[i]) - (p[j].x - p[i].x) * (z[k] - z[i]);
                auto nz = (p[j].x - p[i].x) * (p[k].y - p[i].y) - (p[k].x - p[i].x) * (p[j].y - p[i].y);
                if (LE(0, nz)) {
                    continue;
                }
                std::vector<Pt> s1{p[i], p[j], p[k], p[i]};
                for (int m = 0; m < n; m++) {
                    if (LT(0, nx * (p[m].x - p[i].x) + ny * (p[m].y - p[i].y) + nz * (z[m] - z[i]))) {
                        goto skip;
                    }
                }
                // Reject candidates whose edges properly cross already accepted triangles.
                for (auto [t0, t1, t2] : res_idx) {
                    std::vector<Pt> s2{p[t0], p[t1], p[t2], p[t0]};
                    for (int u = 0; u < 3; u++) {
```

```

        for (int v = 0; v < 3; v++) {
            if (proper_seg_intersection(s1[u], s1[u + 1], s2[v], s2[v + 1])) {
                goto skip;
            }
        }
    }
    res.emplace_back(p[i], p[j], p[k]);
    res_idx.emplace_back(i, j, k);
skip:;
}
}
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
    bool operator<(const Point &p) const { return x != p.x ? x < p.x : y < p.y; }
};

struct PointI {
    int x, y;
    PointI(int x = 0, int y = 0) : x(x), y(y) {}
    bool operator==(const PointI &p) const { return x == p.x && y == p.y; }
    bool operator<(const PointI &p) const { return x != p.x ? x < p.x : y < p.y; }
};

int main() {
    vector<Point> v{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<Point, Point, Point>> t{
        {Point{1, 3}, Point{1, 2}, Point{-1, 3}},
        {Point{1, 3}, Point{2, 1}, Point{1, 2}},
        {Point{1, 2}, Point{2, 1}, Point{0, 0}},
        {Point{1, 2}, Point{0, 0}, Point{-1, 3}}
    };
    assert(delaunay_triangulation(v) == t);

    // Integer-coordinate inputs are triangulated using exact integer arithmetic.
    vector<PointI> iv{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<PointI, PointI, PointI>> ti{
        {PointI{1, 3}, PointI{1, 2}, PointI{-1, 3}},
        {PointI{1, 3}, PointI{2, 1}, PointI{1, 2}},
        {PointI{1, 2}, PointI{2, 1}, PointI{0, 0}},
        {PointI{1, 2}, PointI{0, 0}, PointI{-1, 3}}
    };
    assert(delaunay_triangulation(iv) == ti);
    return 0;
}

```

7.4.6 Delaunay Triangulation (Guibas-Stolfi)

Given a set P of distinct two-dimensional points, a Delaunay triangulation of P is a triangulation of the convex hull of P such that no point of P lies strictly inside the circumcircle of any triangle in the triangulation. The Delaunay triangulation is not necessarily unique when four or more points are cocircular.

This implementation produces one valid triangulation using the Guibas-Stolfi divide-and-conquer algorithm with a quad-edge data structure. The point set is split in half, each half is triangulated recursively, and the halves are stitched together from the bottom up by a sequence of cross edges, with circumcircle tests deciding each connecting edge and deleting invalidated ones.

- `delaunay_triangulation(pts)` sorts `pts` lexicographically and removes duplicate points in place (so the input vector is modified), then returns the triangles of one Delaunay triangulation as a vector of counterclockwise-oriented vertex triples (`std::tuple`) of the input point type. If fewer than three non-collinear unique points remain, the result is empty.

All arithmetic uses the point's own coordinate type, so integer inputs yield an exact triangulation. The in-circle test is a degree-4 polynomial in the coordinates and dominates the overflow budget: a 64-bit integer coordinate type stays exact up to $|C| \sim 1.4e4$, where C is the coordinate magnitude. For floating-point coordinates the predicates are subject to the usual rounding error.

Time Complexity: $O(n \log n)$ per call to `delaunay_triangulation(pts)`, where n is the number of input points.

Space Complexity: $O(n)$ heap space for the quad-edge structure and returned triangulation.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <tuple>
#include <type_traits>
#include <utility>
#include <vector>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool LT(T a, U b) {
    return std::is_integral_v<C> ? C(a) < C(b) : C(a) < C(b) - static_cast<C>(EPS);
}

template<typename T>
struct Point {
    T x, y;
    Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator==(const Point &p) const { return EQ(x, p.x) && EQ(y, p.y); }
};

template<typename T>
T cross(const Point<T> &a, const Point<T> &b, const Point<T> &o) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x); // Overflow warning!
}

template<typename T>
T incircle(const Point<T> &a, const Point<T> &b, const Point<T> &c, const Point<T> &d) {
    T adx = a.x - d.x, ady = a.y - d.y;
    T bdx = b.x - d.x, bdy = b.y - d.y;
    T cdx = c.x - d.x, cdy = c.y - d.y;
    T abdet = adx * bdy - bdx * ady; // Overflow warning!
    T bcdet = bdx * cdy - cdx * bdy;
    T cadet = cdx * ady - adx * cdy;
    T alift = adx * adx + ady * ady;
    T blift = bdx * bdx + bdy * bdy;
    T clift = cdx * cdx + cdy * cdy;
    return alift * bcdet + blift * cadet + clift * abdet;
}

template<typename T>
```

```

struct Edge {
    Point<T> origin;
    Edge *rot, *onext;
    bool primal, deleted, used;

    Edge *rev() const { return rot->rot; }
    Edge *lnext() const { return rot->rev()->onext->rot; }
    Edge *oprev() const { return rot->onext->rot; }
    Point<T> dest() const { return rev()->origin; }
};

template<typename T>
struct QuadEdgePool {
    std::vector<Edge<T>*> edges;

    QuadEdgePool() = default;

    ~QuadEdgePool() {
        for (Edge<T>*e : edges) {
            delete e;
        }
    }

    QuadEdgePool(const QuadEdgePool &) = delete;
    QuadEdgePool &operator=(const QuadEdgePool &) = delete;

    Edge<T>*make_edge(const Point<T> &u, const Point<T> &v) {
        Edge<T>*e1 = new Edge<T>, *e2 = new Edge<T>, *e3 = new Edge<T>, *e4 = new Edge<T>;
        edges.push_back(e1);
        edges.push_back(e2);
        edges.push_back(e3);
        edges.push_back(e4);
        e1->origin = u;
        e2->origin = v;
        e1->rot = e3;
        e2->rot = e4;
        e3->rot = e2;
        e4->rot = e1;
        e1->onext = e1;
        e2->onext = e2;
        e3->onext = e4;
        e4->onext = e3;
        e1->primal = e2->primal = true;
        e3->primal = e4->primal = false;
        e1->deleted = e2->deleted = e3->deleted = e4->deleted = false;
        e1->used = e2->used = e3->used = e4->used = false;
        return e1;
    }

    void delete_edge(Edge<T>*e) {
        splice(e, e->oprev());
        splice(e->rev(), e->rev()->oprev());
        e->deleted = e->rot->deleted = e->rev()->deleted = e->rev()->rot->deleted = true;
    }

    Edge<T>*connect(Edge<T>*a, Edge<T>*b) {
        Edge<T>*e = make_edge(a->dest(), b->origin);
        splice(e, a->lnext());
        splice(e->rev(), b);
        return e;
    }

    static void splice(Edge<T>*a, Edge<T>*b) {
        std::swap(a->onext->rot->onext, b->onext->rot->onext);
        std::swap(a->onext, b->onext);
    }
};

```

```

template<typename T>
bool left_of(const Point<T> &p, Edge<T> *e) {
    return LT(0, cross(e->origin, e->dest(), p));
}

template<typename T>
bool right_of(const Point<T> &p, Edge<T> *e) {
    return LT(cross(e->origin, e->dest(), p), 0);
}

template<typename T>
bool in_circle(const Point<T> &a, const Point<T> &b, const Point<T> &c, const Point<T> &d) {
    return LT(0, incircle(a, b, c, d));
}

template<typename T>
std::pair<Edge<T> *, Edge<T> *> build_triangulation(
    std::vector<Point<T>> &p, int lo, int hi, QuadEdgePool<T> &pool
) {
    if (hi - lo == 1) {
        Edge<T> *a = pool.make_edge(p[lo], p[hi]);
        return {a, a->rev()};
    }
    if (hi - lo == 2) {
        Edge<T> *a = pool.make_edge(p[lo], p[lo + 1]);
        Edge<T> *b = pool.make_edge(p[lo + 1], p[hi]);
        pool.splice(a->rev(), b);
        int side = LT(0, cross(p[lo], p[lo + 1], p[hi]))
            ? 1
            : (LT(cross(p[lo], p[lo + 1], p[hi]), 0) ? -1 : 0);
        if (side == 0) {
            return {a, b->rev()};
        }
        Edge<T> *c = pool.connect(b, a);
        if (side == 1) {
            return {a, b->rev()};
        }
        return {c->rev(), c};
    }
    int mid = lo + (hi - lo) / 2;
    auto [ldo, ldi] = build_triangulation(p, lo, mid, pool);
    auto [rdi, rdo] = build_triangulation(p, mid + 1, hi, pool);
    while (true) {
        if (left_of(rdi->origin, ldi)) {
            ldi = ldi->lnext();
        } else if (right_of(ldi->origin, rdi)) {
            rdi = rdi->rev()->onext;
        } else {
            break;
        }
    }
    Edge<T> *base = pool.connect(rdi->rev(), ldi);
    if (ldi->origin == ldo->origin) {
        ldo = base->rev();
    }
    if (rdi->origin == rdo->origin) {
        rdo = base;
    }
    while (true) {
        Edge<T> *lcand = base->rev()->onext;
        if (right_of(lcand->dest(), base)) {
            while (in_circle(base->dest(), base->origin, lcand->dest(), lcand->onext->dest())) {
                Edge<T> *next = lcand->onext;
                pool.delete_edge(lcand);
                lcand = next;
            }
        }
        Edge<T> *rcand = base->oprev();
    }
}

```



```

    if (right_of(rcand->dest(), base)) {
        while (in_circle(base->dest(), base->origin, rcand->dest(), rcand->oprev()->dest())) {
            Edge<T> *prev = rcand->oprev();
            pool.delete_edge(rcand);
            rcand = prev;
        }
    }
    bool lvalid = right_of(lcand->dest(), base), rvalid = right_of(rcand->dest(), base);
    if (!lvalid && !rvalid) {
        break;
    }
    if (!lvalid ||
        (rvalid && in_circle(lcand->dest(), lcand->origin, rcand->origin, rcand->dest()))) {
        base = pool.connect(rcand, base->rev());
    } else {
        base = pool.connect(base->rev(), lcand->rev());
    }
}
return {ldo, rdo};
}

template<typename Pt>
auto delaunay_triangulation(std::vector<Pt> &p) {
    using T = decltype(Pt::x);
    using Triangle = std::tuple<Pt, Pt, Pt>;
    // Forgo using epsilon comparisons here, since std::sort needs a strict weak ordering to avoid
    // unpredictably sorting/deduping near-coincident points.
    auto ptcmp = [](const Pt &a, const Pt &b) { return a.x == b.x ? a.y < b.y : a.x < b.x; };
    // Rotates triangle vertices so that the lexicographically smallest comes first, preserving the
    // cyclic (counterclockwise) order, to give each triangle a canonical, comparable representation.
    auto canonical = [&](const Pt &a, const Pt &b, const Pt &c) -> Triangle {
        if (ptcmp(b, a) && ptcmp(b, c)) {
            return {b, c, a};
        }
        if (ptcmp(c, a) && ptcmp(c, b)) {
            return {c, a, b};
        }
        return {a, b, c};
    };
    std::sort(p.begin(), p.end(), ptcmp);
    p.erase(std::unique(p.begin(), p.end(), ptcmp), p.end());
    std::vector<Triangle> res;
    if (p.size() < 3) {
        return res;
    }
    QuadEdgePool<T> pool;
    build_triangulation(p, 0, static_cast<int>(p.size()) - 1, pool);
    for (Edge<T> *start : pool.edges) {
        if (!start->primal || start->deleted || start->used) {
            continue;
        }
        std::vector<Pt> face;
        Edge<T> *curr = start;
        do {
            curr->used = true;
            face.push_back(curr->origin);
            curr = curr->lnext();
        } while (curr != start);
        if (face.size() == 3 && LT(0, cross(face[0], face[1], face[2]))) {
            res.push_back(canonical(face[0], face[1], face[2]));
        }
    }
    std::sort(res.begin(), res.end(), [&](const Triangle &t1, const Triangle &t2) {
        const auto &[a1, b1, c1] = t1;
        const auto &[a2, b2, c2] = t2;
        if (ptcmp(a1, a2) || (!ptcmp(a2, a1) && ptcmp(b1, b2))) {
            return true;
        }
    });
}

```

```

    if (ptcmp(a2, a1) || (!ptcmp(a1, a2) && ptcmp(b2, b1))) {
        return false;
    }
    return ptcmp(c1, c2);
};
return res;
}

```

Example Usage

```

#include <cassert>
using namespace std;

using PointL = Point<int64_t>;
using PointD = Point<double>;

int main() {
    vector<PointD> v{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<PointD, PointD, PointD>> t{
        {PointD{-1, 3}, PointD{0, 0}, PointD{1, 2}},
        {PointD{-1, 3}, PointD{1, 2}, PointD{1, 3}},
        {PointD{0, 0}, PointD{2, 1}, PointD{1, 2}},
        {PointD{1, 2}, PointD{2, 1}, PointD{1, 3}}
    };
    assert(delaunay_triangulation(v) == t);

    // Integer-coordinate inputs are triangulated using exact integer arithmetic.
    vector<PointL> iv{{1, 3}, {1, 2}, {2, 1}, {0, 0}, {-1, 3}};
    vector<tuple<PointL, PointL, PointL>> ti{
        {PointL{-1, 3}, PointL{0, 0}, PointL{1, 2}},
        {PointL{-1, 3}, PointL{1, 2}, PointL{1, 3}},
        {PointL{0, 0}, PointL{2, 1}, PointL{1, 2}},
        {PointL{1, 2}, PointL{2, 1}, PointL{1, 3}}
    };
    assert(delaunay_triangulation(iv) == ti);
    return 0;
}

```

7.4.7 Manhattan MST

Given points in the plane, produce a small set of candidate edges guaranteed to contain a minimum spanning tree under Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. The plane is swept in four rotations/reflections; in each sweep, dominance by $x + y$ identifies the only nearby candidates that can matter. Run Kruskal on the returned edges to obtain the MST.

- `manhattan_mst_edges(points)` returns $O(n)$ candidate edges w, u, v .
- `manhattan_mst_weight(points)` is a small Kruskal wrapper that returns the MST weight.

Time Complexity: $O(n \log n)$, where n is the number of points.

Space Complexity: $O(n)$ auxiliary.

```

#include <algorithm>
#include <array>
#include <cassert>
#include <cstdint>
#include <map>
#include <numeric>
#include <vector>

struct Point {
    int64_t x, y;
};

```

```

    Point(int64_t x = 0, int64_t y = 0) : x(x), y(y) {}
};

std::vector<std::tuple<int64_t, int, int>> manhattan_mst_edges(std::vector<Point> p) {
    int n = static_cast<int>(p.size());
    std::vector<int> id(n);
    std::iota(id.begin(), id.end(), 0);
    std::vector<std::tuple<int64_t, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) {
        std::sort(id.begin(), id.end(), [&](int i, int j) {
            return p[i].x + p[i].y < p[j].x + p[j].y;
        });
        std::map<int64_t, int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-p[i].y); it != sweep.end(); it) {
                int j = it->second;
                int64_t dx = p[i].x - p[j].x, dy = p[i].y - p[j].y;
                if (dy > dx) {
                    break;
                }
                edges.emplace_back(dx + dy, i, j);
                it = sweep.erase(it);
            }
            sweep[-p[i].y] = i;
        }
        for (Point &q : p) {
            if (rot & 1) {
                q.x = -q.x;
            } else {
                std::swap(q.x, q.y);
            }
        }
    }
    return edges;
}

struct DSU {
    std::vector<int> parent, rank;

    explicit DSU(int n) : parent(n), rank(n, 0) { std::iota(parent.begin(), parent.end(), 0); }
    int find(int u) { return parent[u] == u ? u : parent[u] = find(parent[u]); }

    bool unite(int u, int v) {
        u = find(u);
        v = find(v);
        if (u == v) {
            return false;
        }
        if (rank[u] < rank[v]) {
            std::swap(u, v);
        }
        parent[v] = u;
        rank[u] += rank[v] == rank[v];
        return true;
    }
};

int64_t manhattan_mst_weight(const std::vector<Point> &p) {
    auto edges = manhattan_mst_edges(p);
    std::sort(edges.begin(), edges.end());
    DSU dsu(static_cast<int>(p.size()));
    int64_t res = 0;
    for (auto [w, u, v] : edges) {
        if (dsu.unite(static_cast<int>(u), static_cast<int>(v))) {
            res += w;
        }
    }
    return res;
}

```

```
}
```

Example Usage

```
using namespace std;

int main() {
    vector<Point> p{{0, 0}, {2, 1}, {3, 4}, {-1, 2}, {5, 0}};
    assert(manhattan_mst_weight(p) == 14);
    auto edges = manhattan_mst_edges(p);
    assert(edges.size() <= 4 * p.size());
    return 0;
}
```

7.5 3D Geometry

7.5.1 3D Point

A 3D point class template supporting vector arithmetic, dot and cross products, distances, angles, unit vectors, normals, and rotation about an axis. Integer coordinates are suitable for exact algebraic predicates (`dot`, `cross`, `sqrnorm`) as long as intermediate products do not overflow; metric operations return floating-point values.

Type aliases:

- `Point3I = TPoint3<int>`
- `Point3L = TPoint3<int64_t>`
- `Point3D = TPoint3<double>`
- `Point3LD = TPoint3<long double>`

Time Complexity: $O(1)$ per operation.

Space Complexity: $O(1)$ for storage and auxiliary space.

```
#include <cassert>
#include <cmath>
#include <cstdint>
#include <tuple>
#include <type_traits>

const double EPS = 1e-9;

template<typename T, typename U, typename C = std::common_type_t<T, U>>
bool EQ(T a, U b) {
    return std::is_integral_v<C> ? C(a) == C(b) : std::fabs(C(a) - C(b)) <= static_cast<C>(EPS);
}

template<typename T>
struct TPoint3 {
    using fp_t = std::conditional_t<std::is_integral<T>::value, double, T>;

    T x, y, z;

    TPoint3(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
    bool operator==(const TPoint3 &p) const { return std::tie(x, y, z) == std::tie(p.x, p.y, p.z); }
    bool operator!=(const TPoint3 &p) const { return !(*this == p); }
    bool operator<(const TPoint3 &p) const { return std::tie(x, y, z) < std::tie(p.x, p.y, p.z); }
    TPoint3 operator+(const TPoint3 &p) const { return {x + p.x, y + p.y, z + p.z}; }
    TPoint3 operator-(const TPoint3 &p) const { return {x - p.x, y - p.y, z - p.z}; }
    TPoint3 operator*(T k) const { return {x * k, y * k, z * k}; }
    TPoint3<fp_t> operator/(fp_t k) const { return {(fp_t)x / k, (fp_t)y / k, (fp_t)z / k}; }
```

```

T dot(const TPoint3 &p) const { return x * p.x + y * p.y + z * p.z; }
T sqnorm() const { return x * x + y * y + z * z; }
fp_t norm() const { return sqrt(static_cast<fp_t>(sqnorm())); }
fp_t phi() const { return atan2(static_cast<fp_t>(y), static_cast<fp_t>(x)); }
TPoint3<fp_t> unit() const { return *this / norm(); }

TPoint3 cross(const TPoint3 &p) const {
    return {y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x};
}

TPoint3<fp_t> normal(const TPoint3 &p) const { return cross(p).unit(); }

fp_t theta() const {
    return atan2(hypot(static_cast<fp_t>(x), static_cast<fp_t>(y)), static_cast<fp_t>(z));
}

TPoint3<fp_t> rotate(fp_t angle, const TPoint3 &axis) const {
    fp_t s = sin(angle), c = cos(angle);
    TPoint3<fp_t> u = axis.unit(), v((fp_t)x, (fp_t)y, (fp_t)z);
    return u * v.dot(u) * (1 - c) + v * c - v.cross(u) * s;
}
};

using Point3I = TPoint3<int>;
using Point3L = TPoint3<int64_t>;
using Point3D = TPoint3<double>;
using Point3LD = TPoint3<long double>;

```

Example Usage

```

int main() {
    Point3I a(1, 0, 0), b(0, 1, 0);
    assert(a.dot(b) == 0);
    assert(a.cross(b) == Point3I(0, 0, 1));
    assert(a.sqnorm() == 1);

    Point3D p(1, 0, 0);
    auto r = p.rotate(acos(-1.0) / 2, Point3D(0, 0, 1));
    assert(EQ(r.x, 0) && EQ(r.y, 1) && EQ(r.z, 0));
    return 0;
}

```

7.5.2 3D Convex Hull

Given points in 3D, compute the triangular faces of their convex hull with an incremental algorithm. Start with a tetrahedron, delete every visible face for each new point, and stitch the horizon edges to that point. Returned faces are oriented outward.

- `convex_hull_3d(points)` returns a list of `Face{a, b, c}` index triples into `points`.

Preconditions: at least four points, and no four points coplanar. For coplanar or highly degenerate inputs, perturbing points slightly or using a more robust library is recommended.

Time Complexity: $O(n^2)$, where n is the number of points.

Space Complexity: $O(n^2)$ auxiliary.

```

#include <algorithm>
#include <array>
#include <cassert>
#include <cmath>
#include <vector>

```

```

const double EPS = 1e-9;

struct Point3D {
    double x, y, z;

    Point3D(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
    Point3D operator-(const Point3D &p) const { return {x - p.x, y - p.y, z - p.z}; }
    Point3D operator*(double k) const { return {x * k, y * k, z * k}; }
    double dot(const Point3D &p) const { return x * p.x + y * p.y + z * p.z; }
    Point3D cross(const Point3D &p) const {
        return {y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x};
    }
};

struct Face {
    int a, b, c;
    Face(int a = 0, int b = 0, int c = 0) : a(a), b(b), c(c) {}
};

std::vector<Face> convex_hull_3d(const std::vector<Point3D> &p) {
    int n = static_cast<int>(p.size());
    assert(n >= 4);
    std::vector<std::vector<std::array<int, 2>>> edge(
        n, std::vector<std::array<int, 2>>(n, {-1, -1})
    );
    std::vector<Face> faces;

    auto normal = [&](const Face &f) { return (p[f.b] - p[f.a]).cross(p[f.c] - p[f.a]); };
    auto visible = [&](const Face &f, int i) { return normal(f).dot(p[i] - p[f.a]) > EPS; };
    auto add_edge = [&](int a, int b, int c) {
        if (a > b) {
            std::swap(a, b);
        }
        auto &e = edge[a][b];
        (e[0] == -1 ? e[0] : e[1]) = c;
    };
    auto remove_edge = [&](int a, int b, int c) {
        if (a > b) {
            std::swap(a, b);
        }
        auto &e = edge[a][b];
        (e[0] == c ? e[0] : e[1]) = -1;
    };
    auto edge_count = [&](int a, int b) {
        if (a > b) {
            std::swap(a, b);
        }
        return (edge[a][b][0] != -1) + (edge[a][b][1] != -1);
    };
    auto make_face = [&](int a, int b, int c, int inside) {
        Face f(a, b, c);
        if (normal(f).dot(p[inside] - p[a]) > 0) {
            std::swap(f.b, f.c);
        }
        add_edge(f.a, f.b, f.c);
        add_edge(f.b, f.c, f.a);
        add_edge(f.c, f.a, f.b);
        faces.push_back(f);
    };

    for (int i = 0; i < 4; i++) {
        for (int j = i + 1; j < 4; j++) {
            for (int k = j + 1; k < 4; k++) {
                make_face(i, j, k, 6 - i - j - k);
            }
        }
    }
}

```

```

for (int i = 4; i < n; i++) {
    for (int j = 0; j < static_cast<int>(faces.size()); j++) {
        Face f = faces[j];
        if (visible(f, i)) {
            remove_edge(f.a, f.b, f.c);
            remove_edge(f.b, f.c, f.a);
            remove_edge(f.c, f.a, f.b);
            std::swap(faces[j--], faces.back());
            faces.pop_back();
        }
    }
    int old_faces = static_cast<int>(faces.size());
    for (int j = 0; j < old_faces; j++) {
        Face f = faces[j];
        if (edge_count(f.a, f.b) != 2) {
            make_face(f.a, f.b, i, f.c);
        }
        if (edge_count(f.b, f.c) != 2) {
            make_face(f.b, f.c, i, f.a);
        }
        if (edge_count(f.c, f.a) != 2) {
            make_face(f.c, f.a, i, f.b);
        }
    }
}
return faces;
}

```

Example Usage

```

using namespace std;

int main() {
    vector<Point3D> tetra{{0, 0, 0}, {1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    auto faces = convex_hull_3d(tetra);
    assert(faces.size() == 4);

    tetra.push_back(Point3D(1, 1, 1));
    faces = convex_hull_3d(tetra);
    assert(faces.size() == 6);
    return 0;
}

```

7.5.3 Polyhedron Volume

Given an oriented triangular mesh of a closed polyhedron, compute its signed volume. Each triangle contributes the signed volume of the tetrahedron formed by the triangle abc and the origin $((a \times b) \cdot c)/6$. If faces are oriented outward, the result is positive for the orientation used by `convex_hull_3d()` in 7.5.2; take `abs()` if only the magnitude is needed.

- `signed_polyhedron_volume(points, faces)` returns the signed volume. Each face is an index triple.

Time Complexity: $O(f)$, where f is the number of triangular faces.

Space Complexity: $O(1)$ auxiliary.

```

#include <cmath>
#include <tuple>
#include <type_traits>
#include <vector>

struct Point3D {

```

```

double x, y, z;

Point3D(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
double dot(const Point3D &p) const { return x * p.x + y * p.y + z * p.z; }
Point3D cross(const Point3D &p) const {
    return {y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x};
}
};

template<typename Pt, typename F>
double signed_polyhedron_volume(const std::vector<Pt> &p, const std::vector<F> &faces) {
    double volume = 0;
    for (const auto &[a, b, c] : faces) {
        volume += p[a].cross(p[b]).dot(p[c]);
    }
    return volume / 6.0;
}

```

Example Usage

```

#include <cassert>
using namespace std;

const double EPS = 1e-9;

bool EQ(double a, double b) {
    return fabs(a - b) < 1e-9;
}

int main() {
    using Face = std::tuple<int, int, int>; // (a, b, c)
    vector<Point3D> p{{0, 0, 0}, {1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    vector<Face> faces{
        Face(0, 2, 1),
        Face(0, 1, 3),
        Face(0, 3, 2),
        Face(1, 2, 3),
    };
    assert(EQ(signed_polyhedron_volume(p, faces), 1.0 / 6.0));
    return 0;
}

```


Chapter 8

Miscellany

8.1 Contest Utilities

Small contest-template helpers that are useful across many algorithms. These snippets avoid algorithm-specific policy and are meant to be pasted near the top of a solution file.

- `y_combinator(f)` wraps a recursive lambda so that the lambda can call itself as its first argument.
- `Rep(i,N)`, `For(i,L,H)`, `Rev(i,N)`, `Dwn(i,H,L)`, and `Each(x,C)` are compact loop macros.
- `sz(c)` returns the signed size of a container.
- `ckmin(a, b)` assigns `a = b` and returns true if `b < a` before the assignment.
- `ckmax(a, b)` assigns `a = b` and returns true if `a < b` before the assignment.
- `between(x, a, b)` returns whether $a \leq x \leq b$.
- `clmp(x, a, b)` returns `x` clamped into the interval $[a, b]$. This is not needed on C++17 and later, where `std::clamp()` is available.
- `ceil_div(a, b)` and `floor_div(a, b)` divide signed integers with mathematical rounding toward positive or negative infinity. Requires `b != 0`.
- `sort_unique(v)` sorts a vector and removes duplicates.
- `erase_one(c, x)` erases one existing value from an associative container, asserting that it is present.
- `min_priority_queue<T>` is a min-heap alias.
- `RNG()` constructs a 64-bit Mersenne Twister seeded from the steady clock.
- `RNG(seed)` constructs a reproducible 64-bit Mersenne Twister.
- `rng.uniform_int(lo, hi)` returns a random integer in the inclusive range $[lo, hi]$.
- `rng.uniform_real(lo, hi)` returns a random real number in the half-open range $[lo, hi)$.
- `rng.shuffle(lo, hi)` randomly shuffles the range $[lo, hi)$.

```
#include <algorithm>
#include <cassert>
#include <chrono>
#include <cstdint>
#include <queue>
#include <random>
#include <type_traits>
#include <utility>
#include <vector>

#define Rep(i, N) for (int i = 0, _##i = (N); i < _##i; ++i) // 0 to N-1
#define For(i, L, H) for (int i = (L), _##i = (H); i <= _##i; ++i) // L to H
#define Rev(i, N) for (int i = (N); --i >= 0;) // N-1 to 0
#define Dwn(i, H, L) for (int i = (H), _##i = (L); i >= _##i; --i) // H to L
#define Each(x, C) for (auto &x : (C))

template<typename Fun>
```

```

class y_combinator_result {
    Fun fun;

public:
    template<typename T>
    explicit y_combinator_result(T &&fun_) : fun(std::forward<T>(fun_)) {}
    template<class... Args>
    decltype(auto) operator()(Args &&...args) {
        return fun(std::ref(*this), std::forward<Args>(args)...);
    }
};

template<typename Fun>
decltype(auto) y_combinator(Fun &&fun) {
    return y_combinator_result<std::decay_t<Fun>>(std::forward<Fun>(fun));
}

template<typename C> int sz(const C &c) { return static_cast<int>(c.size()); }
template<typename T, typename U> bool ckmin(T &a, const U &b) { return b < a ? a = b, true : false; }
template<typename T, typename U> bool ckmax(T &a, const U &b) { return a < b ? a = b, true : false; }
template<typename T> bool between(const T &x, const T &a, const T &b) { return !(x < a) && !(b < x); }
template<typename T> T clmp(const T &x, const T &a, const T &b) { return std::min(std::max(x, a), b); }

template<typename T>
T floor_div(T a, T b) {
    assert(b != 0);
    T q = a / b, r = a % b;
    return q - (r != 0 && ((r < 0) != (b < 0)));
}

template<typename T>
T ceil_div(T a, T b) {
    assert(b != 0);
    T q = a / b, r = a % b;
    return q + (r != 0 && ((r < 0) == (b < 0)));
}

template<typename T>
void sort_unique(std::vector<T> &v) {
    std::sort(v.begin(), v.end());
    v.erase(std::unique(v.begin(), v.end()), v.end());
}

template<typename C, typename T>
void erase_one(C &c, const T &x) {
    auto it = c.find(x);
    assert(it != c.end());
    c.erase(it);
}

template<typename T>
using min_priority_queue = std::priority_queue<T, std::vector<T>, std::greater<T>>;

struct RNG {
    std::mt19937_64 gen;

    RNG() : RNG(std::chrono::steady_clock::now().time_since_epoch().count()) {}
    explicit RNG(uint64_t seed) : gen(seed) {}

    template<typename T>
    T uniform_int(T lo, T hi) {
        static_assert(std::is_integral<T>::value, "uniform_int() requires an integral type");
        return std::uniform_int_distribution<T>(lo, hi)(gen);
    }

    double uniform_real(double lo = 0.0, double hi = 1.0) {
        return std::uniform_real_distribution<double>(lo, hi)(gen);
    }
}

```

```

template<typename It>
void shuffle(It lo, It hi) {
    std::shuffle(lo, hi, gen);
}
};

```

Example Usage

```

#include <set>
using namespace std;

int main() {
    int tot = 0;
    Rep(i, 4) tot += i;    // 0 + 1 + 2 + 3 = 6
    For(i, 2, 5) tot += i; // 6 + (2 + 3 + 4 + 5) = 20
    Rev(i, 3) tot += i;    // 20 + (2 + 1 + 0) = 23
    Dwn(i, 8, 6) tot += i; // 23 + (8 + 7 + 6) = 44
    assert(tot == 44);

    // Recursive lambda with y_combinator.
    auto fib =
        y_combinator([](auto fib, int n) -> int { return n < 2 ? n : fib(n - 1) + fib(n - 2); });
    assert(fib(10) == 55);

    // Recursive lambda without y_combinator: pass the lambda to itself as the first argument.
    vector<vector<int>> adj{{1, 2}, {0}, {0}};
    int seen = 0;
    auto dfs = [&](auto &&dfs, int u, int p) -> void {
        seen++;
        Each(v, adj[u]) if (v != p) dfs(dfs, v, u);
    };
    dfs(dfs, 0, -1);
    assert(seen == 3);

    // Recursive lambdas are automatically supported in C++23.
    #if __cplusplus >= 202302L
        auto fact = [&](this auto fact, int n) -> int { return n ? n * fact(n - 1) : 1; };
        assert(fact(5) == 120);
    #endif

    int x = 10;
    assert(ckmin(x, 7) && x == 7);
    assert(!ckmin(x, 9) && x == 7);
    assert(ckmax(x, 11) && x == 11);

    vector<int> v = {1, 2, 3};
    assert(sz(v) == 3);
    assert(between(2, 1, 3));
    assert(clmp(10, 1, 3) == 3);
    assert(floor_div(-7, 3) == -3);
    assert(ceil_div(-7, 3) == -2);
    v = {3, 1, 3, 2, 1};
    sort_unique(v);
    assert((v == vector<int>{1, 2, 3}));
    multiset<int> ms{1, 2, 2, 3};
    erase_one(ms, 2);
    assert(ms.count(2) == 1);
    min_priority_queue<int> pq;
    pq.push(3);
    pq.push(1);
    assert(pq.top() == 1);

    RNG rng(123);
    int r = rng.uniform_int(1, 6);
    assert(1 <= r && r <= 6);

```

```

    rng.shuffle(v.begin(), v.end());
    assert(sz(v) == 3);
    return 0;
}

```

8.2 Fast IO

Fast input and output wrappers based on `fread()` and `fwrite()`. For most problems, `ios::sync_with_stdio(false); cin.tie(nullptr);` is simpler and fast enough. Use these classes when input is huge or when iostream overhead is measurable.

- `FastInput in(file)` reads tokens from a `FILE*`, defaulting to `stdin`.
- `in >> x` reads a non-whitespace token into `char`, `std::string`, integral types, or floating point types.
- `FastOutput out(file)` writes to a `FILE*`, defaulting to `stdout`.
- `out << x` writes `char`, C strings, `std::string`, integral types, or floating point types.
- `out.flush()` writes any buffered output immediately.
- `set_in(name)`, `set_out(name)`, and `set_io(iname, oname)` redirect standard input/output to files. For example, `set_io("task.in", "task.out")` is convenient for USACO-style problems.

The parser assumes valid input. Floating point input/output is provided for convenience, not as the main performance path.

```

#include <cassert>
#include <cctype>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <string>
#include <type_traits>

void set_in(const std::string &name) {
    assert(freopen(name.c_str(), "r", stdin) != nullptr);
}

void set_out(const std::string &name) {
    assert(freopen(name.c_str(), "w", stdout) != nullptr);
}

void set_io(const std::string &iname, const std::string &oname) {
    set_in(iname);
    set_out(oname);
}

struct FastInput {
    static constexpr int BUF_SIZE = 1 << 20;
    FILE *file;
    char buf[BUF_SIZE];
    int pos = 0, len = 0;

    explicit FastInput(FILE *file_ = stdin) : file(file_) {}

    char get_char() {
        if (pos == len) {
            len = fread(buf, 1, BUF_SIZE, file);
            pos = 0;
            if (len == 0) {
                return 0;
            }
        }
        return buf[pos++];
    }
};

```

```

char peek_char() {
    char c = get_char();
    if (c) {
        --pos;
    }
    return c;
}

void skip_blanks() {
    while (true) {
        char c = get_char();
        if (!c) {
            return;
        }
        if (!std::isspace(static_cast<unsigned char>(c))) {
            --pos;
            return;
        }
    }
}

FastInput &operator>>(char &c) {
    skip_blanks();
    c = get_char();
    return *this;
}

FastInput &operator>>(std::string &s) {
    skip_blanks();
    s.clear();
    while (true) {
        char c = get_char();
        if (!c || std::isspace(static_cast<unsigned char>(c))) {
            break;
        }
        s += c;
    }
    return *this;
}

FastInput &operator>>(bool &x) {
    int val;
    *this >> val;
    x = val != 0;
    return *this;
}

template<typename T>
typename std::enable_if<
    std::is_integral<T>::value && !std::is_same<T, bool>::value, FastInput &>::type
operator>>(T &x) {
    skip_blanks();
    char c = get_char();
    bool neg = false;
    if (c == '-') {
        neg = true;
        c = get_char();
    }
    using U = typename std::make_unsigned<T>::type;
    U val = 0;
    while (c && !std::isspace(static_cast<unsigned char>(c))) {
        val = U(10) * val + U(c - '0');
        c = get_char();
    }
    if constexpr (std::is_signed<T>::value) {
        x = neg ? T(U(0) - val) : T(val);
    } else {

```

```

    x = T(val);
}
return *this;
}

template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, FastInput &>::type operator>>(T &x) {
    std::string s;
    *this >> s;
    x = static_cast<T>(std::strtod(s.c_str(), nullptr));
    return *this;
}
};

struct FastOutput {
    static constexpr int BUF_SIZE = 1 << 20;
    FILE *file;
    char buf[BUF_SIZE];
    int pos = 0;

    explicit FastOutput(FILE *file_ = stdout) : file(file_) {}
    ~FastOutput() { flush(); }

    void flush() {
        if (pos) {
            fwrite(buf, 1, pos, file);
            pos = 0;
        }
    }

    void put_char(char c) {
        if (pos == BUF_SIZE) {
            flush();
        }
        buf[pos++] = c;
    }

    FastOutput &operator<<(char c) {
        put_char(c);
        return *this;
    }

    FastOutput &operator<<(const char *s) {
        while (*s) {
            put_char(*s++);
        }
        return *this;
    }

    FastOutput &operator<<(const std::string &s) {
        for (char c : s) {
            put_char(c);
        }
        return *this;
    }

    FastOutput &operator<<(bool x) {
        put_char(x ? '1' : '0');
        return *this;
    }

    template<typename T>
    typename std::enable_if<
        std::is_integral<T>::value && !std::is_same<T, bool>::value, FastOutput &>::type
        operator<<(T x) {
        if (x == 0) {
            put_char('0');
            return *this;
        }
    }

```

```

    }
    using U = typename std::make_unsigned<T>::type;
    U val;
    if constexpr (std::is_signed<T>::value) {
        if (x < 0) {
            put_char('-');
            val = U(0) - U(x);
        } else {
            val = U(x);
        }
    } else {
        val = x;
    }
    char s[32];
    int n = 0;
    while (val > 0) {
        s[n++] = char('0' + val % 10);
        val /= 10;
    }
    while (n-- > 0) {
        put_char(s[n]);
    }
    return *this;
}

template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, FastOutput &>::type operator<<(T x) {
    char s[64];
    std::snprintf(s, sizeof(s), "%.17g", static_cast<double>(x));
    return *this << s;
}
};

```

Example Usage

```

int main() {
    // set_io("file.in", "file.out");

    FILE *input = tmpfile();
    std::fputs("42 hello 3.5 -2147483648\n", input);
    std::rewind(input);

    FastInput in(input);
    int x;
    std::string s;
    double y;
    int z;
    in >> x >> s >> y >> z;
    assert(x == 42 && s == "hello" && y == 3.5);
    assert(z == -2147483648);
    std::fclose(input);

    FILE *output = tmpfile();
    {
        FastOutput out(output);
        out << x << ' ' << s << ' ' << y << ' ' << z << '\n';
        out.flush();
    }
    std::rewind(output);
    char buf[64] = {};
    std::fgets(buf, sizeof(buf), output);
    assert(std::strncmp(buf, "42 hello 3.5 -2147483648", 24) == 0);
    std::fclose(output);
    return 0;
}

```

8.3 Debugging

Local-only debug printing for contest code. The macro expands to a no-op unless `LOCAL` is defined (e.g. by passing `-DLOCAL` to the compiler flags), so it can remain in submitted code without producing output.

- `dbg_repr(x)` returns a string representation of arithmetic types, strings, pairs, tuples, and iterable containers.
- `dbg(a, b, c)` prints the function name, line number, argument names, and values to `std::cerr` when `LOCAL` is defined.
- `pr(a, b, c)` prints only the values to `std::cerr` when `LOCAL` is defined.
- `dbg(...)` evaluates to no output when `LOCAL` is not defined.
- `pr(...)` evaluates to no output when `LOCAL` is not defined.

Avoid naming these helpers `to_string()` unless you want them mixed into overload resolution with `std::to_string()` and other user-defined `to_string()` functions. A separate name keeps debug formatting local to this snippet.

```
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <tuple>
#include <type_traits>
#include <utility>

template<typename T, typename = void>
struct is_iterable : std::false_type {};

template<typename T>
struct is_iterable<
    T, std::void_t<decltype(std::begin(std::declval<T>())), decltype(std::end(std::declval<T>()))>>
    : std::true_type {};

std::string dbg_repr(const std::string &s) {
    return '"' + s + '"';
}

std::string dbg_repr(const char *s) {
    return dbg_repr(std::string(s));
}

std::string dbg_repr(char c) {
    return std::string('"') + c + '"';
}

std::string dbg_repr(bool b) {
    return b ? "true" : "false";
}

template<typename T>
typename std::enable_if<
    std::is_arithmetic<T>::value && !std::is_same<T, char>::value &&
    !std::is_same<T, bool>::value && !std::is_floating_point<T>::value,
    std::string>::type
dbg_repr(T x) {
    std::ostringstream out;
    out << x;
    return out.str();
}

template<typename T>
typename std::enable_if<std::is_floating_point<T>::value, std::string>::type dbg_repr(T x) {
    std::ostringstream out;
    out << std::setprecision(10) << x; // Set floating point dbg precision here.
    return out.str();
}
```



```

}

template<typename A, typename B>
std::string dbg_repr(const std::pair<A, B> &p) {
    return "(" + dbg_repr(p.first) + ", " + dbg_repr(p.second) + ")";
}

template<typename Tuple, std::size_t... Is>
std::string dbg_tuple_repr(const Tuple &t, std::index_sequence<Is...>) {
    std::string res = "(";
    bool first = true;
    ((res += (first ? (first = false, "") : ", ") + dbg_repr(std::get<Is>(t))), ...);
    return res + ")";
}

template<class... Ts>
std::string dbg_repr(const std::tuple<Ts...> &t) {
    return dbg_tuple_repr(t, std::index_sequence_for<Ts...>{});
}

template<typename T>
typename std::enable_if<
    is_iterable<T>::value && !std::is_convertible<T, std::string>::value, std::string>::type
> dbg_repr(const T &v) {
    std::string res = "{";
    bool first = true;
    for (const auto &x : v) {
        if (!first) {
            res += ", ";
        }
        first = false;
        res += dbg_repr(x);
    }
    return res + "}";
}

void dbg_out(bool) {
    std::cerr << '\n';
}

template<typename Head, class... Tail>
void dbg_out(bool leading_space, const Head &head, const Tail &...tail) {
    if (leading_space) {
        std::cerr << ' ';
    }
    std::cerr << dbg_repr(head);
    ((std::cerr << ' ' << dbg_repr(tail)), ...);
    std::cerr << '\n';
}

#ifdef LOCAL
#define dbg(...) \
    std::cerr << "[" << __func__ << ":" << __LINE__ << "]" " " << #__VA_ARGS__ << ":", \
    dbg_out(true, __VA_ARGS__)
#define pr(...) dbg_out(false, __VA_ARGS__)
#else
#define dbg(...) ((void)0)
#define pr(...) ((void)0)
#endif

```

Example Usage

```

#include <cassert>
#include <map>
#include <set>
#include <vector>

```

```
using namespace std;

struct Point {
    int x, y;
};

// Can overload dbg_repr to dbg your custom structs.
string dbg_repr(const Point &p) {
    return "Point(" + dbg_repr(p.x) + ", " + dbg_repr(p.y) + ")";
}

struct Edge {
    int u, v, w;
};

string dbg_repr(const Edge &e) {
    return "Edge{" + dbg_repr(e.u) + "->" + dbg_repr(e.v) + ", w=" + dbg_repr(e.w) + "}";
}

int main() {
    vector<int> v = {1, 2, 3};
    map<string, int> mp = {{"a", 1}, {"b", 2}};
    auto p = make_pair(4, string("x"));
    auto t = make_tuple(1, true, string("ok"));
    Point pt{2, 3};
    Edge e{0, 1, 5};
    assert(dbg_repr(v) == "{1, 2, 3}");
    assert(dbg_repr(p) == "(4, \"x\")");
    assert(dbg_repr(t) == "(1, true, \"ok\")");
    assert(dbg_repr(mp) == "{(\"a\", 1), (\"b\", 2)}");
    assert(dbg_repr(pt) == "Point(2, 3)");
    assert(dbg_repr(e) == "Edge{0->1, w=5}");
    double d = 3.141592653589793;
    dbg(v, mp);
    dbg(p, t);
    dbg(pt, e);
    pr("plain output", d);
    assert(dbg_repr(1.0 / 3.0) == "0.3333333333");
    assert(dbg_repr(d) == "3.141592654");
    return 0;
}
```

Example Output

```
[main:173] v, mp: {1, 2, 3} {("a", 1), ("b", 2)}
[main:174] p, t: (4, "x") (1, true, "ok")
[main:175] pt, e: Point(2, 3) Edge{0->1, w=5}
"plain output" 3.141592654
```

8.4 Timing and Benchmarking

Small timing helpers for local profiling and codebook examples. Use `std::chrono::steady_clock` instead of wall-clock time so elapsed durations are monotonic.

- `Timer()` starts a timer immediately.
- `timer.reset()` restarts the timer.
- `timer.elapsed()` returns the elapsed time in milliseconds as a double.
- `benchmark(name, iterations, f)` runs `f()` `iterations` times and returns the average elapsed milliseconds per call with the given name.

Benchmarks should be treated as local measurements, not proofs of asymptotic performance. Warm-up, CPU frequency scaling, compiler optimization, and input choice can dominate tiny timings.

```

#include <chrono>
#include <string>
#include <utility>
#include <vector>

class Timer {
    using Clock = std::chrono::steady_clock;
    Clock::time_point start_time;

public:
    Timer() { reset(); }

    void reset() { start_time = Clock::now(); }

    double elapsed() const {
        std::chrono::duration<double, std::milli> diff = Clock::now() - start_time;
        return diff.count();
    }
};

template<typename Fun>
std::pair<std::string, double> benchmark(const std::string &name, int iterations, Fun f) {
    Timer timer;
    for (int i = 0; i < iterations; ++i) {
        f();
    }
    return {name, timer.elapsed() / iterations};
}

```

Example Usage

```

#include <cassert>
#include <numeric>
#include <vector>
using namespace std;

int main() {
    Timer timer;
    std::vector<int> v(1000);
    std::iota(v.begin(), v.end(), 0);
    assert(timer.elapsed() >= 0.0);

    volatile long long sink = 0;
    auto res = benchmark("sum", 5, [&] {
        long long cur = 0;
        for (int x : v) {
            cur += x;
        }
        sink += cur;
    });
    assert(res.first == "sum");
    assert(res.second >= 0.0);
    assert(sink > 0);
    return 0;
}

```

8.5 Stress Testing

A simple stress-testing harness compares a trusted brute force implementation against an optimized implementation on many generated tests. This is most useful for finding small counterexamples while developing an algorithm.

- `stress(trials, generate, solve, brute)` runs `generate()` for each trial, compares `solve(test)` with `brute(test)`, and returns the first failing test if one is found.
- `stress(...)` returns `std::nullopt` if all generated tests pass.

The generated test type only needs to be copyable enough to store in `std::optional`, and both solutions must return comparable results.

```
#include <optional>

template<typename Generate, typename Solve, typename Brute>
auto stress(int trials, Generate generate, Solve solve, Brute brute)
-> std::optional<decltype(generate())> {
    for (int tc = 0; tc < trials; ++tc) {
        auto test = generate();
        if (solve(test) != brute(test)) {
            return test;
        }
    }
    return std::nullopt;
}
```

Example Usage

```
#include <algorithm>
#include <cassert>
#include <random>
#include <vector>
using namespace std;

int main() {
    std::mt19937 rng(123);

    auto generate = [&] {
        int n = std::uniform_int_distribution<int>(1, 8)(rng);
        std::vector<int> a(n);
        for (int &x : a) {
            x = std::uniform_int_distribution<int>(-5, 5)(rng);
        }
        return a;
    };

    auto solve = [](std::vector<int> a) {
        std::sort(a.begin(), a.end());
        return a;
    };

    auto brute = [](std::vector<int> a) {
        do {
            if (std::is_sorted(a.begin(), a.end())) {
                return a;
            }
        } while (std::next_permutation(a.begin(), a.end()));
        std::sort(a.begin(), a.end());
        return a;
    };

    assert(!stress(100, generate, solve, brute));
    return 0;
}
```

8.6 Policy-Based Data Structures

GNU policy-based data structures provide ordered sets and maps with order statistics. They are not part of the C++ standard library, but they are available on many GNU C++ contest judges.

- `OrderedSet<T>` behaves like a set with `find_by_order(k)` and `order_of_key(x)`.
- `OrderedMap<K, V>` behaves like a map with `find_by_order(k)` and `order_of_key(x)`.
- `HashMap<K, V>` and `HashSet<K>` are fast hash tables using `gp_hash_table`. The default `SplitMix64Hasher` is for integer-like keys; pass a custom hash for other key types. It randomizes the hash seed to avoid weak adversarial integer hashes.
- `OrderedMultiset<T>` stores duplicates by pairing each value with a unique ID.
- `OrderedMultimap<K, V>` stores duplicate keys by pairing each key with a unique ID.
- `PairingHeap<T>` is a meldable priority queue; `push()` returns an iterator that can be passed to `modify()` or `erase()`.
- `BinaryHeap<T>`, `BinomialHeap<T>`, and `RcBinomialHeap<T>` are alternative GNU priority queue tags with the same interface.

The order statistic operations are 0-indexed. The priority queue `join()` operation melds another heap into the current one. Since this depends on `__gnu_pbds`, keep a standard library fallback in mind when portability matters.

Time Complexity: $O(\log n)$ per operation.

Space Complexity: $O(n)$.

```
#include <chrono>
#include <cstdint>
#include <cstdint>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <functional>
#include <utility>

template<typename K, typename V, typename Compare = std::less<K>>
using OrderedMap = __gnu_pbds::tree<
    K, V, Compare, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>;

template<typename T, typename Compare = std::less<T>>
using OrderedSet = OrderedMap<T, __gnu_pbds::null_type, Compare>;

struct SplitMix64Hasher {
    // SplitMix64 mixer.
    static uint64_t mix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    std::size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            std::chrono::steady_clock::now().time_since_epoch().count();
        return mix64(x + FIXED_RANDOM);
    }
};

template<typename K, typename V, typename Hash = SplitMix64Hasher>
using HashMap = __gnu_pbds::gp_hash_table<K, V, Hash>;

template<typename K, typename Hash = SplitMix64Hasher>
using HashSet = __gnu_pbds::gp_hash_table<K, __gnu_pbds::null_type, Hash>;

template<typename T>
class OrderedMultiset {
```

```

OrderedSet<std::pair<T, int>> tree;
int timer = 0;

public:
void insert(const T &x) { tree.insert({x, timer++}); }

bool erase_one(const T &x) {
    auto it = tree.lower_bound({x, -1});
    if (it == tree.end() || it->first != x) {
        return false;
    }
    tree.erase(it);
    return true;
}

int order_of_key(const T &x) const { return tree.order_of_key({x, -1}); }
T find_by_order(int k) const { return tree.find_by_order(k)->first; }
int size() const { return static_cast<int>(tree.size()); }
};

template<typename K, typename V>
class OrderedMultimap {
    OrderedMap<std::pair<K, int>, V> tree;
    int timer = 0;

public:
void insert(const K &key, const V &value) { tree.insert({{key, timer++}, value}); }

bool erase_one(const K &key) {
    auto it = tree.lower_bound({key, -1});
    if (it == tree.end() || it->first.first != key) {
        return false;
    }
    tree.erase(it);
    return true;
}

int order_of_key(const K &key) const { return tree.order_of_key({key, -1}); }

std::pair<K, V> find_by_order(int k) const {
    auto it = tree.find_by_order(k);
    return {it->first.first, it->second};
}

int size() const { return static_cast<int>(tree.size()); }
};

template<typename T, typename Compare = std::less<T>>
using PairingHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::pairing_heap_tag>;

template<typename T, typename Compare = std::less<T>>
using BinaryHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::binary_heap_tag>;

template<typename T, typename Compare = std::less<T>>
using BinomialHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::binomial_heap_tag>;

template<typename T, typename Compare = std::less<T>>
using RcBinomialHeap = __gnu_pbds::priority_queue<T, Compare, __gnu_pbds::rc_binomial_heap_tag>;

```

Example Usage

```

#include <cassert>

int main() {
    OrderedSet<int> s;
    s.insert(10);
}

```

```

s.insert(20);
s.insert(30);
assert(*s.find_by_order(1) == 20);
assert(s.order_of_key(25) == 2);

HashMap<int, int> hm;
hm[10] = 1;
hm[20] = 2;
assert(hm[10] == 1);
HashSet<int> hs;
hs.insert(7);
assert(hs.find(7) != hs.end());

OrderedMultiset<int> ms;
ms.insert(5);
ms.insert(5);
ms.insert(7);
assert(ms.size() == 3);
assert(ms.order_of_key(7) == 2);
assert(ms.find_by_order(1) == 5);
assert(ms.erase_one(5));
assert(ms.order_of_key(7) == 1);

OrderedMultimap<int, char> mp;
mp.insert(5, 'a');
mp.insert(5, 'b');
mp.insert(7, 'c');
assert(mp.order_of_key(7) == 2);
assert(mp.find_by_order(1).first == 5);
assert(mp.erase_one(5));
assert(mp.order_of_key(7) == 1);

PairingHeap<int> h1, h2;
auto it = h1.push(10);
h1.push(5);
h1.modify(it, 20);
h2.push(7);
h1.join(h2);
assert(h1.top() == 20);
h1.pop();
assert(h1.top() == 7);
return 0;
}

```

8.7 Multithreading

Basic multithreading can speed up independent local work such as stress tests, batch experiments, or offline preprocessing. It is rarely useful in standard contest submissions, and many judges restrict threads, so use it only when the environment allows it.

- `parallel_for(n, threads, f)` runs `f(i)` once for each $0 \leq i < n$, splitting the indices among at most `threads` worker threads.
- `parallel_cases(cases, threads, solve)` solves independent test cases in parallel and returns the outputs in the original case order.

Each call to `f(i)` must be safe to run concurrently with the others. If it writes shared state, use separate output slots or synchronization.

```

#include <algorithm>
#include <cassert>
#include <numeric>
#include <sstream>

```

```

#include <string>
#include <thread>
#include <vector>

template<typename Fun>
void parallel_for(int n, int threads, Fun f) {
    threads = std::max(1, std::min(threads, n));
    std::vector<std::thread> workers;
    workers.reserve(threads);
    for (int id = 0; id < threads; ++id) {
        workers.emplace_back([=, &f] {
            for (int i = id; i < n; i += threads) {
                f(i);
            }
        });
    }
    for (auto &worker : workers) {
        worker.join();
    }
}

template<typename Case, typename Solve>
std::vector<std::string> parallel_cases(const std::vector<Case> &cases, int threads, Solve solve) {
    std::vector<std::string> out(cases.size());
    parallel_for(static_cast<int>(cases.size()), threads, [&](int i) {
        std::ostringstream os;
        solve(cases[i], os);
        out[i] = os.str();
    });
    return out;
}

```

Example Usage

```

int main() {
    std::vector<long long> squares(1000);
    parallel_for(static_cast<int>(squares.size()), 4, [&](int i) { squares[i] = 1LL * i * i; });
    assert(squares[17] == 289);
    assert(std::accumulate(squares.begin(), squares.end(), 0LL) == 332833500);

    std::vector<int> cases = {3, 1, 4, 2};
    auto out =
        parallel_cases(cases, 2, [](int x, std::ostringstream &os) { os << x << "^2 = " << x * x; });
    assert((out == std::vector<std::string>{"3^2 = 9", "1^2 = 1", "4^2 = 16", "2^2 = 4"}));
    return 0;
}

```

8.8 Memory Allocation

Simple bump allocators for local contest code. These are useful when creating many small objects and individual `new/delete` overhead is measurable. They allocate from one fixed buffer and do not reclaim individual objects.

- The global `operator new` override sends all dynamic allocations through a static buffer. This is the shortest version, but it affects the whole program and should be used only when you are sure the fixed buffer is large enough.
- `BumpAllocator<T>` is an STL allocator for containers such as `std::vector<T, BumpAllocator<T>>`. It is less invasive than overriding global `new`, but still has no real deallocation.

These snippets are not portable production allocators: they are fixed-size, monotonic, and intended for local performance-sensitive contest solutions. The global override below covers ordinary single-object `new`; it does not attempt to be a complete replacement for all allocation forms such as `new[]` or over-aligned allocation.

Time Complexity: $O(1)$ per allocation.

Space Complexity: $O(\text{BUFFER_SIZE})$ static memory.

```
#include <cassert>
#include <cstddef>
#include <cstdint>
#include <new>
#include <type_traits>
#include <vector>

static constexpr std::size_t BUFFER_SIZE = 64 << 20;
alignas(std::max_align_t) static unsigned char bump_buffer[BUFFER_SIZE];
static std::size_t bump_pos = BUFFER_SIZE;

void *operator new(std::size_t size) {
    size = (size + alignof(std::max_align_t) - 1) & ~(alignof(std::max_align_t) - 1);
    assert(size <= bump_pos);
    bump_pos -= size;
    return bump_buffer + bump_pos;
}

void operator delete(void *) noexcept {}

void operator delete(void *, std::size_t) noexcept {}

template<typename T>
struct BumpAllocator {
    using value_type = T;

    BumpAllocator() = default;

    template<typename U>
    BumpAllocator(const BumpAllocator<U> &) {}

    T *allocate(std::size_t n) {
        std::size_t size = n * sizeof(T);
        std::size_t align = alignof(T);
        assert(size <= bump_pos);
        bump_pos = (bump_pos - size) & ~(align - 1);
        return reinterpret_cast<T *>(bump_buffer + bump_pos);
    }

    void deallocate(T *, std::size_t) {}

    template<typename U>
    bool operator==(const BumpAllocator<U> &) const {
        return true;
    }

    template<typename U>
    bool operator!=(const BumpAllocator<U> &) const {
        return false;
    }
};
```

Example Usage

```
int main() {
    int *x = new int(42);
```

```
    assert(*x == 42);
    delete x;    // No-op.

    std::vector<int, BumpAllocator<int>> v;
    for (int i = 0; i < 1000; i++) {
        v.push_back(i);
    }
    assert(v[123] == 123);
    return 0;
}
```